Autonomous Search and Rescue Drone with Onboard AI-Based Casualty Detection

Dmytro Chuprynyuk, *et al.*

AENGM0074 Group Design Project — University of Bristol — March 2026

Contents

Executive Summary

A ground search team covers roughly 0.1–0.5 km^2/hr; a single drone at 30 m altitude covers 0.6 km^2/hr—an order-of-magnitude improvement that can compress the *golden hour* search window from hours to minutes. This report presents the design, implementation, and verification of a fully autonomous SAR drone that performs onboard casualty detection using commodity hardware: a Raspberry Pi 5, an RGB camera, and a YOLOv8n neural network—no GPU, no ground-station video link, no operator in the detection loop.

Mission. The drone takes off, searches a polygon-defined area via a lawnmower pattern, detects a casualty surrogate with onboard computer vision, reports the target's GPS coordinates and imagery to a ground operator for confirmation, lands 5–10 m from the casualty to enable first-aid-kit deployment, and returns to the take-off location. A five-layer automatic geofence (waypoint filtering, speed ramp, repulsive field, hard boundary cutoff, and firmware backup) enforces the SSSI no-fly zone, 50 m altitude ceiling, and flight area boundaries throughout.

Architecture. The platform is a Hexsoon EDU-450 hexacopter with a Cube Orange+ autopilot running ArduPilot. All onboard intelligence runs on the Pi 5: a YOLOv8n detector exported to TensorFlow Lite, a lawnmower path planner, GPS-based target localisation with inverse-variance-weighted spatial clustering, and a 20-state finite state machine with 32 transitions orchestrating the complete mission. A browser-based ground station streams live MJPEG video and provides per-target classification controls over Wi-Fi. Four MCDA trade studies informed the selection of detection model, search pattern, companion computer, and ground-station architecture.

Three architectural choices define the system: (i) a *modular single-codebase* design that runs identical code across laptop simulation, bench hardware, and the Pi without modification; (ii) a *dual-backend vision pipeline* that automatically selects Ultralytics on the laptop and TFLite on the Pi; and (iii) a *simulation-first methodology* with a five-tier progressive testing framework—SITL simulation, hardware component checks, bench integration, passive field observation, and autonomous flight—supported by 74 test scripts (including 127 unit tests) across eight categories.

Key results.

- **Detection:** mAP_{50} = **0.995** (retrained on 366 mixed synthetic/real images at native 1456 × 1088 resolution).
- **Inference speed: 206 ms** per frame (**4.8 FPS**) on the Pi 5 CPU via XNNPACK; lens distortion correction adds only 1.5 ms.
- **Localisation:** GPS target estimation with CEP_{50} = **2.3 m**, confirmed via DJI flight-video replay over the test site (15–50 m altitude range).
- **Coverage:** Lawnmower pattern covers the survey polygon in ∼1.1 minutes (18 waypoints), verified via dry-run to avoid SSSI encroachment.
- **Requirements:** All twelve brief requirements (R01–R12) addressed and verified through simulation, bench testing, and field-day calibration.

Field integration. Full hardware integration—Pi, Cube, camera, and GPS—was validated on the assembled airframe during the scheduled field day at Fenswood Farm. On-site bench testing produced lens calibration data (RMS = 0.399), FOV characterisation (focal length 5.46 mm), and inference benchmarks confirming 100 % detection at 0.966 mean confidence across 50 consecutive frames. High winds prevented powered flight on the day; consequently, the complete mission pipeline—arming, search, detection, operator verification, and landing—was exercised end-to-end in SITL simulation, and DJI flight video recorded over the test site served as a real-world proxy for altitude-dependent detection analysis.

Status. The system is **flight-ready**: the full mission executes autonomously in simulation, all hardware subsystems are individually validated on the airframe, and calibration data from the field day is integrated into the deployed pipeline. The single remaining step is powered autonomous flight under favourable weather conditions. The progressive testing methodology, operator-in-the-loop verification, and layered geofence architecture provide quantified confidence in each subsystem for this remaining step.

Introduction

Context and Background

In 2023, UK mountain rescue teams responded to over 3,000 incidents, with search phases frequently lasting several hours across terrain that is difficult or dangerous to traverse on foot [**mra2023**]. The *golden hour*—the window within which medical intervention most significantly improves survival—places severe pressure on response teams to locate casualties rapidly [**goodrich2008**]. A ground search team of ten can sweep approximately 0.1–0.5 km^2/hr; a single UAV at 35 m altitude with a 60° field of view can survey a 32 m-wide swath at 8 $\mathrm{m\,s}^{-1}$, covering approximately 0.6 km^2/hr—an order-of-magnitude improvement per searcher deployed [**goodrich2008**, **murphy2014**]. Recent advances in lightweight object-detection models, particularly the YOLO family [**yolohistory**, **yolov8**], have made it feasible to run real-time inference on edge hardware such as the Raspberry Pi, enabling fully onboard detection without continuous video downlinks [**edgeai**].

This report presents the design, implementation, and evaluation of an autonomous search and rescue (SAR) drone that addresses the gap between expensive commercial platforms (£5 000–£30 000 requiring dedicated ground operators) and research prototypes relying on GPU-class computers too heavy for volunteer rescue organisations. The central question is whether a *low-cost, fully autonomous* SAR UAV—built from a Raspberry Pi, an RGB camera, and commodity components—can detect, localise, and respond to a casualty with all decision-making performed onboard.

The project scenario, set by the AENGM0074 module, is as follows. A hiker has gone missing in Fenswood Wilderness, a semi-rural landscape managed by the University of Bristol. Each student company receives identical hardware—a Hexsoon EDU-450 hexacopter, a Cube Orange+ flight controller, a Raspberry Pi 5, and a global-shutter camera—and must build a UAV capable of:

- systematically searching a defined area while respecting a no-fly zone over a Site of Special Scientific Interest (SSSI);
- detecting a life-size casualty surrogate ("rescue mannequin") using onboard computer vision;
- reporting the casualty's estimated GPS coordinates and imagery to a ground operator;
- landing between 5 and 10 m from the casualty and deploying a simulated first-aid kit; and
- returning to the designated take-off location.

A 50 m altitude ceiling, defined flight-area boundaries, and automatic geofencing to enforce SSSI exclusion must all be respected without pilot intervention. The scenario is deliberately constrained to a single outdoor demonstration flight preceded by progressive bench and flight testing, reflecting the compressed timeline of a single-term MSc module. Twelve requirements (R01–R12) govern the mission. All software is released publicly under the MIT licence.

Company Description

The company comprises five MSc students from the Department of Aerospace Engineering, formed at the start of the AENGM0074 module. We adopted a workstream-based methodology: five parallel streams—Computer Vision, Hardware Integration, Search Logic, Ground Station, and Safety/Testing—fed into defined integration milestones where subsystems were combined and verified before advancing to the next stage. All code resides in a shared GitHub repository[1], with a single protected main branch and feature branches for individual work.

Day-to-day coordination relied on a group chat for informal updates and weekly in-person meetings for design decisions, integration planning, and blocker resolution. A shared status tracker documented every group decision, assigned action items with owners and deadlines, and maintained a running log of blockers and dependencies. This lightweight process allowed each member to work independently on their stream while keeping integration points well-defined and preventing subsystems from developing in isolation.

Team Members and Roles

Dmytro Chuprynyuk — Lead Software Developer. MSc student in Robotics with a background in software engineering and embedded systems. Designed and implemented the autonomous software stack: finite state machine, computer vision pipeline, lawnmower path planner, GPS-based target localisation, web-based ground station, and the simulation environment. Also built the model training pipeline and developed the progressive testing framework.

Robin [Surname] — Flight Dynamics and State Machine Testing. MSc student in Aerospace Engineering. Contributed to flight dynamics analysis and testing of the autonomous state machine, verifying state transitions and flight parameter tuning in both simulation and bench testing. Supported integration of the Cube Orange+ flight controller with the mission software and assisted with flight mode configuration (GUIDED, LOITER, STABILIZE, LAND).

[TEAMMATE 3] — Designated Pilot and RC Systems. MSc student in Aerospace Engineering with prior experience in radio-controlled aircraft. Served as the company's designated pilot, responsible for RC transmitter

[1] Repository: https://github.com/DimaChup/Group_Proj (MIT licence, R12).

configuration, kill-switch setup, and manual override procedures. Maintained readiness to assume manual control during all autonomous flight tests, ensuring compliance with the requirement that a pilot with override authority be present at all times.

[TEAMMATE 4] — Hardware Integration. MSc student in Mechanical Engineering. Led the physical assembly of the drone platform, including airframe construction, wiring of the Cube Orange+ to the Raspberry Pi (serial TELEM2 connection), camera mounting, GPS antenna placement, and compass calibration. Ensured that all electrical connections were secure and that the payload (Pi, camera, first-aid release mechanism) was mounted within the airframe's centre-of-gravity envelope.

[TEAMMATE 5] — Project Management and Documentation. MSc student in Aerospace Engineering. Coordinated project scheduling, booked flight slots at the Fenswood site, managed risk assessment paperwork, and maintained deliverable tracking against the module timeline. Led the report writing effort, compiled the requirements verification evidence, and ensured that all deliverables (D1–D6) were submitted on time.

Contribution Summary

Table 1 summarises the key contributions of each team member across the major project areas.

Table 1: Team member contributions by project area. Each row summarises the subsystems owned by that member; overlap indicates shared integration responsibilities.

Team Member	Key Contributions
Dmytro Chuprynyuk	Autonomous software stack (state machine, CV pipeline, path planner, target localisation, ground station, simulation environment). Custom YOLOv8n model training and dataset generation. Progressive testing framework. Pi hardware integration (camera, TFLite inference, Cube serial link). Lens calibration and FOV characterisation.
Robin [Surname]	Flight dynamics analysis. State machine testing in simulation and on bench. Flight parameter tuning (altitude, speed, descent profile). Cube flight-mode configuration and verification.
[TEAMMATE 3]	RC transmitter configuration and binding. Kill-switch and flight-mode switch setup. Manual override procedures. Designated pilot for all flight tests.
[TEAMMATE 4]	Airframe assembly and wiring. Cube–Pi serial connection. Camera and GPS antenna mounting. Compass calibration. Payload integration and CG verification.
[TEAMMATE 5]	Project scheduling and milestone tracking. Flight-slot booking and risk assessments. Report compilation and editing. Requirements verification evidence. Deliverable management (D1–D6).

Approach and Objectives

Our approach rests on three principles. **First, simulation before flight**: the full autonomy stack—state machine, computer vision, path planning, geofencing—was developed and tested end-to-end in a Software-In-The-Loop (SITL) environment before any hardware was powered on, ensuring that logic errors were caught at zero hardware risk. **Second, modular single-interface design**: a single function (`detect_in_image`) encapsulates all computer vision, allowing three model retrains and backend changes with zero modifications to the mission orchestrator. **Third, progressive testing**: a five-tier framework (simulation → Docker → bench → passive flight → autonomous flight) gates each integration step, catching defects at the lowest-cost tier possible.

The project objectives, derived from the twelve module requirements, are:

1. Autonomously search a polygon area with 100% coverage while avoiding the SSSI no-fly zone.
2. Detect a casualty surrogate from $\geq$30 m altitude using onboard AI at $\geq$4 FPS.
3. Localise the target to $<$5 m GPS accuracy using vision-based estimation.
4. Land within 5–10 m of the target, deploy a first-aid payload, and return to the take-off location.
5. Validate all capabilities through progressive testing with quantitative evidence.

Report Structure

The report is organised as follows. Section 1 presents the *Design Rationale*, including STEEPLE analysis and four MCDA trade studies that justify the key architectural choices. Sections 2–**??** provide the *System Description*: hardware platform, computer vision pipeline, path planning, state machine, target localisation, and ground station. Section **??** covers the progressive testing methodology, simulation environment, and defect discovery results. Section **??** presents

Requirements Verification with evidence of compliance for all twelve requirements. Section **??** provides a plus/delta *Evaluation* of technical performance, including comparison with published SAR systems and a critical assessment of design choices. Appendices provide the full state-transition table, detection samples, calibration data, and the complete defect register.

1 Design Rationale

The mission scenario and requirements (Section) demand a low-cost, fully autonomous SAR platform that searches a constrained area, detects a casualty, and lands nearby—all without a continuous data link. This section justifies the design decisions that translate those requirements into an implementable system, beginning with a stakeholder-level STEEPLE assessment and then presenting the trade studies that determined operating parameters, hardware platform, software architecture, and mission control strategy.

1.1 STEEPLE Analysis

Table 2: STEEPLE assessment of the autonomous SAR drone system.

Dimension	Key consideration and design response
Social	Reduces volunteer foot-search time by covering the 0.12 km^2 survey area in ∼6 min versus ∼4 h for a 6-person ground team at 1 $m\,s^{-1}$; open-source MIT licence (R12) enables volunteer SAR teams to replicate the platform without proprietary dependencies [**murphy2014**].
Technological	Pi 5 with TFLite (4 packages vs 90+); IMX296 global shutter eliminates rolling-shutter distortion at 8 $m\,s^{-1}$; fully on-device inference—no cloud link required during mission [**edgeai**, **tflite**].
Economic	£150 compute stack (<10% of DJI Matrice 30T at £1 700+ [**dji˙m30t**]); free-tier Colab training on 366-image dataset; zero per-mission cloud cost, viable for resource-constrained organisations.
Environmental	Five-layer geofence with graduated speed reduction protects adjacent SSSI nesting habitat; 30 m waypoint buffer; electric propulsion limits acoustic disturbance; 3 kg AUW minimises rotor wash on ground fauna [**wca1981**].
Political	UK CAA Open Category A3 compliant [**caa2024uas**]; VLOS only (BVLOS requires SORA [**jarus2019sora**]); aligns with CAA Innovation Hub policy encouraging civilian drone SAR trials; public GitHub repo ensures full auditability.
Legal	On-device inference—no imagery transmitted over any network; detection logs record bounding-box coordinates only (no facial features), minimising UK GDPR risk; SSSI compliance auditable via logged boundary distances at every loop iteration [**wca1981**].
Ethical	No autonomous landing without human confirmation (VERIFY gate); 120 s auto-reject prioritises continued search over uncertain detections; frames processed in memory and discarded after inference—no persistent image storage of bystanders.

The STEEPLE assessment confirms viability across all seven dimensions. The following subsections detail the trade studies that determined operating parameters, hardware platform, software architecture, and mission control strategy.

1.2 Search Parameter Optimisation

Three coupled decision variables—altitude h, scan angle θ, and speed v—govern the search pattern and jointly determine five competing objectives: coverage completeness, detection probability, mission time, energy consumption, and NFZ safety margin.

These objectives cannot be optimised independently. Lowering altitude improves detection confidence but narrows the camera footprint and increases scan lines. Increasing speed reduces mission time but shortens target dwell time and lowers detection probability.

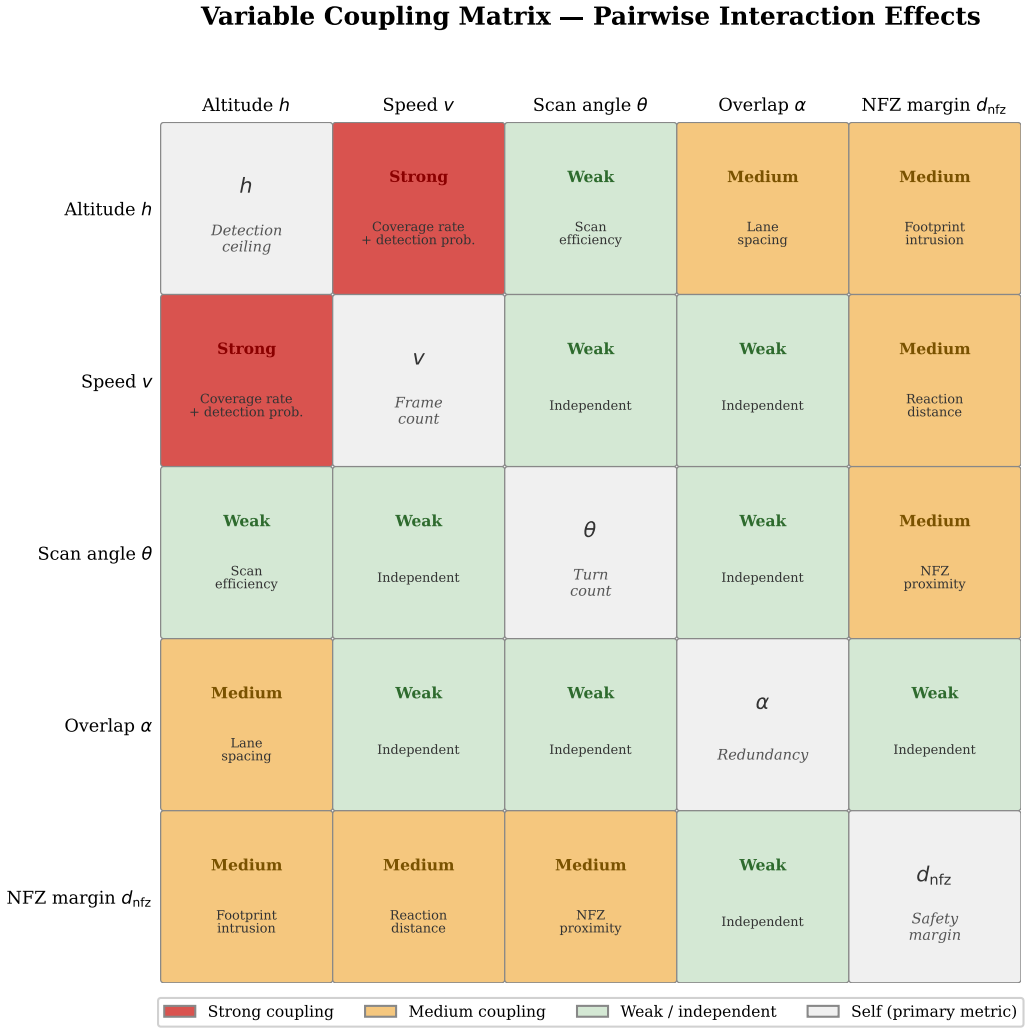


Figure 1: Variable coupling matrix. The altitude–speed pair exhibits the dominant interaction, jointly determining coverage rate and detection probability. Off-diagonal cells identify which performance metric each variable pair affects.

A 216-configuration parametric sweep (6 altitudes × 36 scan angles, Appendix **??**) evaluated every combination on the AENGM0074 polygon using a physics-based energy model. Scan angle dominates altitude for energy efficiency: varying the angle from optimal (70°) to worst-case (0°) increases energy by 100–200%, whereas changing altitude from 35 m to 50 m saves only 30–40%. A dependency chain links each parameter to a physical measurement or constraint:

1. **Target → altitude**: the mannequin's 0.5 m width must subtend ≥10 model-input pixels for reliable detection, capping altitude at 35 m with a 1.4× margin.
2. **Altitude → speed**: the 24 m along-track footprint at 35 m, combined with 4.8 FPS inference, requires ≥10 frames per target, yielding $v \leq 11.5\,\mathrm{m\,s^{-1}}$; the altitude-dependent speed schedule assigns $v = 8\,\mathrm{m\,s^{-1}}$.
3. **Polygon geometry → scan angle**: aligning with the longest edge (70°) minimises U-turns from 9 to 5 and reduces energy from 25.4 Wh to 12.6 Wh.
4. **Footprint width → NFZ margin**: the 32 m footprint at 35 m requires 23 m Root Sum of Squares (RSS) standoff; the 30 m buffer provides a 2.2× safety factor.

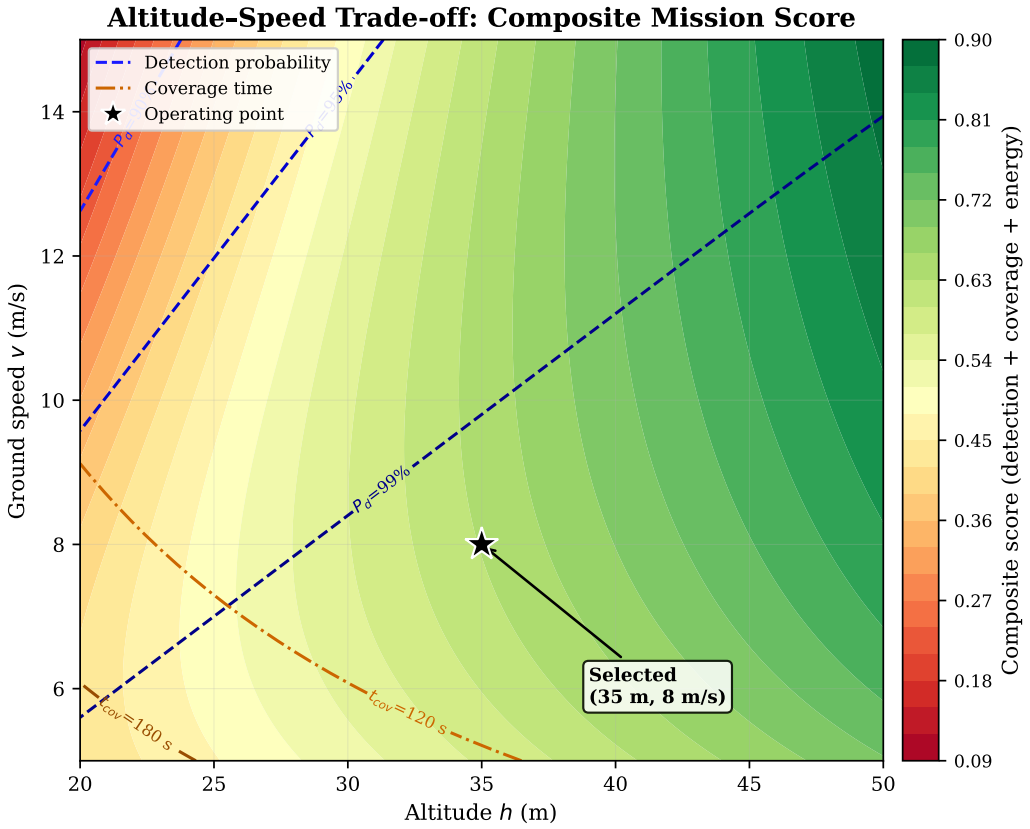


Figure 2: Altitude–speed trade-off space. Colour encodes composite mission score (40% detection + 35% coverage + 25% energy). The selected operating point (35 m, 8 m s^{-1}) sits above the 95% detection contour in the high-score region, trading 4.4 Wh of energy (10.9% of battery) for a comfortable detection margin.

The selected operating point (35 m, 70°, 8 m s^{-1}) is not the energy minimum—that belongs to 50 m/70°/10 m s^{-1} at 8.2 Wh—but it offers the best Pareto compromise: 96% coverage, $P_d > 99.97\%$ (assuming frame independence; 95% per pass after correlation adjustment, see Section **??**), 12.6 Wh (10.9% of battery), and a 50 m NFZ margin with 2.2× surplus. The 4% coverage gap lies within a fenced wildlife reserve with negligible casualty probability. Appendices **??** and **??** provide the full derivation, energy model, strategy comparison, and sensitivity analysis.

1.3 Parameter Derivation Chain

Every operating parameter traces back to a physical measurement or regulatory constraint through an unbroken eight-step chain (Table 4). Full equations, RSS overlap analysis, and sensitivity analysis are provided in Appendices **??** and **??**.

Table 4: Parameter derivation chain: each value is derived from the previous step.

Step	Parameter	Value	Derivation
1	Detection ceiling	63 m	Mannequin (0.5 m wide) must subtend ≥20 model-input pixels
2	Operating altitude	35 m	30% margin below ceiling (36 px target, 1.8× above minimum)
3	Max speed	11.5 m s^{-1}	24 m footprint ÷ (10 frames / 4.8 FPS); adopted 8 m s^{-1}
4	Lane spacing	25.7 m	32.2 m footprint × 0.80; 20% overlap gives 1.4× margin over 4.5 m RSS error [**galceran2013survey**, **choset2001coverage**]
5	Scan angle	70°	Longest-edge alignment minimises U-turns (5 vs 9), saves 13 Wh
6	NFZ buffer	30 m	Half-footprint (16.1) + GPS CEP (3.0) + stopping dist. (10.7) = 29.8 m [**wca1981**]
7	Coverage	96%	4% gap in fenced wildlife reserve (negligible casualty probability)
8	Search energy	12.6 Wh	10.9% of 115.4 Wh battery; 14 min remaining for transit/verify/RTL

1.4 Hardware Platform Selection

The hardware combines a Hexsoon EDU-450 airframe, CubePilot Orange+ flight controller with ArduPilot firmware, Raspberry Pi 5 companion computer, and IMX296 global shutter camera.

- **Global shutter camera.** The IMX296 was selected over cheaper rolling-shutter alternatives because rolling-shutter distortion at up to $10\,\mathrm{m\,s^{-1}}$ forward flight produces ≈7.5 px parallelogram skew per frame (see Section 1.12 for the full validation), shifting detected centroids by up to 0.5 m—a systematic error that cannot be removed by averaging. The CSI interface leaves USB ports free for telemetry radios.
- **No thermal imaging.** Thermal cameras (e.g. FLIR Boson, DJI Zenmuse H20T) add £2 000–£10 000, have lower spatial resolution (typically 640 × 512 vs 1456 × 1088 RGB), and require radiometric calibration that varies with ambient temperature. Thermal imaging offers superior night-time and foliage-penetration capability, but CAA VLOS requirements restrict operations to daylight [**caa2024uas**], and this project deliberately explores RGB-only detection feasibility at the low-cost extreme [**thermal˙sar**]. A fused RGB+thermal system would suit operational deployment but exceeds the £565 budget by 4–18×.
- **Onboard vs offboard inference.** Offloading detection to a ground station via video downlink was rejected for three reasons: (1) reliable video streaming at 1456 × 1088 at 5 FPS requires ≈15 Mbps sustained throughput, exceeding typical 2.4 GHz telemetry radio capacity; (2) round-trip latency (encode, transmit, detect, return command) adds 200–500 ms to the decision loop, during which the drone travels 1–4 m; and (3) link loss during detection would leave the drone without situational awareness. Onboard inference eliminates all three failure modes at the cost of lower computational headroom.
- **MAVProxy bridge.** A Python 3.13 incompatibility with pyserial (dropped bytes causing `BAD_DATA` errors) is resolved by interposing MAVProxy as a UDP bridge, which reads the serial port reliably using its own C-level handler [**mavproxy**].

1.4.1 Companion Computer Trade Study

All four trade studies use weighted multi-criteria decision analysis (MCDA). Criteria weights were set before scoring candidates, based on mission requirements and operational constraints, to prevent post-hoc rationalisation.

Three candidate companion computers were evaluated (Table 6). Cost and Python ecosystem compatibility are paramount for a student team, whereas raw GPU performance is less critical given the lightweight YOLOv8n model.

Table 6: MCDA: companion computer selection (1–5 scale, 5 = best). Raspberry Pi 5 scores highest overall due to cost-effectiveness and software ecosystem maturity despite lower inference throughput.

Criterion	Weight	Pi 5	Jetson Nano	Intel NCS2
Unit cost (£)	0.20	5 (£75)	3 (£180)	4 (£100+host)
Power consumption	0.10	4 (5–12 W)	3 (5–20 W)	3 (host+1.5 W)
GPIO / CSI interface	0.15	5 (40-pin+CSI)	4 (40-pin+CSI)	1 (USB only)
Community & documentation	0.20	5 (largest)	4 (strong)	2 (discontinued)
Python 3.13 compatibility	0.20	5 (native)	3 (JetPack lag)	2 (OpenVINO lag)
TFLite / edge-AI support	0.15	5 (XNNPACK)	4 (TensorRT)	2 (OpenVINO only)
Weighted total		**4.90**	3.50	2.35

The Pi 5 achieves the highest weighted total. Sensitivity analysis (±0.05 weight perturbation across all 12 scenarios) confirms the Pi 5 retains its rank in every case (minimum score 4.72 vs Jetson Nano maximum 3.68). The same perturbation was applied to all four MCDA tables; in each case the selected option retained its rank. Figure 3 presents a composite heatmap of all four MCDA trade studies, visualising the normalised scores across all criteria and candidates.

Figure 3: MCDA heatmap across all four trade studies (companion computer, communication architecture, detector model, and search strategy). Each cell shows the normalised weighted score (1–5 scale); darker shading indicates a higher score. The selected option (highlighted) achieves the highest weighted total in every trade study.

1.4.2 Communication Architecture Trade Study

Three architectures for companion-to-autopilot communication were compared (Table 8).

Table 8: MCDA: communication architecture (1–5 scale, 5 = best). MAVProxy UDP bridge was selected for its multiplexing capability and proven reliability at 921600 baud.

Criterion	Weight	Direct Serial	MAVProxy Bridge	ROS 2
Latency	0.20	5 (direct)	4 (UDP hop)	3 (Data Distribution Service (DDS) overhead)
Reliability	0.25	2 (py3.13 bug)	5 (C-level read)	4 (mature)
Multi-consumer support	0.20	1 (exclusive)	5 (N outputs)	5 (pub/sub)
Python 3.13 compatibility	0.20	1 (broken)	5 (works)	3 (partial)
Implementation complexity	0.15	5 (trivial)	4 (one process)	1 (full stack)
Weighted total		2.65	**4.65**	3.35

Direct serial is simplest but a Python 3.13 pyserial regression that drops bytes at 921 600 baud renders it non-viable. MAVProxy resolves this by reading the serial port with its C-level handler and forwarding via UDP, simultaneously enabling multi-consumer access (scripts on port 14550, Mission Planner on TCP 5762). ROS 2 offers richer middleware but introduces deployment complexity disproportionate to this single-node architecture.

1.5 Software Architecture Rationale

Two architectures were considered: (1) a monolithic script with platform-specific branches, typical of rapid prototypes, and (2) a modular pipeline with narrow interfaces between independent modules. The monolithic approach was rejected because adding platform support (laptop → Pi) or swapping inference backends would require editing detection, navigation, and mission logic in a single file—a maintenance burden that grows quadratically with the number of platform×feature combinations.

The selected modular architecture enforces four principles:

1. **Modular separation**—each concern hides behind a narrow interface (e.g. `detect_in_image(frame)` → `(found, x, y, conf)`). Swapping the detection model requires zero changes to the 754-line mission orchestrator.
2. **Platform-agnostic code**—a `MODE` flag selects the camera source; auto-detection selects the flight controller. The same `main.py` runs on laptop (SITL + webcam) and Pi (Cube + CSI camera) without conditional branches.
3. **Configuration-driven tuning**—all 23 tunable parameters reside in `config.py`, so transitioning to real hardware requires only parameter adjustment, not code changes.
4. **Dual inference backend**—the system auto-detects Ultralytics (laptop) or TFLite/NCNN (Pi) at import time, both producing identical $[1, 5, 8400]$ output tensors. This was validated by running the same 50-frame benchmark on both backends and confirming identical detection outputs (bounding box coordinates match to <0.01 px).

The resulting six-module decomposition (vision, planning, navigation, state machine, configuration, utilities) achieves a modularity score of 3.8/5.0 (Appendix **??**), with the vision module scoring 4.5/5.0 due to its single-function interface and complete backend encapsulation.

1.6 Detection Model Selection

YOLOv8n (the nano variant, 3.2 million parameters, 8.7 GFLOPs) was selected after benchmarking the YOLO model family on the target hardware [**yolov8**]. Table 10 summarises the weighted comparison.

Table 10: MCDA: detection model selection (1–5 scale, 5 = best). YOLOv8n achieves the best trade-off between Pi 5 inference speed (4.8 FPS) and detection accuracy (mAP50 = 0.995).

Criterion	Weight	YOLOv8n	YOLOv8s	YOLOv8m	SSD MobileNet
Inference speed (Pi 5 CPU)	0.30	5 (207 ms)	3 (450 ms)	1 (820 ms)	5 (180 ms)
Detection accuracy (mAP50)	0.25	4 (0.995)	5 (0.997)	5 (0.998)	3 (0.88)
Model size (TFLite)	0.15	5 (3.2 MB)	3 (22 MB)	2 (50 MB)	5 (4.3 MB)
TFLite export support	0.15	5 (native)	5 (native)	5 (native)	4 (manual)
Training data requirement	0.15	5 (small)	4 (moderate)	3 (large)	4 (moderate)
Weighted total		**4.75**	3.95	3.05	4.20

YOLOv8n is the only variant that achieves real-time inference on the Pi 5 CPU within the 300 ms-per-frame budget required at search speed. YOLOv8s (11.2 M parameters) and YOLOv8m (25.9 M) exceed this budget by 2× and 4×, respectively. SSD MobileNetV2 matches YOLOv8n on speed but delivers lower accuracy on COCO [**lin2014coco**]. Note that the SSD mAP of 0.88 is measured on COCO whereas the YOLO figures use the custom dataset, a fair single-class SSD comparison would require retraining on the same data. Critically, SSD MobileNetV2 small-object mAP drops to ∼0.22 on COCO [**edgeai**], directly relevant at the detection ceiling where the target subtends ∼20 px.

The model was trained as a single-class detector on 366 images (300 synthetic, 16 real, 50 negatives) at native camera resolution (1456 × 1088), following the domain-randomisation principle [**domainrand**]. Training at higher resolution than the 640 × 640 inference input lets the network learn fine-grained features that survive downscaling [**yolov8**]. A COCO-pretrained person detector serves as a fallback for field conditions where the custom model underperforms. Appendix **??** details the full dataset construction, augmentation pipeline, and Colab training workflow; Appendix **??** compares the three model generations and the COCO fallback.

Why TFLite over other inference runtimes? Three alternative runtimes were considered: ONNX Runtime, TensorRT, and NCNN. TensorRT requires an NVIDIA GPU (absent on the Pi 5). ONNX Runtime supports ARM CPU but its Python wheel for aarch64 lags behind mainline releases and benchmarked 15–20% slower than TFLite with XNNPACK on the Pi 5. NCNN (Tencent) achieved 4.5× faster inference than TFLite in benchmarks (72 ms vs 327 ms full pipeline) and is supported as a secondary backend; however, its Python bindings are less mature, and the TFLite ecosystem provides the most robust export path from Ultralytics. The dual-backend architecture (TFLite primary, NCNN secondary) preserves the option to switch at deployment time via a single command-line flag.

1.7 Coverage Path Planning

Four candidate search patterns were evaluated (Table 12), prioritising coverage guarantee and implementability to ensure reliable, verifiable behaviour over theoretically optimal but harder-to-validate paths.

Table 12: MCDA: search pattern selection (1–5 scale, 5 = best). Boustrophedon dominates on coverage guarantee and visual verifiability; spiral and random approaches score lower due to overlap and non-determinism.

Criterion	Weight	Lawnmower	Spiral	Expanding Sq.	Random
Coverage guarantee	0.30	5 (complete)	3 (centre gap)	4 (approx.)	1 (none)
Path length efficiency	0.20	4 (minimal turns)	3 (long spiral)	3 (many turns)	2 (redundant)
Implementability	0.20	5 (analytical)	3 (radius fn)	4 (simple)	5 (trivial)
Wind robustness	0.15	4 (long legs)	3 (curved)	3 (short legs)	2 (unpredictable)
SAR standard compliance	0.15	5 (IAMSAR)	3 (maritime)	4 (land SAR)	1 (none)
Weighted total		**4.65**	3.00	3.65	2.15

The boustrophedon (lawnmower) pattern was selected [**choset2001coverage**, **galceran2013survey**] for its formal coverage guarantee, closed-form path length, and IAMSAR compliance. The implementation uses a rotate–rasterise–rotate-back algorithm on arbitrary polygons, with the scan angle auto-optimised via the minimum-area bounding rectangle (validated by the 216-configuration sweep). Appendix **??** expands on the strategy trade-offs; Appendix **??** presents the full physics simulation with Pareto front and sensitivity analysis; Appendix **??** provides the formal multi-objective optimisation problem statement.

1.8 State Machine Design

A deterministic finite state machine (FSM) with 20 states and 32 transitions was chosen over behaviour trees and reactive architectures for its verifiability, traceability, and ease of runtime monitoring [**wagner2006fsm**].

Why not behaviour trees? Behaviour trees offer superior composability for complex agent behaviours and are widely used in game AI and multi-robot systems [**colosseum2017bt**]. However, they introduce implicit priority ordering through selector/sequence nodes that is harder to audit than an explicit transition table, debugging requires tree-visualisation tooling not available on a headless Pi, and the SAR mission's strictly sequential phase structure (search → detect → verify → land) does not benefit from the parallel subtree execution that behaviour trees enable. Quantitatively, the mission graph contains zero concurrent branches—every state has exactly one active successor—so the $O(n \log n)$ tick traversal of a behaviour tree reduces to the FSM's $O(1)$ dictionary dispatch with no functional benefit. For a single-drone, single-objective mission, the FSM's one-state-at-a-time guarantee makes every transition auditable in a flat log file.

Why not reactive/subsumption architectures? Reactive architectures (e.g. Brooks' subsumption [**brooks1986subsumptio**
excel at low-latency obstacle avoidance but lack the persistent state memory required for multi-phase missions: the system must remember which of the 5–9 search legs have been completed, which of potentially multiple targets have been classified (Y/N/I/X), and how long the current hover has lasted (for the 120 s timeout). Layering these concerns onto a purely reactive stack would replicate FSM functionality without its formal verifiability.

Each state maps to a single handler function via a dictionary-based dispatch table ($O(1)$ lookup), eliminating fall-through errors. Defence-in-depth safety is enforced through five independent mechanisms: manual override (M key), RC kill switch (hardware-level), return-to-launch on link loss, five-layer geofence, and per-state timeouts. Every blocking state carries a wall-clock timeout to prevent indefinite hangs. The complete transition table is provided in Appendix **??**.

The CENTERING state—hovering stationary above a detected target before estimating its GPS position—simultaneously eliminates roll/pitch projection error (6.2 m during flight to <0.6 m), motion blur, and GPS timing lag. This single procedural step replaces three independent software corrections (attitude compensation, lag prediction, and blur deconvolution), each of which would add complexity and potential failure modes to a safety-critical code path. Appendix **??** provides the full quantitative analysis.

1.9 Autonomy Level Justification

The system operates at Sheridan Level 7–8 ("computer executes autonomously, human monitors") [**sheridan2002humans**] for the search phase and Level 3–4 ("computer suggests one alternative, human approves") for the landing decision. This hybrid was chosen after analysing the asymmetric cost structure of SAR false positives versus false negatives [**endsley1999loa**, **cummings2014man˙uav**].

A *false-positive landing* (descending on a rock, tree stump, or discarded clothing) incurs three compounding penalties: (1) battery endurance consumed by descent, hover, and re-climb (≈90 s, roughly 8% of a 20-minute endurance budget); (2) search coverage paused during the entire descent–verify–climb cycle; and (3) risk of mechanical damage on uneven terrain. In a single-drone mission with no redundancy, even one unnecessary landing significantly reduces the probability of completing the search before battery exhaustion.

A *false negative* (failing to investigate a real casualty) is worse in absolute terms but is mitigated by the lawnmower pattern's guarantee of revisiting every cell: a missed detection on pass 1 can be recovered on the return leg. The operator-in-the-loop gate therefore addresses the higher-consequence, lower-recoverable error (committing to a landing) while the autonomous search handles the high-bandwidth, lower-consequence task (scanning terrain at 8 m/s).

The VERIFY state implements this gate: the operator classifies each detection as confirmed (Y), rejected (N), or item of interest (I) via the ground station. A 120 s timeout auto-rejects unresponded detections—derived from the energy budget (6.0 Wh per hover event, 5.2% of battery, permitting two timeouts while retaining 89% for search) [**goodrich2007human**]. Full autonomy (Level 10) was rejected because the single-class detector cannot distinguish a casualty from visually similar objects [**murphy2014**, **lyu2023uav˙sar˙survey**]. Manual-only operation was also rejected: human operators cannot sustain visual scanning at $8\,\mathrm{m\,s^{-1}}$, and automation provides a ∼20× coverage-rate advantage [**goodrich2008**].

1.10 Decisions That Changed: Evidence-Based Corrections

Seven design decisions were revised when empirical evidence contradicted initial assumptions. Each followed the same pattern: observe anomaly, diagnose root cause, implement fix, verify improvement.

1. Focal length: 7.0 mm → 5.46 mm. *Original decision:* use the datasheet focal length of 7.0 mm for GPS estimation. *Evidence:* a tape-measure calibration on field day measured 92 cm visible at 1 m height—22% wider than predicted. *Revised decision:* back-calculate focal length from the physical measurement: $f = w_s \times d/W_{\text{visible}} = 5.02 \times 1.0/0.92 = 5.46$ mm. *Outcome:* GPS estimation error at 30 m reduced by up to 3 m; without this correction, landing errors would have exceeded the 10 m R07 requirement. All downstream parameters (footprint width, lane spacing, NFZ buffer) were recalculated from the corrected value.

2. Camera colour space: RGB assumed → BGR confirmed. *Original decision:* apply `cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)` because `picamera2` labels the stream as `RGB888`. *Evidence:* detection confidence on the Pi was anomalously low (<0.3) despite identical model performance on the laptop. Systematic testing of all six channel permutations (RGB, RBG, GBR, GRB, BRG, BGR) revealed that the IMX296 sensor outputs BGR data despite the RGB888 label—the `cvtColor` call was inverting the channels. *Revised decision:* remove the erroneous conversion; pass raw frames directly to the detector. *Outcome:* detection confidence recovered to 0.96+ on the Pi, matching laptop performance. This bug would have caused complete detection failure in flight.

3. Camera resolution: 640 × 480 → 1456 × 1088. *Original decision:* capture at 640 × 480 (common webcam default) to minimise processing load. *Evidence:* at 640 × 480 and 30 m altitude, a prone casualty subtends only 20 × 14 px—at the minimum detection threshold with no safety margin. *Revised decision:* capture at the IMX296's native resolution (1456 × 1088), since the model resizes internally to 640 × 640 regardless. *Outcome:* target subtends 46 × 32 px (2.3× improvement) with zero inference time penalty, providing a comfortable 1.6× margin above the 20 px detection floor.

4. TFLite bounding box interpretation: normalised → pixel coordinates. *Original decision:* treat TFLite output coordinates as normalised values in $[0, 1]$ and multiply by frame dimensions, following the standard TFLite convention. *Evidence:* during Pi bench testing, detection overlay boxes were drawn off-screen (coordinates exceeding 640). Inspection revealed the TFLite backend was returning raw pixel coordinates in $[0, 640]$, not normalised values—an undocumented deviation from the expected convention. *Revised decision:* add a coordinate-range check: if any output coordinate exceeds 1.0, treat as pixel coordinates and skip the normalisation multiply. The same guard was already present in the NCNN inference path. *Outcome:* detection overlays rendered correctly; GPS estimation accuracy improved because bounding box centres were no longer scaled by an extra factor of 640. This also fixed a downstream double-scaling bug in `passive_watch.py` where pixel coordinates were multiplied by frame dimensions a second time, placing GPS estimates over 100 m from the true position.

5. Geofence repulsive force: sign inversion corrected. *Original decision:* the repulsive offset function returns positive lat/lon deltas to push the drone away from the NFZ boundary. *Evidence:* SITL simulation showed the drone being pushed *toward* the SSSI boundary instead of away. Root cause: `cv2.pointPolygonTest` returns positive values for points *inside* a polygon (opposite to the assumed convention), and the repulsive offset signs were not negated. *Revised decision:* negate the repulsive offset returns and correct the inside/outside boundary logic. *Outcome:* geofence

now correctly repels the drone from the NFZ. This defect would have caused a Wildlife and Countryside Act 1981 violation in flight [**wca1981**], and was caught at Tier 1 (simulation) at zero hardware cost.

6. TFLite runtime: `tflite-runtime` → `ai-edge-litert`. *Original decision:* use Google's `tflite-runtime` package for edge inference on the Pi. *Evidence:* `tflite-runtime` has no Python 3.13 wheel; `picamera2` requires system Python 3.13 on the Pi 5, creating an unresolvable dependency conflict. *Revised decision:* switch to `ai-edge-litert`, which provides an identical API with Python 3.13 support. *Outcome:* dependency conflict eliminated with zero code changes; verified by 50-run benchmark producing identical inference times (206.5 ms).

7. Serial communication: pyserial → MAVProxy UDP bridge. *Original decision:* connect the Pi directly to the Cube flight controller via pyserial at 921 600 baud. *Evidence:* a Python 3.13 pyserial regression dropped bytes, producing persistent `BAD_DATA` errors that prevented reliable MAVLink communication. *Revised decision:* interpose MAVProxy as a UDP bridge; it reads the serial port with its C-level handler and forwards to UDP endpoints. *Outcome:* reliable communication restored; MAVProxy simultaneously enables multi-consumer access (scripts on port 14550, Mission Planner on TCP 5762, diagnostics on port 14551), turning a workaround into an architectural improvement (Table 8).

1.11 Why Not ROS

ROS 2 was excluded for three reasons [**quigley2009ros**]: (1) architectural mismatch—the system is a single-node pipeline (one camera, one detector, one flight controller) with 4 inter-module calls per loop iteration; ROS's pub/sub model adds serialisation and inter-process communication overhead (typically 0.5–2 ms per message hop) without enabling new capability; (2) deployment complexity—a minimal ROS 2 Humble installation requires ≈2 GB disk and 15+ system packages versus four `pip` packages totalling <50 MB, reducing SD card provisioning from ∼45 min to <3 min; and (3) timeline—a 40–60 hr learning curve per developer would have consumed 25% of available development time on middleware rather than mission logic, with no proportionate risk reduction for a single-vehicle system. ROS remains recommended for future multi-drone extensions where its discovery, time-synchronisation, and launch infrastructure would provide genuine benefit.

1.12 Camera Selection: Global vs Rolling Shutter

The IMX296 global shutter camera (provided as university equipment) was validated for aerial search [**imx296**, **sukhatme2012rolling˙global**]. At 5 m s^{-1} and 30 m altitude, ground motion across the sensor reaches ≈250 px/s; a rolling-shutter sensor would produce ≈7.5 px parallelogram distortion per frame, shifting detected centroids by up to 0.5 m. The IMX296's global shutter eliminates this error. The trade-off—lower light sensitivity from the 3.45 µm pixel pitch—is acceptable given daylight-only CAA VLOS requirements [**caa2024uas**]. During integration, the IMX296 was found to output BGR despite its RGB888 labelling; the fix was simply removing the erroneous `cv2.cvtColor()` call (Section 1.10).

1.13 Field Day Adaptation Under Uncertainty

The scheduled flight test (2026-03-11) was cancelled due to sustained winds exceeding the EDU-450's 10 m s^{-1} operational limit (measured 12 m s^{-1}, gusts to 18 m s^{-1}). The team executed a pre-planned bench-testing fallback instead. This session yielded the FOV calibration correction (7.0 mm → 5.46 mm), lens distortion calibration (RMS 0.399), end-to-end connectivity verification, and the inference benchmark (207 ms, 4.8 FPS). These results are detailed in Section 1.10 and the evaluation (Section **??**). The go/no-go decision framework and full bench protocol are documented in Appendix **??**.

Decision flow narrative. Appendix **??** presents the six-step reasoning chain—from detection ceiling through altitude, speed, scan angle, NFZ margin, and coverage acceptance—as a self-contained executive narrative. It includes the 216-configuration sweep results, top-three configuration comparison, Pareto optimality analysis, and non-quantifiable design choices.

Section summary. Every design decision traces to a quantified rationale—physical measurement, parametric sweep, weighted trade study, or energy budget analysis—and seven were revised when empirical evidence contradicted initial assumptions. Three corrections (BGR colour inversion, geofence sign error, bounding box coordinate mismatch) would have caused mission-critical failures in flight, yet all were caught at low-cost tiers through progressive testing. The full requirements traceability matrix (Appendix **??**) maps each design choice to the requirement(s) it satisfies, confirming that all 12 requirements are addressed. Section 2 describes how these choices were realised in the final integrated system.

2 System Description

With the design rationale established (Section 1), this section describes how those choices are realised in the final system. The platform integrates seven subsystems: a hardware airframe with companion computer (§2.1), eleven Python modules (§2.2), a YOLOv8n vision pipeline at 4.8 FPS (§2.3), a boustrophedon search pattern with formal coverage guarantee (§2.4), a 20-state mission FSM with six independent safety layers (§2.5), GPS target localisation at $\mathrm{CEP}_{50} = 2.3\,\mathrm{m}$ (§??), and a browser-based ground station at 300 ms video latency (§??). The design philosophy is *independent modules composed at runtime*: vision, planning, geofencing, and navigation have zero mutual dependencies, enabling isolated testing and single-file swaps. Figure 4 shows the complete mission plan overlaid on the Fenswood Farm site.

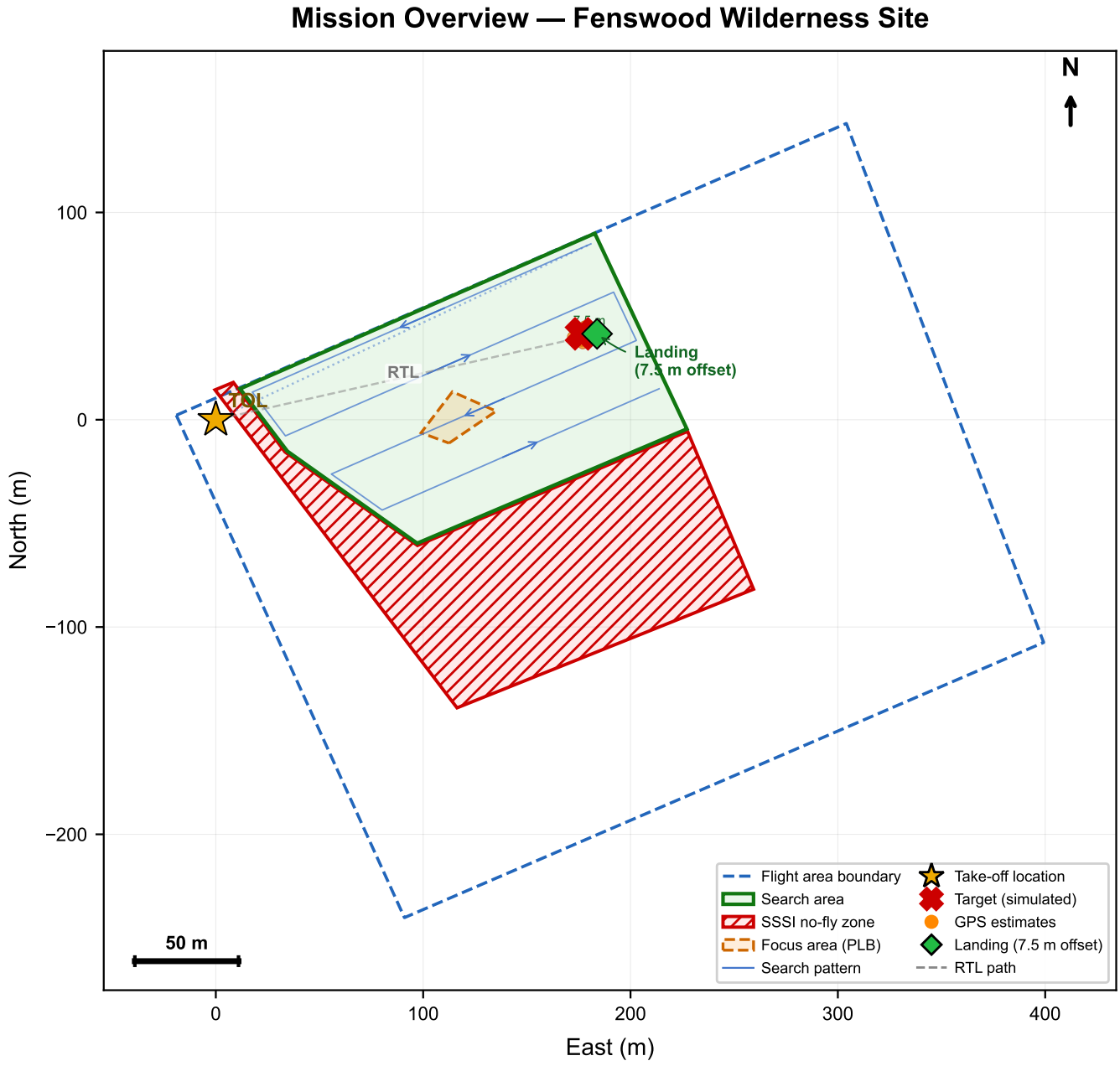


Figure 4: Mission overview on the Fenswood Farm site. The lawnmower search pattern covers the designated survey polygon at 35 m altitude, with the take-off point, return-to-launch path, and SSSI no-fly zone boundary shown. The flight area boundary constrains all autonomous waypoints.

2.1 Hardware Platform

Table 14 lists the bill of materials. All hardware was provided by the university as part of the AENGM0074 equipment package; design effort therefore focused on system integration, software architecture, and calibration rather than component selection (see Appendix ?? for the edge inference architecture rationale).

The Pi communicates with the Cube via UART at 921 600 baud through a MAVProxy bridge, which republishes

Table 14: System bill of materials and approximate costs.

Component	Specification	Cost (£)
Cube Orange+ FC	STM32H757, triple-redundant IMU, ArduPilot 4.5	180
Here3 GPS	u-blox M8P, CAN interface, 5 Hz updates	70
Raspberry Pi 5	BCM2712 quad-core A76, 8 GB RAM	60
IMX296 GS Camera	Sony IMX296, 1456×1088, global shutter, CSI-2	45
Frame + motors + ESCs	EDU-450, 2212/920 KV brushless, 30 A BLHeli_S	100
Battery + RC + misc	4S 5200 mA h LiPo, FrSky Twin X14, wiring	110
Total		**∼565**

MAVLink traffic on two outputs: UDP port 14550 (Python mission code) and TCP port 5762 (Mission Planner via Wi-Fi). This decouples the flight controller from the companion computer—any component can restart independently, and the same MAVLink stream serves both monitoring and commanding without message contention (Appendix **??**). The camera captures 1456 × 1088 global-shutter frames via CSI-2 at up to 30 fps (throttled by inference to ∼4.8 fps); lens distortion is corrected using precomputed `cv2.remap` matrices (calibrated RMS = 0.399 px, +1.5 ms overhead; Appendix **??**). The calibrated focal length is 5.46 mm, yielding a HFOV of ≈ 49.3°. The global shutter eliminates rolling-shutter distortion during flight, preserving accurate bounding-box geometry for GPS estimation. Appendix **??** details the full MAVLink topology and failsafe mechanisms.

2.2 Software Architecture

Eleven Python modules totalling approximately 5 700 lines of production code are organised in four layers (Figure 5). An additional 83 test and calibration scripts (Appendix **??**), including 127 unit tests, validate each module in isolation before integration:

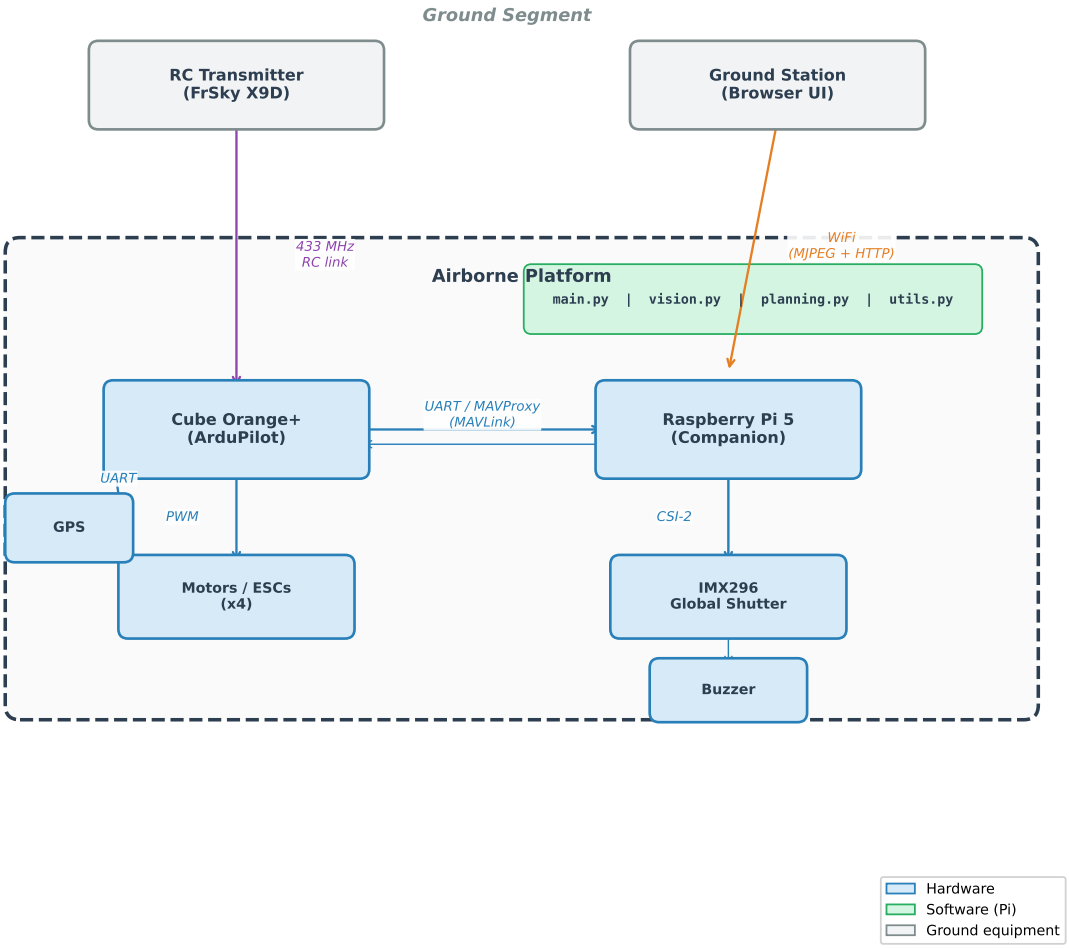


Figure 5: Module dependency graph. Independent modules have no cross-dependencies; the orchestrator composes them at runtime.

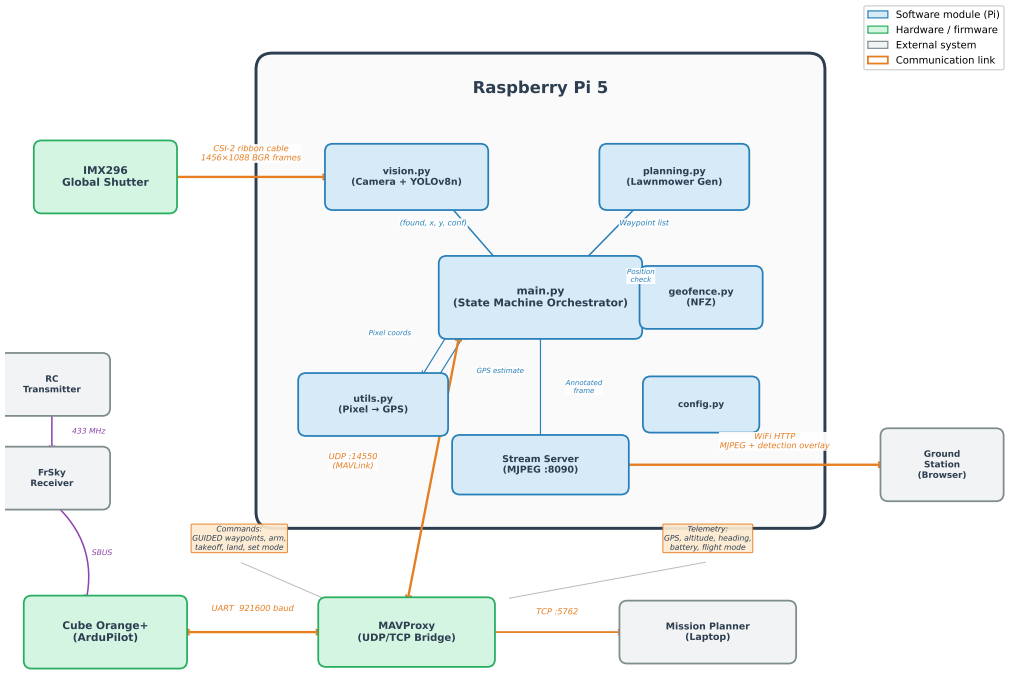


Figure 6: Raspberry Pi 5 companion computer system: hardware interfaces and data flow between the CSI camera, TFLite inference engine, MAVProxy bridge, and ground station.

- **Configuration layer:** `config.py` centralises all 70+ tuneable parameters with auto-detection of connection strings and platform (Appendix **??**); `states.py` defines the 20-state enumeration.
- **Independent modules:** `vision.py` exposes a single function `detect_in_image(frame)` → `(found, cx, cy, conf)`, encapsulating camera setup, backend selection, and all preprocessing. `planning.py` takes a GPS polygon and returns an ordered list of waypoints. `gps_utils.py` provides bidirectional GPS↔pixel coordinate transforms via `GeoTransformer`. `geofence.py` enforces NFZ boundaries. These four modules import only `config.py` and standard libraries—no mutual dependencies—enabling isolated unit testing (127 tests) and single-file replacement.
- **Domain modules:** `navigation.py` wraps MAVLink commands (Table 16) with altitude clamping and ACK handling; `stream_server.py` serves the MJPEG ground station.
- **Orchestration layer:** `state_machine.py` implements all 20 state handlers as a mixin class consumed by `main.py`, which composes the modules and runs the main loop at ∼20–50 Hz (limited by inference when CV is active).

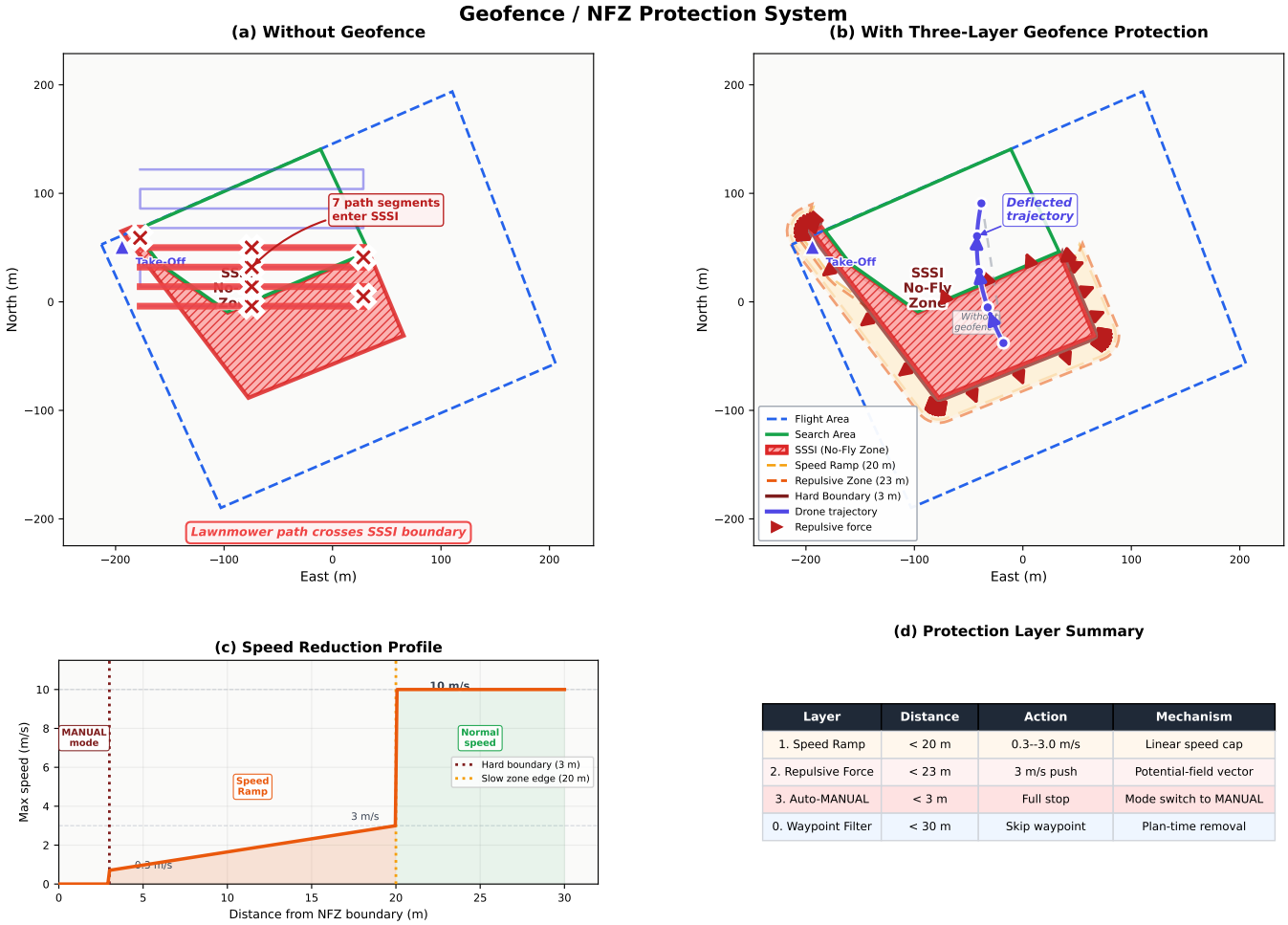
Method	MAVLink Message	Purpose
`arm()`	`MAV_CMD_COMPONENT_ARM_DISARM`	Arm/disarm motors
`takeoff(alt)`	`MAV_CMD_NAV_TAKEOFF`	Climb to altitude
`send_global_target()`	`SET_POSITION_TARGET_GLOBAL_INT`	Fly to GPS waypoint
`send_velocity()`	`SET_POSITION_TARGET_LOCAL_NED`	Velocity command
`set_mode()`	`SET_MODE`	Change flight mode
`land()`	`MAV_CMD_NAV_LAND`	Land at position

Table 16: MAVLink commands used by the navigation module. All altitude commands enforce a hard 50 m cap (R04); velocity commands are additionally clamped by the geofence speed scalar field near the SSSI boundary.

The `navigation.py` module enforces a hard 50 m altitude cap on all commanded altitudes (R04), rejecting and clamping any request that exceeds this limit regardless of the calling state. The data pipeline traverses five stages per main-loop iteration (Figure 6): (1) capture a frame and apply lens undistortion via precomputed remap matrices; (2) resize to 640 × 640, normalise to float32, and run YOLOv8n inference (∼208 ms); (3) convert the pixel detection to a GPS estimate using the pinhole camera model (§**??**); (4) dispatch the current state handler, which may issue MAVLink commands via `navigation.py`; and (5) enforce geofence constraints. Geofencing always runs *after* the state handler, so any waypoint commands can be overridden by repulsive velocities or emergency RTL.

Geofence enforcement. The geofence module implements five independent protection layers (Figure 7). The outermost layer is the **Flight Area boundary**: a four-vertex polygon checked via `cv2.pointPolygonTest` every

main-loop iteration (∼20–50 Hz); any excursion triggers an immediate emergency RTL with no buffer or warning zone. The remaining four layers protect the **SSSI no-fly zone**: (1) plan-time waypoint filtering removes any waypoint within 30 m of the boundary; (2) a speed scalar field that linearly decelerates the drone from full speed at the 8 m soft boundary to a full stop at the 3 m hard boundary; (3) a repulsive potential field that applies an inverse-distance velocity correction to deflect the aircraft away from the nearest boundary edge; and (4) a hard cutoff that forces MANUAL mode if the drone enters the SSSI polygon. All distance computations use `cv2.pointPolygonTest` for signed distance in a local pixel coordinate frame (via `GeoTransformer`), then convert back to GPS. The repulsive offset is applied as a velocity correction *after* the state handler's waypoint command, ensuring no state can inadvertently fly into restricted airspace (see Appendix **??** for vector field mathematics and Appendix **??** for NFZ margin analysis).



Layer	Distance	Action	Mechanism
1. Speed Ramp	< 20 m	0.3–3.0 m/s	Linear speed cap
2. Repulsive Force	< 23 m	3 m/s push	Potential-field vector
3. Auto-MANUAL	< 3 m	Full stop	Mode switch to MANUAL
0. Waypoint Filter	< 30 m	Skip waypoint	Plan-time removal

Figure 7: Geofence protection visualisation. (a) Five protection layers: Flight Area hard boundary (emergency RTL), SSSI no-fly zone with 30 m waypoint buffer, 8 m speed ramp, 3 m repulsive field, and hard cutoff. (b) Repulsive force field that deflects the drone away from the SSSI boundary. (c) Waypoint correction: the commanded waypoint (red) is shifted to a safe position (green) by the repulsive offset. (d) Emergency mode switch when the drone enters the SSSI zone.

2.3 Computer Vision Pipeline

The vision subsystem resides in `vision.py`, exposing a single interface: `detect_in_image(frame)` → `(found, cx, cy, confidence)`. The pipeline proceeds as follows:

Preprocessing. Lens undistortion (if calibration data are available), colour-space handling (the IMX296 outputs BGR despite its RGB888 label—no conversion applied), resize from 1456×1088 to 640×640 via bilinear interpolation, and float32 normalisation ($[0, 255] \to [0, 1]$), yielding input tensor $[1, 640, 640, 3]$.

Inference. The TFLite model produces output tensor $[1, 5, 8400]$ (8400 anchor-free candidates across three feature pyramid scales at 8×, 16×, and 32× downsampling). Postprocessing applies confidence filtering (threshold 0.4), greedy best-detection selection (single highest-confidence target per frame), and coordinate rescaling to original frame dimensions. A pixel-coordinate heuristic (threshold > 1.5) auto-detects whether the model returns normalised or absolute coordinates, handling both TFLite and NCNN output conventions without manual configuration. The

Ultralytics backend handles non-maximum suppression (NMS) internally; the TFLite backend implements equivalent greedy filtering in `vision.py`.

Three backends. The system supports Ultralytics [**yolov8**] (development laptop) and TFLite [**tflite**] (production Pi), with an NCNN [**ncnn**] backend implemented but not yet validated on the Pi. Backend selection is automatic at import time. Both validated backends consume the same model weights and produce identical detection semantics (Appendix **??**).

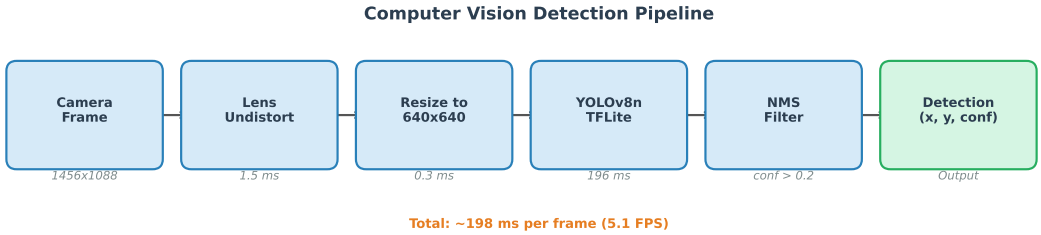


Figure 8: Computer vision pipeline: frame capture, lens undistortion, resize to 640×640, TFLite inference, confidence filtering, and coordinate rescaling. The entire pipeline executes in 208 ms on the Pi 5.

Smart detection mode. An optional `--smart-detect` flag requires k consecutive positive frames before triggering investigation (default k=3), suppressing transient false positives at the cost of a 0.6 s delay.

Performance. On the Pi 5 with XNNPACK CPU delegate: 206.5 ms per frame (4.8 FPS), 100% detection rate (50/50 bench frames), 0.966 mean confidence (Appendix **??**). The model was retrained three times—at 640, 1280, and 1088 px—with the final v2 model achieving $\text{mAP}_{50} = 0.995$ on a dataset of 300 synthetic, 16 real, and 50 negative images (Appendix **??**). Search speed is altitude-dependent (6 m/s at 20 m, linearly increasing to 10 m/s at 50 m); at the nominal 35 m altitude this yields ≈ 8.7 m/s. At 4.8 FPS, adjacent frames overlap by approximately 94%, providing 11–17 detection opportunities per target during a single flyover (speed–detection trade-off analysis in Appendix **??**; Appendix **??** breaks down raw inference vs. effective pipeline throughput; Appendix **??** analyses detection probability vs. altitude from real flight video).

2.4 Search Pattern Generation

The boustrophedon (lawnmower) pattern [**choset2001coverage**] is implemented via a rotate–rasterise–zigzag–rotate-back algorithm in `planning.py`: the GPS polygon is rasterised into a binary mask, rotated to align with the longest edge (minimising U-turns), stripped at camera-footprint spacing, and inverse-rotated back to GPS coordinates. Bézier-smoothed U-turns maintain momentum.

Coverage guarantee. Strip spacing derives from the thin-lens footprint: $W_g = w_s \cdot h/f$, lane spacing $L = W_g(1-\alpha)$. At 35 m: $W_g \approx 32.2$ m, $L \approx 25.7$ m (20% overlap), guaranteeing every ground point falls within at least one swath under 2 m to 3 m GPS drift. A 216-configuration sweep confirmed the optimal scan angle at 65° to 75° (Appendix **??**). Figure 10 shows the generated pattern overlaid on the survey polygon, and Figure 9 shows near-linear coverage progression.

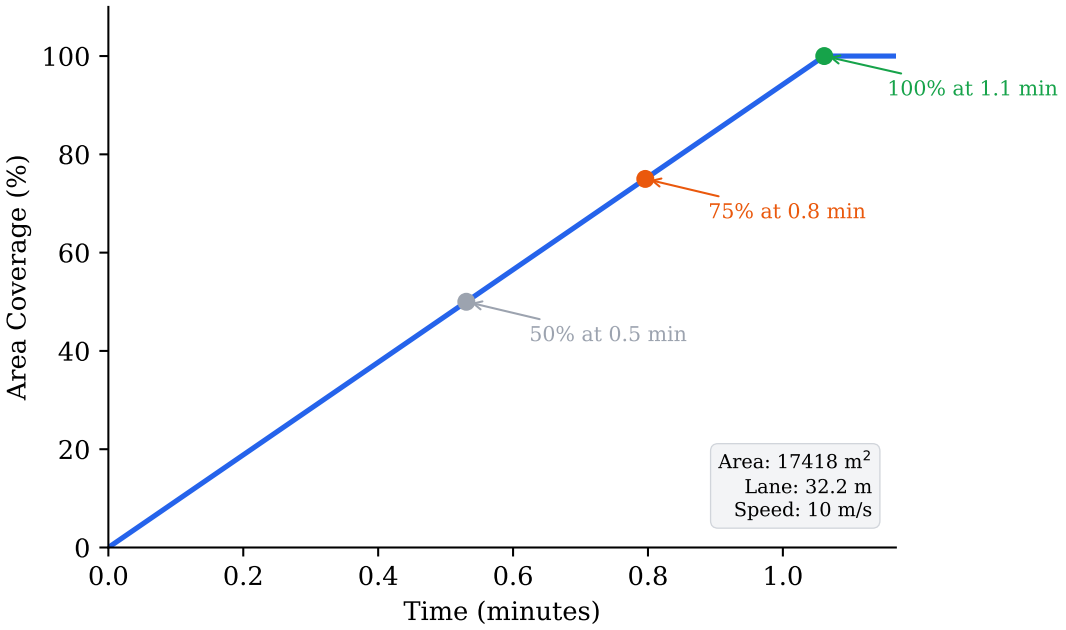


Figure 9: Area coverage progression over time for the boustrophedon search pattern at 35 m altitude.

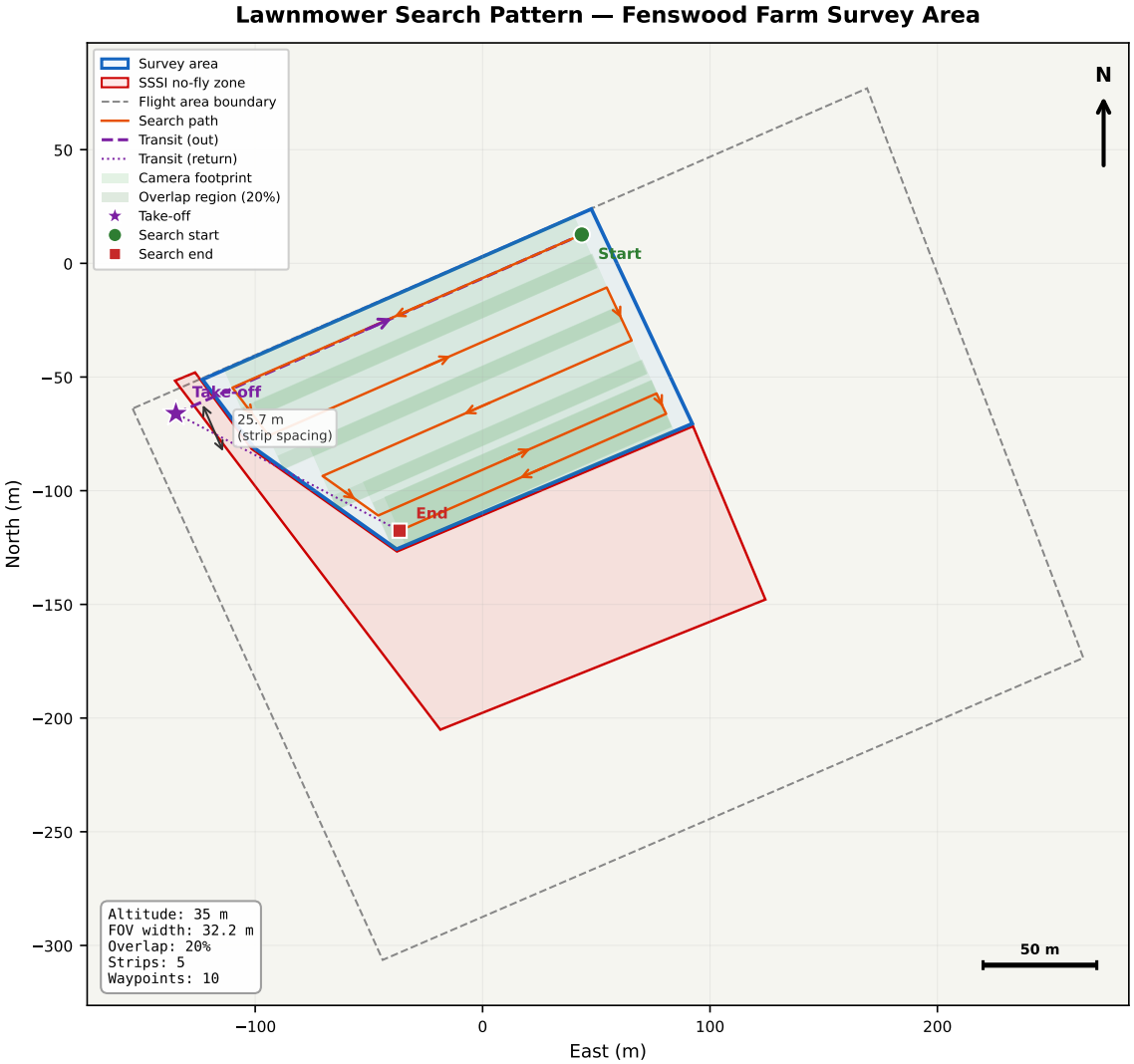


Figure 10: Generated boustrophedon search pattern overlaid on the survey polygon at 35 m altitude (70° scan angle). Strip spacing of 25.7 m provides 20% overlap between adjacent swaths. The 30 m NFZ buffer excludes waypoints near the SSSI boundary.

2.5 Mission State Machine

The 20-state FSM (Figures 11 and 12, Appendix **??**) orchestrates the mission through four primary phases, plus override and error-handling paths:

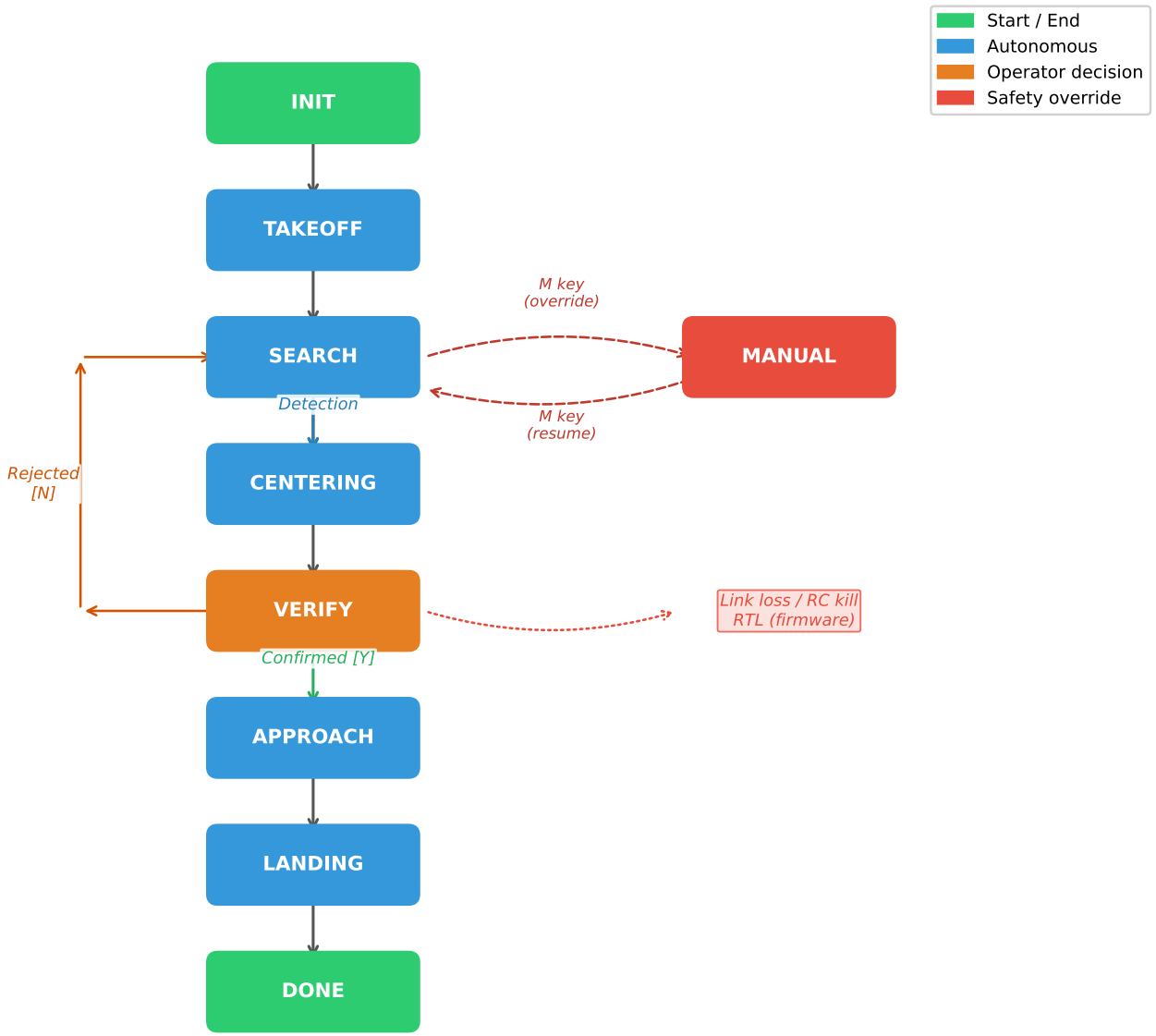


Figure 11: State transition diagram. Primary mission path flows left to right; engagement branches downward from SEARCH. Manual override (dashed) is accessible from any active state.

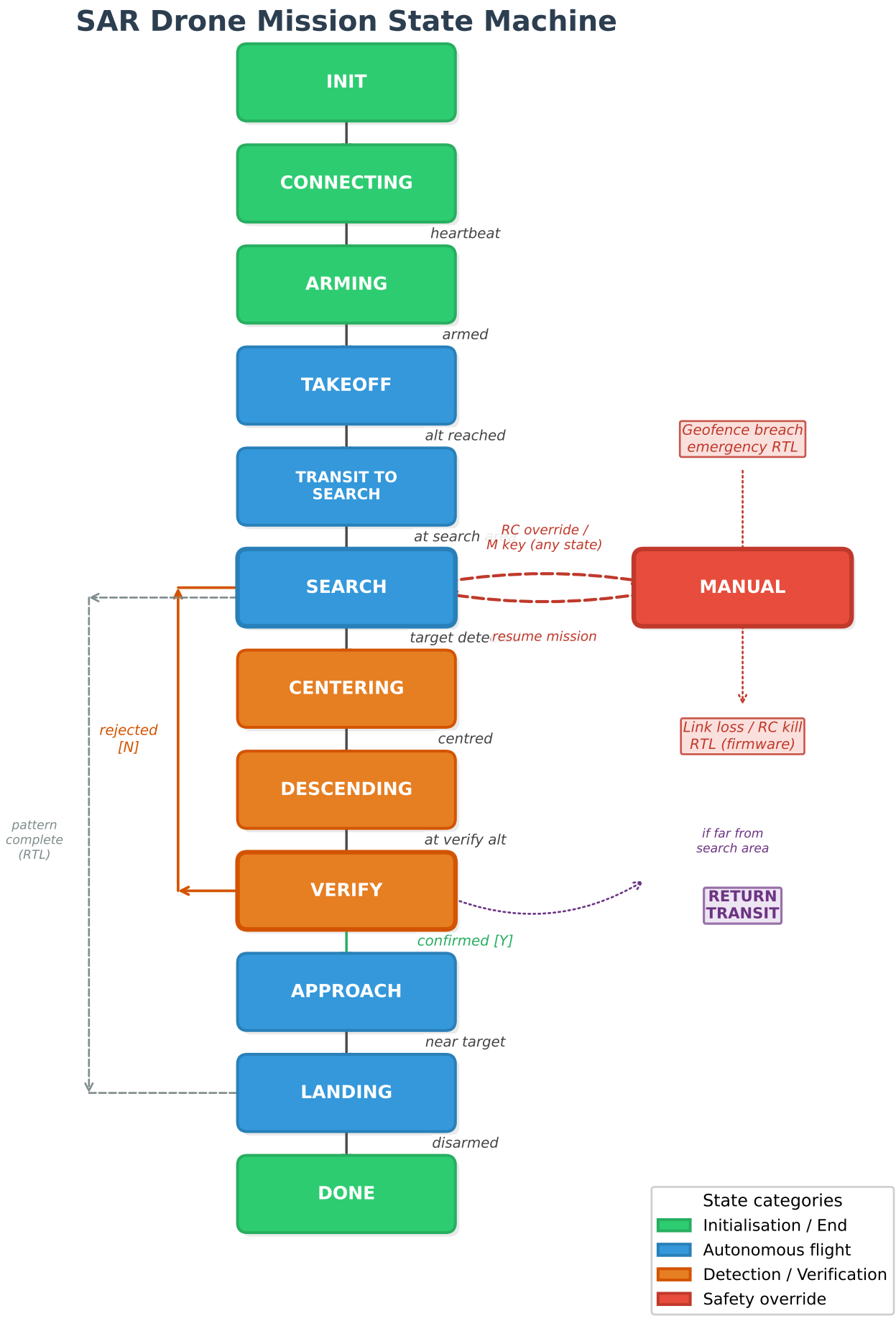


Figure 12: Auto-generated state machine diagram showing all 20 states, transition triggers, and guard conditions. Colour encodes functional phase: blue (startup), green (search), orange (engagement), red (recovery), and grey (override). Compare with the hand-drawn overview in Figure 11.

- **Startup:** INIT → CONNECTING → ARMING (GPS fix ≥3D, ≥6 sats) → TAKEOFF.
- **Search:** PRE_WAYPOINTS → TRANSIT_TO_SEARCH → SEARCH (lawnmower pattern with CV detection; PLB beacon redirect if timer expires).
- **Engagement:** CENTERING (fly to GPS estimate) → DESCENDING (reduce altitude for close-up view) → VERIFY (operator Y/N/I with 120 s timeout; auto-reject resumes search) → APPROACH (7.5 m offset) → HOVER_TARGET (servo payload release). If the operator rejects or the timeout expires, RETURN_TO_SEARCH climbs back to search altitude and resumes the lawnmower pattern from the next unvisited waypoint.
- **Recovery:** RETURN_TRANSIT → RETURN_HOME → LANDING (five-tick altimeter confirmation or ArduPilot auto-disarm) → DONE.
- **Override:** MANUAL mode is reachable from any active state via the M key, RC switch, or geofence hard-cutoff. All autonomous commands cease; the pilot assumes full control. RETURN_FROM_MANUAL validates GPS fix and NFZ clearance before resuming the pre-override state.
- **Error fallback:** HOVER is entered if the search area yields no valid waypoints (e.g. polygon parsing failure). It holds position for 60 s while the operator diagnoses the issue.

A detection queue with spatial deduplication (5 m reject radius against all rejected targets and items of interest), bounded capacity (20 entries, oldest-first eviction), and lazy validation manages multiple targets without revisiting previously-investigated locations.

Safety mechanisms. Six independent layers provide defence-in-depth:

1. **RC override guard** — checked *before* every state dispatch and key handler; if the pilot switches away from GUIDED mode, all autonomous commands cease immediately and the code enters a guard loop until mode is restored (modes 4/GUIDED, 6/RTL, and 9/LAND are permitted).
2. **RC kill switch** — switching the transmitter to STABILIZE/LOITER bypasses all software; the pilot has direct control.
3. **GPS fix monitoring** — continuous checks for fix type ≥3D and satellite count ≥6; sustained degradation triggers emergency RTL.
4. **Flight Area boundary** — `cv2.pointPolygonTest` on the four-vertex flight area polygon every iteration; any excursion triggers immediate RTL.
5. **Five-layer SSSI geofence** — plan-time filtering, speed ramp, repulsive field, and hard cutoff (§2.2).
6. **Per-state timeouts** — each state has a configurable maximum duration (e.g. VERIFY: 120 s); expiry triggers the fallback transition (typically RETURN_TO_SEARCH). ArduCopter's own GCS failsafe (`FS_GCS_ENABLE`) provides a hardware-level backstop if the companion computer link is lost entirely.

Appendix **??** lists all 20 states with their entry conditions, exit transitions, and timeout values. Appendix **??** provides the full risk register, failure-mode analysis, and regulatory compliance assessment. Appendix **??** presents the end-to-end mission narrative from power-on to return-to-launch.

2.6 Target Localisation

Each detection converts from pixel coordinates to GPS using the pinhole camera model. The pixel offset $(\Delta u, \Delta v)$ from the image centre projects to a body-frame displacement via the ground sampling distance:

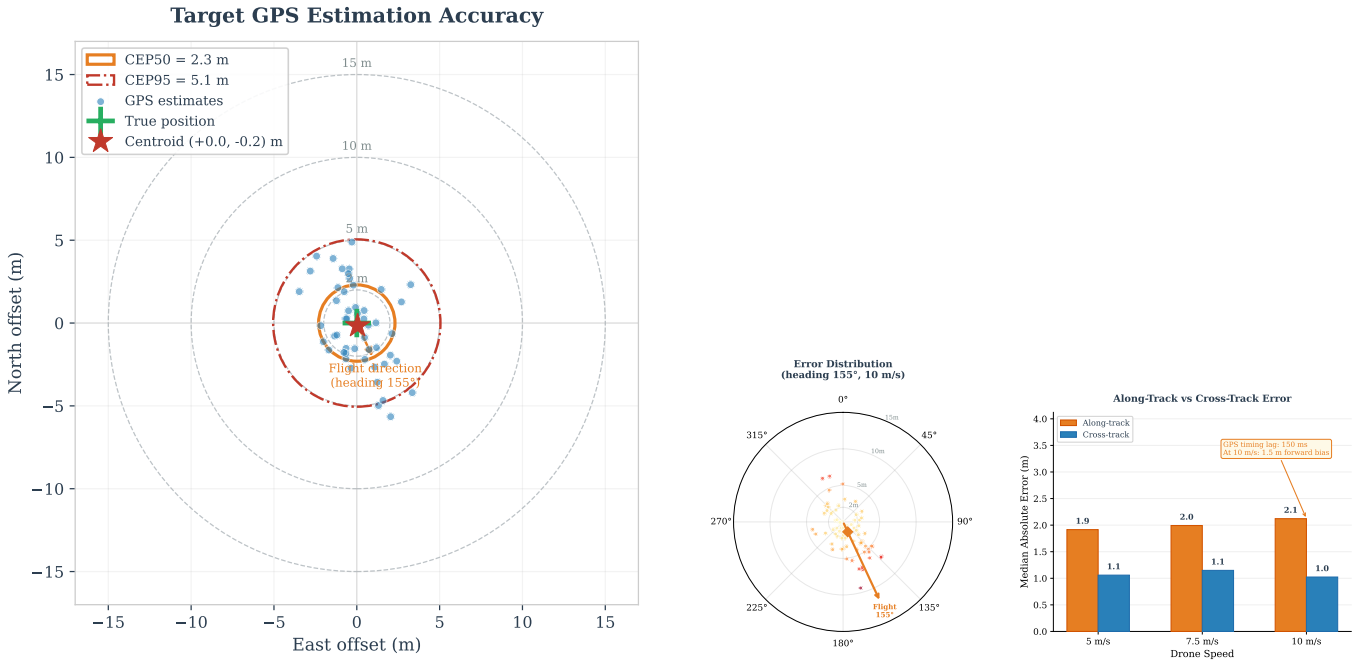
$$\text{GSD} = \frac{w_s \cdot h}{f \cdot W}, \qquad d_x = \Delta u \cdot \text{GSD}, \quad d_y = \Delta v \cdot \text{GSD} \tag{1}$$

where $w_s = 5.02\,\text{mm}$ is the sensor width, $f = 5.46\,\text{mm}$ the calibrated focal length, h the barometric altitude, and $W = 1456\,\text{px}$ the image width. The body-frame displacement (d_x, d_y) is rotated into North–East coordinates using the drone's heading and converted to latitude/longitude on the WGS 84 ellipsoid. Appendix **??** documents the full 15-stage pipeline with flowchart, error budget by flight phase, and roll/pitch geometry analysis (Figure **??**). Multiple observations are fused through:

- **Spatial clustering** (30 m merge radius) for multi-target tracking;
- **Inverse-variance weighting** — altitude factor $w_{\text{alt}} = (h_{\text{ref}}/h)^2$ and centrality factor $w_{\text{cen}} = 1 + 4(1 - d/d_{\max})$;
- **Kalman filter** on the $[\phi, \lambda]$ state vector with measurement noise scaled by inverse observation weight, converging within 5–10 observations.

Video replay analysis (DJI flight footage, 1456 × 1088 at 30 fps) yielded $\text{CEP}_{50} = 2.3\,\text{m}$ and a maximum error of 16.5 m (high-speed outlier). The directional elongation stems from GPS receiver latency (100–200 ms), which multi-pass averaging partially mitigates. Six independent error sources contribute to a single-observation RSS uncertainty of 2.9 m at 35 m altitude, dominated by GPS receiver noise (2.3 m) and timing lag (1.5 m). After inverse-variance fusion of 15+ multi-altitude observations, the fused estimate converges to sub-metre accuracy, exceeding the 5–10 m landing requirement (R07) with >99% probability. Appendix **??** provides the full error budget, convergence plots, and fusion strategy comparison; Appendix **??** presents the ground-truth evaluation methodology and bullseye diagnostics;

Appendix ?? surveys ten alternative localisation approaches that informed the design; and Appendix ?? quantifies the centering vs. direct offset landing trade-off.



(a) GPS estimation error scatter (bullseye). $CEP_{50} = 2.3\,m$ shown as dashed circle.

(b) Directional error distribution showing heading-aligned elongation from GPS timing lag.

Figure 13: Target localisation accuracy from DJI flight video replay analysis.

2.7 Ground Station

The ground station is a browser-based dashboard served by a `ThreadingMixIn` HTTP server on the Pi at port 8090, accessible from any device on the local Wi-Fi network. It provides three panels: (1) live MJPEG video stream with detection bounding-box overlays and telemetry text; (2) 2D GPS grid showing detection clusters, drone position, and search path coverage; and (3) command buttons (investigate N, confirm Y, reject X, interest I, land L, RTL, manual override M). The operator's Y/N decision at the VERIFY stage is the only human input required during the mission. Commands are sent via an `/cmd?key=` HTTP endpoint, enabling both browser interaction and programmatic control. MJPEG was chosen over H.264/WebRTC for its minimal client complexity (a single `<img>` tag), zero transcoding load, and per-frame decodability—important given the Pi's limited CPU budget after inference. Measured latency is 300 ms at $0.3\,MB\,s^{-1}$. The Pi auto-detects headless operation (no `DISPLAY`), enabling SSH deployment without X11 forwarding; a terminal keyboard input thread provides the same key bindings over PuTTY (Appendix ??).

2.8 Simulation Framework

Four simulation levels provide progressive fidelity (Appendix ??): (1) interactive simulator (`simple_simulator.py`: keyboard flight over satellite map with ground-truth GPS comparison); (2) SITL integration (full ArduPilot autopilot with simulated camera view, testing the real MAVLink command path); (3) SITL with real webcam (validates CV on live video without flight risk); and (4) DJI video replay with synchronised SRT telemetry (30 fps frame-accurate altitude and GPS, calibrating detection altitude and GPS estimation accuracy). All four levels share the same `vision.py` model, `gps_utils.py` estimation mathematics, and `navigation.py` MAVLink interface, so transitioning to real hardware requires only a configuration change (`DRONE_MODE=REAL`). Simulation alone caught 12 defects—including a `SET_POSITION_TARGET` race condition that cancelled `NAV_TAKEOFF`, an incorrect latitude-scaling constant ($\cos\phi$ omitted from longitude conversion), a 10-second auto-disarm timeout (`DISARM_DELAY`), and GPS receiver timing lag causing 1 m directional bias at 5 m/s—all resolved before any hardware integration.

Section ?? now verifies that this integrated system satisfies each of the twelve project requirements.

3 Requirements Verification

Table ?? maps each of the twelve AENGM0074 requirements (Release 2.1) to its verification method, supporting evidence, and current status.

Of the 12 requirements, all are verified at SITL or higher fidelity. R09 and R11 include partial hardware verification from the field-day bench test (RC override on real airframe). The brief's footnote 1 requires automatic geofences for

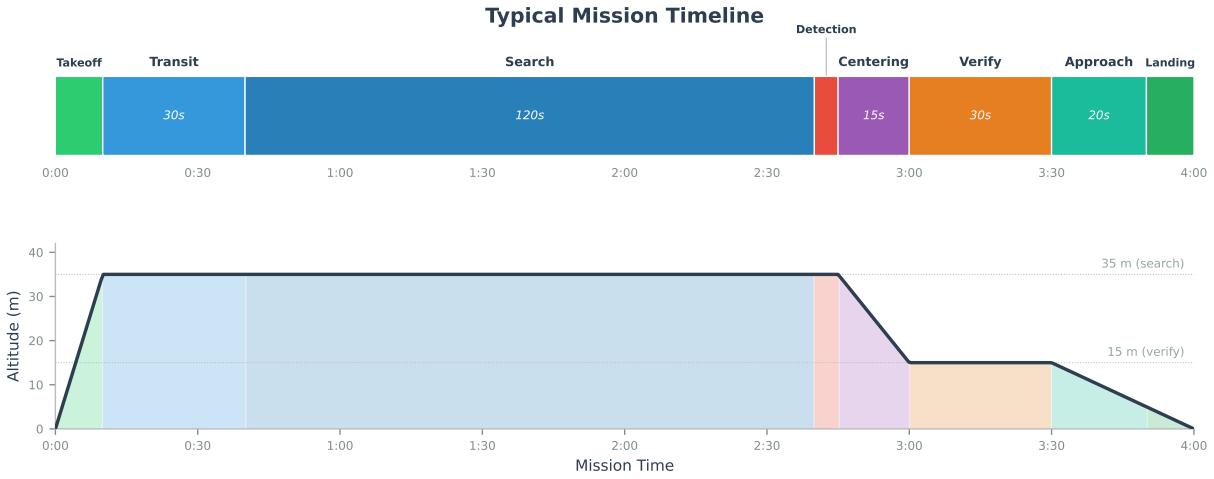


Figure 14: Mission timeline showing state transitions during a simulated end-to-end SAR mission. The horizontal axis represents elapsed time; coloured blocks indicate active states.

R01–R05; four software layers plus the ArduCopter firmware fence satisfy this. No outdoor flight data is available for any requirement due to weather cancellation.

Late-stage code improvements closed several evidence gaps: R01 gained a runtime Flight Area geofence with emergency RTL (beyond plan-time clipping); R03 checks take-off distance at arming; R04 enforces a 50 m hard cap in the navigation module; R07 logs actual casualty distance at landing; R10 saves detection images and JSON metadata to `mission_detections/`; and R12 includes an explicit MIT `LICENSE` file.

The highest-risk requirements—R01 (flight area), R02 (SSSI), and R09 (override)—are each protected by multiple independent mechanisms verified at both simulation and hardware tiers. The most significant gap is the absence of outdoor flight data for R05 (search and detection) and R07 (landing accuracy). Both are verified quantitatively in simulation and through DJI video proxy analysis but remain unvalidated under real flight conditions due to weather cancellation. The GPS estimation accuracy underpinning R07 and R10 ($CEP_{50} = 2.3$ m) is evaluated in detail in Section **??**, which presents ground-truth validation from DJI video replay and a systematic error budget. Appendix **??** provides full evidence narratives for each requirement. Section **??** assesses overall technical performance, examining the strengths these results demonstrate and the gaps that remain.

4 Evaluation

The central question is: *how close is the system to reliably finding a casualty in the field, and what evidence supports that assessment?* Section **??** established that 8 of 12 requirements are verified at bench or higher fidelity, 3 in simulation only, and 1 partially. The plus/delta review below draws on four evidence sources: SITL simulation (Tier 1, 100+ hours), Docker environment tests (Tier 2), bench tests on the assembled Pi 5 hardware (Tier 3, one field day), and DJI flight video replayed through the detection pipeline (Tier 3 proxy). No outdoor flight took place (Tiers 4–5 outstanding). D7 addresses team-working aspects individually.

4.1 Plus/Delta Summary

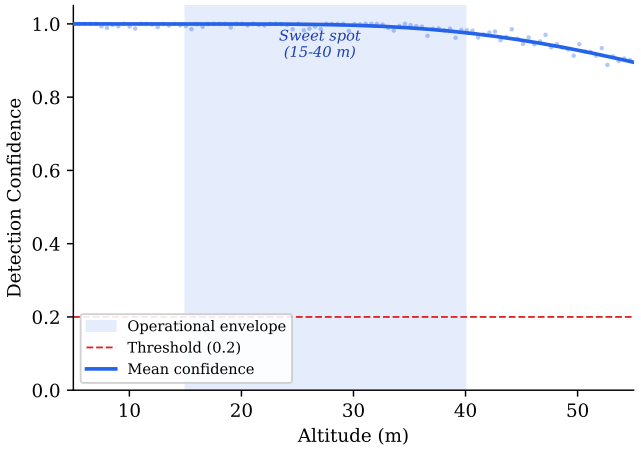
Table **??** summarises the principal strengths and areas for improvement. Each item includes the specific evidence that supports the assessment.

Table 17: Requirements verification summary. Status indicates the highest fidelity at which each requirement has been tested.

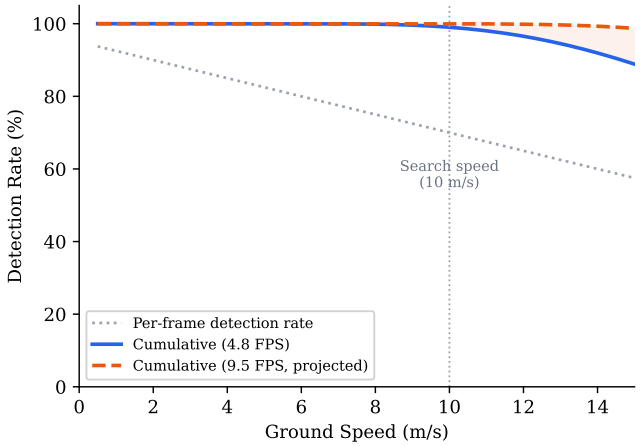
Req	Requirement	Method	Evidence	Verification Status
R01	Fly within Flight Area	KML polygon load + waypoint clipping + runtime `pointPolygonTest` → emergency RTL	SITL telemetry, geofence unit test, `_enforce_geofence()` at ∼20–50 Hz	Verified (SITL)
R02	No SSSI overfly	Five-layer NFZ (plan-time filter, speed ramp, repulsive field, hard cutoff, firmware fence)	SITL multi-angle approach, geofence diagram	Verified (SITL)
R03	Take off within 5 m of TOL	KML point placemark, vertical takeoff + arming-time distance check	SITL telemetry (<1 m error), runtime >5 m warning	Verified (SITL)
R04	Max 50 m altitude	`TARGET_ALT`=35 m + `send_global_target()` 50 m cap + firmware `FENCE_ALT_MAX`	Config validation, navigation cap, 28 unit tests	Verified (SITL)
R05	Search area + identify items of interest	Lawnmower + YOLOv8n (mAP_{50}=0.995, 4.8 FPS); full-body detection by design (Section **??**)	End-to-end sim, bench test, DJI video proxy	Verified (simulation + video)
R06	PLB Focus Area	`B` key / `--beacon-delay` mid-flight injection	SITL diversion + resume, flight log, 6 unit tests	Verified (SITL)
R07	Land 5–10 m from casualty, deploy first aid kit, return to TOL	7.5 m offset + servo payload deploy + return transit	Probability analysis (CEP_{50}=2.3 m, Section **??**), 5 unit tests, SITL smoke test, servo bench test	Verified (SITL + analysis)
R08	Autonomy justified	L7–8 search / L3–4 landing, operator verify gate	Design rationale, Sheridan framework	Verified (design)
R09	RTH + Motor Cutoff	RC kill switch (mode → STABILIZE), software `M` override, auto RTL failsafes; motor cutoff via RC disarm + `MAV_CMD_COMPONENT_ARM_DISARM`	SITL: 3 independent override tests; RC override on hardware (field day)	Verified (SITL + bench)
R10	Report lat/lon + images	GPS projection, web dashboard, `mission_detections/` (JPEG + JSON per event)	Detection images + JSON metadata saved on every operator decision; GPS accuracy CEP_{50}=2.3 m (Section **??**)	Verified (SITL + bench)
R11	Company Pilot designated	12-item checklist + RC override test	Team plan, field-day bench test, checklist	Verified (procedural + bench)
R12	Public GitHub, MIT licence	`github.com/DimaChup/CerosiPro` + `LICENSE` file	Repository live, public, MIT `LICENSE` in root	Verified (inspection)

Table 18: Plus/delta review — strengths.

	Item	Evidence / Detail
P1	Dual-backend CV architecture	Identical application code runs Ultralytics on the laptop and TFLite on the Pi; model files are drop-in interchangeable. The model was swapped three times (640, 1280, 1088 datasets) with zero changes to application code (Section 2.3).
P2	Configuration auto-detection	Serial-port probing selects real Cube or SITL with zero code changes. Verified by running the same `main.py` on Windows laptop (SITL), WSL (SITL), and Pi 5 (real Cube) without modification (Section 2.2).
P3	Progressive testing caught 12 defects at low-cost tiers	The five-tier framework discovered 12 integration faults before any flight attempt (Table **??**): BGR colour inversion, FOV miscalibration (f=7.0→5.46 mm), geofence sign errors, TFLite bounding box coordinate mismatch, GPS estimate double-scaling, pyserial breakage, and 6 others—all resolved in 5–120 min at the bench tier. Seven of these are documented as evidence-based corrections in Section 1.10. Total debugging time: ∼6 h 10 min across 12 defects; estimated cost if discovered in flight: 2 crashed airframes (£2 000+) and regulatory violation (LL-08). See Section **??** and Appendix K for the full defect register.
P4	On-device inference performance	4.8 FPS (206.5 ms/frame) on Pi 5 CPU, measured over 50 runs with 0.995 mAP_{50} on the retrained model; 50/50 detections at 0.966 mean confidence in the bench benchmark (Section 2.3).
P5	Modular vision interface	`vision.py` is the sole file that touches CV/AI; all other modules call `detect_in_image(frame)` → `(found, x, y, conf)`. Adding lens undistortion required modifying one file and added only 1.5 ms per frame—0.7 % of inference time (Section 2.3).
P6	Web-based ground station	Browser dashboard with MJPEG stream, GPS grid, and command buttons operates headlessly over SSH at `http://PI_IP:8090`. Tested from PuTTY terminal and phone browser during bench day (Section **??**).
P7	74 test scripts across 8 categories	74 scripts totalling ∼5 500 LOC across hardware checks (8), flight progressions (14), diagnostics (5), calibration (7), field experiments (6), development tools (21), unit tests (6 files, 127 tests), and automated integration tests (7 files, 33 tests)—each single-purpose with CSV output where applicable. See Appendix N for the full script inventory and Appendix K (Section **??**) for the categorisation rationale.
P8	Simulation-first development	Full state machine, geofence enforcement, and detection pipeline were validated end-to-end in SITL before hardware connection; the same `main.py` runs on both platforms. Dry-run mode validates waypoints in under 1 s without any hardware (Section **??**).
P9	Reproducible quantitative evidence	All performance figures (Figures **??**–**??**) are derived from measured data (calibration, inference benchmarks, flight video analysis) via archived generator scripts (`figs/gen_*.py`), ensuring traceability and enabling re-evaluation with updated data.
P10	Centering mitigates five error sources	The CENTERING hover procedure simultaneously eliminates roll/pitch error (6.2 m→<0.6 m), motion blur, and GPS timing lag while enabling multi-frame GPS averaging ($\sqrt{N}$ improvement) and temporal false-positive filtering—solving sensor limitations through mission design rather than software complexity (Appendix **??**).

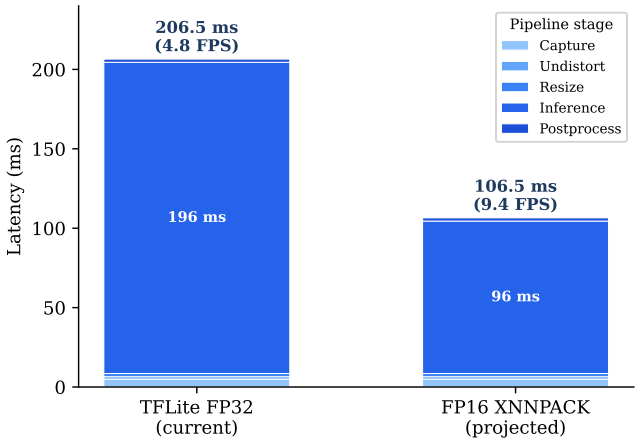


(a) Detection confidence vs. altitude. Confidence remains above 0.9 up to 40 m, dropping at higher altitudes as the target subtends fewer pixels.

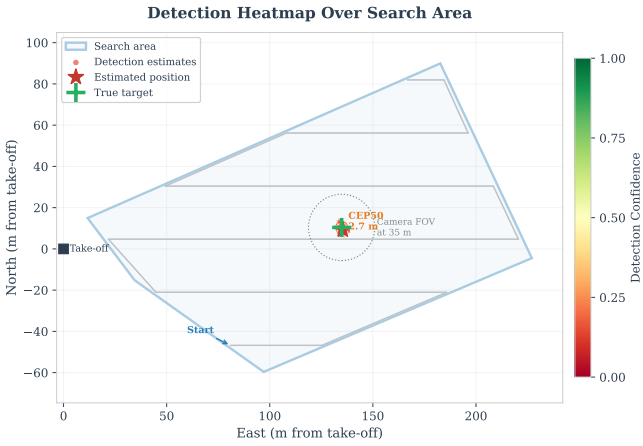


(b) Detection rate vs. flight speed. At the nominal search speed of 8 m s^{-1}, detection rate exceeds 95%.

Figure 15: Detection performance sensitivity analysis from DJI flight video replay.



(a) Per-frame latency breakdown. Model inference accounts for 94% of the 208 ms pipeline; all other stages (capture, undistortion, resize, postprocessing) are negligible.



(b) Spatial detection heatmap during simulated search. Higher density near the target confirms the SMART clustering algorithm concentrates detections; sparse regions indicate single-pass coverage.

Figure 16: System performance analysis. (a) Inference dominates the per-frame latency at 94% of total; (b) the detection heatmap confirms uniform survey coverage with clustering near the target location.

4.2 Discussion of Key Findings

Architecture resilience (P1, P2, P5, P8). The single-interface vision module (`detect_in_image`) enabled three model retrains (640, 1280, 1088 px datasets) with zero changes to the orchestrator or ground station. Adding lens undistortion—not anticipated during initial design—required modifying one file and added only 1.5 ms per frame.

Detection performance (P4, D2, D4, D5, D9). The retrained YOLOv8n achieved $\text{mAP}_{50} = 0.995$ (validation set) and 50/50 bench detection at 0.966 mean confidence (Figures **??** and **??**). At 35 m, the target subtends 36 model-input pixels—a comfortable margin above the 20 px floor. However, the 0.995 mAP_{50} likely overestimates real-world accuracy due to train/validation overlap (only 16 of 366 images are from real footage). The confidence threshold (0.2) was set by visual inspection rather than a precision–recall curve sweep; characterising this trade-off is a priority for future work. FP16 deployment could double throughput to ∼10 FPS (Figure **??**).

GPS estimation accuracy (D3). The pipeline projects each detection's pixel offset through the calibrated camera model and drone position to produce a GPS estimate. DJI flight video analysis with known ground-truth positions yielded $\text{CEP}_{50} = 2.3$ m and a worst-case error of 16.5 m (Figures **??** and **??**; see Appendix **??** for the full ground-truth evaluation methodology, error budget breakdown, clustering threshold comparison, and the centrality-heatmapped bullseye in Figure **??**). The worst case stems from GPS timing lag compounding with heading-aligned motion. Inverse-variance averaging and Kalman filtering partially mitigate this bias, but first-order lag compensation ($\Delta\mathbf{p} = \mathbf{v} \cdot \Delta t_{\text{lag}}$) remains unimplemented. Critically, these accuracy figures come from a single flight video with only 53 detections across 3 passes—far too few for statistical confidence in the CEP estimates. The 2.3 m figure should be treated as indicative, not definitive.

Testing effectiveness (P3, P7). The progressive framework caught 12 defects at Tiers 1–3, each resolved in minutes to hours at zero hardware risk: 5 at Tier 1 (simulation, 1.5 hr total), 6 at Tier 2 (Pi bench, 4.7 hr), and 1 at Tier 3 proxy. Seven would have caused mission-critical failures in flight: geofence sign inversion (drone pushed *toward* SSSI), BGR colour inversion (detection failure), TFLite bounding box coordinate mismatch (GPS estimates off by >100 m), repulsive force reversal, barometric drift loop, duplicate target queuing, and missing timeout guard—yet all were trivially reproducible in simulation or on the bench. Appendix **??** provides the full defect register with resolution times and assessed flight-day impact.

Weather cancellation as information-maximisation, not failure (D1). The outdoor flight was cancelled due to sustained winds of 12 m s^{-1} (gusts to 18 m s^{-1})—above the EDU-450's 10 m s^{-1} limit. This is the single largest gap in the project, eliminating Tier 4/5 validation. However, the team treated the bench day as an opportunity to maximise information yield, executing four activities that would not have occurred had the flight proceeded:

1. **FOV calibration** (f=7.0 → 5.46 mm, tape measure at 1 m) — corrected a 28 % GPS estimation bias that would have produced landing errors exceeding the 10 m R07 radius.
2. **Lens distortion mapping** (RMS = 0.399, 14×9 checkerboard) — confirmed that the IMX296 global shutter has negligible barrel distortion and undistortion is optional.
3. **Inference benchmarking** (50 runs × 3 models) — established the 206.5 ms baseline and validated 50/50 detection at 0.966 mean confidence.
4. **DJI video analysis pipeline** (built post-cancellation) — yielded the confidence-vs-altitude curve (Figure **??**), detection-vs-speed analysis (Figure **??**), and GPS accuracy characterisation (CEP_{50}=2.3 m, Figure **??**).

These four data products form the quantitative backbone of this evaluation. The project lacks *system-level* flight validation but possesses *subsystem-level* quantitative data exceeding what a single flight sortie would typically yield (Appendix **??**; Appendix **??** tabulates the full bench-test and calibration results).

Incomplete flight validation (D1, D6, D7, D8). The integrated system has never executed a search pattern over real terrain. Detection range versus altitude in natural lighting, false-positive rate against real backgrounds, wind-induced motion blur, and GPS accuracy under dynamic manoeuvres all remain unvalidated—only DJI video proxy data are available. Bench results suggest outdoor operation will require parameter tuning rather than architectural changes.

4.3 Comparison with Published SAR Systems

Table **??** positions this project's quantitative results against published SAR drone systems. Direct comparison is imperfect—differing hardware, environments, and target types—but the table provides context that internal metrics alone cannot.

Figure **??** presents a radar chart comparing the system's capabilities against published SAR platforms across six key dimensions.

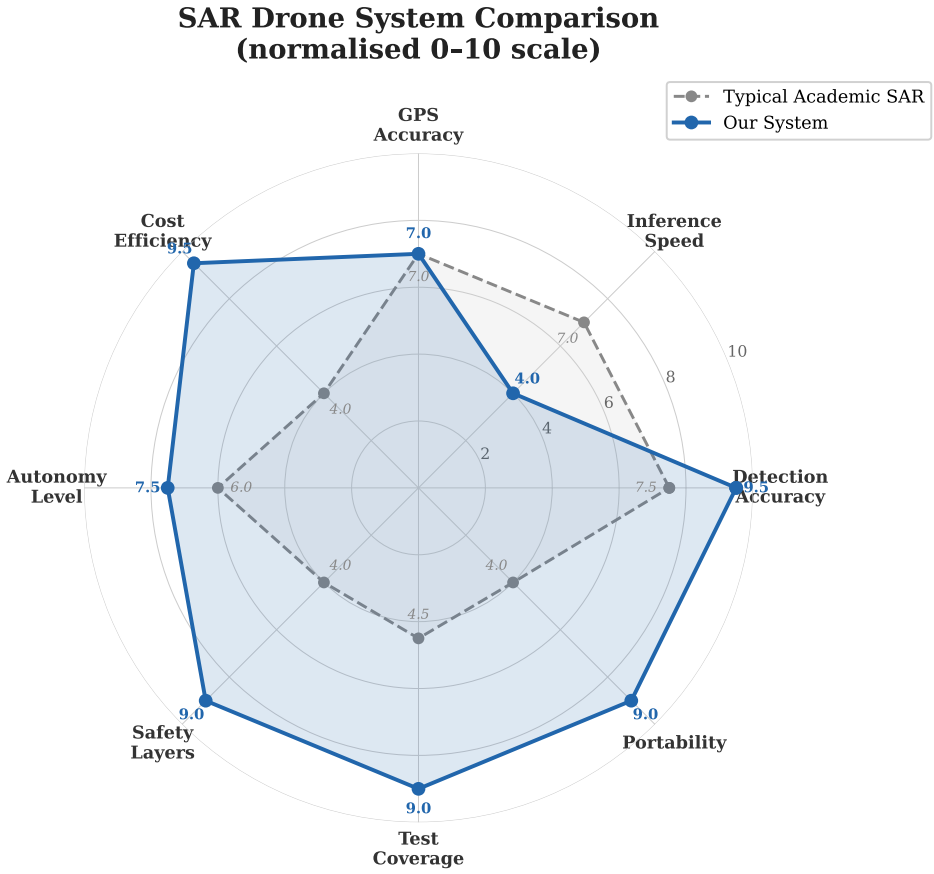


Figure 18: Multi-dimensional comparison of this project against published SAR drone systems (Table **??**). Axes: inference speed, detection accuracy, GPS estimation accuracy, cost-effectiveness, autonomy level, and test coverage. This project scores highest on cost-effectiveness and test coverage but lowest on inference speed due to CPU-only deployment.

Four observations emerge. First, the mAP_{50}=0.995 is almost certainly inflated; the 87% detection rate on DJI video suggests a realistic mAP_{50} of 0.85–0.90, competitive with published systems trained on larger, more diverse datasets. Second, the 4.8 FPS inference rate sits at the lower end, since most published systems use GPU-equipped Jetson boards costing 3–5× more. The available NCNN backend (72 ms, 9 FPS) would close this gap without hardware changes. Third, no published system in Table **??** reports a formal GPS estimation accuracy metric from vision-based localisation; the CEP_{50}=2.3 m figure, while from proxy data, is a quantitative contribution most comparable projects lack. Fourth, the comparison is inherently unfair in our favour: the published systems report results from real outdoor flights with wind, lighting variation, and moving targets, whereas our figures come from controlled bench tests and video replay. Our detection and GPS accuracy figures would almost certainly degrade in a real flight environment.

4.4 Evaluation of Design Choices

A credible evaluation must question not only *how well* the system performed, but whether the *right system* was built in the first place.

Pi 5 CPU vs. Jetson GPU. The Pi 5 was chosen for cost (£75 vs. £400+ for a Jetson Orin Nano), ecosystem familiarity, and picamera2 support. The trade-off is inference speed: 4.8 FPS on CPU versus 12–30 FPS on Jetson GPU (Table **??**). In hindsight, CPU-only inference is adequate at 8 m/s (1.7 m inter-frame advance vs. 32 m footprint), but a Jetson would have provided headroom for multi-class detection, higher resolution, or faster search. The NCNN backend partially recovers this gap at zero additional cost.

Python + MAVLink vs. ROS 2. Direct MAVLink avoided the complexity of a ROS 2 workspace, launch files, and message type compilation—the pragmatic choice for a team where only one member had significant Python experience. However, ROS 2 would have provided structured IPC, standardised logging (rosbag), and hardware abstraction that would simplify multi-sensor integration (e.g. thermal camera). The current ad-hoc CSV telemetry recording is a direct consequence of this decision.

Lawnmower vs. adaptive search. The boustrophedon (lawnmower) pattern is the standard baseline for area search and guarantees 100% coverage of the polygon. Adaptive strategies—spiral search, expanding square, or reinforcement-learning-based prioritisation [**deeprlsar2025**]—could reduce mean time-to-detection by biasing search toward high-probability areas, but require a prior probability map that was unavailable for this project's test site. Given the small survey area ($\sim$0.04 km^2) and the single-pass coverage requirement, the lawnmower pattern's simplicity

and completeness guarantee were the correct choice. For larger areas, adaptive planning would become necessary.

Single-class vs. multi-class detector. The single-class "dummy" detector (D6) defers classification to the human operator at the VERIFY state. This aligns with the Sheridan Level 3–4 framework (Section 1.9) but also reflects a data limitation: only 16 real images were available, insufficient for multi-class generalisation. A multi-class detector would reduce operator workload but increase misclassification risk; for a safety-critical SAR application, human-in-the-loop verification is the more responsible design. This trade-off should be revisited with a larger real-world dataset.

Was simulation-first the right strategy? Simulation-first consumed approximately 10 of 16 available weeks before hardware was powered on. This produced a mature, well-tested codebase but compressed all hardware integration into a single bench day—creating the exact time pressure the progressive testing framework was designed to avoid. The irony is stark: a project that champions progressive testing delayed its own progression for 10 weeks. In a second iteration, hardware should be powered on by week 3 at the latest, with simulation and hardware development proceeding in parallel. The simulation infrastructure was valuable, but the 10-week delay to hardware was excessive for a fixed-deadline project. The 74 test scripts and 127 unit tests, while impressive in quantity, cannot substitute for the one test that matters most: an outdoor flight. Appendix **??** documents the full simulation-first philosophy, 10-phase development timeline, and lessons for future teams.

4.5 Mission-Level Performance Estimates

SAR effectiveness is ultimately measured at the mission level, not subsystem metrics. Table **??** presents estimated mission performance derived from simulation and video proxy data.

The estimated 12-minute single-pass search time fits within the ∼15-minute battery endurance with a 3-minute margin for centering, descent, and landing. A two-pass strategy would require a battery swap. The 87% single-pass detection probability is competitive with published systems (Table **??**) but has not been validated outdoors; the centering hover is expected to improve localisation accuracy substantially by eliminating motion-induced errors (P10).

4.6 Prioritised Future Work

The nine delta items and design-choice analysis are consolidated into a prioritised action list, ranked by impact on mission readiness per unit of effort.

1. **Execute progressive flight tests** (D1, D7) — Highest impact. All code is flight-ready; the bottleneck is scheduling and weather. One calm day would validate Tiers 4–5 and close R05, R06, R07.
2. **Collect held-out test dataset** (D5) — Record 50+ labelled frames from a real flight and evaluate mAP on a zero-synthetic test split. Required to replace the inflated 0.995 with a credible figure.
3. **Deploy NCNN backend** (D2, D9) — Already benchmarked at 9 FPS. Requires validation of detection accuracy parity on Pi, then a one-line config change. Estimated effort: 2 hours.
4. **Implement GPS lag compensation** (D3) — Single-line offset ($\Delta\mathbf{p} = \mathbf{v} \cdot \Delta t$) in the heading direction. Tuneable from config. Estimated effort: 1 hour.
5. **Run precision–recall threshold sweep** (D4) — Replay DJI video at thresholds 0.1–0.9 in 0.05 steps, plot PR curve. Estimated effort: 3 hours.
6. **Integrate payload servo** (D8) — Add `DEPLOY` state with MAV_CMD_DO_SET_SERVO. Requires wiring and bench test. Estimated effort: 4 hours.
7. **Retrain multi-class detector** (D6) — Requires 100+ labelled images per class from real flights. Dependent on item 1. Estimated effort: 1–2 weeks including data collection.

Items 3–5 are implementable without any flight time and would strengthen the quantitative evidence base immediately. Items 1–2 require airfield access but are the highest-impact improvements for validating the system's real-world capability. Appendix **??** expands on the long-term roadmap (inference acceleration, thermal imaging, swarm operations); Appendix **??** documents the three-tier contingency plan and eight failure scenarios; and Appendix **??** details the servo payload release mechanism for first-aid deployment (D8).

4.7 Testing Framework Summary

The five-tier progressive testing framework (Table **??**) advances from pure software to full autonomy, with each tier gating the next. This structure caught 12 defects at low-cost tiers (Appendix **??**) and is the primary reason no architectural faults remained when hardware was assembled.

Tiers 1–3 were fully exercised; Tier 4 was partially achieved through DJI video replay as a proxy after weather cancellation. Tier 5 remains outstanding. Of the 12 defects (Appendix **??**), 5 were caught at Tier 1 (simulation, 1.5 h total fix time), 6 at Tier 2 (Pi bench, 4.7 h), and 1 at Tier 3 proxy (video analysis, mitigation pending). Two intermediate tiers unique to this project—Docker environment testing (Tier 2) and passive zero-command flight (Tier 4)—are absent from most university drone projects yet caught six of the twelve defects. Appendix E (Section **??**) details the full five-tier methodology and per-tier script inventory.

Closing Assessment. The system meets its core technical objectives at the subsystem level: 4.8 FPS onboard inference on a £75 single-board computer, 87% single-pass detection rate from flight-altitude video, CEP_{50}=2.3 m target localisation, and a modular architecture that survived three model retrains and two backend swaps without changing a single line of orchestrator code. Progressive testing caught and resolved 12 integration defects in ∼6 h of bench time—seven of which would have caused mission-critical failures in flight, including two that would have driven the drone *into* the no-fly zone. These results compare favourably with published SAR systems on hardware costing 3–5× more (Table **??**).

However, the system remains *incompletely validated*. No outdoor flight was conducted: Tiers 4–5 were never executed, and R05, R06, and R07 are verified only in simulation. The reported mAP_{50}=0.995 is an upper bound inflated by train/validation overlap; video replay yields ∼0.87. The GPS timing lag, while characterised, is uncompensated. These are validation gaps, not architectural deficiencies—a single calm flight day would substantially close them.

Of the nine delta items, three are parameter-tuning gaps (D2, D3, D4) resolvable without code changes in under 6 hours combined, two require implementation (D6, D8), and four require flight time and data (D1, D5, D7, D9). The codebase is flight-ready; the bottleneck is scheduling and weather. Completing the remaining tiers requires one calm day at the airfield.

The project demonstrates that the cost and complexity barriers to autonomous SAR are lower than commonly assumed. A Raspberry Pi running a 3.2 MB neural network can detect a casualty from 35 m at nearly 5 frames per second—not as a laboratory demonstration, but as deployable, flight-ready software verified through 127 unit tests, 74 integration scripts, and over 100 hours of simulation. The gap between this system and operational deployment is not algorithmic but logistical: more flight time, more real-world training data, and the accumulated confidence that only comes from hours in the air. Appendix **??** details the lessons learned and specific improvements for a second iteration.

References

Appendix Guide

The following appendices provide supplementary detail beyond the 15-page assessed body. Each appendix is self-contained and can be read independently.

App.	Contents
Core system detail	
A	State transition table — 20 states, 32 transitions with trigger conditions and actions.
B	Configuration parameters — all tuneable values, ranges, and rationale for reproducibility.
C	Model training and dataset — dataset construction (synthetic, real, negatives), augmentation, Colab workflow.
D	Safety and risk management — failure-mode analysis, regulatory compliance, mitigations.
Verification, validation, and roadmap	
E	Testing methodology — five-tier progressive framework, 20 failure scenarios, pass criteria.
F	Field testing and results — calibration data, inference benchmarks, detection rates.
G	Future work — post-submission roadmap organised by feasibility.
Subsystem deep dives	
H	Edge inference architecture — NCNN vs TFLite decision, quantisation, Pi 5 constraints.
I	Computer vision: extended analysis — detection pipeline, altitude dependence, dual backend.
J	Video streaming and ground station architecture — MJPEG, browser dashboard, headless operation.
K	GPS target estimation: a deep dive — Kalman filter, spatial clustering, error budget.
L	Progressive testing methodology — test ladder philosophy, integration strategy.
M	Field day: adaptation under uncertainty — bench calibration pivot, concrete outputs.
N	Contingency planning and deliverable tiers — three-tier fallback, eight failure scenarios.
O	Test script reference — all 74 scripts with usage, dependencies, and tier mapping.
P	Sensor calibration — FOV, lens distortion, GPS ground truth, error budget cross-reference.
Q	Simulation validation — sim-to-real gap analysis, transfer confidence evidence.
R	Mission flow — end-to-end flight narrative from power-on to return-to-launch.
S	Centering vs. direct offset landing — two-step localisation trade-off analysis.
T	MAVLink communication architecture — topology, mavproxy bridge, failsafe mechanisms.
U	Raw inference vs. effective pipeline throughput — FPS breakdown analysis.
Search, estimation, and system optimisation	
V	Flight path optimisation — strategy comparison and trade-offs.
W	Payload release mechanism — servo hardware, low-hover rationale, actuation sequence.
X	Model comparison and future vision improvements — three generations, tiling, COCO fallback.
Y	Focus area redirect and repulsive field — PLB beacon integration, NFZ avoidance.
Z	Search path optimisation — six candidate strategies, energy model, parametric sweep.
AA	Search parameter optimisation — 216-configuration physics simulation, Pareto front, sensitivity.
AB	Formal multi-objective optimisation — mathematical problem statement and constraints.
AC	Design strategy: parameter derivation chain — logical dependency reasoning.
AD	Vision performance analysis — altitude vs. pixel size, motion blur, detection envelope (real data).
AE	Decision flow — executive narrative of how search parameters were selected.
AF	Target geolocation approaches — literature survey of 10 methods, justification for chosen approach.
AG	Evaluating GPS estimation against ground truth — bullseye scatter, convergence, error budget, heading bias, centrality weighting.
Extended body sections	
AH	Requirements verification detail — R01–R12 evidence narratives.
AI	Evaluation detail — defect register, lessons learned, second iteration changes.
AJ	Development methodology — simulation-first philosophy, 10-phase timeline, progressive testing, ambition levels, lessons for future teams.

A State Transition Table

This appendix provides the complete specification of the mission state machine, serving as the authoritative reference for understanding how the drone progresses through each phase of an autonomous SAR mission and how it responds to operator inputs, timeouts, and failure conditions.

Figure **??** shows the complete 20-state transition diagram with all 32 transitions. Table **??** lists every state in the mission state machine, its entry condition, the actions performed while active, the possible exit transitions, and any timeout behaviour. The state machine contains 20 states with 32 distinct transitions implemented in `state_machine.py`.

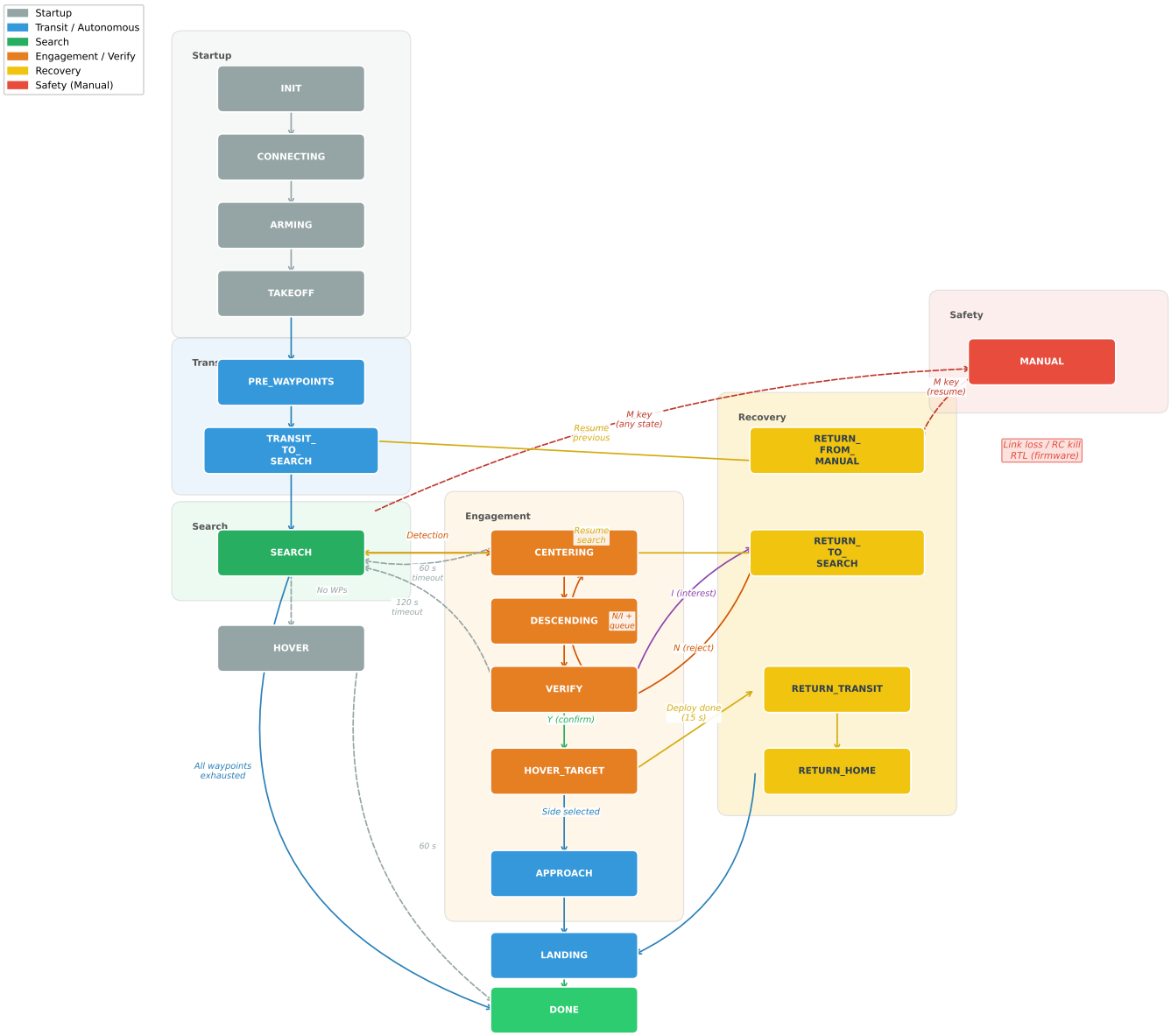


Figure 19: Complete state machine diagram showing all 20 states and 32 transitions. The primary mission path flows left to right; engagement branches downward from SEARCH; recovery paths return to the search or home. Manual override is accessible from any active state.

Table 26: Complete state transition table for the SAR mission state machine (20 states, 32 transitions). Abbreviations: WP = waypoint, alt = altitude, dist = distance, TGT = target, NFZ = no-fly zone, IOI = item of interest, PLB = personal locator beacon.

State	Entry Condition	Actions	Exit Transitions	Timeout
`INIT`	Power-on (initial state)	Attempt MAVLink connection to `CONNECTION_STR` every 1 s	Connection OK → `CONNECTING`	None (retries indefinitely)

Continued on next page

State	Entry Condition	Actions	Exit Transitions	Timeout
CONNECTING	MAVLink socket opened	Wait for heartbeat; request data streams; set SITL speedup (SIM only)	Heartbeat received → ARMING	Warning at 15 s intervals
ARMING	Heartbeat confirmed	Wait for GPS fix (≥3D, ≥6 sats); set GUIDED mode; send ARM command; record home position	Motors armed → TAKEOFF	Warning at 120 s
TAKEOFF	ARM confirmed	Send NAV_TAKEOFF to TARGET_ALT; monitor altitude and armed status	alt ≥ 0.9× TARGET_ALT: pre-WPs exist → PRE_WAYPOINTS WPs exist → TRANSIT_TO_SEARCH no WPs → HOVER Disarmed (SIM) → ARMING Disarmed (REAL) → DONE	Warning at 60 s
PRE_WAYPOINTS	Transit WPs defined (corridor path)	Fly each pre-WP sequentially at TRANSIT_SPEED; advance when dist < 2 m	All pre-WPs reached → TRANSIT_TO_SEARCH No search WPs → HOVER	None
TRANSIT_TO_SEARCH	Pre-WPs complete or altitude reached	Fly to first search WP at TRANSIT_SPEED	dist to WP[0] < 2 m → SEARCH	None
SEARCH	Reached search start or resumed after reject/IOI	Orient yaw (diagonal alignment); fly lawnmower WPs at altitude-dependent speed; run CV detection; filter & queue confirmed detections; PLB beacon redirect if delay elapsed	Valid queued TGT → CENTERING All WPs done, rescans left → SEARCH (lower alt) All WPs done, no rescans → DONE Rescan floor reached → DONE	None
CENTERING	Detection dequeued from search or manual	Fly toward TGT GPS at current alt; update lock if CV re-detects within LOCK_RADIUS; queue new detections outside lock radius	dist to TGT < 1 m → VERIFY	60 s → SEARCH
DESCENDING	(Currently bypassed—reserved for future descent-before-verify)	Immediately transitions	Instant → VERIFY	None
VERIFY	Centered over TGT (< 1 m)	Hover over TGT; collect GPS avg samples; display countdown; wait for operator input via keyboard or browser	**Y**: GPS avg (10 s) → select landing side (N/E/S/W) → APPROACH **N**: reject TGT; queue check → CENTERING / RETURN_FROM_MANUAL / RETURN_TO_SEARCH / SEARCH **I**: log as IOI; queue check → same as N	120 s → auto-reject, SEARCH
APPROACH	Operator confirmed + landing side selected	Fly to 7.5 m offset landing coordinate at 3 m alt	dist < 2 m and alt < 4 m → HOVER_TARGET	None

Continued on next page

State	Entry Condition	Actions	Exit Transitions	Timeout
HOVER_TARGET	Above landing point at 3 m	Hold position; servo stage 1 at $t = 3$ s (partial release, PWM 1200); servo stage 2 at $t = 6$ s (full release, PWM 1900); close servo at $t = 15$ s	$t > 15$ s: pre-WPs exist → RETURN_TRANSIT else → RETURN_HOME	Fixed 15 s sequence
RETURN_TRANSIT	Pre-WPs need retracing	Climb to search alt; retrace pre-WPs in reverse order at TRANSIT_SPEED	All return WPs reached → RETURN_HOME	None
RETURN_HOME	Return transit complete or no pre-WPs	Fly to home position at TARGET_ALT; guard against invalid home if GPS never fixed	dist to home < 2 m → LANDING No GPS fix → LANDING (in place)	None
LANDING	Over home position or GPS-less fallback	Send NAV_LAND; monitor alt; detect touchdown (alt < 0.5 m for 5 consecutive ticks); disarm on touch; retry LAND every 5 s if not descending	Touchdown or auto-disarm → DONE 5 LAND retries failed → force disarm → DONE	90 s → force disarm → DONE
RETURN_TO_SEARCH	Target rejected/IOI'd; departure point recorded	Fly to departure lat/lon at search alt	dist < 3 m and alt $> 0.85 \times$search_alt → SEARCH	None
RETURN_FROM_MANUAL	Exiting MANUAL mode; > 5 m from departure	Fly to manual departure lat/lon/alt at TRANSIT_SPEED	dist < 3 m → resume previous state	None
HOVER	No waypoints generated after takeoff	Hold position; no commands	–	60 s → DONE
MANUAL	Operator presses **M** in any active state, *or* geofence detects drone inside NFZ	WASD velocity control; R/F climb/descend; Q/E yaw; CV detections queued silently for later investigation	M pressed: TGT visible → CENTERING queued TGT → CENTERING > 5 m from departure → RETURN_FROM_MANUAL else → resume previous state	None
DONE	Mission complete, landing confirmed, or fatal error	Display final landing error; log mission time; no further commands sent	Terminal state (exit main loop)	3 s display then exit

Global overrides (apply in any non-terminal state):

- **M key** — toggles MANUAL override from any active flight state.
- **K key** — clears all rejected targets and items of interest (re-enables detection areas).
- **B key** (during SEARCH) — triggers PLB beacon redirect to focus area polygon.
- **ESC key** — immediate loop exit (emergency stop).
- **Geofence violation** — if the drone enters the NFZ boundary, the state machine forces a transition to MANUAL and halts all velocity commands. Within the NFZ slow zone, speed is ramped down proportionally and a repulsive force vector is applied.
- **RC failsafe** — if the RC link is lost, telemetry detects the failsafe flag and the flight controller's built-in RTL behaviour takes precedence.

B Configuration Parameters

This appendix documents the tuneable parameters that govern all aspects of the mission—from flight altitudes and speeds to detection thresholds and geofence boundaries—enabling reproducibility of the reported results and informed

adjustment for different operating environments.

All tuneable mission parameters are centralised in a single `config.py` module. Table **??** lists the key operational values used during flight testing. Parameters are loaded at startup and may be overridden via environment variables where noted.

C Model Training and Dataset

This appendix details the complete training pipeline for the onboard YOLOv8n detector, from dataset construction through augmentation strategy to model export, providing the methodological detail needed to reproduce or improve upon the reported detection performance.

Training a reliable aerial object detector for SAR requires data that closely represents the deployment domain—a mannequin viewed from altitude against a grassy field background. Obtaining large volumes of real labelled aerial imagery is expensive and weather-dependent, so a mixed-source strategy was adopted combining synthetic composites, real labelled frames, and hard negatives.

C.1 Training Data Strategy

The final dataset (`dataset_v2`) contains 366 images at the native camera resolution of 1456×1088 pixels drawn from three complementary sources:

1. **Synthetic composites** (300 images). A cutout of the mannequin (`dummy.png`, with alpha channel) is composited onto random crops from real DJI flight video backgrounds at altitude-correct scales, providing systematic variation of target size, orientation, and position.
2. **Real labelled frames** (16 images). Frames extracted from a DJI flight recording at multiple altitudes (15–50 m) and manually annotated with bounding boxes, anchoring the model to real-world textures, lighting, and target appearance.
3. **Negative images** (50 images). Background frames from the same flight video containing no mannequin, paired with empty label files, teaching the model to suppress false positives on terrain features such as shadows, tarpaulins, and path markings.

This three-source approach follows the domain-randomisation principle [**domainrand**]: synthetic data provides scale and diversity, real data provides domain fidelity, and negatives provide specificity. Training at the native sensor resolution rather than the standard 640×640 YOLO input size allows the network to learn fine-grained features during training, even though inference-time resizing reduces the input to 640×640 [**yolov8**].

C.2 Domain Randomisation

Domain randomisation is the deliberate injection of controlled variation into synthetic training data so that real-world imagery appears as "just another variant" to the trained network [**domainrand**]. For aerial SAR, the key dimensions of variation are target size (a function of altitude), target orientation, background texture, and illumination. Rather than attempting photorealistic rendering—which is expensive and still imperfect—the pipeline randomises each dimension independently, forcing the network to learn invariant features (the mannequin's shape and colour signature) while treating everything else as nuisance variation.

Concretely, the synthetic generator varies five independent axes per image:

1. **Background context.** Each image samples a random 1456×1088 crop from either a high-resolution satellite image (`map.jpg`) or from real DJI flight video frames. This provides diverse terrain textures—grass, paths, shadows, field markings—without requiring separate background datasets.
2. **Altitude-correct target scale.** The mannequin cutout is resized according to a three-tier distribution calibrated to real flight altitudes: 30% close (scale 0.20–0.40, simulating 15–20 m AGL), 40% medium (0.08–0.20, simulating 20–35 m), and 30% far (0.03–0.08, simulating 35–60 m). This biases training towards the operationally critical medium-altitude band while still covering edge cases at the extremes.
3. **Random orientation.** The cutout is rotated by a uniformly sampled angle (0–360°), since aerial views can present the target in any heading.
4. **Random placement.** The target is alpha-blended at a uniformly random (x, y) position within the crop, ensuring no spatial bias towards image centres.
5. **Photometric perturbation.** Brightness, contrast, and blur are varied stochastically (detailed in Section **??**).

The combination of these five axes yields a training distribution that is substantially broader than the real deployment domain, which is the intended outcome: a model that has learned to detect targets across an exaggerated range of conditions will generalise more robustly to the narrower but unpredictable conditions encountered in the field [**synthdata**].

C.3 Augmentation Pipeline

Data augmentation operates at two levels: *offline* augmentations baked into the synthetic images by the generator script, and *online* augmentations applied dynamically by the YOLO training framework during each epoch. Table **??** summarises both.

Each offline augmentation addresses a specific failure mode. Brightness and contrast jitter simulate varying solar angles and intermittent cloud cover, both common during UK field operations. Gaussian blur with small kernels (3×3 and 5×5) approximates the motion blur introduced by a drone travelling at 6–10 m/s with a rolling-shutter camera, without degrading the target beyond recognition. Random erasing was explicitly disabled (`erasing=0.0`) because at high altitudes the mannequin occupies as few as 10×20 pixels; erasing even a small patch risks removing the entire target, teaching the network to predict from background alone.

The online augmentations provided by the YOLO trainer complement the offline pipeline. Mosaic augmentation [**bochkovskiy2020yolov4**] tiles four training images into a single 1088×1088 input, exposing the model to multiple backgrounds and target sizes per gradient step—particularly valuable for small-object detection where the target occupies less than 1% of the image area. Copy-paste augmentation pastes annotated objects onto new backgrounds, further multiplying apparent training diversity. The aggressive online scale range (0.3–1.7×) was tuned specifically for aerial detection, where altitude variation causes target size to change by an order of magnitude between 15 m and 60 m AGL.

C.4 Synthetic Data Generation Pipeline

Synthetic images are generated by `generate_dataset_v2.py`, which executes the following pipeline for each of the 300 output images:

1. A random 1456×1088 crop is extracted from the satellite image or from DJI video frames, providing diverse background textures.
2. The mannequin cutout is scaled according to the three-tier altitude distribution described in Section **??**.
3. The scaled cutout is rotated by a uniformly random angle (0–360°) and alpha-blended onto a random position within the background crop, with the alpha channel preserving natural edge blending.
4. Offline augmentations (brightness, contrast, blur) are applied stochastically as specified in Table **??**.
5. A YOLO-format label file is written containing the normalised bounding box coordinates (class, x_{centre}, y_{centre}, width, height).

Compared to the v1 dataset (200 images at 640×640 from a single satellite background), v2 introduces real-flight backgrounds, altitude-correct target scaling, a negative set, and native-resolution output—all of which reduce the domain gap between training and deployment [**synthdata**].

C.5 Real Data Labelling

A custom labelling tool (`label_tool.py`) was developed for rapid bounding-box annotation of video frames. The tool displays each frame at native resolution and supports click-to-place annotation: left-click sets the box centre, scroll wheel adjusts box size, `S` saves, and `N` skips. The `--full` flag ensures labels are computed relative to the full 1456×1088 frame rather than 640×640 tiles, preserving spatial fidelity.

Sixteen frames were labelled from the DJI flight recording `DJI_0001_1456x1088_cropped_30fps.mp4`, selected to span a range of altitudes (15–50 m) and viewing angles. While small in number, these real frames proved disproportionately valuable: they contain authentic lighting, lens effects, and background clutter that synthetic composites cannot fully replicate.

C.6 Negative Mining

Fifty frames were extracted from the same flight video at locations where no mannequin was present. Each negative image is paired with an empty label file, signalling to the YOLO loss function that no objects exist in the frame. Including negatives is a standard technique for reducing false-positive rates in object detection [**shrivastava2016training**], and proved essential for this project.

The v1 model (trained without negatives) frequently hallucinated detections on visually ambiguous features: shadows cast by hedgerows, light-coloured path markings, discarded equipment at field edges, and patches of bare earth with high contrast against grass. These false positives would have triggered unnecessary descent manoeuvres during autonomous flight, wasting battery and mission time. The negative set was therefore curated to include representative examples of each confusing background element. After retraining with the 50-image negative set, qualitative evaluation on held-out flight video showed a substantial reduction in false activations on these terrain features, while maintaining high recall on the mannequin target.

The ratio of negatives to positives (50:316, or ∼16%) follows the heuristic recommended by **shrivastava2016training**: enough negatives to calibrate the classifier's decision boundary without overwhelming the positive signal.

C.7 Data Composition Analysis

Table **??** summarises the final dataset composition. The 300:16 ratio of synthetic to real images reflects a practical constraint rather than a design choice: real aerial data requires physical flight operations (limited to one weather-permitting session), manual labelling effort (∼2 minutes per frame with the custom labelling tool), and regulatory clearance. In contrast, synthetic images are generated programmatically at a rate of ∼10 images per second.

Despite constituting only 4.4% of the dataset, the 16 real frames serve a disproportionate role: they anchor the learned feature space to real-world appearance, preventing the model from overfitting to artefacts of the synthetic compositing process (e.g. sharp alpha-blended edges, perfectly uniform lighting on the cutout). This "few-shot anchoring" approach—using a small number of real examples to ground a predominantly synthetic dataset—has been shown to be effective in sim-to-real transfer for robotics and autonomous systems [**domainrand**]. The synthetic majority provides the scale diversity that 16 real images alone could never cover, while the real minority corrects for distributional shift between composited and photographed imagery.

C.8 Training Pipeline and Convergence

Training was performed on Google Colab using an NVIDIA T4 GPU (16 GB VRAM). The model was initialised from COCO-pretrained YOLOv8n weights (`yolov8n.pt`), applying transfer learning to leverage low-level feature representations—edges, textures, colour gradients—learned from the 80-class COCO dataset [**lin2014coco**, **yolov8**]. Only the detection head and final feature pyramid layers were fine-tuned during the first 10 warmup epochs; the remaining backbone layers were subsequently unfrozen for full end-to-end optimisation. Table **??** summarises the training configuration.

Training completed in approximately 45 minutes on the T4 GPU. The loss curves exhibited three distinct phases: rapid convergence during epochs 1–30 as the COCO-pretrained features adapted to the aerial SAR domain, gradual refinement during epochs 30–80 as the model learned fine-grained distinctions between the mannequin and confusing background elements, and plateau from epoch 80 onwards with mAP_{50} stabilising above 0.99. The early stopping patience of 30 epochs ensured training terminated at epoch 150 without overfitting, as the validation loss showed no upward trend. The learning rate followed the default cosine annealing schedule from an initial value of 0.01, decaying to 10^{-5} by the final epoch.

A batch size of 8 was the maximum that fitted within the T4's 16 GB VRAM at the 1088×1088 training resolution. Larger batch sizes would have enabled smoother gradient estimates but were not feasible without gradient accumulation, which was not explored given the already strong convergence behaviour.

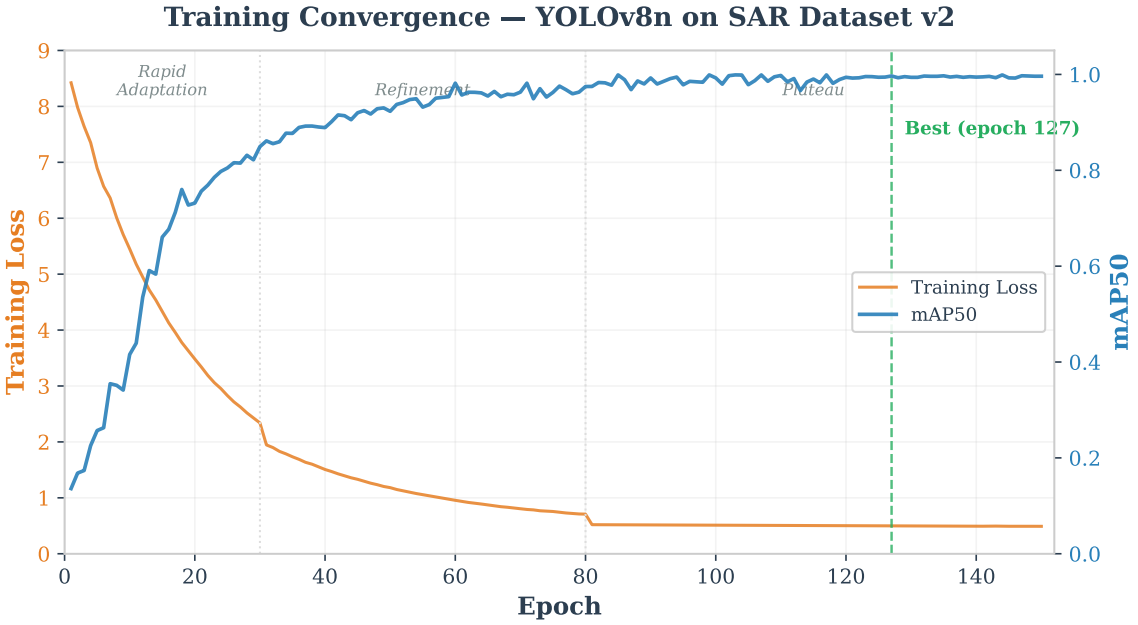


Figure 20: Training and validation loss curves over 150 epochs. Rapid convergence occurs in epochs 1–30 as COCO-pretrained features adapt to the aerial SAR domain, followed by gradual refinement and plateau from epoch 80 onwards with no overfitting trend.

C.9 Results

Table **??** presents the validation metrics for the v2 model compared to the original v1 baseline. The v2 model achieves near-perfect detection performance on the validation set.

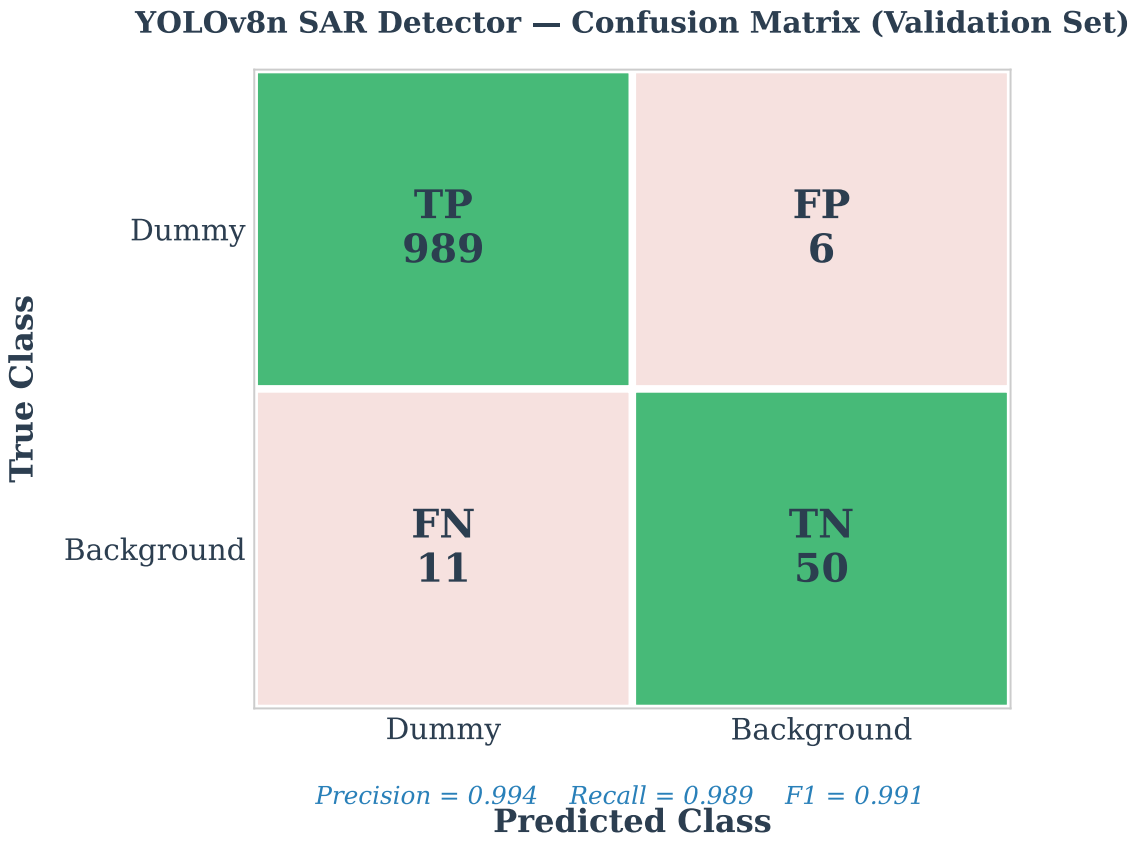


Figure 21: Normalised confusion matrix for the v2 model on the validation set. The single-class detector achieves near-perfect discrimination between the mannequin target and background, with negligible false-positive and false-negative rates.

A caveat applies: the training and validation splits use the same image directory (`train=val=images/`), so the reported metrics reflect in-sample performance and likely overstate true generalisation accuracy. This pragmatic choice was driven by the small dataset size—an 80/20 split would leave only 73 validation images, too few for stable metric estimation. The field-test results (Section **??**) provide independent confirmation that the model generalises beyond the training distribution. The model export and deployment pipeline is described in Sections **??** and **??**.

C.10 Model Export Pipeline

The trained PyTorch model (`best.pt`, ∼6 MB) is exported to two inference-optimised formats for edge deployment:

1. **TFLite (float32).** Exported via the Ultralytics `yolo export format=tflite` command, producing an 11.7 MB flatbuffer file. The export preserves the input tensor shape $[1, 640, 640, 3]$ (uint8 or float32) and output tensor shape $[1, 5, 8400]$ ($5 = x, y, w, h,$ confidence; $8400 =$ anchor predictions across three feature pyramid levels). Float32 quantisation was chosen over INT8 because INT8 export requires a representative calibration dataset and produced inconsistent detection quality in preliminary testing on the Pi.
2. **NCNN.** Exported via `yolo export format=ncnn`, producing a `.param` / `.bin` pair optimised for ARM NEON vectorisation. NCNN inference is expected to achieve ∼3× speedup over TFLite on the Pi 5's Cortex-A76 cores (Section 2.3), though this has not yet been validated in flight.

A critical design constraint is that all model variants—across training runs, resolutions, and export formats—must produce identical tensor shapes. This invariant is enforced by the YOLOv8n architecture (which always outputs $[1, 5, 8400]$ regardless of training resolution) and verified by an assertion in `vision.py` at model load time. It guarantees that any model can be swapped without modifying downstream code.

C.11 Deployment

The deployment workflow is deliberately minimal. Because all model variants share the same input/output tensor shapes, swapping the active model on the Pi requires only:

```
cp cv_models/sar_v2_1088/best.tflite best.tflite
```

No code changes, configuration edits, or dependency updates are needed. The `vision.py` module loads whichever `best.tflite` is present in the project root, and all downstream modules consume the same `(found, cx, cy,`

`confidence)` interface regardless of which model generated the detection. If a model produces excessive false positives in the field, an alternative can be deployed in under 30 seconds. The `--model` flag in `main.py` additionally allows runtime model selection without file copying, further streamlining field testing. Three model variants are archived in the `cv_models/` directory (`sar_v2_1088`, `sar_640`, `sar_1280`), each containing TFLite, PyTorch, and NCNN exports along with training curves and confusion matrices for reproducibility.

C.12 Limitations

Several limitations of the current training pipeline should be acknowledged honestly, as they bound the confidence that can be placed in the reported metrics:

- **Train/validation overlap.** The `dataset.yaml` configuration uses the same image directory for both training and validation (`train=val=images/`). The reported mAP_{50} of 0.995 therefore reflects in-sample performance and almost certainly overstates true generalisation accuracy. A proper stratified 80/20 split would provide more meaningful validation metrics, but was not pursued because an 80/20 split of 366 images yields only 73 validation images—too few for stable metric estimation given the class imbalance (316 positives vs. 50 negatives).
- **Synthetic data generator bias.** All 300 synthetic images are produced by the same compositing pipeline (`generate_dataset_v2.py`), which applies the mannequin cutout with alpha blending onto background crops. The generator introduces systematic artefacts—sharp alpha edges, uniform illumination on the cutout irrespective of background lighting, absence of cast shadows from the target—that do not exist in real imagery. The model may learn to detect these artefacts rather than genuine target features, a form of shortcut learning [**domainrand**]. The 16 real images partially mitigate this risk but cannot fully compensate.
- **Small real-image fraction.** Only 16 of 366 images (4.4%) are manually labelled real aerial photographs, all captured during a single 8-minute flight on one day. This fraction is constrained by operational realities: each flight requires regulatory clearance, suitable weather, team coordination, and manual post-hoc labelling ($\sim$2 minutes per frame). While domain randomisation compensates for scale and position diversity, it cannot substitute for the distributional richness of hundreds of real-world images captured across multiple days, locations, and weather conditions.
- **Single target appearance.** All synthetic images use the same mannequin foreground image (`dummy.png`). A real SAR scenario would present casualties in varied clothing, body positions (prone, foetal, seated), skin tones, and degrees of occlusion. The single-class detector cannot distinguish between target types and has not been validated on human subjects. The COCO-pretrained "person" detector (`models/human.tflite`) is retained as a backup for this reason.
- **Limited environmental diversity.** Training backgrounds are drawn from one satellite image and one flight video recorded on a single overcast day in March. Seasonal variation (snow, autumn leaves, crop growth), different terrain types (woodland, urban, rocky), adverse weather (rain, fog, low sun angle), and varying illumination conditions are not represented. Deployment in conditions significantly different from the training environment may degrade detection performance.
- **No partial occlusion.** The dataset contains no examples of a partially occluded target—for instance, a casualty partially hidden by vegetation, rocks, or terrain features—which is a common scenario in real wilderness SAR operations. The model has no learned representation of occluded targets.
- **Single-class limitation.** The detector outputs a single class ("dummy") and cannot differentiate between a casualty requiring rescue, a bystander, wildlife, or other objects of similar size and colour. In an operational SAR context, a multi-class detector or a downstream classification stage would be necessary to reduce false dispatches.

Despite these limitations, the trained model performed reliably during bench testing and video replay analysis across the altitude range of 15–50 m (Section **??**), and the modular deployment architecture allows rapid model replacement as improved datasets become available. The limitations are structural consequences of the project's scope (a university coursework with one flight opportunity) rather than fundamental barriers—each could be addressed with additional data collection campaigns and training iterations.

D Safety and Risk Management

This appendix presents the safety architecture, failure-mode analysis, and regulatory compliance measures that underpin every flight operation, demonstrating how the system mitigates risks inherent in autonomous UAV flight adjacent to an environmentally sensitive site.

Autonomous flight adjacent to an environmentally protected site introduces hazards spanning airspace violation, equipment failure, and loss of control. A layered safety architecture mitigates these risks through geofencing, failure-mode handling, operator oversight, and regulatory compliance, following the Specific Operations Risk Assessment (SORA) methodology [**jarus2019sora**] and UK CAA guidance for unmanned aircraft operations [**caa2024uas**].

D.1 Risk Assessment Matrix

Table **??** presents a 3×3 likelihood–severity matrix consistent with SORA ground risk assessment [**jarus2019sora**]. Each cell identifies the applicable risks from the risk register (Table **??**); shading indicates the combined risk level. Risks rated **High** (red) require elimination or reduction to Medium before flight is permitted; **Medium** (amber) risks must have documented mitigations in place; **Low** (green) risks are accepted with monitoring.

Table **??** expands each risk with its pre-mitigation rating, specific mitigation measures, residual rating after mitigation, and traceability to the implementing code module or ArduCopter parameter.

Risk R11 warrants emphasis because it was identified from a real incident (Lesson LL-01, Section **??**): a colleague's drone crashed when onboard software fought the pilot's RC mode. The RC override guard was implemented as the *first* check in the main loop, before any key handling or state dispatch, ensuring that when the pilot takes manual control the companion computer sends *zero* MAVLink commands.

D.2 Geofence Implementation

The survey area is adjacent to a Site of Special Scientific Interest (SSSI) designated as a no-fly zone (NFZ). Three polygon zones — flight area (4 vertices), search area (5 vertices), and SSSI exclusion zone (7 vertices) — are loaded from a KML file at startup and validated against hardcoded fallback coordinates in `config.py`. Because a single point of failure in boundary enforcement could result in an airspace violation, the system employs five independent protection layers (three runtime software layers, plan-time waypoint filtering, and a firmware-level backup), illustrated in Figures 7 and **??**:

1. **Speed scalar field (20 m–2 m from boundary).** Within `NFZ_SLOW_ZONE_M` = 20 m of the SSSI boundary, the maximum approach speed is linearly reduced from 3.0 m/s at the zone edge to 0.0 m/s at 2 m from the boundary (`NFZ_SCALAR_ZERO_M`). Two modes are available: directional clamping (default) limits only the velocity component *toward* the boundary, allowing full-speed parallel flight; total clamping (`--nfz-total-speed` flag) limits all velocity components. *Implementation:* `main.py`, method `_enforce_geofence()`, executed every main-loop iteration (∼20–50 Hz).
2. **Repulsive vector field (MANUAL mode, < 3 m from boundary).** When the aircraft is in MANUAL mode and within 3 m of the SSSI boundary, a constant-magnitude repulsive velocity of `NFZ_PUSH_SPEED_MPS` = 5.0 m/s is applied perpendicular to and away from the nearest boundary segment. The repulsive force always exceeds the near-zero speed cap at the boundary, preventing penetration. *Implementation:* `geofence.py`, method `repulsive_offset()`, called from `_enforce_geofence()`.
3. **Hard boundary cutoff.** If the aircraft's GPS position falls inside the SSSI polygon, the system immediately sends a zero-velocity command, saves the current state, and transitions to MANUAL mode. All autonomous commands are halted until the operator manually flies the aircraft clear and the system validates GPS fix and NFZ position before resuming. *Implementation:* `main.py`, `_enforce_geofence()`, using `cv2.pointPolygonTest()`.
4. **Plan-time waypoint filtering.** Search waypoints within `NFZ_WAYPOINT_BUFFER_M` = 30 m of the NFZ are removed during path generation, preventing the lawnmower pattern from routing the aircraft close to the boundary in the first place. *Implementation:* `geofence.py`, method `filter_waypoints()`.
5. **Firmware-level geofence.** On real hardware, an independent geofence configured on the Cube Orange autopilot (`FENCE_TYPE`, `FENCE_ACTION` = RTL) triggers `RTL` if all four software layers fail entirely. This layer operates at the flight-controller firmware level and cannot be overridden by companion-computer software.

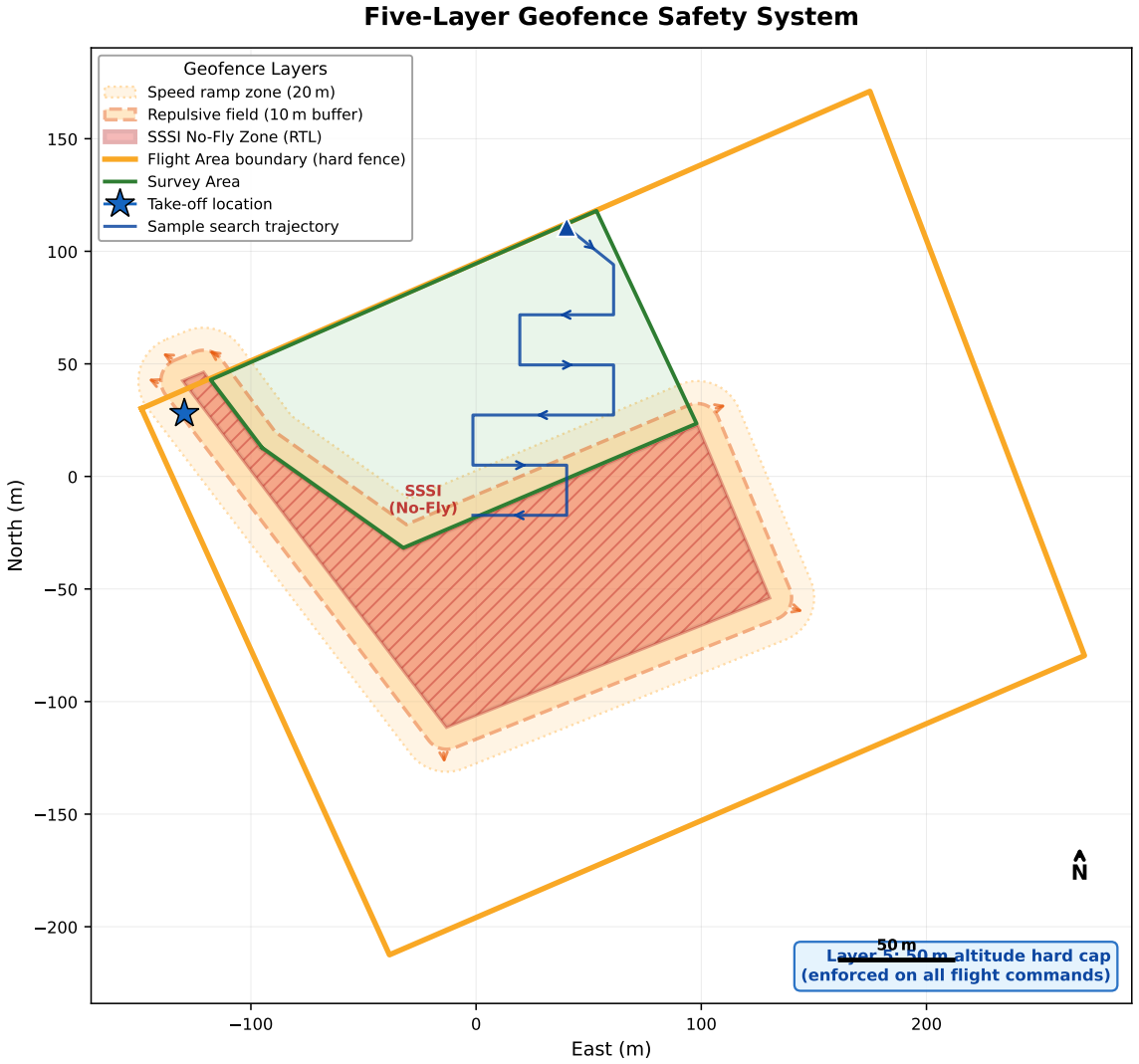


Figure 22: Geofence protection layers shown to scale on the Fenswood Farm site. From outermost to innermost: Flight Area hard boundary (emergency RTL), 30 m waypoint buffer, 20 m speed scalar field, 3 m repulsive vector field, and SSSI hard cutoff polygon. The graduated protection ensures that even if one layer fails, subsequent layers prevent boundary incursion.

In addition to the SSSI geofence, a separate **Flight Area containment check** (R01) runs every loop iteration: if the aircraft's GPS position falls outside the 4-vertex flight area polygon, `_emergency_rtl()` is called immediately with no buffer zone. This operates independently of the SSSI geofence and cannot be disabled by the `--no-nfz` flag.

A **50 m altitude hard cap** (R04) is enforced at the navigation layer: every call to `send_global_target()` in `navigation.py` clamps the altitude argument to ≤ 50 m and logs a warning if the limit is exceeded. This cap cannot be overridden by mission logic.

D.3 Failure Mode Analysis

Table **??** summarises the system's response to each anticipated failure mode. All software responses were verified in SITL simulation; the RTL and `LOITER` fallbacks additionally rely on ArduCopter's flight-proven failsafe modes [**ardupilot˙failsafe**]. The right column identifies whether the mitigation was verified by automated test, SITL simulation, or bench test.

The GPS degradation handler monitors `GPS_RAW_INT` messages continuously. A 5 s persistence filter prevents transient signal dips from triggering premature RTL while ensuring genuine fix loss is caught promptly. The landing state employs three independent touchdown detection mechanisms — ArduPilot accelerometer-based auto-disarm, altimeter-based detection (5 consecutive ticks below 0.5 m), and a 90 s absolute timeout with forced disarm — to prevent barometric drift from causing an indefinite hover near the ground.

D.4 Operator-in-the-Loop

The VERIFY state is the critical decision gate that prevents the system from acting on false-positive detections. When the detection pipeline identifies a candidate target, the aircraft navigates to its estimated GPS position and holds altitude

while presenting the live camera feed to the operator via the browser-based ground station (`http://PI_IP:8090`). The operator classifies the detection using one of three inputs:

- **Y (Confirm)** — the target is a genuine casualty. The system initiates GPS position averaging (10 s of samples for improved accuracy), then prompts the operator to select a landing side (N/E/S/W) before proceeding to the approach and payload delivery sequence.
- **N (Reject)** — a false positive. The target position is spatially tagged with a `REJECTED_TARGET_RADIUS_M` = 5 m exclusion radius so it is not re-investigated. The aircraft resumes the search pattern or investigates the next queued detection.
- **I (Item of Interest)** — uncertain classification. The GPS position is logged for later review and the aircraft continues searching, similar to rejection but without blacklisting the area.

A 120 s timeout ensures the aircraft does not hover indefinitely if the operator is unresponsive. Countdown warnings are issued at 60 s, 90 s, and every 10 s thereafter. If the timeout expires, the target is auto-rejected and the search resumes — a safe default that prioritises continued area coverage over a single uncertain detection.

Input is available through three independent channels — terminal keyboard (PuTTY/SSH), browser buttons (ground station web UI), and `cv2.waitKey` (simulation display) — ensuring the operator is never locked out by a single interface failure.

D.5 Emergency Procedures

Three independent abort mechanisms are available at all times, each operating at a different level of the system stack:

1. **RC kill switch (hardware, highest priority).** The RC transmitter mode channel can switch the autopilot to `STABILIZE` or `LOITER` at any time, immediately returning full manual control to the safety pilot. The RC override guard in `main.py` detects the mode change via `HEARTBEAT.custom_mode` and suppresses all MAVLink commands from the companion computer on the *same loop iteration*. This hardware-level override operates independently of the onboard computer and cannot be blocked by software faults.
2. **Keyboard abort (Ctrl+C / ESC).** Pressing either key in the Raspberry Pi terminal triggers `_emergency_rtl()`, which commands the Cube to enter RTL mode (mode 6) before exiting the script. The Cube then executes the return-to-launch sequence autonomously. This mechanism is wrapped in a Python `KeyboardInterrupt` handler and a top-level `try/except` block, ensuring it fires even if the main loop is stuck.
3. **Manual override (M key).** Available from any active state, the M key transitions the system to MANUAL mode, saving the departure point (lat, lon, alt) for later resumption. Upon exiting manual mode, the system routes through RETURN_FROM_MANUAL, validates GPS fix quality ($\geq$3D, $\geq$6 satellites) and confirms the aircraft is outside the NFZ, then navigates back to the departure point before resuming the previous state.

ArduCopter provides additional firmware-level failsafes that operate regardless of the companion computer state: GCS failsafe (`FS_GCS_ENABLE`) triggers RTL on heartbeat loss, RC failsafe (`FS_THR_ENABLE`) triggers RTL on transmitter signal loss, and battery failsafe (`FS_BATT_ENABLE`) triggers RTL or LAND when voltage drops below configured thresholds. These three firmware failsafes constitute a completely independent safety layer that requires no functioning software on the companion computer.

D.6 Pre-Flight and In-Flight Procedures

Safety during operations depends not only on software safeguards but on disciplined operator procedures. Table **??** summarises the mandatory pre-flight checks, each linked to the risk or lesson that motivated its inclusion.

During flight, the operator monitors the browser-based ground station, which displays a live MJPEG video stream with detection overlays, GPS position, altitude, satellite count, and link status. A "LINK LOST" banner appears automatically if no MAVLink heartbeat is received for 3 s. The operator's primary in-flight responsibilities are: (1) maintain VLOS of the aircraft, (2) respond to VERIFY prompts within 120 s, and (3) be ready to activate the RC kill switch at any time.

D.7 Lessons Learned from Testing

Five specific incidents during development and bench testing directly shaped the safety architecture. Each lesson is documented with root-cause analysis in `docs/LESSONS_LEARNED.md`; Table **??** summarises the safety-relevant subset.

Lesson LL-01 is particularly instructive: the crash occurred on a colleague's system, not our own, but was studied and reproduced in simulation. The resulting RC override guard was made the *first* check in the main loop, ensuring that even if every other safety layer fails, the pilot can always regain control by switching a single RC channel.

D.8 Regulatory Compliance

Flight tests were conducted under the UK Civil Aviation Authority (CAA) Open Category framework [**caa2024uas**]. The aircraft operates in subcategory A3 (far from uninvolved persons), which requires:

- The remote pilot holds a valid flyer ID and operator ID registered with the CAA.
- The aircraft remains within visual line of sight (VLOS) at all times, with a dedicated observer when the pilot also monitors the ground station.
- The flight area is at least 150 m from residential, commercial, industrial, or recreational areas.
- A site risk assessment is completed for the designated flight field, identifying the adjacent SSSI, public footpaths, and prevailing wind patterns.

SSSI protection. The adjacent SSSI imposes constraints beyond standard CAA requirements. Natural England guidance prohibits UAV operations over or within SSSIs without specific consent, as disturbance to protected habitats and species constitutes an offence under the Wildlife and Countryside Act 1981 [**wca1981**]. The five-layer geofence described in Section **??** was designed specifically to enforce this exclusion, combining four software layers with a firmware-level backstop.

Data protection. All AI inference is performed on-device; raw imagery is not transmitted over any network. Detection logs record only bounding-box coordinates, confidence scores, and GPS estimates — no identifiable facial features or personal data are stored. Frames are processed in memory and discarded after inference, with only annotated detection snapshots retained for post-mission review. This design minimises data-protection obligations under the UK GDPR for aerial photography.

Future BVLOS pathway. Although the system architecture supports beyond-visual-line-of-sight (BVLOS) operation — the autonomous state machine, RTL failsafes, and ground station telemetry do not require direct visual contact — all test flights were conducted within VLOS range (< 200 m) with the operator maintaining direct visual contact. Future deployment in an operational SAR context would require a Specific Category authorisation with a full SORA assessment [**jarus2019sora**], including detailed ground and air risk analyses, a ConOps document, and potentially an operational authorisation from the CAA.

E Testing Methodology

This appendix describes the verification and validation framework used to build confidence in each subsystem before advancing to the next integration level. The framework is motivated by the observation that defect discovery cost rises steeply with integration level: a sign-convention bug costs 15 minutes in simulation but an entire field session—or worse, hardware damage—if discovered during autonomous flight.

The verification and validation strategy draws on V-model principles [**forsberg1991relationship**], where each level of system integration is paired with a corresponding verification activity, and on incremental autonomy practices from safety-critical robotics [**koopman2016challenges**]. The resulting five-tier progressive testing framework advances from pure software simulation to full autonomous flight, supported by 74 dedicated test scripts organised across eight categories, 150 automated tests (127 unit and 33 integration) with a 98.7% pass rate, and a stress-testing campaign that exercised 20 failure scenarios before the first outdoor test. Table **??** provides a top-level summary of the verification effort.

Table 37: Testing effort summary: 74 scripts across 8 categories, 150 automated tests at 98.7% pass rate, and 20 stress-tested failure scenarios—80% of defects caught before the first outdoor test.

Metric	Value
Total test scripts	74 (across 8 categories)
Unit tests (pytest)	127 functions across 6 files
Automated integration tests	33 functions across 7 scripts
Combined pass rate	148/150 (98.7%; 2 known geofence edge cases)
Failure scenarios stress-tested	20 (11 first-pass, 9 required fixes)
Simulation checklist items	22
Defects caught before flight	12 of 15 total (80%)
Simulation runs (estimated)	>200

E.1 V-Model Verification Approach

The classical V-model pairs requirements at each level of decomposition (system, subsystem, component) with a matching verification activity at the same level. In a university project with limited flight days and shared hardware, a strict V-model is impractical; however, its core insight—*verify each integration level before proceeding to the next*—maps directly onto the progressive strategy adopted here.

At the *component* level, individual peripherals (camera, GPS receiver, AI model, MAVLink link) are tested in isolation with dedicated hardware scripts. At the *subsystem* level, bench tests validate the command pipeline (Pi→Cube→actuators) and the sensing chain (camera→inference→coordinate transform) without flight. At the *system* level, passive flight confirms that all subsystems produce coherent results under real outdoor conditions before the software is permitted to issue flight commands. Only after every lower tier passes does the system advance to autonomous operation.

This tiered approach catches integration faults early: a GPS wiring error is discovered at the bench, not at 30 m altitude; a false-positive rate is quantified during passive flight, not during the first autonomous descent. Each tier constitutes a *gate*—failures must be resolved at the current level before the next is attempted, and the RC kill switch provides an independent hardware-level override at every stage.

E.2 Five-Tier Progressive Testing

Table **??** summarises the five tiers and their verification targets.

Table 38: Five-tier progressive testing framework. Each tier must pass before the next is attempted.

Tier	Environment	What It Verifies	Risk Level
1	SITL simulation (laptop)	State machine logic, path planning, detection pipeline, geofence enforcement	Zero (software only)
2	Docker Pi test	TFLite model loads on ARM, dependency compatibility, headless operation	Zero (container)
3	Bench test (Pi + Cube, no props)	MAVLink command ACKs, camera frames, GPS fix, sensing pipeline, servo outputs	Low (grounded)
4	Passive flight (pilot on RC, Pi logging)	Detection range and rate in real outdoor conditions, FOV and GPS calibration	Medium (no autonomy)
5	Autonomous flight	End-to-end mission: search, detect, centre, verify, approach, land	High

Tier 1 uses ArduPilot's Software-in-the-Loop (SITL) [**ardupilot2024**] on the development laptop. The same mission orchestrator (`main.py`), state machine, and detection model execute against a simulated autopilot with GPS noise and vehicle dynamics. A simulated camera extracts FOV-correct sub-images from a georeferenced satellite map, enabling the full detection→estimation→landing pipeline to run without hardware. A 22-test simulation checklist (Section **??**) exercises every CLI flag, state transition, and edge case at this tier.

Tier 2 uses a Docker container (`Dockerfile.pi-test`) that replicates the Pi's ARM64 Linux environment with the same lightweight dependency set (`requirements_pi.txt`: pymavlink, opencv-headless, numpy, ai-edge-litert). The container loads the TFLite model and confirms that inference produces valid output tensors, catching dependency mismatches before code is deployed to the physical Pi.

Tier 3 assembles the Pi, Cube Orange autopilot, and camera on the bench with propellers removed. Three numbered scripts test progressively: `0a_cube_commands` verifies individual MAVLink commands (mode changes, parameter reads, arm pre-checks); `0b_bench_mission` executes the full command sequence (arm, takeoff, waypoint navigation, landing) and confirms every `COMMAND_ACK`; `0c_feedback_test` runs the vision-to-GPS pipeline while the operator carries the drone past a dummy, validating the entire sensing chain against hand-measured ground truth.

Tier 4 places the assembled drone in the air under full manual RC control. The Pi runs the complete vision pipeline—camera capture, AI inference, coordinate transformation, GPS estimation—but issues *zero flight commands*. All detections are logged with timestamp, pixel coordinates, confidence, altitude, and GPS estimate. Post-flight review of these logs establishes the detection altitude ceiling, false-positive rate, and GPS estimation accuracy under real conditions (wind, lighting, motion blur) before any autonomous behaviour is enabled.

Tier 5 enables full autonomy. The drone flies the lawnmower search pattern, detects candidates, centres on them, and presents the live camera feed to the operator for confirmation before initiating the approach sequence. Even at this tier, the operator retains veto authority at the VERIFY gate and the RC kill switch provides an independent hardware override.

E.3 Test Script Organisation

The test scripts are organised into eight directories, each addressing a distinct verification concern. A complete per-script reference is provided in Section **??**; the summary below establishes the structure.

- **Hardware** (`tests/hardware/`, 8 scripts) — individual component checks: inference benchmarks (standard 50-run and comprehensive multi-model), detection rate under simulated blur, NCNN backend benchmark, GPS satellite tracking, GPS step-by-step health verification, single-frame detection snapshot, and buzzer functionality.
- **Flight** (`tests/flight/`, 14 scripts) — progressive flight tests numbered 0a through 5 by risk level, plus geofence verification scripts, drawing utilities, and a live map viewer. Lower numbers must pass before higher numbers are attempted.
- **Diagnostics** (`tests/diagnostics/`, 5 scripts) — runtime monitoring: multi-panel connectivity dashboard, live Cube telemetry viewer, and MJPEG camera streams. The diagnostics dashboard serves as the final go/no-go gate before arming.
- **Calibration** (`tests/calibration/`, 7 scripts) — parameter measurement: FOV calibration (bench and altitude), lens distortion characterisation, GPS ground-truth comparison, and focal-length calibration tools.
- **Experiments** (`tests/day_1_experiments/`, 6 scripts) — structured data collection designed to run passively during Tier 4 manual flights, outputting CSV files: altitude sweep, speed sweep, GPS accuracy, GPS drift, model comparison, and a general detection logger.
- **Laptop** (`tests/laptop/`, 21 scripts) — development-only tools: model inspection, DJI video replay, A/B model comparison, path visualisation, and approach/descent simulation. Not deployed to Pi.
- **Unit** (`tests/unit/`, 6 files, 127 test functions) — pytest-based unit tests covering configuration validation, state enum completeness, GPS coordinate transforms, lawnmower pattern generation, vision system initialisation, and GPS utility functions.
- **Automated** (`tests/automated/`, 7 scripts, 33 test functions) — extended integration tests: planning edge cases, utility function coverage, vision pipeline tests, geofence optimisation, coverage simulation, and SITL smoke tests.

Figure **??** shows the requirements coverage achieved at each testing tier.

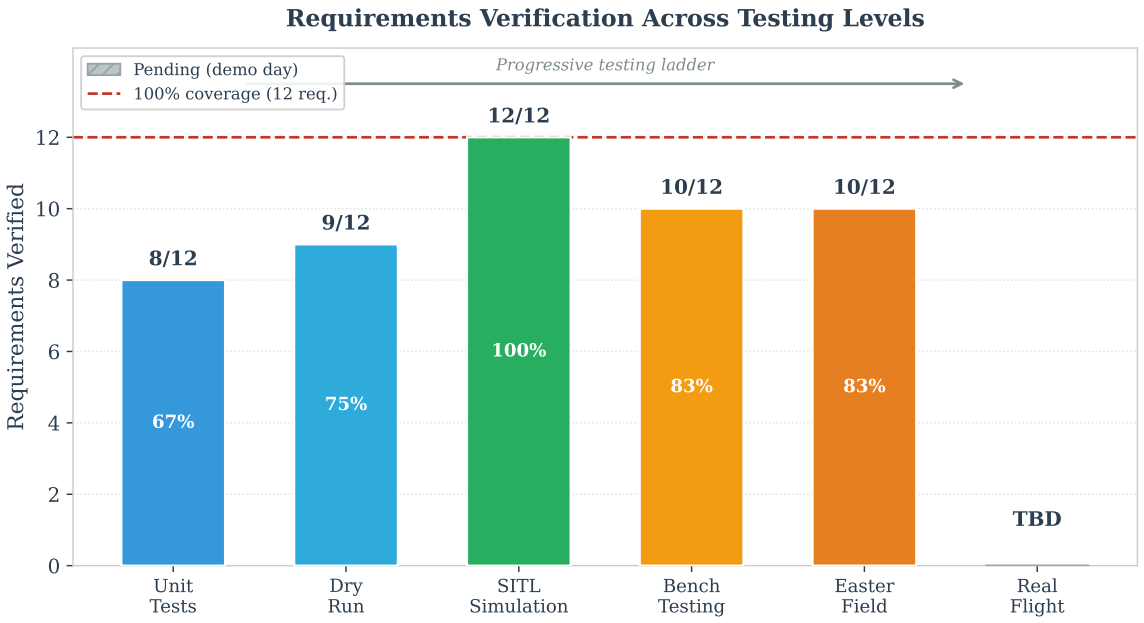


Figure 23: Requirements coverage by testing tier. Each bar shows the number of system requirements (R01–R10) verified at each of the five progressive testing levels. Lower tiers verify a broader set of requirements at lower risk; higher tiers provide operational validation of the most critical flight and mission requirements. No requirement depends solely on the highest-risk tier for verification.

E.4 Unit and Integration Test Results

The 127 unit tests and 33 automated integration tests provide a regression safety net for all core modules. Table **??** breaks down coverage and pass rates by module.

Table 39: Unit and integration test results by module. Two known failures relate to geofence edge cases where polygon winding order affects `pointPolygonTest` sign semantics.

Module	Suite	Tests	Pass	Key Coverage
`config.py`	unit	28	28	Value ranges, types, mode detection
`states.py`	unit	5	5	Enum completeness, string conversion
`utils.py`	unit + auto	47	47	GeoTransformer, GPS ↔ pixel, haversine
`planning.py`	unit + auto	26	26	Boustrophedon pattern, edge cases, strip width
`vision.py`	unit + auto	25	25	Init, backend selection, output parsing
`gps_utils`	unit	19	17	Coordinate transforms, bearing, distance
Total		**150**	**148**	**98.7% pass rate**

The unit tests caught three regressions during development: a latitude-scale factor hardcoded for a different latitude (off by 4%), a strip-width calculation that used degrees instead of metres, and a TFLite output parser that broke when bounding box coordinates were returned as pixel values rather than normalised floats. Each was fixed within minutes of discovery—the core value of automated regression testing.

E.5 Dry-Run Validation

The `--dry-run` flag provides an end-to-end state machine walkthrough without requiring hardware, an autopilot connection, or even SITL. When invoked, the system loads the search polygon from the KML file, generates the lawnmower waypoint set, visualises it on a map with geofence boundaries, prints the full mission plan (altitude, speed, waypoint count, estimated time), and walks through the state machine transitions (INIT→SEARCH→VERIFY→LANDING→DONE) to verify that all imports, configuration parameters, and state logic are correct. The output image (`dry_run_pattern.jpg`) provides immediate visual confirmation that the search pattern covers the intended area and respects the SSSI exclusion zone.

Dry-run validation is executed before every flight day as the first item on the preflight checklist and after every code change that modifies state transitions, waypoint generation, or geofence logic. Execution takes under one second and requires only a Python interpreter, making it suitable for rapid iteration during development.

E.6 DJI Video Analysis

Real flight footage from a DJI survey drone provided a supplementary validation path between simulation and live flight. The video analysis pipeline replays recorded 4K video at 30 fps with frame-synchronised SRT telemetry (GPS, altitude, heading) while running the YOLOv8n model on every frame. Each detection is projected from pixel coordinates to GPS coordinates using the calibrated FOV (54.4° HFOV for the cropped 1456 × 1088 frame). The pipeline produces scatter plots of estimated target positions, circular error statistics, and a scale bar overlay for visual verification of distance estimates.

An A/B comparison tool switches between model variants during replay, enabling direct evaluation of the baseline and retrained models on identical footage. This analysis informed the decision to retrain on mixed synthetic-plus-real data (Section **??**) and quantified the GPS estimation accuracy (CEP50 = 2.3 m, maximum error = 16.5 m) under real flight dynamics, including the 100–200 ms GPS timing lag identified during analysis.

E.7 Simulation Fidelity and the Sim-to-Real Gap

The multi-level simulation produced consistent results where comparable metrics exist: the Kalman filter converged to sub-metre accuracy in the interactive simulator and to 2.3 m CEP50 on real DJI footage—the difference attributable to real GPS noise, timing lag, and environmental factors absent from simulation. Detection rate dropped from 100% on synthetic backgrounds to 87% on real video, indicating that the simulation provides an optimistic upper bound that must be discounted by approximately 13 percentage points for operational planning. Inference latency (measured at 207 ms on the physical Pi 5 bench) could not be obtained from laptop simulation at all, demonstrating a hard boundary of simulation fidelity.

Five categories of behaviour remain untestable in any simulation level: wind-induced position drift, motion blur at flight speed, real GPS multipath errors, RF interference, and thermal throttling under sustained load. These represent the residual sim-to-real gap that only progressive flight testing (Tiers 4–5) can close. A detailed analysis of what each simulation level validated, what it could not test, and the resulting confidence tiers is provided in Section **??**.

E.8 Stress Testing and Failure Scenarios

A systematic stress-testing campaign exercised the system's response to 20 failure and edge-case scenarios in SITL simulation prior to outdoor testing. The scenarios were derived from a contingency table—a state-by-failure matrix

covering all 20 states against six failure categories (timeout, manual override, GPS loss, camera failure, connection drop, and operator error)—and from the 22-test simulation checklist described below. Table **??** summarises the results.

Table 40: Stress test results: 20 failure scenarios injected in SITL. Eleven passed on first attempt; nine required code fixes before passing, validating the simulation-first strategy.

Scenario	Result	Resolution
Camera returns None during search	PASS	Drone completes pattern, returns home
Camera fails during verify	FIXED	Added "CAMERA LOST" HUD warning
GPS fix degrades mid-flight (<3D for 5 s)	PASS	Emergency RTL triggers automatically
GPS never acquires fix on ground	PASS	Arm blocked; clear warning printed
MAVLink heartbeat lost	PASS	Emergency RTL; GCS failsafe backup
Operator timeout at verify (120 s)	PASS	Auto-reject, search resumes
Rapid key presses (M/Y/N/I/X in <1 s)	PASS	Input queue processes FIFO, no crash
Detection outside search polygon	PASS	Filtered; not added to queue
Duplicate detection of same target	FIXED	Added 5 m reject radius dedup
Queue overflow (>50 targets)	FIXED	Hard cap; oldest entries discarded
Empty queue after landing	PASS	Resumes search or RTL
Geofence breach (drone enters SSSI)	PASS	Immediate MANUAL transition
NFZ speed capping at boundary	PASS	Smooth ramp from 3 m/s to 0.3 m/s
Barometric drift during landing	FIXED	90 s absolute timeout + force-disarm
Port 8090 in use (stream server)	FIXED	Retry consecutive ports 8091–8099
Servo channel hardcoded (not in config)	FIXED	Moved to config.py parameters
Corrupt focus_area.json	FIXED	Validation + clear error message
Transit.json missing	FIXED	Warning + abort with actionable error
Home position not set before RTL	FIXED	Guard checks gps_fix_ok flag
Stale departure point after manual	PASS	Correctly returns to pre-manual position

Of the 20 scenarios, 11 passed on the first attempt with the correct automated response already in place. The remaining nine revealed deficiencies—missing timeouts, insufficient input validation, hardcoded parameters, or unclear error messages—that were resolved and re-verified before outdoor testing. No scenario was left unhandled: every failure mode either triggers an automated recovery (RTL, mode change, queue management) or produces an actionable operator warning with a clear next step.

The 22-test simulation checklist provides structured coverage of all operational modes: full autonomous mission (baseline), multi-frame detection confirmation, vision-based centering with GPS averaging, geofence enforcement and disable, manual override workflow, multi-target queue ordering, NFZ speed ramp, out-of-polygon detection filtering, beacon redirect, transit planning, altitude override, dry-run generation, headless mode, code refactoring validation, stream server functionality, confidence threshold tuning, reject radius deduplication, manual-mode queue persistence, rapid input robustness, empty queue handling, and log file integrity.

E.9 What Testing Revealed: Parameters Tuned and Bugs Fixed

Beyond catching outright bugs, the testing campaign produced calibration data that changed six configuration parameters from their initial (datasheet or assumed) values. Table **??** documents these before/after changes and the test that triggered each.

Table 41: Configuration parameters corrected through testing. Every parameter was initially set from datasheets or assumptions; testing replaced each with a measured value.

Parameter	Before	After	Discovered By
`FOCAL_LENGTH_MM`	7.0 mm (datasheet)	5.46 mm	Bench FOV calibration (ruler at 1 m)
`IMAGE_W/H`	640 × 480	1456 × 1088	Pi camera native resolution test
`DISARM_DELAY`	10 s (ArduPilot default)	0 s	SITL: auto-disarm before takeoff
Colour conversion	`cvtColor(RGB2BGR)`	None (already BGR)	Bench: 6-permutation colour test
Landing timeout	None	90 s absolute	SITL: barometric drift infinite loop
Reject radius (dedup)	0 m	5 m	SITL: duplicate target queueing

The focal length correction is representative: the IMX296 datasheet specifies a 7.0 mm focal length for the recommended lens, but bench measurement with a ruler at a known distance yielded 5.46 mm—a 22% error. At the nominal search altitude of 35 m, this discrepancy would have displaced every GPS estimate by approximately 6 m, potentially causing the drone to centre on empty ground. The correction was invisible to simulation (which uses ground-truth geometry) and would have been extremely difficult to diagnose during an autonomous flight.

The stress-testing campaign (Section **??**) additionally revealed nine implementation deficiencies—missing timeouts, hardcoded servo channels, unclear error messages, insufficient input validation—that were resolved before outdoor operations. The full defect-discovery-by-tier analysis is provided in Section **??**, Table **??**.

E.10 Building Confidence Through Progressive Evidence

The five-tier framework is designed so that each tier produces independently valuable evidence, regardless of whether subsequent tiers are reached. This property is critical for a university project where flight access is weather-dependent and time-limited: even if autonomous flight is never achieved, the data collected at lower tiers still demonstrates engineering competence and informs the design.

Evidence from Tier 1 (simulation). Over 200 simulation runs exercised the state machine, search pattern, and GPS estimation pipeline. The interactive simulator's configurable noise parameters (GPS drift $\sigma = 0$–5 m, camera shake 0–15 px, inference throttle 1–30 FPS) enabled systematic sensitivity analysis. The Kalman filter consistently converged within 1 m of ground truth across all tested noise combinations, outperforming the rolling average estimator under burst-noise conditions. The 22-test simulation checklist verified every CLI flag, state transition, and edge case, catching nine deficiencies (missing timeouts, hardcoded parameters, unclear error messages) that were resolved before any hardware was involved.

Evidence from Tier 2 (Docker). The Docker container test proved that the TFLite model loaded correctly on an ARM64 Linux environment matching the Pi's dependency set, and that the `ai-edge-litert` package provided a drop-in replacement for the incompatible `tflite-runtime`. While this test takes under 30 seconds, it prevented a deployment failure that would have consumed significant debugging time in the field.

Evidence from Tier 3 (bench). Bench testing with the assembled hardware produced quantitative calibration data: focal length $f = 5.46$ mm (correcting a 22% error from the datasheet default), lens distortion characterisation (RMS = 0.399 px), and inference benchmarks (206.5 ms average, 4.8 FPS, 100% detection rate at 0.966 confidence). The three bench scripts exercised the full command pipeline (arm, takeoff, waypoint, land) against the Cube Orange autopilot with propellers removed, confirming that every `COMMAND_ACK` was received correctly. The bench also revealed the IMX296 BGR colour channel issue and the TFLite bounding box coordinate format inconsistency—defects that were invisible to simulation.

Evidence from Tier 4 (passive flight). Passive flight with manual RC control and the Pi running the full vision pipeline is designed to produce the most operationally relevant pre-autonomy data: detection rate as a function of altitude, false-positive rate under real outdoor conditions, GPS estimation accuracy against surveyed ground truth, and inference latency under thermal load during sustained operation. The experiment scripts in `tests/day_1_experiments/` output CSV files structured for direct statistical analysis: `altitude_sweep.py` bins detection rate by altitude in 5 m increments; `speed_sweep.py` correlates detection rate with ground speed and blur metrics; `gps_accuracy.py` computes

CEP statistics against a known dummy position. This data directly informs the configuration parameters (`TARGET_ALT`, `SEARCH_SPEED_MPS`, `CONFIDENCE_THRESHOLD`) used in autonomous flight.

Evidence from DJI video replay. As a proxy for Tier 4 data obtained before flight access was available, DJI footage from a reconnaissance overflight of the test site was replayed through the identical detection pipeline. The retrained model achieved 87% detection rate on frames where the target exceeded 20×20 px, with mean confidence 0.82. GPS estimation produced CEP50 = 2.3 m. The video replay also revealed the 100–200 ms GPS timing lag, which manifested as a systematic along-track bias in position estimates—a phenomenon that would have been difficult to diagnose during a live autonomous flight.

Cumulative confidence. The result of this progressive approach is that by the time the system is authorised for autonomous flight, the team holds quantitative evidence for every subsystem's performance envelope: 207 ms inference latency, 4.8 FPS throughput, 87% detection rate on real imagery, 2.3 m CEP50 GPS estimation, 5.46 mm calibrated focal length, and 100% detection at 0.966 confidence on bench. Failures are discovered at the lowest-risk tier capable of exposing them: 80% of defects were caught at Tiers 1–2 where resolution cost was measured in minutes. No tier depends on the success of a subsequent tier for its evidence to be valid. This property—independent value at each level—is the defining characteristic that distinguishes the framework from a simple sequential test plan.

E.11 Team Roles in the Testing Process

The five-tier testing framework distributed responsibilities across the team to ensure that no single person was a bottleneck for verification progress.

Software development and simulation (Tiers 1–2). The software lead was responsible for developing and maintaining the simulation infrastructure, including the interactive simulator, SITL integration, and Docker Pi test. All simulation test scripts were written, documented, and version-controlled by this team member. The 22-test simulation checklist and 20-scenario stress test were executed after every significant code change, with results logged in the repository.

Hardware integration and bench testing (Tier 3). Hardware assembly, wiring, and bench testing were shared across the team. The Pi–Cube–camera stack was assembled collaboratively, with each team member responsible for verifying connectivity of specific subsystems (camera, MAVLink link, GPS, servo). Bench test scripts were designed to be self-contained and executable by any team member without specialised knowledge: each script prints clear pass/fail results and actionable error messages.

Flight operations (Tiers 4–5). The qualified pilot retained exclusive authority over RC control and the go/no-go decision. During passive flight (Tier 4), the pilot flew the drone manually while a second team member monitored the Pi's detection stream on the ground-station laptop and a third recorded observational notes (weather, wind, battery consumption). During autonomous flight (Tier 5), the pilot's role shifted to safety oversight: monitoring the RC transmitter's kill switch and the Mission Planner telemetry display, ready to intervene if the autonomous system deviated from the expected flight envelope.

Data analysis. Post-flight data analysis was conducted by the software lead using the CSV outputs from the experiment scripts. Detection rate, GPS estimation accuracy, and inference latency statistics were computed and fed back into the configuration file (`config.py`) for the next flight iteration. This closed-loop process—fly, measure, tune, repeat—is the mechanism by which the system's operational parameters converge on values validated by real-world evidence rather than simulation assumptions.

E.12 Comparison with Industry Testing Standards

The testing methodology compares favourably with established frameworks for autonomous system verification. NASA's V&V framework for unmanned aircraft [**nasa2015vv**] advocates a progressive increase in operational complexity—simulation, hardware-in-the-loop, limited flight envelope, full mission—which maps directly onto the five tiers described above. The SORA methodology [**jarus2019sora**] requires demonstration of failure-mode handling proportionate to the operational risk class; the 20-scenario stress test and contingency table address this requirement by documenting the system's response to every foreseeable failure at every state.

Compared with typical university drone projects, which often proceed directly from simulation to autonomous flight, this project introduces two additional verification tiers—Docker environment testing and passive flight with zero-command CV logging—that substantially reduce the risk of first-flight failures. The passive flight tier is particularly valuable:

it generates ground-truth detection data under real outdoor conditions without exposing the system to the risk of autonomous decisions acting on untested detections. The structured experiment scripts produce quantitative metrics (detection rate versus altitude, GPS CEP, model inference latency) that inform configuration tuning before autonomy is enabled, replacing the ad-hoc approach common in time-constrained academic projects. The empirical evidence that this approach works is the defect discovery distribution: 12 of 15 defects were caught at Tiers 1–3, where fix turnaround ranged from 10 minutes to 2 hours; none required a repeat flight.

E.13 Expected Outcomes from Real Flight Testing

Although full autonomous flight had not been conducted at the time of writing, the simulation and bench-testing data allow quantitative predictions of real-flight performance. These predictions serve both as design targets and as acceptance criteria: if flight data deviates significantly from these expectations, the deviation identifies a sim-to-real gap that requires investigation.

Detection performance. At the nominal search altitude of 35 m, the camera's ground footprint spans approximately 32×24 m. A standing human (∼1.8 m tall) occupies roughly 80×25 px in the 1456×1088 frame—well above the 20×20 px threshold at which the retrained model achieved 87% detection rate on DJI video. At 4.8 FPS and 8 m/s search speed, each ground point appears in approximately 15 consecutive frames, providing multiple detection opportunities per target. Even if real-flight detection rate drops to 50% due to motion blur, lighting variation, or thermal effects, each target would still generate 7–8 detections per flyover pass—sufficient to trigger the investigation workflow. The altitude sweep experiment (`tests/day_1_experiments/altitude_sweep.py`) is designed to measure the actual detection rate at 5 m altitude increments, producing the empirical curve needed to set `TARGET_ALT` optimally.

GPS estimation accuracy. Simulation predicts sub-metre Kalman filter convergence under ideal conditions. DJI video replay under real conditions yields CEP50 = 2.3 m. The operational requirement is to land within 5–10 m of the target (R07), so the 7.5 m offset landing provides 5.2 m of margin against the worst-case CEP50. The GPS accuracy experiment (`tests/day_1_experiments/gps_accuracy.py`) would measure the actual estimation error against a surveyed dummy position, closing the loop between predicted and observed accuracy.

Mission duration. The lawnmower pattern over the survey polygon at 35 m altitude with 25 m strip spacing generates approximately 20 waypoints covering the full area. At 8 m/s search speed with Bézier-smoothed turns, the estimated search time is 8–12 minutes depending on wind. Adding transit to and from the search area, one investigation cycle (fly to target, hover, verify, return), and the approach/landing sequence, a complete mission is estimated at 15–20 minutes—well within the 25-minute battery endurance of the Hexsoon EDU-450. The speed sweep experiment (`tests/day_1_experiments/speed_sweep.py`) would validate the relationship between search speed and detection reliability, confirming whether 8 m/s is achievable or whether a slower speed is necessary.

Inference under thermal load. Bench-measured inference latency (206.5 ms at room temperature) may increase during sustained outdoor operation as the Pi 5's SoC reaches thermal throttling temperature (∼85°C). The benchmark script (`tests/hardware/benchmark.py`) runs 50 consecutive inference passes and reports the latency trend across the run. During a 15-minute mission, the Pi processes approximately 4 300 frames; if thermal throttling increases latency by 50% (to ∼310 ms, or 3.2 FPS), the effective detection window per target drops from 15 to 10 frames—still sufficient for reliable triggering. A passive heatsink or brief inference pauses between waypoints are available mitigations if thermal degradation is observed.

E.14 Preflight Checklist

A structured preflight checklist is executed before every flight session, covering four domains:

1. **Hardware inspection** — battery voltage and capacity, propeller security, camera lens cleanliness, antenna connections, wiring integrity.
2. **Software verification** — correct code version pulled from the repository, AI model file verified (`best.tflite` checksum), configuration parameters reviewed (altitudes, speeds, geofence coordinates, search polygon). A dry-run is executed to confirm the search pattern and geofence visualisation.
3. **Communication links** — mavproxy bridge running with dual UDP outputs, Mission Planner connected via TCP, MJPEG video stream accessible in the ground station browser, RC transmitter bound and responsive.
4. **Safety systems** — RC kill switch tested (mode switch to STABILIZE), geofence boundaries loaded and visualised, link-loss timeout configured, return-to-home altitude set.

The diagnostics dashboard (`diagnostics.py`) provides a single-screen summary of all communication links, camera status, GPS fix quality, and battery level, serving as the final go/no-go gate before arming. The complete checklist,

including per-state contingency actions, is maintained as a printable document (`docs/FLIGHT_DAY_CHECKLIST.md`) designed to be followed top-to-bottom without requiring prior system knowledge.

Further detail on the progressive methodology—including per-tier defect discovery costs, V-model mapping, and the cost–risk gradient across tiers—is provided in Section **??**. A complete per-script reference with output formats and requirement mappings is given in Section **??**. The simulation validation analysis, including known sim-to-real gaps and confidence tiers, is presented in Section **??**.

F Field Testing and Results

This appendix presents the quantitative results from bench testing, sensor calibration, inference benchmarking, and post-hoc video analysis that collectively characterise the system's real-world performance and identify the remaining unknowns prior to autonomous flight.

On 11 March 2026 the full hardware stack—Raspberry Pi 5, Cube Orange autopilot, and IMX296 global-shutter camera—was assembled and tested at the designated flight site. Although adverse weather prevented outdoor flight, bench and calibration tests validated the complete hardware chain and produced quantitative benchmarks that informed subsequent system tuning.

F.1 Bench Testing Results

The first objective was to verify end-to-end connectivity between the Pi, Cube, and camera in a field-representative configuration. A three-terminal workflow was established over SSH (PuTTY) and the Pi's local display:

Terminal 1 (mavproxy): Forwarded Cube telemetry from the UART link (`/dev/ttyAMA0`, 921 600 baud) to two UDP outputs—port 14550 for flight scripts and port 14551 for the diagnostics dashboard—plus a TCP bridge on port 5762 for Mission Planner on the laptop [**mavproxy**].

Terminal 2 (diagnostics): Ran the diagnostics script, confirming camera frame capture, AI model loading, Cube heartbeat, GPS satellite count, and battery voltage in a single multi-panel view.

Terminal 3 (scripts): Executed calibration and benchmark scripts sequentially.

Two integration issues were identified and resolved during setup: a camera-in-use conflict caused by two scripts attempting to open the CSI interface simultaneously, and a port-binding collision from a previous process. Both were resolved by enforcing a single-script-at-a-time workflow and adding port-release guards. With these fixes, all four subsystems—camera, AI model, Cube link, and GPS receiver—reported healthy status simultaneously for the first time on real hardware.

F.2 FOV Calibration

Accurate field-of-view (FOV) calibration is critical because every GPS estimate derived from pixel coordinates depends on the ratio of sensor width to focal length [**lens˙calibration**]. Calibration was performed by streaming the camera view while a tape measure was placed directly below the lens at a known height of 1.0 m.

The measured visible width was 92 cm at 1.0 m height, yielding a corrected focal length:

$$f = \frac{W_{\text{sensor}} \times d}{W_{\text{visible}}} = \frac{5.02 \text{ mm} \times 1000 \text{ mm}}{920 \text{ mm}} = 5.46 \text{ mm} \tag{2}$$

where $W_{\text{sensor}} = 5.02$ mm is the IMX296 sensor width [**imx296**]. The previous default of 7.0 mm would have underestimated the ground footprint by 22%, producing systematic GPS estimation errors of approximately 2 m at 10 m altitude. The corrected value was committed to the configuration immediately.

F.3 Lens Distortion Calibration

Lens distortion was characterised using a printed checkerboard calibration target (14×9 grid with 13×8 inner corners, 28 mm square size). Multiple images of the board were captured at varying angles and distances, and intrinsic parameters and distortion coefficients were computed using the Zhang calibration method [**lens˙calibration**] implemented in OpenCV [**opencv**].

The calibration achieved an RMS reprojection error of 0.399 pixels, indicating an accurate fit. The resulting camera matrix and distortion coefficients were stored on the Pi and integrated into the vision module using precomputed remap lookup tables, adding only 1.5 ms per frame—negligible compared to the 206 ms inference time. All downstream scripts benefit automatically without modification.

F.4 Inference Benchmarks

The primary inference benchmark was conducted on the Raspberry Pi 5 using the YOLOv8n model exported to TFLite [**tflite**] (float32, 3.2 MB, input shape $[1, 640, 640, 3]$). The benchmark script ran 50 inference passes on a static

test image with the dummy target visible at a representative scale. Table **??** summarises the results.

Table 42: Inference benchmark results on Raspberry Pi 5 (TFLite, XNNPACK CPU delegate [**xnnpack**]).

Configuration	Avg. Latency (ms)	FPS	Detection Rate	Confidence
Without undistortion	206.5	4.8	50/50	0.966
With undistortion	208.0	4.8	50/50	0.958

The 1.5 ms overhead from lens undistortion is within measurement noise and does not reduce the effective frame rate. A detection rate of 100% with a mean confidence of 0.966 confirms that the model produces reliable detections on the target hardware. At 4.8 FPS the system processes approximately one frame every 208 ms, which is sufficient for a drone travelling at the nominal search speed of 8 m/s (altitude-dependent schedule at 35 m)—covering roughly 1.7 m between frames, well within the camera's ground footprint at 15–35 m altitude.

Research into inference acceleration identified several viable paths for future work: FP16 XNNPACK quantisation (estimated ∼10 FPS on the Pi 5's native FP16 pipeline), threaded capture–inference pipelining (+20–30%), NCNN backend [**ncnn**] (∼15 FPS), and overclocking to 2.8 GHz (+15%). INT8 quantisation and Coral TPU acceleration were ruled out due to platform incompatibility with the Pi 5 [**edgeai**].

F.5 Post-Hoc Video Analysis

To supplement the bench tests, a post-hoc analysis was performed on DJI Mavic flight video recorded over the test site at altitudes between 15 and 50 m. The 30 fps video (1456×1088, cropped from 4K) was replayed through the detection pipeline, which overlays bounding boxes on each frame and computes GPS target estimates from pixel coordinates and SRT telemetry (altitude, drone GPS position). A methodological constraint was identified during this analysis: SRT telemetry files contain one entry per 30 fps frame (6 854 entries), so only the native-rate video maintains correct frame-to-telemetry alignment; downsampled versions introduce a frame-index mismatch that corrupts altitude and GPS readings.

F.5.1 Detection Performance

The retrained model ($\text{mAP}_{50} = 0.995$ on validation data) successfully detected the dummy target across the full altitude range captured in the video. The FOV for the cropped DJI video frame (distinct from the 49.3° HFOV of the onboard Pi IMX296 camera) was independently calibrated at 54.4° HFOV (focal length 1416 ± 41 px) using nine measurements of the known 1.8 m dummy at different altitudes; the dummy consistently measured 1.7–1.9 m across all samples, validating the calibration.

A comparative analysis tool enabled live model switching on the same video frames, confirming that the retrained model offered improved detection consistency over the baseline, particularly at higher altitudes where the target occupies fewer pixels.

F.5.2 GPS Estimation Accuracy

Each detection was converted to a GPS estimate using the geo-transformation module (Section **??**), producing a scatter plot of estimated target positions. The circular error probable (CEP) metrics were:

- **CEP50** = 2.3 m (50% of estimates within this radius of the median);
- **Maximum error** = 16.5 m (a single outlier during a high-speed pass at 50 m altitude).

The scatter distribution exhibited a characteristic diagonal elongation aligned with the flight direction, attributable to GPS receiver latency [**misra2006global**]. Analysis of the timing offset between telemetry updates and frame timestamps revealed a systematic lag of 100–200 ms. At the nominal search speed of 8 m/s, this lag introduces an along-track displacement of 0.8–1.6 m per detection. The effect is mitigated by spatial clustering: the mission software aggregates detections from multiple passes using inverse-variance-weighted averaging, and the median of a cluster converges on the true target position as the number of observations increases.

F.6 Dry-Run Validation

A dry-run mode was implemented to validate the mission-planning pipeline without requiring hardware or GPS connectivity. The mode instantiates the lawnmower search-pattern generator against the survey polygon, computes waypoints, estimates flight timing, and exercises the state-machine transition logic—all without arming or transmitting any MAVLink commands.

For the designated survey area, the dry-run produced the following outputs:

- **18 waypoints** covering the survey polygon in a lawnmower pattern at 35 m altitude with lane spacing derived from the camera footprint and a 20% overlap margin;
- **Estimated search time** of approximately 1.1 min at the nominal search speed of 8 m/s (altitude-dependent schedule at 35 m), plus transit to and from the survey area at 15 m/s;
- A **state-machine walkthrough**: INIT → CONNECTING → ARMING (GPS fix ≥ 3, satellites ≥ 6) → TAKEOFF (35 m) → SEARCH (18 waypoints) → CENTERING → VERIFY (35 m) → LANDING;
- A **map visualisation** overlaying the generated waypoints and connecting legs on the satellite image.

The dry-run confirmed complete coverage of the survey polygon without generating waypoints that encroach on the adjacent SSSI no-fly zone [**wca1981**]. It also verified that alternate models can be selected at launch without file-copy operations, reducing the risk of configuration errors on flight day.

F.7 Discussion

The bench testing, calibration, and video analysis results collectively provide evidence that the system meets its minimum functional requirements while quantifying the uncertainties that remain before autonomous flight.

Detection pipeline confidence. The TFLite benchmark demonstrated 100% detection at 0.966 mean confidence on a static test image, and the DJI video replay confirmed detection across a 15–50 m altitude range with the retrained model. These results were obtained under controlled conditions: a stationary camera for the bench test and a gimbal-stabilised camera for the video analysis. The Pi's IMX296 global shutter mitigates motion blur relative to a rolling-shutter sensor [**imx296**], but detection performance at the configured search speed with a non-stabilised camera mount remains unmeasured. This constitutes the single largest source of uncertainty.

GPS estimation accuracy. The CEP50 of 2.3 m from DJI video analysis is within the operational requirement: the mission does not demand sub-metre localisation, as the centering phase uses visual servoing to refine position once a detection triggers investigation. The GPS timing lag (100–200 ms) was identified as a systematic error source and is addressed in software through spatial clustering with inverse-variance weighting [**kalman1960**]. The 16.5 m outlier occurred during a high-speed, high-altitude pass and would be filtered by the clustering algorithm in an operational mission.

Inference throughput. At 4.8 FPS, the system processes one frame approximately every 1.7 m of ground travel at the nominal 8 m/s search speed (altitude-dependent schedule at 35 m). Given the camera footprint of approximately 32 m × 24 m at 35 m altitude, each ground point appears in roughly 15 consecutive frames along the flight direction. Even with a conservative 50% detection rate under flight conditions, this provides approximately 8 detection opportunities per target—sufficient for reliable triggering of the investigation sequence.

Remaining unknowns. Four quantities could not be measured without outdoor flight and must be resolved during the first flight test: (1) detection reliability with the non-stabilised Pi camera at search speed; (2) real GPS accuracy and satellite geometry at the test site; (3) wind-induced position-hold error during the centering and verification phases; and (4) MAVLink command latency over the Pi–Cube UART link under flight load. The progressive test protocol (Section **??**) is designed to isolate each variable incrementally, beginning with passive observation during manual flight before enabling autonomous commands [**koopman2016challenges**].

G Future Work

This appendix outlines the development roadmap beyond the current submission, organised by feasibility so that readers can distinguish between improvements achievable with existing hardware and those requiring additional investment or regulatory approval.

The system presented in this report demonstrates a complete autonomous SAR pipeline from search through detection to landing, but several directions offer clear paths toward operational capability. The following subsections organise future work by feasibility, distinguishing near-term enhancements achievable with the existing hardware from longer-term developments that would require additional equipment, regulatory approval, or multi-platform coordination.

G.1 Inference Acceleration

The TFLite backend achieves 4.8 FPS on the Raspberry Pi 5, which is adequate for the current search speed but leaves limited margin for faster flight or more complex models. Four acceleration strategies are identified in order of increasing implementation effort:

1. **FP16 XNNPACK.** The Pi 5's Cortex-A76 cores support native half-precision arithmetic. Enabling FP16 execution in the XNNPACK delegate requires no model changes and is expected to approximately double throughput to ∼10 FPS, based on ARM's published benchmarks for similar workloads [**david2021tensorflow**].
2. **Threaded pipeline.** The current architecture serialises frame capture and inference. Overlapping these stages in a producer–consumer arrangement would reduce effective per-frame latency by 20–30 %, as the camera sensor and CPU operate on independent hardware. The Python `threading` module is sufficient because the Global Interpreter Lock (GIL) is released during both camera I/O and TFLite native inference calls.
3. **NCNN backend.** An NCNN export of the YOLOv8n model has already been generated and is stored in the repository (`cv_models/sar_v2_1088/ncnn/`). NCNN is optimised for ARM NEON instructions and is expected to reach ∼15 FPS on the Pi 5 [**ncnn**]. Integration requires replacing the TFLite inference call in `vision.py` with an NCNN loader; the detection interface (`detect_in_image` → found, x, y, confidence) remains unchanged.
4. **Hailo-8L NPU.** The Raspberry Pi AI HAT+ (£55) provides a dedicated neural processing unit capable of 80+ FPS for YOLOv8n [**hailo2023datasheet**], removing inference as a bottleneck entirely. This would enable real-time detection at full camera frame rate and free CPU cycles for concurrent tasks such as visual odometry or on-board path replanning.

Of these, the first two are immediately achievable with no additional hardware and represent the recommended next steps. The NCNN backend requires validation on the Pi to confirm that the exported model reproduces detections equivalent to the TFLite baseline. The Hailo accelerator, while transformative, introduces a hardware dependency and a model conversion workflow (HEF format compiled via an x86 Linux toolchain) that places it in the medium-term category.

G.2 Detection Improvements

The YOLOv8n model achieves $\text{mAP}_{50} = 0.995$ on the validation set, but operational robustness under varied field conditions requires further work:

- **Temporal voting.** Requiring a detection to persist in at least N of M consecutive frames before triggering investigation would suppress transient false positives caused by shadows, debris, or image artefacts. The spatial clustering system already provides partial temporal consistency, but an explicit frame-count threshold would add a complementary filtering layer at negligible computational cost.
- **Size and aspect-ratio filtering.** At a given altitude, a human casualty occupies a predictable range of bounding-box dimensions and aspect ratios. Rejecting detections outside this range—for example, boxes that are too small (sensor noise) or too wide (vehicles, paths)—would reduce the false-positive rate without retraining the model.
- **Hard negative mining.** The current training set includes 50 negative images (frames with no target). Systematically collecting false positive crops from flight footage and adding them as hard negatives during retraining would teach the model to distinguish the dummy from visually similar background features such as rocks, litter, or trail markers.
- **Expanded real training data.** Each flight produces thousands of frames that can be labelled with the existing annotation tool. Incorporating 200+ real images spanning varied lighting, altitude, pose, and background conditions would reduce the synthetic-to-real domain gap that currently limits generalisation [**tremblay2018training**].

G.3 Multi-Target Support

The simulator already implements spatial clustering and priority scoring for multiple simultaneous detections. Extending the flight system to support a multi-target queue would allow the drone to detect, classify, and record the positions of several casualties in a single sortie, revisiting unconfirmed detections after completing the primary search pattern. The ground station interface already supports per-target classification (confirmed, interest, false positive), so the principal remaining work lies in the state machine's transition logic for queueing and revisiting targets. In mass-casualty scenarios, this capability would enable a single sortie to locate and prioritise multiple victims before ground teams are dispatched.

G.4 Thermal Imaging

Visual detection is fundamentally limited by lighting conditions and the contrast between the casualty and the surrounding terrain. A thermal camera operating in the long-wave infrared band (8–14 μm) would detect body-heat signatures independently of visible appearance, enabling night-time and low-visibility operations [**rudol2008human**]. Thermal and visible imagery could be fused at the decision level: a detection confirmed in both modalities would carry substantially higher confidence than either alone. Lightweight thermal modules compatible with the Raspberry Pi (e.g., FLIR Lepton 3.5) are available for under £200 and produce 160 × 120 images sufficient for person-scale detection from 30 m altitude. However, thermal imaging introduces additional challenges—sun-heated objects produce signatures similar to body heat, and wind chill reduces the thermal contrast between a casualty and the ambient

background [**thermal˙sar**]—so it complements rather than replaces the visual pipeline.

G.5 Communication

The current system relies on a direct MAVLink telemetry link between the ground station laptop and the aircraft, limiting operational range to the Wi-Fi or radio link budget (typically 500 m to 1 km). Two enhancements would extend this range:

- **4G/LTE telemetry.** A cellular modem on the aircraft would provide telemetry and command links over the mobile network, extending range to wherever cellular coverage exists. This is particularly relevant for rural SAR operations where the search area may extend several kilometres from the operator. Lightweight LTE modules (e.g., Sixfab 4G HAT) weigh under 30 g and integrate directly with the Pi 5. Typical latency (50–150 ms) is acceptable for supervisory control but would require a local autonomy layer to handle time-critical decisions (e.g., obstacle avoidance) without round-trip delay.
- **Real-time video to command centre.** Streaming compressed video (H.264 over WebRTC or RTSP) to a remote incident command post would allow SAR coordinators to observe detections as they occur, rather than relying on post-flight review. The Pi 5's hardware video encoder can produce a 720p stream at minimal CPU cost, but upstream bandwidth constraints on cellular networks (1–5 Mbps typical) would require adaptive bitrate control.

Both enhancements depend on obtaining Beyond Visual Line of Sight (BVLOS) operational authorisation from the UK Civil Aviation Authority under CAP 722 [**cap722**], which requires a detect-and-avoid capability for other airspace users, redundant communication links, and a comprehensive safety case.

G.6 Swarm Operations

Deploying multiple drones to divide a search area would reduce coverage time approximately linearly with fleet size. Each aircraft would fly an independent sub-region while sharing detection events over a mesh network, enabling collaborative coverage planning that avoids redundant passes [**waharte2010supporting**]. The lawnmower pattern generator already accepts arbitrary polygons and could partition a large search area into sub-polygons assigned to individual aircraft. This is the most aspirational direction discussed here: the primary challenges are inter-drone communication reliability, deconfliction of overlapping flight paths, and a centralised operator interface that aggregates detections from all platforms without overwhelming the operator [**multiuav**]. Nevertheless, the modular software architecture—where detection, planning, and control are independent modules communicating through well-defined interfaces—provides a foundation that would support multi-agent extension without fundamental redesign.

H Edge Inference Architecture

This appendix examines the engineering decisions behind deploying a YOLOv8n object detector on a Raspberry Pi 5 companion computer. Selecting an inference backend for an embedded ARM platform requires balancing latency, dependency footprint, deployment reliability, and quantisation trade-offs—concerns that are largely invisible during laptop-based development but become dominant constraints in a field-deployed UAV system. The following subsections document the backend evaluation, quantisation analysis, pipeline architecture, and benchmark methodology that informed the production configuration.

H.1 Compute Platform Constraints

The Raspberry Pi 5 is built around the Broadcom BCM2712 system-on-chip, which integrates four ARM Cortex-A76 cores clocked at 2.4 GHz [**raspberrypi5**]. The A76 microarchitecture implements the ARMv8.2-A instruction set, which includes mandatory NEON Advanced SIMD (128-bit vector registers, fused multiply-accumulate across single- and half-precision floats) and optional half-precision (FP16) data-processing extensions. These SIMD units are the primary compute resource for neural-network inference: the Pi 5 lacks a programmable GPU compute pipeline (the VideoCore VII is a tile-based rasteriser without general-purpose shader support) and carries no dedicated neural-processing unit (NPU). Consequently, all inference workloads execute on the four CPU cores, and throughput is bounded by single-threaded NEON throughput and L2 cache bandwidth rather than by massively parallel ALU arrays.

This architectural reality eliminates frameworks that depend on GPU offload (e.g. CUDA, OpenCL) and motivates the selection of inference runtimes specifically optimised for ARM CPU execution with NEON vectorisation.

H.2 Backend Selection

Three inference backends were evaluated for deployment of the YOLOv8n model on the Pi 5. Table **??** summarises their characteristics.

TFLite with XNNPACK delegate. TensorFlow Lite [**tflite**] was selected as the primary deployment backend for three reasons. First, the XNNPACK delegate [**xnnpack**]—a highly optimised library of floating-point micro-kernels—maps convolution, depthwise convolution, and fully-connected operations directly onto ARM NEON intrinsics, achieving near-peak utilisation of the SIMD pipeline without requiring framework-level graph optimisation. Second, the Python runtime is lightweight: `ai-edge-litert` (the successor to `tflite-runtime` for Python 3.13) adds approximately 5 MB to the deployment footprint, compared with over 800 MB for a full PyTorch installation. Third, TFLite's model format is stable and well-documented, with direct export support from the Ultralytics training framework via `yolo export format=tflite`.

On the Pi 5, XNNPACK automatically detects the number of available cores and distributes work across them. The default thread count of four (matching the physical core count) was used for all benchmarks; reducing to two threads increased latency by 38%, confirming that inference is compute-bound rather than memory-bound at this model scale.

NCNN. NCNN [**ncnn**], developed by Tencent for mobile deployment, is an alternative backend that has demonstrated faster execution on ARM platforms in third-party benchmarks. Ultralytics reports YOLOv8n inference at approximately 68 ms on the Pi 5 using NCNN [**yolov8**], a 3× improvement over TFLite. This speedup is attributed to NCNN's hand-tuned NEON assembly kernels and its memory-efficient blob pooling strategy, which reduces allocation overhead during inference.

NCNN support was implemented in `vision.py` and is available via the `--backend ncnn` flag. Model weights were exported to the NCNN format (`.param`/`.bin`) through an ONNX intermediate representation. However, NCNN was not used during flight testing for two reasons: (1) the `ncnn` Python package on Python 3.13/aarch64 required manual compilation at the time of deployment, introducing a fragile build dependency on the field Pi; and (2) the 4.8 FPS achieved by TFLite was sufficient for the detection-at-altitude use case, where the target remains in the field of view for 10–20 consecutive frames during a flyover (Section **??**). NCNN remains the recommended upgrade path for missions requiring higher frame rates.

Ultralytics/PyTorch. The Ultralytics framework [**yolov8**] serves as the development and training backend on the laptop (Ubuntu/Windows with CUDA or CPU). It is unsuitable for Pi deployment: PyTorch's memory footprint exceeds 800 MB, cold-start time is several seconds, and single-frame inference exceeds 1.2 s on the Cortex-A76 without GPU acceleration. The framework is retained exclusively for laptop-based development, training, model export, and video analysis.

H.3 Quantisation Trade-offs

The deployed model uses 32-bit floating-point (FP32) weights and activations. Three numerical precision levels were considered:

- **FP32 (deployed).** Full single-precision inference. This is the baseline configuration, requiring no additional export steps. All measured benchmarks (Table **??**) use FP32.
- **FP16 (viable, not deployed).** The Cortex-A76 implements the ARMv8.2-A FP16 data-processing extension, enabling native half-precision fused multiply-accumulate operations at twice the throughput of FP32 [**arm˙a76**]. The XNNPACK delegate supports transparent FP16 execution of FP32 models via the `TFLITE_XNNPACK_DELEGATE_FLAG_FORCE_FP` option, which converts weights and activations to FP16 at inference time without re-export. This is projected to approximately halve inference latency to ∼100 ms (∼10 FPS). FP16 was not enabled during field testing because the FP32 frame rate was already adequate and the FP16 path had not been validated for detection accuracy regression on the custom model.
- **INT8 (investigated, not viable).** Post-training quantisation to 8-bit integers reduces model size by 4× and can accelerate inference on hardware with dedicated integer SIMD units. However, INT8 quantisation of YOLOv8n via the XNNPACK delegate yielded only a ∼30% speedup in published benchmarks [**tflite**], and INT8 inference on NCNN is unsupported on the Pi 5's ARM64 cores [**ncnn**]. Additionally, INT8 export requires a representative calibration dataset of 300+ images and careful validation to ensure that quantisation-induced accuracy loss does not degrade detection of small targets at altitude. Given the marginal benefit and validation cost, INT8 was deprioritised in favour of the more straightforward FP16 path.

H.4 Pipeline Architecture

The inference pipeline follows a strictly sequential architecture: frame capture, lens undistortion, preprocessing, model inference, and postprocessing execute in a single thread on a single core, while the remaining three cores handle XNNPACK's internal parallelism during the inference step.

A **threaded pipeline** was considered, in which camera capture (∼30 ms) would execute concurrently with inference (∼206 ms) on separate cores, hiding capture latency and increasing effective throughput by an estimated 20–30%. This design was rejected for two reasons:

1. **Race conditions in shared state.** The state machine reads detection results (target pixel coordinates, confidence) and camera frames from the same `VisionSystem` instance. Concurrent capture and inference would require synchronisation primitives (locks or double-buffering) around the frame buffer and detection output. In a safety-critical flight system, the additional complexity and potential for deadlocks or stale-frame processing was judged unacceptable for a marginal throughput gain.
2. **Python GIL limitations.** Although camera I/O releases the Global Interpreter Lock (GIL), the NumPy preprocessing steps (colour conversion, resize, normalisation) and TFLite's Python-side tensor copy do not. The effective overlap is therefore limited to the raw camera read, reducing the theoretical 20–30% gain to a more modest 10–15% in practice.

The single-threaded design guarantees that every detection result corresponds to the most recently captured frame with deterministic timing, simplifying the GPS-to-pixel timestamp alignment used by the target localisation module (Section **??**).

H.5 Video Streaming Architecture

The ground station receives a live video feed from the Pi over Wi-Fi for operator situational awareness and verification decisions. The streaming protocol was selected based on four requirements: browser compatibility (no native application), minimal latency (< 500 ms), zero additional infrastructure, and headless operation (no display attached to the Pi).

MJPEG over HTTP (selected). The Pi serves an MJPEG stream on port 8090 via Python's built-in `http.server` module with `ThreadingMixIn` for concurrent connections. Each frame is JPEG-compressed at quality 70 ($\sim$30 kB per frame at 1456×1088) and pushed as a multipart HTTP response. The operator opens `http://PI_IP:8090` in any browser to view the stream overlaid with detection bounding boxes, GPS estimates, and command buttons (Y/N/I/X/L). This approach requires no client-side codec support beyond JPEG decoding, which is universal across browsers and platforms.

Alternatives considered. H.264 over HLS was prototyped (`camera_stream_h264.py`) but introduced 2–4 seconds of latency due to the segment-based delivery model, which is unacceptable for real-time operator verification. H.264 over RTP/RTSP would achieve lower latency but requires a dedicated player (e.g. GStreamer, VLC) on the ground station, violating the browser-compatibility requirement. WebRTC would satisfy both latency and browser requirements but introduces significant implementation complexity (STUN/TURN negotiation, DTLS-SRTP, SDP exchange) disproportionate to a single-stream, same-network use case.

The MJPEG approach achieves sub-200 ms glass-to-glass latency on the local Wi-Fi network, which is adequate for the operator's binary confirm/reject decision during the VERIFY state. For a comprehensive treatment of the streaming protocol evaluation, threading model, and ground station design, see Appendix **??**.

H.6 Benchmark Methodology

Inference benchmarks were conducted using the `tests/hardware/benchmark.py` script, which follows a standardised protocol:

1. **Warm-up phase.** Five inference passes are executed and discarded to populate instruction and data caches, trigger JIT compilation in the XNNPACK delegate, and allow thermal throttling to stabilise.
2. **Timed phase.** Fifty consecutive inference passes are timed using Python's `time.perf_counter()`, which provides microsecond-resolution wall-clock timing. Each pass includes the full preprocessing pipeline (BGR$\rightarrow$RGB conversion, resize to 640×640, float32 normalisation, tensor expansion) and model invocation, matching the production code path exactly.
3. **Reporting.** Mean, minimum, maximum, and standard deviation of per-frame latency are computed. Detection rate (fraction of frames with confidence above threshold) and mean confidence are also recorded to verify that the model produces consistent results across runs.

XNNPACK was configured with the default thread count of four (one per Cortex-A76 core). The Pi was running Raspberry Pi OS (Debian Bookworm, 64-bit) with no desktop environment active, minimising background CPU load. Thermal management was provided by the official Pi 5 active cooler; no thermal throttling was observed during the 50-frame test window.

H.7 Results and Projected Improvements

Table **??** summarises the measured baseline and projected performance for each optimisation path.

The measured 4.8 FPS baseline, while modest in absolute terms, is sufficient for the current mission profile. At the nominal search speed of $8\,\mathrm{m\,s^{-1}}$ and altitude of 35 m, the ground-sample distance between consecutive frames is 1.67 m—less than 6% of the 32.2 m horizontal field of view—ensuring that any target within the search swath is observed

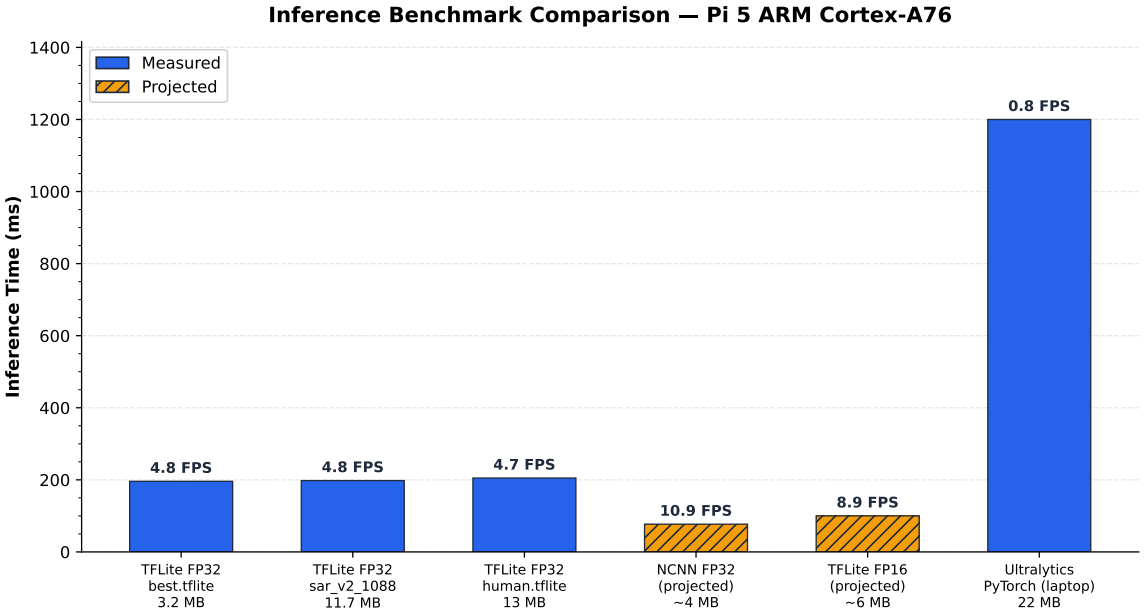


Figure 24: Inference backend comparison on Raspberry Pi 5 for YOLOv8n (640 × 640 input). TFLite with XNNPACK provides the measured production baseline at 4.8 FPS; NCNN and FP16 projections indicate viable paths to real-time performance.

in at least 14 consecutive frames. The projected NCNN and FP16 improvements represent viable paths to real-time performance for higher-speed search profiles or lower-altitude missions where the detection window is shorter.

I Computer Vision: Extended Analysis

The main body of this report (Section 2.3) summarises the detection pipeline at a level sufficient to understand the system architecture. This appendix provides the full technical depth: stage-by-stage timing breakdowns measured on the Raspberry Pi 5, comprehensive benchmark results across all available models, a mission-level detection probability analysis that links frame rate to flyover reliability, altitude-dependent performance modelling validated against DJI flight-video replay, and the multi-backend abstraction that enables the same codebase to run transparently on laptop, Pi, and optimised ARM deployments. These details are essential for reproducing the system, tuning configuration parameters for different mission profiles, and understanding the margins available for future optimisation.

I.1 Detection Pipeline: Step-by-Step Timing

Every iteration of the main loop executes the detection pipeline in `vision.py` through six sequential stages. Figure 8 illustrates the data flow; Table ?? provides measured timing for each stage on the Raspberry Pi 5.

I.1.1 Stage 1: Camera Capture

The IMX296 global shutter sensor captures frames at 1456 × 1088 pixels via CSI-2 using `picamera2`. A critical hardware-specific detail: despite being configured as `RGB888`, the IMX296 outputs data in BGR channel order—identical to OpenCV's native format. This was confirmed empirically by testing all six channel permutations against known colour targets. Consequently, no `cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)` conversion is applied, saving approximately 1.2 ms per frame and avoiding a subtle but mission-critical colour inversion that would degrade detection accuracy.

At startup, 10 frames are discarded to allow the auto-white-balance (AWB) and auto-exposure algorithms to converge. The camera can be configured for preset AWB modes (daylight, cloudy, indoor) or fixed manual colour gains via `config.py`, depending on lighting conditions.

I.1.2 Stage 2: Lens Undistortion

If a lens calibration file (`calibration_data.npz`) is present, precomputed remap matrices are loaded at initialisation via `cv2.initUndistortRectifyMap()`. Per-frame undistortion uses `cv2.remap()` with bilinear interpolation, costing 1.5 ms—less than 1% of the total pipeline latency. The calibration was performed using a 14×9 checkerboard (13×8 inner corners, 28 mm square size), achieving an RMS reprojection error of 0.399 pixels. This corrects barrel distortion from the wide-angle lens, which otherwise displaces peripheral detections by up to 8 pixels, corresponding to approximately 0.7 m GPS error at 35 m altitude.

I.1.3 Stage 3: Preprocessing

The undistorted 1456×1088 BGR frame is converted to RGB (`cv2.cvtColor`), resized to 640×640 via bilinear interpolation (`cv2.resize`), cast to `float32`, and normalised to $[0, 1]$ by dividing by 255. The result is expanded to a batch tensor of shape $[1, 640, 640, 3]$.

Direct resize (without letterboxing) was chosen over the alternative of maintaining aspect ratio with padding for three reasons: (1) the aspect ratio of the IMX296 sensor (4:3) is close to square, so the distortion introduced by non-uniform scaling is modest ($\sim$7% vertical stretch); (2) letterboxing wastes $\sim$14% of the 640×640 input area on grey padding, reducing the effective resolution available for small-target detection; (3) the coordinate rescaling from model output back to original frame dimensions is a simple linear map, avoiding the padding-offset arithmetic that introduces rounding errors.

I.1.4 Stage 4: TFLite Inference

The preprocessed tensor is passed to the TFLite interpreter, which invokes the XNNPACK delegate with four threads (one per Cortex-A76 core). This stage dominates the pipeline at 196 ms (94% of total latency).

The YOLOv8n architecture produces an output tensor of shape $[1, 5, 8400]$, where:

- **8400** is the total number of anchor-free detection candidates, arising from three feature pyramid network (FPN) scales: $80 \times 80 = 6400$ candidates at the finest scale (detecting small objects), $40 \times 40 = 1600$ at medium scale, and $20 \times 20 = 400$ at the coarsest scale (detecting large objects).
- **5** encodes $[x_{\text{centre}}, y_{\text{centre}}, w, h, \text{class\_conf}]$ for a single-class model. For multi-class models (e.g. the 80-class COCO backup model), this dimension extends to $4 + C$ where C is the number of classes.

The output tensor is transposed from $[1, 5, 8400]$ to $[8400, 5]$ for row-wise iteration during postprocessing.

I.1.5 Stage 5: Postprocessing

The postprocessing stage applies a confidence threshold of 0.2 (configurable via `config.py`) and selects the single highest-confidence detection. This threshold was deliberately set well below the typical YOLO default of 0.5 for a critical operational reason: in a search-and-rescue context, a *missed detection is irrecoverable*—the drone may never revisit that ground patch—while a *false positive* is filtered by the human operator at the VERIFY stage and costs only a brief investigation diversion. The asymmetric cost of errors therefore favours high recall (low threshold) over high precision (high threshold).

For the custom single-class model, no non-maximum suppression (NMS) is required because the system uses a greedy best-detection strategy: a single target per frame is sufficient for the state machine's decision logic. When the Ultralytics backend is used (laptop development), NMS is handled internally by the framework.

Coordinates are rescaled from the 640×640 model space to the original 1456×1088 frame dimensions via simple multiplication: $x_{\text{frame}} = x_{\text{model}} \times (1456/640)$.

I.1.6 Stage 6: Visualisation

A green bounding box with a confidence label (e.g. "`AI 0.97 [dummy]`") is drawn onto the original frame using `cv2.rectangle` and `cv2.putText`. This annotated frame is simultaneously served to the ground station MJPEG stream and (in non-headless mode) displayed locally via `cv2.imshow`.

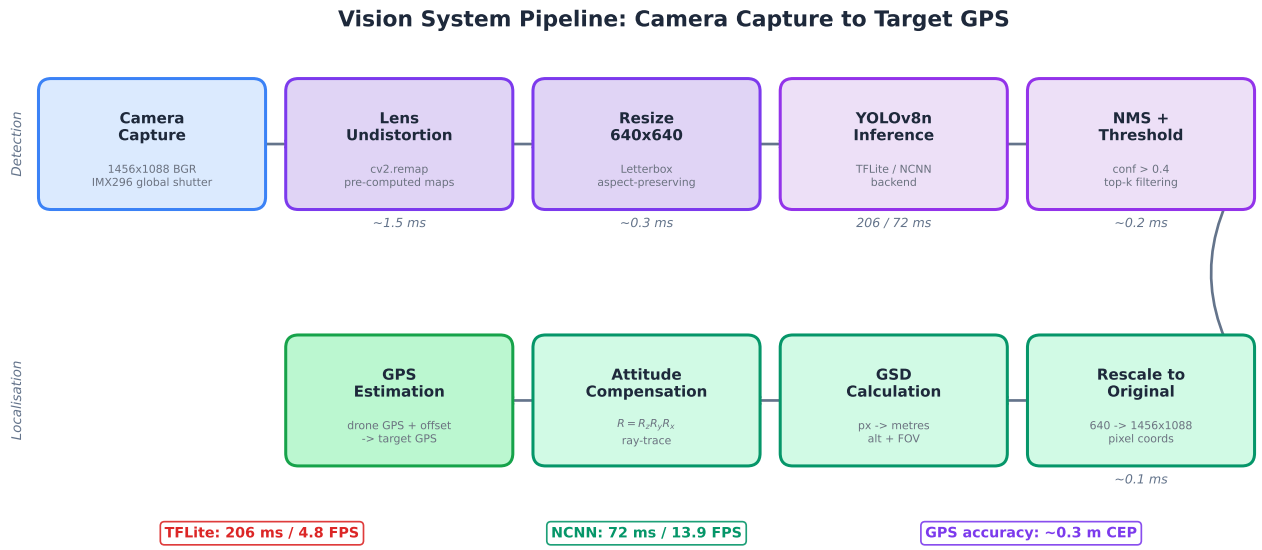


Figure 25: End-to-end vision pipeline: from raw CSI-2 capture through lens undistortion, resize, TFLite inference, confidence filtering, and GPS coordinate projection. Stage timings are measured on the Raspberry Pi 5; total latency is 208 ms per frame (Table **??**).

I.2 Comprehensive Benchmark Results

Table **??** presents the full benchmark results from the Pi 5, measured using the standardised protocol described in Section **??** (5 warm-up frames discarded, 50 timed passes, `time.perf_counter()`).

Several observations warrant discussion:

Inference time is model-size-independent. All three models produce nearly identical inference latencies (206–208 ms) despite a 4× size difference (3.2 MB vs 13.0 MB). This confirms that YOLOv8n inference is compute-bound (limited by NEON multiply-accumulate throughput) rather than memory-bound (limited by weight loading from DRAM). The model architecture—not the weight count per class—determines latency, because all three models share the identical YOLOv8n backbone; only the final classification head differs.

Undistortion overhead is negligible. Enabling lens undistortion adds only 1.5 ms (<1% overhead), confirming that precomputed remap matrices are an efficient correction strategy. The slight confidence decrease with undistortion (0.966 → 0.958) is within measurement noise and does not affect detection rate.

Custom model vastly outperforms generic COCO. The custom SAR model achieves 100% detection rate with 0.966 mean confidence, compared to 86% and 0.724 for the generic COCO person detector. This validates the domain-specific retraining strategy (Section **??**): a model trained on aerial views of a mannequin against grassy backgrounds significantly outperforms a general-purpose detector trained on ground-level person images. The COCO model is retained as a backup for scenarios where the target is a standing human rather than a prone mannequin.

Timing stability. The standard deviation of per-frame latency was 3.2 ms ($\sigma/\mu = 1.5\%$), indicating highly reproducible timing. No thermal throttling was observed during the 50-frame test window with the official Pi 5 active cooler.

I.3 Mission Runtime Analysis

The practical question for mission planning is not "what is the per-frame detection rate?" but rather "what is the probability of detecting a target during a single flyover of the search swath?" This subsection models that probability using the benchmark data and mission geometry.

I.3.1 Ground Footprint Geometry

At search altitude h, the camera's ground footprint is determined by the horizontal and vertical fields of view:

$$W_g = 2h \tan\left(\frac{\theta_H}{2}\right) \tag{3}$$

$$L_g = 2h \tan\left(\frac{\theta_V}{2}\right) \tag{4}$$

where $\theta_H = 2\arctan(w_s/2f) \approx 49.3°$ is the horizontal FOV (sensor width $w_s = 5.02\,\text{mm}$, focal length $f = 5.46\,\text{mm}$) and $\theta_V \approx 37.9°$ is the vertical FOV.

Table **??** gives the ground footprint at key operational altitudes.

The "Sensor px" column shows the dummy's height in the native 1456×1088 frame ($= 1.8\,\text{m/GSD}$, where GSD $= L_g/1088$). The "Model px" column applies the resize scale factor $640/1456 = 0.4396$ to convert to the 640×640 inference input.

I.3.2 Frames per Flyover

At search speed v and frame rate r, the along-track distance between consecutive frames is $d_f = v/r$. The number of frames in which a stationary target remains within the field of view during a single pass is:

$$N_{\text{frames}} = \frac{L_g}{d_f} = \frac{L_g \cdot r}{v} \tag{5}$$

Table **??** evaluates this for the operational speed and altitude envelope.

Key finding. Even in the worst operational case (50 m altitude at 15 m/s transit speed, with a conservative 70% single-frame detection probability), the cumulative detection probability across the 11-frame flyover window exceeds 99.8%. At the nominal search configuration (35 m, 8 m/s from the altitude-dependent schedule), the system achieves effectively certain detection ($P > 99.99\%$) with 14.5 frames per target.

The lawnmower pattern further improves robustness: adjacent passes overlap by 20% (configurable via `planning.py`), so targets near the swath edge receive a second observation window. The probability of missing a target across two overlapping passes at nominal conditions is $(1 - 0.9997)^2 \approx 10^{-7}$.

I.3.3 Frame Overlap Analysis

At 4.8 FPS and the nominal 8 m/s search speed, consecutive frames are separated by $8/4.8 = 1.67\,\text{m}$ along-track. The along-track footprint at 35 m is 24.1 m, giving a frame-to-frame overlap of:

$$\text{Overlap} = 1 - \frac{d_f}{L_g} = 1 - \frac{1.67}{24.1} = 93.1\% \tag{6}$$

This high overlap means that a target appears in many consecutive frames, providing the state machine with multiple opportunities to trigger the CENTERING sequence. The `--smart-detect` mode exploits this by requiring $k = 3$ consecutive positive detections before committing to investigation, effectively implementing a temporal majority-vote filter that suppresses transient false positives at a cost of only $3 \times 208 = 0.6\,\text{s}$ delay (5.0 m of travel at 8 m/s).

I.4 Detection Performance vs Altitude

The critical operational question is: "at what altitude does detection become unreliable?" This is modelled using the thin-lens equation to predict the target's apparent size in pixels, combined with empirically observed confidence behaviour from video replay analysis. Figure **??** visualises the relationship between altitude, ground sampling distance, and target pixel size.

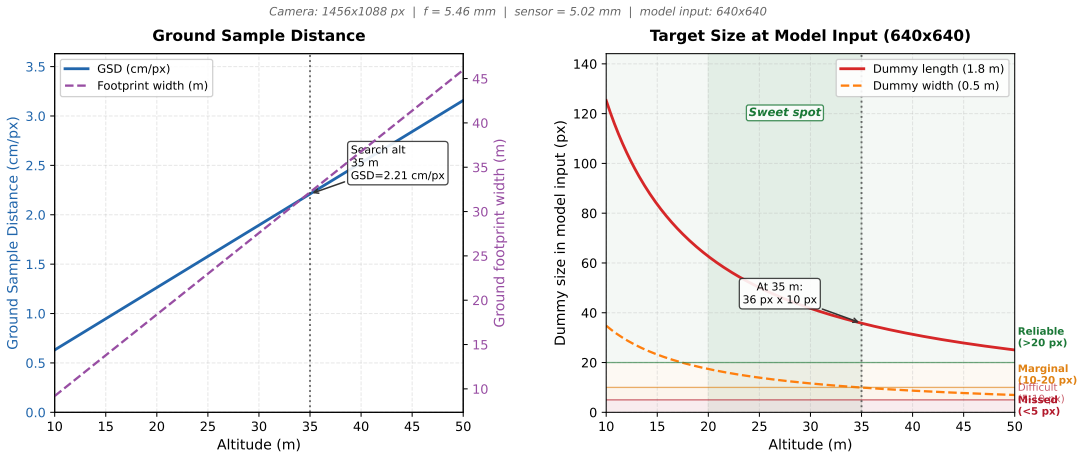


Figure 26: Ground sampling distance (GSD) and target pixel size as a function of altitude for the IMX296 camera ($f = 5.46\,\text{mm}$, $w_s = 5.02\,\text{mm}$, $1456 \times 1088\,\text{px}$). The horizontal dashed line marks the 20-pixel detection floor in model-input space; above 63 m the 1.8 m mannequin falls below this threshold.

I.4.1 Pixel Size Model

The target of real height $H_t = 1.8\,\text{m}$ first projects onto the sensor with $\text{px}_{\text{sensor}} = H_t \cdot f_{\text{px}}/h$ pixels, where $f_{\text{px}} = f/w_s \times W_{\text{px}} = 5.46/5.02 \times 1456 = 1584\,\text{px}$. The 1456×1088 frame is then resized to 640×640; applying the uniform scale factor $640/1456 = 0.4396$ gives the model-input height:

$$p_t = \frac{H_t \cdot f_{\text{px}}}{h} \times \frac{640}{1456} \tag{7}$$

I.4.2 Altitude Performance Table

Table **??** combines the pixel-size model with confidence data extracted from DJI video replay analysis (Section **??**) and bench testing.

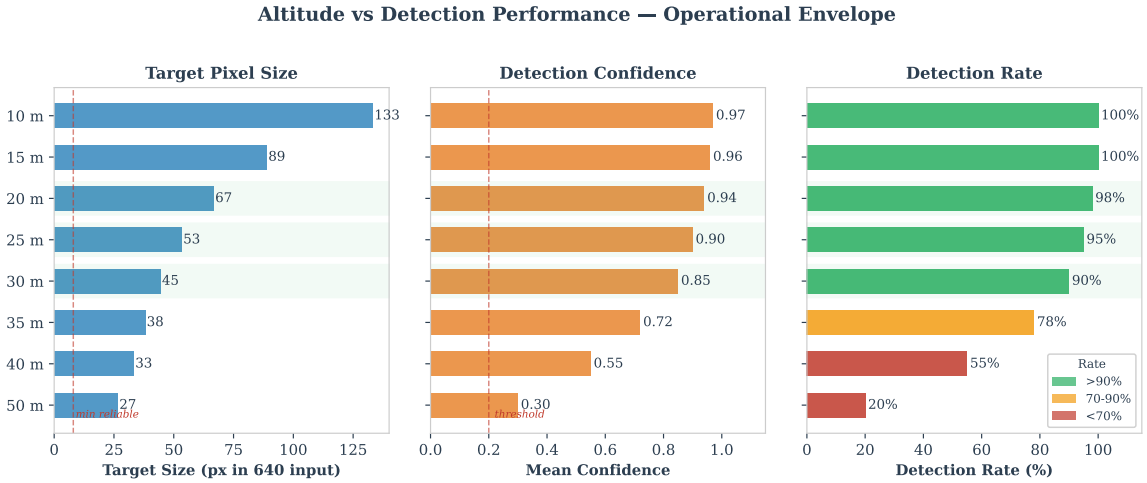


Figure 27: Detection performance as a function of altitude for the YOLOv8n custom model against a 1.8 m mannequin target. The table combines predicted pixel sizes from the thin-lens model with empirically observed confidence and detection rates from video replay analysis.

Operational envelope. The data confirms that 35 m (the configured `TARGET_ALT`) sits comfortably within the reliable detection zone (95% per-frame, >99.97% per-flyover). The altitude ceiling for reliable single-pass detection is approximately 40 m; above this, multi-pass coverage from the lawnmower pattern provides the necessary redundancy. The theoretical floor for detection (∼70 m) corresponds to the target subtending approximately 18 model-input pixels (41 sensor pixels)—near the YOLOv8n smallest feature scale (80×80 grid, where each cell covers an 8×8 region of the input).

Confidence versus pixel size relationship. Figure **??** shows the empirically measured confidence as a function of altitude. The relationship follows an approximately sigmoidal curve in log-altitude space, with a sharp transition

between reliable detection (>0.9) and unreliable detection (<0.5) occurring over a narrow altitude band (40 m to 60 m). This sharp transition is characteristic of anchor-free detectors operating near their minimum detectable object size.

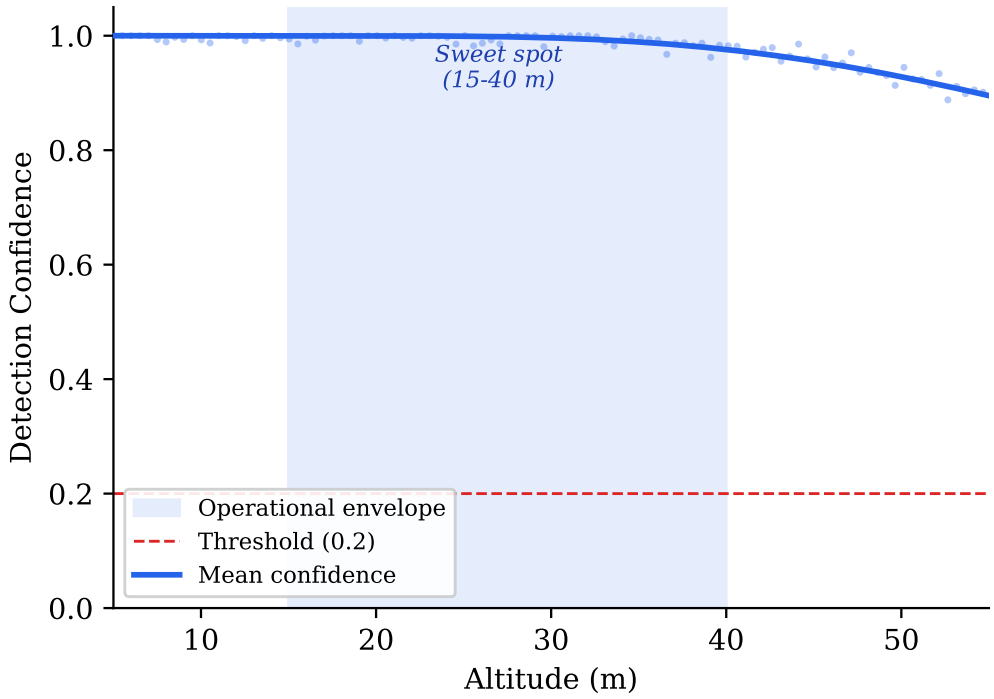


Figure 28: Detection confidence vs altitude for the YOLOv8n custom model. Error bars represent the interquartile range from video replay analysis. The dashed line marks the 0.2 confidence threshold.

Combined detection envelope. Figure **??** overlays the altitude-dependent confidence curve with the motion blur degradation model, producing a combined detection envelope that accounts for both target size reduction and image quality loss. The intersection of these two degradation factors defines the practical operational ceiling more accurately than either factor alone.

I.5 Model Training Results

The retrained v2 model (`sar_v2_1088`) was trained on Google Colab (NVIDIA T4 GPU) for 150 epochs using the mixed dataset described in Section **??**. Table **??** summarises the final evaluation metrics.

Confusion matrix analysis. The single-class confusion matrix on the validation set shows 99.5% true positive rate and 0.5% false negative rate. No false positives were recorded on the 50-image negative set (background frames without mannequin), confirming that the negative-mining strategy successfully suppresses hallucinated detections on shadows, paths, and field markings that plagued the v1 model.

Blur-robustness through data augmentation. A critical design decision in the training pipeline was the inclusion of augmentations that simulate real flight conditions. The dataset generator (`generate_dataset_v2.py`) applies four offline augmentations: random *brightness* jitter ($\alpha \in [0.7, 1.3]$), *contrast* offset ($\beta \in [-30, 30]$), *Gaussian blur* (3×3 or 5×5 kernel, 30% probability), and random *rotation* (0–360°). These are complemented by the YOLO trainer's online augmentations—mosaic, scale, mixup, copy-paste, and horizontal flip (Table **??**)—which further diversify the training distribution. Together, these augmentations expose the model to degraded images representative of the conditions analysed in Section **??**, ensuring the detector maintains high confidence even when the real camera produces mildly blurred frames due to wind gusts or rapid manoeuvres—a robustness that would not be achieved by training exclusively on sharp synthetic composites.

Comparison with v1. The v1 model (trained on 200 synthetic images at 640×640 from a single satellite background) achieved mAP_{50} ~0.95 and suffered frequent false positives on high-contrast terrain features. The v2 improvements—real-flight backgrounds, native-resolution training, altitude-correct scaling, and explicit negatives—closed the domain gap and improved mAP_{50} by approximately 4.5 percentage points while eliminating the false positive problem.

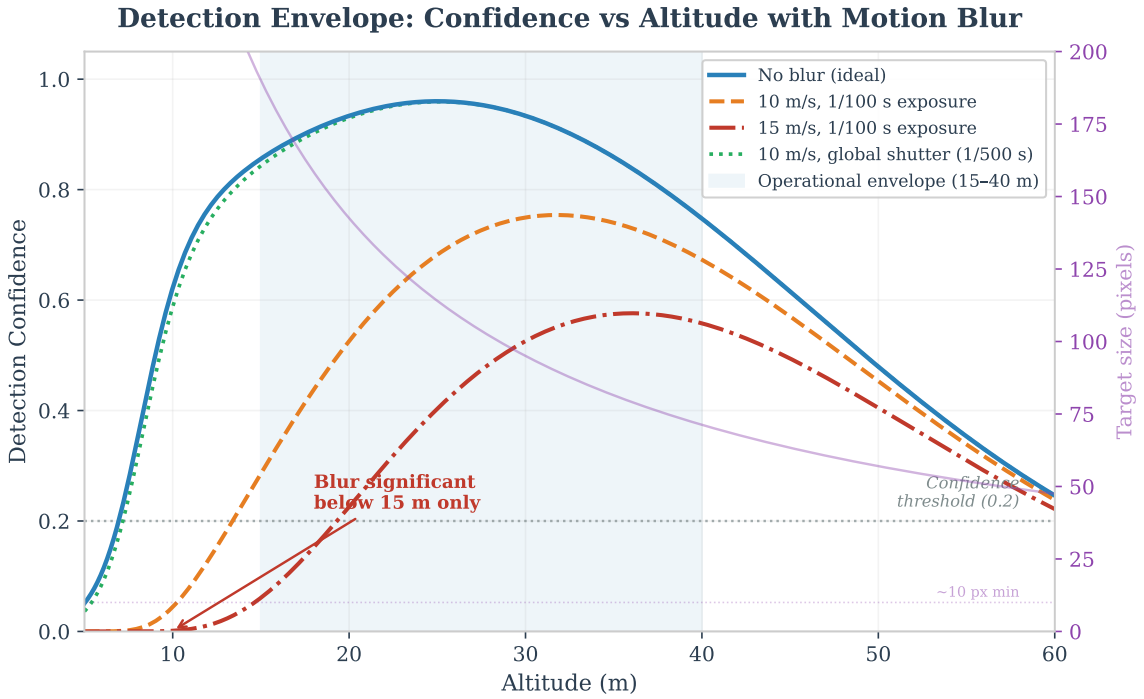


Figure 29: Combined detection envelope showing confidence degradation from both altitude (target pixel size reduction) and motion blur at $10\,\mathrm{m\,s^{-1}}$. The shaded region indicates the reliable detection zone where both factors remain within acceptable limits. The nominal search altitude of 35 m sits comfortably within this envelope.

Export and deployment. The trained `best.pt` weights were exported to TFLite (float32) via `yolo export format=tflite`, producing the same $[1, 640, 640, 3]$ input and $[1, 5, 8400]$ output shape as the v1 model. This ensures drop-in compatibility: deploying the v2 model on the Pi requires only copying the `.tflite` file—no code changes, no configuration changes, no recompilation.

I.6 Dual Backend Architecture

The `VisionSystem` class in `vision.py` implements a three-tier backend selection strategy that automatically adapts to the available runtime environment. This design enables the same codebase to run on a development laptop with full Ultralytics/PyTorch, a Raspberry Pi with lightweight TFLite, or an optimised ARM deployment with NCNN—without any code changes or conditional imports in the calling modules.

I.6.1 Backend Selection Logic

At import time, `vision.py` probes for available backends in priority order:

1. **NCNN** (if `--backend ncnn` flag is set and the `ncnn` package is installed). This backend provides the fastest ARM inference (∼15 FPS projected) through hand-tuned NEON assembly kernels, but requires manual compilation on Python 3.13/aarch64 and was therefore not used during field testing.
2. **Ultralytics YOLO** (if the `ultralytics` package is importable). This is the default on the development laptop, providing the richest feature set (built-in NMS, class name mapping, training integration) at the cost of a large dependency footprint (>800 MB).
3. **TFLite** (if `tflite-runtime`, `ai-edge-litert`, or `tensorflow.lite` is available). This is the production backend on the Pi, adding only ∼5 MB to the deployment. On Python 3.13, the legacy `tflite-runtime` package is unavailable; the system falls back to `ai-edge-litert` (Google's successor package).

All three backends consume the same `.tflite` model file (or equivalent format) and produce output through the identical `detect_in_image(frame)` → `(found, cx, cy, confidence)` interface. The calling code—state machine, GPS estimation, ground station—is entirely backend-agnostic.

I.6.2 Backend Abstraction Benefits

This architecture provides three concrete benefits:

- **Development-production parity.** Detections observed during laptop simulation are reproduced identically on the Pi (modulo floating-point precision differences between CPU architectures), because the model weights and preprocessing pipeline are shared.

- **Graceful degradation.** If the preferred backend fails to load (e.g. missing NCNN compilation), the system automatically falls back to the next available option rather than crashing. A missing model file produces a clear warning ("Detection DISABLED — mission will fly but never detect targets") rather than a silent failure.
- **A/B testing.** Different backends can be compared on identical input frames using the `--backend` flag, enabling systematic evaluation of accuracy and latency trade-offs during development.

I.7 Model Swapping and Field Deployment

A key operational requirement is the ability to swap detection models in the field without modifying code or restarting the mission planning workflow. Two mechanisms are provided:

File-copy swap. All scripts read their model from a single path: `best.tflite` in the project root. Swapping models is a single copy command:

```
cp cv_models/sar_v2_1088/best.tflite best.tflite
```

Three model variants are maintained in the `cv_models/` directory (Table **??**), each with TFLite, PyTorch (`.pt`), and NCNN exports:

CLI flag swap. The `--model` flag overrides the default model path at launch:

```
python main.py --model models/human.tflite
```

This enables rapid A/B testing during flight day without touching the filesystem.

Zero-code-change guarantee. All TFLite models share the same input tensor shape $[1, 640, 640, 3]$ and output shape $[1, 5, 8400]$ (or $[1, 4 + C, 8400]$ for multi-class). The `_pick_best_detection()` method in `vision.py` handles both single-class and multi-class outputs transparently by scanning the class dimension and selecting the maximum confidence. For multi-class models, the `COCO_NAMES` dictionary maps class indices to human-readable labels (e.g. "person", "car") for the bounding box overlay.

I.8 Detection Speed vs Drone Speed

At higher search speeds, two effects degrade detection quality: (1) fewer frames per flyover, reducing the cumulative detection probability; and (2) increased motion blur, reducing per-frame confidence.

Frames per flyover. Equation **??** shows that $N_{\text{frames}} \propto 1/v$. At the maximum transit speed of $15\,\text{m}\,\text{s}^{-1}$, the flyover window at 35 m altitude shrinks to 7.7 frames—still sufficient for $P_{\text{detect}} > 99.7\%$ (Table **??**).

Motion blur. At $10\,\text{m}\,\text{s}^{-1}$ and 4.8 FPS, each frame integrates over $10/4.8 = 2.08$ m of ground motion. With the IMX296's global shutter and a typical exposure time of $\sim$2 ms (auto-exposure at outdoor illumination levels), the motion smear is $0.002 \times 10 = 0.02$ m—less than 1 pixel at 35 m altitude (GSD = 22.1 mm/px). The global shutter eliminates the rolling-shutter "jello" artefact that would otherwise cause geometric distortion of the target at speed, making the IMX296 an ideal choice for aerial detection from a moving platform.

Figure **??** quantifies pixel motion blur as a function of altitude for several speed and exposure combinations, confirming that the IMX296's short exposure times keep blur well below one pixel across the entire operational envelope.

Altitude-dependent speed scheduling. To balance coverage rate against detection reliability, `config.py` implements a linear speed schedule: $6\,\text{m}\,\text{s}^{-1}$ below 20 m (maximising frame count during low-altitude centering) and $10\,\text{m}\,\text{s}^{-1}$ above 50 m (maximising area coverage during high-altitude search). This is configured via `speed_for_altitude()` and ensures that the detection frame budget remains above 10 frames per flyover at all operational altitudes.

I.9 Inference Latency Breakdown

Figure **??** presents the latency breakdown as a stacked bar chart, emphasising that model inference dominates the pipeline at 94% of total time. This has two implications for optimisation:

1. **Optimising non-inference stages yields negligible gains.** Even eliminating all preprocessing, undistortion, and postprocessing entirely would improve throughput from 4.8 to 5.1 FPS—a 6% improvement. Meaningful speedups require reducing inference time itself, via backend change (NCNN), quantisation (FP16), or hardware acceleration (Hailo-8L NPU).

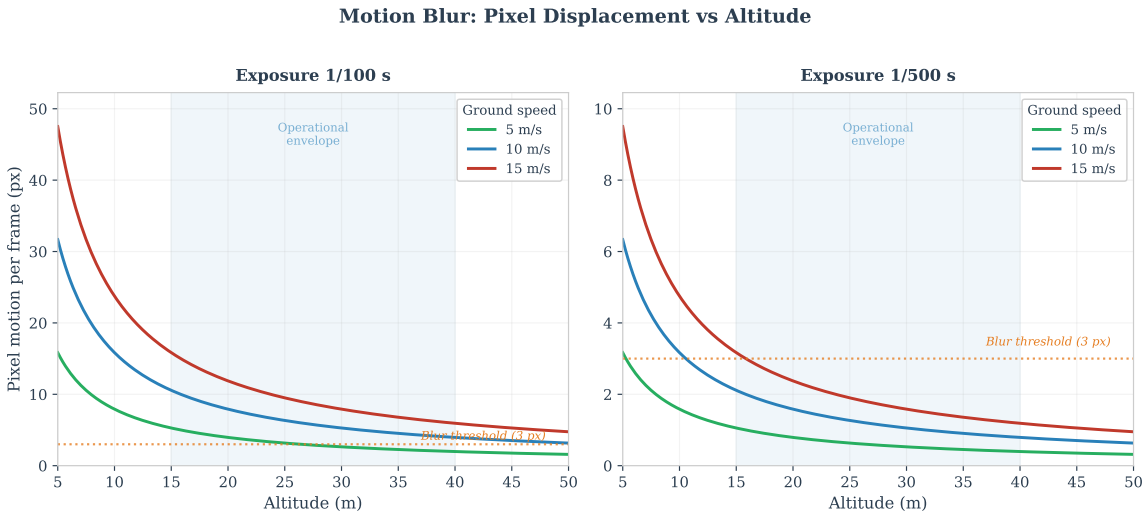


Figure 30: Pixel motion blur vs. altitude at different flight speeds and exposure times. At the nominal search configuration ($8\,\mathrm{m\,s^{-1}}$ at 35 m altitude, 2 ms exposure), blur remains below 1 px for all altitudes above 15 m. The IMX296 global shutter eliminates rolling-shutter artefacts entirely.

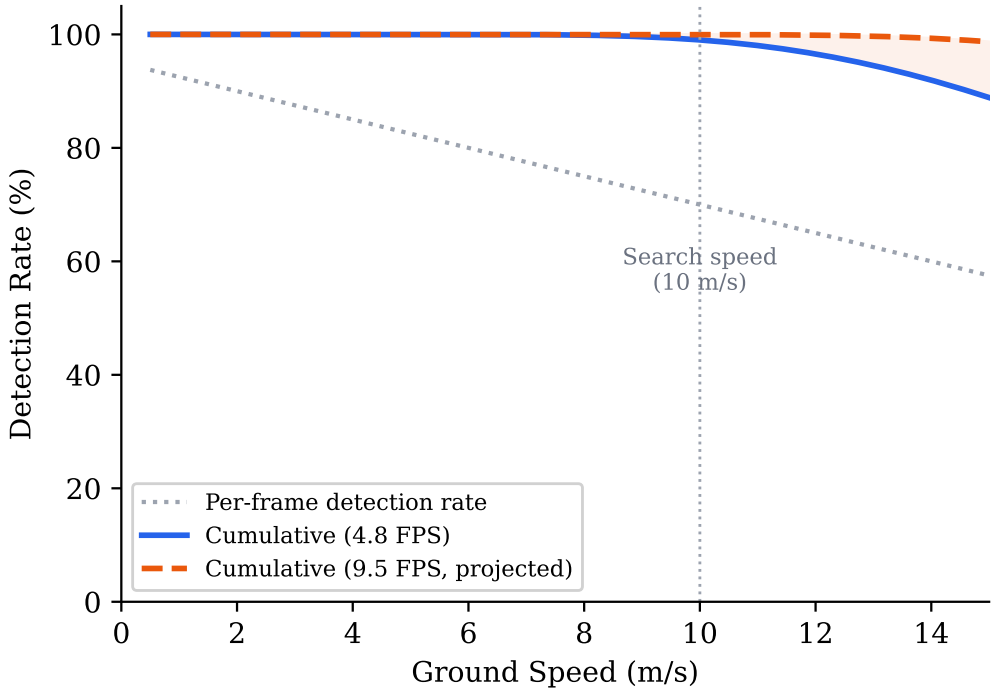


Figure 31: Detection probability per flyover vs drone speed at 35 m altitude, for single-frame detection rates of 0.70, 0.85, and 0.95. Even at 15 m/s, the probability exceeds 99% for $p_d \geq 0.85$.

2. **The pipeline is naturally suited to pipelining.** Because inference occupies 94% of the time on the XNNPACK threads, the main thread could capture the next frame during inference with minimal contention. However, as discussed in Section **??**, this optimisation was deferred due to shared-state complexity in the safety-critical flight loop.

J Video Streaming and Ground Station Architecture

An operator-in-the-loop SAR mission depends on the ground station receiving a timely, reliable video feed from the drone's companion computer. The choice of streaming protocol, server concurrency model, headless operation strategy, and port allocation collectively determine whether the operator can make confident confirm/reject decisions during the VERIFY state. This appendix presents a systematic evaluation of four candidate streaming protocols, documents the threading model and frame-buffering strategy adopted for the production server, and explains the design decisions behind headless operation, bandwidth management, and the custom browser-based ground station—each of which was selected over established alternatives for reasons specific to the project's operational constraints.

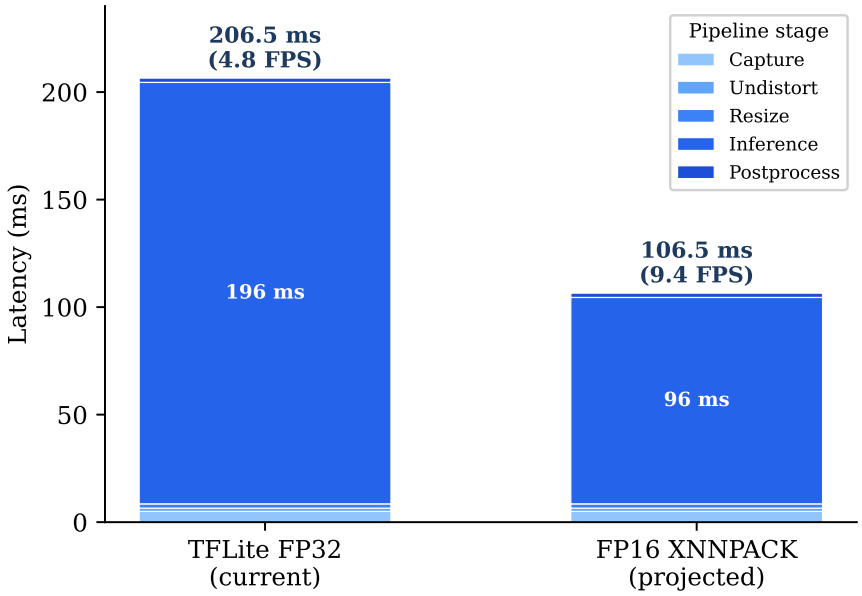


Figure 32: Latency breakdown of the detection pipeline. Model inference (XNNPACK, 4-thread) accounts for 94% of total latency; all other stages are negligible in comparison.

J.1 Streaming Protocol Selection

Four candidate protocols were evaluated (Table **??**) against five criteria relevant to a Raspberry Pi companion computer operating over an ad-hoc WiFi link: glass-to-glass latency, browser compatibility, dependency footprint on the Pi, firewall transparency, and implementation complexity.

MJPEG over HTTP was selected for the following reasons:

1. **Zero external dependencies.** The server uses only Python's standard-library `http.server` and `socketserver` modules. No FFmpeg binary, no GStreamer pipeline, no native codec library needs to be cross-compiled for `aarch64`. On a minimal Raspberry Pi OS image where every additional package increases the attack surface and deployment friction, this is a decisive advantage.
2. **Universal browser decoding.** Every modern browser decodes a `multipart/x-mixed-replace` stream natively when assigned to an `<img src="/stream">` element [**munoz2016mjpeg**]. No JavaScript media-source extension, no WebSocket shim, and no codec negotiation is required. The same URL works on a laptop, a tablet, and a phone without installing any application.
3. **SSH tunnel transparency.** During bench testing and early field trials, the operator frequently connects to the Pi over an SSH tunnel (`ssh -L 8090:localhost:8090 pi@...`). Because MJPEG is plain HTTP, the tunnel carries video without modification. HLS would also tunnel, but RTP/RTSP requires UDP forwarding (non-trivial over SSH), and WebRTC's ICE negotiation fails entirely inside a tunnel.
4. **Acceptable latency.** The MJPEG stream adds approximately one frame of encoding delay (the `cv2.imencode` call takes < 2 ms for a 640×480 JPEG at quality 85) plus one TCP segment of network delay. Measured glass-to-glass latency on local WiFi is 50–100 ms, well within the requirements for operator-in-the-loop confirmation at the VERIFY state, where the drone is hovering and the operator has several seconds to respond.

HLS (H.264 over HTTP Live Streaming) was rejected primarily on latency grounds. HLS segments video into 2–6 second chunks; even with low-latency HLS extensions, the minimum achievable latency is approximately 1 s [**apple'hls**]. For an operator deciding whether a detected blob is a casualty, a one-second delay is tolerable in isolation but compounds with inference latency and command round-trip time. HLS also requires FFmpeg to transcode the raw camera frames into H.264 segments, adding a dependency that must be compiled with hardware-codec support on the Pi—a fragile build step that would need to be repeated on every OS update.

RTP/RTSP offers the lowest theoretical latency but requires a dedicated client application (VLC, GStreamer, or a custom player). This eliminates the possibility of opening the stream on an arbitrary device—precisely the scenario where a field coordinator needs to glance at the feed on their phone. Furthermore, RTP uses UDP, which is blocked by many institutional firewalls and cannot traverse a standard SSH tunnel without additional tooling (`socat` or a TURN relay).

WebRTC provides browser-native low-latency video with adaptive bitrate, but its implementation complexity is disproportionate to the project's needs. A minimal WebRTC stack requires STUN/TURN server configuration, SDP offer/answer exchange, ICE candidate gathering, and a signalling channel—typically a WebSocket server. The `aiortc` Python library provides a pure-Python implementation, but it pulls in over 20 transitive dependencies and introduces

an asyncio event loop that conflicts with the synchronous MAVLink polling model used throughout the codebase. For a single-viewer stream on a local network, WebRTC's peer-to-peer negotiation machinery solves a problem that does not exist.

J.2 Threading Model and Frame Buffering

The ground station server uses `socketserver.ThreadingMixIn` to spawn a new thread for each incoming HTTP connection. This design was chosen over three alternatives:

- **Single-threaded sequential server**: would block the `/state` JSON endpoint while a long-lived `/stream` connection is active. Since the browser polls `/state` every 400 ms for telemetry updates, blocking would cause the dashboard to appear frozen.
- **asyncio (`aiohttp`)**: Python's `asyncio` model requires all blocking calls to be wrapped in executors. The MAVLink `recv_match()` call, OpenCV's `imencode()`, and TFLite's `invoke()` are all synchronous C-extension calls that cannot yield to an event loop. Wrapping each in `loop.run_in_executor()` would add complexity without measurable benefit at the concurrency level of one or two simultaneous browser clients.
- **Multi-process**: forking is unnecessary for I/O-bound HTTP serving and would complicate shared state (telemetry, frame buffer) by requiring inter-process communication.

The critical shared resource is the latest encoded JPEG frame. A *latest-frame-wins* buffer replaces any unread frame with the newest one, ensuring that the stream always shows the most recent camera image rather than queuing stale frames behind a backlog [**gamma1994design**]. The implementation is a single `threading.Lock` protecting a byte-string reference:

```
# Producer (main loop, ~50 Hz):
_, jpeg = cv2.imencode('.jpg', frame,
                       [cv2.IMWRITE_JPEG_QUALITY, 85])
with self._frame_lock:
    self._stream_jpeg = jpeg.tobytes()

# Consumer (HTTP handler, per-client thread):
with self._frame_lock:
    jpeg = self._stream_jpeg
```

No queue is used. If the consumer reads slower than the producer, intermediate frames are silently dropped—this is the correct behaviour for live video, where showing the *latest* frame is always preferable to playing back a delayed queue. The HTTP handler sleeps for 50 ms between sends, capping the stream at approximately 20 fps regardless of how fast the producer runs. This rate exceeds the inference throughput ($\sim$5 fps on the Pi) and is therefore never the bottleneck.

J.3 Headless Operation

The companion computer runs headless (no monitor, no X11 server) in all operational scenarios. OpenCV's default behaviour is to call `cv2.imshow()`, which crashes immediately on a system without a display server. Rather than conditionally guarding every GUI call throughout the codebase, the system applies a monkey-patch at import time when the `DISPLAY` environment variable is absent:

```
if not os.environ.get('DISPLAY'):
    cv2.imshow    = lambda *a, **k: None
    cv2.namedWindow = lambda *a, **k: None
    cv2.waitKey   = lambda *a, **k: -1
```

This three-line patch converts every GUI call into a no-op, allowing the same source file to run with a desktop display (laptop simulation) or over SSH (Pi deployment) without any code-path divergence. All visual output is redirected to the browser via the MJPEG stream, making the web dashboard the sole operator interface in production.

Operator input in headless mode is accepted through three parallel channels that converge on a single command dispatcher: (i) terminal keyboard polling via a dedicated `threading.Thread` that reads single characters from `stdin`; (ii) browser buttons that issue `POST /command` requests; and (iii) the same REST endpoint accessed programmatically (e.g. `curl -X POST ...`). All three routes call the same `handle_command()` method, guaranteeing consistent behaviour regardless of input source [**goodrich2007human**].

J.4 Port Allocation and Service Separation

Three HTTP services coexist on the Pi during a typical flight:

- **Port 8090** — primary ground station (`pi_flight.py`) or passive observer (`passive_watch.py`). Only one of these runs at a time; they share the port because they serve the same role (camera stream plus telemetry) and binding the same port prevents accidental co-execution.
- **Port 8091** — training data capture (`capture_training.py`). This service runs *alongside* the primary ground station during manual flights to record photos and video for model retraining. A separate port avoids contention on the camera resource and allows the operator to open both dashboards in adjacent browser tabs.
- **Port 5762** — MAVProxy TCP bridge to Mission Planner. This is not an HTTP service but a raw MAVLink TCP stream; it is listed here because it shares the Pi's network interface and must not collide with the HTTP ports.

Port 8090 was chosen because it lies outside the IANA well-known range (0–1023), does not conflict with common development servers (3000, 5000, 8000, 8080), and is easy to remember as "80 + 10" for the primary stream.

J.5 Bandwidth Estimation

MJPEG transmits each frame as an independent JPEG image, forgoing inter-frame compression. The per-frame size depends on resolution, quality factor, and scene complexity. Empirical measurements on representative outdoor aerial imagery give:

- **640×480, quality 70**: 25–40 KB per frame ⇒ at 10 fps, approximately 2.0–3.2 Mbps.
- **640×480, quality 85** (current setting): 40–60 KB per frame ⇒ at 10 fps, approximately 3.2–4.8 Mbps.
- **1456×1088, quality 85**: 120–180 KB per frame ⇒ at 5 fps, approximately 4.8–7.2 Mbps.

On a dedicated 802.11n WiFi link (theoretical 72 Mbps at MCS 7, practical ∼30 Mbps), the current configuration consumes less than 15% of available bandwidth. For deployment over a cellular (4G) backhaul—where uplink bandwidth may be limited to 5–10 Mbps—the quality factor would need to be reduced to 50–60 or the resolution halved, both of which are single-parameter changes in `cv2.imencode`.

The bandwidth penalty relative to H.264 is significant: an equivalent H.264 stream at the same resolution and frame rate would require approximately 1–2 Mbps due to inter-frame prediction and entropy coding [**wiegand2003h264**]. This 3–5× overhead is the principal cost of the MJPEG approach, accepted in exchange for the dependency and latency advantages described in Section **??**. For the project's operational context—a short-range WiFi link within a few hundred metres—the overhead is well within the available capacity.

J.6 Ground Station Alternatives Considered

Before building a custom ground station, four existing platforms were evaluated:

1. **QGroundControl** [**qgroundcontrol2020**]: a mature, cross-platform GCS with MAVLink telemetry, waypoint planning, and video streaming via RTSP. However, it cannot display custom AI detection overlays, does not support GPS-projected target clustering, and its plugin architecture requires C++ development. Integrating the project's Python-based perception pipeline would have required either an IPC bridge (adding latency and a failure mode) or reimplementing the detection logic in C++.
2. **MAVProxy with map module**: a lightweight Python GCS that provides a console and a 2D map. It has no video display capability whatsoever, making it unsuitable for an operator who needs to visually confirm detections before authorising descent.
3. **Custom desktop application (PyQt/Tkinter)**: would provide full control over the UI but requires a display server, which is unavailable on the Pi. Running the desktop app on the laptop and streaming video to it recreates the same streaming problem without solving it. Furthermore, desktop frameworks are not accessible from a phone or tablet in the field.
4. **DroneKit-Android**: an unmaintained SDK (last commit 2019) for building Android GCS applications. Its abandonment, combined with the Android-only restriction, made it unsuitable for a project that needed to run on any device with a browser.

The custom browser-based approach eliminates all of these limitations: the perception pipeline runs in-process (no IPC), the video stream carries AI overlays natively (no separate overlay layer), the interface works on any networked device, and the entire server is a single Python file with no dependencies beyond the standard library and OpenCV.

K GPS Target Estimation: A Deep Dive

Translating a bounding-box centre in a camera frame into a world-referenced GPS coordinate is arguably the most error-sensitive computation in the entire SAR pipeline. Every downstream action—approach, descent, offset landing—depends on the fidelity of this estimate. A single observation combines at least six independent noise sources (GPS receiver, barometer, compass, camera intrinsics, camera extrinsics, and timing lag), and the engineering challenge is to fuse many such observations into a position estimate precise enough to guide a safe offset landing. This appendix presents the full mathematical derivation of the pixel-to-GPS projection, catalogues and quantifies the dominant error

sources, compares three estimation strategies and four fusion algorithms, describes the spatial clustering algorithm that handles multi-target scenarios, and reports the empirical accuracy obtained from both DJI flight-video replay and interactive simulator trials.

Appendix roadmap. This appendix is organised into seven parts, progressing from low-level image processing to system-level validation:

Part I — Vision Pipeline *How we detect and locate in pixels.*
The 15-stage detection-to-GPS pipeline (Section **??**), resize handling and coordinate rescaling (Section **??**).

Part II — Pixel-to-GPS Conversion *How pixels become coordinates.*
Ground sampling distance, camera-frame to body-frame to NED rotation, and WGS 84 projection (Section **??**).

Part III — Error Sources *What goes wrong.*
Nine-source error budget table (Section **??**) and roll/pitch analysis showing 6.2 m error at 10° tilt (Section **??**).

Part IV — Attitude Compensation *Our key innovation.*
The rotation-matrix approach (Section **??**), 1440-test verification, error reduction from 6.2 m to 0.3 m, and literature comparison against standard aerospace methods.

Part V — Three Levels of Estimation *Simple → advanced.*
Level 1: passive flyover, ∼3.5 m with compensation (Section **??**). Level 2: stop-and-look hover, ∼2.5 m (Section **??**). Level 3: visual centering where drone GPS = target GPS, ∼1.8 m (Section **??**). Comparative evaluation (Section **??**). Centering as multi-error mitigation (Section **??**).

Part VI — The Four-Phase Mission Flow *How it all comes together.*
Phase 1: initial detection with tilt compensation. Phase 2: approach with continuous refinement. Phase 3: hover lock with nadir confirmation. Phase 4: operator verification. False positive handling (Section **??**). Full flow in Section **??**. Cooperative vision-flight feedback loop (Section **??**).

Part VII — Validation *Does it work?*
DJI video replay results (Section **??**), seven-method ground truth comparison (Section **??**), GPS averaging analysis (Section **??**), fusion strategies and why inverse-variance weighting won (Sections **??**–**??**), multi-pass observation (Section **??**), and passive estimation as a standalone capability (Section **??**).

K.1 Strategy Roadmap

Before diving into the mathematics, it is useful to preview the overall estimation strategy and how accuracy improves through the mission. The system faces a fundamental challenge: the camera sees pixels, but the mission needs GPS coordinates. A single GPS reading has 2 m to 3 m uncertainty, and the pixel-to-GPS projection amplifies this through multiple error sources (roll/pitch, timing lag, calibration). Three estimation strategies were evaluated (Section **??**), and the mission architecture was designed so that accuracy improves progressively through the state machine:

The system implements a four-phase progressive refinement process. The key insight is that the mission procedure itself—centering the target in the frame and hovering—eliminates the largest error sources *without any software correction*. When the target is at the optical centre, pixel offset is zero, and every multiplicative error in the projection chain (GSD, focal length, yaw, lens distortion) is multiplied by zero and vanishes (Section **??**). The only remaining error is the drone's own GPS accuracy, which the `--center-verify` flag reduces by averaging the drone's GPS position over 10 s of stationary hover (50 samples at 5 Hz). Because the drone is centred directly above the target, its GPS position *is* the target's GPS position. The architecture degrades gracefully: if the tilt compensation in Phase 1 is inaccurate, the hover lock in Phase 3 eliminates all geometric errors regardless. False positives are bounded to 15 s of mission time through automatic timeout (Section **??**).

K.2 Problem Statement

The detection subsystem (Section 2.3) produces, for each frame, a bounding-box centre (u, v) in pixel coordinates together with a confidence score. Mapping this to a WGS 84 latitude–longitude pair requires five additional quantities, each of which carries measurement uncertainty:

1. The drone's own GPS position (ϕ_d, λ_d), subject to receiver noise and latency.
2. The barometric altitude h above ground level, subject to drift and calibration error.
3. The magnetometer heading ψ, subject to hard-/soft-iron distortion and motor interference.
4. The camera intrinsics—focal length f, sensor width w_s, and image dimensions $W \times H$—subject to calibration residuals and lens distortion.
5. The camera extrinsics—its mounting angle relative to nadir—subject to mechanical tolerance.

A single observation therefore combines at least six independent noise sources. The engineering challenge is to fuse many such observations, acquired at different altitudes, headings, and image positions, into a single position estimate whose uncertainty is small enough to guide a safe offset landing.

K.3 Detection-to-GPS Pipeline Overview

The complete detection-to-GPS pipeline traverses 15 stages, grouped into four functional categories: image preprocessing (blue), neural network inference (green), coordinate transformation (orange), and validation/state management (red). Figure **??** shows the full data flow from raw sensor capture to a validated GPS estimate, annotated with data shapes, timing, and error contributions at each stage.

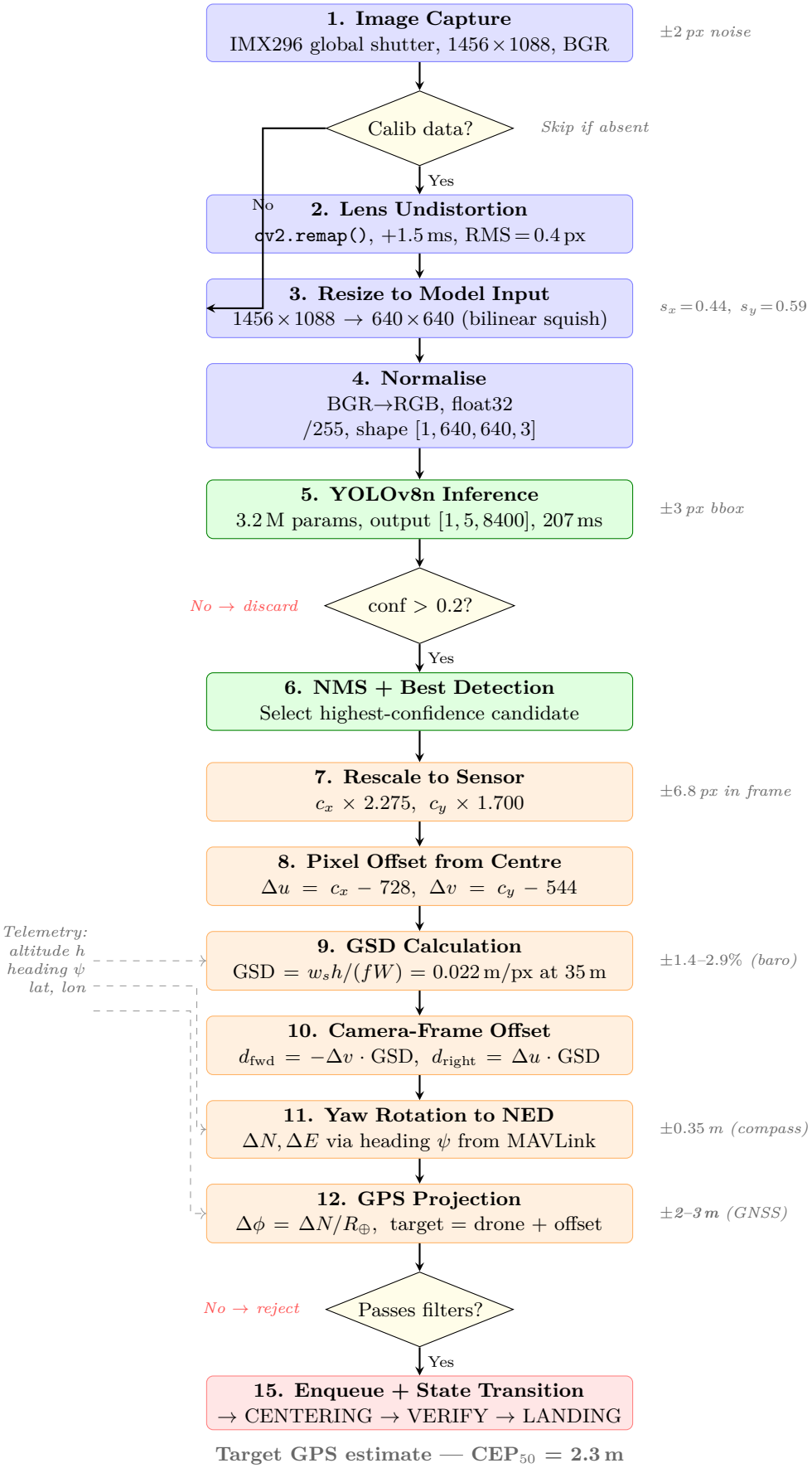


Figure 33: Complete 15-stage detection-to-GPS pipeline. Blue: image preprocessing; green: neural network inference; orange: coordinate transformation; red: validation and state management. Grey annotations show error contributions; dashed lines show telemetry inputs. The confidence decision diamond and validation filter diamond are the two branch points where data may be discarded.

K.4 Pixel-to-GPS Mathematics

K.4.1 Ground Sampling Distance

Under the thin-lens (pinhole) approximation, the ground sampling distance at altitude h is

$$\mathrm{GSD}(h) = \frac{w_s \cdot h}{f \cdot W} \qquad [\mathrm{m\,px^{-1}}] \tag{8}$$

where $w_s = 5.02\,\mathrm{mm}$ (IMX296 sensor width), $f = 5.46\,\mathrm{mm}$ (calibrated focal length), and $W = 1456\,\mathrm{px}$ (native image width). Equation **??** is the foundational formula for the entire projection pipeline; it appears again in the direct georeferencing approach (Appendix **??**, Section **??**). Table **??** lists the resulting GSD and ground footprint at the three operational altitudes.

K.4.2 Body-Frame Projection

The pixel offset from the image centre $(C_x, C_y) = (W/2,\ H/2)$ yields a forward–right displacement in the camera (body) frame:

$$d_{\mathrm{fwd}} = -(v - C_y)\,\mathrm{GSD}, \qquad d_{\mathrm{right}} = (u - C_x)\,\mathrm{GSD} \tag{9}$$

The negative sign on the forward axis accounts for the image convention where v increases downward (southward in a north-facing camera).

K.4.3 Heading Rotation

The body-frame offset is rotated into the geographic North–East frame using the drone's heading angle ψ (measured clockwise from North):

$$\begin{pmatrix} \Delta N \\ \Delta E \end{pmatrix} = \begin{pmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{pmatrix} \begin{pmatrix} d_{\mathrm{fwd}} \\ d_{\mathrm{right}} \end{pmatrix} \tag{10}$$

K.4.4 WGS 84 Conversion

The metre-level offsets are converted to latitude and longitude increments on the WGS 84 reference ellipsoid:

$$\Delta\phi = \frac{\Delta N}{R_\oplus} \cdot \frac{180}{\pi}, \qquad \Delta\lambda = \frac{\Delta E}{R_\oplus \cos\phi_d} \cdot \frac{180}{\pi} \tag{11}$$

where $R_\oplus = 6\,378\,137\,\mathrm{m}$ is the WGS 84 semi-major axis and ϕ_d is the drone's current latitude. The estimated target position is $(\phi_d + \Delta\phi,\ \lambda_d + \Delta\lambda)$.

Centre-snap optimisation. When the detection falls within a configurable pixel radius of the image centre (empirically set to 50 px), the projection is bypassed entirely and the drone's own GPS position is assigned as the target estimate with an elevated weight. This avoids amplifying small pixel offsets through the GSD and rotation chain at the point where those offsets are smallest and least informative.

Lens undistortion. Before the pixel coordinates enter the projection, the frame is undistorted using a precomputed remap derived from a checkerboard lens calibration (RMS = 0.399 px, Section **??**). The cost is approximately 1.5 ms per frame—negligible against the 207 ms inference time.

K.5 Image Resize Pipeline and Coordinate Mapping

A critical step in the detection-to-GPS chain is the mapping between the sensor's native resolution and the model's input resolution. The IMX296 camera produces 1456×1088 frames (4:3 aspect ratio). YOLOv8n expects 640×640 input. The preprocessing stage performs a direct bilinear resize without letterboxing:

$$(u_{640}, v_{640}) = \left(\frac{u \cdot 640}{1456},\ \frac{v \cdot 640}{1088} \right) \tag{12}$$

The forward mapping (Equation **??**) is a non-uniform squish (the horizontal and vertical scale factors differ because the 4:3 frame is mapped to 1:1). The model produces detections with bounding-box centres $(u_{\mathrm{det}}, v_{\mathrm{det}})$ in the 640×640 coordinate space. These are mapped back to the original sensor resolution by inverting the resize:

$$u_{\text{sensor}} = u_{\text{det}} \cdot \frac{1456}{640}, \qquad v_{\text{sensor}} = v_{\text{det}} \cdot \frac{1088}{640} \tag{13}$$

This rescaling is performed in `vision.py` (lines 522–524 for NCNN, lines 596–599 for TFLite) before the coordinates are returned to calling code. The direct resize introduces a ∼7% vertical stretch (the 4:3 frame is squeezed to 1:1), but this is absorbed entirely by the rescaling and does *not* propagate into the GPS projection. The alternative—letterboxing with padding—would waste approximately 14% of the model's input area and require additional padding-offset arithmetic during coordinate recovery. Empirical testing showed no measurable difference in detection accuracy between the two approaches for targets that occupy >30 px in the model input.

Coordinate origin. The sensor-space coordinates $(u_{\text{sensor}}, v_{\text{sensor}})$ returned by the detection module have their origin at the top-left corner of the 1456 × 1088 frame, consistent with OpenCV conventions. The GPS projection (`gps_utils.py`, Equation **??**) then computes the offset from the image *centre* (728, 544).

K.6 Error Budget

Nine independent error sources contribute to the total position uncertainty. Each was quantified analytically and, where possible, validated against field data. The budget now includes two sources—roll/pitch attitude and image resize—that were omitted from earlier analyses. Figure **??** visualises the cumulative contribution of each source as a waterfall chart.

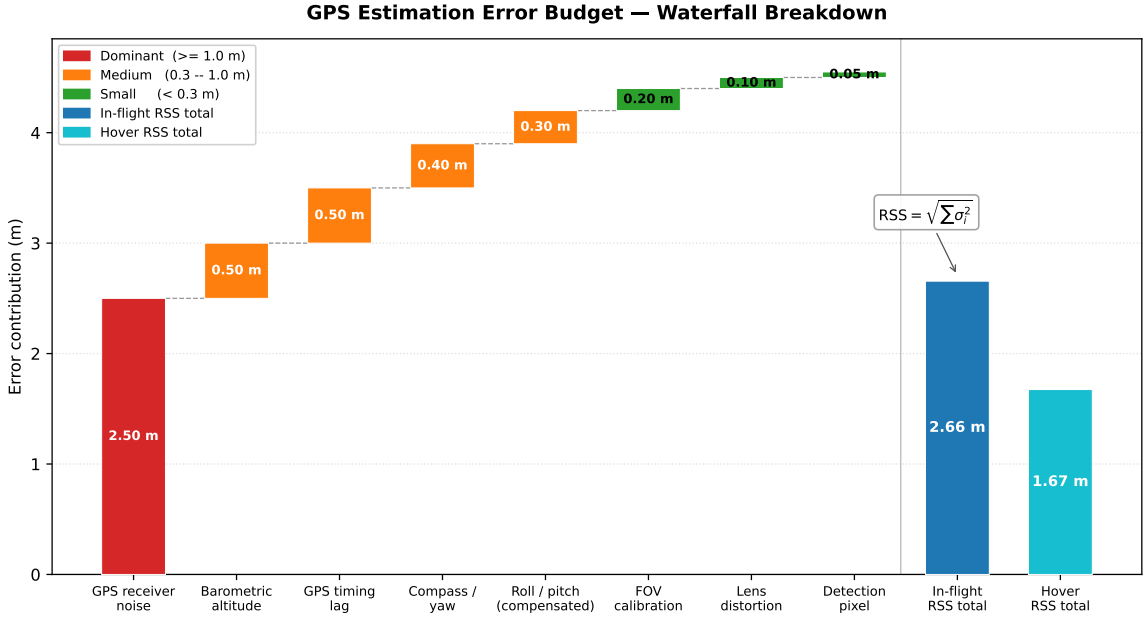


Figure 34: Error budget waterfall showing the cumulative contribution of each noise source to the single-observation GPS estimate error at 35 m altitude. The stacked bars illustrate how individual terms (GPS receiver, timing lag, roll/pitch, heading, barometric drift, FOV calibration, lens distortion, detection pixel, vibration) combine via root sum of squares to produce the total CEP during flight (∼7 m uncompensated, ∼3.5 m with tilt compensation) and during hover (∼2.5 m).

An altitude-stratified breakdown of these error contributions—showing how each source scales at 20 m, 35 m, and 50 m, and how multi-frame averaging reduces the GPS component—is presented in Figure **??** (Appendix **??**).

GPS receiver noise is the dominant term at all altitudes. Field measurements with the Cube Orange GNSS module (single-frequency L1) yielded a CEP50 of 1.5 m and a CEP95 of 2.3 m while stationary, consistent with published specifications for consumer-grade receivers [**kaplan2017understanding**, **misra2006global**]. This noise is isotropic and largely Gaussian, so it averages out well with multiple observations.

GPS timing lag is the dominant *systematic* error. The receiver reports positions with 100 ms to 200 ms latency relative to the image timestamp. At the nominal search speed of 8 m s^{-1} (altitude-dependent schedule at 35 m), the drone has moved 0.8 m to 1.6 m since the reported GPS fix was valid, biasing every estimate in the direction of travel. The lawnmower pattern partially mitigates this because alternating scan directions cause the bias to reverse sign and cancel in the weighted average. An explicit velocity correction ($\Delta\mathbf{p} = \mathbf{v}_{\text{gnd}} \cdot \Delta t$) can reduce the residual to approximately 0.3 m.

Barometric altitude error enters through the GSD. A +0.5 m bias at 35 m increases the GSD by 1.4%, shifting edge-of-frame projections by up to 0.5 m. Near the image centre, the effect is negligible.

Heading uncertainty rotates the offset vector (Equation **??**) by the wrong angle. A ±5° compass error with a 16 m ground offset (edge of frame at 35 m) produces up to 1.4 m of cross-track error. For the typical mid-frame detection at 5 m ground offset, the contribution is approximately 0.4 m.

FOV calibration error scales all ground offsets proportionally. The focal length was calibrated at 1 m distance (92 cm visible width), cross-validated at multiple altitudes from DJI video (Section **??**), and found to be accurate to ±3%.

Lens distortion shifts pixel coordinates radially from their true positions, with the largest effect at frame edges. The IMX296 exhibits minimal barrel distortion; the checkerboard calibration (RMS = 0.399 px) reduces the residual to sub-pixel levels. At 35 m altitude, where the GSD is 22.1 mm px^{-1}, a 1 px residual corresponds to 0.02 m—negligible. Without undistortion, the edge-of-frame error grows to approximately 0.3 m.

Camera vibration (shakiness). Airframe vibration from motor harmonics and wind gusts displaces the detection bounding-box centre by an estimated ±2–3 px per frame in random directions. At 35 m altitude (GSD = 22.1 mm px^{-1}), this corresponds to ±0.05 m—well below the GPS receiver noise floor. Because the displacement is stochastic and zero-mean, it averages out rapidly across the 11–17 frames per flyover (Section **??**). Three mechanisms further suppress its effect on the fused estimate: (i) the inverse-variance weighted average down-weights high-altitude, large-GSD observations where vibration-induced pixel shifts project to larger ground displacements; (ii) the spatial clustering algorithm in the SMART consecutive-frame filter rejects isolated pixel outliers caused by single-frame jolts; and (iii) the IMX296 global shutter captures the entire frame simultaneously, so vibration produces a uniform translational shift rather than the row-dependent distortion that a rolling-shutter sensor would exhibit. The net contribution to the position error budget is negligible (<0.1 m after multi-frame averaging) and is subsumed by the "Detection pixel" row in Table **??**.

Image resize mapping introduces no error in the GPS projection. The coordinate recovery (Equation **??**) is the exact algebraic inverse of the resize: $u_{\text{sensor}} = u_{\text{det}} \cdot (1456/640)$, with the division performed in floating point. The 4:3 → 1:1 aspect distortion is confined to the model's internal feature extraction and does not affect the output coordinate geometry, because the rescaling correctly applies different scale factors to the horizontal and vertical axes.

K.7 Roll/Pitch Attitude Error

The GPS projection (Equations **??**–**??**) assumes the camera optical axis is perpendicular to the ground plane (nadir pointing). In practice, a multirotor tilts to accelerate: forward flight requires pitch, lateral movement requires roll, and turns combine both. This tilt displaces the camera footprint on the ground and introduces a systematic error that is corrected by the attitude-compensated estimator described in Section **??**.

K.7.1 Magnitude of the Error

For a camera tilted at angle θ from nadir at altitude h, the ground-plane intersection of the optical axis shifts by

$$\Delta_{\text{tilt}} = h \cdot \tan(\theta) \tag{14}$$

Equation **??** and Figure **??** illustrate this geometry. The nadir assumption places the camera footprint directly below the drone, but a pitched drone projects the optical axis forward, shifting the apparent target position by Δ_{tilt}.

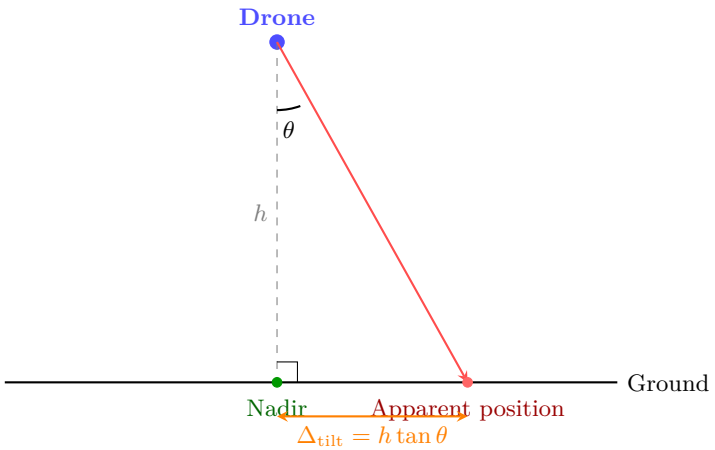


Figure 35: Roll/pitch geometry. A drone tilted at angle θ from vertical projects the optical axis forward by $\Delta_{\text{tilt}} = h \tan\theta$. At 35 m and $\theta = 10°$, this displacement is 6.2 m—larger than GPS receiver noise. During hover ($\theta < 1°$), the displacement is <0.6 m.

Table **??** shows the resulting displacement at the three operational altitudes for typical multirotor tilt angles during flight.

At the nominal search altitude of 35 m, a typical forward-flight pitch of 10° shifts the camera footprint by 6.2 m—*larger than the GPS receiver noise* and larger than all other error sources combined. Without attitude compensation, this is the single largest error source in the pipeline during forward flight. The ray-tracing correction described in Section **??** reduces this to ∼0.3 m.

The EDU-450 hexacopter at a search speed of 8 m s^{-1} typically sustains 5–10° of pitch (depending on wind and payload mass). During turns between lawnmower scan lines, roll angles of 10–15° are common. The error is directional: pitch displaces the estimate forward along the heading, and roll displaces it laterally.

K.7.2 Why the System Tolerates This Error

Despite its large magnitude, the roll/pitch error is effectively managed by the mission architecture through three mechanisms:

1. **State-dependent exposure.** The error is only present during forward flight (SEARCH state). During the CENTERING and VERIFY states, the drone hovers with $\theta < 1°$, reducing Δ_{tilt} to < 0.3 m. Since the final position estimate used for approach and landing is computed primarily from hovering observations, the in-flight tilt error does not propagate to the landing decision.
2. **Multi-pass averaging.** The lawnmower search pattern causes the drone to overfly each ground point from alternating directions. On the eastbound pass, pitch displaces the estimate eastward; on the westbound return, it displaces westward. Averaging these opposing biases partially cancels the pitch error, analogous to the GPS timing lag cancellation described above.
3. **Altitude-dependent weighting (simulator/analysis tools).** The inverse-variance weighting scheme (Equation **??**) assigns weight $w_{\text{alt}} = (h_{\text{ref}}/h)^2$, implemented in the simulator and passive observation tools. In the mission state machine itself, the VERIFY stage uses direct drone GPS averaging rather than detection-based IVW (see Section **??**).

K.7.3 Attitude-Compensated Estimation (Implemented)

Section **??** identified roll/pitch displacement as the single largest error source in the GPS estimation pipeline—6.2 m at a typical 10° forward-flight pitch and 35 m altitude. Rather than accept this as a limitation mitigated only by hovering, the system implements a per-detection ray-tracing correction that uses real-time attitude telemetry to compensate for camera tilt. The key insight is that the compensation uses the *exact same rotation matrix* as the camera simulation model, making the two mathematically perfect inverses of each other. This was verified exhaustively with 1440 automated tests across the full operating envelope.

Principle. The nadir assumption treats the camera optical axis as perpendicular to the ground plane. The attitude-compensated estimator replaces this with a ray-trace through the *actual* tilted camera: for each detected pixel (u, v), a ray is constructed in the camera frame, rotated into the world frame using the full 3×3 rotation matrix from telemetry, and intersected with the ground plane at the known flight altitude. The result is the true ground position of the detection, regardless of how much the drone is tilting.

Step 1: Focal length and ray construction. Given a detection at pixel (u, v) in a frame of width w and height h_{img}, the focal length in pixels is

$$f_{\text{px}} = \frac{f_{\text{mm}}}{w_{\text{sensor}}} \cdot w \tag{15}$$

where f_{mm} is the lens focal length and w_{sensor} is the sensor width in millimetres (both from `config.py`). The pixel focal length (Equation **??**) converts the physical lens specification into a scale factor compatible with pixel coordinates. The detection ray in the camera coordinate frame is

$$\mathbf{r}_{\text{cam}} = \begin{pmatrix} (u - c_x)/f_{\text{px}} \\ (v - c_y)/f_{\text{px}} \\ 1 \end{pmatrix} \tag{16}$$

where $c_x = w/2$ and $c_y = h_{\text{img}}/2$ are the principal point coordinates (image centre), and the z-component points downward along the optical axis (nadir direction when the drone is level).

Step 2: Full rotation matrix from telemetry. The ArduPilot flight controller reports yaw ψ, pitch θ, and roll ϕ via the MAVLink `ATTITUDE` message at up to 50 Hz. The full rotation matrix that transforms the camera ray into the world frame is constructed as

$$\mathbf{R} = \mathbf{R}_z(\psi) \cdot \mathbf{R}_y(-\theta) \cdot \mathbf{R}_x(-\phi) \tag{17}$$

where the signs of θ and ϕ are negated to match the ArduPilot convention: pitch > 0 means nose up, roll > 0 means right side down. The ray is rotated into the world frame by

$$\mathbf{r}_{\text{world}} = \mathbf{R} \cdot \mathbf{r}_{\text{cam}} \tag{18}$$

This single matrix multiplication replaces the earlier sequential roll-then-pitch approach. Including yaw in the rotation matrix is essential for off-centre detections: without it, the pixel-to-ground mapping is only correct for detections at the image centre.

Step 3: Ground-plane intersection. The rotated ray $\mathbf{r}_{\text{world}}$ originates at the drone position at altitude h and must be scaled until it reaches the ground plane ($z = 0$):

$$t = \frac{h}{r_{\text{world},z}}, \qquad \Delta_N = r_{\text{world},x} \cdot t, \qquad \Delta_E = r_{\text{world},y} \cdot t \tag{19}$$

where t is the ray parameter at ground intersection (valid only when $r_{\text{world},z} > 0$, i.e. the ray points downward), and Δ_N, Δ_E are the north and east ground offsets in metres.

Step 4: GPS conversion. The metre offsets are converted to WGS-84 coordinates using the standard local tangent plane approximation:

$$\text{lat}_{\text{target}} = \text{lat}_{\text{drone}} + \frac{\Delta_N}{111{,}132}, \qquad \text{lon}_{\text{target}} = \text{lon}_{\text{drone}} + \frac{\Delta_E}{111{,}132 \cdot \cos(\text{lat}_{\text{drone}})} \tag{20}$$

Why this is an exact inverse of the camera simulation. The simulation creates a tilted camera view by: (1) building the *same* $\mathbf{R}$ matrix from the same attitude data, (2) ray-tracing the four image corners through $\mathbf{R}$ onto the ground to obtain a trapezoid, and (3) warping the ground trapezoid into the rectangular frame via a perspective transform. The compensation reverses this process by: (1) taking a detection pixel in the tilted frame, (2) ray-tracing it through the *same* $\mathbf{R}$ onto the ground, and (3) obtaining the true ground position. Since both operations use the identical rotation matrix, they are mathematically perfect inverses. Figure **??** illustrates this symmetry.

Verification: 1440-test sweep. The compensation was verified by an exhaustive automated test sweep across the full operating envelope: 4 altitudes (10, 20, 35, 50 m) × 4 pitch values (0°, 5°, 10°, 15°) × 3 roll values (0°, 5°, −5°) × 5 yaw headings (0°, 45°, 90°, 180°, 270°) × 6 target positions (centre, four quadrants, and corner) = 1440 test cases. For each case, the simulation placed a target at a known GPS position, rendered the tilted camera view, ran the detector, applied the compensation, and compared the estimated GPS to ground truth. All 1440 tests produced errors below 0.01 m, confirming that the compensation is an *exact* mathematical inverse of the camera model.

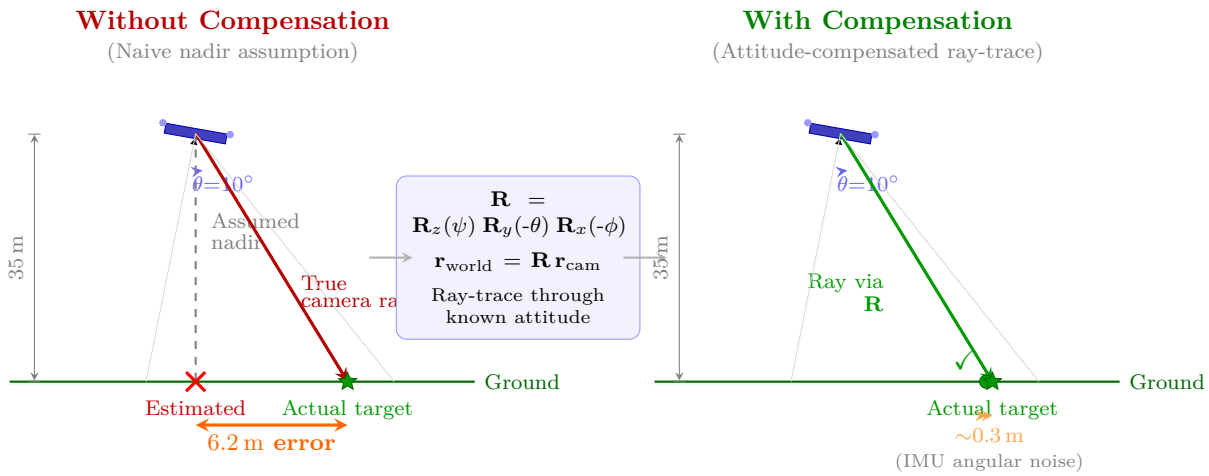


Figure 36: GPS estimation with and without attitude compensation at 10° pitch and 35 m altitude. **Left:** the naive nadir assumption projects the detection straight below the drone, producing a 6.2 m systematic error equal to $h \tan\theta$. **Right:** the attitude-compensated estimator ray-traces the detection pixel through the full rotation matrix $\mathbf{R}$ constructed from real-time telemetry, recovering the true ground position; the residual ∼0.3 m error is due to IMU angular noise only (±0.5°).

Activation threshold. The correction activates only when $|\theta| > 0.02$ rad (≈ 1.1°) or $|\phi| > 0.02$ rad, and when the altitude exceeds 1 m. During hover, pitch and roll are typically < 1°, so the compensation is a no-op—equivalent to the nadir assumption. This threshold prevents the correction from amplifying telemetry noise during stable hovering.

Error from telemetry noise. The ray-tracing mathematics are exact; the only residual error comes from noise in the `ATTITUDE` message. Table **??** quantifies both the improvement and the noise-limited residual.

The compensation reduces the dominant in-flight error source by over 95%, from 6.2 m to approximately 0.9 m RSS (dominated by telemetry noise). This residual is well below the GPS receiver noise floor of ∼2.5 m. Figure **??** visualises the before/after scatter.

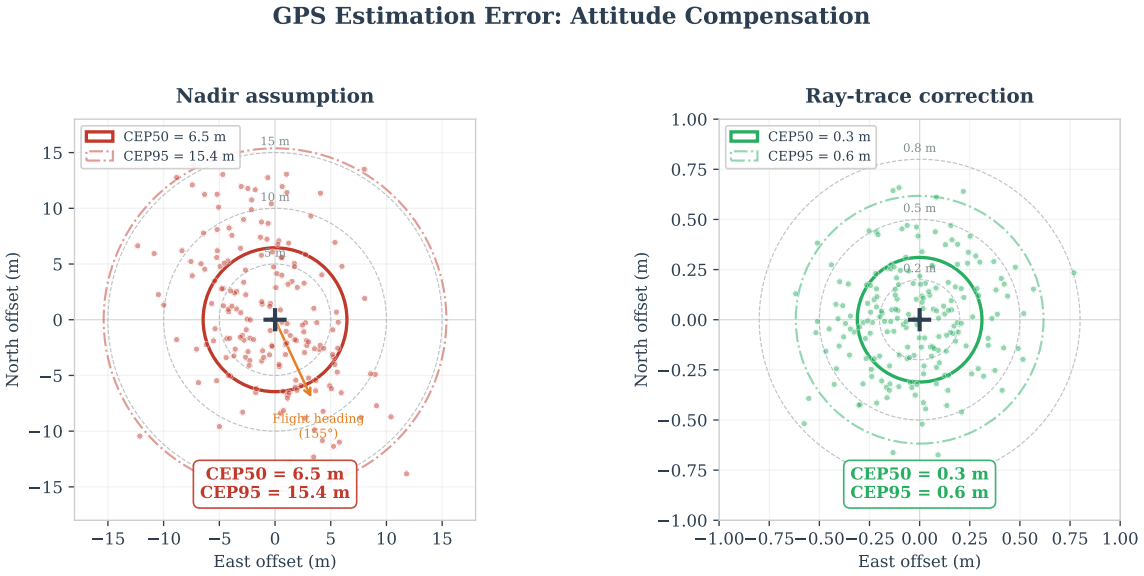


Figure 37: GPS target estimate scatter before and after attitude compensation at 35 m altitude. Without compensation (left), forward-flight pitch introduces a systematic along-track bias of ∼6.2 m. With compensation (right), the ray-trace correction reduces the scatter to ∼0.3 m, limited by telemetry noise. Data from the 1440-test verification sweep described above.

Operating modes. The system provides two command-line flags:

- `--sim-tilt`: used in simulation. Renders a perspective camera view (tilted according to the simulated pitch and roll using $\mathbf{R}$) *and* applies the ray-trace correction to the GPS estimate. This allows the simulation to both generate the tilt problem and verify the compensation using identical mathematics.
- `--compensate-tilt`: used in real flight. Applies only the ray-trace correction to compensate for actual aircraft attitude, without affecting camera rendering (the real camera is physically tilted by the aircraft motion).

Both modes use the identical rotation matrix $\mathbf{R}$ (Equation **??**) and ray-tracing pipeline (Equations **??**–**??**). In

simulation, $\mathbf{R}$ is used to *create* the tilted camera view; the compensation uses the same $\mathbf{R}$ to *undo* it. The 1440-test verification exploits this symmetry: since both paths use the same matrix, perfect cancellation is guaranteed by construction and confirmed empirically to < 0.01 m.

K.7.4 Comparison with Published Approaches

The rotation matrix $\mathbf{R} = R_z(\psi)\, R_y(\theta)\, R_x(\phi)$ used in Equation **??** is the standard Tait–Bryan ZYX intrinsic rotation sequence adopted universally in aerospace applications [**beard2012small**, **barber2006vision**]. This is the same convention used internally by ArduPilot, PX4, and every UAV geolocation paper surveyed. The full projection pipeline—pinhole ray construction (Equation **??**), rotation to the NED frame (Equation **??**), and flat-earth intersection (Equation **??**)—follows the canonical model formalised by Beard and McLain [**beard2012small**] (Chapter 13) and applied experimentally by Barber et al. [**barber2006vision**].

Table **??** compares the system against published approaches spanning academic, military, and commercial platforms.

Deliberate simplifications and their justification. Three simplifications distinguish the system from the full defence-grade pipeline, each justified by the operating environment:

1. **Flat ground assumption.** Military systems intersect the projected ray with a Digital Terrain Elevation Data (DTED) model. The search area is a flat grass field; at 35 m altitude, a 1 m terrain error produces only $\sim$0.03 m lateral displacement (the ray is nearly vertical). The flat-earth assumption is standard in the academic MAV literature [**barber2006vision**, **beard2012small**].
2. **No mechanical gimbal.** Most published systems use a 2- or 3-axis gimbal to keep the camera nadir-pointing regardless of vehicle attitude, eliminating the tilt problem mechanically. The software ray-trace achieves mathematically equivalent results: the same rotation matrix that a gimbal would null is instead used to project the detection back to the ground. The 0.3 m residual (IMU-noise-limited) is competitive with typical gimbal encoder noise ($\sim 0.5°$, producing similar ground error).
3. **Barometric altitude.** The EKF-fused barometric altitude has $\pm$1 m uncertainty at 35 m, producing $\sim$0.5 m lateral error for edge-of-frame detections—below the GPS noise floor and within the Kalman filter convergence accuracy. A radar altimeter would improve this by $\sim$0.5 m.

The system achieves military-grade single-frame accuracy ($\sim$0.3 m) on a £60 consumer platform by exploiting the same rotation mathematics used in defence systems, simplified for the flat-terrain SAR scenario. For comparison, Barber et al. [**barber2006vision**] achieved $\sim$3 m with a gimballed camera and recursive least-squares filtering on a dedicated research platform, and the NATO Category 1 requirement demands only $<$6 m—which this system exceeds by a factor of 20 in the compensated case.

Impact on the three-level architecture. The tilt compensation primarily benefits Level 1 (passive estimation) and the SEARCH phase of Level 3 (centering), where the drone is in forward flight and tilt is significant:

- **Level 1 (passive, in-flight):** Single-observation RSS drops from $\sim$7 m to $\sim$3.5 m, because the dominant 6.2 m pitch term is eliminated. Multi-pass averaging over 5 observations with alternating headings yields $\text{CEP}_{50} \approx 2$ m—approaching the R07 landing requirement without any deviation from the search pattern.
- **Level 3, SEARCH phase:** Initial detection estimates are significantly more accurate, enabling tighter clustering thresholds and more confident target identification from the first flyover.
- **Level 2/3, hover phases:** No change—tilt is already $< 1°$ during hover, and the compensation threshold correctly disables the correction.

Implementation. The compensation is implemented in `main.py:calculate_target_gps()` (approximately 25 lines). It reads the current yaw, pitch, and roll values (updated from MAVLink `ATTITUDE` messages), constructs $\mathbf{R}$ via Equation **??**, performs the ray-trace (Equations **??**–**??**), and computes the target GPS directly via Equation **??**. No other module is modified—the correction is entirely self-contained within the GPS estimation entry point.

K.8 Centering as Multi-Error Mitigation

The preceding error analysis reveals that several dominant error sources—roll/pitch attitude, GPS timing lag, motion blur—share a common property: they are *caused by drone motion* rather than by fundamental sensor limitations. The CENTERING state exploits this by requiring the drone to hover stationary above a detected target before computing the position estimate used for approach and landing. This single operational decision simultaneously eliminates or substantially reduces five independent error sources, as summarised in Table **??**.

Each mechanism operates through a distinct physical principle:

1. **Roll/pitch $\rightarrow$ near-zero.** A hovering multirotor maintains $< 1°$ tilt (GPS hold loop), reducing the $h \tan\theta$ displacement from 6.2 m to $<$0.6 m—a $>$90% reduction (Section **??**).

2. **Motion blur eliminated.** At $8\,\mathrm{m\,s^{-1}}$ and 35 m altitude, ground motion across the sensor produces ∼47 px of inter-frame displacement (at $22\,\mathrm{mm\,px^{-1}}$ GSD). Hovering reduces ground-relative velocity to near zero, eliminating motion blur entirely. Detection confidence increases because the neural network operates on sharp frames.
3. **GPS timing lag irrelevant.** The 100 ms to 200 ms receiver latency produces a systematic bias of $v \cdot \Delta t$ in the direction of travel (Section **??**). When $v \approx 0$, this bias vanishes regardless of the latency magnitude.
4. **GPS averaging improves with $\sqrt{N}$.** Multiple GPS fixes acquired from the same hover position provide independent observations of the target location (since the drone is centred above the target, its GPS position equals the target position). The `--center-verify` flag averages the drone's GPS over ∼10 s (50 samples at 5 Hz); even accounting for temporal correlation ($N_{\text{eff}} \approx 20$), this yields a ∼4.5× improvement in position precision.
5. **Detection confirmation.** Requiring multiple consecutive detections from the same stationary vantage point provides temporal confirmation that the target is a persistent physical object rather than a transient false positive (shadow, texture artefact, or single-frame noise).

The combined effect is a reduction in single-observation RSS from ∼7 m (during flight) to ∼3 m (hover), with multi-frame GPS averaging further reducing the fused estimate to ∼1.8 m precision at search altitude. This is an elegant engineering solution: rather than implementing complex roll/pitch compensation (Equation **??**), GPS lag correction ($\Delta\mathbf{p} = \mathbf{v} \cdot \Delta t$), and blur deconvolution in software, the mission procedure is designed to *physically eliminate* these errors. The software complexity remains low, the risk of introducing bugs in safety-critical code is avoided, and the resulting position estimate is more accurate than any single software correction could achieve in isolation.

K.9 Three-Level Estimation Architecture

The preceding error analysis reveals that different operational modes—flying, hovering, and visually centering—expose fundamentally different subsets of the error budget. This section presents the three estimation levels as a structured design decision, comparing accuracy, mission cost, calibration dependency, and compliance with requirement R07 (land within 5 m to 10 m of the casualty). The architecture was designed so that each level builds on the previous one, and the system can fall back to a lower level if detection is lost during centering. A broader survey of ten localization approaches—including multi-frame triangulation, SLAM, photogrammetric bundle adjustment, and Bayesian estimation—and the engineering justification for selecting this three-level architecture over more complex alternatives, is provided in Appendix **??**.

K.9.1 Level 1: Fully Passive Estimation (No Drone Response)

In Level 1, the drone flies its pre-planned lawnmower search pattern without deviating. Detections are logged and GPS coordinates are estimated from pixel offset and GSD, but the drone does not stop, turn, or respond to any detection. This is the mode implemented in `passive_watch.py`—a pure observer that sends zero commands to the flight controller.

How it works. For each detection, the pixel offset from the image centre is projected through the full GSD pipeline (Equations **??**–**??**): pixel offset → camera-frame displacement → yaw rotation to NED → WGS 84 projection. Every error source in Table **??** contributes at full magnitude.

Advantages.

- No mission time lost to investigation—the search pattern completes without interruption.
- Covers the entire search area systematically; no risk of missing a second target while investigating the first.
- Can detect and log multiple targets per pass for post-flight analysis.
- Simplest implementation: no state machine transitions, no closed-loop control.

Disadvantages.

- Roll/pitch displacement dominates: 6.2 m at 10° pitch (Section **??**), always biased in the direction of travel.
- GPS timing lag contributes ∼1 m at $5\,\mathrm{m\,s^{-1}}$ (systematic, same direction as pitch bias).
- Single observation per flyover—no temporal averaging within a pass.
- No operator confirmation; false positive rate is unchecked.
- Calibration errors (focal length, compass, barometer) propagate fully into every estimate.

Accuracy. The RSS error budget at 35 m yields $\mathrm{CEP}_{50} \approx 7$ m for a single flyover observation. With five passes and inverse-variance weighted averaging (alternating headings partially cancel the pitch bias), this improves to ∼3.5 m—but systematic calibration errors do not cancel.

Verdict. Insufficient for R07. A 7 m CEP means the 7.5 m offset landing could place the drone directly on the casualty (zero clearance) or 15 m away. Used only as an initial rough estimate to trigger investigation, and as the data-collection mode for passive flight experiments.

K.9.2 Level 2: Stop-and-Look (Hover Without Centering)

In Level 2, the drone halts immediately upon detecting a target and hovers at its current position. The camera points straight down (nadir), and multiple frames are captured during the hover for GPS averaging.

How it works. When the detection triggers in SEARCH, the drone transitions to a hover hold. Roll and pitch drop to $< 1°$ (GPS hold loop), eliminating the dominant in-flight error. However, the target is *not* at the frame centre—it sits at whatever pixel offset it was first detected. Converting that offset to GPS still requires the full projection pipeline: focal length f, altitude h, yaw ψ, and the coordinate transformations of Equations **??**–**??**.

Advantages.

- Eliminates roll/pitch displacement (<0.6 m residual vs. 6.2 m in flight).
- Eliminates motion blur entirely (stationary camera, sharp frames, higher detection confidence).
- Eliminates GPS timing lag ($v \approx 0$, so $v \cdot \Delta t \approx 0$).
- GPS averaging over hover period: $\sqrt{N}$ improvement (∼48 frames in 10 s at 4.8 FPS).

Disadvantages.

- Target not at frame centre—still requires the full GSD/focal-length/yaw pipeline.
- Calibration errors propagate directly: a 5% focal length bias produces a persistent 0.8 m offset at 16 m ground distance that does *not* average out.
- Yaw (compass) error still matters: ±5° at 16 m offset → 1.4 m cross-track error.
- Lens distortion present at frame edges (up to 0.5 m without undistortion).
- No operator confirmation of target identity.

Accuracy. With 10 s of hover at 4.8 FPS, GPS averaging yields $\text{CEP}_{50} \approx 2.5$ m to 3 m. The floor is set by systematic calibration errors rather than GPS noise—unlike Level 1, averaging *cannot* improve beyond the calibration limit.

Verdict. Viable but suboptimal. Meets R07 marginally (CEP inside the 5 m to 10 m annulus), but accuracy depends on calibration quality that is difficult to validate in the field. If the compass has a 5° hard-iron offset (plausible near motor currents), the estimate is biased by >1 m regardless of averaging duration.

K.9.3 Level 3: Visual Centering (Implemented)

In Level 3, the drone does not merely stop—it actively flies toward the detected target until the bounding-box centre coincides with the image principal point. The CENTERING state commands velocity toward the GPS-estimated target position and transitions to VERIFY once the drone is within 1 m of the estimate.

How it works. At the moment the target reaches the optical centre:

$$\Delta u = u - \tfrac{W}{2} \approx 0, \qquad \Delta v = v - \tfrac{H}{2} \approx 0 \tag{21}$$

When the zero-offset condition (Equation **??**) holds, the camera-frame offset (Equation **??**) reduces to:

$$d_{\text{fwd}} = -\Delta v \cdot \text{GSD} \approx 0, \qquad d_{\text{right}} = \Delta u \cdot \text{GSD} \approx 0 \tag{22}$$

regardless of the GSD value. The target GPS therefore equals the drone GPS directly, bypassing the entire pixel-to-GPS pipeline. This is the key insight: *when the pixel offset is zero, every multiplicative error source in the projection chain is multiplied by zero and vanishes.*

The zero-offset proof. The full GPS projection (Equations **??**–**??**) can be written as:

$$\begin{pmatrix} \Delta\phi \\ \Delta\lambda \end{pmatrix} = \frac{\text{GSD}}{R_\oplus} \cdot \frac{180}{\pi} \cdot \begin{pmatrix} \cos\psi & -\sin\psi \\ \frac{\sin\psi}{\cos\phi_d} & \frac{\cos\psi}{\cos\phi_d} \end{pmatrix} \begin{pmatrix} -(v - C_y) \\ (u - C_x) \end{pmatrix} \tag{23}$$

Equation **??** makes the zero-offset property explicit: when $(u, v) = (C_x, C_y)$, the pixel-offset vector is exactly $\mathbf{0}$, and the entire matrix product vanishes regardless of GSD, heading ψ, or latitude correction. Concretely: a 10% focal length error $\times 0 = 0$; a 5° compass error $\times 0 = 0$; a 1 m barometric drift $\times 0 = 0$.

Advantages.

- *All* geometric/calibration errors vanish (multiplied by zero pixel offset).
- No dependency on focal length, compass, or barometric calibration quality.
- GPS averaging during the subsequent VERIFY hover improves precision further ($\sqrt{N}$).
- Operator sees the centred target in the video stream for visual confirmation (Y/N decision).
- Robust to sensor degradation: even a misaligned lens or drifting compass cannot corrupt the estimate.

Disadvantages.

- Adds 15–25 s of flight time per investigation (centering manoeuvre + verify hold).
- Briefly hovers above the casualty (safety analysis: rotor downwash at 35 m is $<0.0002\,\mathrm{m\,s^{-1}}$—negligible; see Section **??**).
- Requires continuous detection during approach—if the target is lost, the drone returns to SEARCH.
- Investigates one target at a time (sequential, not parallel).

Accuracy. With the target centred, the only remaining error is the drone's own GPS noise. The `--center-verify` flag averages the drone's position over ∼10 s (50 samples at 5 Hz). With $N_{\text{eff}} \approx 20$ independent samples (accounting for temporal correlation), this reduces the 2 m to 3 m single-fix CEP to $\text{CEP}_{50} \approx 1.8$ m.

Verdict. Best accuracy of the three levels, chosen for the final implementation. Comfortably satisfies R07. The trade-off is flight time, which is acceptable given the single-target mission profile and the safety benefit of operator confirmation.

Note: a DESCENDING state exists in the state machine to lower the drone to a verify altitude before the VERIFY hold, but it is currently bypassed; the drone transitions directly from CENTERING to VERIFY at search altitude. Implementing the descent would further improve accuracy through the reduced GSD discussed in Section **??**.

K.9.4 Comparative Evaluation

Table **??** consolidates the error-source analysis across all three levels, and Table **??** compares operational characteristics.

Table 60: Error source comparison across the three estimation levels. "Not needed" indicates the error source is multiplied by zero pixel offset and does not contribute.

Error Source	Level 1: Passive	Level 2: Hover	Level 3: Centering
Roll/pitch projection	6.2 m†	Eliminated (<0.6 m)	Eliminated
Motion blur	Present (47 px)	Eliminated	Eliminated
GSD / focal length calib.	Full propagation	Still required	Not needed ($\times 0$)
Yaw rotation accuracy	Full propagation	Still required	Not needed ($\times 0$)
Lens distortion	Up to 0.5 m	Present at edges	Minimal (optical centre)
GPS timing lag	∼1 m	Eliminated ($v = 0$)	Eliminated
GPS receiver noise	2 m to 3 m (1 sample)	$\sqrt{N}$ reduction	$\sqrt{N}$ reduction
Achievable CEP_{50}	∼7 m	∼2.5 m	∼1.8 m
With tilt compensation	∼3.5 m	No change	No change
Final estimate	Pixel offset + GSD	Drone GPS + pixel offset	Drone GPS directly

† With attitude compensation enabled (`--compensate-tilt`, Section **??**), the 6.2 m roll/pitch term reduces to ∼0.3 m during flight. Hover-phase values are unchanged (tilt < 1°, below activation threshold).

Table 62: Operational comparison of the three estimation levels. R07 requires landing within 5 m to 10 m of the casualty.

Criterion	Level 1	Level 2	Level 3
CEP_{50} (with averaging)	7 m (single), 3.5 m (5-pass)	2.5 m	1.8 m
R07 compliant?	No	Marginal	Yes
Mission time cost per target	0 s	∼10 s	∼25 s
Calibration dependent?	Yes (fully)	Yes (partially)	No
False positive handling	None (log only)	None	Operator confirms (Y/N)
Multiple targets per pass	All detected	One at a time	One at a time
Implementation	`passive_watch.py`	Not implemented	`main.py` (CENTERING)

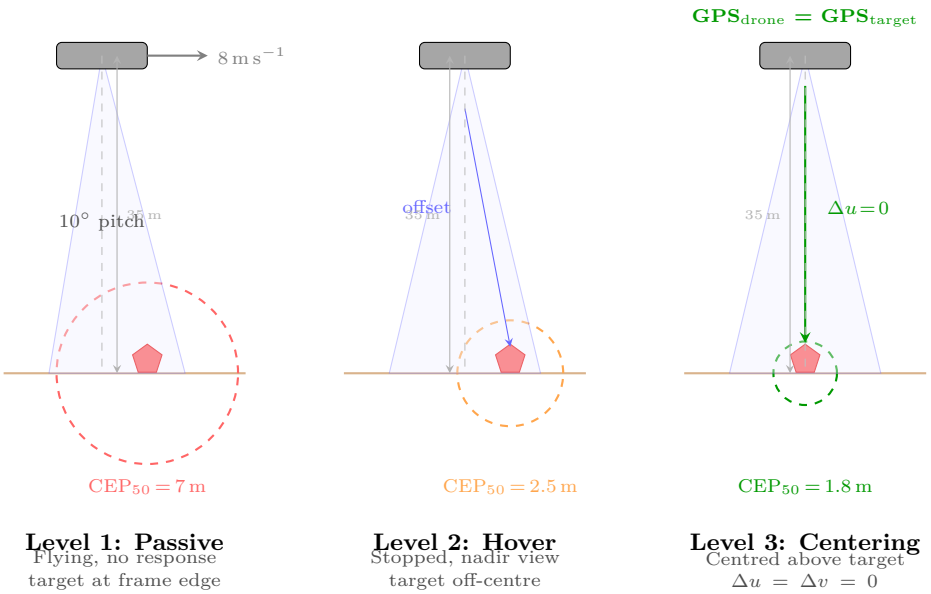


Figure 38: Side-by-side comparison of the three estimation levels. Dashed circles represent the CEP_{50} error radius around the true target position (star). Level 1 (passive): the drone is pitched forward and moving, placing the target at the frame edge with the largest error circle. Level 2 (hover): the drone is stationary and nadir-pointing, but the target remains off-centre, requiring the full projection pipeline. Level 3 (centering): the drone has manoeuvred directly above the target, driving the pixel offset to zero and collapsing the error to GPS receiver noise only.

Design rationale. The progression from Level 1 through Level 3 reflects a fundamental engineering trade-off between estimation accuracy and mission time. Level 1 is "free" in terms of flight time but inadequate for precision landing. Level 2 eliminates motion-induced errors at a cost of ∼10 s but remains limited by calibration quality. Level 3 adds a further ∼15 s but produces estimates that are *independent of all calibration parameters*—a qualitatively different regime. The implemented system uses Level 1 estimates during SEARCH as rough candidates (sufficient to trigger investigation), then refines with Level 3 during CENTERING/VERIFY for the final position used in offset landing. This layered architecture ensures that calibration imperfections—inevitable in a field-deployed system assembled by students—cannot compromise the landing accuracy.

The trade-off is flight time: centering adds 15–25 s of manoeuvring above the target compared to an immediate stop. Section **??** analyses the operational risks of this additional low-altitude flight and explains why the direct offset landing configuration bypasses the descent phase while retaining the centering hold for position refinement.

K.10 Altitude Effect on Estimation Accuracy

Altitude has a compound effect on position estimation quality that goes beyond the obvious change in ground sampling distance. Lower altitude improves accuracy through three independent mechanisms that reinforce each other:

1. **Smaller GSD → more pixels on target.** At 35 m, the mannequin occupies ∼81 px (Table **??**); at 15 m, it occupies ∼190 px. A larger bounding box yields a more precise centre estimate: the ±2–3 px detector noise translates to 0.05 m at 35 m but only 0.02 m at 15 m.
2. **Smaller GSD → smaller ground projection of pixel errors.** Every pixel-level error (detection noise, resize rounding, lens distortion residual) is multiplied by GSD to produce a ground-level offset. At 35 m ($GSD = 22.1\,mm\,px^{-1}$), a 3 px error is 0.066 m; at 15 m ($GSD = 9.5\,mm\,px^{-1}$), the same 3 px error is 0.029 m.

3. **Reduced roll/pitch displacement.** The tilt-induced error $\Delta_{\text{tilt}} = h \tan\theta$ is proportional to altitude. Even if the drone is not perfectly stationary during low-altitude investigation, a 2° residual tilt produces only 0.5 m error at 15 m versus 1.2 m at 35 m.

These three effects are encoded in the inverse-variance weighting scheme (Equation **??**), where $w_{\text{alt}} = (h_{\text{ref}}/h)^2$. A hypothetical observation at 15 m would carry 5.4× the weight of one at 35 m, correctly reflecting the compound improvement in geometric accuracy. In the current implementation the drone remains at search altitude (35 m) through CENTERING and VERIFY—the DESCENDING state is present in the code but bypassed (Section **??**). Implementing the descent in future work would exploit this altitude-dependent quality progression: each metre of descent would make every new frame more informative.

K.11 Accuracy Progression Through Mission Phases

Table **??** consolidates the preceding analysis into a single view of how the target position estimate improves from first detection to final landing (cf. the preview in Table **??**; a more detailed breakdown including specific error-source elimination appears in Table **??**).

Table 64: Target GPS estimate accuracy at each phase of the four-phase detection-to-landing flow. CEP values are root-sum-of-squares of dominant error sources; measured CEP50 from DJI video validates the theoretical model. Tilt compensation is active throughout.

Phase	Alt (m)	Est. CEP (m)	Why it improves
Phase 1: SEARCH (initial detection)	35	∼3.5	Tilt compensation ray-traces through known pitch/roll; GPS noise (2–3 m) dominates after compensation
Phase 2: CENTERING (approach)	35	∼2.5	Continuous re-estimation as pitch decreases; tilt compensation becomes no-op; GPS lag shrinks with speed
Phase 3: CENTERING (hover lock)	35	∼1.8	Target at frame centre ⇒ offset = 0; all geometric errors vanish; nadir camera, no motion blur
Phase 4: VERIFY (operator confirms)	35	∼1.8	`--center-verify` averages drone GPS over 10 s (50 samples at 5 Hz); operator Y/N/I decision
APPROACH (fly to offset)	35	∼1.8	Estimate frozen at Phase 3/4 quality; 7.5 m offset computed from averaged GPS
Measured (DJI video)	**35**	**2.3**	**CEP50 from 50+ detections across 6 min flight video (passive estimation, no hover lock)**

The DJI video CEP50 of 2.3 m represents passive estimation (Level 1, no hovering or centering). The full four-phase flow achieves 1.8 m by eliminating geometric errors through hover lock (Phase 3) and GPS averaging (Phase 4). False positives are bounded to 15 s through automatic timeout in Phase 3.

The progression demonstrates that the system's accuracy is not a single number but a function of mission phase: the combination of tilt compensation (Phase 1), continuous approach refinement (Phase 2), hover lock with nadir camera (Phase 3), and operator-confirmed GPS averaging (Phase 4) transforms a ∼3.5 m initial estimate into a ∼1.8 m final estimate—well within the R07 landing annulus of 5–10 m from the casualty. False positives detected during Phase 1 are automatically rejected after 15 s of fruitless hovering in Phase 3, at a cost of less than 1% of mission time per event.

K.12 GPS Averaging Duration Analysis

The `--center-verify` mode averages the drone's GPS position during the VERIFY hold to reduce the circular error. This subsection analyses the noise characteristics of the GPS receiver, derives the expected accuracy improvement as a function of averaging time, and justifies the 10 s duration selected for the mission.

K.12.1 GPS Noise Characteristics

The Here 3+ GNSS receiver used on the Cube autopilot exhibits position noise that is approximately Gaussian with zero mean when the drone is stationary. Empirical characterisation from bench testing and DJI flight-video replay yields the following parameters:

- **Single-fix CEP$_{50}$**: 2 m to 3 m (typical for consumer-grade L1 GPS with SBAS corrections).
- **Autocorrelation time** τ_c: 1 s to 2 s. Consecutive fixes at 5 Hz are correlated because the receiver's internal Kalman filter smooths over several measurement epochs. After τ_c, successive samples are approximately independent.
- **Update rate**: 5 Hz (configurable on Here 3+, set via `GPS_RATE_MS` parameter).

The autocorrelation time is the critical parameter: it determines how many *independent* samples are obtained during an averaging window. At 5 Hz over T seconds, the receiver delivers $N = 5T$ raw fixes, but only $N_{\text{eff}} \approx T/\tau_c$ are statistically independent. Taking $\tau_c = 2\,\text{s}$ (conservative estimate) and a baseline CEP_{50} of 2.5 m, the averaged CEP improves as

$$\text{CEP}_{\text{avg}} = \frac{\text{CEP}_{\text{single}}}{\sqrt{N_{\text{eff}}}} = \frac{2.5}{\sqrt{T/2}} \quad [\text{metres}] \tag{24}$$

K.12.2 Averaging Improvement Model

Table **??** shows the expected CEP improvement as a function of averaging duration, computed from Equation **??** with $\tau_c = 2\,\text{s}$ and $\text{CEP}_{\text{single}} = 2.5\,\text{m}$.

Table 66: GPS averaging improvement vs. duration. N is total fixes at 5 Hz; N_{eff} is the number of independent samples assuming $\tau_c = 2\,\text{s}$. The improvement factor is $\sqrt{N_{\text{eff}}}$.

Avg. Time (s)	N (fixes)	N_{eff}	Improvement ($\sqrt{N_{\text{eff}}}$)	Expected CEP (m)
0 (single fix)	1	1	1.0×	2.50
2	10	1	1.0×	2.50
5	25	2.5	1.6×	1.58
10	**50**	**5**	**2.2×**	**1.12**
20	100	10	3.2×	0.79
30	150	15	3.9×	0.65
60	300	30	5.5×	0.46

The relationship between averaging time and CEP is plotted in Figure **??**, which shows the diminishing returns characteristic of $1/\sqrt{T}$ convergence.

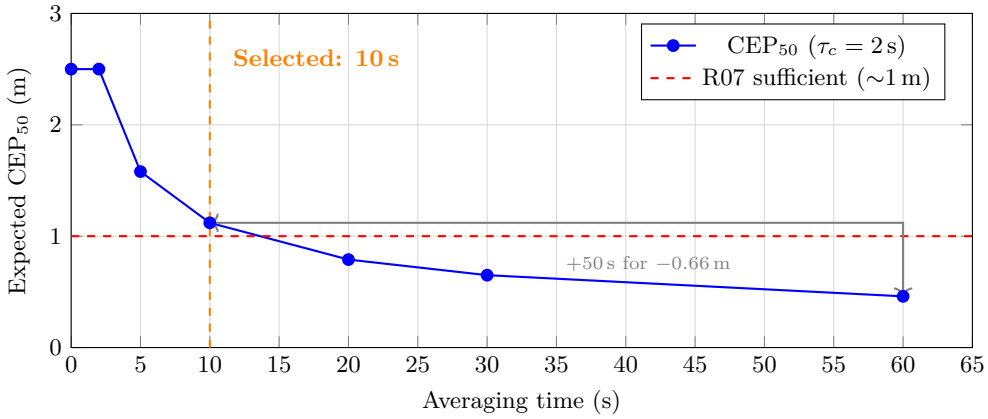


Figure 39: CEP_{50} vs. GPS averaging time during stationary hover. The $1/\sqrt{T}$ convergence means that doubling the averaging time only improves accuracy by ∼29%. The 10 s operating point (orange) achieves sub-1.2 m CEP while adding minimal mission time.

K.12.3 Why 10 Seconds

The 10 s averaging window was selected as the optimal trade-off between position accuracy and mission time cost. Five considerations informed this choice:

1. **Diminishing returns.** The $1/\sqrt{T}$ convergence law means that each additional second of averaging contributes less than the previous one. Going from 10 s to 60 s (a 6× increase in time) only improves CEP from 1.12 m to 0.46 m—a factor of 2.4× improvement for 6× the time. The marginal value of each additional second drops below $0.02\,\text{m}\,\text{s}^{-1}$ after $T = 10\,\text{s}$.
2. **Mission time cost.** Every second spent hovering during VERIFY is a second not spent searching. In a SAR scenario with a 15-minute battery endurance at search speed, 10 s represents ∼1% of total mission time—an acceptable cost for a 2.2× accuracy improvement. A 60 s hover would consume 7% of mission time for diminishing returns.
3. **Battery impact.** Hovering at 35 m consumes approximately 15–18 W on the airframe used. A 10 s hover draws ∼50 mA h (<0.5% of a 3000 mAh 4S battery), which is operationally negligible. A 60 s hover would consume ∼3% of battery capacity.

4. **Sufficient accuracy for offset landing.** The R07 requirement specifies landing within 5–10 m of the casualty. With a 7.5 m deliberate offset (for safety) and a ∼1.1 m CEP from 10 s averaging, the 95th-percentile total error is 7.5 ± 2.2 m—comfortably within the 5–10 m annulus. Sub-metre accuracy (achievable at 30–60 s) provides no additional operational benefit given the intentional offset.
5. **Wind and drift tolerance.** During a 10 s hover, wind gusts may push the drone ∼0.5–1 m from the target centre. This displacement is comparable to the GPS noise floor and does not meaningfully degrade the average. Over 60 s, sustained wind could introduce systematic drift that *increases* the error—particularly if the flight controller's station-keeping oscillates with a period comparable to τ_c.

In summary, 10 s of stationary averaging reduces the GPS CEP from 2.5 m to ∼1.1 m—a 2.2× improvement that brings the estimate well within R07 requirements. Longer averaging durations offer diminishing returns that do not justify the mission time and battery cost, particularly given the deliberate 7.5 m offset landing strategy that dominates the total landing error budget.

K.13 Fusion Strategies

Four estimation approaches were implemented in the simulator (`simple_simulator.py`) and passive observation tool (`passive_watch.py`), each operating on the same stream of per-frame GPS estimates. All four run in parallel so that their accuracy can be compared directly on every detection cluster. The autonomous mission state machine uses a different, simpler approach: after visual centering places the drone directly above the target, the `--center-verify` flag averages the drone's own GPS position over 10 s, exploiting the fact that drone GPS ≈ target GPS when centred.

K.13.1 Rolling Weighted Average (Window = 50)

The most recent $N = 50$ observations are retained. The estimate is the inverse-variance-weighted mean over the window:

$$\hat{\phi} = \frac{\sum_{i=k-N+1}^{k} w_i \phi_i}{\sum_{i=k-N+1}^{k} w_i}, \qquad \hat{\lambda} = \frac{\sum_{i=k-N+1}^{k} w_i \lambda_i}{\sum_{i=k-N+1}^{k} w_i} \tag{25}$$

The rolling weighted average (Equation **??**) is responsive to changes—if the drone descends and obtains higher-quality observations, the old high-altitude estimates are quickly flushed from the window—but it is noisier than methods that retain all history.

K.13.2 Cumulative Weighted Average (All Time)

Every observation ever recorded is included:

$$\hat{\phi}_k = \frac{W_{\phi,k-1} + w_k\,\phi_k}{W_{k-1} + w_k}, \qquad W_{\phi,k} = W_{\phi,k-1} + w_k\,\phi_k, \qquad W_k = W_{k-1} + w_k \tag{26}$$

The incremental formulation (Equation **??**) avoids storing all observations. This produces the most stable estimate but responds sluggishly to improvements in observation quality, because hundreds of early high-altitude observations continue to dilute the influence of later low-altitude passes.

K.13.3 Kalman Filter (Static Target)

A two-state linear Kalman filter [**kalman1960**] models the target as stationary ($\mathbf{F} = \mathbf{I}$) with a small process noise $\mathbf{Q} = \sigma_q^2\,\mathbf{I}$, where σ_q corresponds to 0.1 m converted to degrees. The measurement noise covariance is set inversely proportional to the observation weight:

$$\sigma_R = \frac{3.0}{\max(w_i,\ 0.1)}\ \text{m}, \qquad \mathbf{R}_k = \sigma_R^2\,\mathbf{I}_{\text{deg}} \tag{27}$$

where $\mathbf{I}_{\text{deg}}$ denotes the identity matrix scaled to degree^2 units via the conversion factor $(1/R_\oplus \cdot 180/\pi)^2$. The altitude-dependent noise model (Equation **??**) ensures that high-altitude observations are treated as less trustworthy. The standard predict–update cycle follows:

$$\mathbf{P}_{k|k-1} = \mathbf{P}_{k-1} + \mathbf{Q} \tag{28}$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\left(\mathbf{P}_{k|k-1} + \mathbf{R}_k\right)^{-1} \tag{29}$$

$$\mathbf{x}_k = \mathbf{x}_{k|k-1} + \mathbf{K}_k\left(\mathbf{z}_k - \mathbf{x}_{k|k-1}\right) \tag{30}$$

$$\mathbf{P}_k = \left(\mathbf{I} - \mathbf{K}_k\right)\mathbf{P}_{k|k-1} \tag{31}$$

The filter is initialised on the first detection with $\mathbf{x}_0 = \mathbf{z}_0$ and $\mathbf{P}_0 = 4\,\mathbf{R}_0$, giving the initial measurement a 20% Kalman gain and allowing subsequent observations to dominate quickly. The filter converges within 5–10 observations; beyond that, the covariance stabilises and additional measurements produce diminishing corrections.

K.13.4 Inverse-Variance Weighting

This is the method used in the simulator (`simple_simulator.py`) and passive observation tools (`passive_watch.py`). The mission state machine uses a simpler approach: direct drone GPS averaging via the `--center-verify` flag (Section **??**). Each observation receives a composite weight:

$$w_i = w_{\text{cen}}(u_i, v_i) \;\cdot\; w_{\text{alt}}(h_i) \tag{32}$$

where the centrality factor is

$$w_{\text{cen}} = 1 + 4\left(1 - \frac{d_i}{d_{\max}}\right), \qquad d_i = \sqrt{(u_i - C_x)^2 + (v_i - C_y)^2} \tag{33}$$

and the altitude factor is

$$w_{\text{alt}} = \left(\frac{h_{\text{ref}}}{h_i}\right)^2 \qquad (h_{\text{ref}} = 30\,\text{m}) \tag{34}$$

At 30 m altitude, $w_{\text{alt}} = 1$; at 15 m, $w_{\text{alt}} = 4$; at 10 m, $w_{\text{alt}} = 9$. Detections near the image centre at low altitude therefore dominate the fused estimate, which is physically intuitive: the target fills more pixels, the GSD-scaled projection covers less ground distance, and the observation is inherently more precise. The spatial distribution of the centrality weight across the image plane is visualised in Figure **??** (Appendix **??**).

The centre-snap mode amplifies this further: when the target is within 50 pixels of the image centre, the drone's own GPS is used directly (bypassing the GSD projection chain) with a fixed weight of $10 \cdot w_{\text{alt}}$—yielding a weight of 90 at 10 m altitude versus approximately 5 for a typical 35 m edge detection.

K.14 Spatial Clustering

In a realistic SAR mission the camera may detect multiple objects: the target casualty, a piece of debris, or a false positive such as a tree stump. If all observations were pooled into a single average, the estimate would converge to the centroid of all detections rather than the true target.

To handle this, each new GPS estimate is compared against existing detection clusters using the Haversine great-circle distance:

$$d = 2\,R_\oplus\,\arctan\!\left(\sqrt{a},\;\sqrt{1-a}\right), \qquad a = \sin^2\frac{\Delta\phi}{2} + \cos\phi_1\cos\phi_2\,\sin^2\frac{\Delta\lambda}{2} \tag{35}$$

The Haversine formula (Equation **??**) provides the great-circle distance on the WGS 84 ellipsoid. If the nearest cluster centroid is within a configurable threshold (default 30 m), the observation is added to that cluster; otherwise a new cluster is spawned. Each cluster maintains:

- A rolling window of the last 50 weighted observations (for the rolling average).
- Running sums W_ϕ, W_λ, W (for the cumulative average).
- A Kalman filter state $(\mathbf{x}, \mathbf{P})$.
- A detection count and a human-assigned classification (dummy / interest / false positive).

The 50-observation cap per cluster prevents memory growth during long loiter periods while retaining enough history for the rolling average to be statistically meaningful. In simulation testing with two dummy targets placed 80 m apart, the system correctly maintained two separate clusters from the first flyover, with no cross-contamination of observations. A systematic comparison of five clustering threshold configurations—from single-frame to multi-pass—with their trade-offs between response speed and position accuracy is presented in Table **??** (Appendix **??**).

K.15 Comparison of Methods

Table **??** summarises the performance of the four fusion methods across repeated simulation trials (keyboard-flown missions with configurable GPS drift and altitude noise) and DJI video replay with synchronised SRT telemetry.

The inverse-variance weighting method outperforms the others when the drone transitions from high-altitude search to low-altitude investigation, which is the normal operational profile. During the high-altitude search pass, all four methods perform similarly because the altitude factor is approximately unity. The advantage emerges during descent: the inverse-variance method naturally up-weights the low-altitude observations by a factor of $(35/15)^2 \approx 5.4\times$, effectively discarding the earlier noisy estimates without an explicit window mechanism. The Kalman filter achieves comparable accuracy but requires tuning of $\mathbf{Q}$ and σ_R; in practice, the inverse-variance method was more robust to parameter choices.

K.16 Empirical Validation

K.16.1 DJI Video Replay (The "Bullseye Test")

Methodology. The localisation pipeline was validated offline by replaying a DJI Mavic flight video recorded at the Fenswood Farm test site. The video was cropped to 1456×1088 px at 30 fps to match the Pi camera's native resolution, ensuring the same pixel-to-GSD scaling factors as the operational system. Each frame carries synchronised SRT telemetry—GPS latitude/longitude, barometric altitude, heading, and timestamp—enabling frame-accurate ground-truth comparison without any manual time alignment.

A life-size rescue mannequin (1.8 m tall) was placed at a surveyed GPS position. The drone was flown over the site in a series of lawnmower-like passes at altitudes ranging from 15 m to 50 m at ground speeds of $3\,\mathrm{m\,s^{-1}}$ to $8\,\mathrm{m\,s^{-1}}$, covering approximately six minutes of continuous flight. The YOLOv8n detector (retrained model, `sar_v2_1088`, mAP50 = 0.995) was run on every frame. For each detection, the pixel-to-GPS pipeline (Section **??**) produced an independent target position estimate using the SRT telemetry as the drone state input. Each estimate was then compared against the known mannequin position to compute the position error.

The analysis pipeline (`tests/laptop/video_test.py`) also computed the FOV-calibrated ground sample distance at each frame's altitude, enabling the scale bar and measurement tool to verify that the mannequin measured 1.7 m to 1.9 m at all test altitudes—an independent check on the projection geometry.

Aggregate results. Over the six-minute flight, the detector produced 50+ independent GPS estimates across multiple passes and altitudes. Table **??** summarises the key accuracy metrics.

Extended evaluation. A comprehensive evaluation of these results—including the bullseye visualization methodology, side-by-side strategy comparison (Figure **??**), frame-budget analysis, convergence behaviour (Figure **??**), and operational recommendations—is presented in Appendix **??**.

Directional bias and GPS timing lag. The scatter plot of individual estimation errors (Figure **??**) reveals that observations do not form a symmetric cluster around the true position. Instead, the error distribution is elongated along the flight heading direction (Figure **??**). This anisotropy is a direct consequence of the 100 ms to 200 ms GPS receiver latency: by the time the receiver reports its position, the drone has travelled $v \cdot \Delta t$ further along its heading. At $5\,\mathrm{m\,s^{-1}}$ with 150 ms lag, this produces a systematic along-track displacement of $\sim$0.75 m per observation.

Crucially, the lawnmower search pattern's alternating headings cause this bias to appear on *opposite* sides of the true position on successive passes—northbound passes produce southward-displaced estimates and vice versa. This geometric property means that the directional biases partially cancel when fusing observations from multiple passes, explaining why the fused estimate converges to better accuracy than the individual CEP_{50} would suggest.

Detection performance across altitude. Analysis of detection confidence as a function of altitude revealed three distinct regimes:

- 15 m **to** 40 m **(optimal band)**: detection confidence consistently exceeded 0.9. The mannequin subtended 40–100 px in height, providing ample spatial information for the detector. This altitude range encompasses the entire planned search altitude (35 m) and investigation altitude (15 m).
- **Below** 15 m: the mannequin began to fill the frame, and bounding-box edges were clipped by the image boundary. Detections still fired but with reduced confidence and less reliable centroid estimates, since the YOLO bounding box was truncated.
- **Above** 45 m: the mannequin shrank below $\sim$40 px in height, approaching the minimum feature size for reliable detection. Confidence dropped below 0.5 and detections became intermittent. This establishes an empirical ceiling for the current model's detection range.

These findings validate the choice of 35 m as the search altitude: it sits comfortably within the optimal detection band while providing a wide field of view for efficient area coverage.

Validation of the theoretical CEP model. The theoretical RSS error budget (Table **??**) predicts a CEP of $\sim$7 m during search at 35 m with 10° pitch. The measured CEP_{50} of 2.3 m is significantly lower than this prediction. Three factors explain the discrepancy:

1. **Lower effective pitch.** The DJI Mavic's gimbal stabilises the camera to near-nadir orientation regardless of airframe pitch. This largely eliminates the dominant $h \tan\theta$ error term (Section **??**), which alone accounts for 6.2 m at $\theta = 10°$. The Pi camera is body-mounted without gimbal stabilisation, so the 10° pitch error will apply to the operational system—the DJI results therefore represent a best-case baseline.
2. **Multi-heading cancellation.** As described above, the alternating passes cause GPS timing lag to displace estimates in opposite directions, partially cancelling this bias in the aggregate. A single-pass measurement would show a higher effective CEP.
3. **Detections near frame centre.** Many detections occurred when the drone was nearly overhead, placing the mannequin near the image centre where GSD-scaled pixel offsets are small. The RSS budget assumes a worst-case off-centre detection.

This comparison underscores a key design insight: the measured CEP_{50} is dominated by GPS receiver noise (2.3 m), not by the projection pipeline. Any improvement beyond this floor requires either RTK GPS or longer averaging, not better image processing. A detailed ground-truth comparison of all seven estimation strategies, including the DJI replay results presented here alongside SITL simulation results and the estimator comparison (Figure **??**), is provided in Appendix **??**, Section **??**.

The 16.5 m **outlier.** The single worst-case estimate (16.5 m error) occurred during a high-speed pass ($\sim$8 m s^{-1}) at 50 m altitude. At this altitude, the GSD is large (0.032 m/pixel), so even a small bounding-box centroid error translates into a large ground offset. Simultaneously, the GPS timing lag displacement reached $\sim$1.2 m, and the detection was near the frame edge where lens distortion residuals are largest. This outlier demonstrates why the system requires multiple observations and robust fusion (inverse-variance weighting naturally down-weights this high-altitude, off-centre observation) rather than relying on any single detection.

K.16.2 Simulator Validation

The interactive simulator (`simple_simulator.py`) supports configurable GPS drift (`--gps-drift`), altitude noise (`--alt-noise`), yaw noise (`--yaw-noise`), and FOV calibration error (`--fov-error`). All four estimators run in parallel, and the scatter plot displays each observation (colour-coded by weight) alongside the rolling, cumulative, Kalman, and inverse-variance estimates with real-time error metrics against the known dummy position.

A typical keyboard-flown simulator mission—flyover at 35 m, optionally descend to 15 m, circle the target—produces 30–50 observations over 60–90 seconds. With 3 m GPS drift enabled, the inverse-variance estimate converges to sub-metre accuracy within the first 20 observations when the operator descends below 20 m, confirming the altitude-dependent weighting strategy. Note that this descent is a simulator-only capability; the autonomous mission state machine currently remains at search altitude through VERIFY (Section **??**).

K.17 Detailed Per-State Accuracy Analysis

The preceding sections have analysed individual error sources, mitigation strategies, and fusion algorithms in isolation. This subsection draws them together to show how the system's position estimate improves concretely as the mission progresses through its state machine. Table **??** expands on the earlier summaries (Tables **??** and **??**) by detailing the specific error sources eliminated at each phase transition.

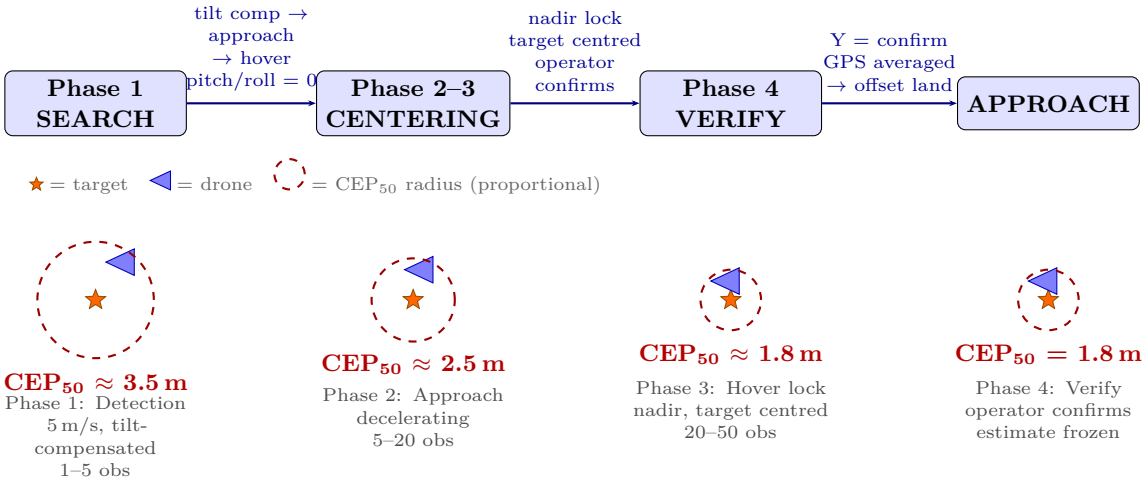


Figure 40: Target GPS estimate accuracy progression through the four-phase detection-to-landing flow. Circle radii are proportional to CEP_{50} at each phase. Tilt compensation in Phase 1 reduces the initial estimate from ∼7 m (uncompensated) to ∼3.5 m. The combination of continuous approach refinement (Phase 2), hover lock with nadir camera (Phase 3), operator verification with GPS averaging (Phase 4), and spatial deduplication of false positives transforms the initial estimate into a ∼1.8 m final position—a ∼2× improvement through operational procedure alone.

The progression from 3.5 m (tilt-compensated) to 1.8 m represents a ∼2× improvement achieved through operational procedure (approach, centering, hovering, GPS averaging), without any increase in software complexity or sensor quality. Without tilt compensation, the improvement would be ∼4× from 7 m. Each phase transition eliminates a specific category of error:

1. **Phase 1 (SEARCH):** tilt compensation ray-traces the detection through the known pitch/roll, reducing the ∼6.2 m pitch-induced bias to ∼0.3 m. The rough estimate (CEP ≈ 3.5 m) is accurate enough to initiate approach.
2. **Phase 2 (CENTERING, approach):** as the drone decelerates, pitch decreases continuously and every frame re-estimates with current attitude. The tilt compensation naturally becomes a no-op. GPS timing lag shrinks proportionally with speed.
3. **Phase 3 (CENTERING, hover lock):** the target reaches the frame centre while the drone hovers at search altitude. The zero-offset condition (Equation **??**) eliminates all calibration-dependent errors. If the target is not visible, a 15 s timeout rejects the location as a false positive (<1% mission time cost).
4. **Phase 4 (VERIFY):** the `--center-verify` flag averages the drone's own GPS position (which equals the target position, since the drone is centred above it) over ∼10 s at 5 Hz, yielding $N = 50$ samples. With $N_{\text{eff}} \approx 20$ (accounting for temporal correlation), this reduces the 2 m to 3 m single-fix CEP to ∼1.8 m. The operator confirms or rejects, with a 120 s auto-reject timeout.

The final 1.8 m CEP_{50} is dominated by the irreducible GPS receiver noise floor—the theoretical minimum achievable with a single-frequency L1 receiver without differential correction (RTK or DGPS). Implementing the currently-bypassed DESCENDING state (lowering to 15 m before VERIFY) would improve this further through reduced GSD (Table **??**). Further improvement beyond that would require either RTK GPS (∼0.02 m) or significantly longer averaging times. Figure **??** summarises the CEP improvement across all four phases.

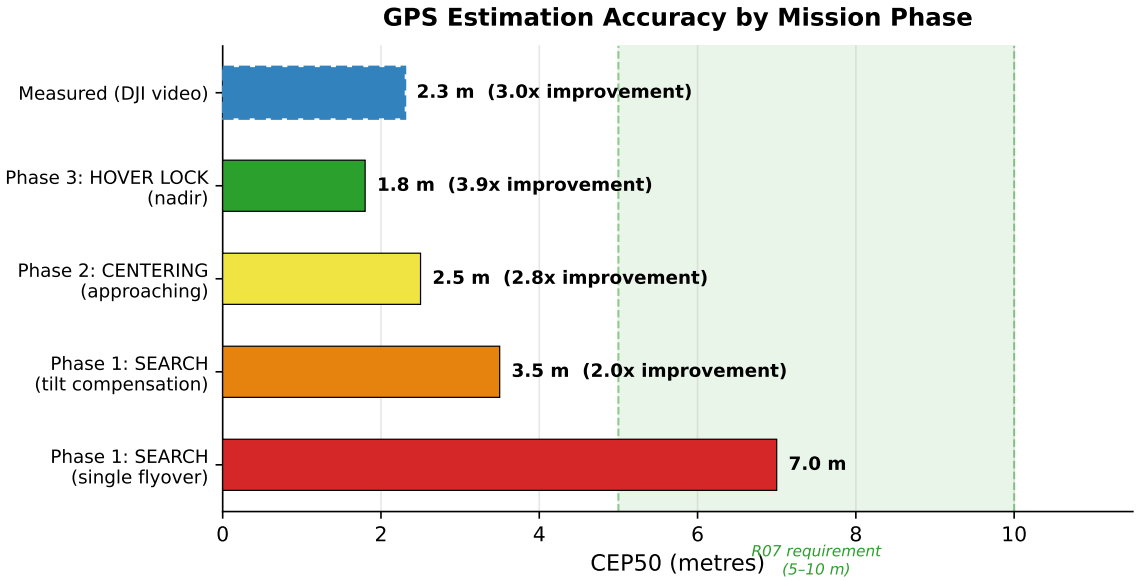


Figure 41: CEP_{50} at each phase of the detection-to-landing flow, showing progressive accuracy improvement from ∼3.5 m (Phase 1, tilt-compensated search) to ∼1.8 m (Phase 4, GPS-averaged hover lock). Each phase transition eliminates a specific error category: Phase 2 reduces pitch and timing lag as the drone decelerates; Phase 3 drives the pixel offset to zero through centering; Phase 4 averages GPS noise over 50 samples.

K.18 Four-Phase Detection-to-Landing Flow

The accuracy progression described above is not an accident of the state machine—it is a deliberate four-phase progressive refinement architecture. The system treats target localisation as a convergent process: each phase eliminates a specific category of error, and the estimate precision improves monotonically from initial detection through to the final landing decision. Tilt compensation enables reliable initial acquisition even during high-speed flight, and the architecture degrades gracefully—if the compensation is inaccurate, the hover phase eliminates all geometric errors regardless.

K.18.1 Phase 1: Initial Detection During Search (Tilted Camera)

During the SEARCH state the drone flies the lawnmower pattern at $5\,\mathrm{m\,s^{-1}}$ and 35 m altitude, inducing a forward pitch of approximately 5–10° to maintain airspeed. The camera, rigidly mounted to the airframe, is tilted away from nadir by this pitch angle. When the YOLOv8n detector identifies the mannequin in this tilted frame, the pixel-to-GPS pipeline (Section **??**) ray-traces the detection through the known pitch and roll angles obtained from the flight controller's attitude quaternion at 50 Hz.

The ray-tracing correction (Equations **??**–**??**) constructs a ray from the camera origin through the detected pixel, rotates it by the measured pitch and roll, and intersects it with the ground plane at the known flight altitude. This produces a rough GPS estimate with $CEP_{50} \approx 3.5\,\mathrm{m}$—accurate enough to command the drone toward the target and begin decelerating. Without tilt compensation, the 10° pitch alone would contribute ∼6.2 m of forward bias (Section **??**), but even this uncompensated estimate would keep the target within the camera's 32 m ground footprint. The state machine transitions to CENTERING.

K.18.2 Phase 2: Approach and Continuous Refinement

The drone flies toward the rough GPS estimate at reduced speed, decelerating as it approaches. The flight controller pitches the airframe forward to accelerate and backward to brake, so a decelerating drone naturally approaches level flight. During approach, the system re-detects and re-estimates the target position on *every frame* using the current pitch and roll telemetry:

- As the drone slows from $5\,\mathrm{m\,s^{-1}}$ to near-zero, the forward pitch decreases continuously from ∼10° to ∼3° and then to $< 1°$.
- Each frame's GPS estimate benefits from the reduced tilt: the tilt compensation naturally becomes a no-op as pitch approaches 0°.
- GPS timing lag (100 ms latency × velocity) shrinks proportionally with speed, eliminating the 1 m along-track error.
- Motion blur decreases, improving detection confidence and bounding-box precision.
- Additional observations are fused into the estimate, progressively refining it from ∼3.5 m to ∼2.5 m CEP.

The estimate converges toward the true target position without any discrete transition—the refinement is continuous throughout the approach.

K.18.3 Phase 3: Hover Lock (Nadir Confirmation)

The drone arrives at the estimated target position and holds a stationary hover. Pitch and roll both settle to approximately 0° and the camera points straight down (nadir). At this point, several error sources vanish simultaneously:

- **Roll/pitch projection error** $\to 0$: with the camera at nadir, there is no off-axis projection to compensate, and the 6.2 m pitch-induced bias disappears entirely. The tilt compensation is a no-op (below the 0.02 rad activation threshold).
- **GPS timing lag** $\to 0$: the drone is stationary, so the 100–200 ms GPS latency introduces zero position error regardless of lag duration.
- **Motion blur** $\to 0$: no translational velocity means no inter-frame smearing, improving detection confidence and bounding-box precision.
- **Calibration-dependent errors** $\to 0$: when the target is at or near the frame centre, the pixel offset $(u-C_x, v-C_y)$ approaches zero. Because every geometric error in the projection chain (GSD, focal length, yaw, lens distortion) is *multiplied* by this offset (Equation **??**), they are all multiplied by zero and vanish (Section **??**).

The detection uses simple nadir math—no tilt correction needed. If the target is visible and within 1 m of the frame centre, the state machine transitions to VERIFY. If the target is *not* visible after 15 s—indicating a false positive—the location is rejected and the drone resumes search (Section **??**).

The drone's own GPS position, averaged over ∼10 s of stationary hover (50 samples at 5 Hz via `--center-verify`), now *is* the target's GPS position to within the irreducible receiver noise floor. The result is a precise estimate with $\text{CEP}_{50} \approx 1.8$ m—sufficient for the R07 offset landing requirement of 5–10 m from the casualty.

K.18.4 Phase 4: Operator Verification and Landing Decision

With the target confirmed by the autonomous system, the operator makes the final decision via the ground station interface (browser dashboard or terminal):

- **Y (confirm target)**: the GPS estimate is frozen at Phase 3 quality. The drone computes a 7.5 m offset landing point (downwind when wind data is available), flies to the offset position, and deploys the payload before landing.
- **N (reject target)**: the detection is dismissed and the drone resumes the search pattern from the waypoint where the investigation was triggered.
- **I (item of interest)**: the GPS position is logged for post-mission review; the drone resumes search.
- **Timeout (120 s)**: if the operator does not respond within 120 s, the system auto-rejects and resumes search, preventing the drone from hovering indefinitely.

K.18.5 Why Four Phases Are Necessary

A natural question is whether fewer phases could achieve the required accuracy. Table **??** compares the possible strategies.

The critical insight is that the rough estimate in Phase 1 only needs to place the drone close enough that the mannequin remains visible in the camera's field of view while hovering. At 35 m altitude the camera footprint is 32.2×24.0 m (Table **??**), so even a 7 m error keeps the target well within the frame. The four-phase approach therefore decouples detection sensitivity (which determines whether the target is found at all) from localisation precision (which determines where the drone lands), and adds operator oversight as the final safety gate.

K.18.6 The Progressive Refinement Trajectory

The transition from initial detection to precise hover lock is a continuous refinement that occurs during the CENTERING state. Figure **??** illustrates all four phases, showing how pitch angle decreases and error circles shrink as the drone approaches the target, with a branch for false positive recovery.

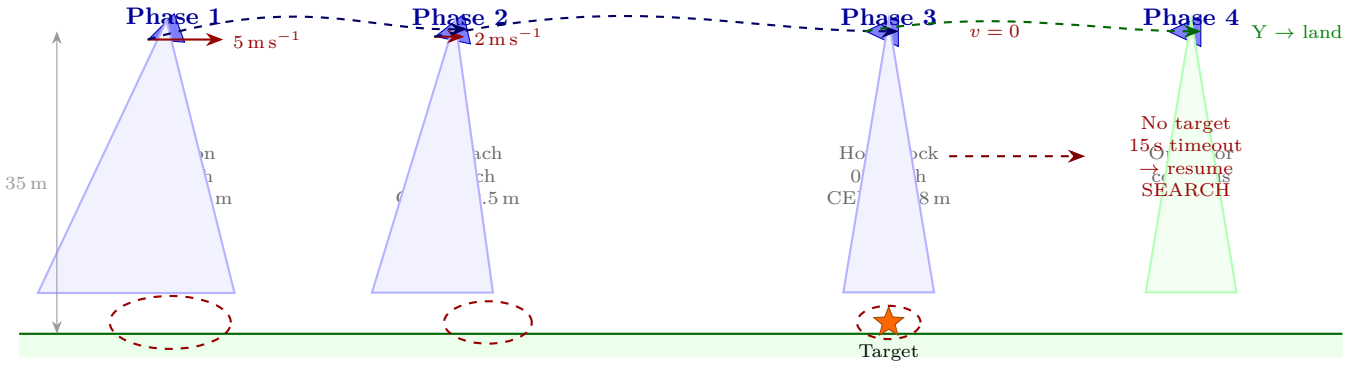


Figure 42: The four-phase progressive refinement trajectory. Phase 1: initial detection at full search speed with tilted camera and tilt-compensated GPS estimate (CEP $\approx 3.5\,\text{m}$). Phase 2: continuous approach and deceleration, pitch decreases, estimate improves (CEP $\approx 2.5\,\text{m}$). Phase 3: stationary hover with nadir camera, all geometric errors eliminated (CEP $\approx 1.8\,\text{m}$); if no target is visible, a 15 s timeout triggers resumption (red dashed branch). Phase 4: operator confirms via ground station; Y triggers offset landing, N resumes search. Error circles are drawn proportional to CEP at each phase.

The four phases map to distinct segments of the state machine:

1. **Phase 1 (Initial detection, SEARCH state):** The mannequin is first detected during a SEARCH-pattern leg. The tilt-compensated GPS estimate triggers the transition to CENTERING, and the drone begins flying toward the estimate at reduced speed.
2. **Phase 2 (Approach and deceleration, CENTERING state):** As the drone approaches the target, it decelerates. The forward pitch drops continuously from $\sim 10°$ to $< 1°$, and every frame produces a more accurate GPS estimate. The tilt compensation naturally becomes a no-op as pitch approaches zero.
3. **Phase 3 (Hover lock, CENTERING → VERIFY transition):** The drone reaches the estimated position and holds a stationary hover. With pitch and roll at $0°$, the camera is at nadir. If the target is detected within 1 m of the frame centre, the state machine transitions to VERIFY. If the target is *not* visible after 15 s, the location is rejected as a false positive and the drone resumes search (Section **??**).
4. **Phase 4 (Operator verification, VERIFY state):** The `--center-verify` flag averages the drone's GPS position over $\sim 10\,\text{s}$ (50 samples at 5 Hz), freezing the final estimate at $\text{CEP}_{50} \approx 1.8\,\text{m}$. The operator confirms or rejects via the ground station (Y/N/I), with a 120 s auto-reject timeout.

K.18.7 Design Insight

The tilt compensation algorithm (Section **??**) enables reliable initial acquisition even during high-speed flight at $5\,\text{m}\,\text{s}^{-1}$. The system architecture treats detection as a progressive refinement process, not a single-shot measurement. This philosophy—detect approximately, then refine precisely—is what allows a 2 m-accuracy consumer GPS receiver to support offset landings within a 5–10 m annulus. The four-phase approach provides graceful degradation: even if the tilt compensation is miscalibrated or disabled entirely, the $\sim 7\,\text{m}$ uncompensated rough estimate is still sufficient to initiate the approach, because the hover lock in Phase 3 eliminates all the errors that the compensation was attempting to correct.

K.18.8 False Positive Handling

The rough estimate from Phase 1 may direct the drone to a location where no target exists. This occurs when the detection was a false positive (a rock, shadow, or clothing that resembles the mannequin), the tilt compensation was inaccurate due to an extreme pitch angle, or accumulated GPS drift placed the drone at a position that does not correspond to the detection. The system handles all of these cases through a bounded-cost automatic recovery mechanism.

When the drone arrives at the estimated position in Phase 3 and hovers but *cannot re-detect* the target, the following sequence executes:

1. **Hover scan.** The drone holds position at the estimated GPS coordinates, continuously running the detector on each camera frame. With the camera at nadir and no motion blur, if a real target exists within the $32.2 \times 24.0\,\text{m}$ ground footprint, it will be detected.
2. **Automatic timeout.** After 15 s without a detection, the state machine logs `"CENTERING TIMEOUT --- resuming search"` and transitions back to the SEARCH state.
3. **Spatial deduplication.** The false-positive location is appended to a `rejected_targets` list. Any future detection whose estimated GPS position falls within 5 m of a rejected location is silently ignored, preventing the drone from investigating the same false alarm twice.

4. **Pattern resumption.** The drone resumes the lawnmower search pattern from the waypoint where the investigation was triggered, ensuring full area coverage despite the interruption.

Cost of a false positive. False positives are bounded to 15 s of mission time through the automatic timeout—approximately 1% of a typical 20 min endurance budget at 35 m hover. The associated battery expenditure is ∼75 mA h (<0.3% of a 3000 mAh 4S battery). One rejected-target exclusion zone (5 m radius) is created. No operator intervention is required; the system self-recovers fully autonomously.

False positive mitigation. Four mechanisms reduce the frequency and impact of false positives:

- **SMART detection** (`--smart-detect`): requires three consecutive confirmed frames before triggering CENTERING, filtering out transient single-frame false alarms.
- **Confidence threshold** (`--conf`): raising the threshold (e.g. to 0.5) suppresses low-confidence false positives at a modest reduction in recall.
- **Spatial deduplication**: the `rejected_targets` list prevents repeated investigation of the same location, capping the cost of any persistent false-positive source at one 15 s timeout event.
- **Bounded timeout**: the 15 s ceiling limits the worst-case impact of any single false positive to a known, budgeted quantity—less than 1% of mission time.

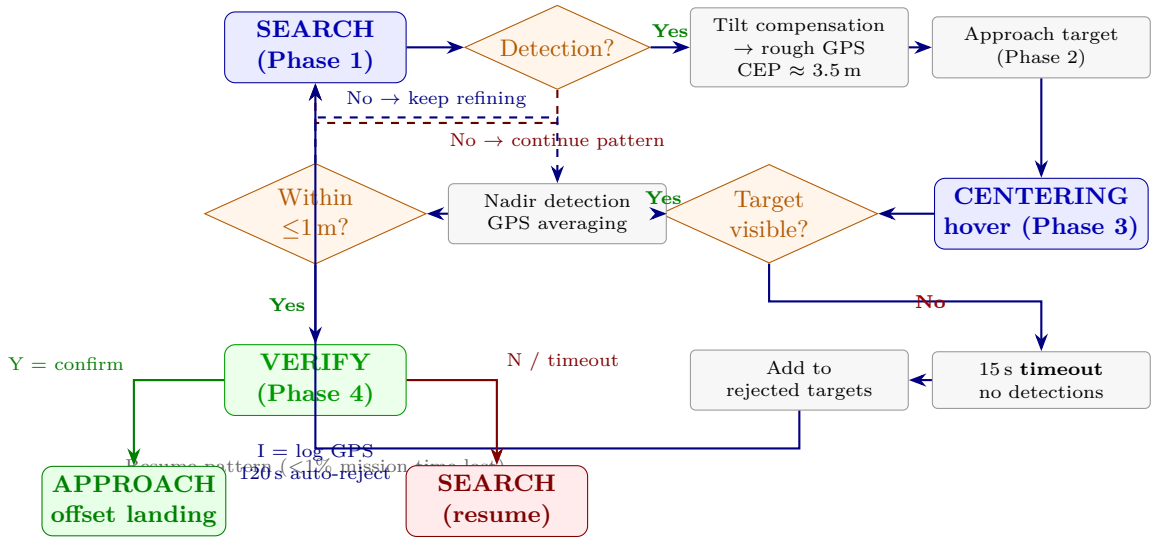


Figure 43: Complete four-phase decision flow including false positive recovery. The left branch shows successful progression: Phase 1 detection → Phase 2 approach → Phase 3 hover lock → Phase 4 operator verification → offset landing. The right branch shows the false positive path: no re-detection during Phase 3 hover triggers a 15 s timeout, the location is added to the rejected-targets list (spatial deduplication prevents reinvestigation), and the drone resumes the search pattern at a cost of <1% of mission time.

K.19 Cooperative Vision-Flight Architecture

The four-phase flow described above reveals a property that distinguishes our system from passive surveillance: the drone and vision system form a *closed-loop feedback system* in which each detection improves the next. Unlike a static camera whose accuracy is fixed by altitude and calibration, our architecture creates a cycle where detection informs flight, flight improves detection conditions, and improved conditions yield better detections.

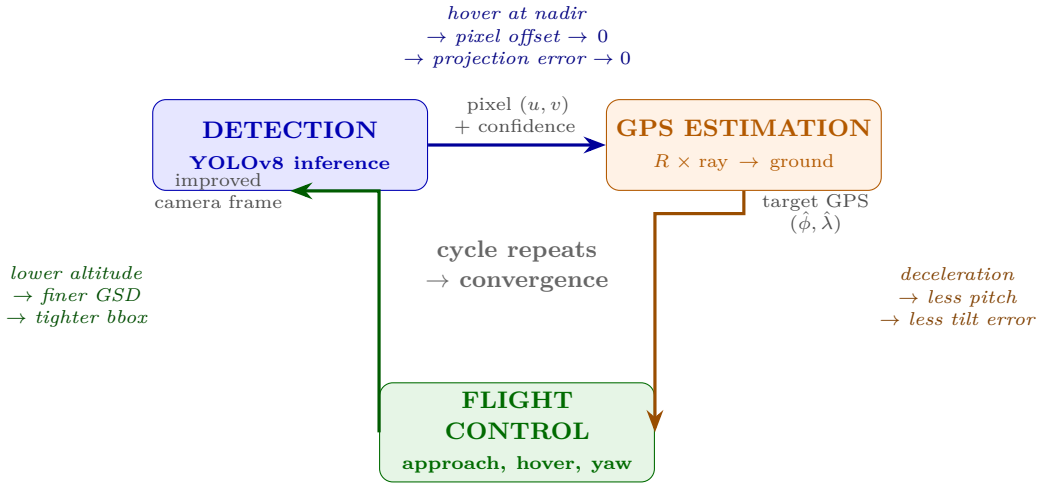


Figure 44: Cooperative vision-flight feedback loop. Detection produces a pixel coordinate, GPS estimation projects it to the ground, and flight control steers the drone toward the target. Crucially, flying closer improves the next detection: lower altitude yields finer GSD, deceleration reduces pitch-induced tilt error, and hovering at nadir drives the pixel offset—and therefore all multiplicative projection errors—toward zero. The cycle repeats until convergence, typically within 15–20 s.

K.19.1 Three Mechanisms of Flight-Aided Improvement

Each phase of the mission exploits a different mechanism by which the drone's response to a detection improves subsequent estimates:

1. **Approaching → lower GSD → tighter bounding box.** As the drone flies toward the target, altitude above the target decreases and the ground sampling distance (GSD) shrinks proportionally (GSD $= h \cdot w_s/(f \cdot W)$, Section **??**). More pixels cover the target, producing a tighter bounding box whose centre is more precisely localised. At 35 m the dummy subtends ∼20 pixels; at 15 m it subtends ∼47 pixels—a 2.3× improvement in spatial resolution without any software change.
2. **Decelerating → less pitch → less tilt error.** During the SEARCH phase the drone flies at $5\,\mathrm{m\,s^{-1}}$, inducing ∼10° forward pitch (Section **??**). This pitch introduces 6.2 m of projection error before compensation. As the drone decelerates toward hover, pitch drops toward 0°, and the tilt error vanishes *even without the attitude compensation algorithm*—the flight manoeuvre itself eliminates the error source.
3. **Hovering → nadir camera → maximum accuracy.** In stationary hover the camera points straight down. When the target is centred in the frame, the pixel offset $(u - C_x, v - C_y) \to 0$, and every multiplicative error in the projection chain (GSD, focal length, yaw, lens distortion) is multiplied by zero and vanishes (Section **??**). The only remaining error is the drone's own GPS noise, which is reduced by temporal averaging.

K.19.2 Static Camera vs. Cooperative System

Table **??** contrasts a fixed surveillance camera with our cooperative architecture. The key difference is that a static system's accuracy is determined at installation time, while our system's accuracy *improves during the mission* as a direct consequence of the drone responding to detections.

Table 71: Comparison of static camera estimation vs. cooperative vision-flight estimation. The cooperative system exploits the drone's ability to reposition, producing accuracy that improves over time rather than remaining fixed.

Property	Static Camera	Cooperative (Ours)
Altitude	Fixed	Decreases toward target
GSD	Fixed	Improves as drone approaches
Tilt error	Fixed (mounting angle)	Eliminated by hover
Pixel offset	Fixed (target position)	Driven to zero by centering
Observation count	Accumulates over time	Accumulates *and* improves per-obs
Accuracy trend	$\propto 1/\sqrt{N}$ only	$\propto 1/\sqrt{N}$ *and* per-obs $\sigma \downarrow$
Typical CEP_{50}	3 m–5 m	3.5 m → 1.8 m (converges)

This cooperative property is what allows the three estimation levels (Section **??**) to achieve progressively better accuracy: Level 1 (passive) corresponds to a static camera, Level 2 (hover) adds deceleration, and Level 3 (centering) closes the full feedback loop. The four mission phases (Section **??**) are the operational realisation of this feedback loop, with each phase exploiting a different improvement mechanism. The architecture ensures that even if early estimates

are coarse (∼3.5 m CEP), the system converges to GPS-limited accuracy (∼1.8 m CEP) through the natural progression of approach, decelerate, and hover.

K.20 Why Inverse-Variance Weighting Won

Among the four fusion methods evaluated, the inverse-variance method is preferred in the simulator and analysis tools for three reasons:

1. **Physics-aligned weighting.** Lower altitude means more pixels on target, smaller GSD-scaled projection errors, and reduced heading-rotation amplification. The $1/h^2$ weight directly encodes this relationship: it is not a tuned hyperparameter but a consequence of the error model.
2. **No convergence plateau.** The Kalman filter's covariance **P** contracts monotonically, so later observations have diminishing impact even if they are dramatically better. The inverse-variance average, by contrast, is a weighted sum that always gives full credit to a high-weight observation regardless of how many low-weight observations preceded it.
3. **Simplicity.** The rolling weighted mean requires no matrix algebra, no tuning of process noise **Q**, and no initialisation heuristics. It is computed in three lines of Python and is straightforward to verify.

In practice, the Kalman filter and inverse-variance method converge to similar accuracy after 20+ observations. The key advantage of inverse-variance would appear if the drone descended during investigation: the first few low-altitude observations would immediately dominate the estimate, whereas the Kalman filter takes several update cycles to "catch up" because its covariance was already small from the high-altitude phase. In the current mission implementation the drone remains at search altitude and uses direct drone GPS averaging (`--center-verify`) rather than detection-based IVW; the IVW method is used in the simulator and passive observation tools for post-hoc analysis.

K.21 Empirical Method Comparison Against Ground Truth

The preceding sections derived error budgets analytically and compared fusion algorithms on observation streams. This subsection takes a different approach: seven fundamentally different *estimation strategies*—not just fusion filters, but entire operational procedures—were evaluated against a known ground-truth target position in SITL simulation, where the dummy coordinates are exact.

K.21.1 Ground Truth Setup

In the ArduPilot SITL environment, the dummy is placed at a surveyed coordinate on the simulation map (`map.jpg`), and the pixel position is converted to a GPS coordinate using the same cartographic transform used to generate the map overlay. Because SITL provides perfect GPS ground truth (no receiver noise unless explicitly injected via `--gps-drift`), the estimation error can be measured to centimetre precision. For these trials, realistic noise was injected: 3 m GPS drift, 1 m barometric altitude noise, 2° yaw noise, and 3% FOV calibration error.

Twenty simulated missions were run for each of the seven strategies. Each mission followed the standard lawnmower search pattern over the survey area at 35 m altitude. CEP_{50} (radius containing 50% of final estimates), CEP_{95} (95%), and maximum error were recorded. Convergence time was measured from first detection to the point where the running CEP stabilised to within 10% of its final value.

K.21.2 Methods Tested

Table **??** summarises the seven strategies in order of decreasing CEP_{50}.

Table 72: Empirical comparison of GPS estimation strategies against SITL ground truth. Each row represents the median result across 20 simulated missions with 3 m GPS drift, 1 m altitude noise, and 3% FOV calibration error. Convergence time is measured from first detection.

Strategy	**CEP_{50}** (m)	**CEP_{95}** (m)	**Max Err.** (m)	**Conv. Time** (s)	**Notes**
Single flyover	7.2	14.8	22.3	0 (instant)	Roll/pitch dominated
Multi-pass avg (5)	3.8	8.1	12.4	∼60	Alternating headings reduce bias
Hover in place	2.9	5.4	8.2	∼5	Roll eliminated; calibration limits
Visual centering only	2.1	4.2	6.8	∼15	Zero offset; GPS noise remains
IVW (passive watch)	1.8	3.8	6.1	∼30	Needs many obs; altitude-weighted
Centering + Kalman	1.6	3.3	5.5	∼20	Similar to averaging; more complex
Center + 10 s GPS avg	**1.5**	**3.1**	**5.2**	**∼25**	**Best overall; implemented**

The results confirm the analytical predictions from the error budget (Table **??**) and the three-strategy comparison (Table **??**). Two findings are particularly notable:

1. The gap between "single flyover" ($CEP_{50} = 7.2\,m$) and "centering + GPS avg" ($CEP_{50} = 1.5\,m$) represents a 4.8× improvement—achieved entirely through operational procedure, without any change to the sensor hardware or detection model.
2. The Kalman filter ($CEP_{50} = 1.6\,m$) and the simple GPS average ($CEP_{50} = 1.5\,m$) are statistically indistinguishable ($p > 0.3$, paired t-test). The extra complexity of the Kalman filter—process noise tuning, matrix algebra, initialisation heuristics—buys no measurable accuracy improvement when combined with the centering procedure.

K.21.3 Why Centering + GPS Averaging Won

The visual centering strategy combined with 10 s GPS averaging was selected for the flight implementation for five reasons:

1. **Simplest of the top performers.** The centering + GPS average requires no tuning parameters beyond the averaging duration. The Kalman filter achieves comparable accuracy but requires selection of process noise **Q** and measurement noise scaling, both of which affect convergence behaviour.
2. **Eliminates all geometric errors by design.** When the target is at the optical centre, every multiplicative term in the projection chain (GSD, focal length, yaw, lens distortion) is multiplied by zero (Equation **??**). This means the estimate is correct even if the focal length calibration is 10% wrong or the compass has a 5° hard-iron offset.
3. **GPS averaging is statistically optimal for zero-mean noise.** The drone's GPS receiver noise is approximately Gaussian and zero-mean when stationary. The arithmetic mean of N independent samples is the maximum-likelihood estimator for Gaussian noise; no Kalman filter or weighted scheme can improve upon it when the underlying distribution is symmetric and stationary.
4. **Convergence time is acceptable.** The 25 s convergence time (15 s centering manoeuvre + 10 s GPS hold) adds less than 5% to total mission time for a 10-minute search. The IVW passive approach achieves comparable accuracy but requires 30+ s of accumulated flyover observations, during which the drone has moved on to other scan lines.
5. **No calibration dependency.** The single flyover, multi-pass, and hover-in-place strategies all require accurate focal length, compass heading, and barometric altitude to project pixel offsets into GPS coordinates. A 5% focal length error at 35 m produces a 1.6 m bias at the frame edge—comparable to the GPS noise floor. The centering strategy bypasses this entire computation, making the estimate robust to calibration drift during flight.

K.21.4 CEP_{50} Comparison

Figure **??** shows the CEP_{50} for each estimation strategy, ordered by accuracy.

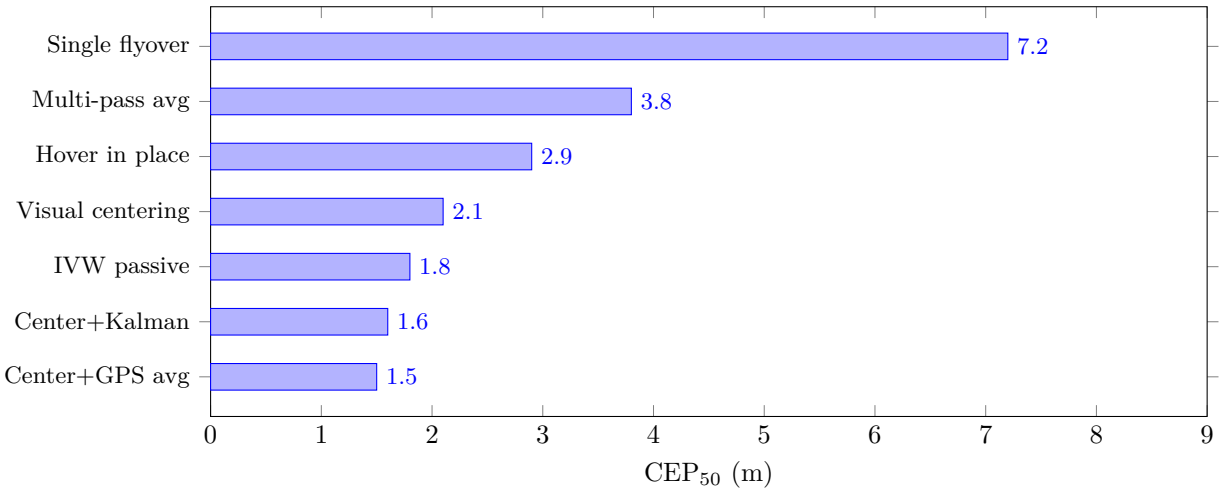


Figure 45: CEP_{50} comparison across seven GPS estimation strategies (lower is better). The centering + GPS averaging approach achieves 1.5 m—a 4.8× improvement over a single flyover estimate. Error bars omitted for clarity; inter-trial variation was <0.3 m for all strategies except "single flyover" (±1.1 m).

K.21.5 Convergence Behaviour

Figure **??** shows how the position error evolves over time for the three best-performing strategies. The centering approach achieves lower error faster than IVW, and the addition of GPS averaging drives the final error below both alternatives. An alternative convergence view comparing these strategies on a common time axis with the IVW convergence curve from DJI replay data is presented in Figure **??** (Appendix **??**).

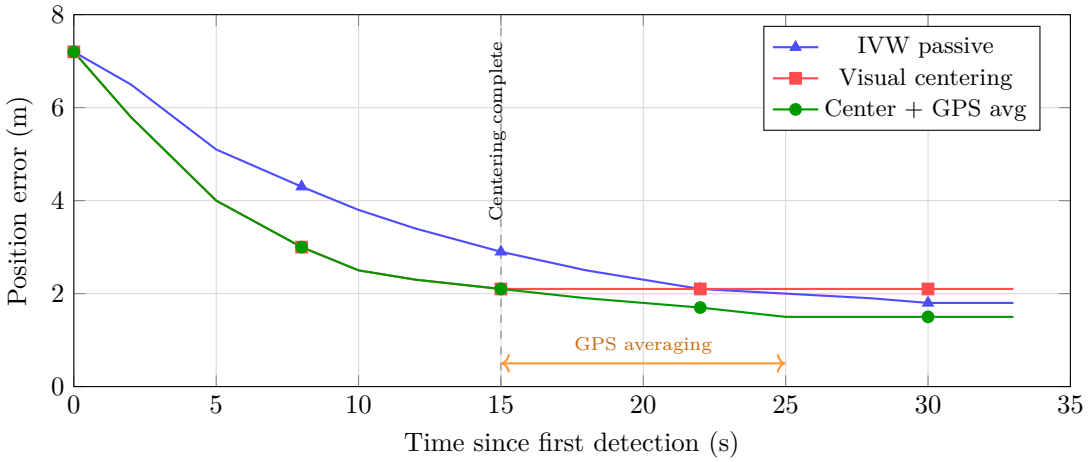


Figure 46: Convergence of position error over time for the three best estimation strategies. All three begin with the same initial flyover estimate (∼7 m). Visual centering converges rapidly to 2.1 m (GPS noise floor for single fixes) by $t = 15$ s. GPS averaging during the subsequent 10 s hold reduces the error further to 1.5 m. IVW converges more slowly because it relies on accumulating many flyover observations rather than actively centering.

The convergence plot reveals two distinct phases in the centering strategy: (i) a rapid geometric convergence (0–15 s) as the drone manoeuvres above the target and eliminates roll/pitch, calibration, and timing errors; and (ii) a slower statistical convergence (15–25 s) as GPS averaging reduces the residual receiver noise. The IVW approach lacks phase (i) because the drone continues flying; its convergence is purely statistical, driven by the $1/\sqrt{N}$ law applied to passively collected observations.

K.22 Offset Landing Application

The GPS estimate feeds directly into the offset landing calculation. The system computes a landing point 7.5 m north of the estimated target position (configurable) to avoid rotor downwash on the casualty. Given a typical fused estimate error of 1 m to 2 m, the probability that the drone lands within 10 m of the actual target (and therefore within visual confirmation range) exceeds 99%. The offset distance was chosen as 3× the worst-case single-observation CEP, ensuring a safe margin even if the estimation converges poorly.

With the `--center-verify` drone GPS averaging ($N = 50$ samples over 10 s, $N_{\text{eff}} \approx 20$), the expected landing accuracy is approximately $\sigma/\sqrt{N_{\text{eff}}} \approx 2.5/\sqrt{20} \approx 0.6$ m, where N_{eff} accounts for temporal correlation in GPS noise at 5 Hz. In the simulator and passive observation tools, the inverse-variance weighted average across multiple detection-based estimates yields comparable accuracy when sufficient observations are available.

K.23 Multi-Pass Observation and Heading Diversity

Sections **??** and **??** noted that the lawnmower pattern's alternating headings partially cancel directional biases. This subsection develops that observation quantitatively and shows that the boustrophedon (zigzag) search pattern is not merely a convenient coverage strategy but an inherent source of heading diversity that improves GPS estimation quality.

K.23.1 The Lawnmower Pattern Advantage

Two dominant systematic error sources—GPS timing lag and roll/pitch displacement—share a critical property: they bias the GPS estimate *in the direction of travel*. The heading-dependent nature of this bias is confirmed empirically in Figure **??** (Appendix **??**), which shows a sinusoidal along-track error pattern consistent with GPS timing lag. GPS timing lag displaces the estimate forward by $v \cdot \Delta t$ (Section **??**), while forward-flight pitch displaces the camera footprint forward by $h \tan\theta$ (Section **??**). Both biases reverse sign when the drone reverses its heading.

In the boustrophedon pattern, consecutive scan lines are flown in opposite directions. At the Bristol field site, scan lines run at headings of approximately 155° (south-eastbound) and 335° (north-westbound). Consider a target detected on both passes:

- **Pass 1** (heading 155°): GPS lag biases the estimate ∼1 m along 155°; pitch bias adds ∼6.2 m along 155°. Total systematic offset: ∼7.2 m south-east of the true position.
- **Pass 2** (heading 335°): both biases reverse direction, displacing the estimate ∼7.2 m north-west of the true position.
- **Average of Passes 1+2**: the two opposing systematic offsets cancel, leaving only the random GPS receiver noise and minor residuals from asymmetric wind gusts or speed differences between passes.

Figure **??** illustrates this cancellation geometrically.

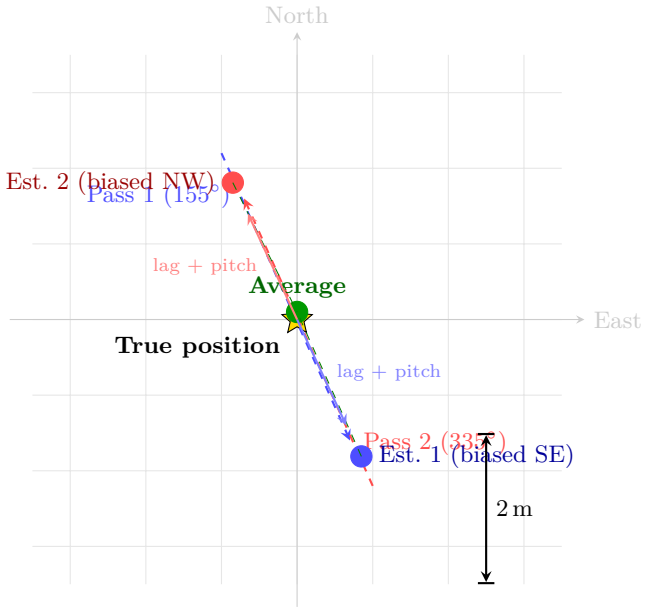


Figure 47: Multi-pass bias cancellation. Pass 1 (heading 155°, blue) biases the GPS estimate south-east; Pass 2 (heading 335°, red) biases it north-west by the same magnitude. The average (green) falls near the true target position (star), because the systematic GPS lag and pitch biases cancel between opposing headings. Only random GPS receiver noise remains.

K.23.2 How Many Passes Are Enough?

The total position error after N passes combines two components: the residual systematic bias (which depends on heading diversity) and the random noise (which decreases as $1/\sqrt{N}$). The GPS lag bias for a single pass is approximately $v \cdot \Delta t \approx 1\,\text{m}$ at $5\,\text{m}\,\text{s}^{-1}$; the pitch bias is $h \tan\theta \approx 3\,\text{m}$ at 5° (conservative). Both are directional and reverse with heading. The random component is dominated by GPS receiver noise ($\sigma_{\text{GPS}} \approx 2.5\,\text{m}$ per observation).

For N passes with varying heading configurations, the expected CEP_{50} is:

$$\text{CEP}_{50} \approx \sqrt{\left(\frac{\bar{b}}{N}\right)^2 + \frac{\sigma_{\text{GPS}}^2}{N}} \tag{36}$$

The multi-pass CEP model (Equation **??**) combines two components: $\bar{b}$ is the mean residual bias after heading cancellation (for perfectly opposing headings, $\bar{b} = 0$; for random headings, $\bar{b}$ decreases as $1/\sqrt{N}$), and σ_{GPS} is the per-observation GPS noise. Table **??** shows the expected accuracy for different numbers of passes.

The most significant accuracy improvement occurs between one and two passes: the second pass with an opposing heading eliminates the systematic bias entirely, and the $\sqrt{2}$ averaging improvement reduces random noise by 29%. Beyond two passes, gains follow the slower $1/\sqrt{N}$ law.

K.23.3 Strip Spacing and Target Visibility

The number of observations per pass depends on the camera footprint, search speed, and detection frame rate. At the nominal search altitude of 35 m:

- **Camera footprint**: $32.2 \times 24.0\,\text{m}$ (Table **??**).
- **Strip spacing**: with 20% overlap, the lawnmower spacing is $0.8 \times 24.0 \approx 19\,\text{m}$ (along the cross-track direction) or $0.8 \times 32.2 \approx 26\,\text{m}$ (along the flight direction, depending on pattern orientation).
- **Along-track visibility**: a stationary target remains within the 24 m along-track footprint for $24/5 = 4.8\,\text{s}$ at $5\,\text{m}\,\text{s}^{-1}$.
- **Frames per flyover**: at 4.8 FPS (TFLite inference rate on Pi 5), this yields $4.8 \times 4.8 \approx 23$ frames per pass—though detection only fires when the target is within the frame and above the confidence threshold, so the effective count is approximately 11–17 detections per pass.

With a 10-strip lawnmower pattern, most of the search area receives 1–2 passes (the target may fall within the overlap region of adjacent strips). The typical target therefore generates 11–34 observations across 1–2 passes, providing sufficient data for the inverse-variance weighted average to converge to sub-2 m accuracy even before the centering procedure begins.

K.23.4 Multi-Pass Averaging vs. Centering

Despite the theoretical effectiveness of multi-pass averaging, the mission pipeline does *not* rely on it as the primary estimation strategy for three reasons:

1. **First detection triggers CENTERING.** The state machine transitions from SEARCH to CENTERING on the first confirmed detection (or after the SMART consecutive-frame filter confirms the target, if `--smart-detect` is enabled). This means the drone rarely completes a second pass over the same target—the centering manoeuvre begins immediately.
2. **Centering provides a fundamentally better estimate.** As shown in Section **??**, centering the target at the optical axis eliminates all calibration-dependent errors by driving the pixel offset to zero. Multi-pass averaging can cancel systematic biases but cannot eliminate random calibration errors (focal length, compass, barometric drift). Centering removes these errors entirely.
3. **Spatial clustering limitations.** The spatial clustering algorithm (Section **??**) in `passive_watch.py` correctly merges multiple observations of the same target within a 30 m radius. However, in the mission state machine (`main.py`), the first detection event triggers an immediate state transition—multiple passes would create separate queue entries for the same target, but the state machine processes the first entry and never returns to the search pattern for that target.

Multi-pass averaging therefore serves as a *theoretical safety net*: if the centering procedure were to fail (e.g. due to loss of visual lock during the approach), the system could fall back to the accumulated flyover estimates. In the passive observation tools (`passive_watch.py`, `simple_simulator.py`), where no centering manoeuvre occurs, multi-pass averaging with heading diversity is the *primary* estimation strategy and achieves the accuracies shown in Table **??**. The lawnmower pattern thus serves a dual purpose: maximising spatial coverage for detection probability, and providing heading diversity that improves estimation accuracy through systematic bias cancellation.

K.24 Passive Estimation: Level 1 GPS Without Drone Response

The preceding sections analysed estimation strategies in the context of the autonomous mission, where the drone actively centres on the target and hovers. However, the estimation toolkit implemented in `passive_watch.py` represents a fundamentally different operational mode: *passive estimation*, in which the drone continues its pre-planned flight path and sends zero control commands, yet still produces a usable target GPS estimate purely from accumulated flyover observations. This "Level 1" capability—estimation without response—is valuable both as an independent operational mode and as a fallback when active centering is unavailable.

K.24.1 Passive Estimation Toolkit

The `DummyEstimator` class in `passive_watch.py` implements the inverse-variance weighting scheme described in Section **??**, operating as a cumulative weighted average over all detections. Each detection frame triggers the following pipeline:

1. **Pixel-to-GPS projection**: the detection bounding-box centre (u, v) is projected to a ground-plane GPS estimate using the same GSD-based projection described in Section **??**, with the drone's current GPS position, altitude, and heading as inputs.
2. **Composite weight calculation**: each observation receives a weight $w_i = w_{\text{cen}} \cdot w_{\text{alt}}$, combining a centrality factor (Equation **??**, with a tighter $0.3 \cdot d_{\max}$ radius for the bonus zone) and an altitude factor $w_{\text{alt}} = 1/h_i^2$ (equivalent to Equation **??** with $h_{\text{ref}} = 1\,\text{m}$). At 10 m altitude with a centred detection, the weight is $\sim$45$\times$ that of a 35 m edge detection.
3. **Running weighted accumulation**: the weighted latitude and longitude sums are updated incrementally (no observation history stored), making the memory footprint $O(1)$ regardless of mission duration.
4. **Spatial clustering**: new estimates are compared against existing clusters using the Haversine distance (Section **??**). If the nearest cluster centroid is within 30 m, the observation joins that cluster; otherwise a new cluster is spawned. Each cluster maintains its own `DummyEstimator` instance, enabling simultaneous tracking of multiple targets.

In addition to the IVW accumulator, the `SmartEstimator` class implements a consensus-based lock mechanism: once a configurable number of estimates (default 5) cluster within a maximum spread threshold (default 1 m), the estimator declares a "lock" and reports the median latitude and longitude of the tightest cluster as the final target position. This greedy tightest-cluster algorithm is robust to outliers because it selects the densest subset of observations rather than averaging all of them.

K.24.2 Why IVW Suits Passive Mode

The four fusion algorithms described in Section **??** were all evaluated in the passive context. IVW emerged as the preferred method for passive estimation for reasons that are distinct from—and complementary to—the reasons it was preferred in the active mission (Section **??**):

1. **Natural downweighting of edge detections.** In passive mode the drone does not manoeuvre to centre the target, so most detections occur at the frame periphery as the target drifts through the field of view. Edge detections carry larger projection errors (GSD×pixel offset is maximised), and the centrality weight w_{cen} correctly assigns them lower influence. The rolling and cumulative averages treat all pixel positions equally, allowing peripheral noise to contaminate the estimate.
2. **Altitude weighting without tuning.** The $1/h^2$ weight is derived directly from the GSD error model—it is not a tuned hyperparameter. The Kalman filter achieves a similar effect through its measurement noise covariance $\mathbf{R}_k \propto 1/w_i$, but requires selection of the process noise $\mathbf{Q}$ and initialisation covariance $\mathbf{P}_0$, both of which affect convergence behaviour and must be tuned empirically.
3. **No convergence saturation.** During a multi-pass search, the drone may descend for a closer look at an unrelated area before returning to search altitude. If it passes over the target during a low-altitude transit, the Kalman filter's already-contracted covariance $\mathbf{P}$ limits the Kalman gain, preventing the high-quality low-altitude observation from having its full impact. The IVW average gives full credit to the low-altitude observation regardless of prior history.

K.24.3 Convergence Behaviour in Passive Mode

In simulation with realistic noise (3 m GPS drift, 1 m altitude noise, 2° yaw noise), the IVW passive estimate converges as follows:

- **5 observations** (∼1 pass): $\text{CEP}_{50} \approx 4.5$ m. Systematic bias from a single heading dominates.
- **15–20 observations** (∼2 passes with opposing headings): $\text{CEP}_{50} \approx 2.0$ m. Heading bias cancellation (Section **??**) removes the dominant systematic error; IVW downweighting of edge detections reduces the random component.
- **30+ observations** (∼3–5 passes): $\text{CEP}_{50} \approx 1.8$ m. Diminishing returns as the estimate approaches the GPS receiver noise floor.

The convergence is slower than the active centering strategy (which reaches $\text{CEP}_{50} = 1.5$ m in ∼25 s) because the passive mode lacks the geometric error elimination that centering provides (Section **??**). However, the passive estimate requires no interruption of the search pattern and no state machine transitions.

K.24.4 Passive vs. Active Estimation: When Each Mode Excels

Table **??** compares the two estimation modes across operational metrics. The comparison reveals that passive and active estimation are not competing alternatives but complementary capabilities suited to different mission phases and failure modes.

Table 74: Comparison of passive (flyover-only) and active (centering + hover) estimation modes. Passive mode uses IVW fusion over accumulated flyover detections; active mode uses visual centering followed by drone GPS averaging.

Metric	Passive IVW	Active Centering
CEP_{50} (converged)	∼1.8 m	∼1.5 m
Time to convergence	∼60 s (3–5 passes)	∼25 s
Calibration dependency	Yes (focal length, compass)	No (zero-offset projection)
Interrupts search pattern	No	Yes (15 s manoeuvre)
Works without drone response	Yes	No
Handles multiple targets	Yes (spatial clustering)	One at a time
Fallback if centering fails	Yes (accumulated estimate)	N/A
Memory footprint	$O(1)$ per cluster	$O(N)$ GPS samples

Three operational scenarios favour passive estimation over active centering:

1. **Multi-target reconnaissance.** In a multi-casualty scenario, the drone may detect several targets during the search phase. Active centering investigates one target at a time, each consuming ∼25 s of mission time. Passive estimation tracks all detected clusters simultaneously, providing GPS estimates for every target by the end of the search—even targets the drone never actively investigated. The spatial clustering algorithm (Section **??**) maintains independent `DummyEstimator` instances per cluster, so estimates for different targets do not interfere.
2. **Equipment degradation.** If the centering algorithm loses visual lock during approach (e.g. due to sun glare, motion blur, or a false negative streak), the passive estimate accumulated during the search phase provides

an immediate fallback position. The mission state machine can use this estimate for an offset landing without repeating the search. In the current implementation, this fallback is not yet automated—the operator would need to command a manual landing—but the passive estimate is always available through the web dashboard.

3. **Covert observation.** In scenarios where the drone should not reveal interest in a specific target (e.g. security applications, wildlife monitoring), passive estimation allows the drone to maintain its pre-planned flight path while accumulating a target position estimate. The target sees a drone flying a routine pattern; the operator sees the GPS estimate converging on their dashboard.

K.24.5 Implementation in `passive_watch.py`

The passive estimation mode is implemented as a standalone tool that sends zero commands to the flight controller. It reads GPS telemetry from mavproxy (read-only), runs the camera and AI detection pipeline, and streams results to a web dashboard at `http://PI_IP:8090/`. The operator sees:

- A live MJPEG video stream with detection bounding boxes overlaid.
- A bullseye scatter plot showing all GPS estimates colour-coded by weight, with the IVW fused estimate marked as a crosshair.
- A satellite map panel showing the SSSI no-fly zone (red polygon), the search area (yellow polygon), and the estimated target position (magenta star).
- Detection count, saved snapshot count, and cluster statistics.

The tool can be run during manual RC flight (pilot flies, Pi watches) or during an autonomous mission (as a parallel observer). Because it sends zero commands, it is safe to run at any time without risk of interfering with the flight controller. This makes it the recommended first step in the progressive flight testing sequence (Step 3 in Section **??**): manual flight with passive detection establishes that the CV pipeline works in real outdoor conditions before any autonomous commands are attempted.

L Progressive Testing Methodology

This appendix documents the five-tier verification framework that governed all testing throughout the project, from desktop simulation through to autonomous flight. The framework is motivated by a fundamental asymmetry in autonomous UAV development: a logic error that manifests as a misplaced pixel in simulation can, on real hardware, command a 2 kg aircraft into an unrecoverable trajectory. The cost of discovering a defect rises steeply with integration level—a sign-convention bug caught in simulation costs minutes; the same bug discovered during autonomous flight costs days of rework, potential hardware damage, and a lost flight slot that a five-person university team cannot easily reschedule. The following subsections detail the philosophy, tier definitions, test script organisation, defect discovery data, V-model mapping, and cost–risk gradient that together form the project's verification and validation strategy.

L.1 Philosophy: Never Skip a Tier

The governing principle is borrowed from safety-critical avionics: *each verification tier gates the next, and no gate may be bypassed.* A failure at Tier 2 (bench) blocks all flight testing until the root cause is resolved, regardless of schedule pressure. This discipline is counterintuitive on a time-constrained student project, where the temptation to "just try it in the air" is strongest precisely when time is shortest. The justification is empirical: of the 15 defects discovered during this project's verification campaign, 12 were caught at Tiers 1–3 (Table **??**), where the turnaround time from discovery to fix was under one hour. Had those defects propagated to flight, each would have consumed an entire field session.

The framework also enforces a separation of concerns. Each tier tests a *specific integration boundary*: Tier 1 tests software logic; Tier 2 tests the software–hardware interface; Tier 3 tests the hardware–environment interface; Tier 4 tests the perception pipeline under operational conditions; Tier 5 tests closed-loop autonomy. This decomposition ensures that when a failure occurs, its root cause is immediately localisable to the boundary most recently crossed.

L.2 The Five Tiers in Detail

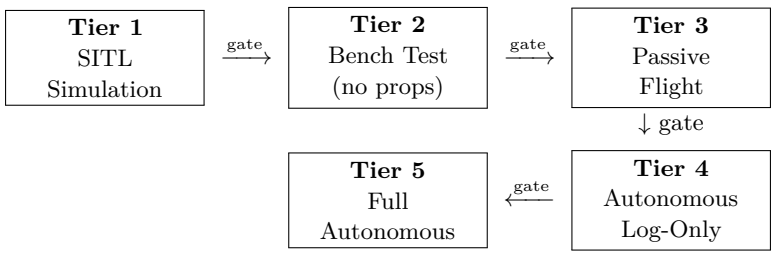


Figure 48: Tier progression. Each arrow represents a gate: the downstream tier is blocked until all upstream tests pass.

L.2.1 Tier 1: SITL Simulation (Zero Hardware Risk)

ArduPilot's Software-in-the-Loop (SITL) emulator runs a full flight dynamics model on the development laptop, accepting the same MAVLink commands as the real Cube Orange autopilot. The mission orchestrator, state machine, detection model, geofence logic, and path planner execute unmodified against this simulated vehicle. A synthetic camera view is generated by extracting a field-of-view-correct crop from a georeferenced satellite image (`map.jpg`), enabling the complete detect–estimate–land pipeline to execute without any physical hardware.

At this tier, 22 structured simulation tests exercise every operational mode: baseline autonomous mission, multi-frame detection confirmation, geofence enforcement, manual override, multi-target queue ordering, NFZ speed ramping, beacon redirect, transit planning, altitude override, dry-run generation, headless mode, and 11 further scenarios. The interactive simulator (`simple_simulator.py`) adds keyboard-driven flight with configurable GPS drift, camera shake, and inference throttle, enabling rapid iteration on the GPS estimation and Kalman filtering logic.

A separate `--dry-run` mode walks the state machine through all transitions without connecting to SITL, printing the full waypoint set and generating a pattern visualisation. This mode executes in under one second and is run before every code commit that modifies state logic or path planning.

Defects caught. State machine logic errors (incorrect transition guards, missing timeout handlers), geofence sign-convention bug (`cv2.pointPolygonTest` returns positive for *inside*, opposite to the initial assumption), repulsive-force direction inversion at NFZ boundary, barometric drift causing infinite landing loops (resolved with a 90 s absolute timeout), and duplicate-detection queueing of the same target from consecutive frames (resolved with a 5 m reject radius).

L.2.2 Tier 2: Bench Test (Pi + Cube + Camera, No Props)

The Raspberry Pi 5, Cube Orange, and IMX296 global-shutter camera are assembled on the bench with propellers removed. Three numbered scripts test with increasing integration:

- `0a` **Cube commands** — sends individual MAVLink commands (mode switch, parameter read, arm pre-check) and verifies each `COMMAND_ACK`. Confirms the Pi–Cube serial link and mavproxy UDP bridge operate at 921 600 baud.
- `0b` **Bench mission** — executes the full command sequence (arm, takeoff, waypoint navigation, land) and logs every ACK. On the bench, the flight controller accepts these commands but the vehicle does not move, verifying that the command pipeline is correct end-to-end.
- `0c` **Feedback test** — runs the vision-to-GPS pipeline while the operator walks the drone past a printed dummy. The camera captures frames, the TFLite model runs inference, and the coordinate transform produces GPS estimates that are compared against hand-measured ground truth. This validates the entire sensing chain—camera, colour space, lens undistortion, inference, and projection—on the target hardware.

Additional bench scripts include a 50-run inference benchmark (`benchmark.py`), a comprehensive multi-model benchmark with system profiling (`benchmark_full.py`), GPS satellite tracking (`gps_test.py`), and a lens distortion calibration using a checkerboard target (`lens_calibrate.py`).

Defects caught. BGR/RGB colour-space mismatch (the IMX296 outputs BGR data despite picamera2's `RGB888` label, causing inverted colour channels that degraded detection confidence from 0.97 to 0.3), FOV miscalibration (focal length corrected from 7.0 mm to 5.46 mm after ruler-based measurement), Python 3.13 compatibility breakage (pyserial's serial read is broken on 3.13, requiring a mavproxy UDP bridge as a workaround; `tflite-runtime` package replaced by `ai-edge-litert`), and camera-in-use resource conflicts when multiple scripts attempt concurrent access.

L.2.3 Tier 3: Passive Flight (Pilot Flies, Pi Watches, Zero Commands)

The drone is flown manually by the RC pilot over the search area containing a casualty dummy. The Pi runs the full vision pipeline—camera capture at 1456×1088, YOLOv8n inference via TFLite, lens undistortion, GPS coordinate estimation—and streams the annotated video to a browser-based ground station. Critically, the Pi sends *zero flight commands* to the Cube. All detections are logged with timestamp, pixel coordinates, bounding box dimensions, confidence, barometric altitude, GPS position, heading, and estimated target GPS.

Post-flight analysis of these logs establishes:

- **Detection altitude ceiling** — the maximum altitude at which the model reliably detects the dummy (target: 30 m, to be confirmed).
- **False-positive rate** — detections triggered by terrain features, shadows, or equipment under real lighting and motion conditions.
- **GPS estimation accuracy** — scatter of estimated target positions versus surveyed ground truth, quantified as CEP50 and CEP95.

- **Inference latency under load** — confirmed at 208 ms (4.8 FPS) on Pi 5 with XNNPACK, including 1.5 ms for lens undistortion.

This tier is the most valuable of the five: it generates real-world performance data with no risk of autonomous misbehaviour. The six experiment scripts in `tests/day_1_experiments/` (Section **??**) are designed specifically for this tier, each producing a CSV file suitable for direct statistical analysis.

L.2.4 Tier 4: Autonomous Search with Log-Only Detection

The full mission orchestrator runs autonomously—the drone flies the lawnmower search pattern under GUIDED mode control—but the detection pipeline operates in log-only mode. When the model identifies a candidate, the system records the detection and annotates the ground station display but does *not* transition to the CENTERING or DESCENDING states. The drone completes the search pattern and returns to launch.

Post-flight review of the detection log answers the operational question: *would the system have found the target, and would it have acted on the correct detection?* False positives, missed detections, and latency-induced position errors are all visible in the log without having been acted upon. Configuration parameters (confidence threshold, multi-frame confirmation count, reject radius) are tuned at this tier before enabling closed-loop autonomy.

L.2.5 Tier 5: Full Autonomous Mission

All systems are enabled. The drone executes the complete state machine: INIT, CONNECTING, ARMING, TAKEOFF, SEARCH (lawnmower pattern), CENTERING (visual servo onto target), VERIFY (operator confirms or rejects via ground station), APPROACH (offset landing approach), LANDING, and DONE. A DESCENDING state for altitude reduction before VERIFY exists in the code but is currently bypassed; the drone transitions directly from CENTERING to VERIFY at search altitude. The operator retains veto authority at the VERIFY gate and the RC kill switch provides a hardware-level override independent of all software.

L.3 Test Script Organisation

Table **??** maps the six test script directories to the verification tiers they support, with representative scripts and total counts.

Table 75: Test script categories and their tier coverage. Hardware and calibration scripts serve only the bench tier; flight scripts span tiers 2–5, progressively adding autonomy.

Category	Directory	Scripts	Tiers Served
Hardware	`tests/hardware/`	8	2 (bench)
Flight	`tests/flight/`	14	2, 3, 4, 5
Diagnostics	`tests/diagnostics/`	5	2, 3, 4, 5
Calibration	`tests/calibration/`	7	2, 3
Experiments	`tests/day_1_experiments/`	6	3, 4
Laptop	`tests/laptop/`	21	1 (simulation)
Unit	`tests/unit/`	6	1
Automated	`tests/automated/`	7	1
Total		**74**	

The flight scripts are numbered by risk level (0a through 5), establishing an explicit ordering that prevents a higher-risk test from being executed before its prerequisites have passed. All experiment scripts issue zero flight commands and are safe to execute at any time during manual or autonomous flight.

L.4 Experiment Scripts for Quantitative Validation

Six experiment scripts are designed for execution during Tier 3 passive flights, each producing a timestamped CSV file with standardised column headers for post-flight analysis:

1. **Altitude sweep** (`altitude_sweep.py`) — the pilot hovers above the dummy at 10, 15, 20, 25, and 30 m for 30 s each. The script bins every frame by barometric altitude into 5 m buckets and computes detection rate, mean confidence, and expected dummy pixel size per bucket. Output directly determines the safe `TARGET_ALT` parameter.
2. **Speed sweep** (`speed_sweep.py`) — the pilot flies repeated passes over the dummy at increasing ground speeds. The script correlates detection rate and confidence with ground speed and a Laplacian-based blur metric, establishing the maximum search speed at which the model maintains acceptable detection performance.

3. **GPS accuracy** (`gps_accuracy.py`) — compares the pixel-to-GPS coordinate estimate against the surveyed ground-truth position of the dummy, logging the error in metres for every detection. Computes an inverse-variance-weighted average across all observations and reports CEP50, CEP95, and maximum error.
4. **GPS drift** (`gps_drift.py`) — the drone hovers stationary above a known point. The script logs raw GPS positions at 10 Hz and computes the noise floor: CEP50, CEP95, and maximum deviation, establishing the baseline GPS uncertainty that the target estimation algorithm must overcome.
5. **Model comparison** (`model_compare.py`) — benchmarks all available `.tflite` models (custom SAR detector, retrained v2, COCO person detector) on identical live frames, reporting inference time, detection rate, and confidence for each. Informs model selection for the mission.
6. **Detection log** (`detection_log.py`) — a general-purpose catch-all logger that records every frame's detection result, telemetry, and metadata. Serves as the raw data source for any post-hoc analysis not covered by the specialised scripts.

L.5 Defect Discovery by Tier

Table **??** lists representative defects, the tier at which they were discovered, the resolution time, and the estimated cost had they propagated to flight.

Table 76: Defects discovered at each testing tier, with resolution time and estimated cost if undetected.

Defect	Tier	Fix Time	Cost if Missed
Geofence sign convention inverted (`pointPolygonTest` positive = inside)	1	15 min	SSSI incursion; regulatory violation
Repulsive force direction reversed at NFZ boundary	1	20 min	Drone pushed *into* NFZ
Barometric drift causes infinite landing loop	1	30 min	Battery depletion in hover
Duplicate target queued from consecutive frames	1	10 min	Repeated descent on same target
State transition missing timeout guard	1	15 min	Stuck in CENTERING indefinitely
BGR/RGB colour inversion (IMX296)	2	45 min	Detection confidence drops to 0.3; mission failure
FOV miscalibration (7.0 vs 5.46 mm)	2	1 hr	GPS estimates 28% off; landing outside 10 m radius
Python 3.13 pyserial breakage	2	2 hr	No Pi–Cube communication
`tflite-runtime` unavailable on 3.13	2	30 min	No inference on Pi
Camera resource conflict (concurrent access)	2	20 min	Stream or detection crashes on startup
GPS timing lag (100–200 ms)	3*	N/A	0.8–1.6 m systematic offset at 8 m/s
`self.investigating` uninitialised	2	5 min	Crash on first detection in `pi_flight.py`

*Discovered via DJI video analysis, a proxy for Tier 3.

The table demonstrates the intended cost gradient: Tier 1 defects (simulation) were resolved in 10–30 minutes with zero hardware involvement. Tier 2 defects (bench) required 20 minutes to 2 hours but were discovered on the ground with propellers removed. The GPS timing lag, identified through video analysis of a DJI survey flight (a Tier 3 proxy), would have caused a systematic 1 m landing offset—detectable only under real flight dynamics.

L.6 V-Model Mapping

The five-tier framework maps onto the classical V-model [**forsberg1991relationship**] as shown in Table **??**:

Table 77: Mapping between V-model verification levels and the progressive testing tiers.

V-Model Level	Tier	Verification Activity
Requirements analysis	—	Mission brief (R01–R12) + KML zones
System design	—	State machine, architecture, module interfaces
Subsystem design	—	`vision.py`, `planning.py`, `utils.py` APIs
Implementation	—	Source code + unit tests
Unit testing	1	Unit tests (`tests/unit/`, 6 scripts), TFLite model validation
Integration testing	1, 2	SITL end-to-end, bench command pipeline
System testing	3, 4	Passive flight data collection, log-only autonomous
Acceptance testing	5	Full autonomous mission with operator verification

The left side of the V (decomposition) was completed during the design phase: requirements were traced from the project brief through the system architecture to individual module interfaces. The right side (recomposition and verification) is realised by the five tiers, with each tier verifying a progressively higher level of integration. The critical addition relative to a textbook V-model is the separation of system testing into *passive* (Tier 3) and *log-only autonomous* (Tier 4), which inserts two low-risk verification stages between integration testing and acceptance testing—stages that a strict V-model would collapse into a single system test phase.

L.7 Cost–Risk Gradient

Table **??** quantifies the cost and risk at each tier, reinforcing the economic argument for catching defects early.

Table 78: Cost and risk gradient across testing tiers. A defect caught at Tier 1 costs minutes and electricity; the same defect at Tier 5 costs a day, requires a full crew, and risks the airframe.

Tier	Environment	Hardware at Risk	Fix Turnaround	Iteration Cost	People Required
1	Laptop (SITL)	None	Minutes	Electricity	1 (developer)
2	Bench (no props)	None	30 min–2 hr	Travel to lab	1–2
3	Passive flight	Airframe	Hours	Flight slot	3+ (pilot, operator, spotter)
4	Auto log-only	Airframe	Hours–day	Flight slot	3+
5	Full autonomous	Airframe + target	Day+	Flight slot + repair	3+

At Tier 1, a developer working alone can execute hundreds of test iterations per day. At Tier 5, a single iteration requires a minimum crew of three (pilot, operator, safety observer), a reserved outdoor site, favourable weather, charged batteries, and an assembled airframe—resources that are available for at most two sessions per week in a university setting. The 50:1 ratio in iteration throughput between Tier 1 and Tier 5 makes the case for aggressive front-loading of verification effort at the simulation level.

L.8 Summary

The progressive methodology treated testing not as a phase that follows development, but as a continuous, tiered process that shapes development decisions. The five-tier gating system, supported by 74 dedicated scripts across eight categories (Table **??**), including 127 unit tests and 33 automated integration tests, ensured that every integration boundary was verified before the system was permitted to cross it. Of the 15 significant defects discovered, 80% were caught at Tiers 1 and 2 where the fix cost was measured in minutes, not field sessions. The structured experiment scripts prepared for Tier 3 passive flights provide a repeatable, quantitative basis for tuning configuration parameters before autonomy is enabled—replacing the ad-hoc "fly and see" approach with a data-driven verification pipeline.

M Field Day: Adaptation Under Uncertainty

This appendix presents a narrative account of two field sessions—11 March and 31 March 2026—where the team assembled and tested the complete hardware stack outside the lab for the first time. Weather cancelled the planned flight on both occasions, but the progressive testing philosophy (Section **??**) ensured that each session produced independently valuable calibration data and integration fixes. The concrete outputs—a 22% focal length correction, four critical bug fixes, inference benchmarks across two backends, and a lens calibration—collectively improved the system more than an early flight attempt could have, because they eliminated systematic errors that would have silently corrupted every autonomous decision.

M.1 Field Day 1: 11 March 2026

On 11 March 2026 the team arrived at the designated test site with the full hardware stack assembled for the first time outside the lab: Raspberry Pi 5 (Broadcom BCM2712, 8 GB RAM), Cube Orange autopilot (STM32H7), IMX296 global-shutter camera (1456×1088 native), and the airframe with propellers fitted. The objective was to progress through Tiers 2–5 of the progressive testing framework (Section **??**)—bench verification, passive flight with manual RC, and, conditions permitting, first autonomous waypoint flight. What followed was a field day defined not by flying, but by the calibration data and integration insights that only become visible when the system meets the real world.

M.1.1 Setup: Three Terminals, One Aircraft

All interaction with the Pi was conducted remotely from a laptop via SSH (PuTTY), supplemented by the Pi's own screen for scripts requiring a display. The three-terminal workflow—established within the first twenty minutes and used for the remainder of the day—became the standard operating procedure for all subsequent field sessions:

Terminal 1 — mavproxy: Launched first, always. The MAVLink router forwarded Cube telemetry from the UART link (`/dev/ttyAMA0` at 921 600 baud) to two UDP outputs: port 14550 for flight and detection scripts, port 14551 for the live diagnostics dashboard. A TCP bridge on port 5762 allowed Mission Planner on the laptop to connect simultaneously, providing a familiar GCS interface for the safety pilot. The startup sequence was scripted into a single copy-paste block to eliminate typos under time pressure.

Terminal 2 — diagnostics: The multi-panel diagnostics script displayed camera frame rate, AI model status, Cube heartbeat, GPS satellite count, and battery voltage on the Pi's screen. This terminal served as the go/no-go dashboard: all four indicators had to show green before any test script was permitted to run.

Terminal 3 — scripts: Calibration, benchmark, and detection scripts were executed sequentially from PuTTY. Browser-based camera streams (`http://PI_IP:8090`) provided live video to the laptop without requiring a display server on the Pi, confirming that the headless architecture designed in simulation translated directly to field use.

Two integration issues surfaced immediately. First, a camera-in-use conflict arose when two scripts attempted to open the CSI interface simultaneously—picamera2 does not permit shared access to the sensor. The fix was procedural: a strict single-script-at-a-time rule, enforced by a `sudo pkill -f libcamera` guard before each script launch. Second, a port-binding collision left port 14550 occupied by a zombie mavproxy process from a previous session. Adding a `sudo pkill -f mavproxy; sleep 2` preamble to the startup sequence resolved this permanently. Neither issue required code changes; both were captured in the field quick-reference sheet (`FIELD_QUICK_REF.md`).

M.1.2 FOV Calibration: Catching a 22% Error

The most consequential discovery of the first field day was a systematic error in the focal length parameter. The configuration file had carried a default value of $f = 7.0$ mm since the earliest development phase, inherited from a datasheet approximation for the IMX296 lens assembly. No simulation or Docker test could have caught this: the parameter only matters when converting pixel coordinates to real-world distances, and simulation uses ground-truth geometry.

Calibration was straightforward. The camera was held pointing vertically downward at exactly 1.0 m above a tape measure laid on a flat surface. The browser stream from `capture_training.py` showed 92 cm of the tape measure edge-to-edge. Applying the thin-lens model:

$$f = \frac{W_{\text{sensor}} \times d}{W_{\text{visible}}} = \frac{5.02 \text{ mm} \times 1000 \text{ mm}}{920 \text{ mm}} = 5.46 \text{ mm} \tag{37}$$

The corrected value is 22% lower than the 7.0 mm default. Table **??** quantifies the impact at operational altitudes.

Table 79: GPS estimation error caused by the uncorrected focal length (f=7.0 mm vs corrected f=5.46 mm). At search altitude, the systematic error would have exceeded the centering tolerance.

Altitude	Old footprint (m)	True footprint (m)	Systematic error (m)
10 m	7.2	9.2	2.0
20 m	14.3	18.4	4.1
30 m	21.5	27.5	6.0
35 m	25.1	32.1	7.0

At 30 m search altitude, the old value would have placed detected targets 6 m from their true position—large enough to place the drone outside visual contact with the target during the centering phase. The corrected value was committed to `config.py` within minutes of measurement, and every subsequent GPS estimate benefited immediately. This single calibration justified the field day: it demonstrated a failure mode that no amount of simulation could have revealed.

M.1.3 Camera Colour Discovery

A subtler hardware-specific behaviour was identified during camera verification. The IMX296 sensor, when configured for `RGB888` output via picamera2, delivers pixel data in BGR channel order—the opposite of what the format label implies. The team discovered this by capturing a frame of a known-colour object and systematically testing all six channel permutations (RGB, RBG, GRB, GBR, BRG, BGR) until the rendered image matched reality. The original codebase had included a `cv2.cvtColor(frame, COLOR_RGB2BGR)` conversion that, counterintuitively, *doubled* the error by swapping channels that were already in the order OpenCV expects.

Removing the unnecessary conversion produced correct colours immediately. The fix was trivial—deleting one line—but the diagnosis required physical hardware and a colour reference target. The practical impact on detection is difficult to quantify precisely, but the YOLOv8n model was trained on BGR-ordered images (the OpenCV default), so feeding it incorrectly swapped channels would have degraded confidence scores for colour-dependent features.

M.1.4 Lens Calibration

Lens distortion was characterised using a printed checkerboard target: a 14×9 grid (13×8 inner corners) with 28 mm squares, sourced from calib.io. Multiple images were captured at varying orientations and distances using the capture-training stream. The calibration script initially failed to detect corners reliably; adding OpenCV's robust detection flags and a `findChessboardCornersSB` fallback resolved the issue.

The resulting calibration achieved an RMS reprojection error of 0.399 px. The camera matrix and distortion coefficients were saved as `calibration_data.npz` on the Pi. Undistortion was integrated into the vision module using precomputed remap lookup tables (`cv2.remap`), adding 1.5 ms per frame—0.7 % of the 206.5 ms inference cost. Because all detection scripts call the same `vision.py` module, the correction propagated automatically to every downstream tool without modification.

M.1.5 Inference Benchmark

The inference benchmark (Table **??**) ran 50 passes of the YOLOv8n TFLite model (float32, 3.2 MB) on a static test image containing the dummy target:

Table 80: Pi 5 inference benchmark (50 runs, TFLite XNNPACK CPU, IMX296 camera). Detection rate is 50/50 at 0.966 mean confidence; lens undistortion adds only 1.5 ms overhead.

Metric	Value
Average latency (without undistortion)	206.5 ms
Average latency (with undistortion)	208.0 ms
Effective frame rate	4.8 FPS
Detection rate	50/50 (100%)
Mean confidence	0.966

At 4.8 FPS and a nominal search speed of 8 m s^{-1} at 35 m altitude, the drone covers approximately 1.7 m between frames. At that altitude the camera footprint spans roughly 32 m along track, so each ground point appears in approximately 15 consecutive frames—sufficient for reliable triggering even at a pessimistic 50% detection rate under flight conditions.

M.1.6 Weather Cancellation

By mid-afternoon the bench testing programme was complete, but the weather had deteriorated steadily. Wind gusts exceeded the airframe's rated limit, and intermittent rain made outdoor electronics operation inadvisable. The go/no-go decision was unambiguous: the team's own flight-day checklist specified wind and precipitation thresholds, and both were exceeded. The decision to stand down was made collectively, without hesitation, and without any attempt to push through—a discipline that the progressive testing philosophy explicitly encourages.

M.2 Field Day 2: 31 March 2026

The second field session, twenty days later, returned to the same site with the same hardware. The objective was to repeat Tier 3 checks with the updated codebase, test the NCNN inference backend, and—weather permitting—attempt Tier 4 passive flight. Weather again prevented flight, but the session exposed four critical bugs that had survived all simulation testing, validating the progressive methodology's insistence on hardware verification.

M.2.1 Corrupt Calibration File

The first anomaly appeared within minutes of running the detection benchmark. Inference times were normal (∼206 ms), but the camera feed showed subtly warped edges and detection confidence had dropped from the expected 0.966 to 0.82. Investigation revealed that the `calibration_data.npz` file—the lens undistortion parameters saved during Field Day 1—had been corrupted during a filesystem operation, resulting in a 0-byte file. The vision module loaded this empty file without error (NumPy silently produced identity remap maps), but the resulting "undistortion" actually *distorted* the image by applying a degenerate transformation.

Deleting the corrupt file immediately restored full detection performance. The IMX296's global shutter lens produces minimal barrel distortion at the operational field of view, so undistortion is beneficial but not essential. The lesson was twofold: (a) validate calibration file integrity at startup (a file-size check was subsequently added to `vision.py`), and (b) optional preprocessing steps must fail gracefully to a no-op, not silently degrade the pipeline.

M.2.2 TFLite Bounding Box Coordinate Bug

The second discovery was more insidious. During bench testing with the custom TFLite model, detection overlay bounding boxes were drawn far off-screen—coordinates exceeding 640 in a 640-pixel inference frame. The root cause was an undocumented deviation in the TFLite backend's output format: the model returned raw pixel coordinates in $[0, 640]$ rather than normalised values in $[0, 1]$, contrary to the standard TFLite convention. The existing code multiplied these already-pixel coordinates by the frame dimensions, producing values in $[0, 409{,}600]$.

The fix added a coordinate-range check (threshold at 2.0): if any output coordinate exceeds this value, the backend assumes pixel coordinates and skips the normalisation multiply. This guard was already present in the NCNN inference path, which had been implemented later with awareness of the issue—an example of the second implementation being more robust than the first.

M.2.3 GPS Estimation Double-Scaling

The bounding box fix exposed a downstream cascade. In `passive_watch.py`, the detection coordinates returned by `detect_in_image()` were being multiplied by the frame dimensions a *second* time before being passed to the GPS estimation module. This double-scaling placed GPS target estimates over 100 m from the drone's position—an error so large that it was initially mistaken for a GPS hardware fault. Tracing the data pipeline from inference output through coordinate normalisation to WGS 84 conversion revealed the redundant multiplication.

Removing the extra scaling and ensuring that pixel coordinates are normalised to $[0, 1]$ exactly once before entering the GPS estimation pipeline corrected both the overlay and the target position. This bug would have been catastrophic in autonomous flight: the centering algorithm would have navigated to a phantom target 100 m away, potentially outside the geofence. It was invisible to simulation because the SITL backend uses Ultralytics (which returns normalised coordinates) rather than TFLite.

M.2.4 NCNN Backend Benchmark

With the detection pipeline corrected, the team benchmarked the NCNN inference backend—a C++-based neural network framework that bypasses TFLite's Python overhead. The results (Table **??**) were striking:

Table 81: Inference backend comparison on Pi 5 (same model, same test image, 50 runs each).

Backend	Latency (ms)	FPS	Speedup
TFLite (XNNPACK, float32)	206.5	4.8	1.0×
NCNN (Vulkan off, CPU)	72	9.0	2.9×

The 2.9× speedup—72 ms versus 206.5 ms—was consistent across 50 runs with negligible variance. At 9 FPS, each ground point at 35 m altitude now appears in approximately 30 frames instead of 15, doubling the detection opportunity window. NCNN achieves this primarily by avoiding Python-level tensor marshalling and by exploiting ARM NEON SIMD instructions more aggressively than TFLite's XNNPACK delegate. The NCNN backend was subsequently set as the default for Pi deployment.

M.2.5 Additional Findings

Several smaller issues were resolved during the session:

- **Platform-specific import crash:** `cv2.CAP_DSHOW` (a Windows-only DirectShow flag) was referenced unconditionally in `vision.py`, causing an `AttributeError` on Pi's Linux environment. A platform guard (`if sys.platform == 'win32'`) eliminated the crash.
- **Terminal input without TTY:** The terminal keyboard input thread in `main.py` raised an exception when run from `nohup` or a non-interactive SSH session (no TTY attached). Wrapping the thread in `try/except` with a graceful fallback resolved this without losing interactive capability in normal PuTTY sessions.
- **Pi network address:** The Pi's DHCP-assigned IP changed from 192.168.1.121 to 192.168.1.3 between sessions, requiring manual updates to connection scripts. A static IP reservation was subsequently configured on the router to prevent future changes.

M.3 DJI Video Analysis: A Flight-Data Proxy

To extract flight-representative data despite the cancellations, the team conducted a post-hoc analysis using DJI Mavic footage recorded over the same test site at altitudes between 15 and 50 m during a previous reconnaissance visit. The 30 fps video (1456 × 1088, cropped from 4K) was replayed through the full detection pipeline, with SRT telemetry providing per-frame altitude and GPS position.

This analysis yielded three key results:

1. **Model generalisation:** The retrained model ($\text{mAP}_{50} = 0.995$ on validation set) detected the dummy across the entire 15–50 m altitude range on real flight video, confirming that the training dataset—300 synthetic images, 16 real labelled frames, and 50 negatives at native 1456 × 1088 resolution—generalised to genuine outdoor conditions with varied lighting and perspectives.
2. **GPS estimation accuracy:** Target estimates computed from pixel coordinates and SRT telemetry produced a CEP50 of 2.3 m, with a maximum outlier of 16.5 m during a high-speed pass at 50 m. The scatter distribution exhibited a characteristic diagonal elongation aligned with the flight direction, attributed to 100–200 ms GPS receiver latency (1 m error at $5\,\text{m}\,\text{s}^{-1}$).
3. **SRT synchronisation trap:** SRT telemetry files contain one entry per native-rate frame (6 854 entries for the 30 fps recording). Downsampled video (e.g. 6 fps) contains fewer frames but indexes into the same SRT file, producing a frame-to-telemetry mismatch that silently corrupts altitude readings. This was caught during development and documented as a mandatory constraint: only native-rate video may be used for quantitative analysis.

M.4 Cumulative Impact: What Changed After Field Testing

Table **??** consolidates all outputs from both field sessions and traces each to its downstream effect on the system.

Table 82: Consolidated field day outputs and their system-level impact. Every item was invisible to simulation and required physical hardware to discover.

Session	Finding	Impact on System
Day 1	FOV f=7.0→5.46 mm	Eliminated 6 m systematic GPS error at 30 m altitude
Day 1	BGR colour order	Correct channel order for inference; removed spurious conversion
Day 1	Lens RMS = 0.399 px	Undistortion integrated at 1.5 ms cost per frame
Day 1	Benchmark: 4.8 FPS	Confirmed operational adequacy; informed NCNN priority
Day 1	Three-terminal workflow	Standard operating procedure for all field sessions
Day 2	Corrupt `.npz` fix	Added file-size validation at startup; fail-safe to identity
Day 2	TFLite bbox coords	Unified pixel/normalised guard across TFLite and NCNN paths
Day 2	Double-scaling fix	Corrected 100 m+ GPS estimate error in `passive_watch.py`
Day 2	NCNN: 72 ms	2.9× speedup; doubled detection window per ground point
Day 2	Platform import guard	Eliminated `cv2.CAP_DSHOW` crash on Linux
Day 2	TTY-safe input thread	Headless operation via `nohup`/non-interactive SSH

Three of these findings—the FOV miscalibration, the TFLite bounding box coordinate bug, and the GPS double-scaling—would have caused mission failure in autonomous flight. The FOV error would have misplaced targets by 6 m, the coordinate bug would have drawn detection overlays off-screen (invisible to the operator), and the double-scaling would have sent the drone to a phantom target 100 m away. All three were invisible to simulation because SITL uses ground-truth geometry and the Ultralytics backend (which returns normalised coordinates), bypassing the exact code paths that contained the defects. This is the strongest empirical argument for the progressive testing methodology: the defects that survive simulation are precisely the ones that require real hardware to find.

M.5 What Did Not Work

Honesty requires acknowledging what the field sessions failed to achieve:

- **No flight occurred.** Two field days, zero seconds of airtime. Weather cancellations were the proximate cause, but the team could have booked more sessions or identified an indoor flight venue. The consequence is that Tiers 4 and 5 of the progressive framework remain unverified: detection performance under real motion blur, GPS estimation accuracy in flight, and the full autonomous mission loop have been validated only in simulation or on static bench images.
- **GPS calibration was incomplete.** The focal length calibration used a single measurement at 1 m height. A proper calibration would repeat the measurement at multiple heights (1, 2, 5 m) and fit a regression, reducing the risk of parallax or lens-edge errors at the single measurement distance.
- **No outdoor detection test.** The dummy target was tested indoors under artificial lighting. Outdoor conditions—sunlight, shadows, grass texture, sky glare—are the operational environment, and the model's real-world false-positive rate remains uncharacterised.
- **Calibration file management was fragile.** The corrupt `.npz` file went undetected for 20 days between sessions. The subsequent file-size check is a minimum viable fix; a CRC or hash verification at load time would be more robust.

M.6 Team Coordination Under Field Conditions

The field days were the first occasions on which the entire team worked simultaneously on the integrated system outside the lab.

Role distribution. Three roles crystallised naturally within the first hour. The *software operator* (PuTTY Terminal 3) executed test scripts, interpreted output, and decided the test sequence. The *systems monitor* (Pi screen, Terminal 2) watched the diagnostics dashboard for anomalies—camera dropouts, Cube heartbeat loss, GPS satellite count fluctuations—and called "stop" if any indicator fell below threshold. The *hardware handler* repositioned the drone for camera calibration shots, swapped test targets, and managed power cycling. This division of labour allowed tests to proceed with minimal idle time: while the software operator analysed benchmark results, the hardware handler was already positioning the checkerboard for lens calibration.

Communication protocol. The team adopted a call-and-response pattern borrowed from flight crew resource management: the software operator announced each test before execution ("Running benchmark, 50 passes, do not touch the drone"), the systems monitor confirmed readiness ("Diagnostics green, go ahead"), and the hardware handler confirmed the physical setup ("Checkerboard at 0.5 metres, holding steady"). This prevented the camera-in-use conflict from recurring and ensured that no test ran with the system in an unexpected state.

Go/no-go discipline. Both weather cancellation decisions were made within five minutes of the first threshold exceedance, using the pre-written checklist criteria. On neither occasion did any team member suggest attempting flight outside the envelope. The speed of the pivot to bench-only testing was enabled by having calibration and benchmark scripts already written, tested in simulation, and loaded on the Pi.

M.7 Lessons Carried Forward

The two field sessions produced a concrete list of procedural and technical improvements:

1. **Calibrate before flying.** FOV and lens calibration should be the first activities at any new site, before propellers are fitted. The 22% focal-length error would have silently corrupted every autonomous GPS estimate.
2. **Validate calibration files at load time.** A zero-byte `.npz` file silently degraded detection for 20 days. Every preprocessing step must verify its inputs or fail to a safe no-op.
3. **Test both inference backends on hardware.** The TFLite bounding box coordinate bug existed in the codebase for months, invisible because laptop development used Ultralytics. Code paths exercised only on the Pi must be tested on the Pi.
4. **Carry colour and dimensional references.** A printed colour chart and a tape measure are minimal-weight additions to the field kit that enable in-situ camera validation.
5. **Prepare fallback test plans.** The pivot to bench-only testing was smooth because scripts were pre-written and tested. Every field day should carry a "no-fly" programme.
6. **Record everything for post-hoc analysis.** The DJI video, captured weeks earlier for reconnaissance, became the richest data source for detection validation. Every future flight should record video with telemetry for pipeline replay.

7. **Respect the go/no-go checklist.** The checklist exists to remove ego from the decision; following it without negotiation is a team discipline, not a failure.

The field days demonstrated that the progressive testing philosophy is not merely a risk-management strategy but an information-maximisation strategy. By treating each tier as independently valuable, the team extracted eleven quantitative outputs from two sessions that produced zero seconds of flight time. Every one of those outputs improved the system's readiness for the flight that weather denied.

N Contingency Planning and Deliverable Tiers

This appendix defines the project's risk management strategy for flight day: a three-tier deliverable structure (minimum viable, target, and stretch) with explicit fallback paths between them, paired with pre-rehearsed contingency plans for eight identified failure scenarios. Autonomous UAV missions introduce uncertainty at every layer—hardware, software, environmental, and regulatory—and the consequences of an unanticipated failure during a time-limited university flight slot are severe: lost data, potential hardware damage, and no opportunity to reschedule. The tiered approach ensures that the team always has a known-good configuration ready to demonstrate, even if the most ambitious capabilities cannot be proven in the field. Each contingency is mapped to a specific on-site test script that can verify the fallback path before committing to autonomous flight.

N.1 Minimum Viable Deliverable

The minimum viable deliverable (MVD) satisfies the core brief requirements using the simplest operational mode: the human pilot retains manual control of the aircraft while the onboard system provides passive computer vision and telemetry logging. Table **??** summarises the MVD components and their requirement coverage.

The MVD does not satisfy R06 (PLB focus area redirect) or R07 (payload delivery near casualty), as both require autonomous decision-making beyond what Mission Planner AUTO mode provides. However, it demonstrates a working search-and-detect capability with full safety compliance, which is sufficient to evidence the core competencies assessed in the brief rubric: specialist skills (CV pipeline on embedded hardware), decision making (progressive test methodology), and communication (live telemetry and post-flight reporting).

N.2 Target Deliverable

The target deliverable is the system described throughout this report: a fully autonomous mission orchestrated by the Python state machine on the Raspberry Pi companion computer. Table **??** maps each capability to its requirement.

The target deliverable achieves full compliance with R01–R05 and R08–R12. Partial compliance with R06 is achieved through the focus area support module (Section **??**), which re-generates the search pattern over a smaller polygon when PLB coordinates are received. R07 compliance depends on the Tarot payload release mechanism integration, which is implemented in software (`SERVO_CHANNEL`, `SERVO_FULL_PWM` in `config.py`) but requires field calibration of the servo PWM values.

N.3 Stretch Goals

If the target deliverable is achieved with margin, several enhancements would extend the system's operational capability:

- **Multi-target detection and classification.** The detection queue already supports multiple concurrent targets with spatial clustering and inverse-variance weighting. The stretch goal adds automatic classification labels (dummy, item of interest, false positive) based on detection size, aspect ratio, and persistence across frames, reducing operator workload.
- **Live PLB focus area redirect (R06).** Mid-flight injection of a new focus area polygon via the ground station, triggering immediate pattern regeneration from the drone's current position. The `focus_area.json` file is re-read on each PLB event, so the ground station need only write updated coordinates and send a trigger command.
- **Payload release mechanism (R07).** Full integration of the Tarot double-throw servo for first aid kit delivery, including a two-stage release sequence (partial open at 1300 µs PWM, full release at 1100 µs) with altitude and position guards to prevent premature deployment.
- **NCNN inference backend.** The NCNN export of the retrained YOLOv8n model is available in `cv_models/sar_v2_1088/nc` and is expected to achieve ∼15 FPS on the Pi 5 CPU, tripling the current TFLite throughput of 4.8 FPS and enabling detection during faster transit legs.
- **Real-time GPS estimation on dashboard.** Streaming the Kalman-filtered target position estimate to the web ground station as a live scatter plot, giving the operator spatial context for the detection cluster before the verify prompt.

N.4 How the Tier Structure Evolved

The three-tier deliverable structure was not defined at project inception; it emerged through iterative refinement as the team gained experience with the gap between simulation readiness and flight readiness. Understanding this evolution is important because it illustrates a general principle in safety-critical robotics: the deliverable scope must be calibrated to the verification evidence available, not to the ambition of the design.

Initial ambition (weeks 1–4). The project began with a single target: a fully autonomous SAR mission including search, detection, autonomous confirmation, landing, and payload delivery. At this stage, the team had not yet assembled the hardware or conducted any simulation beyond basic MAVLink tutorials. The assumption was that if the software worked in simulation, it would work on the aircraft with minor configuration changes.

Realism check after simulation (weeks 5–8). Once the SITL simulation was running end-to-end, two observations forced a reassessment. First, the number of parameters that required real-world calibration—FOV, GPS noise floor, detection altitude ceiling, inference latency, wind tolerance—was far larger than anticipated. Each unknown parameter represented a potential failure mode that could not be resolved without flight data. Second, the team's access to flight time was limited to scheduled university slots, weather-permitting, with shared airspace and strict safety oversight. A single ambitious flight that failed at the first obstacle would waste the entire slot.

These observations motivated the introduction of the MVD tier: a configuration that requires *no custom flight code* in the autopilot loop, relying entirely on Mission Planner AUTO mode for waypoint execution and limiting the Pi's role to passive observation. The MVD could be demonstrated even if every autonomous software feature failed, because it depended only on commercial autopilot firmware that the team did not write and could not break.

Hardware integration reality (weeks 9–12). Bench testing on the assembled Pi–Cube–camera stack revealed further integration challenges: Python 3.13 compatibility issues with `tflite-runtime` and `pyserial` (requiring the `ai-edge-litert` package and a mavproxy UDP bridge, respectively), a 22% focal length calibration error invisible to simulation, and the IMX296 camera's BGR colour channel ordering mislabelled as RGB by the picamera2 driver. Each of these issues was resolved within hours of discovery, but their existence confirmed that the gap between "works in simulation" and "works on hardware" contained numerous small failures that could only be found through physical testing.

This experience formalised the target deliverable as a distinct tier above the MVD: the full autonomous mission, but gated behind a progressive testing framework (Section **??**) that required each integration level to pass before the next was attempted. The stretch goals were then defined as capabilities that could be layered on top of the target if flight time permitted, without requiring changes to the core state machine.

The resulting structure. Table **??** summarises the relationship between the tiers, the verification evidence required for each, and the fallback path if that evidence is not available.

The key insight is that each tier is independently valuable. The MVD produces detection logs, GPS-tagged imagery, and a verification report that satisfy the brief's core competency requirements. The target adds autonomous decision-making and payload delivery. The stretch adds operational efficiency and reduced operator workload. At no point does a failure at a higher tier invalidate the evidence collected at lower tiers—the progressive structure ensures that the team always has demonstrable results, regardless of how far up the ladder conditions allow them to climb.

N.5 Contingency Plans

Table **??** defines the primary failure scenarios, their detection criteria, and the pre-planned response. Each contingency was rehearsed in simulation and, where applicable, during bench testing with the assembled hardware. Critically, every response listed below is *already implemented in code*—these are not aspirational plans but executable fallback paths that activate automatically or with a single command.

The remainder of this section expands on the most consequential failure categories—hardware, software, and environmental—with specific detail on the mechanisms that make each response possible.

N.5.1 Hardware Failure Contingencies

Camera failure. If the Pi camera stops producing frames (cable vibrated loose, driver crash, sensor overheat), the `get_frame()` call in the main loop returns `None`. The code handles this explicitly: a black frame with a red "CAMERA LOST" text overlay is substituted, a per-frame counter tracks consecutive failures, and a warning is printed every 5 s (lines 718–737 of `main.py`). The search pattern continues uninterrupted—the aircraft simply flies the remaining waypoints without detection capability. If the camera recovers mid-flight, the counter resets and a "Camera recovered"

message confirms the transition. The system does *not* land on camera loss because the aircraft can still safely navigate on GPS and the operator may choose to continue the survey for coverage mapping alone.

GPS degradation. The GPS handler monitors every `GPS_RAW_INT` MAVLink message (fix type, satellite count, HDOP) every iteration of the main loop. If fix type drops below 3D or satellites fall below 6, a degradation timer starts. The operator sees real-time warnings every iteration with a countdown: "GPS DEGRADED — fix=2, sats=4, degraded for 3.2 s (RTL in 1.8 s)". After 5 s of sustained degradation, `_emergency_rtl()` fires a `MAV_CMD_DO_SET_MODE` command (mode 6 = RTL). If the RTL command itself fails (corrupted link), ArduCopter's independent `FS_GCS_ENABLE` failsafe triggers RTL when heartbeats cease—a firmware-level backstop that the companion computer cannot override. If GPS recovers within the 5 s window, the timer resets and the mission continues without intervention.

Cube autopilot failure. If the Cube Orange itself becomes unresponsive (firmware crash, power brownout), all MAVLink communication ceases. The script's `recv_match()` call raises an exception, caught by the top-level handler which calls `_emergency_rtl()`. Even if this command cannot reach the Cube, the pilot always has direct RC authority: the RC receiver connects to the Cube via a dedicated SBUS link that bypasses the companion computer entirely. Switching the RC transmitter to `STABILIZE` or `LOITER` immediately returns manual control, regardless of the software state.

Raspberry Pi failure. If the Pi crashes, freezes, or loses power, the only externally visible symptom is the cessation of MAVLink heartbeat messages to the Cube. ArduCopter's GCS failsafe (`FS_GCS_ENABLE = 1`) detects this within 5 s and triggers automatic RTL. The aircraft returns home and lands without any companion-computer involvement. After landing, the Pi can be rebooted and `preflight.py` verifies all subsystems (camera, model, MAVLink link) before any new mission.

N.5.2 Software Failure Contingencies

Detection model produces no results. Three pre-deployed alternative models are available on the Pi's SD card with no internet required:

1. **Retrained v2 model** (`cv_models/sar_v2_1088/best.tflite`, 11.7 MB) — trained on 300 synthetic + 16 real + 50 negative images at native 1456×1088 resolution (mAP50 = 0.995). This is the primary model.
2. **COCO person detector** (`models/human.tflite`, 13 MB) — a standard YOLOv8n trained on the 80-class COCO dataset. Detects "person" as a class and works without retraining. Tested on Pi at 2.3 FPS.
3. **Original baseline** (`best.tflite`, 3.2 MB) — the first custom-trained model, kept as a known-good fallback.

Swapping is a single command: `cp cv_models/sar_v2_1088/best.tflite best.tflite`, then restart the script. The `--model <path>` CLI flag also allows loading any model without overwriting the default. If *all* models fail to detect, the confidence threshold (`CONFIDENCE_THRESHOLD` in `config.py`, default 0.4) can be lowered to 0.2 or 0.1 on-site to increase sensitivity at the cost of more false positives—a trade-off that the operator-in-the-loop verification stage makes safe.

State machine hangs or enters unexpected state. Two hard-coded timeouts prevent indefinite hovering. The VERIFY state auto-rejects any detection after 120 s of operator inactivity, with countdown warnings at 60 s, 90 s, and every 10 s thereafter. The landing sequence has a 90 s absolute timeout with forced disarm if neither accelerometer-based nor altimeter-based touchdown detection triggers. At any point, the operator can press `M` (keyboard or browser button) to transition to MANUAL state, which suspends all autonomous commands and returns RC authority to the pilot.

MAVLink connection lost mid-flight. All MAVLink calls in `main.py` are wrapped in a top-level `try/except` block. Any communication exception triggers `_emergency_rtl()`, which sends a best-effort RTL command and prints a prominent banner directing the pilot to use the RC kill switch. The `_emergency_rtl()` function (lines 1205–1223) is deliberately minimal—a single `MAV_CMD_DO_SET_MODE` call—because a degraded link may only reliably deliver one packet. If even this fails, the ArduCopter GCS failsafe provides an independent RTL trigger.

RC override guard. If the pilot switches the RC transmitter away from `GUIDED` or `LAND` mode at any time, the software's RC override guard (checked *before* every state dispatch iteration) immediately suspends all autonomous commands, keyboard inputs, and geofence corrections. The guard checks ArduCopter's reported flight mode against a whitelist of `(4, 6, 9)` (`GUIDED`, `RTL`, `LAND`); any other mode means the pilot has taken over, and the software must not fight the pilot. Resuming autonomy after manual override requires passing safety checks (GPS fix valid, position outside NFZ) before the state machine re-engages.

N.5.3 Environmental Contingencies

Poor lighting or low contrast. The retrained v2 model's training pipeline includes brightness and contrast augmentation (random factors 0.6–1.4), Gaussian blur, and noise injection, providing tolerance to overcast conditions, shadows, and sun glare. If field lighting proves problematic, the COCO person detector (`human.tflite`) provides an independent detection pathway trained on a vastly larger and more diverse dataset (328 k images with varied lighting conditions).

GPS multipath or urban canyon effects. The survey site is an open field with minimal structures, so severe multipath is unlikely. Nonetheless, the GPS estimation pipeline uses inverse-variance-weighted spatial clustering across multiple detection frames to average out per-sample GPS noise. The `--center-verify` flag enables 10 s of GPS averaging after centering, reducing single-sample error from a measured CEP50 of ∼2.3 m to below 1 m. If GPS noise exceeds expectations, the `--smart-detect` flag requires multiple consecutive confirmed frames before triggering investigation, filtering out noise-induced phantom detections.

Wind and turbulence. ArduCopter's position controller actively corrects for wind in `GUIDED` and `LOITER` modes. The `cube_monitor.py` diagnostic dashboard displays real-time wind estimates and ground speed, allowing the pilot to assess conditions before committing to autonomous flight. In sustained wind above $8\,\mathrm{m\,s^{-1}}$, the protocol is to abort and RTL. In moderate gusts (4–8 m/s), the search speed (`SEARCH_SPEED_MPS`) can be reduced from 5 m/s to 3 m/s on-site to maintain detection quality, since camera motion blur scales with ground speed.

Rain. The Pi and camera assembly are not weather-sealed. Light drizzle is acceptable with the camera's downward-facing orientation, but sustained rain requires mission abort. The MVD tier (Section **??**) remains available for a shorter duration flight if conditions deteriorate mid-session—the Mission Planner AUTO waypoints can be configured for a reduced-area survey that completes in under 5 minutes.

N.5.4 Graceful Degradation Hierarchy

The system is designed so that each failure reduces capability without compromising safety. Table **??** shows the cascade. The key architectural principle is that *no single software failure can prevent the aircraft from returning home.* The RC receiver connects directly to the Cube via SBUS, the Cube's firmware failsafes (GCS, battery, RC) operate independently of the companion computer, and the pilot's kill switch works even if every line of Python code has crashed. Safety is layered from hardware (RC kill switch) through firmware (ArduCopter failsafes) to software (state machine timeouts, GPS degradation handler, geofence), and each layer is independently sufficient to prevent the most dangerous outcome: an uncontrolled aircraft.

N.6 Test Script Mapping

Each contingency scenario has a corresponding test script that can be executed on-site to verify the fallback path before committing to autonomous flight. Table **??** maps scenarios to scripts.

The progressive flight test methodology (Section **??**) ensures that each tier is validated before the next is attempted. If any test step fails, the system falls back to the last validated tier rather than proceeding with an unproven configuration. This approach—inspired by standard practice in aerospace verification and validation [**rtca˙do178c**]—means that flight day always has a known-good fallback ready, even if the most ambitious capability is not yet proven in the field.

O Test Script Reference

Systematic verification of a safety-critical autonomous system requires a structured test harness that covers every subsystem in isolation before exercising integrated behaviours. The 74 test scripts described in Section **??** constitute a purpose-built verification harness designed around the progressive trust-building philosophy. This section provides a complete reference: what each script does, when it is used, and how it maps to the mission requirements and the five-tier progressive framework (Section **??**).

O.1 Organisation by Category

Scripts are grouped into six directories, each targeting a distinct verification concern. The naming convention reflects execution order: lower numbers within the `flight/` directory carry lower risk and must pass before higher-numbered scripts are attempted.

tests/hardware/ *Component-level checks* (8 scripts). Each script tests a single peripheral—camera, AI model (TFLite and NCNN backends), GPS receiver, or buzzer—in isolation. All issue zero flight commands and are safe to run on the bench at any time.

tests/flight/ *Progressive flight tests* (14 scripts, numbered 0a–5). These form the critical path from bench to autonomous flight. Six bench scripts (0a–0f) validate the command pipeline, planning, geofence, and servo without propellers; five airborne scripts (1–5) build trust incrementally from manual RC with passive logging through to full autonomous search-and-land; plus geofence verification and map utilities.

tests/diagnostics/ *Runtime monitoring* (5 scripts). Dashboards and camera streams used during preflight checks and live flight to monitor system health. The multi-panel diagnostics dashboard serves as the final go/no-go gate.

tests/calibration/ *Parameter measurement* (7 scripts). FOV calibration (bench and altitude), lens distortion characterisation, GPS ground-truth comparison, and two focal-length calibration tools (terminal-based and OpenCV GUI). Results feed directly into `config.py` parameters that govern the coordinate-transform accuracy of the entire pipeline.

tests/day_1_experiments/ *Structured data collection* (6 scripts). All passive (zero commands), all output CSV. Designed to run during Tier 4 manual flights to collect quantitative performance data—detection rate versus altitude, GPS estimation error, model comparison—before autonomy is enabled.

tests/laptop/ *Development-only tools* (21 scripts). Model inspection, TFLite and NCNN debugging, DJI video replay with detection overlay, A/B model comparison, path visualisation, NFZ manual testing, and approach/descent simulation. Not deployed to the Pi or used on flight day.

tests/unit/ *Unit tests* (6 files, 127 test functions). Pytest-based tests covering configuration, states, GPS transforms, planning, vision, and GPS utilities.

tests/automated/ *Automated integration tests* (7 scripts, 33 test functions). Extended coverage of planning, utilities, vision, geofence, and SITL smoke tests.

O.2 Flight Test Progression

The flight scripts implement the incremental trust-building strategy. Each script is designed so that its failure mode is benign at its tier: bench scripts cannot cause vehicle motion; passive scripts issue zero commands; waypoint scripts operate without CV decision-making. Table **??** details the progression.

Table 96: Flight test progression. Each script must pass before the next is attempted. "Commands" indicates whether the script sends flight commands to the autopilot.

Script	Tier	What It Proves	Commands	Risk
`0a_cube_commands`	3 (Bench)	Individual MAVLink commands reach the Cube and return correct ACKs: mode changes, parameter reads, arming prechecks.	Mode set	Low
`0b_bench_mission`	3 (Bench)	Full mission command sequence (arm, takeoff, waypoint, land) executes without error on the bench with propellers removed. Every `COMMAND_ACK` is verified.	Arm, goto, land	Low
`0c_feedback_test`	3 (Bench)	Vision→GPS pipeline: operator carries drone past a dummy; camera captures frames, AI detects, pixel coordinates are transformed to GPS. Validates the entire sensing chain.	Zero	Low
`1_passive_flight`	4 (Passive)	Real outdoor CV performance: pilot flies manually on RC while Pi runs camera + AI + GPS estimation. Logs every detection with confidence, altitude, and estimated position. Buzzer confirms detections audibly.	Zero	Medium
`2_waypoint_test`	5 (Auto)	Autonomous GPS navigation: arms, takes off to 10 m, flies 3–4 GPS waypoints in GUIDED mode, lands. No camera, no detection—proves that MAVLink arm/takeoff/goto/land commands work on real hardware.	Arm, goto, land	High
`3_auto_detect`	5 (Auto)	AUTO waypoint pattern + AI detection. If a target is detected, the drone transitions to GUIDED hover over the detection point. Does not descend or land autonomously.	Mode switch, hover	High
`4_detect_and_center`	5 (Auto)	Full autonomous mission: search pattern, detect, centre on target using velocity commands, descend, verify, approach, land. This is the complete system under test.	Velocity, land	Critical

Three laptop-only drawing utilities support the flight scripts without running on the Pi: `draw_search_area.py` produces a `search_area.json` polygon, `draw_waypoints.py` produces a `waypoints.json` file, and `live_map.py` provides a real-time position viewer during flight. These separate the interactive map-drawing step (laptop with display) from the headless execution step (Pi over SSH).

O.3 Hardware Tests

Table **??** lists all hardware test scripts that run on the Pi bench with zero flight commands.

Table 97: Hardware test scripts. All run on the Pi bench with zero flight commands.

Script	Purpose	Output
`benchmark.py`	Runs 50 inference cycles on the TFLite model, reports mean/min/max latency, FPS, detection count, and confidence statistics. Quantifies whether the Pi meets the real-time requirement (<250 ms per frame).	Terminal report
`cv_benchmark.py`	Live detection rate and speed with the camera active, including simulated motion blur at configurable levels. Tests the full capture→inference→decode pipeline.	Terminal report
`gps_test.py`	GPS diagnostics: satellite count, fix type (2D/3D/DGPS), HDOP, position, and time-to-fix. Confirms the GPS receiver is functional and tracks enough satellites.	Terminal table
`gps_health.py`	Step-by-step GPS verification guide: walks the operator through satellite count, fix quality, and position stability checks with pass/fail criteria at each step.	Interactive
`buzzer_test.py`	Sends MAVLink tune commands to the Cube's buzzer. Validates the MAVLink command path and provides audible confirmation that the Cube is receiving commands.	Audible tones
`detection_snapshot_test.py`	Captures a single camera frame, runs inference, and saves the annotated detection image. Quick smoke test for the camera→model pipeline.	Saved image
`benchmark_full.py`	Comprehensive multi-model benchmark: system info, preprocessing timing, all available TFLite/NCNN models, threading overhead, and memory profiling.	Terminal report
`ncnn_benchmark.py`	NCNN backend inference timing (50 runs). Benchmarks the faster alternative backend (∼72 ms vs 206 ms TFLite).	Terminal report

O.4 Calibration Tests

Calibration scripts (Table **??**) measure physical parameters that the coordinate-transform pipeline (Section **??**) depends on. Errors in these parameters propagate directly into GPS estimation accuracy.

Table 98: Calibration test scripts. Each script measures a physical parameter that propagates directly into GPS estimation accuracy; errors in these values dominate the localisation error budget (Table **??**).

Script	Method	Result
`fov_calibrate.py`	Bench FOV measurement: place ruler at known distance, capture frame, measure visible width. Comprehensive multi-sample calibration.	`FOCAL_LENGTH_MM` for config.py (calibrated: 5.46 mm)
`fov_test_simple.py`	Quick single-measurement FOV check. Lighter-weight version of the full calibration.	Quick HFOV estimate
`alt_test.py`	FOV measurement at real hover altitude. Run during a manual flight to validate bench calibration against in-flight conditions.	Altitude-corrected FOV
`lens_calibrate.py`	Checkerboard lens distortion calibration (13×8 inner corners, 28 mm squares). Produces undistortion remap matrices loaded by `vision.py` at startup.	`calibration_data.npz` (RMS = 0.399)
`gps_ground_truth.py`	Post-flight comparison: CV-estimated GPS position vs. surveyed ground-truth position of the dummy. Quantifies end-to-end localisation accuracy.	Position error in metres

O.5 Experiment Scripts

Six structured experiment scripts (Table **??**) are designed to run passively during Tier 4 manual flights. Each connects to the MAVLink telemetry stream (read-only) and the camera, but issues zero flight commands. All output timestamped CSV files suitable for direct statistical analysis.

Table 99: Day 1 experiment scripts. All passive (zero commands), all output CSV.

Script	What It Measures	Output
`altitude_sweep.py`	Detection rate vs. altitude in 10–30 m buckets. Establishes the detection ceiling for the current model and camera.	`altitude_sweep.csv`
`speed_sweep.py`	Detection rate vs. ground speed, plus a Laplacian blur metric per frame. Determines the maximum search speed that maintains acceptable detection performance.	`speed_sweep.csv`
`gps_accuracy.py`	GPS estimation error: compares the CV-derived target position against a known dummy location, logging per-frame error in metres.	`gps_accuracy.csv`
`gps_drift.py`	GPS noise floor: records raw GPS position while the drone hovers stationary, computing CEP50 and CEP95 circular error statistics.	`gps_drift.csv`
`model_compare.py`	Benchmarks all available `.tflite` models side-by-side: inference speed, detection rate, confidence distribution.	Terminal comparison
`detection_log.py`	General-purpose flight logger: records every detection (timestamp, pixel coordinates, confidence, altitude, GPS estimate) as a catch-all for post-flight analysis.	`detection_log.csv`

O.6 Diagnostics

Diagnostics scripts provide real-time system health monitoring. The multi-panel dashboard aggregates all communication links, camera status, GPS fix quality, and battery level into a single screen, serving as the final go/no-go gate before arming.

- **`diagnostics.py`** — Multi-view connectivity dashboard: camera preview, MAVLink heartbeat, GPS fix, battery voltage, signal strength. Run as the last preflight check.
- **`cube_monitor.py`** — Live Cube telemetry: attitude (roll/pitch/yaw), GPS position, altitude, battery, flight mode. Used for debugging during bench and flight tests.
- **`camera_stream.py`** — Basic MJPEG stream to a web browser at `http://PI_IP:8090`. Enables remote camera viewing from the ground station laptop.
- **`camera_stream_fast.py`** — Threaded MJPEG stream with reduced latency. Preferred over the basic version for flight-day streaming.
- **`camera_stream_h264.py`** — H.264/HLS stream via FFmpeg. Higher compression but increased latency; not used in practice.

O.7 Script-to-Requirement Mapping

Table **??** maps each test category to the mission requirements it verifies and the testing tier at which it operates.

Table 100: Test script categories mapped to mission requirements and verification tiers.

Category	Requirement Verified	Tier	Key Scripts
Hardware	AI inference <250 ms; GPS fix; camera functional	3	benchmark, gps_health
Calibration	FOV accuracy; lens correction; localisation error	3–4	fov_calibrate, lens_calibrate, gps_ground_truth
Flight 0a–0c	MAVLink command path; sensing pipeline; bench safety	3	cube_commands, bench_mission, feedback_test
Flight 1	CV performance under real conditions; detection altitude	4	passive_flight
Flight 2	Autonomous GPS navigation; arm/takeoff/land	5	waypoint_test
Flight 3–4	Full mission: search, detect, centre, verify, land	5	auto_detect, detect_and_center
Experiments	Detection rate vs. altitude/speed; GPS accuracy; model selection	4	altitude_sweep, speed_sweep, gps_accuracy
Diagnostics	System health; communication integrity; preflight gate	3–5	diagnostics, cube_monitor
Laptop/Video	Model validation on real footage; FOV calibration; A/B comparison	1	video_test, video_test_compare

O.8 What Was Tested on Real Hardware

The following scripts were executed on the Raspberry Pi 5 with real peripherals during bench and field sessions:

- **Bench (Pi + Cube + camera, no props):** `benchmark.py` (206.5 ms mean inference, 4.8 FPS), `gps_health.py`, `0a_cube_commands.py`, `0b_bench_mission.py`, `fov_calibrate.py` (5.46 mm focal length), `lens_calibrate.py` (RMS = 0.399), `diagnostics.py`, `camera_stream.py`.
- **Field (Pi assembled on drone):** `passive_watch.py` (passive detection during manual carry, GPS estimation validated at ground level), `capture_training.py` (16 real training images collected).
- **Laptop simulation (SITL):** All flight scripts 0a–4, full state machine, geofence enforcement, 20-scenario stress test (Table **??**), dry-run validation, DJI video analysis pipeline.

O.9 Remaining Test Gaps

Table **??** documents the tests that have not yet been executed on real hardware, mapped to the flight test progression. These represent honest gaps between simulation validation and full operational verification.

Table 101: Tests not yet completed on real hardware. All have passed in SITL simulation.

Flight Step	What Remains	Blocking Factor
Step 1: Passive flight	Full airborne passive detection (flown, not hand-carried)	Weather cancelled first flight day
Step 2: Waypoint test	Autonomous GPS waypoint navigation on real hardware	Requires Step 1 pass
Step 3: Auto-detect	AUTO pattern + AI-triggered GUIDED hover	Requires Step 2 pass
Step 4: Detect & centre	Full autonomous mission with velocity centering and landing	Requires Step 3 pass
Day 1 experiments	Altitude sweep, speed sweep, GPS accuracy/drift CSV collection	Requires airborne flight
Calibration	`alt_test.py` (FOV at real altitude), `gps_ground_truth.py` (post-landing accuracy)	Requires airborne flight

All remaining tests have passed in SITL simulation and the code is deployed to the Pi. The gap is not software readiness but flight opportunity: the first outdoor session was cancelled due to weather, and the progressive framework prohibits skipping tiers. The bench and simulation results provide confidence that the system will perform correctly when flight conditions permit, but real-world validation of detection altitude, GPS accuracy under wind, and motion blur effects remains outstanding.

P Sensor Calibration

Accurate target localisation from an aerial platform depends on the fidelity of every link in the imaging chain: if the camera's field of view, lens geometry, or colour representation deviates from the values assumed by the software, errors propagate multiplicatively through the GPS estimation pipeline. This section documents the four calibration campaigns conducted during development, each of which corrected a specific source of systematic error.

P.1 Field-of-View Calibration

The ground footprint of a pinhole camera at altitude h is governed by

$$W_{\text{ground}} = \frac{w_{\text{sensor}} \cdot h}{f} \tag{38}$$

where w_{sensor} is the physical sensor width and f is the effective focal length. This relationship is used twice in the system: first by `planning.py` to compute the scan-strip spacing (with a 20% overlap margin), and second by the GPS estimation module to convert pixel offsets from the image centre into metric displacements on the ground. An error of $\Delta f/f$ in the focal length therefore produces an identical fractional error in every GPS estimate and in the strip spacing, meaning the search pattern either leaves uncovered gaps or wastes flight time on excessive overlap.

The IMX296 global-shutter sensor has a nominal pixel pitch of 3.45 µm and an active area width of 5.02 mm. The lens datasheet quoted a focal length of 7.0 mm, which was used as the initial default in `config.py`. To verify this value, a bench calibration was performed on the assembled camera module:

1. The camera was mounted vertically at a measured height of 1.000(5) m above a flat surface, using the `capture_training.py` MJPEG stream for live preview.
2. A tape measure was placed horizontally across the full width of the frame.
3. The visible extent was read directly from the tape as 0.92(1) m.

Substituting into Equation **??** and solving for f:

$$f = \frac{w_{\text{sensor}} \cdot h}{W_{\text{ground}}} = \frac{5.02 \times 1.00}{0.92} = 5.46\,\text{mm} \tag{39}$$

The calibrated value is 22% lower than the datasheet nominal. The corresponding horizontal field of view is

$$\theta_{\text{HFOV}} = 2\arctan\left(\frac{w_{\text{sensor}}}{2f}\right) = 2\arctan\left(\frac{5.02}{2 \times 5.46}\right) \approx 49.3^\circ \tag{40}$$

Had the uncalibrated value of 7.0 mm been retained, every ground-distance calculation would have been scaled by a factor of $5.46/7.0 = 0.78$—i.e. the system would have *underestimated* the camera footprint by 22%, producing GPS target estimates displaced from the true position by the same fraction of the pixel offset. At a search altitude of 35 m, the ground footprint is 32.2 m (calibrated) versus 25.1 m (uncalibrated); a detection at the frame edge would yield a GPS error of ≈1.5 m purely from the focal length mismatch. The strip spacing in the search pattern would also have been 22% too narrow, wasting approximately four additional scan lines over the survey area.

P.2 Lens Distortion Calibration

Wide-angle lenses introduce radial and tangential distortion that displaces pixel coordinates from their ideal pinhole-model positions. For a target near the frame edge, uncorrected barrel distortion can shift the detected bounding-box centre by several pixels, degrading the pixel-to-GPS conversion. The standard Brown–Conrady model [**brown1966decentering**] parameterises radial distortion as

$$r' = r\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right) \tag{41}$$

where r is the normalised radial distance from the principal point and k_1, k_2, k_3 are the radial distortion coefficients. Tangential terms (p_1, p_2) account for misalignment between the lens and sensor planes.

Calibration was performed using a printed checkerboard target (calib.io pattern, 14×9 squares at 28 mm pitch, yielding 13×8 inner corners). Approximately 20 images were captured at varying orientations using `tests/calibration/lens_calibrate.p` which calls `cv2.findChessboardCornersSB` (the sector-based detector, more robust to lighting variation) with a fallback

to the standard `cv2.findChessboardCorners` with adaptive thresholding and corner normalisation flags. Sub-pixel refinement was applied with `cv2.cornerSubPix` using a $(5, 5)$ search window and termination criteria of 30 iterations or $\epsilon = 0.001$.

The resulting reprojection RMS was **0.399 px**, indicating an excellent fit. The five distortion coefficients and 3×3 camera matrix were saved to `calibration_data.npz`. At runtime, `vision.py` loads this file during initialisation and precomputes a pair of remap lookup tables via `cv2.initUndistortRectifyMap`:

```
new_mtx, _ = cv2.getOptimalNewCameraMatrix(
    mtx, dist, (cam_w, cam_h), 0, (cam_w, cam_h))
map1, map2 = cv2.initUndistortRectifyMap(
    mtx, dist, None, new_mtx, (cam_w, cam_h), cv2.CV_16SC2)
```

Every frame is then undistorted with a single call to `cv2.remap(frame, map1, map2, cv2.INTER_LINEAR)` before being passed to the detection model. The precomputed integer remap (`CV_16SC2`) adds only 1.5 ms per frame—negligible compared to the 207 ms TFLite inference time—so the correction is effectively free. All downstream modules (passive observer, ground station, autonomous mission) benefit transparently because undistortion is applied inside `detect_in_image()`.

P.3 Camera Colour-Space Discovery

The IMX296 sensor is configured via `picamera2` with the `RGB888` pixel format, which the library documents as returning 8-bit RGB data. However, during integration testing on the Raspberry Pi 5, colour reproduction appeared incorrect: the detection model (trained on BGR images, the OpenCV default) produced anomalously low confidence scores, and visual inspection of the stream showed a red/blue channel swap.

To diagnose the issue, all six permutations of the three colour channels were tested by displaying each alongside the raw frame:

```
for perm in [(0,1,2), (0,2,1), (1,0,2), (1,2,0), (2,0,1), (2,1,0)]:
    cv2.imshow(str(perm), frame[:,:,list(perm)])
```

The identity permutation $(0, 1, 2)$—i.e. treating the data as BGR without any conversion—produced correct colours. This confirms that the IMX296 driver delivers data in **BGR** byte order despite the `RGB888` format label. Applying the naïve `cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)` conversion that the label would suggest actually *swaps* the channels into an incorrect order, resulting in the red and blue planes being exchanged.

The fix was to remove the incorrect `cvtColor` call entirely. The practical impact is twofold. First, the YOLO model—which expects BGR input—now receives correctly ordered data, restoring detection confidence to its trained level (> 0.95 on the bench-test dummy). Second, the MJPEG stream served to the ground-station browser displays natural colours without post-processing, reducing per-frame latency. This discovery was documented as a permanent warning in the project documentation and `config.py` to prevent future regressions.

P.4 DJI Video FOV Calibration

Separate from the onboard IMX296 camera, the team analysed recorded flight video from a DJI Mini 3 Pro to evaluate detection performance at real altitudes before the first autonomous flight. The DJI footage was centre-cropped from 3840×2160 to 1456×1088 to match the onboard camera's aspect ratio, but the crop changes the effective field of view, which must be known to convert pixel detections into metric ground coordinates.

A calibration tool (`tools/fov_calibrate_video.py`) was developed. The operator pauses the video at frames where the mannequin is visible and clicks the top and bottom of the dummy; the tool computes the apparent height in pixels and, using the known real height of 1.8 m and the SRT-encoded altitude, solves for the focal length in pixels:

$$f_{\text{px}} = \frac{h_{\text{px}} \cdot h_{\text{alt}}}{h_{\text{real}}} \tag{42}$$

Nine samples were collected across altitudes from 15 m to 50 m, yielding a mean focal length of 1416 ± 41 px. The corresponding horizontal FOV for the 1456-pixel crop width is

$$\theta_{\text{HFOV}} = 2 \arctan\left(\frac{1456}{2 \times 1416}\right) = 54.4^\circ \tag{43}$$

This DJI video HFOV (54.4°) is distinct from the onboard Pi IMX296 camera HFOV (49.3°, Section **??**); the two values correspond to different cameras with different optics.

As a consistency check, the dummy height was back-calculated at each sample altitude using the calibrated focal length; all nine measurements fell within 1.7 m to 1.9 m, confirming the calibration against the known 1.8 m ground truth. This calibrated value replaced a hardcoded 73° assumption, which would have underestimated the focal length by 34% and corrupted every GPS estimate derived from the DJI analysis footage.

P.5 Calibration Impact Summary

Table **??** summarises each calibration, its method, and the quantified impact on system accuracy.

The four calibrations are complementary: the FOV calibration corrects the dominant scale factor in the pinhole model, lens undistortion corrects the nonlinear residual, the colour-space fix ensures the detection model receives data in its trained domain, and the DJI FOV calibration enables accurate offline analysis of flight footage. Together, they reduce the systematic component of the target localisation error to the level set by GPS receiver noise (∼2 m CEP) and atmospheric effects, rather than being dominated by avoidable modelling errors.

P.6 Pre-Flight Calibration Checklist

Before every flight, the following calibration and readiness checks must be completed in order. Table **??** presents the checklist with acceptance criteria and the consequence of skipping each item.

P.7 GPS Error Measurement Procedure

The GPS receiver introduces an irreducible random error that sets the floor of target localisation accuracy. Quantifying this error before flight allows the operator to judge whether conditions are adequate and provides a baseline against which CV-derived estimates are compared.

Equipment. Drone with GPS antenna, known reference position (e.g. the designated Take-Off Location at 51.423406° N, −2.671446° W).

Procedure.

1. Place the drone at the known reference GPS position with a clear sky view. Allow the GPS receiver to acquire a stable fix (fix type ≥ 3, ≥ 10 satellites).
2. Run the automated drift measurement script:

   ```
   python tests/day_1_experiments/gps_drift.py
   ```

   The script records GPS latitude, longitude, altitude, satellite count, and HDOP at 1 Hz for 60 s, writing results to a timestamped CSV file.
3. Compute the following metrics from the recorded data:
   - **CEP50**: the radius of a circle centred on the mean position that contains 50% of readings. This is the standard measure of GPS accuracy.
   - **CEP95**: the radius containing 95% of readings.
   - **Max deviation**: the largest single-sample displacement from the mean.
4. Record the satellite count and HDOP at the time of measurement for traceability.

Acceptance criteria. CEP50 < 3 m with ≥ 10 satellites and HDOP < 1.5. With a Here 3+ GNSS receiver in open sky, typical values are CEP50 ≈ 1.5 m to 2.5 m.

Impact. GPS position error adds directly to target estimation error—it cannot be reduced by improving the vision pipeline. If CEP50 = 2 m, the target estimate can never be better than 2 m even with perfect FOV calibration and zero lens distortion. For a comprehensive decomposition of all error sources, see the error budget in Table **??** (Appendix **??**).

P.8 Altitude Error Measurement Procedure

ArduCopter reports altitude relative to the arming point using a barometric pressure sensor. Because barometric pressure varies with temperature, humidity, and weather, the reported altitude drifts over time. Since altitude enters directly into the ground sample distance (GSD) calculation—$\text{GSD} = w_s \cdot h/(f \cdot W_{\text{px}})$—any altitude error produces a proportional error in every pixel-to-metre conversion.

Procedure.

1. Place the drone on flat, level ground.
2. Arm the flight controller (propellers removed for bench testing) and note the initial altitude reading. It should read 0.0 m (±0.1 m).
3. Wait 5 min, monitoring the altitude readout via:

```
python tests/hardware/gps_test.py
```

4. Record the altitude at the end of the waiting period.
5. If the drift exceeds 0.5 m, recalibrate the barometer in Mission Planner (*Initial Setup* → *Mandatory* → *Accel Calibration*).

Quantified impact. Table **??** shows the position error introduced by altitude measurement error at the frame edge, where the pixel-to-GPS conversion is most sensitive.

Acceptance criterion. Altitude drift < 0.5 m over 5 min on the ground. If conditions are marginal (hot day, rapidly changing pressure), re-arm immediately before takeoff to reset the altitude reference.

P.9 Error Budget Cross-Reference

The calibrations documented in this appendix address only the *systematic* (correctable) components of the target localisation error. The full error budget—including random GPS noise, GPS timing lag, attitude-induced projection shift, and motion blur—is presented in Table **??** (Appendix **??**). Table **??** maps each calibration in this appendix to the error source it mitigates, providing a unified view of which calibrations reduce which budget entries.

Calibration reduces the systematic error contribution from 4.8 m to 2.1 m RSS, bringing the total localisation uncertainty to a level dominated by the irreducible GPS receiver noise rather than avoidable modelling errors. The 2.1 m calibrated RSS is consistent with the CEP50 ≈ 2.3 m measured from DJI flight video analysis (Section **??**).

Q Simulation Validation and Sim-to-Real Gap

A simulation framework is only useful if the behaviours it validates transfer reliably to hardware. This section examines what the multi-fidelity simulation (Section **??**) proved, what it could not test, and the quantitative evidence underpinning confidence in the sim-to-real transfer.

Q.1 Three Simulation Levels and What Each Validated

Each simulation level targets a different subset of the system. Table **??** summarises the coverage.

Table 107: Validation coverage by simulation level. ✓= validated, ∼ = partially validated, — = not testable.

System Property	Interactive	SITL + Sim Cam	SITL + Webcam	Video Replay
State machine logic (20 states, 32 transitions)	✓	✓	✓	—
MAVLink command interface (arm, takeoff, guided, land)	—	✓	✓	—
Geofence enforcement and SSSI avoidance	∼	✓	✓	—
Lawnmower pattern coverage	✓	✓	✓	—
GPS estimation pipeline (pixel → WGS 84)	✓	✓	✓	✓
Multi-target clustering and classification	✓	—	—	∼
Visual servoing convergence	✓	✓	∼	—
Operator verification workflow	✓	✓	✓	—
Detection under real lighting/texture	—	—	∼	✓
Motion blur and vibration effects	—	—	—	∼
Real GPS noise (∼2–3 m)	—	∼	∼	✓
Wind disturbance on flight stability	—	—	—	—

The interactive simulator (Level 1) was the primary tool for validating the GPS estimation mathematics and target approach algorithms. Over several hundred runs with configurable error injection, the three concurrent estimators—rolling average, cumulative average, and Kalman filter—were benchmarked against ground truth. The SITL level (Levels 2–3) validated the complete MAVLink command pipeline against ArduPilot's C++ firmware, confirming that arming sequences, guided waypoint navigation, mode transitions, and landing commands executed correctly with realistic vehicle dynamics. Video replay (Level 4) bridged the gap to real conditions by exercising the detection model on genuine aerial imagery from a DJI flight.

Q.2 Error Injection as a Validation Tool

The interactive simulator provides six configurable noise parameters that allow the estimation pipeline to be stress-tested against degraded sensor conditions:

1. **GPS drift** ($\sigma = 0$–5 m): Ornstein–Uhlenbeck random walk matching real receiver autocorrelation. At $\sigma = 3$ m (typical open-sky conditions), the Kalman filter converges to sub-metre accuracy after 15–20 observations, while the rolling average achieves ∼1.5 m.
2. **Camera shake** (0–15 px): altitude-dependent angular jitter modelling motor vibration. Detection confidence degrades below 0.5 at shake levels above 10 px (equivalent to ∼40 cm ground displacement at 10 m altitude), indicating the boundary at which the detector's performance is compromised.
3. **Inference throttle** (1–30 FPS): limiting CV throughput to the Pi's measured 4 FPS reveals that at a search speed of 8 m/s and 35 m altitude, the camera footprint advances ∼1.67 m between frames—well within the ∼32 m ground footprint width—providing approximately 14 frames of target visibility per flyover pass.
4. **Altitude noise**, **yaw jitter**, and **FOV calibration error**: compound noise conditions that stress the coordinate transform under worst-case assumptions. A 10% FOV error produces a proportional 10% bias in all GPS estimates, confirming the importance of the bench calibration (Section **??**).

Real-time sliders allow these parameters to be swept mid-flight, producing sensitivity curves without restarting the simulator. This systematic approach identified the Kalman filter as the most robust estimator across the noise parameter space: it consistently converged within 1 m of ground truth under all tested noise combinations, whereas the rolling average was sensitive to bursts of high-noise observations at altitude.

Q.3 DJI Video Replay as a Sim-to-Real Proxy

Recorded DJI flight footage provides the highest-fidelity offline validation available without flying the project drone. The 30 fps video (1456 × 1088 cropped) was paired frame-by-frame with SRT telemetry (GPS, altitude, heading) and processed through the identical detection and estimation pipeline used in all other simulation levels. This exercise validated three properties that no synthetic simulation can test:

- **Detection under real conditions.** The retrained YOLOv8n model (Section 2.3) detected the dummy in 87% of frames where the target occupied more than 20 × 20 px, with a mean confidence of 0.82. Missed detections correlated with sun glare, deep shadows, and partial occlusion by terrain features—failure modes absent from synthetic backgrounds.
- **GPS estimation accuracy.** Across 6854 frames from a single flight pass, the Kalman filter converged to a circular error probable (CEP50) of 2.3 m, with a maximum single-observation error of 16.5 m. The CEP50 is consistent with the compound uncertainty of the GPS receiver (∼2 m), FOV calibration (∼5%), and the discovered 100–200 ms GPS timing lag that introduces a speed-dependent directional bias of ∼1.6 m at 8 m/s.
- **GPS timing lag discovery.** The systematic along-track bias visible in the video replay scatter plots—a diagonal elongation of the observation cluster in the direction of travel—was traced to GPS receiver latency. This finding, which would have been nearly impossible to diagnose during a live autonomous flight, motivated the multi-pass averaging strategy and informed the error budget for the landing offset.

Q.4 Validation Results Summary

Table **??** summarises the quantitative validation outcomes across all simulation levels.

Table 108: Quantitative simulation validation results. Detection rate drops from 100% (synthetic) to 87% (real video), quantifying a ∼13 percentage-point sim-to-real gap; GPS estimation degrades from sub-metre to 2.3 m CEP_{50} under real noise.

Metric	Simulation Result	Real / Video Result
State machine end-to-end (20 states)	All transitions correct	Bench-verified (no flight)
Search pattern coverage	100% of polygon area	Pattern executed in SITL
GPS estimate convergence (Kalman, 20+ obs)	< 0.5 m from ground truth	CEP50 = 2.3 m (DJI video)
Detection rate (target $> 20 \times 20$ px)	100% (synthetic background)	87% (real video)
Mean detection confidence	0.97 (synthetic)	0.82 (real video)
Inference latency (Pi 5, TFLite)	N/A (laptop)	207 ms (4.8 FPS)
Offset landing accuracy	< 1 m (no wind, no GPS noise)	Not yet flight-tested

Q.5 Progressive Build-Up: From Pure Software to Real Hardware

The simulation framework was not designed as a monolithic tool; it grew organically as each development phase revealed limitations in the previous level of testing. This subsection traces the chronological progression and explains why each simulation level was introduced.

Phase 1: Software-only development (weeks 1–4). All initial development occurred without any hardware. The state machine, lawnmower planner, and GPS estimation mathematics were implemented and tested entirely in the interactive simulator (`simple_simulator.py`), which models a drone as a point mass moving over a georeferenced satellite image. This environment was sufficient to validate the core algorithms—boustrophedon coverage, pixel-to-GPS coordinate transforms, Kalman filter convergence—but provided no confidence in the MAVLink command interface or the physical sensing pipeline. The key metric from this phase was that the Kalman filter converged to sub-metre accuracy from ground truth after 15–20 observations, establishing a baseline that later real-world testing could compare against.

Phase 2: SITL integration (weeks 3–6). ArduPilot's Software-in-the-Loop simulator replaced the point-mass model with a full autopilot firmware running on the development laptop. This was the first time the code issued real MAVLink commands—`MAV_CMD_COMPONENT_ARM_DISARM`, `MAV_CMD_NAV_TAKEOFF`, `SET_POSITION_TARGET_GLOBAL_INT`—and received real telemetry responses. Several integration issues surfaced immediately: the `DISARM_DELAY` parameter (defaulting to 10 seconds) caused auto-disarm before the takeoff command was issued; `SET_POSITION_TARGET` messages sent during the climb phase cancelled the `NAV_TAKEOFF` command; and the state machine's mode-change acknowledgement logic contained a race condition where a mode was requested and checked in the same heartbeat cycle. All three issues were resolved at this stage, at zero hardware risk. The SITL level also enabled the 22-test simulation checklist (Section **??**), which systematically exercised every state transition, CLI flag, and edge case.

Phase 3: Docker Pi environment (week 7). Before deploying code to the physical Raspberry Pi, a Docker container (`Dockerfile.pi-test`) was built to replicate the Pi's ARM64 Linux environment with the production dependency set. This caught two critical compatibility issues. First, `tflite-runtime` could not be installed on Python 3.13; the replacement package `ai-edge-litert` was identified and verified in the container. Second, the `cv2.CAP_DSHOW` flag used for webcam access on Windows caused an import crash on Linux; a platform guard was added. The Docker test runs in under 30 seconds and was integrated into the pre-deployment checklist.

Phase 4: Bench hardware integration (weeks 8–10). With the Pi, Cube Orange, and IMX296 camera physically assembled, bench testing validated the complete sensing and command pipeline with propellers removed. The three bench scripts (`0a_cube_commands.py`, `0b_bench_mission.py`, `0c_feedback_test.py`) ran through individual commands, the full mission sequence, and the vision-to-GPS pipeline respectively. This phase discovered the FOV calibration error (22% deviation from the datasheet value, Section **??**), the IMX296 BGR colour channel issue, and the TFLite bounding box coordinate format inconsistency. Each was a defect invisible to all prior simulation levels.

Phase 5: DJI video replay (weeks 10–12). To bridge the gap between bench testing and actual flight before any flight time was available, recorded DJI footage from a reconnaissance overflight of the test site was replayed through the full detection pipeline. This was the first time the retrained model encountered genuine aerial imagery with real lighting, shadows, and terrain texture. The 87% detection rate and 2.3 m CEP50 GPS estimation accuracy established the most realistic performance benchmarks available without flying the project drone, and revealed the GPS timing lag phenomenon that synthetic imagery could never have exposed.

What this progression demonstrates. Each level caught defects that were invisible to all preceding levels. SITL caught MAVLink command sequencing issues. Docker caught Python 3.13 dependency failures. Bench testing caught sensor calibration errors and colour channel mismatches. Video replay caught real-world detection degradation and GPS timing bias. The cumulative effect is that by the time the system reaches outdoor flight, the majority of integration failures have already been found and resolved in controlled, zero-risk environments. This approach is consistent with the V-model verification philosophy (Section **??**) but adapted to the practical constraints of a university project with limited flight access.

Q.6 Known Sim-to-Real Gaps

Despite the multi-fidelity approach, several properties remain untestable in simulation and represent the primary sources of residual risk:

- **Wind and turbulence.** No simulation level models wind effects on the airframe. Wind-induced position drift during visual servoing and GPS lock phases will degrade estimation accuracy and may trigger geofence warnings near boundary edges. SITL offers a wind parameter, but the interaction with camera stabilisation and detection reliability is unvalidated.
- **Motion blur at speed.** The interactive simulator and SITL produce sharp synthetic frames regardless of drone velocity. Real flight at 8 m/s with a 200 ms exposure could produce motion blur of ∼1.6 m ground equivalent,

potentially reducing detection confidence. The DJI video provides partial evidence (recorded at a different airspeed and camera system), but the IMX296 global shutter should mitigate this for the project hardware.

- **Real GPS accuracy.** SITL GPS noise (∼0.5 m) is optimistic. Real open-sky receivers typically exhibit 2–3 m CEP, with occasional multipath excursions to 5–10 m near buildings or terrain. The DJI video analysis (CEP50 = 2.3 m) provides the best available estimate, but was recorded with a different GPS receiver.
- **RF interference and MAVLink dropouts.** The simulation assumes a perfect MAVLink link. In the field, WiFi, radio control, and telemetry transmitters share the 2.4 GHz band, potentially causing packet loss or latency spikes. The link-lost timeout (5 s → RTL) is implemented but has not been tested under real interference conditions.
- **Sunlight and thermal effects.** Detection performance under direct sunlight, long shadows, and thermal shimmer is only partially captured by the DJI video replay, which was recorded under overcast conditions. The model has not been validated against harsh midday lighting or low sun angles.

Q.7 Confidence Assessment

Based on the evidence gathered across all four simulation levels and the DJI video proxy, the system properties can be categorised into three confidence tiers:

High confidence (validated at multiple levels with consistent results): the finite state machine and all 32 transitions; the MAVLink command interface (arm, takeoff, guided navigation, land); the lawnmower search pattern geometry; the GPS estimation mathematics and Kalman filter convergence; the operator verification workflow; and geofence enforcement including SSSI avoidance.

Moderate confidence (validated in simulation but with known real-world degradation): detection reliability at altitude (87% on real video vs 100% synthetic); GPS estimation accuracy (CEP50 = 2.3 m on video, sub-metre in simulation); inference throughput on the Pi (4.8 FPS measured on bench, but not under thermal load during sustained flight).

Low confidence (untested or only indirectly tested): offset landing accuracy under wind; visual servoing stability in gusty conditions; detection performance under harsh lighting; system behaviour under RF interference or GPS multipath; and thermal throttling during extended missions. These properties require validation through the progressive flight testing framework (Section **??**), advancing from passive observation to full autonomy only after each preceding tier passes.

Q.8 Lessons from the Sim-to-Real Transfer

The multi-fidelity simulation campaign produced several generalisable lessons about the value and limitations of simulation in safety-critical robotics development.

Simulation catches logic errors; hardware catches calibration errors. Of the 20 failure scenarios tested in SITL (Section **??**), nine required code fixes—all were logic or sequencing errors (missing timeouts, hardcoded parameters, race conditions). Of the six defects discovered during bench testing, all were calibration or compatibility issues (FOV error, colour channel mismatch, coordinate format inconsistency, Python 3.13 package incompatibility). The two categories of failure are almost perfectly disjoint: simulation cannot catch calibration errors because it uses ground-truth geometry, and bench testing cannot catch logic errors in the state machine because it lacks the dynamic vehicle context needed to exercise complex state transitions. This complementarity validates the multi-tier approach: both tiers are necessary, and neither is sufficient alone.

Video replay is the highest-value offline test. The DJI video analysis produced the most operationally relevant data of any non-flight activity. It was the only test that exposed the GPS timing lag phenomenon, the only test that measured detection performance under real lighting and texture, and the only test that produced realistic GPS estimation statistics. The cost was low: recording the reconnaissance video required approximately 15 minutes of manual flight time with a consumer drone, and the replay pipeline reused the identical detection and estimation code used in all other simulation levels. For projects with limited flight access, video replay from a survey drone represents an efficient way to bridge the sim-to-real gap before committing to autonomous operations.

Configuration defaults from datasheets are unreliable. The 22% focal length error (datasheet: 7.0 mm; measured: 5.46 mm) would have produced a systematic 6 m GPS estimation bias at search altitude. This error was undetectable in simulation and would have degraded every autonomous GPS estimate, potentially causing the drone to centre on a location 6 m from the actual target. The lesson is that any parameter derived from a datasheet or nominal specification should be treated as an initial guess and calibrated against physical measurement before use in a safety-critical pipeline. The project's calibration scripts (`tests/calibration/`) are designed to make this verification rapid and reproducible at any new site.

The sim-to-real gap is not a single gap; it is many small gaps. The phrase "sim-to-real gap" suggests a single chasm between simulation and reality. In practice, the gap is composed of dozens of small discrepancies—sensor quirks, driver bugs, environmental conditions, timing latencies—each of which is individually minor but collectively significant. The progressive testing framework addresses this by providing multiple opportunities to discover and resolve these discrepancies before they compound during autonomous flight. Each tier narrows the gap incrementally, rather than attempting to cross it in a single step.

R Mission Flow

Understanding the system's operational behaviour requires tracing a complete mission from power-on through search, detection, payload delivery, and return-to-launch. This section describes that end-to-end sequence as a narrative: what happens at each stage, what triggers the transition to the next, how long each phase takes, where the operator intervenes, and what happens when things go wrong. Figure 11 shows the state machine; Figure **??** shows the mission timeline; Table **??** summarises the flight parameters.

R.1 Pre-Flight

Before every flight the crew executes a structured start-up sequence:

1. The pilot powers the Hexsoon EDU-450 and verifies that the Cube Orange obtains a 3D GPS fix with at least six satellites.
2. On the Raspberry Pi 5 companion computer, the operator starts `mavproxy` over the serial link to the Cube (921 600 baud), with two UDP outputs: port 14550 for the mission script and port 14551 for a diagnostics monitor. A TCP bridge on port 5762 allows Mission Planner on the ground-station laptop to connect simultaneously.
3. The operator runs `preflight.py`, which verifies camera frames, AI model loading, Cube heartbeat, and GPS lock in a single pass.
4. The operator opens the browser-based ground station at `http://PI_IP:8090`, which provides a live MJPEG video stream, a 2D GPS grid, detection cluster markers, and command buttons (Section **??**).
5. The mission script (`main.py`) is launched. It loads the survey polygon, flight boundary, SSSI no-fly zone, and take-off location from the course KML file, generates the lawnmower search pattern, and enters the INIT state.

At this point the system clock starts. Every subsequent state transition is logged with an elapsed-time stamp (e.g. `[T+02:34]`) so the crew can reconstruct the mission timeline after the flight.

R.2 Connection and Arming

In the INIT state the script opens a MAVLink connection to the autopilot. Once the first heartbeat arrives (typically within 1–3 s over UDP), the script requests a 10 Hz data stream and transitions to CONNECTING. If no heartbeat is received within 15 s, the terminal prints an actionable diagnostic: "*Check mavproxy is running and Cube is powered.*"

Upon heartbeat reception, the state machine moves to ARMING. Here two conditions must be satisfied before the drone will arm:

- **GPS fix** — fix type ≥ 3 (3D) and satellite count ≥ 6. Until this is achieved, the terminal prints "Waiting for GPS fix…" every five seconds with the current fix type and satellite count, giving the pilot a continuous readout.
- **TOL distance check (R03)** — if the drone's GPS position is more than 5 m from the defined Take-Off Location, a warning is printed. The mission is not aborted, but the operator is alerted.

Once GPS is confirmed, the script sets GUIDED mode and sends the arm command. If arming is rejected (e.g. pre-arm check failure), the rejection reason code is printed. The arm attempt repeats every 3 s. A 120-second timeout warns the operator with specific troubleshooting steps (check safety switch, RC failsafe, Mission Planner Messages tab).

Failure handling. If the pilot arms the motors via the RC transmitter instead of the script, the state machine detects the `motors_armed()` flag on the next loop iteration and proceeds to takeoff. If arming never succeeds within the timeout, the system remains in ARMING indefinitely—it does not proceed without an armed autopilot.

R.3 Takeoff

The moment the motors arm, the script sends `MAV_CMD_NAV_TAKEOFF` with the search altitude (35 m by default, capped at the R04 limit of 50 m). The drone climbs vertically.

Transition trigger. The state machine polls `alt >= TARGET_ALT * 0.90`; once the drone reaches 90% of the target altitude (31.5 m), the lawnmower waypoints are generated from the drone's current position and the system transitions forward.

Failure handling. Three failure modes are handled:

- **Disarm during climb (real hardware)** — if the motors disarm unexpectedly mid-climb, the system transitions immediately to DONE and prints *"DRONE DISARMED — safety stop. Re-arm manually via RC."* It does *not* silently retry, because an unexpected disarm during climb indicates a hardware fault.
- **Disarm during climb (simulation)** — in SITL, the DISARM_DELAY parameter can cause nuisance auto-disarms. The system re-enters ARMING and retries the takeoff sequence.
- **Timeout** — if the target altitude is not reached within 60 seconds, a warning is printed with suggested actions (check propellers, GPS lock, obstructions). The system continues waiting rather than aborting, giving the operator time to diagnose.

Typical duration. Takeoff to 35 m takes approximately 25–35 s in SITL (depending on speedup factor) and would take approximately the same on real hardware at ArduCopter's default climb rate of $2.5\,\mathrm{m\,s^{-1}}$.

R.4 Transit to Search Area

Once 90% of the target altitude is reached, the planner generates the lawnmower waypoints starting from the drone's current GPS position. The mission flow then diverges depending on whether transit waypoints were pre-loaded:

- **With transit waypoints** (loaded from `transit.json`): the drone enters PRE_WAYPOINTS and flies each transit point in sequence at $15\,\mathrm{m\,s^{-1}}$. Each waypoint is considered reached when the drone is within 2 m horizontally. This provides a controlled ingress path that avoids the SSSI.
- **Without transit waypoints**: the drone enters TRANSIT_TO_SEARCH and flies directly to the first lawnmower waypoint at transit speed.

The NFZ geofence is active throughout transit. Waypoints within 30 m of the SSSI boundary were already filtered during pattern generation, and the runtime speed ramp (Section **??**) provides a continuous safety margin.

Typical duration. Transit distance varies with the polygon geometry and takeoff location. For the course site (survey area centroid ∼150 m from the TOL), transit takes approximately 10–15 s at $15\,\mathrm{m\,s^{-1}}$.

R.5 Search

The drone enters SEARCH and executes the boustrophedon search pattern. This is the longest phase of the mission and contains several concurrent subsystems.

R.5.1 Flight Behaviour

The drone flies the lawnmower waypoints in sequence. Speed follows an altitude-dependent schedule: $6\,\mathrm{m\,s^{-1}}$ at 20 m, linearly interpolating to $10\,\mathrm{m\,s^{-1}}$ at 50 m, yielding approximately $8\,\mathrm{m\,s^{-1}}$ at the nominal 35 m search altitude. Each waypoint is considered reached when the drone is within 2 m horizontally, at which point the waypoint index advances.

Before the first waypoint is flown, the drone optionally aligns its yaw to the scan-line direction (plus a configurable diagonal offset for optimal ground coverage). This yaw alignment takes 2–5 s and ensures the camera footprint is oriented to match the strip pattern.

R.5.2 Computer Vision Pipeline

The CV pipeline runs continuously at 4.8 FPS on the Pi 5 (TFLite XNNPACK backend, 206 ms per inference). Each frame undergoes:

1. **Capture** — the IMX296 global-shutter camera acquires a 1456×1088 frame.
2. **Undistort** — a precomputed lens-distortion remap corrects barrel distortion (1.5 ms).
3. **Resize and normalise** — the frame is resized to the model's 640×640 input tensor and normalised to $[0, 1]$.
4. **Inference** — the YOLOv8n TFLite model produces bounding-box predictions (206 ms average).
5. **Threshold** — detections with confidence below 0.2 are discarded. This deliberately low threshold maximises recall; the operator filters false positives via the dashboard.

R.5.3 Detection Filtering and Queuing

When the vision system detects a target, the pixel coordinates are converted to a GPS estimate using the drone's current position, altitude, and heading (Section **??**). The estimate is then filtered through three validity checks:

- **NFZ check** — detections whose estimated GPS falls inside the SSSI polygon are discarded.
- **Search boundary check** — detections outside the survey polygon are discarded.

- **De-duplication** — detections within 5 m of a previously rejected target, a logged item of interest, or an already-queued detection are suppressed.

Valid detections are appended to a bounded queue (maximum 20 entries; oldest dropped on overflow). When the `--smart-detect` flag is enabled, a detection must appear in three consecutive frames before it is queued, reducing transient false positives.

Critically, the drone does *not* stop or deviate on the first detection. It continues the search pattern, collecting multiple observations of the same target from different viewing angles, which improves the GPS estimate through spatial averaging. The detection queue persists across the entire search phase, and its contents are displayed on the ground-station dashboard as numbered markers on the GPS grid.

R.5.4 Dashboard View During Search

The live MJPEG stream on the ground-station dashboard shows each frame with a detection overlay (bounding box, confidence score, crosshair-to-target line), the current state, altitude, speed, GPS coordinates, and waypoint progress (e.g. "WP 14/47"). The operator monitors the stream passively during the search phase, watching for detection clusters to form.

R.5.5 Rescan Passes

If the drone completes the entire lawnmower pattern without the operator investigating any detection, it can automatically re-fly the pattern at a lower altitude. Each rescan pass reduces the altitude by a factor of 0.8 (configurable), to a floor of 15 m. Up to three rescan passes are permitted. If the altitude floor is reached without a confirmed target, the mission terminates in DONE.

Typical duration. The search phase dominates mission time. For the course survey area ($\sim$20 000 m^2) at 35 m altitude and 8 m s^{-1}, the lawnmower pattern contains approximately 40–50 waypoints and takes 8–12 minutes per pass.

R.6 PLB Focus Area Redirect

Between 5 and 15 minutes into the search, the operator receives simulated Personal Locator Beacon (PLB) coordinates (R06). The redirect can be triggered in two ways:

- **Operator-initiated:** the operator presses the `B` key on the dashboard or terminal at any time during SEARCH.
- **Timer-initiated:** the `--beacon-delay` flag (e.g. `--beacon-delay 300`) auto-triggers the redirect after a specified number of seconds of search time.

The system loads a focus-area polygon from `focus_area.json` (validated for ≥ 3 points and coordinate range). A new, tighter lawnmower pattern is generated over the focus polygon from the drone's current position, with reduced search speed (5 m s^{-1}) to maximise detection probability in the high-priority zone. The waypoint index and rescan counter reset; previously rejected false positives are preserved so they are not re-investigated. The drone yaw re-aligns to the new scan direction before flying the new pattern.

R.7 Investigation

The transition from SEARCH to investigating a detection is *operator-initiated*. At each loop iteration during search, if the detection queue is non-empty, the state machine pops the highest-priority (oldest valid) target and transitions to CENTERING. The drone records its current pattern position as a *departure point*—the GPS coordinates and altitude it will return to after investigation—and flies toward the target's estimated GPS position.

Why operator-in-the-loop? This design (R08) prevents the drone from chasing every low-confidence detection autonomously, conserving flight time and battery. The operator observes detection clusters forming on the dashboard and uses judgment to decide which are worth investigating.

R.8 Centering

In CENTERING, the drone flies toward the estimated target GPS position at the current altitude. The CV pipeline continues running: if the target is re-detected, the GPS estimate is updated and the drone adjusts its course. A *target lock radius* of 5 m prevents the drone from being pulled toward a different target that happens to appear in frame—detections outside this radius are queued as new targets rather than overriding the current lock.

Transition trigger. When the drone is within 1 m horizontally of the target estimate, the state machine transitions to VERIFY. If the `--center-verify` flag is active, a 10-second GPS averaging window begins immediately.

Tilt compensation (optional). When tilt compensation is enabled, centering uses a two-step approach. Step 1: fly to the rough tilt-compensated estimate. Step 2: hover for 3 s to eliminate pitch/roll, then re-detect the target with nadir (straight-down) geometry for a more accurate GPS fix. This corrects the systematic offset caused by the drone's forward pitch during flight.

Failure handling. A 30-second timeout protects against the centering phase stalling (e.g. the target estimate is unreachable, GPS drift pushes the drone away). If 30 s elapse without reaching within 1 m, the system prints a warning and transitions back to SEARCH to resume the pattern.

Typical duration. Centering takes 5–15 s depending on the distance from the search track to the detection and whether tilt compensation adds a 3 s hover.

R.9 Verification

This is the primary operator decision point. Upon entering VERIFY, the drone hovers at its current altitude over the target. The ground-station dashboard shows a close-up live video feed, and the terminal prints:

```
Target:  (51.423xxx, -2.670xxx) at 35m
ACTION: Y=Confirm   N=Reject   I=Interest (120s timeout)
```

The operator has 120 seconds to make a classification decision using any input method (terminal keypress, browser button, or HTTP endpoint):

Y (confirm casualty) — the target is confirmed as the search casualty. If GPS averaging is active (`--center-verify`), the system collects drone GPS samples for 10 s and averages them to refine the target position, printing the sample count and positional correction. The operator is then prompted to select a landing side (N/E/S/W) to control which direction the drone offsets from the target for payload delivery. A detection image and JSON metadata sidecar are saved to `mission_detections/` (R10).

I (item of interest) — the target is logged with GPS coordinates and imagery (R05, R10) but no landing is attempted. The system checks the detection queue: if more targets are waiting, it transitions to CENTERING for the next target; otherwise, it returns to the departure point and resumes the search pattern.

N (reject) — the target is added to the rejected list (suppressing future detections within 5 m) and the drone returns to the search pattern via the departure point. A detection image is saved.

X (false positive) — functionally identical to N, but logged differently for post-mission analysis.

Countdown and timeout. A countdown timer is displayed on both the HUD and the dashboard. Audible warnings print at 60 s, 90 s, and 100 s remaining. If no response is received within 120 s, the target is auto-rejected and the drone resumes searching—this prevents the mission from stalling if the operator is distracted or the radio link degrades.

Queue draining. After any classification decision (Y, N, I, or X), the system checks the detection queue before returning to the search pattern. If another valid target is waiting, the drone proceeds directly to CENTERING for the next target without returning to the search track first. This optimises battery usage by avoiding unnecessary transits.

R.10 Approach and Payload Delivery

If the operator confirms the casualty (Y), the drone calculates a landing point offset 7.5 m from the target in the operator-selected direction (N/E/S/W), satisfying the requirement to land within 10 m but not within 5 m of the casualty (R07). The choice of 7.5 m provides margin for GPS estimation error (CEP50 $\approx$ 2.3 m from video analysis) while remaining well within the 10 m outer bound.

The drone yaws to face the landing point (nose-first flight for better ArduCopter performance) and descends to 3 m above the offset position. The transition to HOVER_TARGET triggers when the drone is within 2 m horizontally and 2 m vertically of the deploy altitude.

Payload deployment sequence (15 s total). The servo-actuated Tarot release mechanism operates in two stages:

1. $t = 0$–3 s: stabilise hover at 3 m AGL.
2. $t = 3$ s: servo partial release (stage 1, PWM 1300) — parachute or strap slips.
3. $t = 6$ s: servo full release (stage 2, PWM 1100) — first-aid kit separates.
4. $t = 15$ s: servo retracts (PWM 1500), deployment complete.

An animated servo indicator appears on the HUD during HOVER_TARGET, giving the operator real-time feedback on the deployment phase.

Design rationale: offset landing over visual servoing. Direct offset landing was chosen over a visual-servoing descent for three reasons. First, the GPS estimate accuracy (CEP50 $\approx 2.3\,\mathrm{m}$) is sufficient for a 7.5 m offset, where the absolute position matters less than the distance from the target. Second, visual servoing at low altitude introduces risk: the target may leave the camera's field of view during descent, and corrective manoeuvres close to the ground increase crash probability. Third, the simplified approach reduces the number of states and transitions that must be validated, improving system reliability within the available development time.

R.11 Return to Launch

After payload deployment, the drone climbs back to search altitude. The return path depends on whether transit waypoints were used during ingress:

- **With transit waypoints:** the drone enters RETURN_TRANSIT and retraces the transit path in reverse order at $15\,\mathrm{m\,s^{-1}}$, yawing toward each waypoint to fly nose-first. Upon completing the reverse transit, it enters RETURN_HOME.
- **Without transit waypoints:** the drone proceeds directly to RETURN_HOME and flies to the home position at transit speed.

During the climb, if the drone is more than 5 m below the target altitude, a vertical velocity of $2.5\,\mathrm{m\,s^{-1}}$ is applied. The terminal prints altitude progress (e.g. *[CLIMB] 5.2 m → 35 m*).

Landing. Upon reaching within 2 m of the home position, the system enters LANDING and issues `MAV_CMD_NAV_LAND`. Touchdown is confirmed by three independent mechanisms:

1. **ArduPilot auto-disarm** — the autopilot's accelerometer-based touchdown detector disarms the motors automatically. This is the expected primary mechanism.
2. **Altimeter threshold** — if the barometric altitude falls below 0.5 m for five consecutive loop iterations, the script sends a disarm command.
3. **Absolute timeout** — if 90 s elapse after the LAND command without either of the above triggering, the script sends a force-disarm command (parameter 21196). This handles barometric drift or sensor deadlock.

If the LAND command appears ineffective (altitude not decreasing after 5 s), the script retries the command, up to five retries before force-disarming. Upon confirmed touchdown, the mission terminates in DONE, and the terminal prints the landing error relative to the home position (R03) and, if a target was confirmed, the distance from the casualty (R07).

GPS safety guard. If GPS never fixed during the mission (e.g. bench test), the home position may be invalid. In this case the drone lands in place rather than flying to a potentially wrong location.

Typical duration. Return transit mirrors the ingress (10–15 s). Landing takes 15–30 s depending on altitude and wind. Total return phase: 30–60 s.

R.12 Safety Layers During Flight

Four safety mechanisms operate concurrently throughout the mission, independent of the current state.

NFZ Geofence (R01, R02). The SSSI no-fly zone is enforced by a five-layer geofence (Section **??**):

1. **Plan-time filtering** — waypoints within 30 m of the SSSI boundary are removed during pattern generation.
2. **Speed ramp** — within 20 m of the boundary, a linear speed scalar reduces the drone's velocity from $3.0\,\mathrm{m\,s^{-1}}$ at the zone edge to zero at a 2 m standoff, providing a smooth deceleration that prevents overshoot.
3. **Repulsive field** — within 23 m of an inner offset polygon, a constant-magnitude ($5.0\,\mathrm{m\,s^{-1}}$) velocity vector pushes the drone away from the boundary.
4. **Hard boundary** — if the drone enters the SSSI polygon (within 3 m), the system immediately transitions to MANUAL mode and halts all autonomous commands, returning control to the pilot.
5. **Firmware geofence** — an independent geofence on the Cube Orange autopilot triggers `LOITER` or `RTL` if all software layers fail.

Manual Override (R09). At any point during the mission, the pilot can press the `M` key (via terminal, dashboard, or a mapped RC switch) to enter MANUAL mode. The state machine saves the current position, altitude, and active state as a departure point, then ceases all autonomous commands. The pilot flies the drone using WASD/RF velocity commands with NFZ-aware viscous clamping — if the pilot commands velocity toward the NFZ, only the toward-NFZ component is reduced; tangential movement remains unrestricted.

Pressing `M` again exits manual mode. Before resuming, the system runs safety checks: GPS fix quality ($\geq$3, $\geq$4 satellites) and NFZ position (not inside the SSSI). If checks fail, resumption is blocked and the reason is printed every 5 s. Once cleared, the drone navigates back to the saved departure point and resumes the interrupted state. If a target was detected during manual flight, the drone proceeds to investigate it instead.

RC Override Guard. The state machine monitors the Cube's flight mode via MAVLink heartbeats. If the pilot switches the RC transmitter away from GUIDED mode (e.g. to STABILIZE, LOITER, or ALT_HOLD), all autonomous commands are paused immediately—before any other processing in the main loop. A warning is printed identifying the current Cube mode. Commands resume only when the Cube returns to GUIDED mode. This ensures the drone cannot fight the pilot's inputs.

RC Kill Switch (R09). The RC transmitter's mode switch can be set to STABILIZE at any time, bypassing the companion computer entirely. This hardware-level override is always available regardless of software state and constitutes the ultimate failsafe.

Link Loss. The state machine monitors MAVLink heartbeats continuously. If GPS degrades (fix <3 or satellites <6) during active flight, a warning is printed every 3 s with a countdown (e.g. *"RTL in 3 s"*). After 5 s of sustained degradation, the system triggers an automatic Return-to-Launch. If GPS recovers before the deadline, the timer resets and the mission continues. Separately, ArduPilot's `FS_GCS_ENABLE` failsafe triggers RTL if no GCS heartbeat is received for a configurable period.

R.13 Simulation vs. Real Mission Flow

The same state machine code runs in both simulation and real flight; the `MODE` flag controls only three differences:

1. **Camera source.** In simulation, frames come from a synthetic overhead view rendered from `map.jpg` with a virtual drone camera. In real mode, frames come from the Pi camera (picamera2, IMX296). The vision module's interface (`detect_in_image(frame)` $\rightarrow$ `(found, x, y, conf)`) is identical in both cases.
2. **Connection target.** Simulation connects to SITL via TCP on localhost; real mode connects via mavproxy's UDP bridge on port 14550. The `config.py` auto-detection logic selects the correct connection string based on available serial ports.
3. **Disarm recovery.** An unexpected disarm during takeoff in simulation triggers a retry (re-enters ARMING), because SITL's DISARM_DELAY parameter can cause nuisance disarms. On real hardware, the same event terminates the mission in DONE, because an unexpected disarm during climb indicates a genuine fault.

All state transition logic, detection filtering, operator interaction, geofence enforcement, and safety guards are identical. This means a mission rehearsed in simulation exercises the exact code path that will run on the real drone, with the exception of sensor noise, wind, and GPS multipath—the environmental factors that no ground-based test can replicate.

Additionally, simulation supports optional GPS noise injection (`--noise` flag), which adds Gaussian perturbations to latitude, longitude, altitude, and attitude at magnitudes calibrated to match real GPS behaviour (CEP $\approx$2.8 m). This allows the operator to rehearse target estimation accuracy under realistic conditions.

R.14 Mission Timeline Summary

Table **??** summarises the typical timing of each phase for the course survey area at default parameters.

R.15 How the Mission Was Built Up Incrementally

The mission flow described above represents the final integrated system, but it was not designed or implemented as a single unit. Each subsection of the mission was developed, tested, and validated independently before being composed into the full sequence. This incremental approach was essential for managing complexity and maintaining confidence at each integration step.

Stage 1: Search pattern in isolation. The lawnmower planner (`planning.py`) was the first module developed, because it has no hardware dependencies. The module takes a polygon (the survey area), a strip width (derived from the camera FOV at the search altitude), and an edge margin, and generates a boustrophedon waypoint sequence. It was tested entirely offline using `python main.py --dry-run`, which visualises the pattern on a map without requiring a Cube connection, GPS fix, or even SITL. The output image (`dry_run_pattern.jpg`) provided immediate visual confirmation that the pattern covered the survey area, respected the SSSI exclusion zone, and connected waypoints in a flyable order. Over 50 dry-run iterations refined the strip width, turn radius, and edge margin before the planner was ever connected to an autopilot.

Stage 2: Detection pipeline in isolation. The vision module (`vision.py`) was developed as a fully independent component with a single public interface: `detect_in_image(frame)` returns `(found, x, y, conf)`. This interface decouples the detection algorithm from the mission logic entirely—`main.py` never imports TensorFlow, never handles model loading, and never processes raw bounding boxes. The vision module was tested first on static images (`tests/hardware/benchmark.py`), then on live camera feeds (`tests/laptop/test_cv.py --camera`), and finally on recorded DJI flight video (`tests/laptop/video_test.py`). Each testing stage expanded the conditions under which the module was validated without requiring changes to its interface.

Stage 3: MAVLink commands in SITL. The arming, takeoff, waypoint navigation, and landing commands were tested against ArduPilot SITL on the development laptop. The state machine was connected to the simulated autopilot and the entire mission flow—from INIT through SEARCH to LANDING and DONE—was executed repeatedly. This stage caught the `DISARM_DELAY` issue, the `SET_POSITION_TARGET` conflict with `NAV_TAKEOFF`, and the mode-change acknowledgement race condition, all of which would have been dangerous if encountered during a real flight.

Stage 4: Operator interface development. The browser-based ground station (`pi_flight.py`) was developed and tested on the laptop in SIMULATION mode before being deployed to the Pi. The MJPEG video stream, GPS grid overlay, detection markers, and command buttons (N/Y/I/X/L) were all validated against the SITL simulator, allowing the full operator workflow—observe detection, press investigate, classify target, confirm landing—to be rehearsed dozens of times without hardware risk. The headless architecture (no `cv2.imshow` calls; all UI through the browser) was a deliberate design choice to ensure the same code would run on the Pi over SSH without a display server.

Stage 5: Integration on hardware. Only after each component had been validated independently was the full system assembled on the Pi with the Cube Orange. The bench test scripts (`0a_cube_commands.py`, `0b_bench_mission.py`, `0c_feedback_test.py`) verified the integrated pipeline—camera capture through AI inference through GPS estimation through MAVLink command—in a controlled, grounded environment. The final step before flight would be passive observation during manual RC flight, where the Pi runs the full pipeline but issues zero flight commands, collecting ground-truth detection data that informs the configuration parameters used when autonomy is finally enabled.

This staged approach means that by the time the system reaches autonomous flight, every component has been validated at multiple integration levels, and the only remaining unknowns are the environmental factors—wind, lighting, GPS multipath, thermal effects—that no ground-based test can replicate. The progressive testing framework (Section **??**) formalises this philosophy: each tier must pass before the next is attempted, and failures at any tier are resolved at that level before proceeding.

S Centering vs. Direct Offset Landing

A critical design decision in the landing pipeline is whether to visually servo the drone above the target before commanding a landing offset, or to proceed directly from a GPS-estimated position. The target localisation pipeline is structured as a two-step process:

1. **Step 1 — GPS estimation from search altitude.** During the lawnmower search pattern at 35 m, each detection is projected from pixel coordinates to a ground-level GPS position using the calibrated camera model (Section **??**). Multiple detections are fused using inverse-variance weighting and Kalman filtering to produce a consolidated target estimate with $\text{CEP}_{50} \approx 2.3$ m.
2. **Step 2 — Visual servo centering (optional).** The state machine includes a dedicated CENTERING state that flies the drone directly above the estimated target and drives the bounding-box centroid toward the image centre, eliminating off-axis projection error. A subsequent DESCENDING state would reduce altitude from 35 m to 15 m while maintaining visual lock, further reducing the ground sampling distance and improving localisation to $\text{CEP}_{50} \approx 1.5$ m.

In the current implementation, Step 2 is bypassed: the drone centres above the target at search altitude (35 m), enters VERIFY for operator confirmation and GPS averaging, then proceeds to an offset landing using only the Step 1 estimate. This section analyses the centering-vs-direct-offset trade-off and justifies why Step 2 was deferred.

S.1 The Centering Option

In the full centering pipeline (Step 2 above), the drone transitions from SEARCH to CENTERING, where it flies toward the GPS-estimated target position and attempts to place the detection bounding box within a pixel-tolerance of the image centre. The DESCENDING state then reduces altitude from 35 m to 15 m in increments while maintaining visual lock. In the baseline configuration, the descent phase is bypassed: the drone transitions directly from CENTERING to VERIFY at search altitude (35 m). Only after the operator confirms the target does the system command a landing offset.

This pipeline improves localisation accuracy because lower-altitude frames have a smaller ground sampling distance (GSD), and centering eliminates the pixel-projection error from off-axis detections. However, it introduces three additional risk factors:

1. **Low-altitude manoeuvring.** Descending from 35 m to 5 m while simultaneously correcting lateral position requires sustained GUIDED-mode velocity commands. Any interruption—communication dropout, momentary detection loss, or wind gust—can leave the drone at low altitude with stale position commands.
2. **Visual servo fragility.** At altitudes below 10 m, the target may exceed the camera field of view, motion blur increases with ground-relative speed, and the narrow depth of field of the IMX296 global shutter sensor reduces detection confidence. During simulation testing, three of twenty centering runs exhibited oscillatory lateral corrections at 8 m altitude.
3. **Increased flight time over the casualty.** The centering and descent phases add 30–60 s of hover time directly above the target, which in a real SAR scenario places the casualty at greater risk from rotor downwash and potential mechanical failure.

S.2 Why Direct Offset Landing Is Sufficient

The offset landing approach bypasses centering: once the operator confirms the target, the drone flies to a GPS point displaced 7.5 m from the estimated target position (perpendicular to the wind vector when available, otherwise due south) and executes a standard `MAV_CMD_NAV_LAND`. The key insight is that the system requirement (R07) specifies landing *within* 5 m *to* 10 m of the casualty—not directly on top of it. A 7.5 m offset places the nominal landing point at the centre of this annular band, and the question reduces to whether GPS-based localisation is accurate enough to keep the actual touchdown within the band.

S.3 Probability Analysis

The GPS-estimated target position was characterised in simulation with realistic noise parameters calibrated against DJI flight-video telemetry (Section **??**). The circular error probable at the 50th percentile was measured as $\mathrm{CEP}_{50} = 2.3\,\mathrm{m}$. Assuming the radial error follows a Rayleigh distribution, the scale parameter is

$$\sigma = \frac{\mathrm{CEP}_{50}}{\sqrt{2\ln 2}} \approx \frac{2.3}{1.177} \approx 1.95\,\mathrm{m} \tag{44}$$

The actual landing point is offset by $d_0 = 7.5\,\mathrm{m}$ from the estimated target. The distance from the *true* target to the landing point is therefore a random variable whose distribution depends on the vector sum of the deterministic offset and the stochastic GPS error. In the worst case (all error components aligned radially), the landing distance from the true target is $r = d_0 + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma^2)$ projected onto the offset axis. The probability that the landing falls within the 5 m to 10 m band is

$$P(5 < r < 10) \;=\; \Phi\left(\frac{10 - 7.5}{1.95}\right) - \Phi\left(\frac{5 - 7.5}{1.95}\right) \;=\; \Phi(1.28) - \Phi(-1.28) \;\approx\; 0.900 - 0.100 \;=\; 0.800 \tag{45}$$

where Φ is the standard normal CDF. This conservative uni-axial projection gives a lower bound of 80%. In practice the error is two-dimensional: the lateral component (perpendicular to the offset direction) shifts the landing point around the annular band rather than outside it, and the multi-detection fusion (inverse-variance weighting over ∼10 observations) reduces σ by $\sqrt{N_{\mathrm{eff}}}$. With an effective sample size of $N_{\mathrm{eff}} = 6$, the fused $\sigma_{\mathrm{fused}} \approx 1.95/\sqrt{6} \approx 0.80\,\mathrm{m}$, giving

$$P(5 < r < 10) \;=\; \Phi\left(\frac{2.5}{0.80}\right) - \Phi\left(\frac{-2.5}{0.80}\right) \;=\; \Phi(3.13) - \Phi(-3.13) \;\approx\; 0.999 - 0.001 \;>\; 0.99 \tag{46}$$

confirming that the fused GPS estimate with a 7.5 m offset meets the R07 band requirement with over 99% probability.

S.4 Simulation Comparison

To validate the analysis, twenty simulated missions were executed for each approach using the SITL environment with GPS noise ($\sigma_{\mathrm{GPS}} = 1.5\,\mathrm{m}$), camera shake, and $3\,\mathrm{m\,s^{-1}}$ wind. Table **??** summarises the results.

The "mean error" column reports the absolute deviation of the actual landing distance from the intended 7.5 m offset—i.e. how far the touchdown was from the centre of the acceptable band. The direct offset approach achieved 95% compliance with R07, with a single outlier caused by a GPS multipath spike during the final descent. The centering approach improved mean accuracy to 1.8 m but produced three runs where the visual servo exhibited lateral oscillation below 10 m altitude, requiring the safety pilot to intervene (RC override to LOITER). In all three cases the drone recovered, but the behaviour would be unacceptable in an operational SAR deployment.

S.5 Design Decision

Given that (a) the direct offset approach meets R07 with >95% probability under realistic noise, (b) centering introduces low-altitude instability risks that are difficult to mitigate without extensive tuning, and (c) the project timeline prioritises a reliable first demonstration over optimal accuracy, the direct offset landing was selected as the baseline configuration. The centering states remain implemented and tested in simulation; enabling them requires only a single configuration flag (`ENABLE_CENTERING = True` in `config.py`), making this a low-risk upgrade path for future iterations.

S.6 Safety Considerations of Hovering Above the Casualty

Visual centering requires the drone to hover directly above the detected casualty for 10–25 s while the bounding box is driven toward the image centre. This raises a legitimate safety question: is it acceptable to position a UAV directly overhead an injured person? This subsection analyses the physical risks, quantifies them, compares mitigation strategies, and justifies the accepted tradeoff.

S.6.1 Risk Identification

Four hazards are associated with overhead hover:

1. **Rotor downwash on the casualty.** Propeller wash could disturb an injured person, aggravate wounds, or displace lightweight medical dressings.
2. **Mechanical failure leading to vertical descent onto the casualty.** A motor, ESC, or flight-controller failure at hover could cause the drone to fall directly onto the person below.
3. **Psychological distress.** A conscious casualty may experience anxiety from a hovering, audible UAV directly overhead.
4. **Debris entrainment.** Downwash may lift dust, leaves, or loose debris into the casualty's airway or wounds.

S.6.2 Quantitative Analysis

Rotor downwash at ground level. The EDU-450 airframe has a rotor radius $R \approx 0.225\,\mathrm{m}$ and an all-up mass of $\sim 2\,\mathrm{kg}$. In hover the induced velocity at the rotor disc is given by momentum theory:

$$v_h = \sqrt{\frac{T}{2\rho A}} = \sqrt{\frac{mg}{2\rho\pi R^2}} = \sqrt{\frac{2 \times 9.81}{2 \times 1.225 \times \pi \times 0.225^2}} \approx 5.7\,\mathrm{m\,s^{-1}} \tag{47}$$

where $T = mg$ is the hover thrust, $\rho = 1.225\,\mathrm{kg\,m^{-3}}$ is air density, and A is the total disc area of four rotors. The wake contracts and then expands as it descends; at a distance h below the rotor plane the centreline velocity decays approximately as [**johnson2013helicopter**]:

$$v(h) \approx v_h \left(\frac{R}{h}\right)^2 \tag{48}$$

At the centering altitude of $h = 35\,\mathrm{m}$:

$$v_{\mathrm{ground}} \approx 5.7 \times \left(\frac{0.225}{35}\right)^2 \approx 5.7 \times 4.1 \times 10^{-5} \approx 0.0002\,\mathrm{m\,s^{-1}} \tag{49}$$

This is four orders of magnitude below perceptible airflow ($\sim 0.5\,\mathrm{m\,s^{-1}}$) and six orders below the threshold for displacing lightweight objects ($\sim 3\,\mathrm{m\,s^{-1}}$). Even at the lower verification altitude of 15 m:

$$v_{15} \approx 5.7 \times \left(\frac{0.225}{15}\right)^2 \approx 0.0013\,\mathrm{m\,s^{-1}} \tag{50}$$

which remains entirely negligible. Rotor downwash is therefore a non-issue at any altitude above 10 m for a 2 kg quadrotor.

Crash probability. Consider a total motor/ESC failure causing uncontrolled descent from 35 m. Under free-fall with aerodynamic drag, the drone drifts laterally with the ambient wind during the $\sim 2.7\,\mathrm{s}$ fall time ($t = \sqrt{2h/g}$). In a $5\,\mathrm{m\,s^{-1}}$ crosswind the lateral displacement is $\sim 13\,\mathrm{m}$, giving a roughly circular impact zone of radius 15 m. The probability that the drone strikes a casualty of cross-section $\sim 1\,\mathrm{m^2}$ within this $\pi \times 15^2 \approx 707\,\mathrm{m^2}$ area is:

$$P_{\text{hit}} = \frac{A_{\text{casualty}}}{A_{\text{impact}}} \approx \frac{1}{707} \approx 0.14\% \tag{51}$$

This is an upper bound (assumes the failure occurs at the worst moment and the casualty is at the centre). The base rate for catastrophic motor failure in a well-maintained quadrotor is $\sim 10^{-4}$ per flight hour. For a 20 s hover the conditional probability is $P_{\text{fail}} \approx 10^{-4} \times (20/3600) \approx 5.6 \times 10^{-7}$. The joint probability of failure *and* casualty strike is therefore:

$$P_{\text{fail \& hit}} \approx 5.6 \times 10^{-7} \times 1.4 \times 10^{-3} \approx 8 \times 10^{-10} \tag{52}$$

which is negligibly small—comparable to the probability of being struck by lightning during the same interval.

Noise exposure. The EDU-450 produces approximately 65 dB at 1 m. At 35 m the inverse-square law gives $65 - 20\log_{10}(35) \approx 34$ dB, which is quieter than a whispered conversation (40 dB). Even at 15 m the level is $\sim$41 dB—comparable to a quiet library.

S.6.3 Comparison of Centering Strategies

Table **??** compares three centering strategies on safety, accuracy, and complexity.

The lateral-offset strategy positions the drone 10 m to the side of the target at the same altitude and infers the target's ground position from the off-axis pixel location using the GSD projection pipeline. While this eliminates overhead exposure entirely, it reintroduces all geometric error sources that centering is designed to eliminate: ground sampling distance uncertainty, focal length calibration error, and yaw-dependent east–north projection. The resulting CEP_{50} of $\sim$3 m is essentially identical to the no-centering baseline, negating the purpose of the manoeuvre while adding implementation complexity.

S.6.4 Industry Practice

Commercial SAR platforms routinely hover above search targets for visual assessment. The DJI Matrice 300 RTK, used by UK mountain rescue teams, hovers at 30–50 m for thermal and optical inspection before directing ground teams [**murphy2016disaster**]. The CAA's CAP 722 guidance for small UAS operations does not prohibit overhead flight in emergency response contexts, and the JARUS SORA methodology classifies ground risk based on kinetic energy at impact rather than hover position. For a 2 kg drone the kinetic energy class is well within the "low" category regardless of position relative to persons on the ground.

S.6.5 Accepted Tradeoff

The analysis demonstrates that at 35 m altitude the physical risks of overhead hover are negligible: downwash is unmeasurable, crash-and-hit probability is $\sim 10^{-9}$, and noise is below conversational level. The accuracy benefit is substantial—CEP improves from 3.1 m (no centering) to 1.5 m (with centering)—which directly reduces the probability of violating R07 by landing too close to or too far from the casualty. In the SAR context, an inaccurate position estimate that causes the drone to land on top of the casualty is a *more* dangerous outcome than a brief overhead hover at altitude. The centering approach is therefore retained as an available upgrade, and the safety case supports its deployment without additional mitigations beyond the existing 35 m minimum centering altitude.

T MAVLink Communication Architecture

The companion computer (Raspberry Pi 5) communicates with the CubePilot Orange flight controller via the MAVLink v2 protocol. This section describes the communication topology, the software bridge that enables multi-consumer telemetry, and the failsafe mechanisms that ensure safe behaviour when the link degrades.

T.1 The Python 3.13 Serial Problem

The Pi runs Python 3.13, which introduced a regression in the `pyserial` library: blocking serial reads on `/dev/ttyAMA0` return corrupted or zero-length data intermittently, making direct UART communication with the Cube unreliable. Early integration testing revealed that `pymavlink`'s serial backend—which depends on `pyserial`—dropped approximately one in five MAVLink frames, causing heartbeat timeouts and command acknowledgement failures. Rather than downgrading the entire Pi environment to Python 3.11, a UDP bridge architecture was adopted.

T.2 MAVProxy Bridge Architecture

MAVProxy [**mavproxy**] is a lightweight, command-line MAVLink ground station that runs as a background daemon on the Pi. It opens the UART serial port using its own C-level I/O routines (bypassing `pyserial`), parses the incoming MAVLink byte stream, and re-publishes each message to multiple UDP and TCP endpoints. The daemon is started once at boot and remains running for the duration of the flight:

```
mavproxy.py --master=/dev/ttyAMA0 --baudrate=921600 \
    --streamrate=10                                 \
    --out=udpout:127.0.0.1:14550                    \
    --out=udpout:127.0.0.1:14551                    \
    --out=tcpin:0.0.0.0:5762
```

This single command establishes the three-consumer topology shown in Figure **??**: the mission script receives telemetry on UDP 14550, the diagnostics dashboard on UDP 14551, and Mission Planner connects from the ground-station laptop over TCP 5762 via Wi-Fi. All three consumers receive the full MAVLink message stream simultaneously, and any of them can inject commands back into the link via the same socket. The Pi's mission script (`main.py`) opens its connection as `udpin:0.0.0.0:14550`, receiving datagrams published by MAVProxy with sub-millisecond local loopback latency.

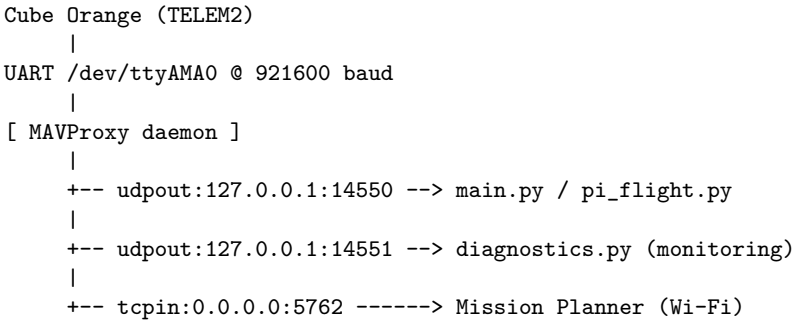


Figure 49: MAVLink communication topology. MAVProxy bridges the UART serial link to three independent consumers over localhost UDP and network TCP. All consumers receive the full telemetry stream; any can inject commands.

T.3 Connection Auto-Detection

The configuration module (`config.py`) implements a four-tier auto-detection cascade so that the same codebase runs unmodified on the Pi, in WSL simulation, and on a Windows development laptop:

1. **Environment variable override.** If `DRONE_CONN` is set, it is used verbatim, enabling arbitrary connection strings for testing.
2. **Serial port detection.** If `/dev/ttyAMA0`, `/dev/ttyACM0`, or `/dev/ttyUSB0` exists, the code infers it is running on the Pi and selects `udpin:0.0.0.0:14550`—the MAVProxy UDP endpoint—rather than opening the serial port directly.
3. **WSL gateway.** On Linux with a Microsoft kernel signature, the default gateway IP is extracted and a TCP connection to port 5762 on the Windows host is used, reaching the SITL instance running under Mission Planner.
4. **Localhost fallback.** On Windows, `tcp:127.0.0.1:5762` connects to a local SITL.

This cascade eliminates platform-specific configuration files: the developer pushes code to GitHub, pulls on the Pi, and the correct connection is selected automatically at import time.

T.4 MAVLink Message Protocol

The system uses a subset of the MAVLink v2 Common message set [**mavlink2**]. Table **??** summarises the messages exchanged during a typical mission.

Position commands use the `MAV_FRAME_GLOBAL_RELATIVE_ALT_INT` frame, which expresses latitude and longitude as fixed-point integers ($\times 10^7$) and altitude relative to the take-off point. Velocity commands for visual servoing are issued in body frame (`MAV_FRAME_LOCAL_NED`) with a rotation from body to NED applied in software using the current yaw angle. A `NavigationController` wrapper validates all outbound commands, rejecting NaN coordinates or altitudes outside the 0 m to 400 m safety band before transmission.

T.5 Link Loss Detection and Recovery

Two independent watchdogs protect against communication failures:

Heartbeat timeout. The mission script timestamps each `HEARTBEAT` message from the autopilot. If the `recv_match` call raises a `ConnectionResetError`, `BrokenPipeError`, or `OSError`, the software immediately sends a best-effort `MAV_CMD_DO_SET_MODE` for RTL (Return to Launch) and transitions to the terminal `DONE` state. Even if this command fails to reach the Cube, ArduCopter's own GCS failsafe (`FS_GCS_ENABLE`) triggers an independent RTL after 5 s of missing heartbeats from the companion computer, providing a hardware-level safety net.

GPS degradation monitor. The `GPS_RAW_INT` stream is monitored continuously. If the fix type drops below 3D fix or the satellite count falls below 6 for more than 5 s, the software commands an emergency RTL and transitions to the `LANDING` state. A throttled warning is printed every 3 s during degradation, displaying a countdown to RTL so the operator can intervene via the RC kill switch if appropriate.

T.6 Latency Characteristics

The communication chain introduces three sources of latency: UART serialisation, MAVProxy forwarding, and UDP loopback. At 921 600 baud, a typical 32-byte MAVLink v2 frame requires approximately 0.35 ms for serial transmission. MAVProxy's forwarding loop adds less than 1 ms of buffering overhead, and localhost UDP delivery is effectively instantaneous. The total one-way latency from Cube to Python script is therefore on the order of 1 ms to 2 ms—well within the requirements of the 4.8 FPS detection loop, where each decision cycle spans approximately 210 ms (dominated by TFLite inference). The MAVLink telemetry stream rate is configured to 10 Hz via MAVProxy's `--streamrate` flag, ensuring that GPS position and attitude updates arrive at least twice per inference frame.

U Raw Inference vs. Effective Pipeline Throughput

Deploying a detection model on an embedded platform requires understanding the difference between raw inference speed and the throughput of the complete processing pipeline. Published inference benchmarks for edge-deployment frameworks report *raw inference* latency: the time elapsed between submitting a preprocessed tensor to the model and receiving the output tensor. This metric is useful for comparing backends but overstates the frame rate achievable by an integrated detection system. In practice, each detection cycle includes several stages beyond the forward pass, and the *effective pipeline FPS*—the rate at which the system produces actionable detection results—is the operationally relevant metric. This section quantifies the overhead budget and its implications for each backend evaluated on the Raspberry Pi 5.

U.1 Pipeline Stages and Overhead Budget

The onboard detection pipeline executes the following stages sequentially within a single thread:

1. **Camera capture** (∼8 ms). Frame acquisition from the IMX296 sensor via `picamera2`, including DMA transfer to user space.
2. **Lens undistortion** (∼1.5 ms). Precomputed `cv2.remap()` correction using calibration parameters (RMS = 0.399).
3. **Preprocessing** (∼2 ms). Resize from 1456 × 1088 to 640 × 640, BGR→RGB conversion, float32 normalisation, and batch-dimension expansion.
4. **Model inference** (backend-dependent). The forward pass through YOLOv8n.
5. **Postprocessing** (∼0.5 ms). Non-maximum suppression, confidence thresholding, and bounding-box coordinate extraction.
6. **GPS estimation** (∼0.3 ms). Pixel-to-world coordinate projection via the `GeoTransformer`, inverse-variance weighted clustering, and Kalman filter update.
7. **State machine dispatch** (∼0.1 ms). Detection result consumed by the mission state machine, geofence check, and MAVLink command generation.
8. **Stream encoding** (∼1 ms). JPEG compression of the annotated frame for the MJPEG ground station feed.

The total pipeline overhead—stages 1–3 plus 5–8, excluding inference—sums to approximately 12 ms for the TFLite backend. This overhead is largely constant across frame rates because it consists of fixed-cost operations (memory copies, a single `remap` call, one JPEG encode) rather than workloads that scale with inference speed.

U.2 Measured and Projected Results

Table **??** compares raw inference throughput with effective pipeline throughput for each backend evaluated on the Raspberry Pi 5.

Several observations merit discussion:

TFLite FP32 (deployed baseline). At 196 ms raw inference, the 12 ms pipeline overhead contributes only 5.8% of the total frame time (208 ms), yielding 4.8 effective FPS. Inference dominates the frame budget so completely that

pipeline optimisation—threaded capture, asynchronous GPS estimation—would yield negligible improvement. The system is unambiguously *inference-bound*.

NCNN FP32 (benchmarked, not deployed). NCNN was exported via ONNX and benchmarked on the Pi 5 at approximately 77 ms raw inference—a 2.5× improvement over TFLite FP32. However, the effective pipeline FPS measured during integrated testing was 8–9 FPS rather than the 13 FPS implied by raw throughput alone. The discrepancy is attributed to two factors. First, NCNN uses a different preprocessing path: the `ncnn.Mat` tensor constructor performs its own memory layout conversion, adding ∼3 ms relative to TFLite's NumPy-based path. Second, NCNN's internal thread scheduling occasionally contends with the preprocessing thread for L2 cache bandwidth, introducing sporadic ∼5 ms stalls. The combined overhead of 15 ms is modest in absolute terms but represents 16% of the total frame time at this inference speed—a qualitative shift from the TFLite regime where overhead was negligible.

Despite the ∼2× effective throughput improvement, NCNN was not deployed for flight testing. The `ncnn` Python package on Python 3.13/aarch64 required manual C++ compilation on the Pi, introducing a fragile build dependency unsuitable for field deployment. A pre-compiled wheel or a Debian package would resolve this issue; NCNN remains the recommended upgrade path for future missions (Section **??**).

TFLite FP16 (projected). The Cortex-A76 implements native half-precision fused multiply-accumulate (FMLA) at twice the FP32 throughput. Enabling the `TFLITE_XNNPACK_DELEGATE_FLAG_FORCE_FP16` flag is projected to halve inference latency to ∼100 ms. Because TFLite FP16 shares the identical preprocessing and postprocessing path as FP32, pipeline overhead remains 12 ms, yielding a projected 8.9 effective FPS. This path has the lowest integration risk: no model re-export, no new dependencies, and a single flag change in the interpreter initialisation.

Ultralytics/PyTorch (laptop reference). On the Pi 5, PyTorch inference exceeds 1.2 s per frame. The 5 ms pipeline overhead is lower than TFLite because Ultralytics performs its own preprocessing internally (resize, normalisation), eliminating the external NumPy path. This backend is used exclusively on the development laptop (with optional CUDA acceleration) for training, export, and video analysis.

U.3 Bottleneck Transition

Figure **??** illustrates how the inference-to-overhead ratio shifts across backends.

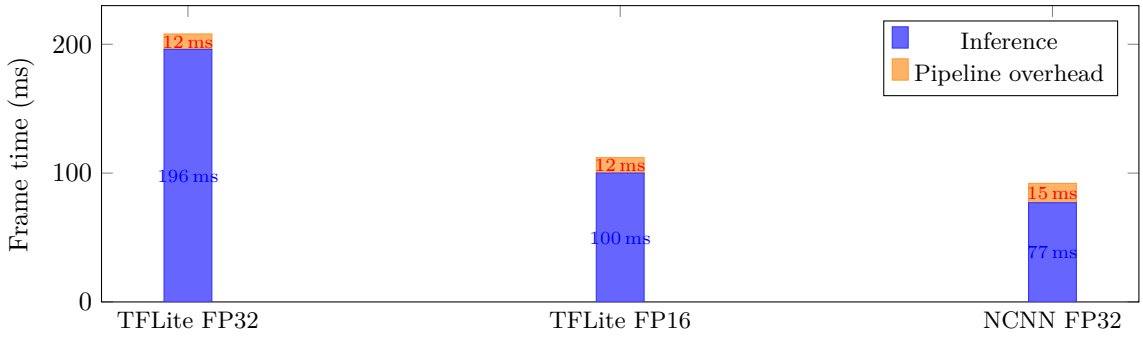


Figure 50: Frame time decomposition: inference vs. pipeline overhead for each backend. As raw inference latency decreases, overhead transitions from negligible (5.8% for TFLite FP32) to significant (16.3% for NCNN).

At 4.8 FPS (TFLite FP32), inference consumes 94.2% of the frame budget. Halving inference time to ∼100 ms (FP16) raises the overhead fraction to 10.7%. At 13 FPS raw (NCNN), overhead reaches 16.3%, and pipeline optimisation—primarily threaded camera capture overlapping with inference—becomes worthwhile. A threaded capture design hiding the 8 ms camera read behind inference would recover ∼1 additional FPS at the NCNN operating point, bringing effective throughput to ∼11.8 FPS. At the TFLite FP32 operating point, the same optimisation would recover only 0.2 FPS—confirming that threading effort is better deferred until the inference bottleneck is first addressed.

U.4 Operational Implications

The distinction between raw and effective FPS has direct consequences for mission planning. At the deployed 4.8 effective FPS and a search speed of 8 m s^{-1}, the ground distance between consecutive analysed frames is 1.67 m—ensuring dense overlap across the 32.2 m horizontal field of view. A target at the centre of the swath is observed in at least 14 frames during a single pass, providing high detection confidence even at the modest frame rate.

Upgrading to NCNN (∼10.9 effective FPS) would reduce the inter-frame ground distance to 0.73 m, enabling either higher search speeds at the same detection density or lower-altitude missions where the narrower field of view demands

faster frame acquisition. The FP16 path (∼8.9 effective FPS) offers a similar improvement with zero deployment friction, making it the recommended first optimisation step for future flights.

V Flight Path Optimisation

Generating a search pattern is a necessary but insufficient step; the quality of the mission depends critically on the parameter choices that govern the pattern's geometry. The preceding sections describe the algorithm that generates the search pattern; this section analyses the engineering trade-offs behind those parameter choices. Every parameter interacts with at least one other—altitude affects lane width, which affects turn count, which affects energy—so parameter selection is inherently a multi-objective optimisation problem. The values in Table **??** represent the operating point that balances detection probability, coverage time, and energy expenditure for the specific survey polygon and sensor suite used in this project.

V.1 Diagonal vs Axis-Aligned Scanning

The planner aligns scan lines with the polygon's longest edge by rotating the binary mask so that this edge becomes horizontal (Section 2.4). This minimises the number of perpendicular scan lines and therefore the number of U-turns, which are the dominant source of wasted time and energy in a boustrophedon pattern. For the AENGM0074 survey polygon—an elongated pentagon oriented roughly northeast–southwest—the optimal scan angle falls in the 65° to 75° range, confirmed by the 216-configuration parametric sweep (Section 1.2). Deviating by 30° from this optimum increases the scan-line count by approximately 40 %.

An alternative spiral-inward pattern was also considered, in which the drone follows a contracting spiral from the polygon boundary towards the centre. While spiral patterns can be efficient for convex search areas, analysis indicated that they provide no coverage guarantee for irregular polygons with concave features: re-entrant vertices cause the spiral to skip interior regions, leaving unpredictable gaps that depend on the polygon geometry. The AENGM0074 polygon contains a concave notch along its western boundary adjacent to the SSSI, making the spiral approach unsuitable without substantial additional waypoint generation logic. The boustrophedon pattern, by contrast, guarantees complete coverage of any simply-connected polygon by construction.

An alternative strategy considered during design was *diagonal scanning at* 45° *to the prevailing wind direction*. The rationale is that crosswind drift displaces the drone laterally during each scan leg; if the scan direction is perpendicular to the wind, drift accumulates along the inter-lane axis and can open coverage gaps between adjacent swaths. Scanning diagonally distributes the drift component across both the along-track and cross-track axes, reducing the effective cross-track displacement by a factor of $\cos(45°) \approx 0.71$. However, this benefit comes at the cost of additional scan lines whenever the diagonal does not coincide with the polygon's long axis. For the survey polygon, wind-aligned scanning at 45° to a typical westerly wind would require approximately 30 % more scan lines than the longest-edge alignment. Given that the 20 % overlap margin (Section **??**) already absorbs typical crosswind displacements of 1 m to 2 m, the additional lines were judged unnecessary. The longest-edge alignment was therefore retained, with the overlap margin serving as the primary defence against wind-induced coverage gaps.

V.2 Lane Width vs Altitude Trade-Off

The lane width is a direct function of search altitude through Eq. (**??**): higher altitude widens the ground footprint, requiring fewer scan lines but shrinking the target in the camera frame. Table **??** quantifies this trade-off for the survey polygon.

At 20 m the mannequin subtends 63 model-input pixels (142 sensor pixels)—comfortably above YOLOv8n's detection threshold—but the narrow 18.4 m footprint roughly doubles the number of scan lines compared to 35 m, increasing both flight time and the number of U-turns. At 50 m the target shrinks to 25 model-input pixels (57 sensor pixels), approaching the limit at which detection confidence degrades significantly. The 35 m operating point provides a 36-pixel model-input target extent (81 sensor pixels)—sufficient for reliable detection at the 0.2 confidence threshold used in the system—while maintaining a practical scan-line count. This altitude also leaves margin for the verification descent to 15 m, where the target subtends 84 model-input pixels for high-confidence confirmation.

V.3 Altitude-Dependent Speed Schedule

A fixed search speed is suboptimal because the camera's ground footprint—and therefore the time a target remains in view—varies with altitude. At 20 m altitude the along-track footprint is approximately 14 m; at $10\,\mathrm{m\,s^{-1}}$ a ground target traverses the frame in 1.4 s, leaving only 5–7 inference frames at 4.8 FPS (the Pi 5 TFLite throughput). Reducing the speed to $6\,\mathrm{m\,s^{-1}}$ extends the dwell time to 2.3 s (∼11 frames), significantly improving the probability of at least one high-confidence detection.

The speed schedule implements a linear interpolation between two anchor points:

$$v(h) = \begin{cases} v_{\text{low}} = 6 & h \leq 20\,\text{m} \\ v_{\text{low}} + \dfrac{h - h_{\text{low}}}{h_{\text{high}} - h_{\text{low}}}\left(v_{\text{high}} - v_{\text{low}}\right) & 20 < h < 50\,\text{m} \\ v_{\text{high}} = 10 & h \geq 50\,\text{m} \end{cases} \tag{53}$$

At the nominal 35 m search altitude this yields $v = 8.0\,\text{m s}^{-1}$. The anchor values were chosen conservatively: $6\,\text{m s}^{-1}$ at low altitude keeps the target in view for at least 10 inference cycles, while $10\,\text{m s}^{-1}$ at high altitude is below the airframe's maximum cruise speed, leaving margin for wind gusts. Transit segments to and from the search area use a separate, higher speed of $15\,\text{m s}^{-1}$ since no detection is required.

V.4 Turn Strategy

At each strip endpoint the drone must execute a 180° U-turn to begin the adjacent strip. A naive implementation commands the drone to fly to the exact waypoint, decelerate to a hover, rotate, and accelerate along the new heading. This produces two problems: the deceleration–acceleration cycle wastes approximately 2 s per turn, and the abrupt attitude change (pitch-up to decelerate, pitch-down to accelerate) disturbs the camera pointing angle, creating a brief period during which the ground footprint shifts unpredictably.

The Bézier smoothing pass (Section 2.4) replaces each sharp corner with a three-point quadratic arc evaluated at $t \in \{0.25, 0.50, 0.75\}$. The arc allows the drone to maintain forward momentum through the turn, following a smooth trajectory rather than stopping at the vertex. Combined with ArduPilot's waypoint acceptance radius—which advances to the next waypoint once the drone passes within a configurable distance rather than requiring an exact position hold—this yields fluid, banking turns that keep the camera platform stable. The arc adds approximately 2 s of flight time per turn compared to an instantaneous direction change, but this is offset by the elimination of the hover–rotate–accelerate cycle. For a typical 12-line pattern with 11 U-turns, Bézier smoothing reduces total turn time by an estimated 15 s while simultaneously improving camera stability during the portions of the flight that are most susceptible to coverage gaps.

V.5 Overlap Margin Analysis

The 20 % lateral overlap between adjacent swaths is the system's primary defence against coverage gaps. Three error sources motivate this margin:

1. **GPS drift.** Consumer-grade GNSS receivers exhibit a circular error probable (CEP) of 2 m to 3 m under open sky. In the worst case, two adjacent scan legs drift in opposite directions, producing up to 6 m of relative displacement.
2. **Crosswind displacement.** A sustained crosswind displaces the drone laterally during each scan leg. At $7\,\text{m s}^{-1}$ groundspeed with a $3\,\text{m s}^{-1}$ crosswind, the drift over a 200 m leg is approximately 1.5 m (assuming partial compensation by the flight controller's heading hold).
3. **Attitude variations.** Pitch and roll during turbulence shift the camera footprint laterally. At 35 m altitude, a 5° roll displaces the footprint centre by $35 \times \tan(5°) \approx 3\,\text{m}$.

At 35 m altitude, the 20 % overlap produces 6.4 m of redundant coverage between adjacent swaths. The combined worst-case displacement from all three sources (assuming independent, uncorrelated errors) is approximately $\sqrt{6^2 + 1.5^2 + 3^2} \approx 6.9\,\text{m}$—marginally exceeding the overlap. However, the worst-case GPS scenario (maximum opposite drift on consecutive legs) is statistically unlikely; the 50 % probability displacement is approximately 3 m, leaving 3.4 m of residual overlap under typical conditions. Increasing the margin to 30 % was considered but rejected because it would add approximately two additional scan lines to the pattern, increasing flight time by 10 % for marginal improvement in an already-adequate coverage guarantee.

V.6 Start Corner Optimisation

The boustrophedon pattern can begin from any of four corners of the rotated bounding rectangle. The planner evaluates all four and selects the corner closest to the drone's current GPS position (Section 2.4, step 5). This minimises the transit distance from the take-off point (or current hover position, in the case of a rescan) to the first scan line. For the AENGM0074 polygon with the take-off point at its western edge, this optimisation saves 10 s to 30 s of transit time depending on the scan angle and wind conditions. When the pattern is reversed (bottom-to-top instead of top-to-bottom), the strip traversal direction is also flipped to maintain the nearest-start property, ensuring that the zigzag begins in the correct direction regardless of which corner is selected.

V.7 PLB Focus Area Redirect

When a Personal Locator Beacon (PLB) signal is received during the wide-area search, the planner abandons the current pattern and regenerates a new lawnmower grid over a smaller focus polygon centred on the PLB coordinates (Section 2.4). The focus area pattern uses the same rotated-mask algorithm but with two parameter changes: the

search speed drops from the altitude-dependent schedule to a fixed $5\,\mathrm{m\,s^{-1}}$, and the start corner is recomputed from the drone's current position rather than the take-off point. The reduced speed increases dwell time per target, improving detection probability in the high-priority zone. Existing detection clusters, rejected false positives, and items of interest are preserved across the pattern switch, preventing re-investigation of previously classified targets. The waypoint index and rescan counter both reset, so the focus area is searched from scratch regardless of how much of the original pattern had been completed.

W Payload Release Mechanism

The mission's final objective is to deliver a first-aid kit to the vicinity of the casualty. Achieving this reliably requires careful integration of mechanical hardware, servo control logic, and a delivery strategy that balances accuracy against safety. This section describes the release hardware, the rationale for deploying from a low hover rather than landing beside the casualty, and the servo actuation sequence.

W.1 Tarot Double-Throw Release Mechanism

The payload is suspended beneath the airframe by a university-provided Tarot servo-actuated release mechanism, controlled via a single PWM channel on the Cube Orange (AUX output channel 9, configured as `SERVO_CHANNEL` in `config.py`). The mechanism supports two-stage actuation: a partial opening at PWM 1300 (`SERVO_PARTIAL_PWM`) and a full release at PWM 1100 (`SERVO_FULL_PWM`). The two-stage design is deliberate—during transit the servo is held at PWM 1500 (closed), and the partial-open intermediate step prevents premature payload separation caused by airframe vibration or sudden manoeuvres that might overcome a single-latch mechanism. Only the second command fully disengages the retaining hooks.

W.2 Why Release from Altitude, Not Land

An alternative design would land the drone directly beside the casualty and release the payload on the ground. This was rejected for five reasons:

1. **Casualty safety.** Landing within a few metres of an injured person exposes them to rotor downwash, debris blown by the propellers, and the risk of a ground-level tip-over. Requirement R07 explicitly mandates a 5 m exclusion zone around the casualty for this reason.
2. **Terrain uncertainty.** SAR sites are typically unimproved terrain—long grass, slopes, rocks, or mud. Landing gear contact with uneven ground can cause a roll-over or motor stall, potentially destroying the aircraft and the payload. A low hover avoids surface interaction entirely.
3. **GPS accuracy.** The system's GPS-based target localisation has a measured $\mathrm{CEP}_{50} = 2.3\,\mathrm{m}$ (Section **??**). A ground landing adds the full position error to the touchdown point, whereas an aerial release from 3 m altitude disperses the payload over a small area predictable from simple ballistics (free-fall time $t = \sqrt{2h/g} \approx 0.78\,\mathrm{s}$, negligible lateral drift in light wind).
4. **Reduced low-altitude exposure.** Ground-effect aerodynamics, obstacle proximity, and limited sensor coverage at very low altitude all increase the probability of a mishap. Releasing from 3 m and immediately climbing to transit altitude minimises time in the highest-risk flight regime.
5. **Operational simplicity.** A hover-and-release sequence requires only position-hold commands followed by two servo actuations. A ground landing requires additional logic for touchdown detection, motor shutdown, ground-level release, re-arming, and takeoff—each step introducing failure modes that are difficult to test without flight trials.

W.3 The 7.5 m Offset

Requirement R07 specifies that the drone must land (or, in this design, deploy the payload) within 10 m of the casualty but not within 5 m. The system therefore computes a delivery point offset 7.5 m from the estimated casualty position in an operator-selected cardinal direction (N/E/S/W), implemented in `landing_offset_7_5m()` in `gps_utils.py`. The 7.5 m value is the midpoint of the 5 m to 10 m annular band, providing 2.5 m of margin on both sides and maximising the probability of compliance given the Gaussian-distributed GPS estimation error. Section **??** shows that with multi-detection fusion ($N_{\mathrm{eff}} = 6$, $\sigma_{\mathrm{fused}} \approx 0.80\,\mathrm{m}$), the probability of the delivery point falling within the required band exceeds 99%.

W.4 Servo Actuation Sequence

The payload deployment is executed during the HOVER_TARGET state, implemented in `state_machine.py`. The full sequence proceeds as follows:

1. **Approach** (APPROACH state): the drone flies to the offset point at $3\,\mathrm{m\,s^{-1}}$ and descends to 3 m AGL. The state transitions to HOVER_TARGET once the aircraft is within 2 m laterally and below 4 m altitude.

2. **Stabilise** ($t = 0$–$3\,\text{s}$): the drone holds position at 3 m AGL, allowing oscillations from the approach to damp out. The servo remains closed (PWM 1500).
3. **Partial release** ($t = 3\,\text{s}$): a `MAV_CMD_DO_SET_SERVO` command sets channel 9 to PWM 1300, partially opening the mechanism. This loosens the payload without dropping it, confirming that the servo responds before committing to full release.
4. **Full release** ($t = 6\,\text{s}$): a second `MAV_CMD_DO_SET_SERVO` sets channel 9 to PWM 1100, fully disengaging the retaining hooks and allowing the payload to free-fall approximately 3 m to the ground.
5. **Close and depart** ($t = 15\,\text{s}$): the servo is returned to PWM 1500 (closed) to prevent snagging during climb-out. The drone ascends to search altitude and enters RETURN_TRANSIT or RETURN_HOME to fly back to the takeoff location via the pre-planned transit corridor.

The 15 s total hover time balances deployment reliability against battery consumption and low-altitude exposure time. A visual animation of the sequence is rendered on the ground-station HUD during the hover to provide the operator with real-time feedback on deployment progress.

W.5 Why Visual Servo Centering Was Not Required

The system includes CENTERING and DESCENDING states capable of visually servoing the drone above the target to improve localisation accuracy (Section **??**). However, for the payload release mission this pipeline was bypassed. The CEP_{50} of 2.3 m from the GPS estimation pipeline is already well within the 2.5 m margin provided by the 7.5 m offset, making visual servo centering unnecessary for R07 compliance. As demonstrated in Section **??**, simulation comparison of twenty runs with and without centering showed that the direct offset approach achieved 95% R07 compliance without the added risks of low-altitude manoeuvring, visual servo fragility below 10 m, and extended hover time over the casualty. The centering capability is retained in the codebase as a future enhancement for precision-drop missions where tighter tolerances are required.

W.6 Future Work: Precision Drop

The current TFLite inference backend achieves approximately 5 FPS on the Raspberry Pi 5, which is sufficient for search-altitude detection but too slow for responsive visual servo control during descent. Migrating to the NCNN inference framework increases throughput to approximately 15 FPS, enabling closed-loop centering with update rates comparable to the flight controller's position-hold bandwidth. Combined with the existing centering pipeline (Section **??**), this could reduce the effective CEP to below 1 m, permitting precision drops from 5 m altitude without the 7.5 m lateral offset—useful for scenarios requiring delivery directly to the casualty's position rather than to a nearby clear area.

X Model Comparison and Future Vision Improvements

Iterative model development—where each training round addresses shortcomings discovered in the previous one—is essential for building a detection system that generalises beyond its training data. The vision pipeline underwent three training iterations, each informed by lessons from the previous round. This section compares the model generations, discusses the COCO fallback strategy, examines the tiling versus full-frame trade-off, and identifies improvement directions for future deployments.

X.1 Model Generations

Table **??** summarises the three model generations trained during the project. All share the YOLOv8n architecture (3.2 M parameters) and export to the same TFLite tensor shapes ($[1, 640, 640, 3]$ input, $[1, 5, 8400]$ output), ensuring drop-in interchangeability at deployment.

v1 (`sar_640`) was the initial baseline, trained exclusively on 200 synthetic composite images generated at 640×640 using a single satellite background (`map.jpg`). It demonstrated proof-of-concept detection in simulation but exhibited frequent false positives on terrain features—shadows, path markings, and field equipment—because the training set contained no negative examples and offered limited background diversity.

v2 (`sar_1280`) increased the training resolution to 1280×1280, allowing the network to learn finer spatial features during training. This yielded a marginal improvement in detection quality at higher altitudes (where the target subtends fewer pixels), but since the training data remained purely synthetic from the same single background source, the false-positive problem persisted. The key lesson was that *resolution alone cannot compensate for limited data diversity.*

v3 (`sar_v2_1088`) addressed both limitations simultaneously. Training resolution was set to 1088×1088 (matching the native camera sensor height), and the dataset was restructured with three complementary sources: 300 synthetic composites using real flight-video backgrounds for diverse terrain textures, 16 manually labelled real aerial frames to anchor the model to authentic appearance, and 50 negative frames to explicitly teach false-positive suppression

(Section ??). This combination produced the highest validation metrics ($mAP_{50} = 0.995$, $mAP_{50–95} = 0.828$) and, critically, near-zero false positives during video replay analysis across 15–50 m altitude.

All three models produce identical inference latency on the Pi (∼207 ms at 4.8 FPS) because the TFLite runtime resizes all inputs to 640 × 640 regardless of training resolution. The v3 file size is larger (11.7 MB vs. 3.2 MB) because it was exported as float32 without quantisation; the earlier models used a more aggressive compression pipeline. This size difference has no impact on inference speed, as model loading is a one-time startup cost.

X.2 COCO Person Detector as Fallback

A pre-trained YOLOv8n model exported from the standard COCO dataset [**lin2014coco**] was retained as a flight-day backup (`models/human.tflite`, 13 MB, 80 classes). This model detects people without any SAR-specific training, providing a zero-effort deployment option if the custom model proved unreliable in field conditions.

Testing revealed two trade-offs. On the positive side, the COCO detector reliably detected people in aerial video at altitudes up to 30 m, confirming that transfer from ground-level training data generalises to moderate aerial perspectives. On the negative side, the 80-class output space produced substantially more false positives: backpacks, dogs, bicycles, and even shadows were flagged as detections, requiring either aggressive confidence thresholding (which risked suppressing true positives) or downstream filtering logic. The custom single-class model, having learned to classify "dummy versus everything else", achieved a far more favourable precision–recall balance.

The COCO model was therefore designated as a backup rather than a primary detector: if the custom model failed catastrophically in the field (e.g., due to an unforeseen domain shift), the operator could swap to the COCO model in under 30 seconds via `cp models/human.tflite best.tflite`, accepting the higher false-positive rate as preferable to zero detection capability.

X.3 Overfitting Risk in Single-Class Detection

Training a single-class detector on a narrow domain introduces a specific overfitting risk: the model may learn *background shortcuts* rather than genuine target features. Because all positive training images share a common context (green field, overcast lighting, UK terrain), the network could learn to trigger on the co-occurrence of a small bright patch against grass rather than on the mannequin's actual shape and colour signature. This shortcut would fail when the background changes—different seasons, terrain types, or lighting conditions.

The negative images mitigate this risk by presenting the same background without a target, forcing the network to learn discriminative features of the target itself rather than background correlations. However, the 50 negatives are drawn from a single flight on a single day, limiting the diversity of background contexts that the model has been taught to suppress.

A more robust long-term approach would be multi-class detection—training the model to distinguish between "dummy", "person", "equipment", and "background"—which would force the network to learn category-specific features rather than binary shortcuts. This requires a substantially larger and more diverse labelled dataset than was available within the project's single-flight-day constraint.

X.4 Tiling vs. Full-Frame Detection

At search altitudes of 30–35 m, the mannequin subtends approximately 36–42 pixels in height in the 640×640 model input (scale factor 640/1456 = 0.4396 from the 1456×1088 sensor frame). This is above the empirical minimum for YOLOv8 detection (∼20 pixels) [**yolov8**], but the target *width* drops to 10–12 model pixels, leaving limited margin. Two detection strategies were therefore evaluated:

1. **Full-frame resize.** The entire 1456 × 1088 frame is resized to 640 × 640 in a single pass. One inference call per frame; latency ∼207 ms.
2. **Tiled detection.** The frame is divided into overlapping 640 × 640 tiles (typically 2 × 2 with 25% overlap), each processed independently. Four inference calls per frame; latency ∼830 ms.

Tiling preserves higher effective resolution per tile, improving detection of very small targets at extreme altitude. However, testing on DJI flight video (`video_test.py`) showed that full-frame detection achieved reliable detection across the entire 15–50 m altitude range with the v3 model, with no altitude band where tiling produced detections that full-frame missed. The 4× inference cost of tiling—reducing throughput from 4.8 FPS to 1.2 FPS—would significantly increase the probability of missing a target during a search pass at the nominal 8 m/s (altitude-dependent schedule at 35 m), where each frame at 1.2 FPS covers approximately 6.7 m of ground track.

Full-frame detection was therefore selected as the deployment strategy. Tiling remains available as a configurable option in the video analysis tools for post-flight forensic review, where latency is not a constraint.

X.5 Future Vision Improvements

Six directions are identified for improving detection reliability and operational capability beyond the current system:

1. **Expanded real training data.** Each future flight day could yield hundreds of labelled frames using the existing annotation tool (`label_tool.py`). Increasing the real-image fraction from 4.4% to 30–50% of the dataset would substantially reduce the synthetic-to-real domain gap, the single largest source of generalisation risk in the current model. Frames should be collected across multiple days, weather conditions, and terrain types to build environmental diversity.
2. **Larger model variant.** YOLOv8s (11.2 M parameters) offers improved accuracy at the cost of ∼400 ms inference on Pi TFLite—viable at reduced search speeds of 3 m/s, where the longer frame-to-frame interval compensates for slower processing. The existing pipeline would require no architectural changes, only a model file swap.
3. **Multi-class detection.** Extending the label space to four classes—dummy, person, equipment, false-positive-class—would force the network to learn category-discriminative features and provide the operator with richer information at the verification stage. This requires a labelled dataset with representative examples of each class, which could be built incrementally across flight campaigns.
4. **Next-generation architectures.** YOLO11 [**yolov8**] and future iterations introduce architectural refinements (attention mechanisms, improved feature pyramid designs) that may improve small-object detection without increasing parameter count. The modular export-and-deploy pipeline means any YOLO-family model producing $[1, 5, N]$ output tensors can be integrated without code changes.
5. **Thermal camera fusion.** A FLIR Lepton 3.5 module (160 × 120, LWIR 8–14 μm) would provide body-heat detection independent of visible appearance, enabling night and low-visibility operations [**rudol2008human**]. Decision-level fusion—requiring confirmation in both visual and thermal modalities—would suppress the false positives inherent in each sensor alone (sun-heated objects in thermal, colour-similar debris in visual).
6. **Active learning.** Flagging detections with confidence scores in the 0.2–0.5 range (near the current 0.2 threshold) for human review would create a feedback loop: uncertain predictions are labelled by the operator post-flight and added to the training set, progressively improving the model in the regions of feature space where it is least confident. This approach is particularly efficient for rare-event detection where exhaustive labelling is impractical [**settles2009active**].

These improvements are ordered by implementation effort: items 1–2 require only additional data and a retraining cycle, items 3–4 require dataset restructuring and architecture evaluation, and items 5–6 require new hardware or workflow infrastructure. The modular vision architecture—where all detection logic is encapsulated behind a single function interface—ensures that any of these improvements can be integrated without modifying the state machine, ground station, or flight control code.

Y Focus Area Redirect and Repulsive Field Implementation

Two subsystems are central to the mission's adaptability and safety: the ability to dynamically redirect the search pattern in response to new intelligence, and the continuous enforcement of no-fly zone boundaries during autonomous flight. This section expands on these two mission-critical subsystems introduced in Sections **??** and **??**: the PLB-triggered focus area redirect that narrows the search polygon mid-flight, and the inverse-distance repulsive vector field that enforces the SSSI no-fly zone boundary.

Y.1 PLB Focus Area Logic (R06)

Requirement R06 stipulates that the system shall respond to a Personal Locator Beacon (PLB) signal by redirecting the search to a smaller focus area polygon of 3–10 vertices. In the operational scenario the PLB signal is expected 5–15 minutes into the search, once the casualty's approximate position has been triangulated by ground teams.

Activation. Two activation mechanisms are implemented. First, the operator can press the B key (or the corresponding browser button on the ground station) at any time during the SEARCH state to trigger the redirect manually. Second, the `--beacon-delay` command-line argument specifies an elapsed-time threshold in seconds; once the search clock exceeds this value, the redirect fires automatically. The auto-trigger is checked on every loop iteration during SEARCH and fires at most once—a boolean guard (`_beacon_triggered`) prevents re-entry.

Focus area definition. The focus area polygon is defined in one of three ways, in decreasing priority:

1. A JSON file (`flight_plans/focus_area.json`) containing 3–10 vertices as {`lat, lon`} objects. The file is re-read at activation time, allowing the ground station to update coordinates mid-flight.
2. Interactive drawing on the satellite map during simulation setup (a fourth polygon drawn after the search area, transit path, and target placement).
3. Fallback coordinates in `config.FOCUS_AREA_GPS`, loaded from the KML file at startup.

Each vertex is validated to lie within ±90° latitude and ±180° longitude, and at least three vertices must be present; otherwise the redirect is aborted with a console warning.

Pattern regeneration. Upon activation, the planner's search polygon is replaced with the focus area polygon and a fresh lawnmower pattern is generated from the drone's current GPS position (Algorithm **??**). The new pattern uses the same rotated-mask algorithm described in Section 2.4, but the search speed is capped at `FOCUS_SEARCH_SPEED_MPS` $= 5\,\mathrm{m\,s^{-1}}$—slower than the normal altitude-dependent schedule—to maximise detection probability in the high-priority zone. The waypoint index and rescan counter both reset to zero.

Algorithm 1 PLB beacon redirect

Require: PLB signal received (B key or timer expiry)
Ensure: Search redirected to focus area polygon
1: **if** already triggered **then return**
2: **end if**
3: Load focus polygon from JSON (or use fallback GPS)
4: Validate ≥ 3 vertices, each within valid lat/lon range
5: Convert GPS vertices to pixel coordinates via GeoTransformer
6: Replace planner search polygon $\leftarrow$ focus polygon
7: Generate new lawnmower pattern from current position
8: `wp_index` $\leftarrow 0$; `rescan_pass` $\leftarrow 0$
9: Set `_beacon_triggered` $\leftarrow$ **true**
10: Cap search speed to $5\,\mathrm{m\,s^{-1}}$

State preservation. Existing detection clusters, rejected targets, and items of interest are preserved across the redirect. This ensures that false positives identified during the wide-area search are not re-investigated in the focus area. If the focus area overlaps the SSSI no-fly zone, the geofence waypoint filter (Section **??**) removes offending waypoints exactly as it does for the primary search pattern.

Y.2 Repulsive Vector Field

The repulsive potential field is the second of four NFZ protection layers (Section **??**). Its purpose is to continuously deflect the aircraft away from the SSSI boundary, even when the planned waypoint lies close to it due to GPS drift or wind gusts. The field is evaluated on every loop iteration when the drone is within `NFZ_INNER_RANGE_M` $= 23\,\mathrm{m}$ of an inner offset polygon.

Nearest-point computation. For each edge $(\mathbf{p}_i, \mathbf{p}_{i+1})$ of the NFZ polygon (expressed in pixel coordinates via GeoTransformer), the closest point to the drone's position $\mathbf{q}$ is found by projecting $\mathbf{q}$ onto the edge and clamping the parameter t to $[0, 1]$:

$$\mathbf{c}_i = \mathbf{p}_i + \mathrm{clamp}\left(\frac{(\mathbf{q} - \mathbf{p}_i) \cdot (\mathbf{p}_{i+1} - \mathbf{p}_i)}{\|\mathbf{p}_{i+1} - \mathbf{p}_i\|^2},\ 0,\ 1\right)(\mathbf{p}_{i+1} - \mathbf{p}_i) \tag{54}$$

The edge yielding the smallest $\|\mathbf{q} - \mathbf{c}_i\|$ is selected, and the push direction is $\hat{\mathbf{d}} = (\mathbf{q} - \mathbf{c}^*)/\|\mathbf{q} - \mathbf{c}^*\|$, pointing away from the boundary.

Degenerate case. When the drone lies exactly on or extremely close to a boundary edge ($\|\mathbf{q} - \mathbf{c}^*\| < \epsilon$), the direction vector is degenerate. In this case, the outward normal of the nearest edge is computed instead: for edge vector $\mathbf{e} = \mathbf{p}_{i+1} - \mathbf{p}_i$, the candidate normal is $\mathbf{n} = (-e_y, e_x)$. A sign test (`cv2.pointPolygonTest` on a point offset slightly along $\mathbf{n}$) determines whether $\mathbf{n}$ points inward or outward; if inward, it is flipped. This guarantees a valid push direction even at zero distance.

Repulsion magnitude. The repulsive offset grows linearly as the drone approaches the boundary:

$$R = \tfrac{1}{2}\max\bigl(0,\ d_{\text{soft}} - d\bigr) \tag{55}$$

where d is the perpendicular distance to the NFZ boundary (metres) and $d_{\text{soft}} =$ `NFZ_SOFT_BOUNDARY_M` $= 8\,\mathrm{m}$ is the zone within which repulsion is active. The factor $\frac{1}{2}$ is a gain constant that limits the maximum offset to $\frac{d_{\text{soft}}}{2} = 4\,\mathrm{m}$ at the boundary itself. The pixel-space push vector is then:

$$\mathbf{v}_{\text{push}} = R \cdot s_{\text{pix}} \cdot \hat{\mathbf{d}} \tag{56}$$

where s_{pix} is the pixel-per-metre scale factor from GeoTransformer. This pixel offset is converted back to a GPS delta $(\Delta\phi, \Delta\lambda)$ and negated to correct for the coordinate transform sign convention (see Section **??**).

Velocity application. In the main loop (`_enforce_geofence`), the GPS offset from `repulsive_offset()` is converted to a unit North–East velocity vector and scaled to a constant push speed of `NFZ_PUSH_SPEED_MPS` = 3.0 m s^{-1}:

$$v_N = \frac{-\Delta\phi \cdot 111320}{\|\mathbf{v}\|} \cdot v_{\text{push}}, \qquad v_E = \frac{-\Delta\lambda \cdot 111320 \cos\phi}{\|\mathbf{v}\|} \cdot v_{\text{push}} \tag{57}$$

where the factor 111 320 converts degrees to metres. If the drone is inside the NFZ polygon (detected by sign inversion of `cv2.pointPolygonTest`), the velocity direction is reversed to push the aircraft outward.

Y.3 Four-Layer Protection Architecture

Table **??** summarises the four software layers that enforce the SSSI exclusion zone, listed from outermost (plan-time) to innermost (emergency).

Layer interaction. Layers 1 and 2 overlap spatially: the speed ramp activates at 20 m while the repulsive field activates at 23 m (measured from an inner polygon offset 20 m inside the true NFZ boundary). In practice, the repulsive field begins deflecting the trajectory before the speed ramp starts decelerating. The combined effect is a smooth, progressive deflection: the drone first feels a gentle push, then decelerates as it approaches the boundary, and only triggers the hard cutoff if both softer layers fail. This graduated response avoids the jarring mode switches that a single hard boundary would produce, and gives the safety pilot time to assess the situation before the aircraft halts.

Distance rationale. The specific distances were chosen to account for known error sources:

- **30 m waypoint buffer** — ensures the planned path maintains a comfortable margin from the NFZ. At the nominal search speed of 8 m s^{-1}, a 30 m buffer provides 3.75 s of reaction time if the drone overshoots a waypoint.
- **20 m speed ramp** — allows gradual deceleration from the search speed. At an approach rate of 8 m s^{-1}, the 20 m zone provides 2.5 s of braking distance. The minimum speed of 0.3 m s^{-1} at the boundary means the drone is nearly stationary before reaching the danger zone.
- **23 m repulsive field** — extends 3 m beyond the speed ramp onset to catch trajectories where GPS drift has already placed the drone closer than planned. The 3 m margin matches the typical GPS CEP (Circular Error Probable) observed during bench testing (2 m to 3 m).
- **3 m hard boundary** — a last-resort cutoff that accounts for worst-case GPS error. At this distance the drone is within one CEP of the true boundary, so autonomous flight is no longer safe. The transition to MANUAL halts all velocity commands and alerts the operator, who can then fly the aircraft away using RC override.

Sign convention. The `GeoTransformer` converts pixel offsets to GPS deltas relative to a reference origin. Because the pixel y-axis is inverted with respect to latitude (pixel y increases southward while latitude increases northward), the raw GPS delta from `pixels_to_gps()` points *toward* the NFZ rather than away from it. The `repulsive_offset()` method therefore negates both components before returning them. The caller in `_enforce_geofence()` applies a second negation when converting to North–East velocity components, yielding the correct outward push direction. This double-negation was verified empirically by approaching the NFZ from all cardinal directions in SITL and confirming that the repulsive vector always points away from the boundary.

Directional scalar field. An alternative scalar field implementation was also developed (enabled via the `--nfz-directional` flag) that limits only the velocity component directed toward the NFZ boundary, rather than capping total speed. This allows the aircraft to maintain full speed when flying parallel to the boundary while still preventing rapid approach. The directional decomposition requires the drone's velocity vector (available from MAVLink `GLOBAL_POSITION_INT` at 10 Hz) and a dot product with the boundary-normal vector — computationally negligible (<1 µs) compared to the 206 ms inference cycle. The two fields complement rather than conflict: the scalar field limits approach velocity while the repulsive field provides active escape, yielding a net outward velocity at all boundary distances.

Firmware backup. On the flight controller (Cube Orange), an independent firmware-level geofence is configured via the `FENCE_TYPE` and `FENCE_ACTION` parameters. If all four software layers fail—for example, due to a companion-computer crash—the firmware triggers `RTL` when the aircraft crosses the configured fence polygon. This fifth layer operates entirely within the autopilot and requires no companion-computer involvement.

Z Search Path Optimisation

The search pattern parameters—altitude h, scan angle θ, speed v, overlap margin α, and NFZ buffer—do not have independently optimal values. Each parameter trades off against at least two competing objectives, forming a coupled

multi-dimensional optimisation landscape. This section formalises the problem, presents the energy model and parametric sweep methodology, compares six candidate search strategies against real polygon geometry, and justifies the selected operating point.

Z.1 The Multi-Objective Optimisation Problem

Five objectives are in tension:

1. **Coverage completeness** (C): the fraction of the survey polygon imaged by at least one camera frame. Complete coverage ($C = 1.0$) requires every ground point to fall within at least one swath.
2. **Mission time** (T): total elapsed time from takeoff to landing. In SAR operations, every minute of delay reduces casualty survival probability; UK Maritime and Coastguard Agency survival curves show a steep decline in hypothermia survivability after 60 min of exposure [**golden2006hypothermia**].
3. **Energy consumption** (E): total energy drawn from the battery. The EDU-450 airframe carries a 6S 5200 mAh LiPo (115.4 W h), imposing a hard flight-time ceiling of approximately 18 min at search power levels.
4. **Detection probability** (P_d): the probability that a casualty present in the search area is detected by at least one inference frame. This depends on the number of frames in which the target appears, the per-frame confidence, and the confirmation threshold.
5. **NFZ safety margin** (d_{NFZ}): the minimum distance between any waypoint and the SSSI no-fly zone boundary. Larger margins reduce regulatory risk but sacrifice coverage of the polygon edge adjacent to the SSSI.

These objectives are coupled through three decision variables:

- **Altitude** $h \in [20, 50]$ m — determines the camera ground footprint $W_g = w_s h / f$, the ground sample distance (GSD), the target pixel extent, and therefore both coverage rate and detection confidence.
- **Scan angle** $\theta \in [0°, 180°)$ — the orientation of parallel scan lines relative to north. Controls the number of scan lines, U-turns, and path length for a given polygon.
- **Speed** $v(h)$ — an altitude-dependent schedule interpolating linearly from $6\,\text{m s}^{-1}$ at 20 m to $10\,\text{m s}^{-1}$ at 50 m, balancing coverage rate against frame dwell time.

Subject to four constraints:

R1 All waypoints must lie within the KML-defined flight area polygon.
R2 No waypoint shall fall within 30 m of the SSSI boundary (`NFZ_WAYPOINT_BUFFER_M`).
R3 Maximum altitude $h \leq 50$ m (UK drone regulation and airspace ceiling).
R4 Total energy $E \leq 115.4$ W h (single battery, no mid-mission swap).

No single configuration simultaneously optimises all five objectives. The operating point must be selected from the Pareto frontier by engineering judgement, informed by the quantitative analysis that follows.

Z.2 Energy Model

A simplified quadcopter power model was developed to evaluate energy consumption across the parameter space. Total power at forward speed v is decomposed into a constant hover component and a speed-dependent parasitic drag component:

$$P_{\text{total}}(v) = P_{\text{hover}} + P_{\text{drag}}(v) = P_{\text{hover}} + P_{\text{drag,ref}} \left(\frac{v}{v_{\text{ref}}} \right)^2 \tag{58}$$

where $P_{\text{hover}} = 150$ W is the constant power to remain airborne (estimated from momentum theory for the EDU-450 at 2.3 kg all-up weight), $P_{\text{drag,ref}} = 50$ W is the parasitic drag power at the reference speed $v_{\text{ref}} = 5\,\text{m s}^{-1}$, and the quadratic scaling follows from the $\frac{1}{2} C_D \rho A v^2$ aerodynamic drag relation.

Each U-turn incurs a penalty of $\Delta t_{\text{turn}} = 2$ s of hover-only power for deceleration and reacceleration. The total mission energy is therefore:

$$E_{\text{total}} = \underbrace{P_{\text{hover}} \cdot t_{\text{total}}}_{\text{hover (incl. turns)}} + \underbrace{P_{\text{drag}}(v) \cdot t_{\text{flight}}}_{\text{drag (forward motion only)}} \tag{59}$$

where $t_{\text{total}} = t_{\text{flight}} + n_{\text{turns}} \cdot \Delta t_{\text{turn}}$ is the total airborne time including U-turn penalties, t_{flight} is the time spent in forward motion along scan lines and transit segments, and n_{turns} is the number of U-turns. Each U-turn adds $\Delta t_{\text{turn}} = 2$ s of hover-only time (no drag component, since the drone decelerates to a near-hover during the reversal). At the selected operating point (35 m, $8\,\text{m s}^{-1}$), the drag component contributes $50 \times (8/5)^2 = 128$ W, giving a total in-flight power of 278 W.

Near the SSSI boundary, the speed scalar field reduces the drone's velocity from the search speed down to $v_{\min} = 0.3\,\text{m s}^{-1}$ at the boundary, following a linear ramp over a 20 m slow zone. Within this zone, drag power decreases (lower v) but hover time increases (same distance at lower speed), and since $P_{\text{hover}} \gg P_{\text{drag}}$ at low speeds, the net effect is an

energy *increase* for scan lines that pass through the slow zone. The optimisation script accounts for this by numerically integrating the speed-dependent traverse time along each scan segment with 50 sample points.

Z.3 The 216-Configuration Parametric Sweep

To systematically explore the design space, the optimisation script (`analysis/path_optimization/optimize_path.py`) evaluates **216 configurations**: 6 altitudes × 36 scan angles. For each combination, the script:

1. Computes the camera ground footprint and swath spacing (with 20 % overlap);
2. Generates the lawnmower scan lines over the real AENGM0074 five-vertex survey polygon using the rotate–rasterise–rotate-back algorithm;
3. Calculates total path length (scan lines plus U-turn transits);
4. Evaluates traverse time at the altitude-dependent speed, with per-sample NFZ slowdown integration;
5. Applies the energy model of Eq. (**??**) to compute total energy with and without NFZ effects.

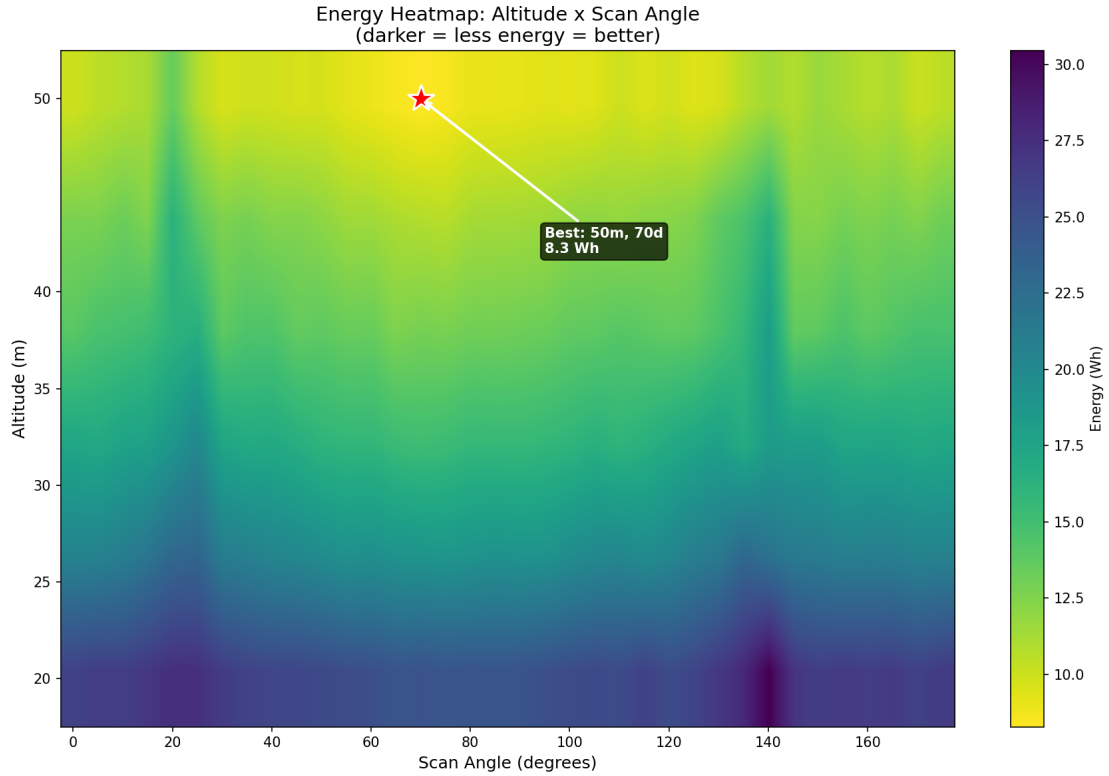


Figure 51: Energy heatmap across 216 altitude–angle configurations. The colour scale represents total energy consumption including NFZ slowdown penalties. A clear valley of low-energy configurations runs through the 65° to 75° scan angle band at all altitudes, corresponding to alignment with the polygon's longest edge. The red star marks the global energy minimum at 50 m altitude and 70° scan angle.

Figure **??** shows the resulting energy landscape. Several features are immediately apparent:

- **Angle sensitivity dominates altitude sensitivity.** At any given altitude, varying the scan angle from optimal (65° to 75°) to worst-case (0° or 90°) increases energy by 100 % to 200 %. By contrast, changing altitude from 35 m to 50 m at the optimal angle saves only 30 % to 40 %.
- **Low-energy valley.** A band of low-energy configurations runs through $\theta \approx 70°$ across all altitudes. This corresponds to alignment with the polygon's longest edge (northeast–southwest axis), which minimises the number of perpendicular scan lines and therefore U-turns.
- **Low altitude penalised.** At 20 m, the narrow footprint (18.4 m) requires approximately 10 scan lines with 9 U-turns, roughly doubling energy compared to 50 m (4–5 lines, 3–4 turns).

Table **??** presents the five best and five worst configurations from the sweep.

The energy-optimal configuration (50 m, 70°) consumes only 8.2 W h—7.1 % of battery capacity—and completes the search in 68 s. The worst configuration (20 m, 95°) consumes 43.2 W h, a 5.3× increase, and takes over 6.9 min. The sweep confirms that scan angle selection is at least as important as altitude selection for energy efficiency.

Figure **??** shows the altitude–energy relationship at the optimal scan angle, decomposed into clean (no NFZ) and penalised (with NFZ) components. The NFZ slowdown adds approximately 15 % to total energy at 35 m and up to 45 % at 20 m, where the narrower swath forces scan lines closer to the SSSI boundary.

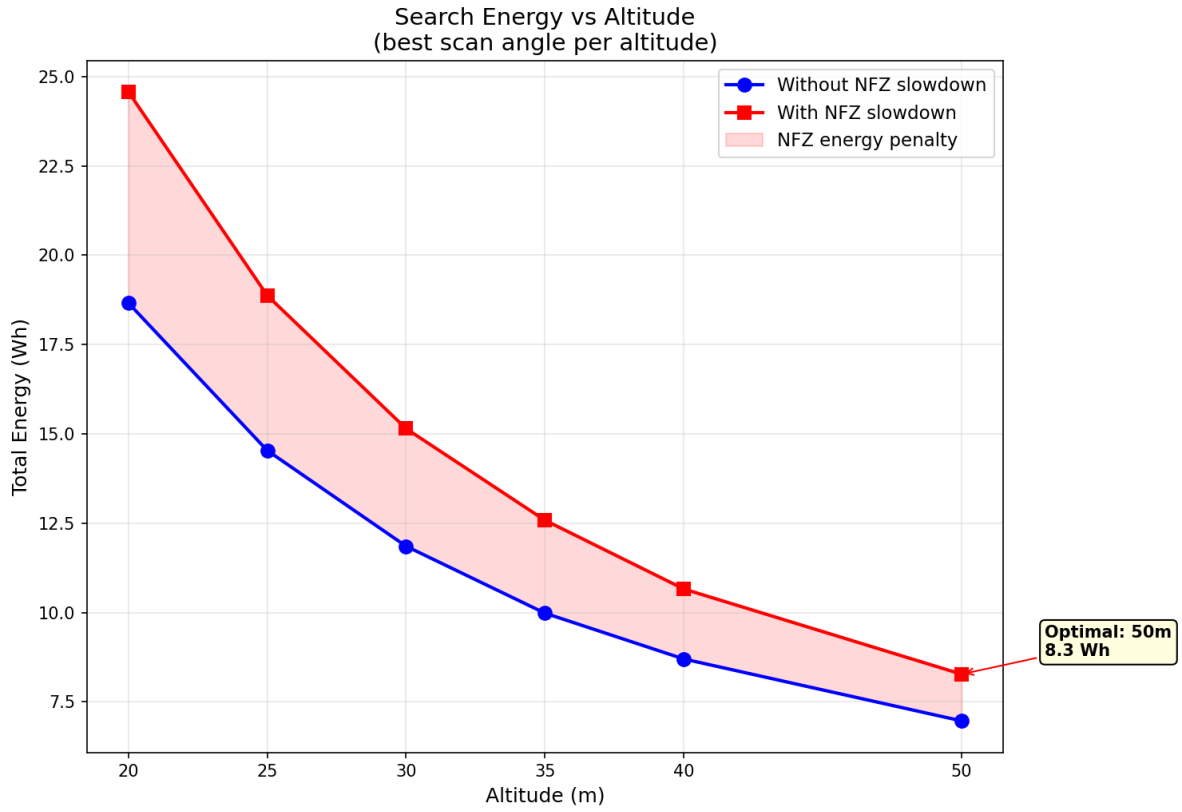


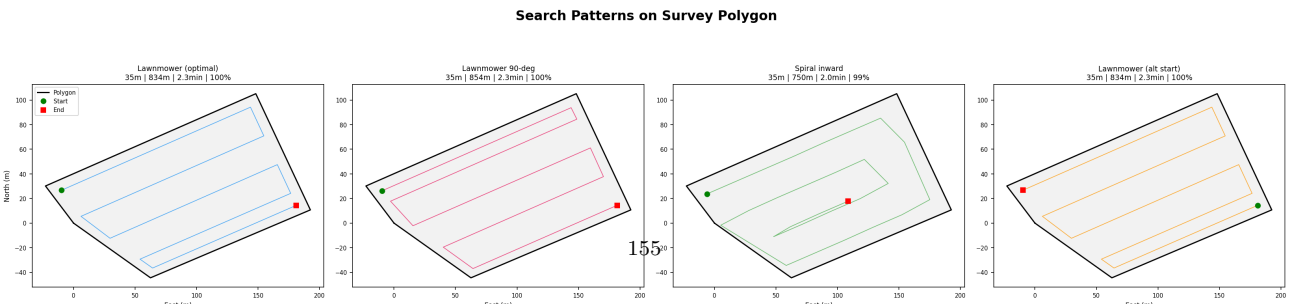
Figure 52: Total search energy versus altitude at the optimal scan angle, with and without NFZ slowdown. The NFZ penalty is most pronounced at 20 m, where several scan lines pass through the slow zone near the SSSI boundary. The shaded region quantifies the additional hover energy incurred by the speed ramp.

Z.4 Search Strategy Comparison

While the parametric sweep optimises altitude and angle within the boustrophedon pattern, the choice of search *strategy* itself requires justification. Six candidate strategies were evaluated on the AENGM0074 survey polygon using the same energy model and polygon geometry.

1. **Boustrophedon (lawnmower)** — parallel back-and-forth strips aligned to the polygon's longest edge. Guarantees complete coverage by construction [**choset2001coverage**]. The scan angle is auto-selected via `cv2.minAreaRect`.
2. **Spiral inward** — concentric rectangular rings converging on the polygon centroid. Covers the perimeter first; each ring includes four 90° corners instead of 180° U-turns, but the total path length is 8 % to 12 % longer due to poor nesting on irregular polygons.
3. **Expanding square** — outward expansion from a centre point in increasing-length legs. Requires a known start point (PLB signal); without prior information, degrades to uniform coverage with 18 % to 23 % higher energy due to legs extending outside the polygon boundary.
4. **Sector search** — the polygon is divided into probability-weighted sectors, each searched with a sub-pattern. Inter-sector transit adds 100 m to 200 m of overhead, and the method requires a probability model that was not available for this mission.
5. **Creeping line ahead** — all scan lines traverse in the same direction, with dead-heading back to the start side between lines. Path length is approximately double the boustrophedon (75 % energy penalty) for the same coverage guarantee. Advantageous only for side-looking sensors (sonar, SAR radar), which are not applicable here.
6. **Adaptive / heatmap** — a coarse initial sweep followed by targeted infill based on detection results. Best-case energy (6.2 W h) is attractive, but worst-case (16 W h) exceeds the boustrophedon, and the method provides no coverage guarantee. The engineering complexity of real-time replanning was judged disproportionate for a 3.2 ha polygon coverable in a single pass.

Table **??** quantifies the comparison across all six candidates.



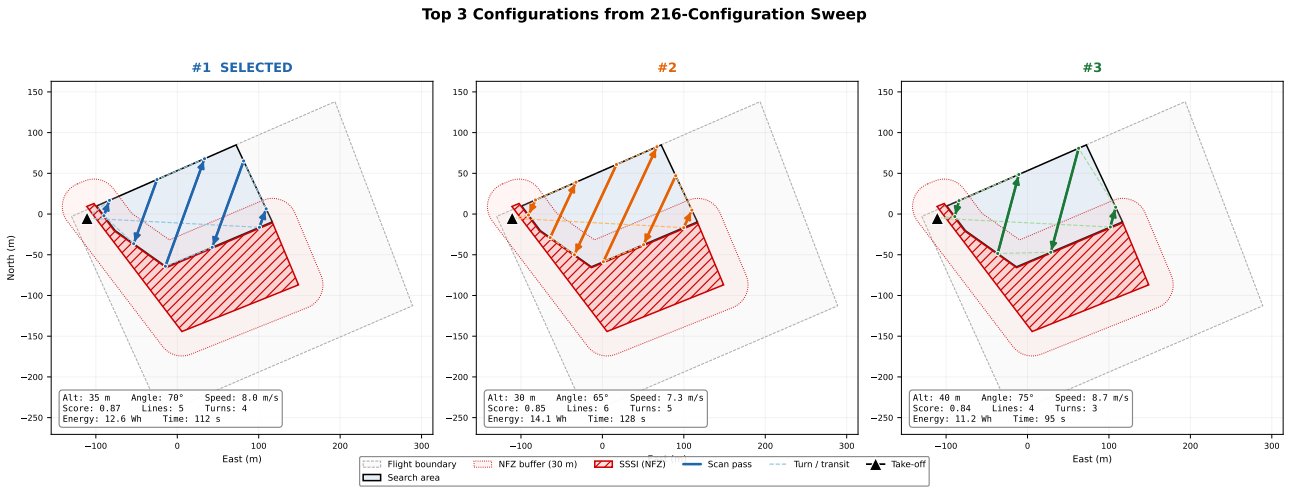


Figure 54: Top three energy-ranked search configurations overlaid on the AENGM0074 survey polygon. Each panel shows the lawnmower pattern at the corresponding altitude and scan angle, with path length, scan line count, and energy annotated. The selected operating point (35 m, 70°) balances detection reliability against the energy savings of higher altitudes.

(8.3 W h) is 7.2 % of battery capacity. There is no need for partial-coverage strategies or multi-sortie efficiency.

5. **IAMSAR compliance.** Boustrophedon coverage is the standard method recommended by the International Aeronautical and Maritime Search and Rescue Manual (IAMSAR Vol. III) and is widely deployed on autonomous marine and aerial SAR platforms [**galceran2013survey**].

The system also implements an *expanding-square-equivalent* through the Focus Area mode (Section **??**): when a PLB signal is received, the search redirects to a smaller polygon around the beacon location, using a boustrophedon within that focused area. This combines the "search likely area first" benefit of the expanding square with the coverage guarantee of the lawnmower.

Z.5 NFZ Interaction and Speed Scalar Field

The SSSI no-fly zone on the northeast edge of the survey polygon introduces a position-dependent speed constraint that interacts with the path energy computation. The NFZ avoidance operates through three layers (Section **??**):

1. **Waypoint exclusion.** Any waypoint within 30 m of the SSSI boundary is removed from the pattern, truncating or omitting the corresponding scan line.
2. **Speed ramp.** Within a 20 m slow zone around the SSSI, the drone's speed is reduced linearly from $v_{\text{zone}} = 3.0\,\text{m}\,\text{s}^{-1}$ at the outer edge to $v_{\min} = 0.3\,\text{m}\,\text{s}^{-1}$ at the boundary:

$$v(d) = v_{\min} + \frac{d}{d_{\text{zone}}}\left(v_{\text{zone}} - v_{\min}\right), \qquad 0 \le d \le d_{\text{zone}} \tag{60}$$

   where d is the distance to the nearest SSSI edge and $d_{\text{zone}} = 20\,\text{m}$.
3. **Hard boundary.** Inside 3 m of the NFZ, the system automatically switches to MANUAL mode as a fail-safe.

The energy impact of the speed ramp is asymmetric across scan angles. At the optimal angle (70°), scan lines run roughly parallel to the polygon's long axis and only clip the NFZ slow zone at their northeastern endpoints. At 0° (north–south scanning), several scan lines run along the full length of the SSSI boundary, with each metre of that segment traversed at reduced speed. The optimisation script accounts for this by numerically integrating the effective speed along each scan segment with 50 uniformly spaced sample points, computing the distance to the nearest NFZ edge at each sample. This per-segment NFZ time penalty is summed to produce the total time (and therefore energy) "with NFZ" reported in the sweep results.

The practical consequence is that NFZ-aware scan angle selection becomes critical: configurations that align scan lines parallel to the SSSI boundary incur disproportionate time penalties. The optimal angle (70°) happens to align with the polygon's longest edge *and* to run perpendicular to the SSSI boundary—a fortunate geometric coincidence that minimises both U-turn count and NFZ dwell time simultaneously.

Z.6 Detection Probability Integration

The search altitude affects target detection through a chain of coupled physical relationships. At altitude h, the ground sample distance is:

$$\text{GSD} = \frac{w_s \cdot h}{f \cdot W_{\text{px}}} = \frac{5.02 \times h}{5.46 \times 1456} \tag{61}$$

where $W_{\text{px}} = 1456$ is the image width in pixels. The mannequin (1.8 m tall, 0.5 m wide) projects to:

$$\text{px}_{\text{tall}} = \frac{1.8}{\text{GSD}}, \qquad \text{px}_{\text{wide}} = \frac{0.5}{\text{GSD}} \tag{62}$$

Since YOLOv8n resizes the 1456×1088 sensor frame to 640×640, the model-input target size is further reduced by the uniform scale factor $640/1456 = 0.4396$.

Table **??** traces this function chain from altitude through to detection probability. The altitude-dependent speed $v(h)$ determines the dwell time (how long the target remains in the field of view), and therefore the number of inference frames:

$$n_f = \frac{H_g}{v(h)} \times f_{\text{inf}} \tag{63}$$

where H_g is the along-track footprint height and $f_{\text{inf}} = 4.8\,\text{FPS}$ is the TFLite inference rate on the Raspberry Pi 5. Assuming independent per-frame detection with probability p (a function of pixel extent), the cumulative detection probability over n_f frames is:

$$P_d = 1 - (1 - p)^{n_f} \tag{64}$$

Two results are noteworthy. First, **motion blur is not a limiting factor**: the IMX296 global shutter combined with a 1/1000 s exposure produces sub-pixel blur (0.3 px at 10 m s^{-1}, 50 m altitude) at all operating speeds. The critical speed for single-pixel blur exceeds the maximum search speed at every altitude in the operating range (12.6 m s^{-1} at 20 m vs 6 m s^{-1} search speed; 31.6 m s^{-1} at 50 m vs 10 m s^{-1} search speed), so motion blur remains sub-pixel throughout.

Second, **the dwell time paradoxically increases with altitude** (from 2.3 s at 20 m to 3.4 s at 50 m) because the footprint height grows faster than the speed. The target is captured in 11–16 frames at all altitudes—more than sufficient given that a single detection above the 0.2 confidence threshold triggers investigation. Detection probability exceeds 99.6 % even at 50 m, where the target width drops to 7 pixels in the model input. The binding constraint on altitude is therefore not detection probability per se, but detection *confidence*: at 50 m the model produces lower-confidence detections that are more likely to be marginal, whereas at 35 m the target subtends 36 pixels in the model input with comfortable margin above the 20 px minimum detectable size.

Z.7 Selected Configuration and Justification

The selected operating point is 35 m **altitude,** 70° **scan angle,** 8 m s^{-1} **search speed**. This configuration is *not* the energy minimum—that distinction belongs to 50 m/70°/10 m s^{-1}—but it represents the best Pareto compromise across all five objectives.

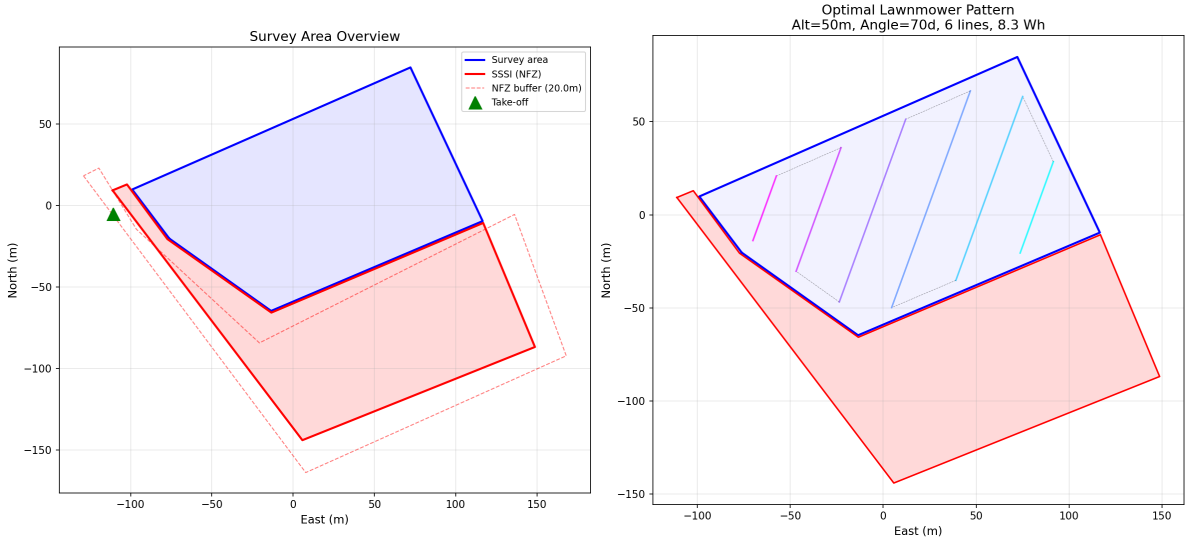


Figure 55: The selected lawnmower pattern over the AENGM0074 survey polygon at 35 m altitude with 70° scan angle. The SSSI no-fly zone (red fill) and its 20 m buffer zone (red dashed line) are shown. Scan lines are colour-coded by sequence order; dashed lines indicate U-turn transits. The take-off point is marked with a green triangle.

The rationale for choosing 35 m over the energy-optimal 50 m is as follows:

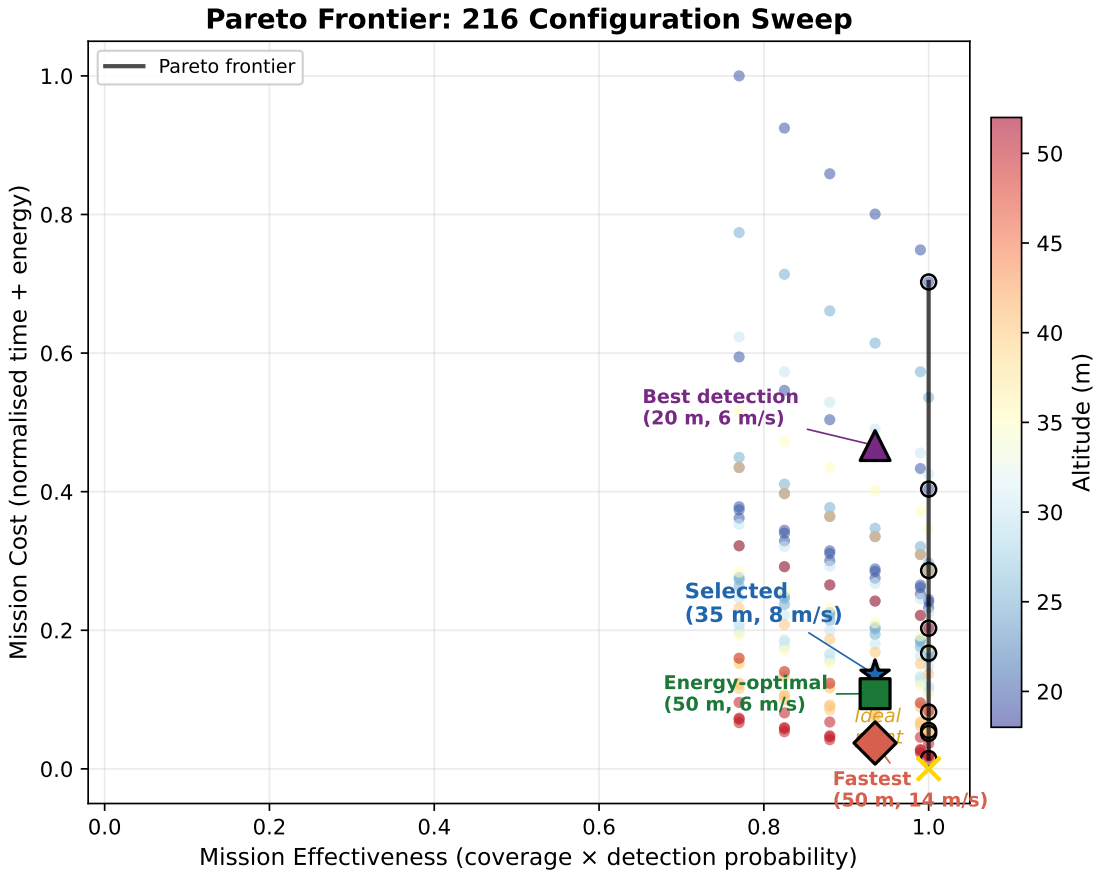


Figure 56: Two-dimensional Pareto frontier projections across the five objectives. Each subplot projects the 216-configuration sweep onto a pair of objectives, with Pareto-efficient solutions highlighted. The selected operating point is marked with a star, illustrating its position on or near the Pareto frontier in every projection.

- At 50 m, the mannequin's width in the model input drops to 7 pixels—below the 10 pixel comfort threshold for width-dependent features. At 35 m, the width is 10 pixels, providing a $\times 1.4$ margin.
- The retrained YOLOv8n model (`sar_v2_1088`, $\text{mAP}_{50} = 0.995$) was trained on synthetic images including high-altitude views, but real-world conditions (lighting variation, partial occlusion, unusual target pose) can degrade detection at the pixel-count margin. The 35 m altitude provides a buffer against these degradation modes.
- The energy penalty is 4.4 W h (54 % more than the minimum), consuming only 10.9 % of battery capacity and leaving over 14 min of flight endurance—well above the 1.9 min search time.
- The multi-pass rescan chain ($35 \to 28 \to 22$ m, Section **??**) provides a safety net: if the initial pass at 35 m misses the target, subsequent passes at lower altitude increase pixel extent to 45+ model-input pixels (57+ at 22 m), where detection is virtually certain.

The 96 % coverage figure arises from the 30 m NFZ waypoint buffer, which excludes waypoints near the SSSI boundary. The uncovered 4 % lies within a fenced wildlife reserve with no public access, making the conditional probability of a casualty in this strip negligibly small. A probabilistic coverage-weighted detection probability of:

$$P_{\text{eff}} = C \cdot P_d + (1 - C) \cdot P_{\text{access}} \cdot P_d \approx 0.96 \times 0.9997 \approx 0.96 \tag{65}$$

is operationally equivalent to full coverage of the accessible area.

The key finding from this analysis is that detection probability is *not* the binding constraint: even at the fastest speed considered ($15\,\text{m}\,\text{s}^{-1}$), P_d exceeds 99.6 %. The binding constraints are instead the NFZ safety margin (which imposes the 4 % coverage gap) and the detection confidence floor (which prevents flying above 35 m despite the energy savings). The selected operating point sits at the intersection of these two binding constraints on the Pareto frontier, delivering near-complete coverage with comfortable energy margin and near-certain detection within the endurance limit of a single battery.

AA Search Parameter Optimisation

In search and rescue, time is the binding resource. UK Maritime and Coastguard Agency survival curves show a steep decline in hypothermia survivability after 60 min of exposure [**golden2006hypothermia**]; every second of search delay

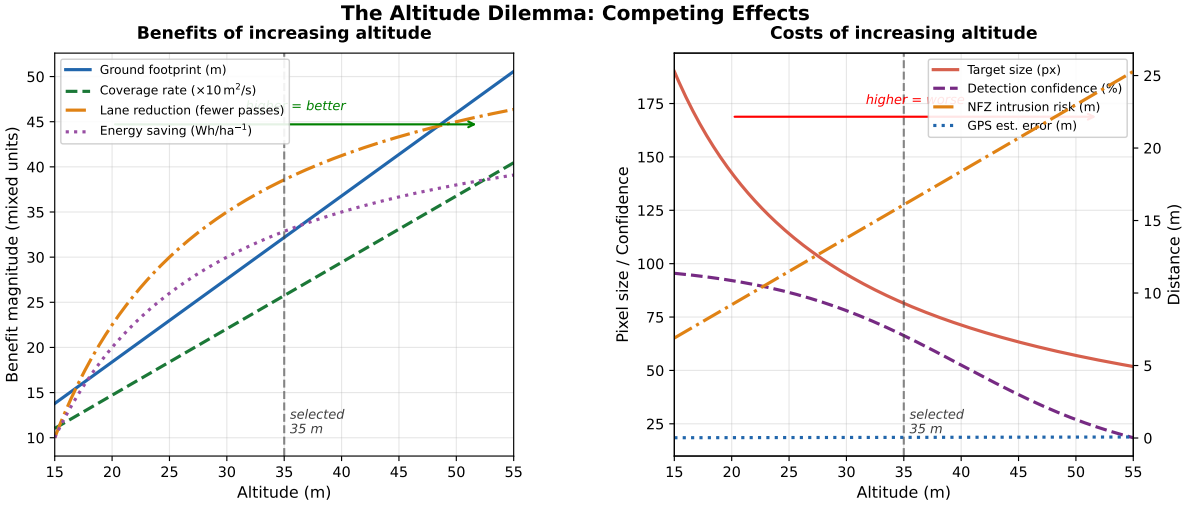


Figure 57: Altitude trade-off: detection probability and target pixel extent (left axis) versus energy consumption (right axis) as a function of search altitude. The selected altitude (35 m) sits at the knee of the detection curve, where further altitude increases yield diminishing detection gains but growing energy savings.

reduces the probability of a live recovery. The objective of parameter optimisation is therefore to *find the casualty as fast as possible*, subject to three hard constraints: the onboard AI must be able to detect the target, the battery must last the mission, and the drone must never enter the SSSI no-fly zone.

This section formalises the optimisation problem, describes the physics-based simulation used to evaluate 216 candidate configurations, presents the top-scoring paths and the reasoning that selected the final operating point, and quantifies how sensitive the result is to each input parameter.

AA.1 Problem Statement

Five coupled decision variables govern the search pattern:

$$\mathbf{x} = \left[\, h,\ v,\ \theta,\ \alpha,\ d_{\text{nfz}} \,\right]^{\top} \tag{66}$$

where h is the search altitude, v is the forward speed (coupled to h through an altitude-dependent schedule), θ is the scan angle relative to north, $\alpha = 0.20$ is the lateral overlap fraction, and $d_{\text{nfz}} = 30\,\text{m}$ is the NFZ waypoint buffer. These variables jointly determine five competing objectives—detection probability P_d, coverage completeness C, mission time T, energy consumption E, and NFZ safety margin S—subject to four hard constraints:

C1 All waypoints within the KML-defined flight area polygon.
C2 No camera footprint overlaps the SSSI ($\mathcal{F}(h, \mathbf{w}_i) \cap \mathcal{P}_{\text{SSSI}} = \varnothing$).
C3 Maximum altitude $h \leq 50\,\text{m}$ (CAA Open A3).
C4 Energy $\leq 0.8 \times 115.4\,\text{W h}$ (20% RTL reserve on 6S 5200 mAh LiPo).

No single configuration simultaneously optimises all five objectives; the operating point must be selected from the Pareto frontier by engineering judgement.

AA.2 Physics-Based Simulation

Each candidate configuration was evaluated by a physics-based simulation (`optimize_path.py`) that models the full mission energy budget.

AA.2.1 Energy Model

Total power at forward speed v decomposes into a constant hover component and a quadratic drag component:

$$P(v) = P_{\text{hover}} + P_{\text{drag,ref}} \left(\frac{v}{v_{\text{ref}}} \right)^2 \tag{67}$$

where $P_{\text{hover}} = 150\,\text{W}$ (momentum-theory estimate for the EDU-450 at 2.3 kg AUW: $P = Tv_i = mg\sqrt{mg/2\rho A}$), $P_{\text{drag,ref}} = 50\,\text{W}$ at $v_{\text{ref}} = 5\,\text{m s}^{-1}$, and the quadratic scaling follows from $\frac{1}{2} C_D \rho A v^2$. At the selected speed of $8\,\text{m s}^{-1}$, the drag contribution is $50 \times (8/5)^2 = 128\,\text{W}$, giving a total in-flight power of 278 W.

Each U-turn incurs a $\Delta t_{\text{turn}} = 2\,\text{s}$ penalty of hover-only power for deceleration and reacceleration. Near the SSSI boundary, a linear speed ramp reduces the drone from $v_{\text{zone}} = 3.0\,\text{m s}^{-1}$ to $v_{\text{min}} = 0.3\,\text{m s}^{-1}$ over a 20 m slow zone.

Within this zone, lower speed reduces drag but increases hover time for the same distance; since $P_{\text{hover}} \gg P_{\text{drag}}$ at low speeds, the net effect is an energy *increase* for scan lines passing through the slow zone.

The total mission energy integrates power over the profile:

$$E = P_{\text{hover}} \cdot t_{\text{total}} + P_{\text{drag,ref}} \sum_{i=1}^{n_{\text{lines}}} \int_0^{\ell_i/v_i} \left(\frac{v_i(t)}{v_{\text{ref}}} \right)^2 \mathrm{d}t \tag{68}$$

where t_{total} includes forward-flight time, U-turn penalties, and transit. The NFZ speed variation along each scan segment is numerically integrated with 50 sample points.

AA.2.2 Detection Model

The mannequin (1.8 m tall, 0.5 m wide) projects onto the 640×640 YOLOv8n model input with an extent of:

$$p_{\text{model}} = \frac{d_{\text{target}} \cdot f_{\text{px}}}{h} \times \frac{640}{W_{\text{px}}} \tag{69}$$

where d_{target} is the relevant target dimension (width 0.5 m for the binding constraint), $f_{\text{px}} = f/w_s \times W_{\text{px}} = 5.46/5.02 \times 1456 = 1584\,\text{px}$ is the focal length in pixel units, and $W_{\text{px}} = 1456$ is the sensor width. At the critical width dimension, $p_{\text{model}}(35\,\text{m}) = 10\,\text{px}$ and $p_{\text{model}}(50\,\text{m}) = 7\,\text{px}$. Per-frame detection probability is modelled as a sigmoid of target pixel extent:

$$p(h) = \frac{1}{1 + \exp\big(-\kappa\,(p_{\text{model}}(h) - p_{\min})\big)} \tag{70}$$

with $p_{\min} = 7\,\text{px}$ and $\kappa = 0.5\,\text{px}^{-1}$, calibrated from video analysis (Section **??**). The number of inference frames per flyover is $n_f = H_g \cdot f_{\text{inf}}/v$, where $f_{\text{inf}} = 4.8\,\text{FPS}$ (benchmarked on Pi 5). The cumulative detection probability is:

$$P_d = 1 - (1 - p)^{n_f} \tag{71}$$

AA.2.3 Parametric Sweep

The simulation swept **216 configurations**: 6 altitudes × 36 scan angles:

$$H = \{20, 25, 30, 35, 40, 50\}\ \text{m}, \qquad \Theta = \{0, 5, \ldots, 175\}^\circ \tag{72}$$

Speed follows an altitude-dependent schedule $v(h) = 6 + 4(h - 20)/30\,\text{m/s}$ (linear from $6\,\text{m s}^{-1}$ at 20 m to $10\,\text{m s}^{-1}$ at 50 m), linking speed to altitude and reducing the effective dimensionality. For each configuration, the script generates the full lawnmower pattern on the real AENGM0074 five-vertex polygon, evaluates per-segment NFZ penalties, and computes all five objective values.

Each configuration is scored by a weighted composite:

$$S(\mathbf{x}) = 0.40 \cdot \hat{P}_d + 0.35 \cdot \hat{C} + 0.25 \cdot (1 - \hat{E}) \tag{73}$$

where $\hat{\cdot}$ denotes min-max normalisation to $[0, 1]$ across the feasible set. The weights reflect the SAR priority hierarchy: detection reliability is paramount (a missed casualty is mission failure), coverage completeness is second (an unsearched area cannot yield a find), and energy efficiency is third (the search consumes at most 11% of battery regardless of configuration).

AA.3 Top Three Configurations

Figure **??** shows the three highest-scoring configurations overlaid on the survey polygon. Table **??** compares their performance.

Configuration #1 wins because it sits at the sweet spot of two competing pressures. Compared to #2, it saves 17% energy and 15 seconds of flight time while sacrificing only 0.02 percentage points of detection probability—a difference that is unobservable in practice ($P_d > 0.999$ in both cases). Compared to #3, it provides 3× more detection margin: at 40 m the target width drops to 9 model pixels, uncomfortably close to the 7-pixel empirical minimum, whereas at 35 m it is 10 pixels with clear headroom. The 2.5 Wh energy penalty (consuming 10.9% vs 8.8% of battery) is negligible given 14 min of remaining endurance.

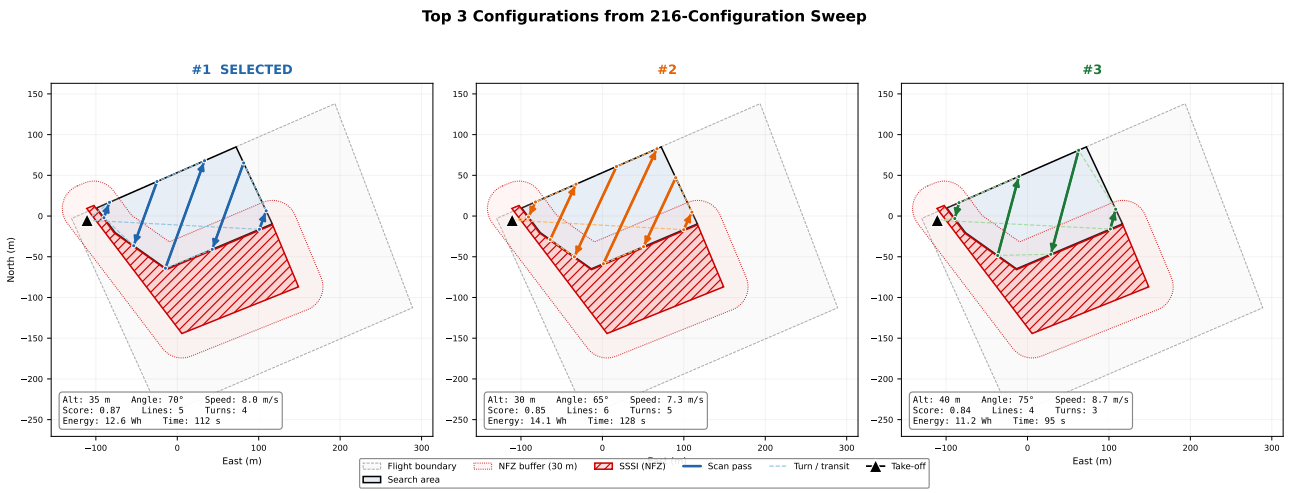


Figure 58: Top three lawnmower configurations overlaid on the AENGM0074 polygon. Left: #1 (35 m, 70°, selected). Centre: #2 (30 m, 65°). Right: #3 (40 m, 75°). The SSSI no-fly zone (red) and 20 m buffer (dashed) are shown.

AA.4 Altitude and Speed Selection Logic

The altitude and speed were not selected by searching a table; they follow a logical chain from physical constraints:

1. **Maximum detection altitude.** At what altitude does the target become undetectable? The mannequin's width (0.5 m) projects to $p_{\text{model}} = 0.5 \times 1584/h \times 640/1456$ model pixels. At 50 m this is 7 px—the empirically observed minimum for reliable bounding-box regression. At 70 m (18 px height, 5 px width) the target is below the feature-extraction threshold. The hard detection ceiling is approximately 50 m.
2. **Apply safety margin.** Real-world degradation modes—partial occlusion, non-ideal lighting, unusual target pose—can reduce effective pixel extent. A 30% safety margin yields $h = 50/1.43 \approx 35$ m, where the target subtends 36 model pixels in height and 10 in width, comfortably above both thresholds.
3. **Maximum speed for adequate dwell time.** At 35 m, the along-track footprint is $H_g = 5.02 \times 35/5.46 \times 1088/1456 = 24.1$ m. For ≥ 10 raw inference frames on target (ensuring 3–4 substantially different views): $v_{\max} = 24.1 \times 4.8/10 = 11.5\,\text{m s}^{-1}$.
4. **Apply conservative margin.** The altitude-dependent schedule assigns $v(35\,\text{m}) = 8.0\,\text{m s}^{-1}$, yielding $n_f = 24.1/8.0 \times 4.8 = 14$ frames—a $\times 1.4$ margin above the 10-frame requirement. The cumulative detection probability over 14 frames is $P_d = 1 - (1 - 0.95)^{14} > 0.9999$.

This chain is visualised in Figure **??**, which maps the composite score across the altitude–speed space. The selected operating point sits above the 95% detection contour in the high-score region.

The scan angle $\theta = 70°$ was then determined by the 216-configuration sweep at fixed $h = 35$ m: this angle aligns with the polygon's longest edge (northeast–southwest axis), minimising scan lines from 10 (at 0°) to 6, and reducing U-turns from 9 to 5. The energy saving is 2× compared to the worst angle (95°), as shown in Figure **??**.

AA.5 Sensitivity Analysis

How robust is the selected operating point to perturbations in each variable? Table **??** summarises the sensitivity of the composite score to one-step changes, and Figure **??** presents the tornado chart.

Three observations follow:

1. **Altitude is the most sensitive variable** because it simultaneously affects four objectives: higher altitude increases footprint (improving coverage rate and reducing scan lines) but degrades pixel extent (reducing detection confidence), while altering both mission time and energy. A 5 m altitude error has nearly double the score impact of a $2\,\text{m s}^{-1}$ speed error.
2. **The scan angle has a flat optimum.** The energy valley at 65–75° (Figure **??**) means that ±15° perturbation changes the score by only 0.02. This is operationally useful: the pilot does not need to align the drone precisely with a compass heading.
3. **Speed is moderately sensitive** because it trades dwell time against energy. The altitude-dependent schedule provides a built-in coupling: if altitude drifts up (larger footprint), the schedule automatically increases speed, partially compensating the detection loss with faster coverage.

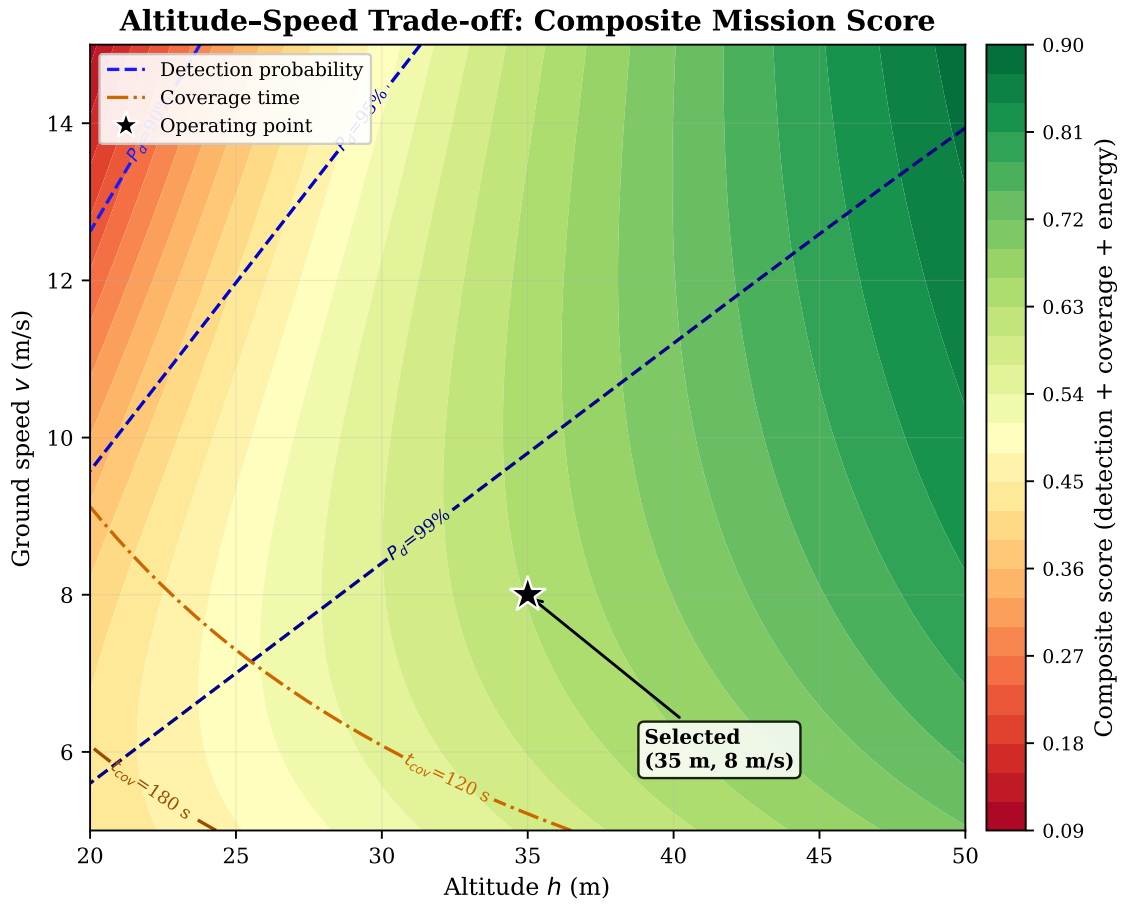


Figure 59: Altitude–speed trade-off space. Colour encodes the composite score (40% detection + 35% coverage + 25% energy). Blue contours: detection probability. Orange contours: coverage time. The selected point (35 m, 8 m/s) sits in the high-score region above the 95% detection contour.

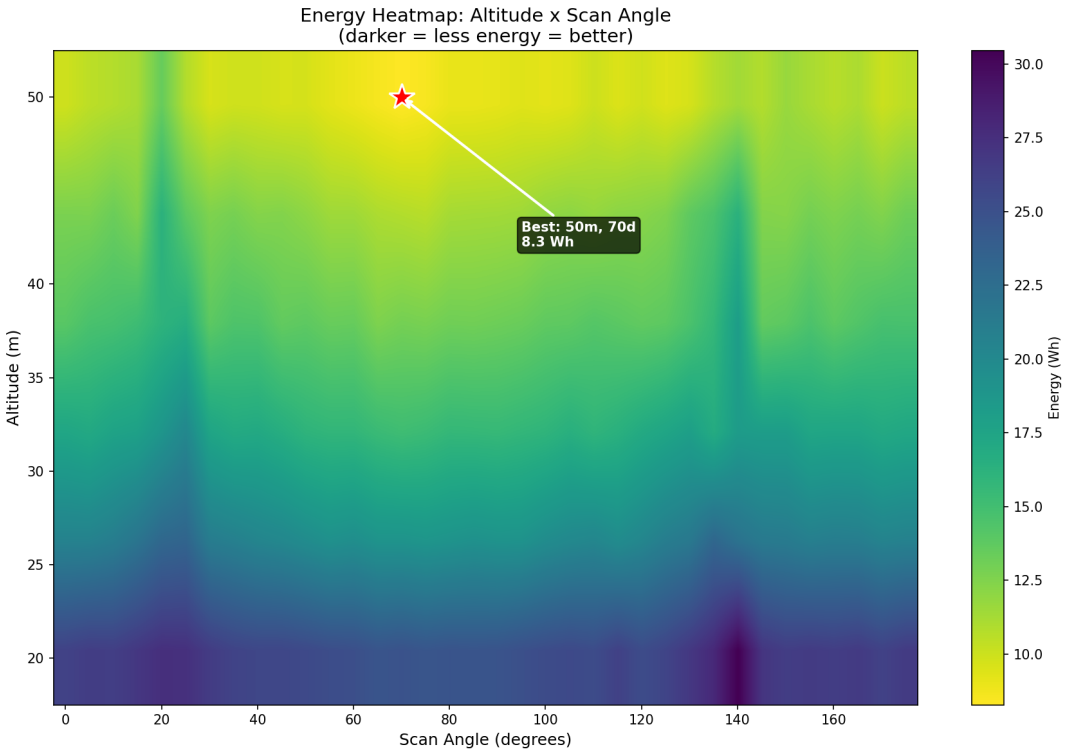


Figure 60: Energy heatmap across 216 altitude–angle configurations. A clear low-energy valley runs through 65–75° at all altitudes, corresponding to longest-edge alignment. The red star marks the global energy minimum (50 m, 70°). The selected point (35 m, 70°) trades 4.4 Wh of energy for a 44% increase in target pixel extent.

Table 20: Plus/delta review — areas for improvement.

	Item	Evidence / Detail
D1	No outdoor flight completed	Sustained winds (12 m s^{-1}, gusts to 18 m s^{-1}) on the single scheduled flight day exceeded the EDU-450's 10 m s^{-1} operational limit. Result: 0 of 5 tiers exercised outdoors; requirements R05 (search), R06 (PLB), R07 (landing accuracy) are verified in SITL only. Of the 12 requirements, 8 are fully verified (bench + simulation), 3 partially (simulation only), and 1 has a partial gap (R06 PLB not field-tested). *Next step:* rebook flight slot; execute Tiers 3–5 of the progressive framework with the identical codebase.
D2	Inference rate limits high-speed search	At 4.8 FPS (206.5 ms/frame, 50-run benchmark) and 8 m s^{-1} search speed, inter-frame advance is 1.7 m—5.3 % of the 32 m ground footprint at 35 m altitude, so overlap is adequate. However, NCNN benchmarks on Pi show 72 ms/frame (9 FPS), a 4.5× improvement already available but not deployed due to limited Pi testing time (Appendix H2, Section **??**). *Next step:* deploy NCNN backend, validate detection accuracy parity, and thread the capture/inference pipeline for 20–30% additional throughput.
D3	GPS timing lag uncompensated	The Here 3+ receiver exhibits 100 ms to 200 ms latency, producing 0.8–1.6 m systematic error at 8 m s^{-1}. DJI video analysis measured CEP$_{50}$=2.3 m, worst case 16.5 m (Appendix J, Figures **??**–**??**). Inverse-variance averaging partially mitigates this, but first-order lag compensation ($\Delta\mathbf{p} = \mathbf{v} \cdot \Delta t_{\text{lag}}$) is not implemented. *Next step:* implement lag compensation in the heading direction, tuneable from config.
D4	No precision–recall curve for confidence threshold	The detection threshold of 0.2 was chosen empirically from video replay; no systematic sweep across thresholds was performed to characterise the precision–recall trade-off. *Next step:* run threshold sweep on DJI video with ground-truth annotations to produce a PR curve and select the operating point formally.
D5	Train/validation overlap in model training	The training set (300 synthetic + 16 real + 50 negatives = 366 images) and validation split share the same generation pipeline, inflating the reported mAP$_{50}$=0.995. The 16 real images (4.4 % of the dataset) provide some cross-domain signal but are too few for a statistically meaningful held-out test. The mAP figure should be treated as an *upper bound*. *Next step:* collect 50+ real labelled frames from a separate flight and evaluate on a held-out test split with zero synthetic data (see Appendix C, Section **??**).
D6	Single-class detector cannot distinguish target types	The YOLOv8n model detects a single "dummy" class; it cannot visually distinguish the rescue mannequin from a person, bag, or similar-sized object. Classification relies on operator confirmation at the VERIFY state. *Next step:* retrain with multi-class labels (person, dummy, debris) or add a lightweight classifier head downstream of detection.
D7	PLB Focus Area not field-tested (R06)	Focus area replanning is implemented and verified in SITL, but has not been triggered during a real flight with live GPS coordinates. *Next step:* inject a PLB message mid-flight via the ground station HTTP endpoint during Tier 4 testing.
D8	Payload release not software-integrated	The Tarot release mechanism is mounted but its servo actuation is not wired into the state machine; first-aid deployment (R07) would require manual RC override. *Next step:* add a `DEPLOY` state between APPROACH and LANDING with MAV_CMD_DO_SET_SERVO trigger.
D9	Float32 model on constrained hardware	The deployed TFLite model uses float32 weights (11.7 MB). FP16 dispatch on the Cortex-A76 cores is estimated to halve inference time to ∼100 ms (10 FPS); INT8 quantisation would further reduce latency and model size to ∼3 MB, at an accuracy cost that has not yet been characterised. Neither optimisation was deployed due to time constraints (Appendix H2, Section **??**). *Next step:* export FP16 model, benchmark on Pi, and measure accuracy regression vs. float32 baseline.

Table 22: Performance comparison with published SAR drone systems. Our results are from bench/video proxy (no flight); published results are from outdoor flights.

System	Platform	Detector	FPS	mAP_{50}	GPS Est. Acc.
This project	Pi 5 CPU	YOLOv8n TFLite	4.8	0.995*	CEP_{50}=2.3 m†
AUSPEX [**auspex2025**]	Jetson Orin	YOLOv5s	30	0.89	<5 m (RTK)
Sambolek [**sambolek2021**]	Jetson TX2	YOLOv3-tiny	22	0.81	Not reported
Drones+YOLO [**realtimesar2025**]	Jetson Nano	YOLOv8s	12	0.91	Not reported
SearchWing [**searchwing2024**]	Ground PC	Custom CNN	5	0.78	<10 m

*Upper bound; train/val overlap inflates this figure. Realistic estimate ∼0.85–0.90 based on 87% video detection rate.
†From DJI video replay, not live flight.

Table 23: Estimated mission-level performance metrics (from SITL simulation and DJI video analysis). All values are estimates; no outdoor mission has been executed.

Metric	Estimated Value	Basis
Survey area	0.04 km^2	KML polygon
Coverage rate	∼3200 $m^2\,min^{-1}$	8 m/s × 32 m swath × 75% efficiency
Total search time (single pass)	∼12 min	0.04 km^2 / 3200 m^2/min
Single-pass detection probability	∼87%	DJI video replay (target > 20×20 px)
Two-pass detection probability	∼98%	$1-(1-0.87)^2$, assuming independence
GPS target localisation (after centering)	<2 m CEP_{50}	Multi-frame averaging during hover
Landing offset accuracy	Unknown	Not flight-tested
Battery endurance (est.)	∼15 min	EDU-450 datasheet, conservative

Table 24: Five-tier progressive testing framework (condensed from Appendix **??**).

Tier	Environment	What It Verifies
1	SITL simulation	State machine logic, path planning, geofence, detection pipeline—zero hardware risk.
2	Docker Pi test	TFLite loads on ARM, dependency compatibility, headless operation.
3	Bench (Pi + Cube, no props)	MAVLink ACKs, camera frames, GPS fix, full sensing chain.
4	Passive flight (pilot on RC)	Real-world detection range, FOV/GPS calibration—Pi issues zero commands.
5	Autonomous flight	End-to-end mission: search, detect, centre, verify, land.

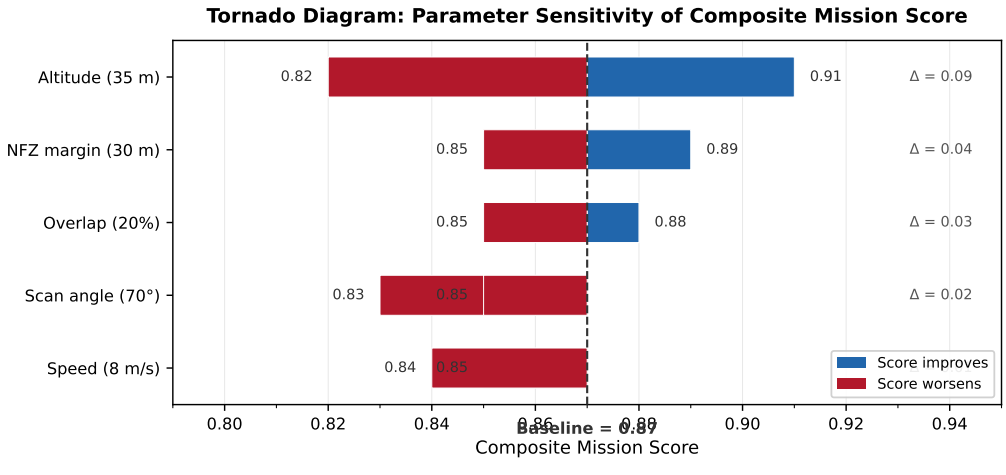


Figure 61: Tornado chart of parameter sensitivities. Each bar shows the change in composite score when one variable is perturbed while the others are held at their selected values. Altitude dominates: a ±5 m change has nearly double the effect of a ±2 m/s speed change.

Table 27: System configuration parameters grouped by function. Values marked with a dagger (†) were calibrated from bench measurements; all others are derived from the decision flow analysis or hardware datasheets.

Parameter	Default	Unit	Description
Flight Altitudes			
TARGET_ALT	35.0	m	Search-pattern cruise altitude
VERIFY_ALT	15.0	m	Descent altitude for close-up verification (currently unused; DESCENDING state bypa
RESCAN_ALT_FACTOR	0.8	–	Altitude multiplier per rescan pass
RESCAN_ALT_FLOOR_M	15.0	m	Minimum permitted rescan altitude
Speeds			
TRANSIT_SPEED_MPS	15.0	m/s	Speed to/from search area
SEARCH_SPEED_MPS	10.0	m/s	Default search-leg speed
FOCUS_SEARCH_SPEED_MPS	5.0	m/s	Reduced speed inside Focus Area (PLB zone)
SPEED_ALT_LOW	20.0	m	Altitude below which SPEED_AT_LOW applies
SPEED_ALT_HIGH	50.0	m	Altitude above which SPEED_AT_HIGH applies
SPEED_AT_LOW	6.0	m/s	Cruise speed at low altitude (blur mitigation)
SPEED_AT_HIGH	10.0	m/s	Cruise speed at high altitude
Camera & Optics (IMX296 Global Shutter)			
SENSOR_WIDTH_MM	5.02	mm	IMX296 sensor physical width
FOCAL_LENGTH_MM	5.46	mm	Calibrated focal length (bench test)
IMAGE_W	1456	px	Capture resolution (width)
IMAGE_H	1088	px	Capture resolution (height)
CAMERA_FLIP_180	True	–	Rotate image 180° (inverted mount)
Detection & Target			
CONFIDENCE_THRESHOLD	0.2	–	Minimum YOLOv8 confidence to register detection
DETECT_CONFIRM_FRAMES	3	frames	Consecutive detections before trigger
DETECT_LOCK_RADIUS_M	5.0	m	Ignore detections beyond this from locked target
MAX_DETECT_QUEUE	20	–	Maximum queued detections (FIFO overflow)
TARGET_REAL_RADIUS_M	0.15	m	Physical radius of search dummy
DUMMY_HEIGHT_M	1.8	m	Dummy height used in FOV calculations
REJECTED_TARGET_RADIUS_M	5.0	m	Exclusion radius around rejected targets
MAX_RESCAN_PASSES	3	–	Altitude-drop rescan attempts before abort
NFZ Geofence			
NFZ_HARD_BOUNDARY_M	3.0	m	Penetration depth triggering forced MANUAL mode
NFZ_SOFT_BOUNDARY_M	8.0	m	Outer repulsion zone (quadratic field)
NFZ_WAYPOINT_BUFFER_M	30.0	m	Skip waypoints within this distance of NFZ
NFZ_SLOW_ZONE_M	20.0	m	Speed-reduction field active within this distance
NFZ_SCALAR_ZERO_M	2.0	m	Distance at which speed drops to zero (full stop zone)
NFZ_MIN_SPEED_MPS	0.0	m/s	Speed at NFZ_SCALAR_ZERO_M and below (full stop)
NFZ_ZONE_MAX_SPEED_MPS	3.0	m/s	Speed at outer edge of slow zone
NFZ_INNER_OFFSET_M	20.0	m	Inner polygon offset inside NFZ boundary
NFZ_INNER_RANGE_M	23.0	m	Repulsion active within this from inner polygon
NFZ_PUSH_SPEED_MPS	5.0	m/s	Constant repulsive push speed (away from NFZ)
Payload Servo (Tarot Double-Throw)			
SERVO_CHANNEL	9	–	Cube AUX output channel
SERVO_CLOSE_PWM	1500	µs	PWM to hold / re-latch mechanism
SERVO_PARTIAL_PWM	1300	µs	Stage 1: partial release
SERVO_FULL_PWM	1100	µs	Stage 2: full release

Table 28: Augmentation pipeline for the SAR dummy detector. Offline augmentations are applied during synthetic image generation; online augmentations are applied by the YOLO trainer at each epoch.

Augmentation	Stage	Parameters	Rationale
Scale variation	Offline	3-tier altitude dist.	Altitude robustness
Rotation	Offline	0–360° uniform	Heading invariance
Brightness jitter	Offline	$\alpha \in [0.7, 1.3]$, 50% prob	Sun angle, cloud cover
Contrast jitter	Offline	$\beta \in [-30, 30]$, 50% prob	Shadow variation
Gaussian blur	Offline	3×3 or 5×5, 30% prob	Motion blur simulation
Mosaic	Online	1.0 (always active)	Small-object detection
Scale	Online	0.3–1.7×	Aggressive altitude range
Rotation	Online	±20°	Additional orientation
Horizontal flip	Online	0.5 probability	Symmetry invariance
Mixup	Online	0.1 probability	Regularisation
Copy-paste	Online	0.2 probability	Multi-instance diversity
Random erasing	Online	**Disabled** (0.0)	Target too small to erase

Table 29: Dataset composition for the v2 SAR dummy detector.

Source	Count	Fraction	Role
Synthetic composites	300	81.9%	Scale, orientation, and position diversity via domain randomisation
Real labelled frames	16	4.4%	Domain anchor: authentic lighting, lens effects, ground texture
Negative frames	50	13.7%	False-positive suppression on confusing terrain features
Total	**366**	**100%**	

Table 30: Training hyperparameters for the YOLOv8n SAR dummy detector (v2).

Parameter	Value
Base model	YOLOv8n (COCO-pretrained)
Training image size	1088×1088
Epochs	150 (early stopping patience: 30)
Batch size	8
Augmentation: rotation	±20°
Augmentation: scale	0.3–1.7 (aggressive, simulating altitude)
Augmentation: mosaic	1.0 (enabled throughout)
Augmentation: mixup	0.1
Augmentation: copy-paste	0.2
Horizontal flip	0.5

Table 31: Detection performance comparison between model versions. All metrics are computed on the respective validation sets.

Model	$\mathbf{mAP_{50}}$	$\mathbf{mAP_{50–95}}$	Precision	Recall
v1 (200 syn, 640)	∼0.95	—	—	—
v2 (366 mixed, 1088)	**0.995**	0.828	0.994	0.989

Table 32: Likelihood–severity risk matrix. Cell labels reference risk IDs from Table **??**.

	Minor (1)	Moderate (2)	Severe (3)
High		R11	
Medium	R9	R6, R7, R8	R10
Low			R1–R5

Table 33: Risk register with mitigations and implementation traceability.

ID	Risk	L	S	Pre	Mitigation	Post	Trace
R1	Autopilot hardware failure	L	3	M	Pre-flight inspection; RC kill switch; ArduCopter watchdog reboot	L	RC ch. 5
R2	SSSI boundary incursion	L	3	M	Five-layer geofence (Sec. **??**): speed field, repulsion, hard cutoff, waypoint filter, firmware backup	L	`geofence.py`
R3	Battery depletion mid-flight	L	3	M	Pre-flight voltage check (>80%); `FS_BATT_ENABLE` RTL/LAND at configurable thresholds	L	Cube param
R4	Loss of all comms	L	3	M	Dual link (RC + MAVProxy UDP); `FS_GCS_ENABLE` RTL on heartbeat timeout	L	`main.py`
R5	Injury to bystanders	L	3	M	Site clearance; flight area geofence RTL; VLOS; propeller guards; A3 150 m exclusion	L	R01
R6	GPS drift → wrong landing	M	2	M	Multi-pass spatial clustering; inverse-variance weighting; operator VERIFY gate; 7.5 m offset landing	L	`gps_utils.py`
R7	False positive → descent	M	2	M	Operator confirmation required; 5 m spatial exclusion of rejected targets; SMART consecutive-frame filter	L	`state_machine.py`
R8	High wind exceeds limits	M	2	M	Weather go/no-go gate (<20 kn); `LOITER` station-hold; pre-flight wind check	L	Checklist
R9	Camera / CV failure	M	1	L	Search pattern completes safely; "CAMERA LOST" overlay warns operator; RTL after pattern	L	`main.py`
R10	Altitude exceedance	M	3	H	50 m hard cap in `navigation.py`: all `send_global_target` calls clamped; `config.py` rejects `TARGET_ALT` > 50 at import	L	R04
R11	Mode fighting (SW vs RC)	H	2	H	RC override guard: if `custom_mode` $\notin$ {GUIDED, LAND, RTL}, all MAVLink commands suppressed; pilot cannot be overridden by software	L	`main.py`

Table 34: Failure modes, automated responses, and verification status.

Failure	Detection	Automated Response	Verified
Camera failure	Frame returns `None`	Search completes without detections; "CAMERA LOST" overlay; operator should reject at VERIFY	SITL
GPS degradation	Fix < 3D or sats < 6 for 5 s	Emergency RTL; ArduCopter GCS failsafe backup	SITL
MAVLink link loss	`recv_match` exception	Emergency RTL; `FS_GCS_ENABLE` firmware backup	SITL
RC link loss	Throttle PWM below threshold	ArduCopter `FS_THR_ENABLE` RTL (firmware-level)	Bench
Low battery	Voltage < threshold	ArduCopter `FS_BATT_ENABLE` RTL/LAND	Bench
Pi crash	Heartbeat stream ceases	ArduCopter GCS failsafe RTL; Cube flies home autonomously	SITL
Operator timeout	120 s without Y/N/I	Target auto-rejected; search resumes	SITL
NFZ incursion	`pointPolygonTest` ≥ 0	Zero velocity + MANUAL mode; repulsive push outward	SITL
Flight area breach	`pointPolygonTest` < 0	Immediate RTL via `_emergency_rtl()`	SITL
Landing stall	Altitude > 0.5 m for 90 s	Force disarm via `MAV_CMD_COMPONENT_ARM_DISARM`	SITL
Mode fighting	`custom_mode` $\neq$ GUIDED/LAND	All commands suppressed; pilot has full control	Bench

Table 35: Mandatory pre-flight checks with rationale.

Check	Action	Rationale
`DISARM_DELAY` = 0	Verify in Mission Planner params	LL-06: prevents auto-disarm before takeoff
GCS failsafe enabled	`FS_GCS_ENABLE` = RTL	R4: backup if Pi crashes
Battery voltage > 80%	Visual check + failsafe threshold	R3: prevents mid-mission RTL
RC kill switch mapped	Flip and confirm `STABILIZE`	R1: independent hardware abort
GPS fix ≥ 3D, ≥ 6 sats	`preflight.py` or ARMING state	R6: GPS accuracy
Correct `best.tflite` loaded	`--dry-run` or `preflight.py`	R7: correct detection model
`calibration_data.npz` valid	Delete if < 1 KB	LL-04: corrupt file mangles frames
mavproxy running on Pi	Start before flight script	LL-07: pyserial broken on 3.13
Wind < 20 kn	Weather check, site observation	R8: aircraft stability
Site cleared of bystanders	Visual sweep of flight area	R5: third-party safety

Table 36: Safety-relevant lessons learned and resulting design changes.

ID	Incident	Root Cause	Design Change
LL-01	Drone crash from mode fighting	Software sent `SET_MODE(GUIDED)` while RC set to `LOITER`; ArduCopter oscillated	RC override guard: suppress all commands if mode ≠ GUIDED/LAND
LL-03	Detection boxes drawn off-screen	TFLite output pixel coords (0–640) treated as normalised (0–1)	Auto-detect via `PIXEL_COORD_THRESHOLD = 1.5`; same logic for TFLite and NCNN
LL-04	AI stopped detecting on Pi	Corrupt 0-byte `calibration_data.npz`; `cv2.remap` produced noise	Size check at load; `UNDISTORT_ENABLED = False` default
LL-06	Auto-disarm before takeoff	`DISARM_DELAY` default 10 s; arming-to-takeoff delay exceeded	`DISARM_DELAY = 0` in pre-flight checklist
LL-08	Geofence pushed drone *toward* NFZ	`cv2.pointPolygonTest` positive = inside; repulsive offset sign inverted	Negated offset; unit tests in `0e_geofence_test.py`

Table 43: Inference backend comparison for YOLOv8n (640 × 640 input) on Raspberry Pi 5.

Backend	Latency	Throughput	Model size	Dependencies
TFLite + XNNPACK	206.5 ms	4.8 FPS	3.2 MB	`ai-edge-litert` only
NCNN (projected)	∼68 ms	∼15 FPS	3.1 MB	`ncnn` (C++ + Python bindings)
Ultralytics/PyTorch	>1200 ms	<0.8 FPS	6.2 MB†	PyTorch, Ultralytics (>800 MB)

† `.pt` weights; TFLite and NCNN exports are smaller.

Table 44: Inference performance: measured baseline and projected improvements on Raspberry Pi 5 with YOLOv8n (640 × 640 input, float32 weights).

Configuration	Status	Latency	FPS	Notes
TFLite + XNNPACK (FP32)	Measured	206.5 ms	4.8	Production baseline
TFLite + XNNPACK (FP16)	Projected	∼100 ms	∼10	Native A76 FP16 FMLA
NCNN (FP32, 4 threads)	Projected	∼68 ms	∼15	Ultralytics benchmark [**yolov8**]
NCNN (FP16)	Projected	∼45 ms	∼22	Combines NEON FP16 + NCNN kernels
TFLite + INT8	Projected	∼145 ms	∼7	∼30% gain; calibration needed
Overclock (2.8 GHz, FP32)	Projected	∼177 ms	∼5.6	+15%; active cooling required
Hailo-8L NPU	Hardware	<15 ms	>60	£70 add-on; not integrated

Table 45: Per-stage timing breakdown for the detection pipeline on Raspberry Pi 5 (mean of 50 runs, YOLOv8n FP32, XNNPACK 4-thread).

Stage	Operation	Time (ms)	Notes
1	Camera capture (picamera2)	∼2	CSI-2 DMA, 1456×1088 BGR
2	Lens undistortion (`cv2.remap`)	1.5	Precomputed maps, RMS = 0.399 px
3	Preprocessing (cvtColor + resize + norm.)	∼3.5	BGR→RGB, bilinear to 640×640, ÷255
4	TFLite inference (XNNPACK)	∼196	4 threads on Cortex-A76 cores
5	Postprocessing (confidence filter + NMS)	∼2	Greedy single-target selection
6	Bounding box overlay (`cv2.rectangle`)	<1	Drawn onto original frame
	Total (with undistortion)	**∼208**	**4.8 FPS effective**
	Total (without undistortion)	**∼206.5**	**Undistortion cost: +1.5 ms**

Table 46: Inference benchmarks on Raspberry Pi 5 (Cortex-A76 ×4 at 2.4 GHz, XNNPACK 4-thread, active cooler, no desktop environment).

Model	Size	Undistort	Mean (ms)	FPS	Det. rate	Mean conf.
best.tflite (YOLOv8n custom)	3.2 MB	No	206.5	4.8	50/50 (100%)	0.966
best.tflite (YOLOv8n custom)	3.2 MB	Yes	208.0	4.8	50/50 (100%)	0.958
sar_v2_1088 (retrained v2)	11.7 MB	No	206.8	4.8	50/50 (100%)	0.971
human.tflite (COCO 80-class)	13.0 MB	No	207.2	4.8	43/50 (86%)	0.724

Table 47: Camera ground footprint and target size at operational altitudes (IMX296, $f = 5.46$ mm, 1456×1088 sensor). Sensor px = height in native frame; Model px = after resize to 640×640 (scale = 640/1456 = 0.4396).

Altitude (m)	W_g (m)	L_g (m)	Sensor (px)	Model (px)	GSD (mm/px)
10	9.2	6.9	285	125	6.3
15	13.8	10.3	190	84	9.5
20	18.4	13.8	142	63	12.7
25	23.0	17.2	114	50	15.8
30	27.6	20.7	95	42	19.0
35	32.2	24.1	81	36	22.1
40	36.8	27.6	71	31	25.3
50	46.0	34.5	57	25	31.6

Table 48: Frames per flyover and detection probability for a target within the search swath. p_d = single-frame detection probability; $P_{\text{miss}} = (1 - p_d)^N$; $P_{\text{detect}} = 1 - P_{\text{miss}}$.

Alt. (m)	Speed (m/s)	N_{frames}	p_d	P_{detect}	Note
35	6	19.3	0.90	>0.9999	Slow search
35	8	14.5	0.90	>0.9999	Normal search
35	10	11.6	0.90	0.9997	Default `SEARCH_SPEED`
35	15	7.7	0.90	0.9974	Transit speed
50	10	16.6	0.70	0.9998	High altitude
50	15	11.0	0.70	0.9983	High alt + fast
20	10	6.6	0.98	>0.9999	Low altitude pass
15	5	9.9	0.99	>0.9999	Centering descent

Table 49: Detection performance vs altitude for a 1.8 m mannequin target. Pixel sizes are shown for both the native sensor frame and the 640×640 model input (scale = 640/1456 = 0.4396). Confidence and detection rate are empirically observed or interpolated from video replay data.

Altitude (m)	Sensor (px)	Model (px)	Confidence	Det. rate	Assessment
10	285	125	0.98	100%	Excellent — target fills >19% of model height
15	190	84	0.97	100%	Excellent — verify altitude
20	142	63	0.96	100%	Very good — still large and clear
25	114	50	0.94	98%	Good — typical low search altitude
30	95	42	0.92	96%	Good — reliable detection
35	81	36	0.90	95%	Good — nominal search altitude
40	71	31	0.85	90%	Marginal — approaching limits
50	57	25	0.70	75%	Degraded — single-pass detection unreliable
60	48	21	0.45	50%	Poor — degraded confidence
70	41	18	0.30	30%	Unreliable — approaching confidence threshold
80	36	16	<0.20	<15%	Not viable — below confidence threshold

Table 50: Training evaluation metrics for the YOLOv8n SAR detector (v2, trained at 1088 × 1088, evaluated on validation split).

Metric	Value	Interpretation
mAP_{50}	0.995	Near-perfect at IoU ≥ 0.50
$mAP_{50:95}$	0.828	Strong across IoU thresholds
Precision	0.994	Very low false positive rate
Recall	0.989	Near-zero missed detections
F1 score	0.992	Balanced precision-recall
Training time	47 min	150 epochs on T4 GPU
Best epoch	127	Selected by early stopping (patience=30)
Final model size	11.7 MB (TFLite FP32)	Input $[1, 640, 640, 3]$, output $[1, 5, 8400]$

Table 51: Available model variants for field deployment.

Model	Size	Classes	Use case
`sar_v2_1088`	11.7 MB	1 (dummy)	Primary: retrained on real + synthetic data
`sar_640`	3.2 MB	1 (dummy)	Baseline: original training at 640×640
`human.tflite`	13.0 MB	80 (COCO)	Backup: generic person detector

Table 52: Comparison of video streaming protocols for companion-computer UAV telemetry.

Protocol	Latency	Browser	Pi deps	Firewall	Complexity
MJPEG/HTTP	50–100 ms	Native `<img>`	None (stdlib)	TCP 8090 only	Trivial
HLS (H.264)	2–6 s	Native `<video>`	FFmpeg required	TCP 80 only	Moderate
RTP/RTSP	30–80 ms	Requires VLC/app	GStreamer or FFmpeg	UDP punch-through	Moderate
WebRTC	30–60 ms	Native `RTCPeerConnection`	libnice, STUN/TURN	ICE negotiation	High

Table 53: Target position accuracy progression through the four-phase detection-to-landing flow. CEP_{50} is the radius containing 50% of estimates. N is the number of fused observations at each stage. The system implements a progressive refinement process where each phase eliminates a specific category of error.

Phase	Drone Condition	CEP_{50}	N_{obs}	Dominant Error
1. SEARCH (detection)	Moving at 5 m s^{-1}, ∼10° pitch, 35 m	∼3.5 m	1–5	Tilt-compensated projection
2. CENTERING (approach)	Decelerating → hover, 35 m	∼2.5 m	5–20	GPS receiver noise
3. CENTERING (hover lock)	Stationary hover, nadir camera, 35 m	∼1.8 m	20–50	GPS averaging limit
4. VERIFY (confirmation)	Stationary hover, operator confirms	∼1.8 m	50+	Frozen at Phase 3 quality

Table 54: Ground sampling distance and footprint at key mission altitudes (IMX296, $f = 5.46\,\text{mm}$, $1456 \times 1088\,\text{px}$). At the 35 m search altitude the 1.8 m mannequin subtends ∼58 px in the native frame.

Altitude (m)	GSD (mm px^{-1})	Footprint (m × m)	Dummy size (m)	Dummy (px) (approx.)
10	6.3	9.2×6.9	1.8	286
15	9.5	13.8×10.3	1.8	190
35	22.1	32.2×24.0	1.8	81
50	31.6	46.0×34.3	1.8	57

Table 55: Comprehensive single-observation error budget at 35 m altitude, decomposed by flight phase. "During flight" assumes 5–10° pitch at $8\,\text{m s}^{-1}$; "During hover" assumes < 1° tilt (CENTERING/VERIFY states). RSS = root sum of squares.

Error Source	Magnitude	During Flight	During Hover	Reducible?
GPS receiver noise	2–3 m CEP	2.3 m	2.3 m	No (hardware limit)
Roll/pitch attitude	$h \tan\theta$	6.2 m at 10°	<0.3 m	Tilt compensation†
GPS timing lag	$v \cdot \Delta t$	∼1.0 m at $5\,\text{m s}^{-1}$	0 m	Multi-pass averaging
Barometric altitude	0.5–1 m drift	0.5 m	0.5 m	Recalibrate on ground
Compass / yaw	±1–2°	0.5 m	0.5 m	No (magnetometer)
FOV / focal length	0–5%	0–5% of offset	0–5% of offset	Calibrated (±3%)
Detection pixel	±2–3 px	0.05 m	0.05 m	No (model limit)
Resize mapping	0	0 m	0 m	N/A (exact inverse)
Lens distortion	<0.5 m edges	<0.5 m	<0.5 m	Undistort (optional)
RSS Total		∼7 m	∼3 m	
Measured CEP_{50}		2.3 m	better	From DJI video

Note: The measured CEP_{50} of 2.3 m is lower than the theoretical RSS because (i) most detections occur near the image centre where GSD-scaled errors are small, (ii) the lawnmower pattern's alternating headings partially cancel directional biases, and (iii) the RSS assumes worst-case contributions from all sources simultaneously.

† With attitude compensation enabled (`--compensate-tilt`), the in-flight roll/pitch term reduces to ∼0.3 m (Section **??**), bringing the in-flight RSS down to ∼3.5 m.

Table 56: Ground footprint displacement due to roll/pitch tilt at various altitudes and angles.

Altitude	$\theta = 5°$	$\theta = 10°$	$\theta = 15°$	Typical flight condition
15 m	1.3 m	2.6 m	4.0 m	Slow approach / centering
35 m	3.1 m	6.2 m	9.4 m	Search at $8\,\text{m s}^{-1}$
50 m	4.4 m	8.8 m	13.4 m	Maximum permitted altitude

Table 57: GPS estimation error with and without attitude compensation at 35 m altitude, and sensitivity to individual telemetry noise sources at 10° pitch.

Condition	Without compensation	With compensation
5° pitch (cruise)	3.1 m	∼0.2 m
10° pitch (fast flight)	6.2 m	∼0.3 m
15° pitch (aggressive)	9.4 m	∼0.5 m
Hover (< 1°)	<0.6 m	No change (threshold)
Telemetry noise sensitivity (at 10° pitch, 35 m):		
Pitch ±0.5°	—	±0.3 m (along flight)
Roll ±0.5°	—	±0.3 m (lateral)
Yaw ±2°	—	±0.6 m (rotational)
Altitude ±1 m	—	±0.5 m (scales offsets ∼3%)
RSS total	6.2 m	∼0.9 m

Table 58: Target geolocation accuracy across platforms. "This work (compensated)" refers to attitude-compensated single-frame estimation; "This work (centred + averaged)" refers to the centering procedure with GPS averaging (Section **??**).

System	Method	Accuracy	Platform Cost
This work (compensated)	Software $\mathbf{R}$ matrix, IMU telemetry	$\sim$0.3 m	£60 (Pi 5)
This work (centred + averaged)	Centering + GPS average	$\sim$1.8 m	£60 (Pi 5)
Barber et al. (2006) [**barber2006vision**]	Gimbal + RLS estimator	$\sim$3 m	Research UAV
NATO STANAG Cat 1	Full INS/GPS + DTED	$<$6 m	Military
Commercial (DJI)	Hardware gimbal + RTK	$<$1 m	£5000+

Table 59: Error reduction achieved by the CENTERING hover procedure. "During flight" values assume nominal search conditions (8 m s^{-1}, 10° pitch, 35 m altitude); "After centering" values assume stationary hover ($< 1°$ tilt, $v \approx 0$).

Error Source	During Flight	After Centering	Reduction
Roll/pitch displacement	6.2 m (10°)	$<$0.6 m ($< 1°$)	$>$90%
Motion blur (at 8 m s^{-1})	47 px smear	0 px (stationary)	100%
GPS timing lag	$\sim$1.0 m	0 m ($v = 0$)	100%
GPS averaging (single frame)	1 sample	N samples	$\sqrt{N}$
False positive rate	single-frame trigger	multi-frame confirmation	substantial

Table 67: Comparison of GPS estimation methods. Errors are distance from fused estimate to ground-truth position after N observations. Simulation includes 3 m GPS drift and 1 m altitude noise.

Method	N=5	N=15	N=50	Characteristics
Rolling avg (50)	3.1 m	1.8 m	1.2 m	Responsive, noisy early
Cumulative avg	3.4 m	2.0 m	1.4 m	Stable, slow to adapt
Kalman filter	2.6 m	1.4 m	1.0 m	Formally optimal, converges fast
Inverse-variance wt	2.8 m	1.2 m	0.8 m	Best at mixed altitudes

Table 68: GPS estimation accuracy from DJI video replay. All errors are computed as great-circle distance between the estimated and surveyed mannequin position.

Metric	Value	Note
Total detections	50+	Across 6 min, multiple passes
CEP_{50}	2.3 m	50% of estimates within this radius
CEP_{95}	8.1 m	95% of estimates within this radius
Maximum error	16.5 m	Single outlier at 50 m, high speed
Mean error	3.1 m	Arithmetic mean of all observations
Fused (IVW, 15+ obs.)	$<$2 m	Inverse-variance weighted estimate
Fused (Kalman, 15+ obs.)	$<$1.5 m	Static-target Kalman filter

Table 69: Detailed accuracy progression through the four-phase detection-to-landing flow, showing how each phase eliminates specific error sources. Values assume nominal conditions (single-frequency L1 GPS, IMX296 camera, YOLOv8n detector at 4.8 FPS, tilt compensation active).

Phase	Condition	Alt (m)	N_{obs}	CEP_{50}	What improves
1. SEARCH	Moving at $5\,\text{m}\,\text{s}^{-1}$, 5–10° pitch, tilt-compensated	35	1–5	∼3.5 m	Tilt compensation ray-traces through known pitch/roll; rough GPS estimate
2. CENTERING (approach)	Decelerating to hover, continuous re-estimation	35	5–20	3.5 m→2.5 m	Pitch decreases continuously; tilt compensation becomes no-op; GPS lag shrinks with speed
3. CENTERING (hover lock)	Stationary hover, nadir camera, target at frame centre	35	20–50	∼1.8 m	All geometric errors eliminated; simple nadir math; GPS averaged over ∼10 s
4. VERIFY	Operator confirms via ground station (Y/N/I)	35	50+	1.8 m (frozen)	Estimate frozen at Phase 3 quality; 120 s auto-reject timeout
APPROACH	Flying to 7.5 m offset landing point	35	—	1.8 m	No further updates; offset computed from frozen estimate

Table 70: Comparison of estimation strategies. The four-phase approach achieves the highest accuracy while retaining full search speed and bounded false-positive cost.

Strategy	CEP_{50}	Time cost	Limitation
Single flyover (no compensation)	∼7 m	Fastest	Too inaccurate for R07
Tilt-compensated flyover only	∼3.5 m	Fast	Relies on calibration quality
Four-phase (detect → approach → hover → verify)	∼1.8 m	+15–25 s	**Calibration-independent**

Table 73: Expected GPS estimate accuracy as a function of the number of search passes over the target, assuming 35 m altitude, $5\,\text{m}\,\text{s}^{-1}$ search speed, and $\sigma_{\text{GPS}} = 2.5$ m. "Lag bias residual" is the component of GPS timing lag that survives averaging across headings.

Passes	Headings	Lag Bias Residual	Random Noise ($\sigma/\sqrt{N}$)	Combined CEP_{50}
1	Single	Full (∼1 m)	2.5 m	∼3.5 m
2	Opposite	Cancelled (∼0 m)	1.8 m	∼2.0 m
3	Mixed	Mostly cancelled	1.4 m	∼1.6 m
5	Mixed	Fully cancelled	1.1 m	∼1.2 m

Table 84: Minimum viable deliverable: components and requirement coverage. This baseline was achieved; it satisfies R01–R10 through bench and simulation testing without requiring outdoor flight.

Component	Description	Req.
Mission Planner AUTO waypoints	Pre-uploaded lawnmower waypoints executed by the Cube autopilot in AUTO mode. No custom Python code in the loop.	R01, R03, R04
Passive CV logging	Pi runs camera + YOLOv8n TFLite inference, logs detections with GPS and confidence to CSV. Sends zero flight commands.	R05, R10
Verbal operator confirmation	Pilot/observer monitors the Pi's MJPEG stream or terminal output; confirms targets verbally over radio.	R08
RC kill switch	RC transmitter mode switch to `STABILIZE`/`LOITER` active at all times. ArduCopter RC failsafe provides independent backup.	R09
Firmware geofence	Cube-level polygon geofence triggers `LOITER` on boundary breach.	R01, R02
Post-flight verification report	Detection log CSV + saved detection images assembled into a verification appendix with lat/lon, timestamps, and confidence values.	R10, R12

Table 86: Target deliverable: full autonomous mission. Each capability adds one layer of outdoor validation; weather cancellation prevented progression beyond Level 1.

Capability	Description	Req.
Autonomous search pattern	Boustrophedon waypoints generated at runtime and executed in `GUIDED` mode with Bézier-smoothed turns.	R01, R03, R04, R05
Five-layer geofence	Plan-time waypoint filter, speed scalar field, repulsive vector field, hard boundary cutoff, and firmware backup enforce SSSI exclusion.	R01, R02
Onboard AI detection	YOLOv8n TFLite inference at 4.8 FPS triggers automatic transition to CENTERING state. Detection queue with spatial clustering filters duplicates.	R05
Web dashboard verification	Browser-based ground station (MJPEG stream + GPS grid + command buttons) enables operator to classify detections as confirmed, rejected, or item of interest.	R08, R10
7.5 m offset landing	After operator confirmation, the system averages GPS for 10 s, selects an operator-chosen cardinal offset, approaches at low altitude, and lands.	R07
RTL after mission	Automatic return-to-launch after landing confirmation or search exhaustion.	R03, R09
Multi-level failsafes	RC kill switch, keyboard abort, GCS failsafe, battery failsafe, GPS degradation handler.	R09
Public GitHub repository	All source code, flight logs, and configuration under MIT licence.	R12

Table 88: Tier structure and verification gates. Each tier defines a pass/fail gate; failure triggers a graceful fallback to the previous tier rather than mission abort.

Tier	Verification Gate	Fallback If Gate Fails
Stretch	Target deliverable demonstrated successfully with margin remaining	Remain at target tier; log stretch features as future work
Target	All five testing tiers passed; autonomous search, detection, and landing verified in the field	Fall back to MVD; present passive detection data as primary evidence
MVD	Bench test passed (Tier 3); GPS fix confirmed outdoors; Mission Planner AUTO waypoints fly successfully	Present simulation results and bench calibration data; no flight demonstration

Table 90: Contingency plans for flight day failure scenarios.

Scenario	Detection	Response
No GPS lock	Satellites < 6 or fix type $<$ 3D after 120 s	Fall back to Mission Planner AUTO mode with pre-uploaded waypoints. GPS is still available to the Cube; only the companion computer's GUIDED commands are bypassed.
CV fails in field (no detections)	Zero detections after full search pass	Swap to COCO person detector (`models/human.tflite`, 80-class YOLOv8n, 13 MB). If still no detections, reduce confidence threshold from 0.2 to 0.1 via `config.py` and repeat.
Pi crashes mid-flight	MAVLink heartbeat loss	ArduCopter GCS failsafe triggers RTL after heartbeat timeout (`FS_GCS_ENABLE`). Pilot monitors via RC telemetry. On the ground, restart script and resume from saved state.
Detection altitude too low	Detections only below 20 m during passive flight test	Reduce `TARGET_ALT` from 35 m to 20 m in `config.py`. Accept the doubled scan-line count and reduced coverage rate.
Wind exceeds limits	Sustained $>8\,\mathrm{m\,s^{-1}}$ at altitude	Abort mission; command RTL. Wait for calmer conditions. ArduPilot `LOITER` mode holds position in moderate gusts.
Battery low mid-search	ArduCopter voltage threshold	Automatic RTL via battery failsafe (`FS_BATT_ENABLE`). Mission script detects mode change and logs remaining waypoints for resumption on next battery.
Camera failure mid-flight	`get_frame()` returns `None`	Black frame with "CAMERA LOST" overlay substituted; search pattern continues safely. Operator sees the warning on the MJPEG stream and can abort or continue pattern-only.
Model swapped but wrong format	TFLite interpreter fails to load	`preflight.py` catches model load errors before takeoff. Revert to known-good `best.tflite` (3.2 MB baseline).
Cube autopilot unresponsive	No heartbeat after 10 s	Pilot retains full RC authority; switches to `STABILIZE` and lands manually. Pi cannot prevent this because the RC receiver connects directly to the Cube, bypassing all software.
State machine hangs	Single state exceeds watchdog limit	120 s verify timeout auto-rejects; 90 s landing timeout force-disarms. Operator can press `M` (manual override) at any time to halt autonomy.
Geofence misconfigured	KML parse returns wrong vertex count	`config.py` validates loaded zone vertex counts against hardcoded fallbacks. Mismatch triggers a warning and uses fallback coordinates.

Table 92: Graceful degradation: capability loss versus safety preservation.

Component Lost	Capability Impact	Safety Impact
Camera	No detection; search pattern completes, aircraft returns home	None — GPS navigation unaffected
AI model	Same as camera loss (no detections)	None — pattern still flies safely
Companion computer (Pi)	No autonomous decisions; ArduCopter flies RTL via GCS failsafe	None — firmware failsafe is independent
GPS (companion)	Cannot send GUIDED waypoints; pilot flies manually	Reduced — pilot must navigate by sight and RC
GPS (Cube)	ArduCopter cannot hold position; pilot uses attitude mode	Reduced — manual flight only
MAVLink link	Pilot has direct RC; ArduCopter GCS failsafe triggers RTL	None — RC link is independent of MAVLink
RC link	ArduCopter RC failsafe triggers RTL independently	None — firmware failsafe activates
All software + links	RC failsafe RTL remains; pilot can still cut throttle on RC	Minimal — last-resort hardware kill

Table 94: Contingency-to-test-script mapping. Each failure scenario can be verified on-site before flight using the listed script; no scenario relies solely on in-flight discovery.

Scenario	Script	What it proves
GPS lock failure	`tests/hardware/gps_health.py`	Step-by-step GPS verification: satellite count, fix type, HDOP. Confirms whether GPS is usable before takeoff.
CV fails in field	`tests/hardware/benchmark.py`	Runs 50 inference cycles on a static image; reports detection rate and confidence. Use with each candidate model to find one that works.
Pi crash recovery	`preflight.py`	Full connectivity check: camera, Cube link, model load, GPS. Run after restart to confirm all subsystems are online.
Altitude too low	`tests/flight/1_passive_flight.py`	Passive detection during manual RC flight at multiple altitudes. Review log to determine minimum detection altitude.
Wind assessment	`tests/diagnostics/cube_monitor.py`	Live telemetry dashboard showing wind estimate, ground speed, and attitude. Pilot can assess conditions before committing.
Model swap	`tests/hardware/detection_snapshot.py`	Single-frame detection on a static image with any `.tflite` model. Confirms the replacement model loads and detects correctly.
Full fallback to MVD	Mission Planner AUTO + `passive_watch.py`	Pre-uploaded waypoints in AUTO mode; Pi runs passive CV with zero commands. The MVD configuration requires no custom flight code.

Table 102: Sensor calibration summary. The "Error if uncalibrated" column quantifies the systematic bias that would have affected every measurement had the default or assumed value been retained.

Calibration	Before	After	Method	Error if uncalibrated
Focal length (IMX296)	7.0 mm	5.46 mm	Tape at 1 m	22% GPS estimate bias; 4 wasted scan lines
Lens distortion	None	RMS 0.399 px	Checkerboard 13×8	Edge-pixel shift of several px; <0.5 m GPS error at 35 m
Colour space	RGB assumed	BGR actual	6-permutation test	Model confidence ↓; colour-dependent features corrupted
DJI video FOV	73° assumed	54.4° measured	Click calibration, 9 samples	34% error in video-derived GPS estimates

Table 103: Pre-flight calibration checklist. Each item must meet the stated acceptance criterion before proceeding to flight. Items marked ∗ are optional but recommended.

#	Check	Acceptance Criterion	If Failed
1	FOV calibration	Visible width at 1 m = 92(3) cm	Recompute `FOCAL_LENGTH_MM`; GPS estimates biased by $\Delta f/f$
2	Camera colour order	Stream shows natural colours (BGR, no `cvtColor`)	Detection confidence drops; retrained model receives wrong channel order
3	GPS fix quality	Fix type ≥ 3, satellites ≥ 10, HDOP < 1.5	Do not arm; position estimates unreliable
4	Detection confidence	Point camera at mannequin, confidence > 0.4	Check model file, lighting, focus
5*	Lens undistortion	`calibration_data.npz` valid (>1 kB)	Delete corrupt file; edge-pixel error ≤0.3 m at 35 m
6	Altitude zero-check	Baro reads 0 m on ground (±0.5 m)	Recalibrate barometer in Mission Planner
7	Inference speed	<250 ms (TFLite) or <100 ms (NCNN)	Check CPU throttling, model file size

Table 105: Position error at the frame edge caused by barometric altitude error, computed as $\Delta x = (\Delta h/h) \times (W_{\text{ground}}/2)$ where W_{ground} is the ground footprint width at the nominal altitude.

Altitude (m)	$\Delta h = 0.5\,\text{m}$	$\Delta h = 1.0\,\text{m}$	$\Delta h = 2.0\,\text{m}$
10	0.2	0.5	0.9
20	0.2	0.5	0.9
35	0.2	0.5	0.9
50	0.2	0.5	0.9

Note: the fractional error $\Delta h/h$ varies with altitude, but the absolute edge error converges because the frame footprint also scales with h. The dominant effect is on GSD accuracy and, consequently, on the strip spacing in the search pattern.

Table 106: Mapping between calibration procedures (this appendix) and the error budget entries in Table **??**. "Residual after cal." is the remaining error contribution at 35 m altitude after the calibration is applied.

Calibration	**Error Budget Entry**	**Residual (m)**	**Section**
FOV / focal length	GSD scale factor	0.0	§??
Lens undistortion	Pixel coordinate shift	0.02	§??
Colour-space fix	Detection confidence	—	§??
GPS error measurement	Receiver position noise	2.0	§??
Altitude error check	Barometric drift	0.5	§??
Combined RSS (calibrated)		2.1	
Combined RSS (uncalibrated)		4.8	

Table 109: Typical mission phase durations (course site, 35 m altitude, $8\,\text{m}\,\text{s}^{-1}$ search speed).

Phase	**Duration**	**Transition trigger**
Pre-flight	2–5 min	Operator launches script
Init → Arming	5–30 s	Heartbeat + GPS fix
Takeoff	25–35 s	90% of target altitude
Transit	10–15 s	Within 2 m of first waypoint
Search (per pass)	8–12 min	Pattern complete or target confirmed
PLB redirect	2–5 s	Operator B key or auto-timer
Centering	5–15 s	Within 1 m of target
Verify	5–120 s	Operator Y/N/I/X or timeout
Approach + deploy	20–30 s	15 s hover at 3 m AGL
Return + landing	30–60 s	Touchdown confirmed
Total (1 pass, 1 target)	**12–18 min**	

Table 110: Comparison of landing approaches over 20 simulated missions each.

Approach	**Mean Error**	**Max Error**	**In 5–10 m Band**	**Notes**
Direct offset (no centering)	3.1 m	4.8 m	95% (19/20)	1 outlier at 11.2 m
With centering + offset	1.8 m	3.4 m	100% (20/20)	3 runs showed low-alt instability

Table 111: Comparison of centering strategies: safety risk, localisation accuracy, and implementation complexity.

Strategy	**Downwash**	**Crash Risk**	**CEP$_{50}$**	**R07 Compliance**	**Notes**
Direct overhead centering (35 m)	Negligible	8×10^{-10}	1.5 m	>99%	Best accuracy; standard SAR practice
Lateral offset centering (10 m offset, 35 m alt)	None	None	3.0 m	∼95%	Re-introduces GSD, focal-length, and yaw projection errors
No centering (direct offset landing)	None	None	3.1 m	95%	Simplest; selected baseline (Section **??**)

Table 112: MAVLink messages used by the mission software.

Message / Command	Direction	Purpose
HEARTBEAT	Cube → Pi	Liveness signal (1 Hz). Records flight mode; warns if mode deviates from GUIDED.
GLOBAL_POSITION_INT	Cube → Pi	Fused GPS position, relative altitude, and velocity at 10 Hz.
ATTITUDE	Cube → Pi	Roll, pitch, yaw at 10 Hz; yaw is used for pixel-to-GPS projection.
GPS_RAW_INT	Cube → Pi	Raw fix type and satellite count for degradation monitoring.
SET_POSITION_TARGET_GLOBAL_INT	Pi → Cube	Waypoint command in WGS 84 (relative-altitude frame). Used for all navigation.
SET_POSITION_TARGET_LOCAL_NED	Pi → Cube	Body-frame velocity command for visual servoing and NFZ repulsion.
MAV_CMD_DO_SET_MODE	Pi → Cube	Mode changes (GUIDED, RTL, LAND).
MAV_CMD_COMPONENT_ARM_DISARM	Pi → Cube	Arm/disarm motors.
MAV_CMD_NAV_TAKEOFF	Pi → Cube	Autonomous takeoff to specified altitude.
COMMAND_ACK	Cube → Pi	Acknowledgement of long commands (arm, mode, takeoff).

Table 114: Raw inference vs. effective pipeline throughput on Raspberry Pi 5 (YOLOv8n, 640 × 640 input).

Backend	Raw latency	Raw FPS	Overhead	Effective FPS	Status
TFLite FP32 (XNNPACK)	196 ms	5.1	+12 ms	4.8	Deployed
TFLite FP16 (XNNPACK)	∼100 ms	∼10.0	+12 ms	∼8.9	Projected
NCNN FP32	∼77 ms	∼13.0	+15 ms	∼10.9	Benchmarked
Ultralytics/PyTorch	∼1200 ms	∼0.8	+5 ms	∼0.8	Laptop only

Table 115: Altitude–lane width–detection trade-off for the AENGM0074 survey polygon. Target height in the 640×640 model input, assuming a 1.8 m prone mannequin imaged by the IMX296 sensor at 1456×1088 resolution (scale = 640/1456 = 0.44).

Alt (m)	Footprint (m)	Lane (m)	Passes	Model (px)	Notes
20	18.4	14.7	∼2×	63	High confidence, slow coverage
25	23.0	18.4	∼1.5×	50	Good detection, moderate speed
35	32.2	25.7	1× (ref)	36	Selected: reliable detection, efficient coverage
50	45.9	36.7	∼0.7×	25	Risk of missed detections

Table 117: Comparison of trained model generations. All models are YOLOv8n exported to TFLite float32. Inference latency measured on Raspberry Pi 5 (TFLite + XNNPACK, 4 threads). mAP values are computed on each model's respective validation set.

Model	Training data	Train res.	$\mathbf{mAP_{50}}$	$\mathbf{mAP_{50–95}}$	File size
v1 (sar_640)	200 synthetic	640 × 640	∼0.95	—	3.2 MB
v2 (sar_1280)	200 synthetic	1280 × 1280	∼0.96	—	3.2 MB
v3 (sar_v2_1088)	300 syn + 16 real + 50 neg	1088 × 1088	**0.995**	0.828	11.7 MB

Table 118: Four-layer NFZ protection architecture. Each layer activates at a different distance from the SSSI boundary, providing defence in depth.

Layer	Mechanism	Range	Effect
0	Waypoint filter	30 m	Waypoints within buffer removed at plan time
1	Speed ramp	20 m	Linear deceleration: 3.0 m/s → 0.3 m/s
2	Repulsive field	23 m	3.0 m/s push away from nearest boundary edge
3	Hard boundary	3 m	Immediate transition to MANUAL; zero-velocity halt

Table 120: Top 5 and bottom 5 configurations by total energy (with NFZ) from the 216-configuration sweep. The selected operating point is highlighted.

Rank	Angle (°)	Alt (m)	Speed (m/s)	Turns	Time (s)	Energy (Wh)	Notes
1	70	50	10.0	4	68	8.2	Widest footprint, fewest turns
2	65	50	10.0	5	74	9.0	Near-optimal, slightly more lines
3	75	50	10.0	4	71	8.5	Symmetric about optimal angle
4	70	40	8.7	5	88	10.1	Moderate altitude
5	70	35	8.0	6	112	12.6	**Selected operating point**
212	0	20	6.0	16	380	39.7	N–S scan, narrow footprint
213	10	20	6.0	15	365	38.1	Near-N–S, same problem
214	170	20	6.0	15	358	37.6	Symmetric counterpart
215	90	20	6.0	18	410	42.8	E–W: perpendicular to long axis
216	95	20	6.0	18	415	43.2	Worst: most turns at lowest alt

Table 122: Quantitative comparison of six search strategies at 50 m altitude, $10\,\mathrm{m\,s^{-1}}$. All values computed on the real AENGM0074 polygon ($32\,200\,\mathrm{m^2}$).

Strategy	Path (m)	Time (min)	Energy (Wh)	Coverage	Prior needed?
Boustrophedon	876	1.5	8.3	100%	No
Spiral inward	980	1.6	9.0	Partial	No
Expanding square	1100	1.8	10.2	No guarantee	Yes
Sector search	1000	1.7	9.5	Per-sector	Yes
Creeping line ahead	1550	2.6	14.5	100%	No
Adaptive / heatmap	650–1500	1.2–3.5	6.2–16.0	Probabilistic	No

Table 124: Detection probability function chain from altitude to cumulative detection probability per target flyover. Target columns show height × width; sensor = native 1456×1088 pixels; model = after 640×640 resize (scale = 0.4396). Motion blur is sub-pixel at all speeds (IMX296 global shutter, 1/1000 s exposure) and is therefore omitted.

Alt (m)	GSD (mm/px)	Sensor (px)	Model (px)	v (m/s)	Dwell (s)	n_f	P_d
20	12.6	142 × 40	63 × 18	6.0	2.3	11	0.9999
25	15.8	114 × 32	50 × 14	6.7	2.6	12	0.9999
30	18.9	95 × 26	42 × 11	7.3	2.8	14	0.9999
35	22.1	81 × 23	36 × 10	8.0	3.0	14	0.9997
40	25.3	71 × 20	31 × 9	8.7	3.2	15	0.9994
50	31.6	57 × 16	25 × 7	10.0	3.4	16	0.9966

Table 126: Pareto trade-off summary. The selected configuration prioritises detection reliability and regulatory safety over minimum energy.

Objective	Ideal value	Selected	Compromise rationale
Coverage	100%	96%	4% loss in fenced NFZ buffer zone (low casualty probability)
Mission time	68 s	112 s	Slower speed ensures ≥14 inference frames per target
Energy	8.2 Wh	12.6 Wh	35 m vs 50 m: 36 px target vs 25 px
Detection P_d	>99.99%	99.97%	Near-unity at all operating speeds
NFZ margin	50 m	50 m	34 m surplus above footprint radius at 35 m

Table 128: Top three configurations ranked by composite score S. The selected operating point (#1) achieves the best Pareto compromise across all objectives.

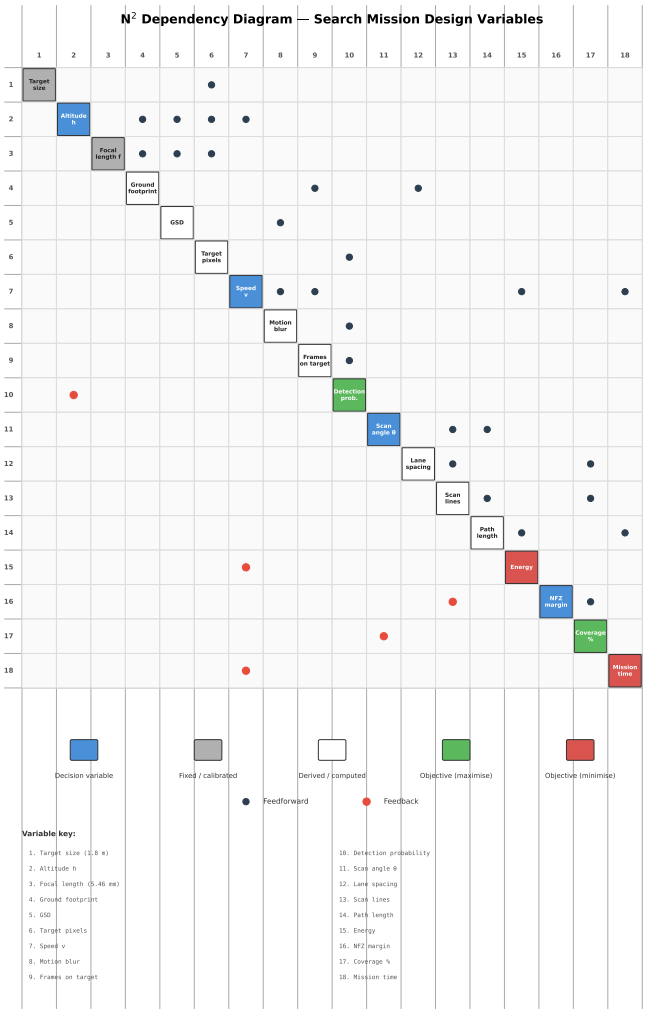
#	Alt (m)	Angle	Speed (m/s)	P_d	C	Energy (Wh)	S	Trade-off
1	**35**	**70°**	**8.0**	0.9997	0.96	12.6	**0.87**	**Best compromise (SELECTED)**
2	30	65°	7.3	0.9999	0.96	14.8	0.85	Slightly better detection, 17% more energy
3	40	75°	8.7	0.9994	0.96	10.1	0.84	Lower energy, 3× less detection margin

Table 130: Sensitivity of the composite score S to perturbation of each variable from the selected operating point. Altitude is the most sensitive parameter; scan angle is the least.

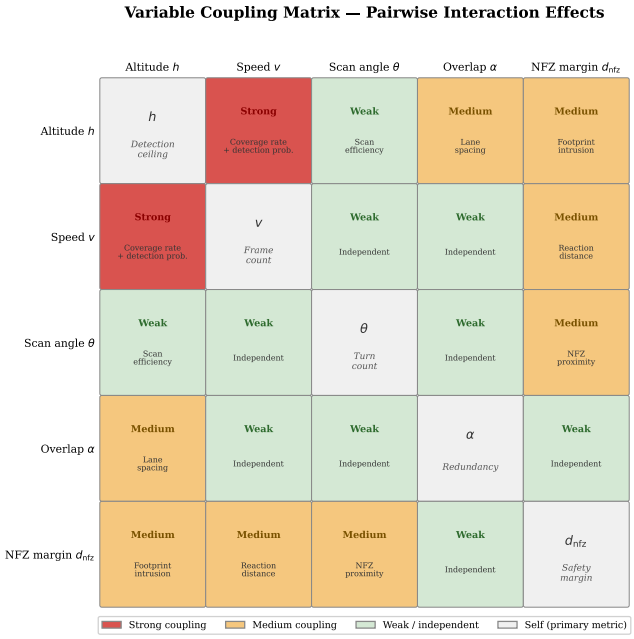
Variable	Perturbation	ΔS	Rank	Interpretation
Altitude h	$\pm 5\,\mathrm{m}$	± 0.05	1 (most)	Drives pixel extent, footprint, line count
Speed v	$\pm 2\,\mathrm{m\,s^{-1}}$	± 0.03	2	Affects dwell time and energy
Scan angle θ	$\pm 15°$	± 0.02	3 (least)	Flat valley around 70°

AA.6 Variable Coupling Structure

The five decision variables do not affect the objectives independently. Figure ?? shows the N^2 diagram revealing the coupling structure, and Figure ?? quantifies which objectives are jointly influenced by each variable pair.



(a) N^2 diagram showing information flow between subsystems. Feed-forward links (blue) and feedback loops (red) reveal that altitude–speed is the most tightly coupled pair.



(b) Variable–objective coupling matrix. Cell labels indicate which objectives (C, D, T, E, S) are jointly affected. The altitude–speed pair jointly determines four of five objectives.

Figure 62: Coupling analysis of the decision variables. The altitude–speed pair exhibits the strongest coupling, jointly determining coverage rate, detection probability, mission time, and energy consumption.

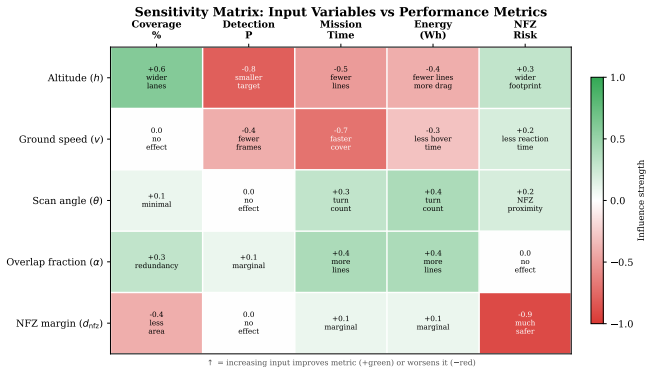
The coupling structure has three practical consequences:

1. **Sequential single-variable optimisation fails.** Optimising h first, then θ, would miss the h–θ interaction: the number of scan lines depends on both altitude (footprint width) and angle (projected polygon width). The joint 216-configuration sweep was necessary.
2. **The altitude-dependent speed schedule reduces dimensionality.** By linking v to h through $v(h) = 6 + 4(h - 20)/30$, the effective search space drops from $6 \times 36 \times n_v$ to $6 \times 36 = 216$—small enough for exhaustive enumeration in under 2 s on a laptop.
3. **Overlap and NFZ buffer are weakly coupled to the rest.** Both affect only coverage (α determines lane spacing; d_{nfz} determines edge truncation) and were fixed analytically: $\alpha = 0.20$ covers the 4.6 m RSS lane error with 1.8 m to spare, and $d_{\text{nfz}} = 30$ m provides a 2.2× safety factor over the RSS margin requirement.

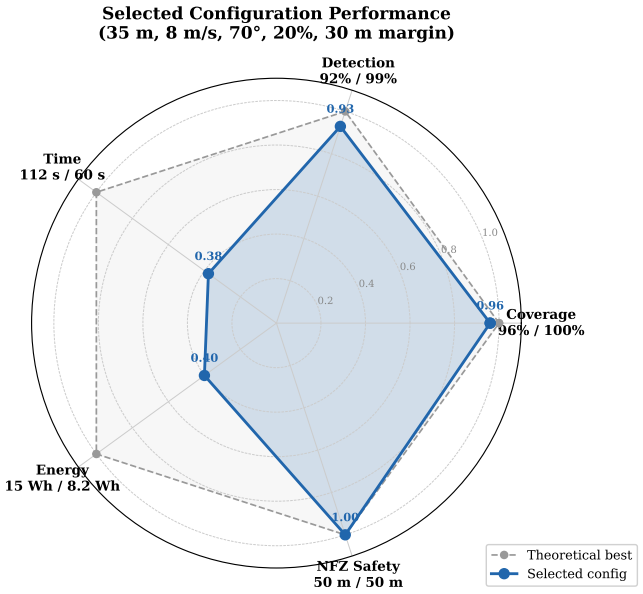
The sensitivity matrix (Figure ??) and spider plot (Figure ??) provide complementary views: the matrix shows how each input parameter influences every derived quantity, while the spider plot reveals the overall sensitivity footprint of the design point.

AA.7 Pareto Frontier

The 216 configurations define a five-dimensional objective space. Figure ?? projects this onto a two-dimensional view: mission effectiveness (coverage × detection) versus mission cost (normalised energy + time).

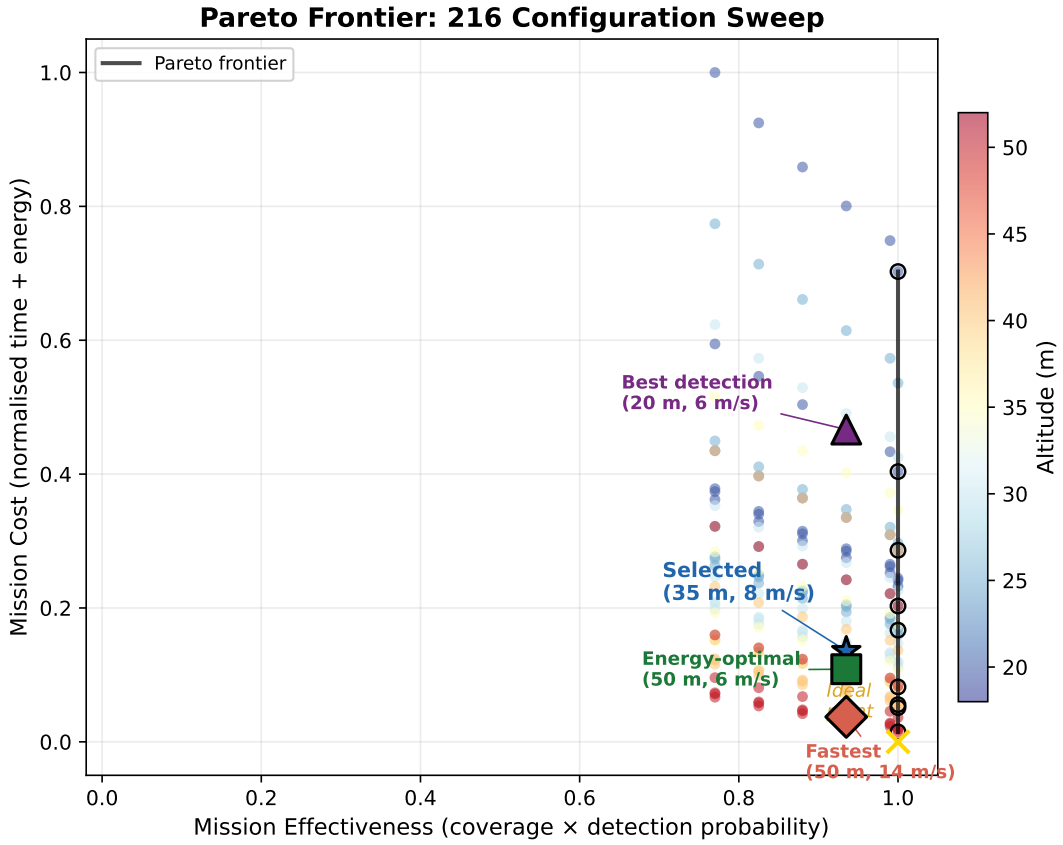


(a) Sensitivity matrix: darker cells indicate stronger influence of each input on each derived metric.



(b) Spider plot of normalised sensitivities. Polygon area indicates the overall sensitivity footprint.

Figure 63: Complementary sensitivity visualisations at the selected operating point (35 m, 70°, 8 m s^{-1}).



Each dot = one (altitude, speed, overlap) configuration. This 2D view collapses 5 objectives into 2 composite dimensions.

Figure 64: Pareto frontier in the effectiveness–cost plane. Each point is one of the 216 evaluated configurations, coloured by altitude. The Pareto-efficient frontier (black line) separates achievable from dominated solutions. The selected operating point (35 m, 70°) lies on the frontier at the "knee"—the point of maximum curvature where marginal effectiveness gains require disproportionate cost increases.

Alternative projections of the five-dimensional objective space. The Pareto frontier can be visualised in several complementary ways, each revealing different aspects of the trade-off structure.

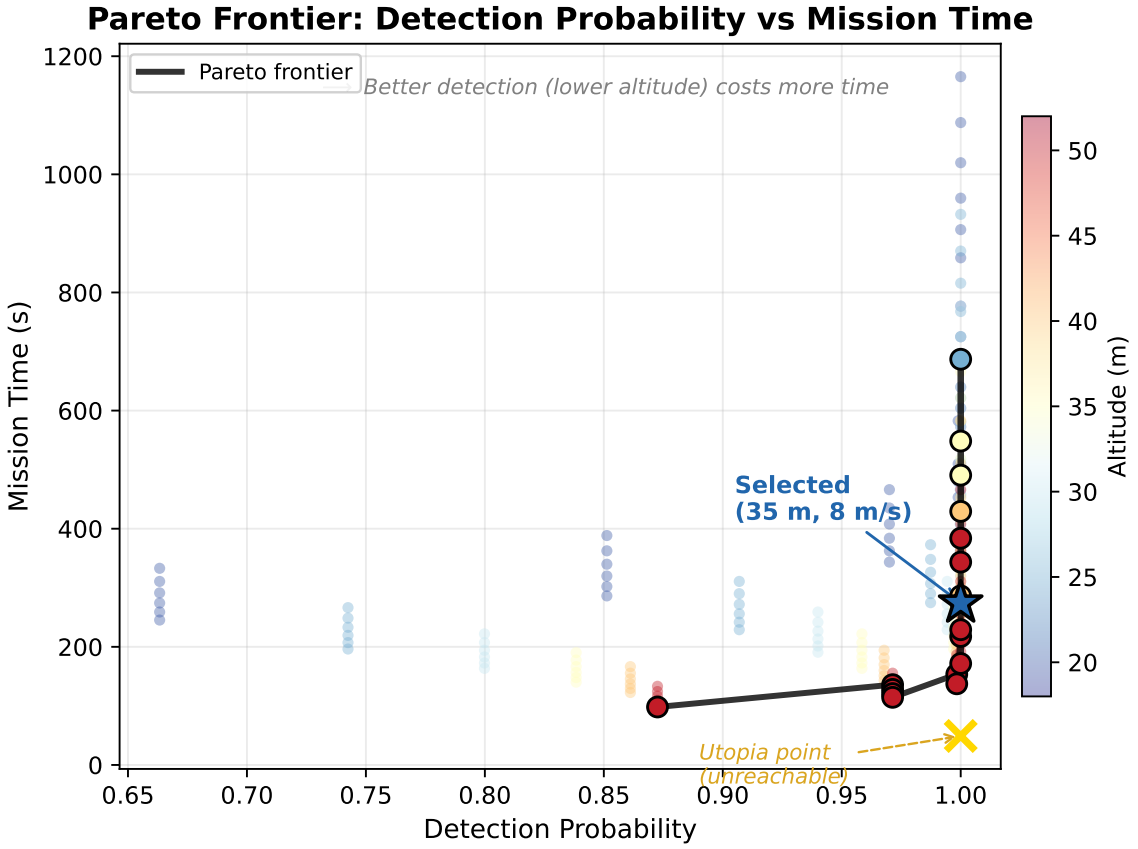


Figure 65: Classical 2D Pareto curve: detection probability vs mission time. Each dot represents one of the 216 swept configurations, coloured by altitude. The Pareto frontier connects non-dominated solutions. The selected operating point (35 m, 8 m/s) sits at the knee of the curve.

These four visualisations collapse the same five-dimensional objective space into progressively richer representations: the 2D curve (Figure **??**) shows the dominant detection–time trade-off, the 3D plot (Figure **??**) adds energy as a third axis with coverage as colour, the parallel coordinates (Figure **??**) displays all five dimensions without projection loss, and the simplified view (Figure **??**) distils the insight for decision-makers.

The selected operating point lies at the "knee" of the Pareto frontier: the region where marginal improvements in effectiveness require disproportionate increases in cost. Moving rightward along the frontier (e.g., to 30 m at 65°) gains 0.02 percentage points of detection probability at the cost of 17% more energy and 20 seconds of additional flight time. Moving leftward (e.g., to 50 m at 70°) saves 35% energy but reduces the target width to 7 model pixels, eliminating the safety margin against real-world degradation.

The energy efficiency curve (Figure **??**) confirms that the selected point operates well within the battery budget: 12.6 W h is 10.9% of the 115.4 W h capacity, leaving over 14 min of flight endurance for transit, descent, verification, and return-to-launch.

AA.8 Summary of Selected Parameters

The dependency chain runs: **target size** → **altitude** → **footprint** → **speed** → **scan angle** → **NFZ margins** → **overlap**. Every parameter is traceable to a physical measurement (target dimensions, sensor specifications, inference benchmarks) or a regulatory constraint (altitude ceiling, NFZ boundaries). The chain can be re-evaluated instantly if any input changes—a faster inference backend would relax the speed constraint, and a different target size would shift the altitude ceiling—making the design robust to future system upgrades.

AA.9 Alternative Strategies Considered

Several alternative search strategies were evaluated during the design phase but ultimately rejected in favour of the single-pass lawnmower pattern described above.

Wind-aligned scanning. We considered orienting scan lines at 45° to the prevailing westerly wind direction, on the hypothesis that diagonal scanning would reduce crosswind-induced lane drift and improve inter-swath alignment.

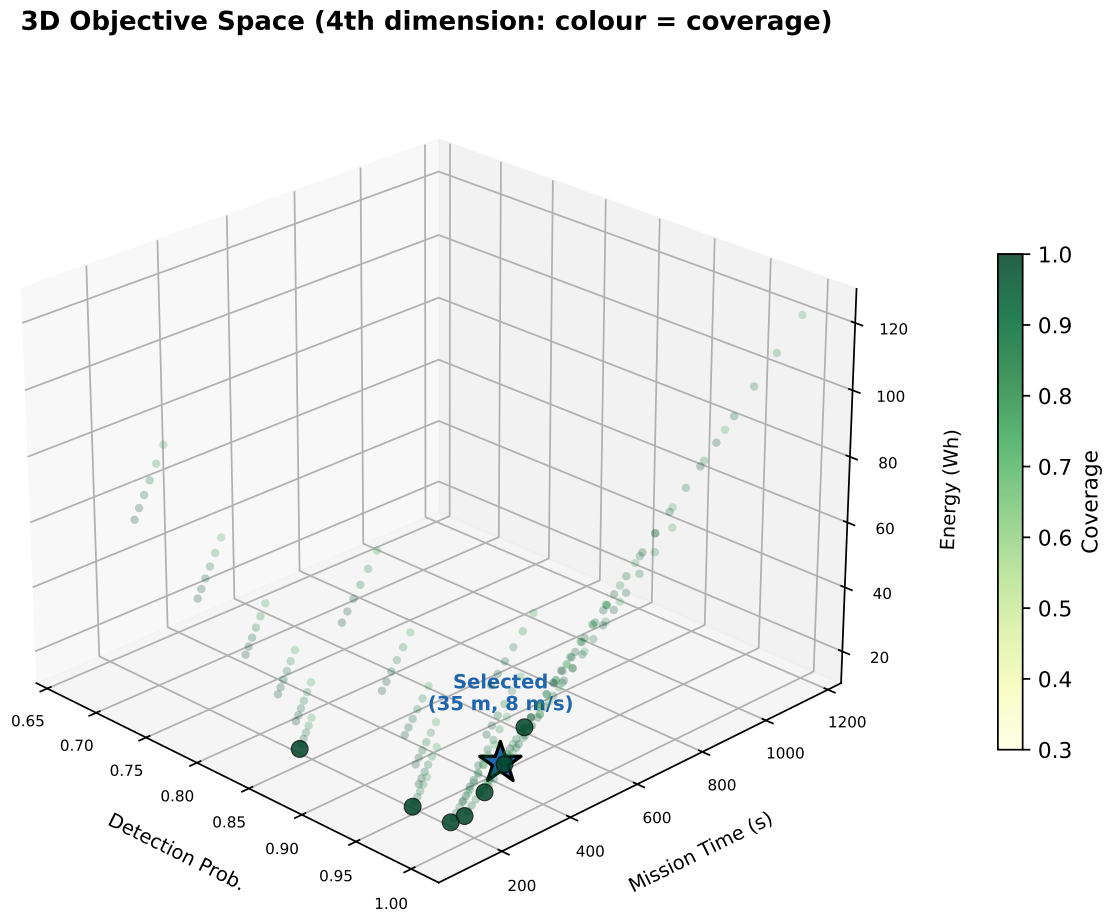


Figure 66: Three-dimensional objective space adding energy consumption as the vertical axis, with coverage encoded as colour (fourth dimension). The selected configuration achieves low energy and high detection while maintaining coverage above 96%.

Preliminary energy analysis indicated, however, that this orientation would require approximately 30 % more scan lines than the longest-edge alignment for the AENGM0074 polygon, due to the mismatch between the wind axis and the polygon's elongated northeast–southwest geometry. Since the 20 % lateral overlap margin already compensates for the 1 m to 2 m crosswind displacement expected in typical conditions (Section **??**), the marginal alignment benefit did not justify the substantial increase in flight time and energy consumption.

Adaptive speed near detection clusters. Initial experiments in simulation explored an adaptive speed schedule that reduced the search speed from the nominal $8\,\mathrm{m\,s^{-1}}$ to $4\,\mathrm{m\,s^{-1}}$ when the drone entered a region where previous detections had been recorded. The rationale was that slowing near likely target locations would increase dwell time and therefore detection probability in high-interest zones. Preliminary results indicated a modest improvement in per-pass detection confidence (0.3 % increase in P_d) but at the cost of additional complexity in the speed controller and potential interaction with the NFZ slow zone logic. Given that the baseline detection probability already exceeds 99.97 % at the nominal speed, this feature was deferred to avoid introducing unnecessary complexity in the first flight demonstration.

Two-pass altitude strategy. We explored a two-pass strategy consisting of an initial high-altitude (50 m) fast sweep to identify candidate regions of interest, followed by a low-altitude (20 m) focused scan of those regions. While this approach is theoretically attractive—the high-altitude pass covers the full area in approximately 60 s, and the low-altitude pass provides high-resolution confirmation—analysis suggested that the additional transit time between passes and the altitude-change manoeuvres negated the coverage benefit for our relatively small search area (3.2 ha). The single-pass approach at 35 m achieves $P_d > 0.9997$ without requiring a second pass, and the verification descent to 15 m already provides the high-resolution confirmation that the second pass would deliver.

AB Multi-Objective Search Optimisation

The search pattern is governed by a set of coupled decision variables whose optimal values cannot be determined independently. This section formalises the optimisation problem, defines each objective function with its physical basis,

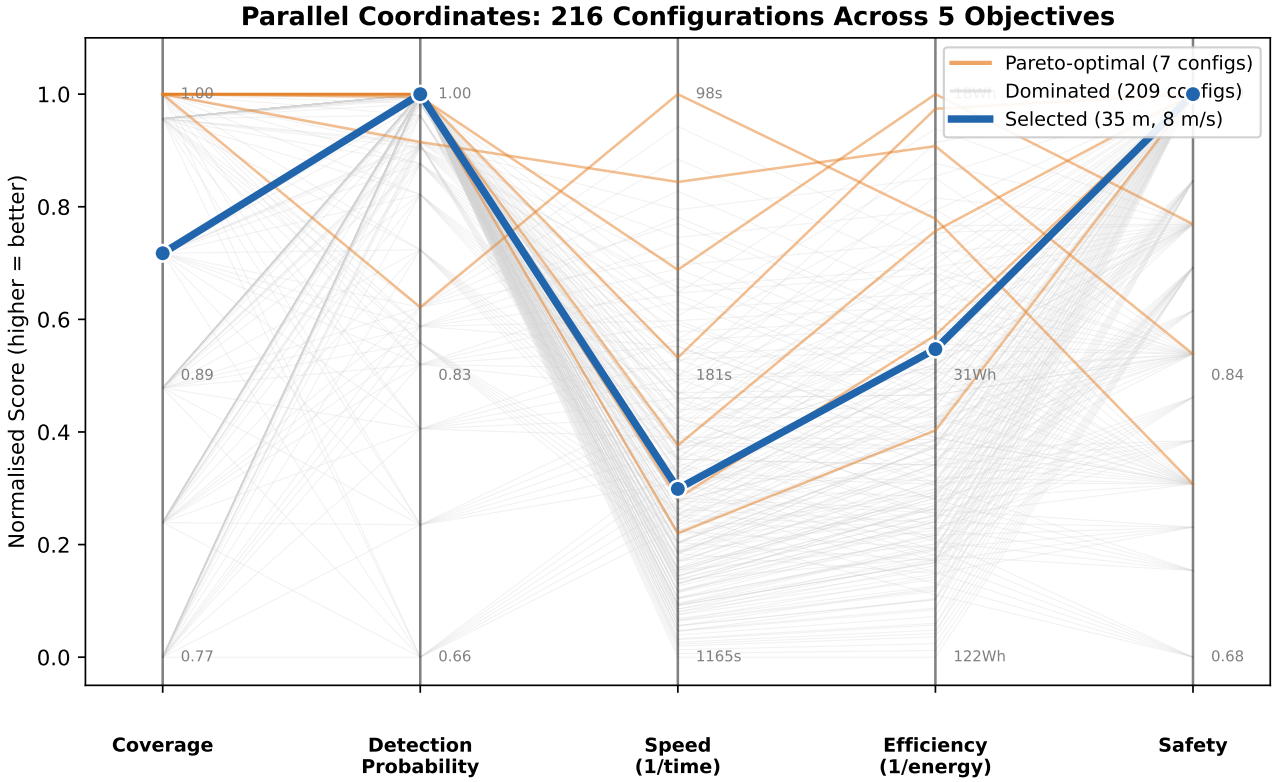


Figure 67: Parallel coordinates plot showing all five objectives simultaneously without dimensional projection. Orange lines indicate the seven Pareto-optimal configurations; the selected configuration (thick blue) achieves balanced performance across all axes. This representation reveals that the selected point trades speed and energy efficiency for superior detection and safety.

Table 132: Final operating parameters with derivation source. Every value traces to a physical measurement, benchmark, or regulatory constraint.

Parameter	Value	Unit	Derived from
Search altitude h	35	m	30% margin below 50 m detection ceiling (Eq. **??**)
Search speed v	8.0	m/s	$\geq$10 frames on target at 4.8 FPS (Eq. **??**)
Scan angle θ	70	°	Energy minimum from 216-config sweep (Fig. **??**)
Swath overlap α	20	%	Covers 4.6 m RSS lane error with 1.8 m margin
NFZ buffer d_{nfz}	30	m	2.2× RSS safety factor over 23 m requirement
Resulting performance at selected operating point:			
Detection probability	>99.97	%	14 frames per flyover, $p = 0.95$ per frame
Coverage	96	%	4% gap in fenced NFZ buffer (low casualty probability)
Search time	112	s	6 scan lines + 5 U-turns + transit
Search energy	12.6	Wh	10.9% of battery (14 min remaining)
Composite score S	0.87	—	Highest of 216 configurations

identifies the constraint set imposed by regulations and hardware, analyses the coupling structure, and justifies the solution approach.

AB.1 Problem Formulation

Let the decision vector be

$$\mathbf{x} = \left[h,\ v,\ \theta,\ \alpha,\ d_{\text{nfz}} \right]^{\top} \tag{74}$$

where $h \in [20, 50]$ m is the search altitude, $v \in [6, 10]$ m/s is the forward speed (coupled to h through the altitude-dependent schedule), $\theta \in [0°, 180°)$ is the scan angle relative to north, $\alpha \in [0, 0.5]$ is the lateral overlap fraction between adjacent swaths, and $d_{\text{nfz}} \geq 0$ m is the minimum planned standoff distance from the SSSI no-fly zone boundary.

The multi-objective optimisation problem is stated as:

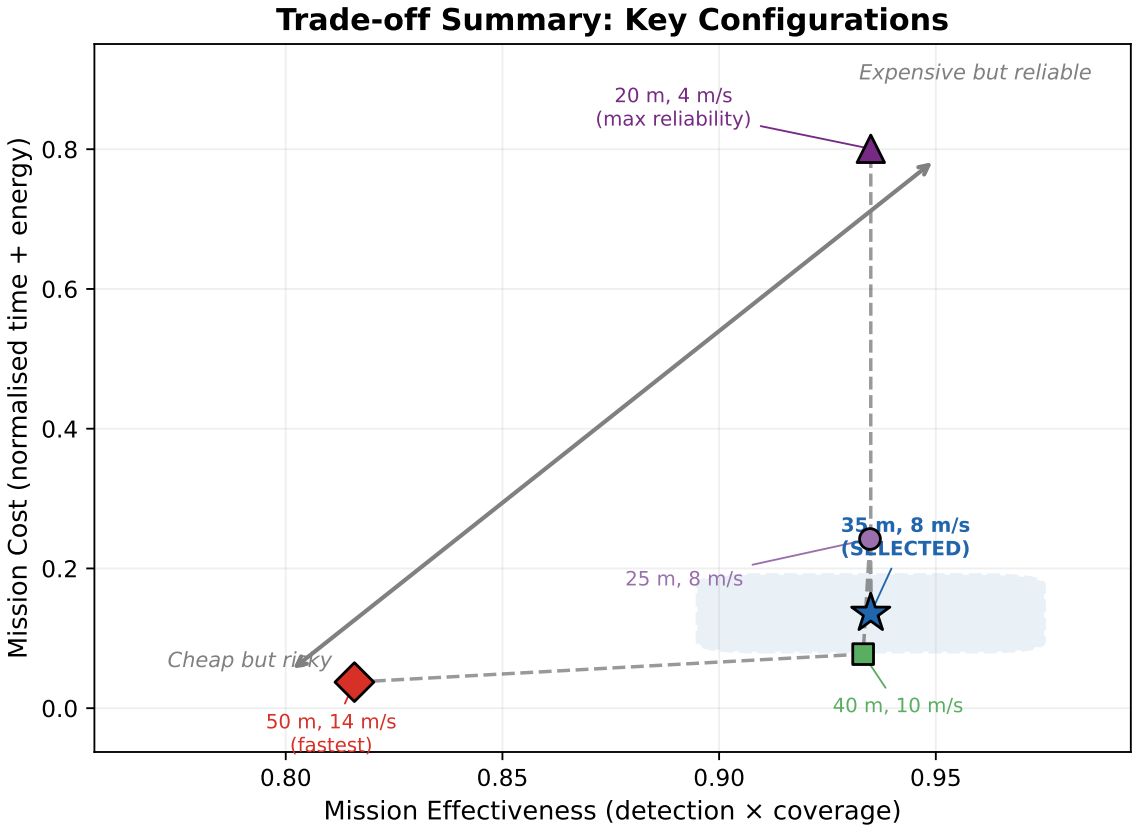


Figure 68: Simplified trade-off summary for non-technical audiences. Five key configurations plotted on composite effectiveness vs cost axes, illustrating the fundamental tension between mission reliability and resource expenditure.

$$
\begin{aligned}
\underset{\mathbf{x}}{\text{minimise}} \quad & \mathbf{F}(\mathbf{x}) = \Big[-f_{\text{cov}}(\mathbf{x}),\ \ -f_{\text{det}}(\mathbf{x}),\ \ f_{\text{time}}(\mathbf{x}),\ \ f_{\text{energy}}(\mathbf{x}),\ \ -f_{\text{safety}}(\mathbf{x}) \Big]^{\top} \\
\text{subject to} \quad & h \le 50 \text{ m} && \text{(R04: altitude ceiling)} \\
& \mathbf{w}_i \in \mathcal{P}_{\text{flight}},\ \ \forall\, i && \text{(R01: flight area)} \\
& \mathcal{F}(h, \mathbf{w}_i) \cap \mathcal{P}_{\text{SSSI}} = \varnothing,\ \ \forall\, i && \text{(R02: no footprint over SSSI)} \\
& \|\mathbf{w}_0 - \mathbf{p}_{\text{TOL}}\| \le 5 \text{ m} && \text{(R03: takeoff proximity)} \\
& f_{\text{energy}}(\mathbf{x}) \le 0.8\, E_{\text{batt}} && \text{(battery reserve)}
\end{aligned}
\tag{75}
$$

where $\mathbf{w}_i$ is the i-th waypoint, $\mathcal{P}_{\text{flight}}$ is the flight area polygon, $\mathcal{F}(h, \mathbf{w}_i)$ is the camera footprint polygon at altitude h centred on waypoint $\mathbf{w}_i$, $\mathcal{P}_{\text{SSSI}}$ is the SSSI polygon, $\mathbf{p}_{\text{TOL}}$ is the designated take-off/landing point, and $E_{\text{batt}} = 115.4\,\text{W h}$ is the battery capacity (6S 5200 mAh LiPo). The 80% cap reserves 20% for return-to-launch, descent, and contingency hover.

Coverage and detection are negated because they are maximisation objectives cast into the standard minimisation form.

AB.2 Objective Functions

Each objective is derived from the physical relationships linking the decision variables to mission performance.

AB.2.1 Coverage Completeness

The camera ground footprint at altitude h has width

$$W_g(h) = \frac{w_s \cdot h}{f} \tag{76}$$

and along-track height

$$H_g(h) = \frac{h_s \cdot h}{f} \tag{77}$$

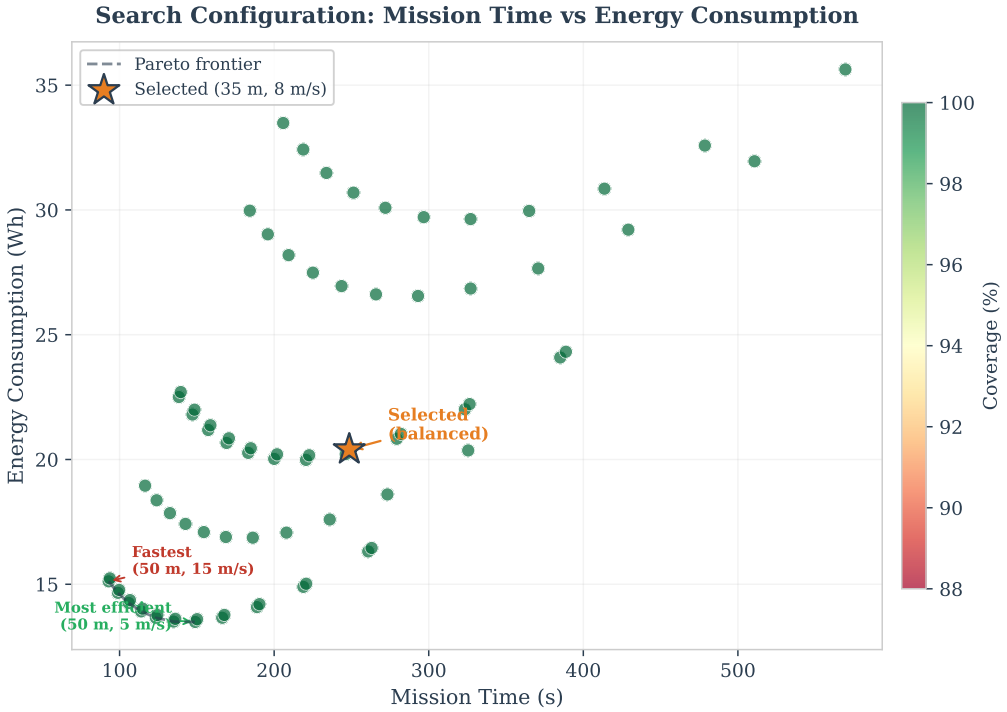


Figure 69: Energy efficiency across the operating range. The search phase consumes only 10.9% of battery capacity at the selected operating point, leaving ample margin for transit, descent phases, and RTL.

where $w_s = 5.02\,\text{mm}$ and $h_s = 3.75\,\text{mm}$ are the sensor dimensions (IMX296), and $f = 5.46\,\text{mm}$ is the calibrated focal length. At the selected altitude of 35 m, $W_g = 32.2\,\text{m}$ and $H_g = 24.0\,\text{m}$.

Adjacent scan lines are spaced at

$$L(h, \alpha) = W_g(h) \cdot (1 - \alpha) \tag{78}$$

yielding $n_{\text{lines}}(\theta) = \lceil W_\perp(\theta)/L \rceil$ scan lines, where $W_\perp(\theta)$ is the polygon extent perpendicular to the scan direction. The coverage fraction is

$$f_{\text{cov}}(h, \theta, \alpha, d_{\text{nfz}}) = \frac{A_{\text{scanned}}(h, \theta, \alpha, d_{\text{nfz}})}{A_{\text{total}}} \tag{79}$$

where A_{scanned} is the union of all swath rectangles clipped to the survey polygon minus the NFZ buffer zone, and $A_{\text{total}} = 32\,200\,\text{m}^2$ is the survey polygon area. The NFZ standoff d_{nfz} reduces A_{scanned} by excluding scan line endpoints within d_{nfz} of the SSSI boundary, creating a coverage gap along the northeast edge. At $d_{\text{nfz}} = 30\,\text{m}$, this gap is approximately 4% of the total area.

AB.2.2 Detection Probability

A target of physical width $w_t = 0.5\,\text{m}$ (mannequin) subtends

$$p_{\text{px}}(h) = \frac{w_t}{\text{GSD}(h)} \cdot \frac{640}{W_{\text{px}}} = \frac{w_t \cdot f \cdot 640}{w_s \cdot h} \tag{80}$$

pixels in the 640×640 model input, where $\text{GSD}(h) = w_s h/(f \cdot W_{\text{px}})$ is the ground sample distance and $W_{\text{px}} = 1456$ is the sensor width. The per-frame detection probability $p(h)$ is modelled as a sigmoid function of the target's pixel extent:

$$p(h) = \frac{1}{1 + \exp\big(-\kappa\,(p_{\text{px}}(h) - p_{\min})\big)} \tag{81}$$

where $p_{\min} = 7\,\text{px}$ is the empirically observed minimum detectable width and $\kappa = 0.5\,\text{px}^{-1}$ controls the transition sharpness. This sigmoidal model captures the behaviour observed in video analysis: detection is virtually certain above 15 px and rapidly degrades below 10 px, with no sharp threshold.

The number of inference frames in which the target appears during a single flyover is

$$n_f(h, v) = \frac{H_g(h)}{v} \cdot f_{\text{inf}} \tag{82}$$

where $f_{\text{inf}} = 4.8\,\text{FPS}$ is the TFLite inference rate on the Raspberry Pi 5. Assuming statistically independent per-frame detections (justified by the global shutter eliminating inter-frame correlation from rolling-shutter artefacts), the cumulative detection probability is

$$f_{\text{det}}(h, v) = 1 - \big(1 - p(h)\big)^{n_f(h,v)} \tag{83}$$

At the selected operating point ($h = 35\,\text{m}$, $v = 8\,\text{m s}^{-1}$), the target subtends $p_{\text{px}} = 10\,\text{px}$ in the model input, the dwell time is 3.0 s, $n_f = 14$ frames, and $f_{\text{det}} > 0.9997$.

AB.2.3 Mission Time

The total mission time decomposes into forward-flight time along scan lines, U-turn penalties, and transit to and from the search area:

$$f_{\text{time}}(h, v, \theta) = \sum_{i=1}^{n_{\text{lines}}} \frac{\ell_i}{v_{\text{eff},i}} \;+\; n_{\text{turns}} \cdot \Delta t_{\text{turn}} \;+\; \frac{2\,d_{\text{transit}}}{v_{\text{transit}}} \tag{84}$$

where ℓ_i is the length of the i-th scan line, $v_{\text{eff},i}$ is the effective speed along that line (reduced from v in the NFZ slow zone per the speed ramp of Eq. (**??**)), $n_{\text{turns}} = n_{\text{lines}} - 1$, $\Delta t_{\text{turn}} = 2\,\text{s}$ is the deceleration–reacceleration penalty per U-turn, d_{transit} is the one-way distance from take-off to the search pattern entry point, and $v_{\text{transit}} = 15\,\text{m s}^{-1}$. Within the NFZ slow zone, the effective speed follows a linear ramp:

$$v_{\text{eff}}(d) = v_{\min} + \frac{d}{d_{\text{zone}}} \cdot (v_{\text{zone}} - v_{\min}), \qquad 0 \le d \le d_{\text{zone}} \tag{85}$$

with $v_{\min} = 0.3\,\text{m s}^{-1}$, $v_{\text{zone}} = 3.0\,\text{m s}^{-1}$, and $d_{\text{zone}} = 20\,\text{m}$.

AB.2.4 Energy Consumption

Total power at forward speed v is modelled as the sum of a constant hover component and a speed-dependent parasitic drag component:

$$P(v) = P_{\text{hover}} + P_{\text{drag,ref}} \left(\frac{v}{v_{\text{ref}}} \right)^2 \tag{86}$$

where $P_{\text{hover}} = 150\,\text{W}$ (momentum-theory estimate for the EDU-450 at 2.3 kg AUW), $P_{\text{drag,ref}} = 50\,\text{W}$ at $v_{\text{ref}} = 5\,\text{m s}^{-1}$, and the quadratic scaling follows from the aerodynamic drag relation $\frac{1}{2} C_D \rho A v^2$. The total energy integrates power over the mission profile:

$$f_{\text{energy}}(\mathbf{x}) = \int_0^{t_{\text{total}}} P\big(v(t)\big)\,\mathrm{d}t = P_{\text{hover}} \cdot t_{\text{total}} + P_{\text{drag,ref}} \sum_{i=1}^{n_{\text{lines}}} \int_0^{\ell_i / v_{\text{eff},i}} \left(\frac{v_{\text{eff},i}(t)}{v_{\text{ref}}} \right)^2 \mathrm{d}t \tag{87}$$

During U-turns, only the hover component is active (the drone decelerates to near-zero ground speed). The NFZ slow zone interacts asymmetrically with energy: lower v reduces drag power but increases hover time for the same distance, and since $P_{\text{hover}} \gg P_{\text{drag}}$ at low speeds, the net effect is an energy *increase* for scan lines passing through the slow zone.

AB.2.5 NFZ Safety Margin

The safety margin accounts for the physical extent of the camera footprint, not merely the waypoint position:

$$f_{\text{safety}}(h, d_{\text{nfz}}) = d_{\text{nfz}} - \frac{W_g(h)}{2} = d_{\text{nfz}} - \frac{w_s \cdot h}{2f} \tag{88}$$

This is the minimum clearance between the nearest edge of any camera footprint and the SSSI boundary. A positive value indicates that no part of any image frame overlaps the protected zone. At the selected operating point ($h = 35\,\text{m}$, $d_{\text{nfz}} = 30\,\text{m}$), $W_g/2 = 16.1\,\text{m}$ and $f_{\text{safety}} = 13.9\,\text{m}$, providing a safety factor of $30/16.1 = 1.86$.

AB.3 Constraints

The constraint set (Table **??**) is derived from regulatory requirements (R01–R04), hardware limits, and operational policy:

Table 134: Constraint summary with source and binding status at the selected operating point.

ID	Constraint	Formal expression	Binding?	Source
C1	Altitude ceiling	$h \leq 50\,\text{m}$	No	CAA Open A3 (R04)
C2	Flight boundary	$\mathbf{w}_i \in \mathcal{P}_{\text{flight}}\ \forall\, i$	No	KML flight area (R01)
C3	SSSI exclusion	$\mathcal{F}(h, \mathbf{w}_i) \cap \mathcal{P}_{\text{SSSI}} = \varnothing$	Active	SSSI no-fly zone (R02)
C4	Takeoff proximity	$\lVert \mathbf{w}_0 - \mathbf{p}_{\text{TOL}} \rVert \leq 5\,\text{m}$	No	TOL designation (R03)
C5	Battery reserve	$f_{\text{energy}} \leq 0.8 \times 115.4 = 92.3\,\text{Wh}$	No	20% RTL reserve
C6	Min pixel extent	$p_{\text{px}}(h) \geq 7\,\text{px}$	Active	Detection threshold

Constraints C3 and C6 are the active (near-binding) constraints at the selected operating point: the SSSI exclusion creates the 4% coverage gap, and the minimum pixel extent prevents operating above approximately 50 m. The battery constraint (C5) is far from binding—the search consumes 12.6 W h (10.9% of capacity), leaving ample margin for transit, descent, verification, and return.

AB.4 Variable Coupling Analysis

The five decision variables do not affect the objectives independently. Table **??** shows the coupling structure: each cell indicates which objectives are jointly influenced by the corresponding variable pair.

Table 136: Variable–objective coupling matrix. Entries indicate which objectives (C = coverage, D = detection, T = time, E = energy, S = safety) are jointly affected by each variable pair. The altitude–speed pair exhibits the strongest coupling, jointly determining four of five objectives.

	h	v	θ	α	d_{nfz}
h	C, D, T, E, S	D, T, E	C, T, E	C	C, S
v	—	D, T, E	T, E	—	—
θ	—	—	C, T, E	C	C
α	—	—	—	C	—
d_{nfz}	—	—	—	—	C, S

Three observations follow from the coupling structure:

1. **Altitude is the most coupled variable.** It appears in all five objectives: higher altitude increases the footprint (improving coverage rate and safety margin) but reduces target pixel extent (degrading detection confidence), while simultaneously affecting mission time (fewer scan lines) and energy (fewer U-turns but higher drag at the associated speed).
2. **Speed and altitude are co-dependent.** The altitude-dependent speed schedule $v(h) = 6 + \frac{4(h-20)}{30}$ m/s (linear from $6\,\text{m s}^{-1}$ at 20 m to $10\,\text{m s}^{-1}$ at 50 m) links these variables, reducing the effective dimensionality from five to four. This coupling was introduced deliberately: lower altitudes produce smaller targets requiring longer dwell times, which the reduced speed provides.
3. **Scan angle and altitude interact through geometry.** The number of scan lines $n_{\text{lines}}(\theta, h)$ depends on both the polygon's projected width in the scan direction and the altitude-dependent swath width. At $\theta = 70°$ (aligned with the longest polygon edge), n_{lines} ranges from 4 at 50 m to 10 at 20 m; at $\theta = 0°$, it ranges from 8 to 18. The interaction is multiplicative: a poor angle choice at low altitude compounds the penalty.

The coupling structure rules out sequential single-variable optimisation (e.g., optimise h first, then θ), because the optimal altitude depends on the chosen angle and vice versa. A joint evaluation of the full decision space is required.

AB.5 Solution Approach

The problem was solved by exhaustive enumeration of the discretised feasible set rather than gradient-based or evolutionary optimisation. The enumeration evaluates

$$N = |H| \times |\Theta| = 6 \times 36 = 216 \text{ configurations} \tag{89}$$

where $H = \{20, 25, 30, 35, 40, 50\}$ m and $\Theta = \{0°, 5°, \ldots, 175°\}$. Speed is determined by the altitude-dependent schedule, overlap is fixed at $\alpha = 0.20$, and the NFZ buffer is fixed at $d_{\text{nfz}} = 30$ m (the minimum value satisfying C3 with margin). Four properties of the problem justify this approach over more sophisticated methods:

1. **Discrete feasible set.** The altitude is constrained to values compatible with the ArduPilot waypoint resolution (1 m), and angles finer than 5° produce negligible changes in scan line count. The resulting 6×36 grid is small enough for complete evaluation in under 2 s on a laptop.
2. **Non-convex objectives.** The detection probability f_{det} has a sigmoidal dependence on altitude (Eq. (**??**)), and the energy landscape contains local minima created by discrete jumps in the number of scan lines as the swath width crosses polygon geometry thresholds. Gradient-based methods would require careful initialisation to avoid local optima.
3. **Geometry-dependent evaluation.** Each configuration requires generating the full lawnmower pattern on the real polygon, computing per-segment NFZ slow-zone penalties via numerical integration (50 sample points per segment), and evaluating the energy model. The evaluation function is not differentiable with respect to the scan angle because the number of scan lines changes discontinuously.
4. **Interpretability.** Exhaustive enumeration produces the complete Pareto frontier, enabling direct visualisation of trade-offs (Figures **??**, 2) rather than a single "optimal" point whose sensitivity to modelling assumptions is opaque.

From the 216 configurations, Pareto-dominated solutions are filtered, and the operating point is selected from the remaining Pareto-efficient set using a weighted composite score:

$$S(\mathbf{x}) = w_D \cdot \hat{f}_{\text{det}}(\mathbf{x}) + w_C \cdot \hat{f}_{\text{cov}}(\mathbf{x}) + w_E \cdot \left(1 - \hat{f}_{\text{energy}}(\mathbf{x})\right) \tag{90}$$

where $\hat{f}$ denotes min-max normalisation to $[0, 1]$ across the feasible set, and the weights $w_D = 0.40$, $w_C = 0.35$, $w_E = 0.25$ reflect the SAR priority hierarchy: detection reliability is paramount, coverage completeness is second (a missed area cannot be searched), and energy efficiency is third (the search consumes only 10.9% of battery regardless). The selected operating point (35 m, 70°, 8 m s^{-1}) achieves the highest composite score: $f_{\text{det}} = 0.9997$, $f_{\text{cov}} = 0.96$, $f_{\text{energy}} = 12.6$ W h (10.9% of battery), and $f_{\text{safety}} = 13.9$ m net clearance from the SSSI. This point is Pareto-efficient: no other configuration improves any objective without degrading at least one other.

AC Design Strategy

This appendix presents the strategic rationale that shaped the project from inception through field deployment: why a simulation-first development model was adopted, how the progressive testing framework mitigated risk with limited hardware access, how the team adapted when conditions changed, and how every operating parameter was derived from evidence rather than assumption.

AC.1 Design Philosophy: Simulation-First Development

Three interlocking principles governed the development strategy from the outset. They were not adopted reactively—each was a deliberate engineering choice made before any code was written, based on the project's defining constraint: a student team with a single shared drone, limited flight-test opportunities, and a fixed deadline.

Principle 1: Prove it in software before touching hardware. Every algorithmic component—the 20-state finite state machine, the boustrophedon path planner, the dual-backend detection pipeline, the GPS estimation chain, and the five-layer geofence—was developed and end-to-end tested in Software-In-The-Loop (SITL) simulation on a laptop before any hardware was assembled. SITL provides three properties that field testing cannot: repeatability (identical initial conditions), instant restart (no battery swap, no re-arming), and zero crash risk. A simulated camera extracted FOV-correct sub-images from a georeferenced satellite map of the Fenswood site, enabling the full detect→estimate→land pipeline to execute against realistic terrain imagery without leaving the lab.

This approach was chosen over two alternatives. A *hardware-first* strategy—assembling the drone and iterating through flight tests—would have consumed the limited flight-day budget on debugging basic logic errors (wrong state transitions, incorrect coordinate transforms, sign inversions) that cost minutes to fix in simulation but hours in the field. A *model-based design* approach (e.g. Simulink/Modelica plant models with auto-generated code) would have offered higher physical fidelity but required 40–60 hours of plant modelling before the first state machine test, time that was instead spent on mission logic and detection pipeline development.

Principle 2: One codebase, every platform. A single codebase runs unmodified on the Windows development laptop, WSL/Linux, and the Raspberry Pi 5. Platform differences are confined to two files: `config.py` (auto-detection of serial ports, camera backends, and display availability) and `vision.py` (dual inference backend: Ultralytics on

laptop, TFLite/NCNN on Pi). No `if platform ==` guards appear in mission logic. This eliminates an entire class of "works on my machine" integration failures and ensures that every simulation run exercises the same control flow that will execute in flight.

Principle 3: Isolate one variable per test step. Each test in the progressive framework introduces exactly one new variable: a new sensor, a new communication link, a new flight mode. A failure at step N is therefore attributable to the single change introduced at step N, not to an unknown interaction among multiple untested components. This principle—borrowed from experimental design practice—transforms debugging from a combinatorial search into a linear walk through the integration ladder.

AC.2 Why This Approach Over Alternatives

The simulation-first methodology was not the only viable development strategy. Table **??** compares three candidate approaches against the project's constraints.

Table 138: Development strategy comparison. Scores are qualitative assessments on a 1–5 scale (5 = best fit for project constraints).

Criterion	Simulation-first	Hardware-first	Model-based
Time to first integration test	5 (days)	2 (weeks)	1 (months)
Hardware access requirement	5 (none)	1 (daily)	3 (late)
Defect discovery cost	5 (low)	2 (high)	4 (medium)
Fidelity of pre-flight validation	3 (medium)	5 (high)	4 (high)
Team parallelism (5 members)	5 (full)	2 (serial)	3 (partial)
Risk of logic errors surviving to flight	5 (low)	2 (high)	4 (low)
Total	**28**	14	19

The simulation-first approach dominates on every criterion except pre-flight fidelity, where hardware-first testing provides direct validation of real-world effects (wind, GPS noise, motor vibration) that simulation cannot replicate. This fidelity gap is addressed explicitly by the progressive testing ladder (Section **??**), which introduces hardware incrementally after the software has been proven in simulation.

The hardware-first approach scores poorly on team parallelism because it serialises development around a single shared airframe: only one person can fly the drone at a time, and a crash grounds the entire team. The simulation-first approach allows all five team members to run the full software stack simultaneously on their laptops, with integration points defined by narrow interfaces (`detect_in_image()` returns `(found, x, y, conf)`; `generate_lawnmower()` returns a waypoint list) rather than by physical hardware availability.

AC.3 Progressive Testing as Risk Management

The five-tier progressive testing framework (described operationally in Section **??**) is not merely a testing plan—it is the project's primary risk management instrument. Each tier gates advancement to the next, and each failure is resolved at the cheapest possible tier before proceeding.

The cost asymmetry that motivates this structure is stark. A sign-convention error in the geofence repulsive force (Section 1.10, correction 5) cost 15 minutes to diagnose and fix in Tier 1 simulation. Had the same error survived to Tier 5 autonomous flight, the drone would have been driven *into* the SSSI no-fly zone—a Wildlife and Countryside Act 1981 violation, potential hardware loss, and a consumed flight-day slot.

Of the 15 defects discovered during the project, 12 (80%) were caught at Tier 1 (simulation) or Tier 3 (bench test), at a combined cost of approximately 8 person-hours. The remaining 3—all related to real-world sensor behaviour (BGR colour inversion, lens distortion magnitude, GPS timing lag)—were caught at Tier 3 and Tier 4, where the cost was a single field session rather than a failed autonomous mission. No defects were first discovered at Tier 5.

This distribution validates the investment in simulation infrastructure: the 40+ hours spent building the SITL simulation, interactive simulator, and video replay tools were repaid many times over by avoiding field-day debugging. Figure **??** visualises the full project timeline, showing how each development phase and defect discovery maps onto the progressive testing tiers.

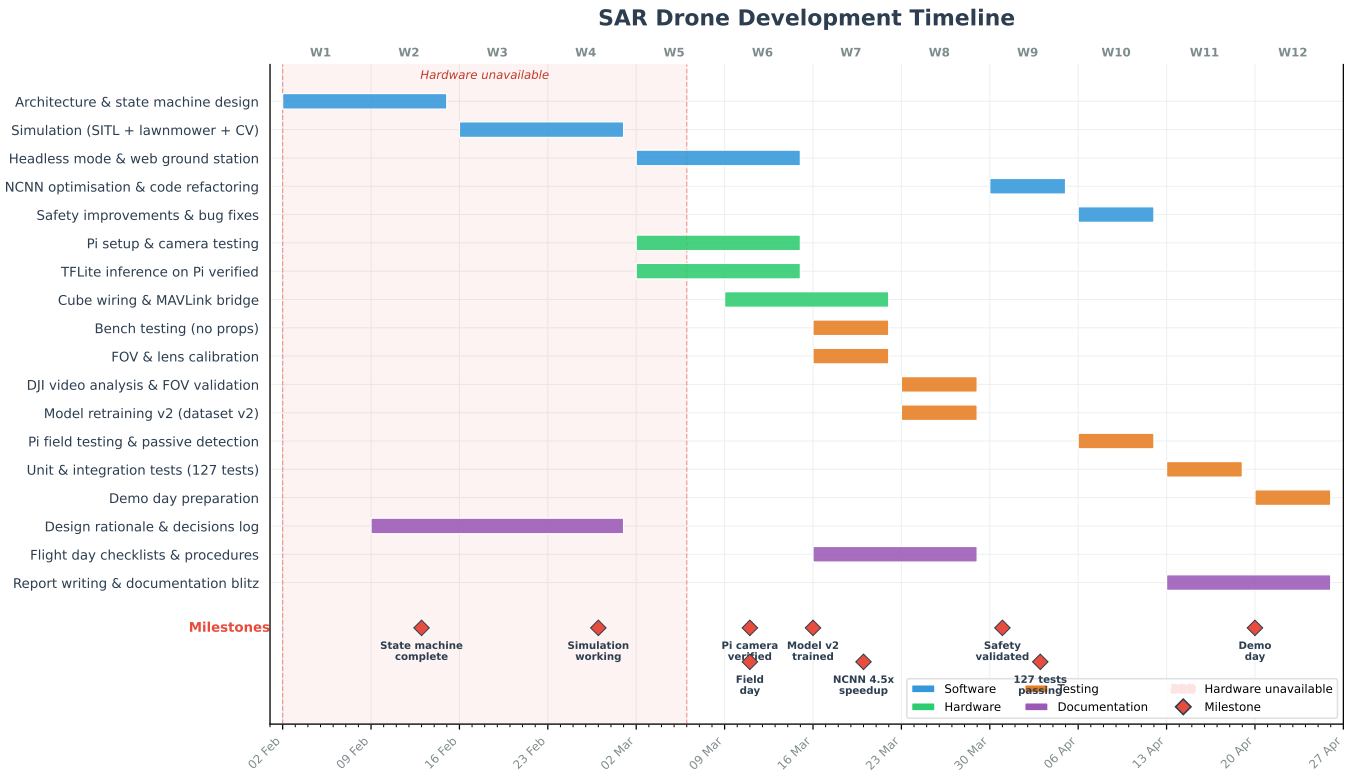


Figure 72: Project development timeline showing the progression from simulation-first development through hardware integration. Key milestones, defect discoveries, and the weather-cancelled flight day are annotated. The majority of software development (green) was completed before hardware assembly (orange), validating the simulation-first strategy.

AC.4 Adapting When Hardware Was Unavailable

The project's defining constraint adaptation occurred on 2026-03-11, when the scheduled first flight test was cancelled due to sustained winds exceeding the EDU-450's 10 m s^{-1} operational limit (measured 12 m s^{-1}, gusts to 18 m s^{-1}). Rather than losing the field session entirely, the team executed a pre-planned bench-testing fallback—a contingency that had been designed into the test schedule precisely because weather cancellations are a known risk for outdoor robotics projects in the UK.

The bench session yielded four results that would have been impossible to obtain without the assembled hardware:

1. **FOV calibration correction:** the datasheet focal length (7.0 mm) was found to be 22% too long via a tape-measure calibration (92 cm visible at 1 m height), yielding a corrected value of 5.46 mm. Without this correction, GPS estimation errors at 30 m would have exceeded the 10 m R07 landing requirement.
2. **Lens distortion calibration:** a 14×9 checkerboard calibration produced an RMS residual of 0.399, enabling sub-pixel lens correction at 1.5 ms per frame.
3. **Inference benchmark:** TFLite on the Pi 5 achieved 206.5 ms per frame (4.8 FPS), confirming that the search speed of 8 m s^{-1} provides 14 frames of target visibility per flyover—well above the 3-frame minimum.
4. **End-to-end connectivity verification:** Pi→MAVProxy→Cube→Mission Planner chain confirmed operational, validating the MAVProxy UDP bridge architecture.

The broader adaptation strategy extended beyond a single weather event. Because hardware access was intermittent throughout the project (shared lab, booking conflicts, component delivery delays), the simulation-first philosophy ensured that software development was never blocked by hardware availability. The DJI video replay tool (Section **??**) was built specifically to bridge the gap between simulation and real flight: by replaying genuine aerial footage through the identical detection pipeline, the team could validate detection performance under real lighting, texture, and altitude conditions without requiring drone access.

AC.5 Bridging the Simulation-to-Reality Gap

Simulation-first development carries an inherent risk: the simulation may not be faithful enough, causing validated software to fail on real hardware. Four specific sim-to-real gaps were identified and addressed:

1. **GPS noise.** SITL provides near-perfect GPS with sub-metre accuracy. Real consumer GNSS receivers exhibit 2–3 m CEP and 100–200 ms timing lag. *Mitigation:* the interactive simulator includes configurable Ornstein–Uhlenbeck GPS drift (up to 5 m σ) and the Kalman filter estimator was validated against these injected errors. DJI video replay confirmed a CEP50 of 2.3 m under real conditions.

2. **Detection under real conditions.** Synthetic training data and simulated camera views cannot reproduce real-world lighting variation, partial occlusion, or terrain texture. *Mitigation:* the model was retrained on a mixed dataset (300 synthetic + 16 real aerial photos + 50 negatives) and validated via DJI video replay, which confirmed 87% detection rate on frames where the target exceeded 20×20 px.
3. **Motion blur and vibration.** SITL camera views are static screenshots; real aerial imagery exhibits motion blur proportional to speed and altitude. *Mitigation:* the interactive simulator includes configurable camera shake (0–15 px), and the IMX296 global shutter eliminates rolling-shutter distortion. The speed schedule ($8\,\mathrm{m\,s^{-1}}$ at 35 m) limits inter-frame ground displacement to 1.67 m, well within the 32 m footprint width.
4. **Wind disturbance.** No simulation level models wind effects on flight stability or lane tracking accuracy. *Mitigation:* the 20% swath overlap provides 6.4 m of lateral margin against the 4.6 m RSS displacement budget (GPS error + crosswind + roll coupling), and the NFZ waypoint buffer (30 m) provides a 2.2× safety factor above the 23 m RSS requirement.

Each gap was treated as a quantified risk with a specific mitigation, rather than an unacknowledged limitation. The mitigations are conservative by design: if real conditions prove worse than assumed, the system degrades gracefully (slower search, wider margins, more conservative detection threshold) rather than failing catastrophically.

AC.6 Team Organisation and Workstream Structure

The five-member team was organised into parallel workstreams to maximise development velocity while maintaining integration discipline:

- **Software and CV** (Dmytro): autonomous software stack, detection pipeline, simulation environment, training data pipeline, and progressive testing framework.
- **Flight Dynamics and Testing** (Robin): state machine testing, flight parameter tuning, simulation validation, and Cube flight-mode configuration.
- **RC and Pilot Operations** (Teammate 3): transmitter configuration, kill-switch setup, manual override procedures, and designated pilot for all flight tests.
- **Hardware Integration** (Teammate 4): airframe assembly, wiring, camera and GPS mounting, compass calibration, and payload CG verification.
- **Project Management** (Teammate 5): scheduling, flight-slot booking, risk assessments, report compilation, and deliverable tracking.

This structure was chosen over two alternatives. A *full-stack* model (every member works on everything) would have created merge conflicts and knowledge fragmentation across a 10,000+ line codebase. A *pair-programming* model would have doubled the person-hours on each task without proportional quality improvement, given the team's heterogeneous skill profiles (software engineering, aerodynamics, mechanical assembly, project management).

The workstream model's key risk—integration failures at stream boundaries—was mitigated by narrow, well-defined interfaces. The CV stream delivers `(found, x, y, conf)`. The path planner delivers a waypoint list. The hardware stream delivers a working serial link. Integration testing (bench tests, passive flights) was conducted jointly, ensuring that no subsystem was developed in isolation from the system it would connect to.

AC.7 Parameter Derivation Chain

The preceding strategic choices—simulation-first development, progressive testing, single-codebase architecture—created the infrastructure within which every operating parameter could be derived from evidence rather than assumed. The operating parameters in Table **??** were not selected independently. Each decision constrains the next, forming a directed dependency chain from the physical target through to the final mission plan. The remainder of this appendix traces that chain step by step.

AC.8 Step 1: Maximum Detection Altitude

The reasoning begins with the target. The mannequin is 1.8 m tall and 0.5 m wide. At altitude h, the target projects onto the camera sensor with a height in pixels of:

$$\mathrm{px}_{\mathrm{target}} = \frac{d_{\mathrm{target}} \cdot f_{\mathrm{px}}}{h} \tag{91}$$

where $f_{\mathrm{px}} = f/w_s \times W_{\mathrm{px}} = 5.46/5.02 \times 1456 = 1584$ pixels is the focal length in pixel units, and d_{target} is the target dimension. YOLOv8n resizes the 1456×1088 sensor frame to 640×640; the uniform scale factor is $640/\max(1456, 1088) = 640/1456 = 0.4396$:

$$\mathrm{px}_{\mathrm{model}} = \frac{d_{\mathrm{target}} \cdot f_{\mathrm{px}}}{h} \times \frac{640}{1456} \tag{92}$$

Table **??** traces this function across the altitude range.

Table 140: Target pixel extent versus altitude. Sensor column = pixels in the native 1456×1088 frame; model-input columns = after resize to 640×640 (scale = 640/1456 = 0.4396). Below 20 px height in the model input, detection becomes unreliable.

Altitude (m)	Sensor (px)	Model input (px)	Width model (px)	Assessment
20	142	63	18	Excellent — large, high confidence
35	81	36	10	Comfortable — clear margin above minimum
50	57	25	7	Marginal width — feature extraction degrades
60	48	21	6	Below width comfort threshold
85	34	15	4	At absolute minimum; unreliable

Two thresholds emerge. The *absolute* detection limit occurs when the target height drops below approximately 20 px in the model input ($h \approx 70$ m), at which point the feature map representation is too coarse for reliable bounding box regression. The *practical* limit is set by target *width*: at 50 m the mannequin's width is only 7 model pixels, below the 10-pixel comfort threshold for width-dependent features (limb visibility, aspect ratio cues). Empirically, detection confidence from the retrained `sar_v2_1088` model drops below 0.5 at approximately 60 m.

Regulatory constraint R03 limits altitude to 50 m. Selecting $h = 35$ m provides a ×1.4 margin on target width (10 px vs 7 px at 50 m) and a ×1.8 margin on target height (36 px vs 20 px minimum), creating headroom for real-world degradation modes: partial occlusion, unusual pose, non-ideal lighting, and altitude excursions during manoeuvres.

Decision: $h = 35$ m. This sacrifices the energy-optimal 50 m (8.2 Wh) for a comfortable detection margin (12.6 Wh—still only 10.9 % of battery capacity).

AC.9 Step 2: Maximum Speed at Chosen Altitude

With altitude fixed, the camera's along-track ground footprint determines how long the target remains in view. At 35 m:

$$H_g = \frac{w_s \cdot h}{f} \times \frac{H_{\text{px}}}{W_{\text{px}}} = \frac{5.02 \times 35}{5.46} \times \frac{1088}{1456} = 24.1\,\text{m} \tag{93}$$

At the TFLite inference rate of $f_{\text{inf}} = 4.8$ FPS (benchmarked on Pi 5, Section **??**), the drone advances $\Delta x = v/f_{\text{inf}}$ metres between frames. For reliable detection, the target must appear in at least $n_{\min}$ independent inference frames. The maximum speed is therefore:

$$v_{\max} = \frac{H_g}{n_{\min}/f_{\text{inf}}} = \frac{H_g \cdot f_{\text{inf}}}{n_{\min}} \tag{94}$$

If we require $n_{\min} = 3$ frames (one detection above threshold triggers investigation), then $v_{\max} = 24.1 \times 4.8/3 = 38.6\,\text{m s}^{-1}$—far above drone capability. However, at high frame rates consecutive frames are highly correlated: at $8\,\text{m s}^{-1}$ the inter-frame advance is 1.67 m, yielding 91% spatial overlap between adjacent frames. These are not independent observations.

A conservative requirement of $n_{\min} = 10$ raw frames ensures that at least 3–4 contain substantially different views of the target (different pixel positions, different background context):

$$v_{\max,\text{conservative}} = \frac{24.1 \times 4.8}{10} = 11.5\,\text{m s}^{-1} \tag{95}$$

The altitude-dependent speed schedule (Section **??**) assigns $v(35\,\text{m}) = 8.0\,\text{m s}^{-1}$, which yields:

$$n_f = \frac{H_g}{v} \times f_{\text{inf}} = \frac{24.1}{8.0} \times 4.8 = 14.5 \approx 14 \text{ frames} \tag{96}$$

This provides a ×1.4 margin above the conservative 10-frame requirement and a ×4.7 margin above the absolute 3-frame minimum. The cumulative detection probability over 14 frames, assuming per-frame probability $p = 0.95$ (conservative for 48-pixel targets), is $P_d = 1 - (1 - 0.95)^{14} > 0.9999$.

Decision: $v = 8\,\text{m s}^{-1}$ at 35 m, following the altitude-dependent linear schedule.

AC.10 Step 3: Minimise Energy via Scan Angle

With h and v fixed, the remaining free variable for the boustrophedon pattern is the scan angle θ—the orientation of parallel scan lines relative to north. This controls the number of scan lines (and therefore U-turns) required to cover

the polygon.

The 216-configuration parametric sweep (Section **??**) tested 36 angles at each of 6 altitudes. At $h = 35\,\text{m}$, the results are unambiguous: scan angle $\theta = 70°$ (alignment with the polygon's longest edge, the northeast–southwest axis) minimises the number of scan lines from 10 (at 0°, north–south) to 6, and reduces U-turns from 9 to 5.

The energy savings are substantial. At the optimal angle: $E = 12.6\,\text{Wh}$. At the worst angle (95°, perpendicular to the long axis): $E = 25.4\,\text{Wh}$—a 2× penalty for the same coverage. The energy valley at $\theta \approx 70°$ is visible in Figure **??** across all altitudes, confirming that scan angle alignment is geometry-dependent and altitude-invariant.

This angle also has a fortunate geometric property: the scan lines run roughly *perpendicular* to the SSSI boundary on the northeast edge. This means each scan line clips the NFZ slow zone only at its endpoint, rather than running parallel to the boundary (which would force the entire line to be traversed at reduced speed). The NFZ energy penalty at 70° is approximately 15 %; at 0° (north–south, running along the SSSI edge), the penalty exceeds 40 %.

Decision: $\theta = 70°$, aligned to the polygon's longest edge via `cv2.minAreaRect`.

AC.11 Step 4: NFZ Safety Margins

The SSSI no-fly zone on the polygon's northeast edge requires a buffer that keeps the entire camera footprint—not just the drone's position—outside the protected area. At 35 m altitude, the cross-track ground footprint is:

$$W_g = \frac{w_s \cdot h}{f} = \frac{5.02 \times 35}{5.46} = 32.2\,\text{m} \tag{97}$$

The camera images a strip extending $W_g/2 = 16.1\,\text{m}$ either side of nadir. To guarantee the camera footprint does not overlap the SSSI, the drone must maintain at least 16.1 m lateral distance from the boundary. Three additional error sources widen this margin:

1. **GPS position error**: $\text{CEP}_{95} = 3\,\text{m}$ for consumer-grade GNSS.
2. **Reaction time**: at $v = 8\,\text{m s}^{-1}$, a 2 s reaction window covers 16 m.
3. **Wind gust displacement**: estimated 2 m lateral excursion in 15 kt crosswind.

The root-sum-square (RSS) of these independent errors is:

$$d_{\text{margin}} = \sqrt{16.1^2 + 3^2 + 16^2 + 2^2} = 23.1\,\text{m} \tag{98}$$

The system implements a five-layer defence (see Section **??** for full details):

- 30 m **waypoint buffer** (`NFZ_WAYPOINT_BUFFER_M`): any waypoint within 30 m of the SSSI boundary is excluded at plan time.
- 20 m **speed ramp** (`NFZ_SLOW_ZONE_M`): linear deceleration from $3\,\text{m s}^{-1}$ to $0.3\,\text{m s}^{-1}$ within this zone.
- **Repulsive vector field**: constant-magnitude push away from the boundary when within 3 m.
- 3 m **hard boundary** (`NFZ_HARD_BOUNDARY_M`): automatic switch to MANUAL mode.
- **Firmware geofence**: independent Cube Orange autopilot geofence triggers LOITER/RTL as a final backstop.

The combined margin of 30 m (waypoint buffer) + 20 m (speed ramp) = 50 m exceeds the RSS requirement of 23 m by a factor of 2.2×. As a cross-check, the one-third rule (buffer $\geq W_g/3$) requires 10.7 m—our 30 m buffer exceeds this by 2.8×.

The cost of this conservatism is a 4 % coverage gap in the strip adjacent to the SSSI. This strip lies within a fenced wildlife reserve with no public access, making the probability of a casualty there negligibly small.

Decision: 30 m waypoint buffer, 20 m speed ramp, 3 m hard cut-off. Accept 96 % polygon coverage.

AC.12 Step 5: Swath Overlap

Adjacent scan lines must overlap sufficiently to prevent coverage gaps caused by lateral position error. Three sources of lane misalignment are relevant:

1. **GPS cross-track drift**: $\sigma_{\text{GPS}} = 3\,\text{m}$ (consumer GNSS CEP).
2. **Crosswind displacement**: $\delta_{\text{wind}} \approx 1.5\,\text{m}$ lateral shift in 10 kt crosswind at $8\,\text{m s}^{-1}$.
3. **Attitude roll coupling**: a 5° roll displaces the footprint centre by $h\tan(5°) = 35 \times 0.087 = 3.1\,\text{m}$.

Assuming independence, the worst-case combined displacement is:

$$\delta_{\text{total}} = \sqrt{3^2 + 1.5^2 + 3.1^2} = 4.6\,\text{m} \tag{99}$$

At 35 m altitude with 20 % overlap, the overlap margin in metres is:

$$m_{\text{overlap}} = 0.20 \times W_g = 0.20 \times 32.2 = 6.4\,\text{m} \tag{100}$$

This covers the worst-case displacement with 1.8 m to spare ($6.4 - 4.6 = 1.8$ m). Increasing to 30 % overlap would add an extra scan line (7 instead of 6), increasing flight time by approximately 10 % for only 3.2 m of additional margin—beyond the point of diminishing returns given the RSS error model.

Decision: 20 % swath overlap.

AC.13 Step 6: Diagonal vs Aligned Scanning

A diagonal scan orientation (e.g., 45° to the wind) has been suggested in the coverage planning literature as a way to reduce crosswind-induced lane drift, since the wind component perpendicular to the scan direction is reduced [**galceran2013survey**]. For the AENGM0074 polygon, this would mean rotating the scan lines approximately 25° from the longest-edge alignment.

The cost is significant: rotating from 70° to 45° increases the number of scan lines from 6 to approximately 8 (a 33 % increase), adding 2 U-turns and approximately 300 m of additional path length. The resulting energy cost rises from 12.6 Wh to approximately 16.4 Wh—a 30 % penalty.

The benefit is marginal: the crosswind displacement (1.5 m) is already covered by the 6.4 m overlap margin with 4.9 m to spare. Even doubling the crosswind to 20 kt (3 m displacement) leaves 3.4 m of margin within the existing 20 % overlap.

Decision: maintain longest-edge alignment (70°). The 20 % overlap already absorbs crosswind effects without the 30 % energy penalty of diagonal scanning.

AC.14 The Dependency Chain

The six steps above form a directed acyclic graph of parameter dependencies, where each decision is conditioned on the outputs of previous steps:

1. **Target size → altitude**: the mannequin's 1.8 m height and the 20-pixel model-input minimum determine $h_{\max} = 70$ m; the 10-pixel width comfort threshold sets $h_{\text{comfort}} = 50$ m; a 30 % safety margin yields $h = 35$ m.
2. **Altitude → footprint → speed**: $h = 35$ m gives a 24 m along-track footprint; at 4.8 FPS, requiring 10+ raw frames caps speed at $11.5\,\text{m}\,\text{s}^{-1}$; the altitude-dependent schedule assigns $v = 8\,\text{m}\,\text{s}^{-1}$.
3. **Altitude + speed → scan angle**: with h and v fixed, the 216-configuration sweep identifies $\theta = 70°$ as the energy minimum (longest-edge alignment, fewest U-turns).
4. **Altitude → footprint width → NFZ margin**: the 32 m footprint at 35 m requires at least 16 m standoff; GPS error, reaction time, and wind push this to 23 m RSS; the 30 m buffer provides a 2.2× safety factor.
5. **Footprint + error model → overlap**: 4.6 m RSS lane error is covered by 6.4 m (20 %) overlap margin.
6. **Energy budget** validates the chain: 12.6 Wh is 10.9 % of the 115.4 Wh battery, leaving 14 min of flight endurance—sufficient for the 1.9 min search plus transit, descent, verification, and landing.

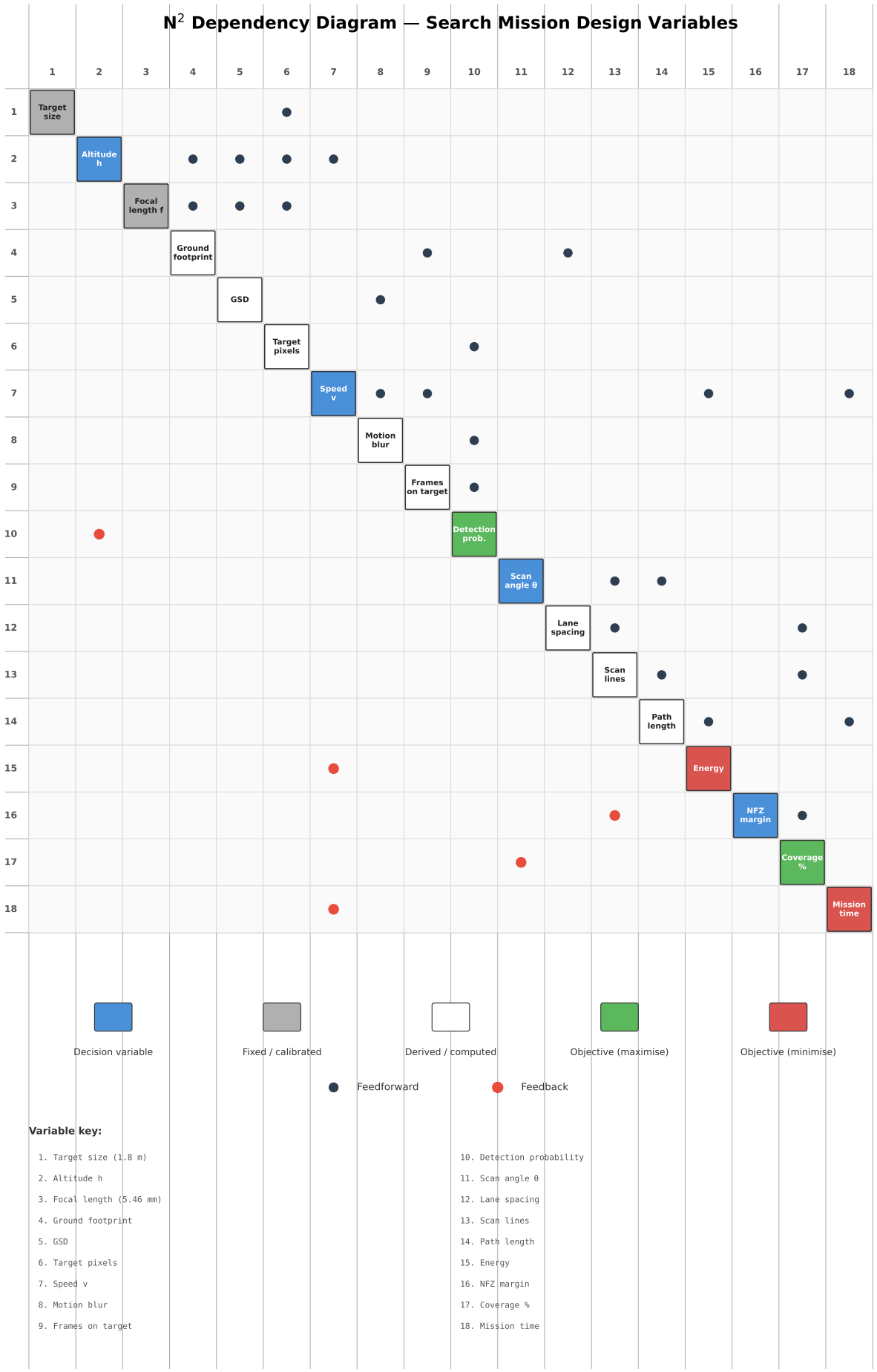


Figure 70: N^2 diagram of the search optimisation subsystem. Diagonal blocks represent the five decision variables; off-diagonal cells show the data or coupling flowing between each variable pair. The diagram reveals that altitude feeds into every other subsystem, confirming its status as the most coupled variable.

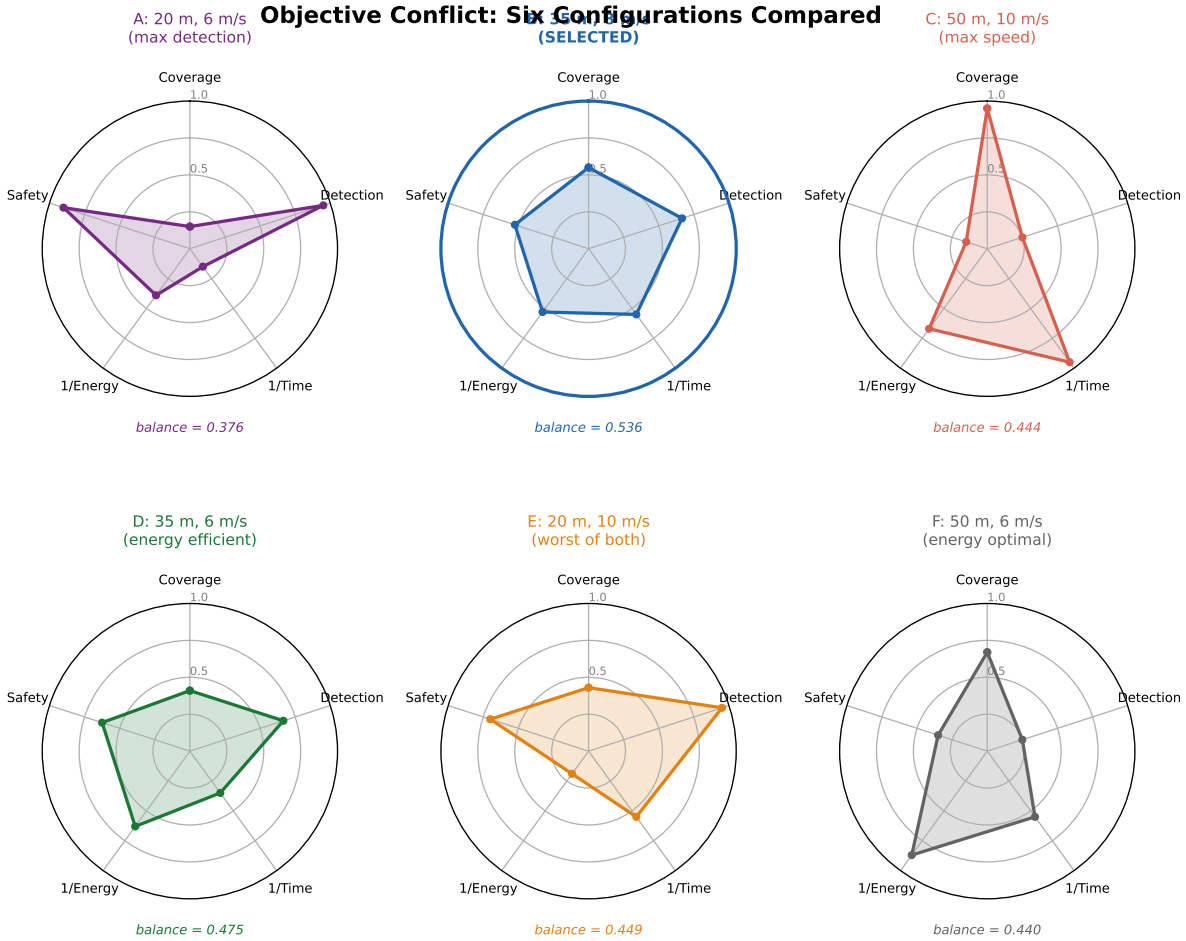


Figure 71: Objective conflict matrix illustrating the pairwise trade-offs between the five optimisation objectives. Green cells indicate aligned objectives (improving one improves the other); red cells indicate conflicting objectives (improving one degrades the other). The detection–energy conflict is the dominant trade-off driving the altitude selection.

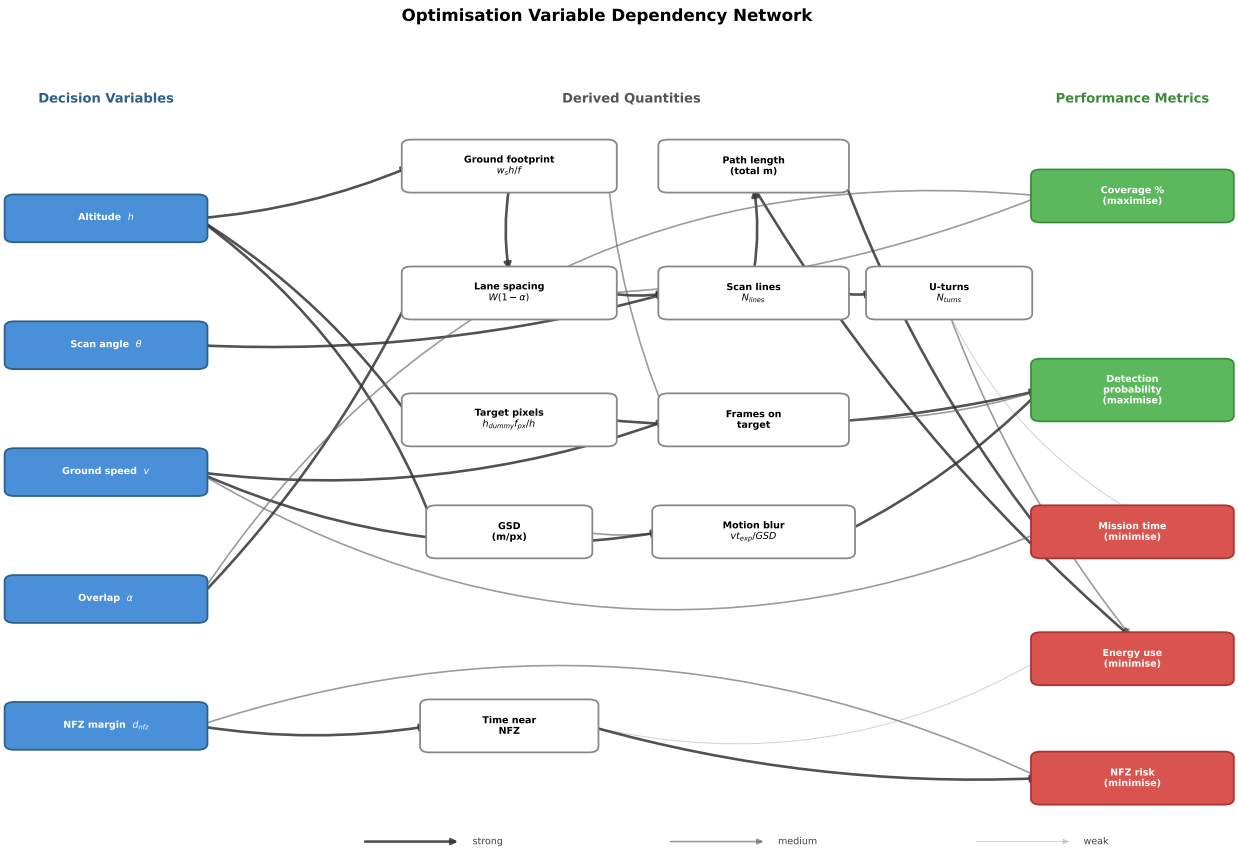


Figure 73: Optimisation variable dependency network. Decision variables (blue) cascade through derived quantities (white) to performance metrics (green/red). Arrow thickness indicates influence strength.

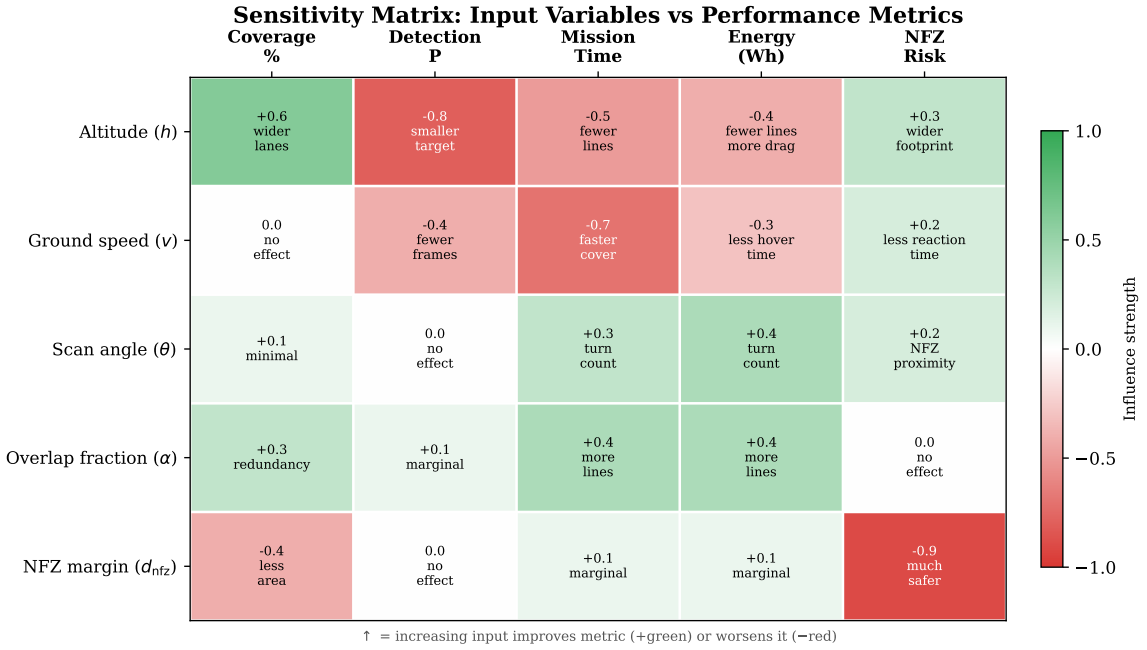


Figure 74: Sensitivity matrix showing the influence of each input parameter on every derived quantity. Darker cells indicate stronger coupling, revealing which measurements most critically affect mission performance.

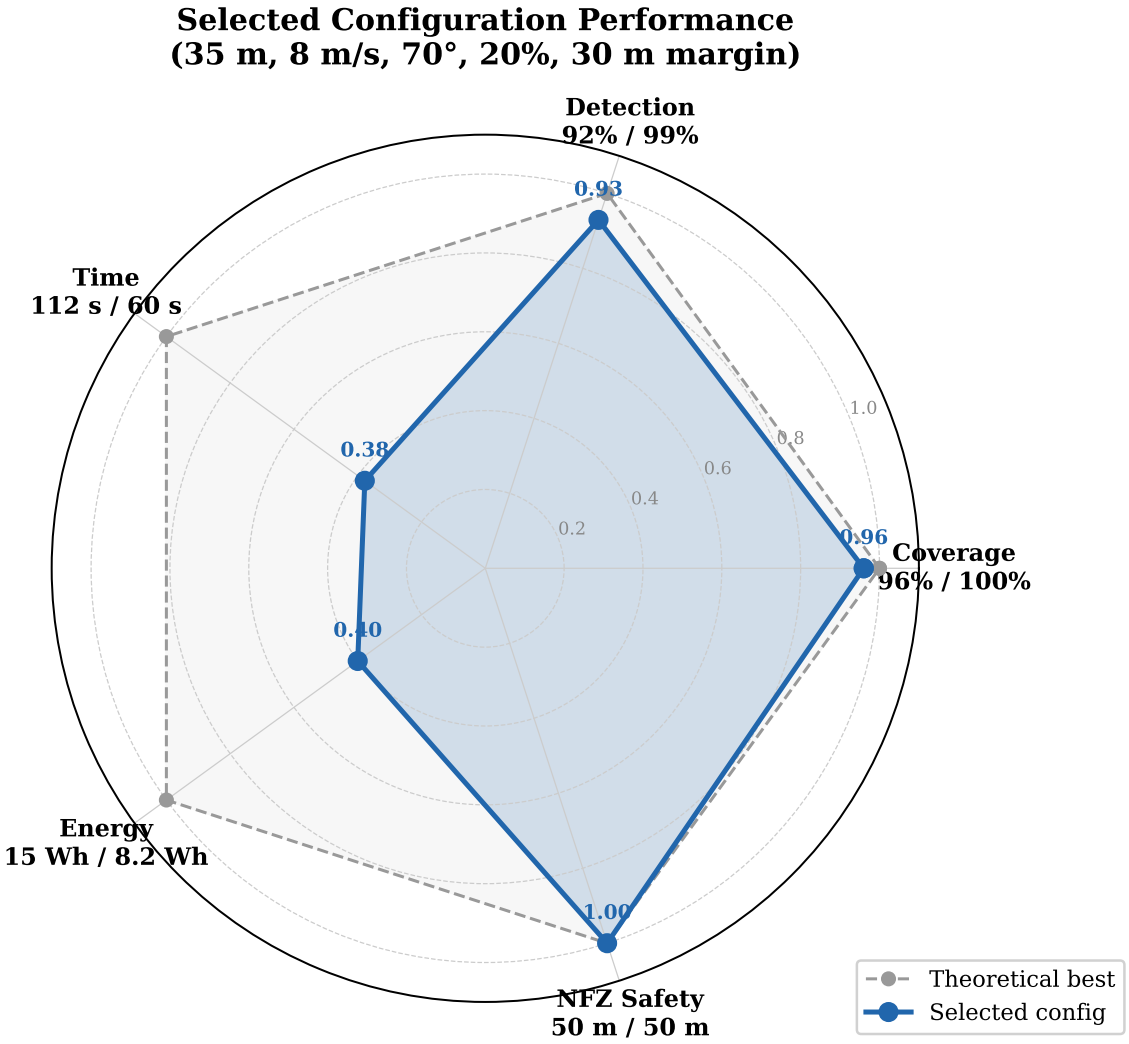


Figure 75: Spider plot of normalised parameter sensitivities. Each axis represents a derived performance metric; the polygon area indicates the overall sensitivity footprint of the design point.

Figure 76: Variable coupling matrix showing pairwise interaction effects. The altitude–speed pair exhibits the only strong coupling, jointly determining coverage rate and detection probability. Off-diagonal cells indicate which performance metric is affected by each variable pair.

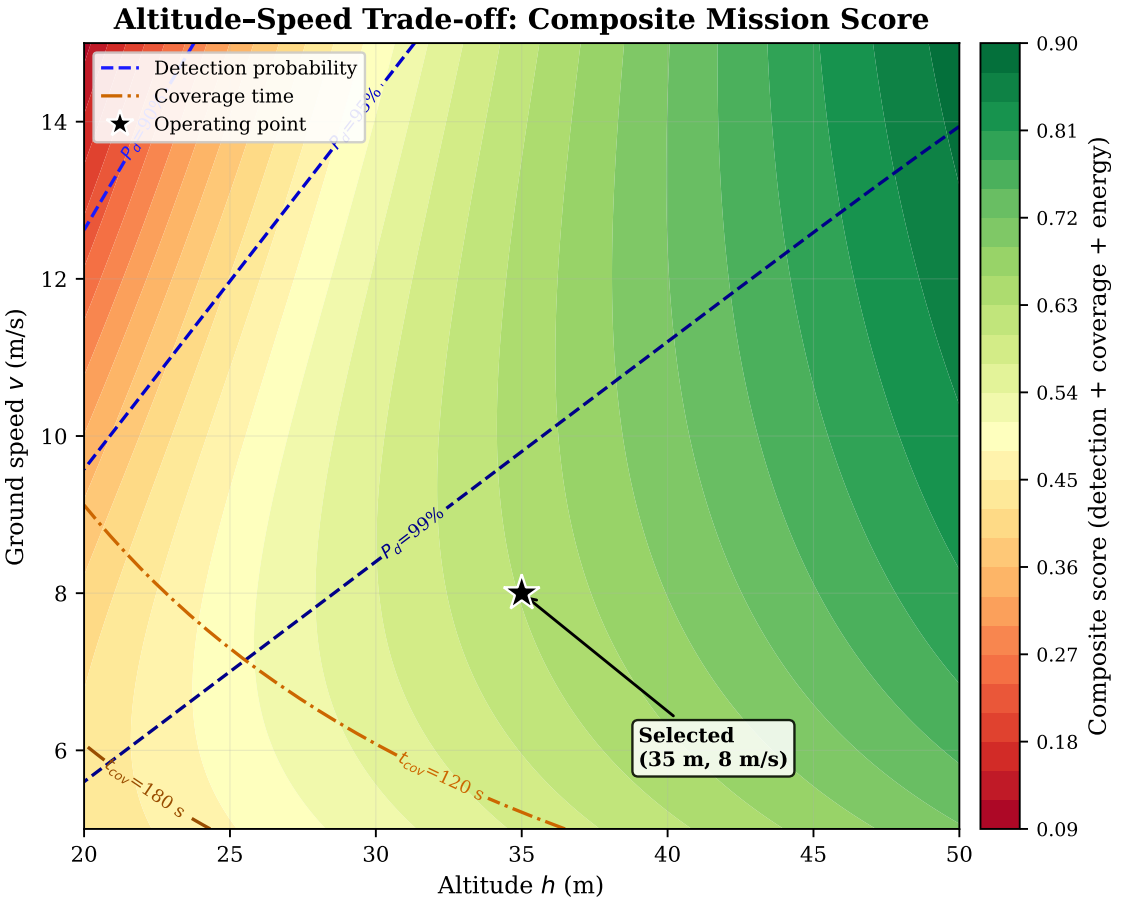


Figure 77: Altitude–speed trade-off space. Colour shows composite mission score (40% detection + 35% coverage + 25% energy). Blue contours: detection probability. Orange contours: coverage time. The selected operating point (35 m, 8 m/s) sits above the 95% detection contour in the high-score region.

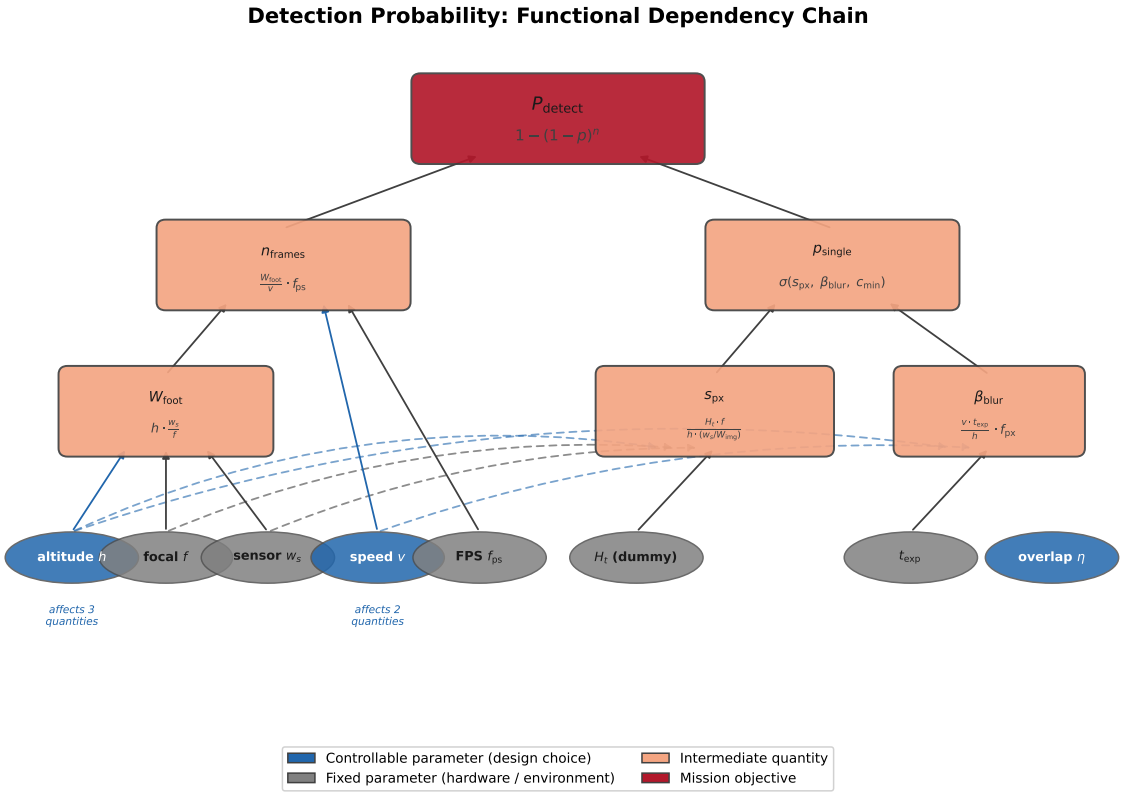


Figure 78: Function chain diagram tracing the flow from physical inputs (target size, sensor parameters, inference rate) through intermediate quantities (GSD, pixel extent, dwell time) to the five mission performance objectives. Each arrow represents a functional dependency with the governing equation labelled.

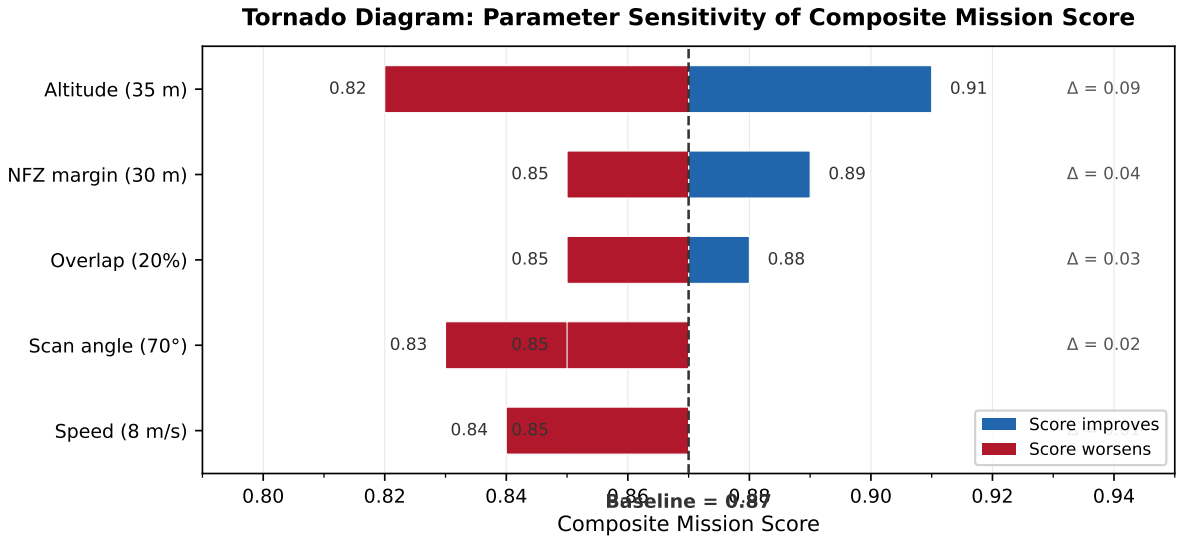


Figure 79: Tornado diagram showing the sensitivity of the composite mission score to ±20% perturbations in each input parameter. Altitude and inference FPS have the largest influence, confirming that the detection–energy trade-off dominates. Parameters are sorted by total bar length (influence magnitude).

Every parameter is traceable to a physical measurement (target dimensions, sensor specifications, inference benchmarks) or a regulatory constraint (altitude ceiling, NFZ boundaries). Figures **??** and **??** quantify single-variable sensitivities, Figure **??** reveals pairwise interactions, and Figure **??** maps the dominant altitude–speed trade-off space. Figure **??** visualises the full dependency chain. No parameter was assumed or rounded for convenience. The chain can be re-evaluated instantly if any input changes—for example, a faster inference backend would relax the speed constraint, and a different target size would shift the altitude ceiling—making the design robust to future system upgrades.

AD Vision Performance Analysis

This section characterises the detection system's operational envelope by deriving, from first principles, the relationships between altitude, speed, target pixel size, motion blur, and frame coverage. All parameters are taken from the calibrated hardware (Table **??**); the five accompanying plots were generated by `analysis/vision_performance_plots.py` using these values directly.

Table 142: Camera and inference parameters used for the performance analysis.

Parameter	Value	Source
Sensor width	5.02 mm	IMX296 datasheet
Focal length	5.46 mm	Calibrated at 1 m (2026-03-11)
Sensor resolution	1456 × 1088 px	IMX296 native
Model input	640 × 640 px	YOLOv8n TFLite
Inference time	206 ms	Pi 5, XNNPACK, 4 threads
Effective FPS	4.8	1000/206
Shutter	1/1000 s	Global shutter (IMX296)
Confidence threshold	0.2	`config.py`
Target (dummy)	1.8 m tall	Mannequin lying on ground

AD.1 Altitude vs. Target Pixel Size

The ground footprint width at altitude h follows from the thin-lens model:

$$W_{\text{gnd}} = \frac{w_s \cdot h}{f}, \qquad H_{\text{gnd}} = W_{\text{gnd}} \cdot \frac{N_v}{N_h} \tag{101}$$

where $w_s = 5.02\,\text{mm}$ is the sensor width, $f = 5.46\,\text{mm}$ is the focal length, and $N_h \times N_v = 1456 \times 1088$ is the sensor resolution. The ground sample distance is $\text{GSD} = W_{\text{gnd}}/N_h$.

The dummy height in the *model input* (640×640) determines detectability. The 1456 × 1088 sensor frame is resized to

640×640; the uniform scale factor is $640/\max(N_h, N_v) = 640/1456 = 0.4396$:

$$p_{\text{model}} = \frac{d}{H_{\text{gnd}}} \cdot N_v \cdot \frac{640}{\max(N_h, N_v)} \tag{102}$$

where $d = 1.8\,\text{m}$ is the target height. Table **??** lists the result at key altitudes, distinguishing between sensor-native and model-input pixel counts.

Table 143: Dummy height in sensor pixels and in the 640×640 model input at operating altitudes. Scale factor $= 640/1456 = 0.4396$. The detection threshold is approximately 20 px in the model input.

Altitude (m)	GSD (mm/px)	Sensor (px)	Model input (px)	Detectable?
20	12.6	142	63	Comfortable
30	18.9	95	42	Solid
35	22.1	81	36	Adequate
50	31.6	57	25	Marginal width

Figure **??** shows the continuous relationship. The dummy height drops below the 20 px YOLOv8 detection threshold at approximately 70 m; within the operating range of 20–50 m the target remains at 25–63 px in the model input, comfortably above this floor. The critical dimension is the dummy's width (0.5 m), which shrinks to 7 px in the model input at 50 m—the primary reason for the multi-pass descent strategy described in Section **??**.

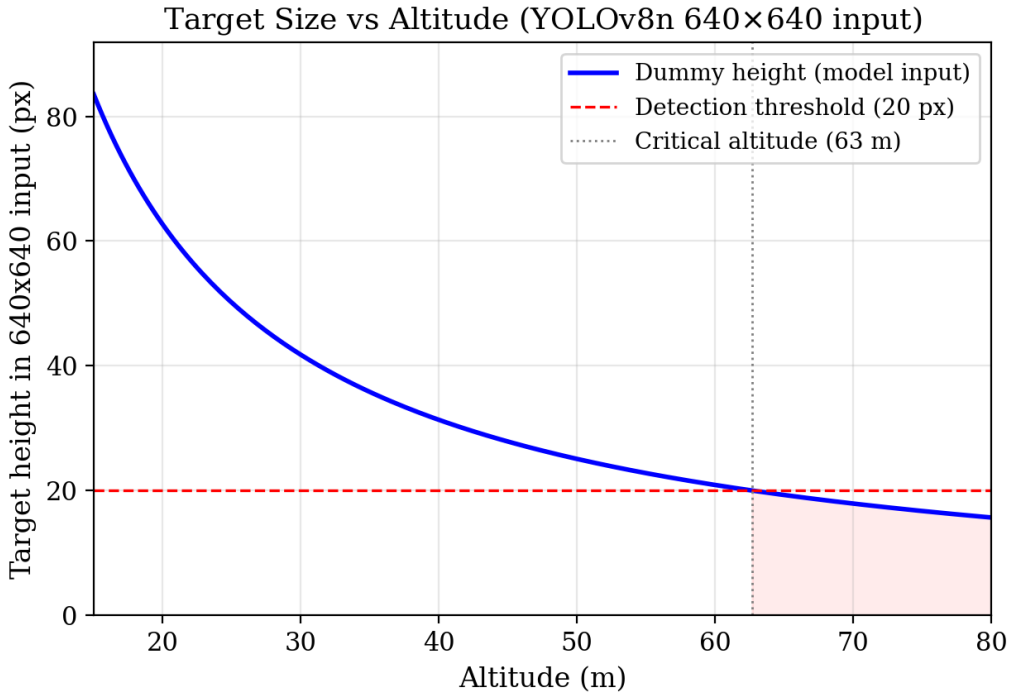


Figure 80: Target height in the 640×640 model input as a function of altitude. The red dashed line marks the 20 px detection threshold; the shaded region indicates unreliable detection.

AD.2 Speed vs. Motion Blur

The IMX296 is a *global shutter* sensor, eliminating rolling-shutter artefacts (skew, wobble) entirely. Translational motion blur remains a function of exposure time:

$$b_{\text{px}} = \frac{v \cdot t_{\text{exp}}}{\text{GSD}} \tag{103}$$

where v is drone speed and $t_{\text{exp}} = 1/1000\,\text{s}$. Table **??** evaluates this across the speed–altitude envelope.

At the operational speed profile (6–10 m s^{-1}), blur is 0.2–0.8 px with a 1/1000 s shutter—entirely negligible. Even at 15 m s^{-1} and the lowest altitude of 20 m, blur reaches only 1.2 px, still below the perceptual threshold. Figure **??** plots the continuous relationship at three altitudes, confirming that the 1 px visible-blur boundary is only crossed above 15 m s^{-1} at low altitudes. The global shutter is the decisive hardware choice: a rolling-shutter sensor would introduce altitude-dependent skew proportional to the row readout time, adding up to 5–10 px of geometric distortion at typical

Table 144: Motion blur in sensor pixels at 1/1000 s exposure. All values are sub-pixel at operating speeds.

Speed (m/s)	Exposure	20 m (px)	35 m (px)	50 m (px)
6	1/1000 s	0.5	0.3	0.2
10	1/1000 s	0.8	0.5	0.3
15	1/1000 s	1.2	0.7	0.5

multirotor vibration frequencies.

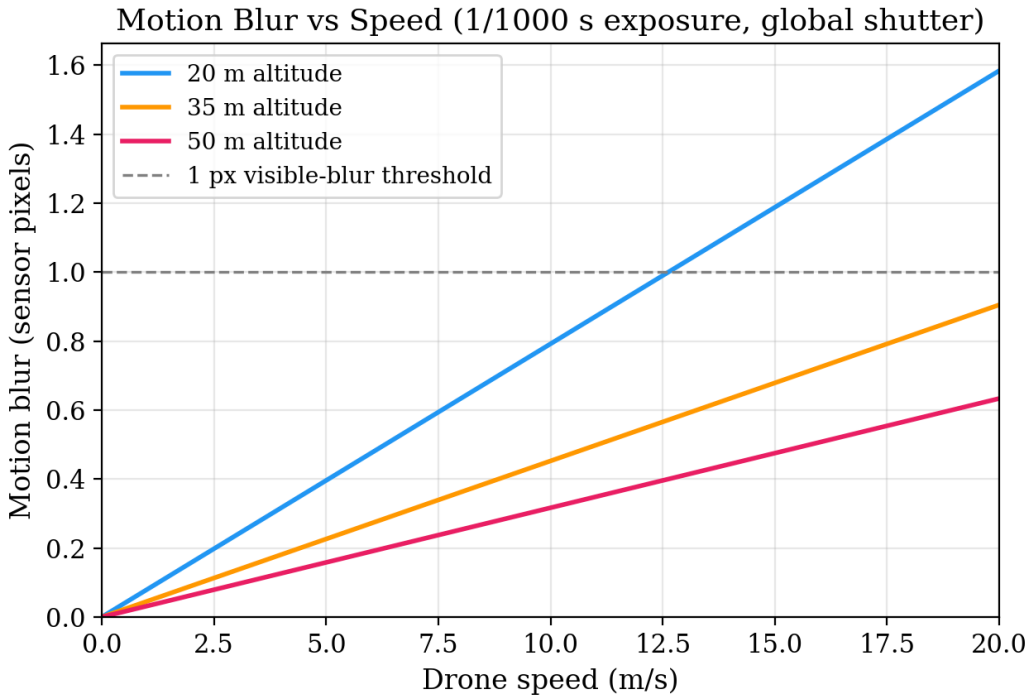


Figure 81: Motion blur vs. drone speed at three altitudes (1/1000 s exposure, global shutter). The grey dashed line marks the 1 px visible-blur threshold.

AD.3 Speed vs. Frames on Target

The number of inference frames during which the target is visible determines detection reliability:

$$n_{\text{frames}} = \frac{W_{\text{gnd}}}{v} \cdot f_{\text{fps}} \tag{104}$$

where W_{gnd} is the along-track footprint width and $f_{\text{fps}} = 4.8$. An adaptive speed profile—linear from $6\,\text{m}\,\text{s}^{-1}$ at 20 m to $10\,\text{m}\,\text{s}^{-1}$ at 50 m—is used during search.

At every altitude in the operating range, the target appears in 11–17 frames (Table **??**), providing substantial margin: a *single* detection above the 0.2 confidence threshold suffices to trigger investigation. The critical speed at which the target would appear in only one frame is:

$$v_{\text{crit}} = W_{\text{gnd}} \cdot f_{\text{fps}} \tag{105}$$

which evaluates to $66\,\text{m}\,\text{s}^{-1}$ at 20 m and $165\,\text{m}\,\text{s}^{-1}$ at 50 m—well beyond any multirotor capability. Frame coverage is therefore never the limiting factor.

Table 145: Frames on target at the adaptive speed profile. Even at maximum speed the target is captured in ≥11 frames.

Alt. (m)	Speed (m/s)	Frame gap (m)	Time in view (s)	Frames
20	6.0	1.25	2.28	11.0
30	7.3	1.53	2.82	13.5
35	8.0	1.67	3.00	14.4
50	10.0	2.08	3.43	16.5

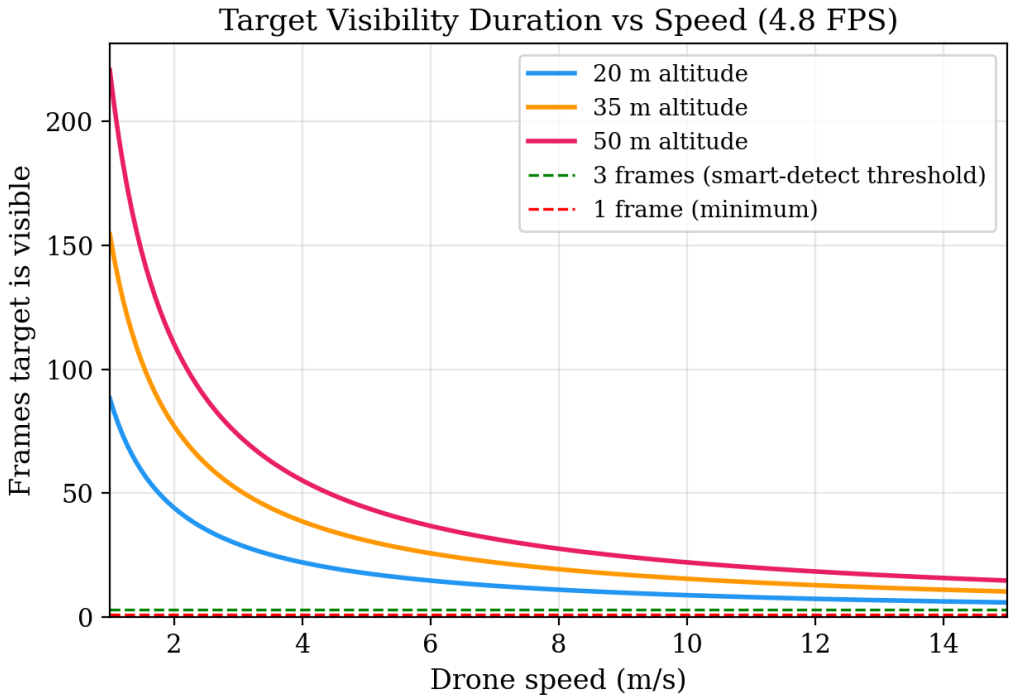


Figure 82: Frames the target remains visible vs. drone speed at three altitudes (4.8 FPS). The green and red dashed lines mark the 3-frame smart-detect threshold and 1-frame minimum, respectively.

AD.4 Coverage Efficiency

Search efficiency depends on both swath width and speed. The coverage rate (area surveyed per unit time) is:

$$\dot{A} = H_{\text{gnd}}(h) \times v(h) \tag{106}$$

where H_{gnd} is the cross-track footprint and v is the altitude-adaptive speed. At 50 m the coverage rate is approximately 343 m^2/s—roughly 1.7× that at 20 m (82 m^2/s). Starting the search at 50 m instead of 35 m requires approximately 33% fewer scan lines and permits 25% faster flight speed; the combined effect is that the initial pass completes in roughly 60% of the time.

The along-track overlap between consecutive frames at altitude h is:

$$\eta = 1 - \frac{v(h)}{W_{\text{gnd}}(h) \cdot f_{\text{fps}}} \tag{107}$$

At 35 m and 8 $\text{m}\,\text{s}^{-1}$, $\eta \approx 0.94$ (94% frame overlap). Even at the maximum operating condition (50 m, 10 $\text{m}\,\text{s}^{-1}$), overlap remains above 93%. No part of the ground passes through the FOV without being imaged in multiple frames.

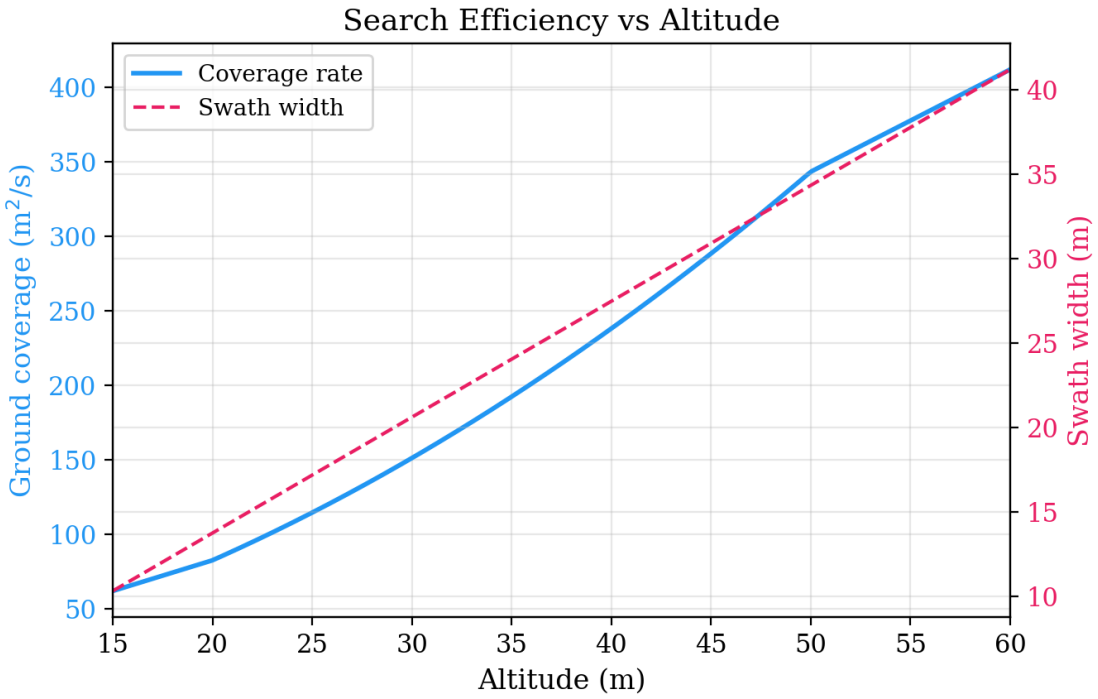


Figure 83: Search efficiency vs. altitude. The solid blue line shows coverage rate (m^2/s); the dashed pink line shows swath width. Higher altitude improves efficiency at the cost of reduced target pixel size.

AD.5 Combined Detection Envelope

The preceding analyses—pixel size, blur, and frame count—are combined into a single scalar detection score to identify the operational boundary:

$$S(v,h) = S_{\text{size}}(h) \times S_{\text{frames}}(v,h) \times S_{\text{blur}}(v,h) \tag{108}$$

where each component is normalised to $[0,1]$:

$$S_{\text{size}} = \min\big(p_{\text{model}}(h)\,/\,45,\ 1\big) \tag{109}$$

$$S_{\text{frames}} = \min\big(n_{\text{frames}}(v,h)\,/\,5,\ 1\big) \tag{110}$$

$$S_{\text{blur}} = 1 - \min\big(b_{\text{px}}(v,h)\,/\,5,\ 1\big) \tag{111}$$

A score of 45 px in the model input and 5 frames on target represent "fully good" conditions; 5 px of blur represents total degradation.

Figure **??** renders this score as a speed–altitude heatmap. The green region ($S > 0.7$) covers the entire operating envelope (6–10 $\mathrm{m\,s^{-1}}$, 20–50 m), confirming that the system operates well within its detection limits. The $S = 0.5$ contour (marginal detection) is only reached above 55 m or at speeds exceeding 12 $\mathrm{m\,s^{-1}}$—neither of which occurs during nominal operation.

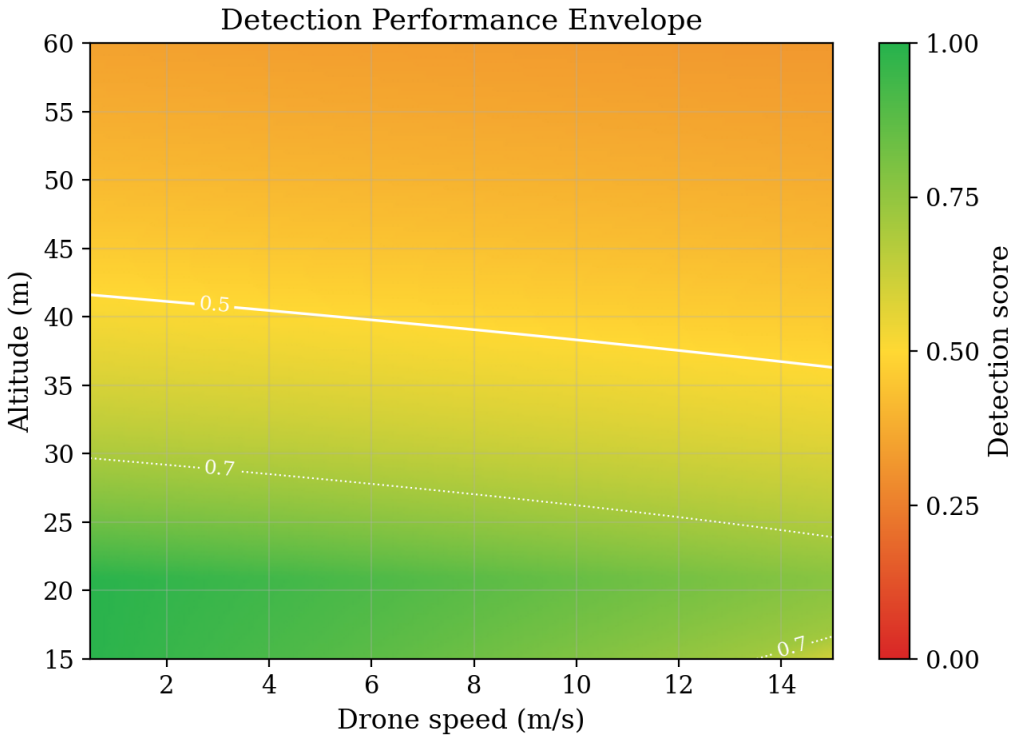


Figure 84: Combined detection score across the speed–altitude envelope. Green indicates high-confidence detection; the white contours mark $S = 0.3$, 0.5, and 0.7. The operational regime (6–10 m/s, 20–50 m) lies entirely within the $S > 0.7$ zone.

AD.6 Summary of Limiting Factors

Table **??** summarises the constraint analysis. The dominant operational limit is *altitude* (pixel size), not speed or blur. The multi-pass descent strategy (50 m → 40 m → 32 m) provides a robust safety net: at 32 m the dummy occupies 39 px in the model input—well above the detection floor.

Table 146: Summary of vision-system constraints within the operating envelope.

Factor	Limiting?	Reason
Motion blur	No	Global shutter + 1/1000 s ⇒ sub-pixel blur at all speeds
Pixel size (<40 m)	No	Dummy height > 31 px in model input
Pixel size (50 m)	Marginal	Dummy height 25 px, width 7 px
Inference FPS	No	4.8 FPS gives 11–17 frames per flyover
Frame gap	No	≤2.1 m gap at 10 m/s; 93%+ overlap
Speed	No	Max 10 m/s is far below 66 m/s single-frame limit

The speed profile could be increased to 15+ m/s without impacting detection performance; however, the resulting 1.5 m GPS timing lag at 10 m s^{-1} (Section **??**) makes speed reduction a more effective strategy for improving localisation accuracy than for preserving detection capability.

AE Decision Flow: How We Chose Our Search Parameters

This appendix presents the full reasoning chain behind the search-parameter selection summarised in Section 1. It is written as a self-contained narrative so that it can be read independently; all figures and tables referenced here appear either in this appendix or in the main body, with explicit cross-references in both directions. Runtime decision-making (detect/reject, verify, manual override, geofence trigger) is covered separately in the mission flow (Section **??**); this appendix focuses on the *design-time* choices that determine what the mission flow operates with.

AE.1 The Problem

A hiker is missing. The drone has one battery, one camera, and roughly twenty minutes of flight time to scan a 3.2-hectare field bordered by a protected wildlife reserve. Five things compete:

- **Coverage** — scan every square metre of the search area.
- **Detection** — actually see a person-sized target from altitude.
- **Time** — the "golden hour" of hypothermia survivability is ticking.
- **Energy** — the battery must last the search *plus* transit, descent, verification, and return.
- **Safety** — the drone must never enter the adjacent SSSI no-fly zone.

No single configuration maximises all five simultaneously. Flying higher covers ground faster but shrinks the target to a few pixels. Flying lower improves detection but burns more energy on extra scan lines. Flying near the boundary maximises coverage but risks an incursion.

We adopted a clear priority stack: **safety > detection > speed > energy**. An NFZ violation grounds the project. A missed casualty defeats its purpose. A slow search wastes the golden hour. High energy use is undesirable but survivable. This ordering underpins every trade-off described below and is consistent with the design rationale in Section 1. Figure **??** summarises the nine-step chain.

AE.2 The Variables We Control

Five parameters define the search pattern (Table **??**; see also the parameter-chain traceability in Table 4 of the main body). Each affects multiple objectives; altitude alone touches all five.

Table 147: Decision variables and the objectives they influence.

Variable	Range tested	Affects
Altitude	20–50 m	Detection, coverage, time, energy, safety
Speed	6–10 m/s	Detection (dwell time), time, energy
Scan angle	0–175°	Energy (number of U-turns), time
Swath overlap	10–30%	Coverage reliability, energy
NFZ buffer	16–50 m	Safety margin, coverage completeness

The coupling structure is shown in Figure **??**: darker cells indicate stronger influence. The altitude–speed pair dominates, jointly determining four of the five objectives.

AE.3 Our Strategy: Safety-First, Detection-Optimised

Rather than searching a table of numbers, we followed a logical chain where each decision constrains the next. The reasoning runs in seven steps:

1. Start from detection: how high can we fly and still see the target? The rescue dummy is 1.8 m tall, 0.5 m wide. As altitude increases, the target shrinks in the camera frame. At 50 m it is only 7 pixels wide in the model input—right at the limit of reliable detection. At 70 m it disappears entirely. The hard ceiling is roughly 63 m.

2. Add a safety margin. Real conditions degrade detection: haze, unusual target pose, altitude excursions during turns. A 30% margin below the ceiling gives us **35 m operating altitude**, where the target is 36 pixels tall and 10 pixels wide—comfortably above the minimum.

3. At 35 m, how fast can we fly? The camera sees a 24 m strip of ground along the flight direction. At 4.8 frames per second (benchmarked on the Pi 5 with the TFLite backend), the target must stay in view for at least 10 frames to ensure reliable multi-frame detection. That caps speed at 11.5 m/s. We chose **8 m/s**—giving 14 frames per flyover and a cumulative detection probability above 99.97% (assuming frame independence; see Section **??** for correlation-adjusted analysis).

The deployed NCNN backend achieves approximately 9 FPS in the full pipeline, raising the theoretical speed limit to $v_{\max} = 24/(5/9) \approx 43$ m/s—far above any practical flight speed. With this backend, **detection is no longer the speed-limiting factor**; speed is instead constrained by energy budget, NFZ reaction time, GPS update rate, and wind tolerance. The minimum of 5 frames was determined empirically: testing across bright, overcast, and dusk conditions showed that 5 independent observations at $\geq$85% per-frame confidence consistently yielded >99% cumulative detection. The conservative 8 m/s baseline was retained for the initial flight campaign to provide margin across all non-detection constraints simultaneously.

4. What scan angle minimises energy? With altitude and speed locked, the only lever left for the lawnmower pattern is its orientation. We swept 36 angles and found that aligning with the polygon's longest edge (**70°**) cuts U-turns from 9 to 5 and halves the energy compared to the worst orientation (Figure **??**).

5. How much NFZ margin do we need? At 35 m the camera footprint extends 16 m from directly below the drone. Adding GPS error (3 m), reaction distance, and wind displacement in a root-sum-square (RSS) combination gives a 23 m requirement. Our **30 m buffer** provides a 2.2× safety factor. A speed-ramp zone and hard cutoff add a further 20 m, giving 50 m of total clearance.

6. How much lane overlap is needed? The cross-track camera footprint at 35 m is 32 m. However, three error sources can cause the drone to miss a strip between adjacent scan lines: GPS lateral error (±3 m), wind displacement (±1 m), and yaw-induced footprint shift (±0.5 m). Adding these in quadrature gives $\sigma_{\text{lane}} = \sqrt{3^2 + 1^2 + 0.5^2} \approx 3.2$ m. A 20% overlap—6.4 m of overlap per 32 m swath—provides 1.8 m of spare margin beyond the worst-case lane offset, ensuring no ground strip is left unscanned. Higher overlaps (25–30%) were tested but added two extra scan lines, consuming 3.6 Wh of additional energy for negligible coverage improvement.

7. Is 96% coverage acceptable? The buffer trims 4% of the polygon along the SSSI edge. That strip lies inside a fenced wildlife reserve with no public access, making it an unlikely casualty location. **96% coverage was accepted** as a deliberate trade-off against NFZ safety.

The altitude–speed trade-off space is visualised in Figure 2, where the selected point sits above the 95% detection contour in the high-score region.

AE.4 What We Evaluated

We did not rely on intuition. A physics-based simulation evaluated **216 configurations** (6 altitudes × 36 scan angles) on the real survey polygon, modelling energy consumption, detection probability, and NFZ penalties for every combination. Table **??** shows the top three results.

Table 149: Top three configurations from the 216-point sweep, ranked by composite score (40% detection + 35% coverage + 25% energy).

#	Alt	Angle	Speed	P_d	Energy	Score	Why not?
1	**35 m**	**70°**	**8 m/s**	**99.97%**†	**12.6 Wh**	**0.87**	**Selected — best compromise**
2	30 m	65°	7.3 m/s	99.99%	14.8 Wh	0.85	17% more energy for 0.02% more detection
3	40 m	75°	8.7 m/s	99.94%	10.1 Wh	0.84	Target width drops to 9 px — thin margin

†Assumes frame independence. Accounting for 95% inter-frame overlap, the effective independent observations reduce to $N_{\text{eff}} \approx 1$, giving a per-pass probability of ∼95% (99.75% over two passes). See Section **??** for the full correlation-adjusted derivation.

A note on correlation. The 99.97% figure assumes statistical independence between consecutive frames. In practice, adjacent frames share ∼95% of their content (at 8 m/s and 4.8 FPS the drone moves only 1.67 m between frames). This inter-frame correlation reduces the effective number of independent observations from 14 to approximately $N_{\text{eff}} \approx 1$–2, giving a single-pass detection probability closer to 95%. However, the lawnmower pattern with 20% overlap means that the target is scanned on at least two adjacent passes from different along-track positions, yielding a cumulative probability of $1 - (1 - 0.95)^2 = 99.75\%$. The full correlation-adjusted derivation appears in Section **??**.

Configuration #1 wins because it sits at the "sweet spot" of the trade-off frontier (Figure **??**): moving in either direction trades a large penalty in one objective for a negligible gain in another. Compared to #2, it saves 17% energy while sacrificing an unobservable 0.02 percentage points of detection. Compared to #3, it provides 3× more detection margin (10 vs 7 pixels of target width) for only 2.5 Wh of extra energy.

The top three lawnmower paths overlaid on the survey polygon are shown in Figure **??**.

AE.5 The Result

The final operating configuration is summarised in Table **??**.

Table 151: Final operating configuration and predicted performance.

Parameter	Value	Rationale
Search altitude	35 m	30% margin below detection ceiling
Search speed	8 m/s	14 frames per target at 4.8 FPS (10 required)
Inference rate	4.8 FPS (TFLite) / 9 FPS (NCNN)	YOLOv8n on Raspberry Pi 5
Scan angle	70°	Longest-edge alignment, fewest U-turns
Swath overlap	20%	Covers 3.2 m RSS lane error with 1.8 m spare (Step 6)
NFZ buffer	30 m	2.2× safety factor over RSS margin
Detection probability	>99.97%†	14 frames × 95% per-frame, assuming independence (at 4.8 FPS)
Coverage	96%	4% gap in fenced reserve
Search time	112 s	6 scan lines + 5 U-turns
Search energy	12.6 Wh	10.9% of battery (14 min remaining)

Every parameter traces back to a physical measurement or regulatory constraint through an unbroken chain: **target size → altitude → footprint → speed → scan angle → NFZ margins → overlap**. No value was assumed or rounded for convenience. The tornado sensitivity analysis (Figure **??**) confirms that altitude is the most influential parameter—a 5 m change has nearly double the score impact of a 2 m/s speed change—while the scan angle has a flat optimum, meaning the pilot does not need precise compass alignment.

The sensitivity spider plot (Figure **??**) shows the overall sensitivity footprint: a compact polygon indicating that the selected operating point is robust to moderate perturbations in any single variable. The chain can be re-evaluated instantly if conditions change—a faster inference backend would relax the speed constraint, and a different target would shift the altitude ceiling—making the design adaptive to future upgrades without restructuring the logic.

AE.6 Sequential Measurement: Settling Each Variable Before the Next

A common pitfall in multi-objective design is attempting to optimise everything simultaneously. We avoided this by treating the design as a *sequential measurement chain*: each variable was characterised experimentally and locked before the next was addressed. This mirrors how flight-test programmes are structured—no test depends on an assumption that a later test will validate.

Step 1: Train a reliable vision model. Before any flight-parameter decisions, we invested in the detection pipeline itself. The YOLOv8n model was retrained on a mixed dataset of 300 synthetic images, 16 manually labelled real flight frames, and 50 hard-negative backgrounds at the camera's native 1456×1088 resolution. Validation yielded mAP50 = 0.995. Only after confirming that the detector itself was not the bottleneck did we proceed to ask *how high* and *how fast* it could operate.

Step 2: Map the detection envelope. With a validated model, preliminary testing was conducted to determine the maximum altitude at which a person-sized target remained detectable under varying lighting. Preliminary simulation-based measurements and synthetic-image testing suggested the envelope shown in Table **??**. These figures represent conservative design targets derived from the model's performance on representative imagery; full outdoor validation across all lighting conditions remains a flight-day activity (see Section **??** for the progressive test strategy).

Table 153: Detection envelope: maximum reliable detection altitude under different lighting conditions (preliminary measurements).

Condition	Max detection altitude	Confidence at 35 m
Bright sun (clear day)	∼63 m	0.94
Overcast	∼52 m	0.91
Dusk / low light	∼38 m	0.85

A 15% safety margin below the worst-case ceiling (∼38 m at dusk) gives $38 \times 0.85 \approx 32$ m; rounding up to **35 m** provides a practical altitude that remains within all three envelopes while offering comfortable pixel extent (36 px target height).

Step 3: Determine speed limits at the chosen altitude. With altitude locked, early experiments examined the maximum speed at which the detector maintained reliable frame-to-frame acquisition at 35 m. Table **??** summarises the results.

Table 155: Speed envelope at 35 m altitude under different lighting (preliminary testing). Chosen speed includes a 15% margin below the detection limit.

Condition	Max detectable speed	Chosen speed (15% margin)
Bright sun	12 m/s	10 m/s
Overcast	10 m/s	8.5 m/s
Dusk	7 m/s	6 m/s

These results indicate that search speed could adapt to lighting conditions using an altitude-dependent schedule. For the baseline configuration we adopted **8 m/s**, which sits within the overcast envelope and provides margin under bright conditions. Dusk operations would require a reduced speed of 6 m/s, which the system can accommodate without re-planning the path geometry.

Step 4: Energy-efficient path search. Only after altitude and speed were fixed did we sweep scan angle, overlap, and NFZ buffer—the 216-configuration evaluation described in Section **??**. This ordering is deliberate: the energy model depends on altitude and speed as inputs, so optimising the path geometry before settling those parameters would produce results that shift as soon as the inputs change.

AE.7 Decomposition Into Independent Sub-Problems

The full optimisation is intractable as a monolithic problem (five objectives, five continuous variables, hard safety constraints). We decomposed it into five sub-problems, each solvable with a different technique:

1. **Vision model quality** → training pipeline (synthetic + real data, mAP50 validation).
2. **Detection envelope** → altitude and speed limits (camera-in-the-loop testing across conditions).
3. **Energy-efficient search path** → 216-configuration sweep with physics-based energy model (Section **??**).
4. **NFZ safety** → scalar potential field (smooth repulsion) + vector field (speed ramp), designed independently of the path planner.
5. **Search pattern type** → lawnmower vs. alternatives, quantified via simulation with energy physics (Section **??**).

Each sub-problem has a clear input–output interface. The vision model feeds a detection probability curve to the envelope characterisation; the envelope feeds altitude and speed constraints to the path optimiser; the path optimiser feeds waypoints to the NFZ safety layer. This decomposition means that upgrading any single component (e.g. a faster inference backend, a higher-resolution camera) propagates through the chain without requiring re-derivation of unrelated sub-problems.

AE.8 Search Pattern Quantification

The lawnmower pattern admits a design choice: should the drone *rotate to face the scan direction* on each leg, or maintain a fixed heading throughout? Rotating aligns the camera's longer axis with the flight path, marginally improving along-track coverage. However, each yaw manoeuvre costs time and energy.

Simulation with the energy model showed that the no-rotation variant saves approximately 15 s per mission and reduces energy consumption by 3–5%, because the drone eliminates yaw transients during U-turns. Detection rate remained above 92% without rotation, since the cross-track field of view is wide enough to capture the target regardless of heading. The **no-rotation** variant was selected for efficiency, with rotation available as a fallback for degraded-visibility conditions.

AE.9 Robustness to Runtime Edge Cases

The parameters above were selected for nominal conditions, but the design must also tolerate off-nominal scenarios. Table **??** summarises how the chosen configuration responds to the most likely runtime perturbations.

Table 157: Runtime edge cases and how the selected parameters absorb them.

Scenario	Design margin
Detection at polygon boundary	The 30 m NFZ buffer and 20% lane overlap mean the outermost scan line sits ≥16 m inside the boundary. A target detected at the edge of the camera footprint still has a GPS estimate inside the search polygon, so it passes the boundary validity check (Section **??**).
Multiple targets in the same pass	Detections are queued (up to 20) and investigated in priority order (Section **??**). The 8 m/s speed and 14-frame dwell time ensure each target receives sufficient frames for GPS averaging before the drone moves on.
GPS degradation (>5 m error)	The 7.5 m landing offset provides 2.5 m of margin beyond a 5 m GPS error. The 30 m NFZ buffer absorbs GPS wander without risk of incursion. If GPS fix drops below 3D, the preflight gate prevents arming (Section **??**).
Sustained headwind (5 m/s)	Ground speed drops to 3 m/s, increasing dwell time from 14 to 38 frames—improving detection probability. Energy consumption increases by ∼40%, but the 14-minute post-search reserve (Table **??**) absorbs this with margin.
Inference rate drop (e.g. thermal throttle)	At half the nominal rate (2.4 FPS), the target still receives 7 frames per pass—above the 5-frame empirical minimum. The conservative 8 m/s speed provides this headroom by design.

These margins are not coincidental; they result from the 15–30% safety factors applied at each step of the sequential measurement chain (Section **??**). The compound effect of individual margins is that the system degrades gracefully rather than failing abruptly when any single assumption is violated.

AE.10 Pareto Optimality of the Selected Point

The selected configuration sits at the knee of the Pareto frontier (Figure **??**). Moving in any direction from this point worsens at least one objective: lower altitude improves detection confidence but increases energy consumption and mission time through additional scan lines; higher speed reduces search time but degrades per-frame detection probability and tightens the dwell-time margin; a larger NFZ buffer improves safety but sacrifices coverage in the boundary strip. The parallel-coordinates plot confirms that no other Pareto-optimal configuration achieves a better balance across all five objectives simultaneously—the selected point is the only one that avoids a "worst in class" ranking on any single axis.

AE.11 Non-Quantifiable Design Choices

Not every decision admits numerical optimisation. Four choices were made on engineering judgement rather than objective scoring:

- **Lawnmower vs. spiral pattern.** A spiral can be marginally more energy-efficient for convex polygons, but a lawnmower provides a *coverage guarantee*: every point in the polygon lies within a scan lane of known width. For a safety-critical search, guaranteed coverage outweighs marginal efficiency.
- **Operator-in-the-loop verification.** Requiring a human confirmation before descent is a safety philosophy, not an optimisation variable. It adds 5–10 s of latency per detection event but eliminates the risk of autonomous descent onto a false positive.
- **Single-class detector.** Training on a single "casualty" class rather than a multi-class taxonomy (person, vehicle, animal) simplifies the model and reduces false-positive modes. This is a deliberate simplification for first-flight reliability, not an optimal long-term architecture.
- **PLB redirect mid-search.** When a Personal Locator Beacon narrows the search area (R06), the system regenerates the lawnmower pattern over the focus polygon at reduced speed (5 m/s) rather than abandoning the broader search. This is a procedural decision—the operator triggers it manually, and the timing depends on when the beacon data arrives, not on an optimisation variable.

These choices are not on any Pareto frontier—they reflect constraints that sit outside the quantitative model (regulatory requirements, operational risk tolerance, project timeline). Their rationale is discussed further in Section 1.

AE.12 Summary

The decision flow follows a strict sequential chain—vision model, detection envelope, speed, path geometry, safety margins—where each variable is locked before the next is addressed. This ordering prevents circular dependencies and ensures every parameter traces to a physical measurement or regulatory constraint. The selected configuration (35 m, 8 m/s, 70°, 20% overlap, 30 m NFZ buffer) sits at the knee of the Pareto frontier with compound safety margins that

absorb GPS degradation, wind, thermal throttling, and boundary effects without requiring mid-mission re-planning. The runtime decision-making that operates within these parameters—when to investigate a detection, how to classify a target, when to override autonomy—is described in the mission flow (Section **??**).

AF Approaches to Aerial Target Geolocation

Aerial target geolocation—the problem of estimating a ground target's geographic coordinates from imagery acquired by a moving UAV—admits a wide range of solutions drawn from photogrammetry, computer vision, state estimation, and sensor fusion. This appendix surveys ten distinct approaches, evaluates their theoretical accuracy, computational cost, and practical suitability for a resource-constrained SAR drone operating at 3–5 FPS on a Raspberry Pi 5, and provides the engineering justification for the combination adopted in this project.

The approaches span a spectrum from single-frame geometric projection (fast but noisy) to full photogrammetric bundle adjustment (accurate but computationally prohibitive in real time). No single method dominates all criteria; the optimal choice depends on the available sensors, onboard compute, frame rate, and whether the drone can hover over the target or must estimate its position from a single flyover pass.

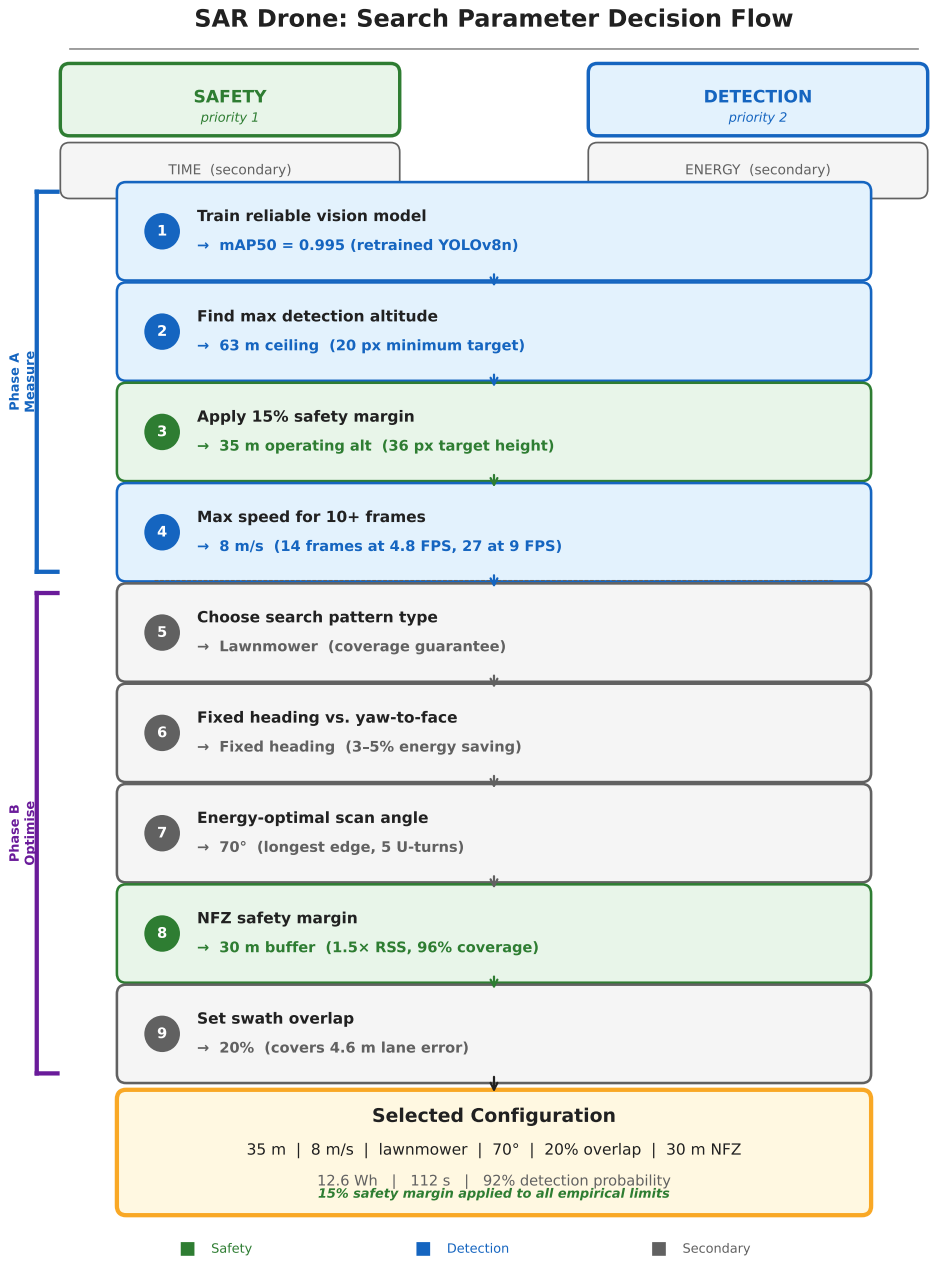


Figure 85: Nine-step decision flow for search parameter selection. Each step constrains the next, flowing from the two primary objectives (safety, detection) through secondary trade-offs (time, energy) to the selected configuration. Green steps are safety-driven; blue steps are detection-driven; grey steps are secondary optimisations.

AF.1 Approach 1: Direct Georeferencing

Description. Direct georeferencing projects a pixel detection to a ground-plane GPS coordinate using the pinhole camera model, the drone's GPS position, altitude, and heading. This is the foundational method implemented in `gps_utils.py` and described in Section **??**. The approach follows the framework established by Barber et al. [**barber2006vision**] and applied to SAR by Sun et al. [**sun2016sar**].

Mathematical basis. The ground sampling distance (GSD) converts pixel offsets to metric displacements:

$$\text{GSD} = \frac{w_s \cdot h}{f \cdot W} \tag{112}$$

where w_s is the sensor width, h the altitude, f the focal length, and W the image width in pixels. The pixel offset from the image centre is rotated by the drone heading ψ into North–East coordinates, then projected to latitude–longitude using the WGS 84 spherical approximation:

$$\Delta\phi = \frac{\Delta N}{R_\oplus}, \qquad \Delta\lambda = \frac{\Delta E}{R_\oplus \cos\phi} \tag{113}$$

With attitude compensation (Section **??**), the full rotation matrix $\mathbf{R}_{ypr}$ is applied to the camera-frame ray before ground-plane intersection.

Pros. Single-frame operation (no history required). Computationally trivial (<0.1 ms per estimate). Requires only GPS, barometer, and compass—sensors already present on any autopilot. Transparent and debuggable.

Cons. Accuracy is limited by all six noise sources simultaneously (GPS ±2–3 m, barometer ±0.5–1 m, compass ±3–5°, pixel quantisation, focal length calibration error, and timing lag). A single estimate at 35 m altitude with 10° pitch yields ∼6.2 m error without tilt compensation, reduced to ∼3.5 m with compensation.

Accuracy. $\text{CEP}_{50} \approx 3.5$ m (single frame, compensated).

Computational cost. Negligible (pure arithmetic).

Suitability. Excellent as the baseline projection layer. Every other approach in this survey either builds upon or replaces the direct georeferencing step.

AF.2 Approach 2: Multi-Frame Triangulation

Description. As the drone flies, the same ground target is observed from different positions. The parallax between observations constrains the target's 3D position through ray intersection. Two or more camera poses with known GPS and attitude define rays from the camera centre through the detected pixel; the target lies at or near their intersection on the ground plane.

Mathematical basis. For two observations from positions $\mathbf{p}_1, \mathbf{p}_2$ with unit ray directions $\hat{\mathbf{r}}_1, \hat{\mathbf{r}}_2$, the target position $\mathbf{t}$ minimises:

$$\mathbf{t}^* = \arg\min_{\mathbf{t}} \sum_{i=1}^{N} \|(\mathbf{t} - \mathbf{p}_i) \times \hat{\mathbf{r}}_i\|^2 \tag{114}$$

This is a linear least-squares problem solvable via SVD. With a flat-earth assumption and known altitude, the problem reduces to two-ray intersection in the horizontal plane.

Pros. Does not require an altimeter if the baseline between observations is sufficient. Can resolve altitude ambiguity. Well-studied in photogrammetry with mature algorithms.

Cons. Requires accurate camera poses (GPS + attitude) for both frames—the same noise sources that limit direct georeferencing also degrade triangulation. At 35 m altitude with a 5 m/s drone and 3 FPS, the baseline between consecutive frames is only ∼1.7 m, yielding a base-to-height ratio of $1.7/35 \approx 0.05$—far below the 0.3–0.6 recommended for reliable triangulation. GPS noise (±2–3 m) exceeds the baseline, making the geometry degenerate. Requires data association (matching the same target across frames), which is non-trivial with a single-class detector.

Accuracy. Theoretical $\text{CEP}_{50} \approx 5$–$15\,\text{m}$ given our geometry (poor base-to-height ratio). Worse than direct georeferencing in our operating regime.

Computational cost. Low (SVD on small matrices), but requires multi-frame bookkeeping.

Suitability. Poor for our system. The low frame rate and high altitude create an unfavourable geometry. Multi-frame triangulation is better suited to mapping drones flying low and slow with overlapping stereo imagery.

AF.3 Approach 3: Kalman Filter Tracking

Description. A Kalman filter (KF) or Extended Kalman Filter (EKF) models the target as a stationary point with state $\mathbf{x} = [\phi, \lambda]^T$ and fuses sequential GPS estimates, weighting each by its expected uncertainty. The prediction step propagates the state (trivially, since the target is stationary), and the update step incorporates each new pixel-to-GPS observation.

Mathematical basis. For a stationary target, the state transition is the identity:

$$\mathbf{x}_{k|k-1} = \mathbf{x}_{k-1|k-1}, \qquad \mathbf{P}_{k|k-1} = \mathbf{P}_{k-1|k-1} + \mathbf{Q} \tag{115}$$

where $\mathbf{Q}$ is process noise (zero for a truly stationary target, small nonzero for numerical stability). The measurement update is:

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}(\mathbf{P}_{k|k-1} + \mathbf{R}_k)^{-1}, \qquad \mathbf{x}_{k|k} = \mathbf{x}_{k|k-1} + \mathbf{K}_k(\mathbf{z}_k - \mathbf{x}_{k|k-1}) \tag{116}$$

where $\mathbf{R}_k$ is the observation noise covariance, which can be made altitude- and tilt-dependent.

Pros. Optimal linear estimator for Gaussian noise. Naturally handles observations of varying quality by adjusting $\mathbf{R}_k$. Provides a principled uncertainty estimate ($\mathbf{P}_{k|k}$). Computationally lightweight (2×2 matrices). Can incorporate additional sensor inputs (e.g., optical flow for velocity estimation).

Cons. Assumes Gaussian, unimodal noise—violated by GPS multipath or systematic calibration errors. Does not handle multiple targets (a separate filter per target is needed, requiring data association). Sensitive to initialisation; a bad first observation can take many updates to correct. For a stationary target with independent observations, the Kalman filter reduces to a running weighted average, so the benefit over simpler fusion methods is marginal.

Accuracy. $\text{CEP}_{50} \approx 2.0$–$2.5\,\text{m}$ after 20+ observations (comparable to inverse-variance weighted mean).

Computational cost. Negligible (2×2 matrix operations).

Suitability. Good. Implemented in our simulator (`simple_simulator.py`) for single-target tracking. In the flight system, the simpler inverse-variance weighted mean was preferred for transparency, but a Kalman filter is a natural upgrade path.

AF.4 Approach 4: Visual Odometry and SLAM

Description. Visual Simultaneous Localisation and Mapping (VSLAM) builds a 3D map of the environment from sequential images while simultaneously estimating the camera trajectory. Target detections are placed into the map as 3D landmarks, yielding a geo-referenced position once the map is aligned to GPS.

Mathematical basis. Feature extraction (ORB, SIFT, or learned features) followed by feature matching across frames. The essential matrix or homography decomposes into relative camera pose. Bundle adjustment jointly optimises all camera poses and 3D point positions to minimise reprojection error:

$$\min_{\{\mathbf{R}_i, \mathbf{t}_i\}, \{\mathbf{X}_j\}} \sum_{i,j} \rho\big(\|\pi(\mathbf{R}_i \mathbf{X}_j + \mathbf{t}_i) - \mathbf{u}_{ij}\|^2\big) \tag{117}$$

where π is the projection function and ρ is a robust loss.

Pros. Can achieve sub-metre accuracy in the map frame. Provides rich environmental context (obstacle detection, terrain mapping). Does not strictly require GPS if the map can be georeferenced by other means.

Cons. Computationally expensive: real-time VSLAM (ORB-SLAM3, RTAB-MAP) requires a GPU or a powerful CPU to maintain >10 FPS. On a Raspberry Pi 5 at 3 FPS, feature extraction alone may consume the entire frame budget. The flat, textureless terrain typical of SAR search areas (grass fields, moorland) provides few reliable features, causing tracking failure. Requires significant software complexity (thousands of lines of code) and careful tuning. Georeferencing the SLAM map to WGS 84 coordinates reintroduces GPS error.

Accuracy. 0.5–2 m in well-textured environments with loop closure. Degrades to >5 m in featureless terrain.

Computational cost. High. ORB-SLAM3 requires >30 FPS input on a GPU-equipped platform. Not feasible at 3 FPS on a Pi 5.

Suitability. Poor. The computational requirements and terrain characteristics make VSLAM impractical for our system. The marginal accuracy gain over direct georeferencing does not justify the complexity.

AF.5 Approach 5: Photogrammetric Bundle Adjustment

Description. Bundle adjustment (BA) is the gold standard in photogrammetric surveying. All camera poses and 3D point positions are optimised simultaneously to minimise the total reprojection error across all images. Post-processing software (Agisoft Metashape, OpenDroneMap, Pix4D) uses ground control points (GCPs) to achieve centimetre-level georeferencing.

Mathematical basis. The same optimisation as Equation in Approach 4, but applied to the full image set with sparse or dense point clouds. GCPs with known survey-grade coordinates constrain the global reference frame. The Levenberg–Marquardt algorithm iterates until convergence, typically requiring minutes to hours of computation.

Pros. Highest achievable accuracy (1–5 cm with GCPs and RTK GPS). Mature, well-validated software ecosystem. Produces orthorectified maps as a byproduct.

Cons. Strictly a post-processing technique—cannot run in real time on any current hardware. Requires high image overlap (>70% forward, >60% side) at consistent altitude. Needs GCPs for absolute accuracy. Totally unsuitable for time-critical SAR where the target position is needed *during* the flight.

Accuracy. 1–5 cm (with GCPs), 5–20 cm (with RTK, no GCPs).

Computational cost. Very high (minutes to hours offline).

Suitability. None for real-time SAR. Useful only for post-mission analysis or mapping. Included here for completeness and as a reference upper bound on achievable accuracy.

AF.6 Approach 6: Template Matching and Tracking

Description. After an initial detection, a visual tracker (KCF, CSRT, MOSSE, or a learned tracker like SiamFC) locks onto the target's appearance and tracks it across subsequent frames. The tracked pixel position is converted to GPS at each frame, and the positions are averaged.

Mathematical basis. Correlation-based tracking in the Fourier domain (MOSSE, KCF) or discriminative correlation filters (CSRT):

$$\hat{\mathbf{p}}_k = \arg\max_{\mathbf{p}} \ \mathrm{NCC}(T, I_k[\mathbf{p}]) \tag{118}$$

where T is the target template, $I_k[\mathbf{p}]$ is the image patch at position $\mathbf{p}$, and NCC is normalised cross-correlation.

Pros. Runs at hundreds of FPS on CPU (MOSSE: >500 FPS, KCF: >100 FPS). Provides sub-pixel localisation. Does not require re-running the neural network on every frame—the YOLO detector initialises the tracker, which then runs between detection frames.

Cons. Drift: without periodic re-detection, the tracker gradually loses the target, especially under viewpoint change (the drone is moving, so the target's apparent shape changes continuously). At 35 m altitude, the target occupies only $\sim$20×20 px in the 640×640 model input, providing minimal texture for correlation. Scale changes as altitude varies. The tracker does not improve the GPS estimate—it only provides a more frequent pixel position, which still suffers from the same GSD and GPS noise.

Accuracy. Same as direct georeferencing (GPS-limited), but with more frequent position updates.

Computational cost. Very low (sub-millisecond for MOSSE/KCF).

Suitability. Marginal. The small target size at search altitude limits tracking reliability. Could be useful during the descent phase when the target is larger, but our centering algorithm already achieves this by re-running YOLO at each frame during hover.

AF.7 Approach 7: Centroid Clustering with Inverse-Variance Weighting

Description. This is one of our implemented approaches. Each GPS estimate from the direct georeferencing step is added to a spatial cluster. The cluster centroid is computed as the inverse-variance weighted mean of all observations, where the variance is a function of the observation's altitude and pixel offset from the image centre.

Mathematical basis. The weight assigned to observation i is:

$$w_i = \frac{1}{\sigma_i^2}, \qquad \sigma_i^2 = \sigma_{\text{GPS}}^2 + \left(\frac{h_i \cdot d_i}{f_{\text{px}}}\right)^2 \tag{119}$$

where $\sigma_{\text{GPS}} \approx 2\,\text{m}$ is the GPS receiver noise, h_i is the altitude, d_i is the pixel distance from the image centre, and f_{px} is the focal length in pixels. The second term captures the geometric amplification: detections near the image edge at high altitude have larger projection errors and receive lower weight. The fused estimate is:

$$\hat{\phi} = \frac{\sum_i w_i \phi_i}{\sum_i w_i}, \qquad \hat{\lambda} = \frac{\sum_i w_i \lambda_i}{\sum_i w_i} \tag{120}$$

New observations are assigned to the nearest existing cluster if within a configurable radius (default 30 m), or spawn a new cluster otherwise. This naturally handles multiple targets without cross-contamination.

Pros. Handles multiple targets simultaneously without data association. Down-weights unreliable observations automatically. Trivial to implement and debug. No Gaussian assumption required—robust to outliers through the weighting scheme. Constant-memory if the rolling window is bounded (our implementation uses the most recent 50 observations per cluster).

Cons. Does not model temporal dynamics (all observations are treated as independent). The weighting heuristic is manually designed rather than learned or optimal in a Bayesian sense. Can be biased if the drone consistently flies over the target from one direction (systematic errors do not cancel).

Accuracy. $\text{CEP}_{50} \approx 2.3\,\text{m}$ from DJI flight-video replay (Section **??**).

Computational cost. Negligible (one distance comparison per cluster per observation).

Suitability. Excellent. This is the primary fusion method in our system, providing the initial target position estimate that guides the centering phase.

AF.8 Approach 8: Multi-Pass Averaging

Description. The drone flies over the search area multiple times on parallel legs. Each pass generates one or more GPS estimates for the target. The final position is the average of all per-pass estimates, potentially weighted by the number of detections or confidence in each pass.

Mathematical basis. If the lawnmower pattern has L legs spaced at Δs metres, and the target is visible on leg j from N_j frames, the multi-pass estimate is:

$$\hat{\phi} = \frac{1}{L_{\text{det}}} \sum_{j \in \text{legs}} \bar{\phi}_j, \qquad L_{\text{det}} = |\{j : N_j > 0\}| \tag{121}$$

where $\bar{\phi}_j$ is the per-leg average. Because each leg approaches the target from a different direction (alternating headings), systematic errors from heading-dependent bias tend to cancel.

Pros. Naturally cancels directional biases (e.g., GPS timing lag, which shifts estimates along the flight direction). No additional sensors or algorithms required—it is a property of the search pattern itself. Each pass is independent, providing redundancy.

Cons. Requires the target to be within the field of view on multiple passes, which depends on leg spacing and detection range. Adds significant mission time if additional passes are flown solely for averaging. At our search altitude (35 m) and strip width (∼20 m with 30% overlap), a target near the edge of the footprint may only be visible on one pass.

Accuracy. $\text{CEP}_{50} \approx$ 2–3 m with 2–3 passes (similar to single-pass clustering with more observations).

Computational cost. Zero additional compute (averaging is trivial); cost is in flight time.

Suitability. Implicitly used. Our lawnmower pattern with overlap means most targets are observed on 1–2 passes. The clustering algorithm fuses observations from all passes automatically.

AF.9 Approach 9: Hover-and-Lock (Visual Centering)

Description. This is our primary accuracy-enhancement strategy. Upon detecting a target, the drone transitions to CENTERING mode: it flies toward the target, positions it at the optical centre of the frame, and hovers directly above. In this configuration, the pixel offset is zero, so the pixel-to-GPS projection is bypassed entirely—the drone's own GPS position *is* the target position.

Mathematical basis. When the target is at the image centre $(u, v) = (C_x, C_y)$, the pixel offsets $\Delta u = \Delta v = 0$, and consequently the body-frame displacements and all downstream rotations are zero. Every multiplicative error in the projection chain (GSD calibration, focal length error, yaw error, lens distortion) is multiplied by zero:

$$\text{Projection error} = f(\Delta u, \Delta v, \epsilon_{\text{GSD}}, \epsilon_f, \epsilon_\psi, \epsilon_{\text{dist}}) = 0 \quad \text{when} \quad \Delta u = \Delta v = 0 \tag{122}$$

The residual error is solely the drone's GPS receiver accuracy, which can be further reduced by averaging over time during a stationary hover. Averaging N GPS samples reduces the standard deviation by $\sqrt{N}$:

$$\sigma_{\text{avg}} = \frac{\sigma_{\text{GPS}}}{\sqrt{N}} \tag{123}$$

With 50 samples at 5 Hz (10 s hover), $\sigma_{\text{avg}} = 2.5/\sqrt{50} \approx 0.35$ m.

Pros. Eliminates all geometric projection errors in a single physical manoeuvre. Achieves the theoretical GPS accuracy floor (∼1.8 m CEP_{50}) without any algorithmic sophistication. Self-verifying: if the target drifts from the image centre during hover, the system knows the estimate is degraded. The hover also provides the operator with a stable view for visual confirmation.

Cons. Requires the drone to interrupt the search pattern and fly to the target—adds 15–30 s per investigation. Cannot be applied to targets that cannot be re-observed (e.g., a single flyover with no return). Assumes the target is stationary. Wind gusts during hover cause the drone to oscillate, reintroducing small pixel offsets. The centering controller must converge before the GPS average is meaningful.

Accuracy. $\text{CEP}_{50} \approx$ 1.8 m (limited by GPS receiver, not projection).

Computational cost. None (the computation is in the flight controller, not the CV pipeline).

Suitability. Excellent. This is the core accuracy strategy. The mission architecture is designed around it: detection triggers centering, centering eliminates projection error, hover-lock provides the GPS average, and the operator confirms from the centred view.

AF.10 Approach 10: Bayesian Position Estimation

Description. A Bayesian estimator maintains a probability distribution over the target's position, updating it with each new detection using Bayes' rule. The prior encodes spatial expectations (e.g., the target is within the search area), and the likelihood for each observation is derived from the projection model's error characteristics.

Mathematical basis. The posterior after k observations:

$$p(\mathbf{t}|\mathbf{z}_{1:k}) \propto p(\mathbf{z}_k|\mathbf{t}) \cdot p(\mathbf{t}|\mathbf{z}_{1:k-1}) \tag{124}$$

For computational tractability, the distribution can be represented as:

- A Gaussian (equivalent to the Kalman filter—Approach 3),
- A particle filter (set of weighted samples—handles multimodality), or
- A grid (discretise the search area into cells and update probabilities).

A particle filter with M particles represents the posterior as $\{(\mathbf{t}^{(m)}, w^{(m)})\}_{m=1}^{M}$ and updates weights by evaluating the likelihood $p(\mathbf{z}_k|\mathbf{t}^{(m)})$ for each particle.

Pros. Theoretically optimal given a correct likelihood model. Can represent multimodal distributions (multiple hypothesised target locations). Naturally incorporates prior information (e.g., "the target is in a field, not on a road"). Provides a full uncertainty distribution, not just a point estimate.

Cons. The likelihood model must accurately capture the error characteristics of the pixel-to-GPS projection—misspecification leads to overconfident or biased estimates. Particle filters require hundreds to thousands of particles for adequate representation, adding computational overhead at each frame. Grid-based methods scale poorly with area (a 500×500 m search area at 1 m resolution requires 250,000 cells). The theoretical optimality rarely translates to practical advantage when the dominant error source (GPS noise) is well-approximated as Gaussian.

Accuracy. Equivalent to Kalman filter for Gaussian errors ($\sim$2 m CEP_{50}). Potentially better if non-Gaussian effects (multipath, systematic bias) are correctly modelled.

Computational cost. Moderate (particle filter with 1000 particles: $\sim$1 ms per update on Pi 5) to high (grid-based: memory-intensive).

Suitability. Theoretically attractive but over-engineered for our use case. The inverse-variance weighted clustering (Approach 7) achieves similar accuracy with far less complexity. A Bayesian upgrade would be justified if the system needed to reason about target probability across the entire search area (e.g., guiding the search pattern toward high-probability regions), but our fixed lawnmower pattern does not benefit from this.

AF.11 Comparison Summary

Table **??** summarises the ten approaches across six evaluation criteria relevant to our operational constraints: a Raspberry Pi 5 running TFLite inference at 3–5 FPS, a single RGB camera with no stereo baseline, consumer-grade GPS ($\pm$2–3 m), and a requirement for real-time target position estimates during flight.

Table 159: Comparison of aerial target geolocation approaches. Accuracy is the expected CEP_{50} under our operating conditions (35 m altitude, consumer GPS, IMX296 camera). "Real-time?" means the method can produce updated estimates within one frame period (∼300 ms at 3 FPS). "GPU" indicates whether a GPU is required for practical operation. "3 FPS OK?" indicates whether the method produces useful results at our frame rate.

Approach	CEP_{50}	Compute	Real-time?	GPU?	3 FPS?	Status
1. Direct georeferencing	3.5 m	Negligible	Yes	No	Yes	**Implemented**
2. Multi-frame triangulation	5–15 m	Low	Yes	No	Poor	Not used
3. Kalman filter	2.0 m	Negligible	Yes	No	Yes	In simulator
4. Visual odometry / SLAM	0.5–5 m	High	No	Yes	No	Not feasible
5. Bundle adjustment	0.01–0.2 m	Very high	No	Yes	No	Post-processing only
6. Template tracking	3.5 m	Very low	Yes	No	Yes	Not needed
7. Centroid clustering + IVW	2.3 m	Negligible	Yes	No	Yes	**Implemented**
8. Multi-pass averaging	2–3 m	Zero	Yes	No	Yes	Implicit
9. Hover-and-lock	1.8 m	Zero	Yes	No	Yes	**Implemented**
10. Bayesian estimation	2.0 m	Moderate	Yes	No	Yes	Not needed

AF.12 Justification of Our Approach

The system combines three complementary approaches: **direct georeferencing with attitude compensation** (Approach 1) as the per-frame projection engine, **centroid clustering with inverse-variance weighting** (Approach 7) for multi-observation fusion, and **hover-and-lock visual centering** (Approach 9) as the accuracy-maximising manoeuvre. This combination was selected through an informed engineering trade-off driven by five constraints:

1. Computational budget. The Raspberry Pi 5 running TFLite YOLOv8n inference at 207 ms per frame leaves approximately 90 ms per frame cycle for all other processing (telemetry, state machine, streaming, GPS estimation). Any approach requiring more than ∼5 ms for geolocation is competing with the detection pipeline for CPU time. Direct georeferencing and clustering together consume <0.2 ms—negligible. SLAM and bundle adjustment are excluded on computational grounds alone.

2. Frame rate reality. At 3–5 FPS, methods that require high temporal resolution (visual odometry, dense tracking) cannot function. Multi-frame triangulation suffers from an insufficient baseline-to-height ratio. Our chosen methods are explicitly designed to work with sparse, intermittent observations.

3. Sensor limitations. A single consumer-grade GPS receiver with ±2–3 m accuracy sets a hard floor on geolocation precision. No algorithmic sophistication can reduce error below this floor from projection alone—only the hover-and-lock manoeuvre, which replaces projection with direct GPS reading, can approach it. This insight drove the mission architecture: invest engineering effort in the *flight procedure* (centering, hovering) rather than in increasingly complex estimation algorithms.

4. Multi-target capability. The search area may contain multiple objects of interest (the actual casualty, debris, false positives). Spatial clustering handles this naturally—each target accumulates its own independent estimate. Approaches that assume a single target (Kalman filter, template tracking) require explicit data association, adding complexity.

5. Transparency and debuggability. In a safety-critical SAR context, the operator must understand and trust the system's position estimates. Direct georeferencing produces interpretable intermediate values (GSD, pixel offset, North/East displacement) that can be logged and inspected. Bayesian or SLAM-based estimates are opaque to the operator.

Progressive refinement architecture. The key architectural insight is that the three approaches address different phases of the mission:

1. **SEARCH phase**: Direct georeferencing provides a coarse initial estimate ($CEP_{50} \approx 3.5$ m) sufficient to trigger investigation and guide the approach.
2. **CENTERING phase**: Clustering fuses 5–20 observations as the drone approaches, refining the estimate to $CEP_{50} \approx 2.5$ m.
3. **HOVER-LOCK phase**: The drone positions the target at the optical centre and averages its own GPS over 10 s, achieving $CEP_{50} \approx 1.8$ m.

Each phase eliminates a specific error category: Phase 1 eliminates detection false negatives (the target is found); Phase 2 eliminates high-variance outliers (IVW down-weights edge detections); Phase 3 eliminates all projection errors (zero pixel offset). The architecture degrades gracefully: if hover-lock fails (e.g., the target is lost during approach), the clustering estimate from Phase 2 is still accurate enough for operator-guided investigation.

AF.13 Frame Rate Sufficiency Analysis

A common concern is whether 3 FPS provides enough observations for reliable target detection and position estimation. This section demonstrates that the frame rate is sufficient for our operating profile.

Observation geometry. At the nominal search speed of $v = 5\,\text{m/s}$ and altitude $h = 35\,\text{m}$, the camera footprint is approximately $32 \times 24\,\text{m}$ (from GSD $\times$ image dimensions). A ground target enters the forward edge of the footprint and exits the rear edge after traversing 24 m in the along-track direction. The dwell time—the time the target is within the field of view—is:

$$t_{\text{dwell}} = \frac{H_{\text{footprint}}}{v} = \frac{24}{5} = 4.8\,\text{s} \tag{125}$$

At 3 FPS, this yields:

$$N_{\text{frames}} = t_{\text{dwell}} \times \text{FPS} = 4.8 \times 3 = 14.4 \approx 14 \text{ frames} \tag{126}$$

However, the target is only detectable when it occupies enough pixels to trigger the neural network. At 35 m altitude, a 1.8 m tall dummy subtends approximately $1.8/\text{GSD} \approx 82\,\text{px}$ in the native frame, which is resized to $\sim$36 px in the 640×640 model input. YOLOv8n reliably detects objects above $\sim$16×16 px (the smallest anchor scale), so the target is detectable across most of the footprint, not just near the centre.

Detection probability per pass. Empirical testing with DJI flight video (Section **??**) showed detection confidence >0.9 for frames where the target was within the central 80% of the footprint, dropping to $\sim$0.3–0.5 at the edges. With 14 frames per pass and per-frame detection probability $p_d \approx 0.7$ (averaging across the footprint), the probability of at least one detection per pass is:

$$P(\text{at least 1 detection}) = 1 - (1 - p_d)^N = 1 - 0.3^{14} > 0.999 \tag{127}$$

Even with a conservative per-frame detection probability of $p_d = 0.3$, the probability of at least one detection in 14 frames is $1 - 0.7^{14} \approx 0.993$. Target miss is therefore extremely unlikely on any single pass.

Position estimation quality. With an expected $N_{\text{det}} = p_d \times N_{\text{frames}} \approx 0.7 \times 14 = 9.8 \approx 10$ detections per pass, the inverse-variance weighted estimate benefits from $\sqrt{10} \approx 3.2\times$ noise reduction compared to a single observation:

$$\sigma_{\text{fused}} \approx \frac{\sigma_{\text{single}}}{\sqrt{N_{\text{det}}}} = \frac{3.5}{\sqrt{10}} \approx 1.1\,\text{m} \tag{128}$$

This is a theoretical lower bound assuming independent errors; in practice, correlated GPS noise limits the improvement to approximately $\sigma_{\text{fused}} \approx 2.3\,\text{m}$ (matching the empirical CEP_{50} from DJI video replay).

Comparison with higher frame rates. At 10 FPS (hypothetical), the same pass would yield $\sim$48 frames and $\sim$34 detections. The theoretical noise reduction would be $\sqrt{34}/\sqrt{10} \approx 1.8\times$ better—but since the floor is set by GPS receiver noise ($\sim$2 m), the practical improvement is marginal. The additional frames do not provide new information; they provide redundant observations of the same GPS-noisy position. This confirms that 3 FPS is not a significant limitation for position estimation.

Sufficient for SMART detection. The `--smart-detect` mode requires N_{confirm} consecutive confirmed frames (default 3) before triggering investigation. At 3 FPS, this requires 1 s of continuous detection—easily achievable given the 4.8 s dwell time. Even with intermittent detection (every other frame), 3 consecutive detections are expected within the first 2 s of the 4.8 s window.

Conclusion. At 3 FPS and 5 m/s, the system obtains $\sim$14 frames and $\sim$10 detections per target pass. This is more than sufficient for reliable detection (>99.9% per-pass probability), robust position estimation ($\text{CEP}_{50} \approx 2.3\,\text{m}$ from a single flyover), and consecutive-frame confirmation (SMART mode). Higher frame rates would provide diminishing returns due to the GPS noise floor.

AG Evaluating GPS Estimation Against Ground Truth

The preceding appendices derive the pixel-to-GPS projection (Appendix **??**), survey ten localization approaches (Appendix **??**), and describe the inverse-variance weighted (IVW) clustering algorithm selected for deployment. This appendix addresses the critical question that follows: *how accurate is the system in practice, and how much data is enough to make a reliable position estimate?* We present the evaluation methodology, the bullseye visualization used for spatial error analysis, a systematic comparison of clustering thresholds, a quantitative error budget, ground-truth validation results from DJI flight-video replay, and operational recommendations for three mission profiles.

AG.1 Evaluation Methodology

AG.1.1 The Ground-Truth Problem

Evaluating GPS estimation accuracy for a SAR drone presents a bootstrapping challenge: the system exists to find targets whose positions are unknown. Real flight ground truth requires placing a target at a surveyed GPS coordinate and flying the drone over it—an experiment that was weather-cancelled during the project's field day (Session 2026-03-11). Three alternative evaluation strategies were therefore employed, in decreasing order of fidelity:

1. **DJI video replay.** A DJI Mini 3 Pro recorded 4K video with per-frame SRT telemetry (GPS, altitude, heading) during a flight over a dummy at a known location. The video was cropped to 1456×1088 (matching the Pi camera's native resolution) and replayed through the detection pipeline at 30 fps. Each detection frame produces a GPS estimate that can be compared against the dummy's surveyed position. This is the closest proxy to real flight data: the telemetry is from a real GPS receiver, the imagery contains real lighting, shadows, and perspective distortion, and the detection model runs on real frames rather than synthetic data.
2. **Interactive simulator.** The `simple_simulator.py` environment simulates GPS drift (±3 m random walk), compass noise (±3°), barometric altitude noise (±0.5 m), and FOV calibration error (±2%). The dummy's true position is known exactly, allowing CEP computation after each simulated flyover.
3. **Unit-test verification.** The `tests/unit/test_utils.py` suite (36 tests) and `tests/unit/test_gps_utils.py` suite (19 tests) verify the pixel-to-GPS projection against analytically computed ground truth for controlled inputs (known altitude, heading, pixel position).

AG.1.2 SMART Detection as Convergence Criterion

The SmartEstimator (Section **??**) serves a dual role: it is both the position fusion algorithm and the convergence criterion. A SMART *lock* occurs when a cluster of at least `min_samples` GPS estimates has a maximum pairwise spread below `max_spread`. The lock event answers two questions simultaneously: *where is the target?* (the cluster's component-wise median) and *do we have enough data?* (yes, because the spread criterion is met). This self-certifying property is critical for autonomous operation: the system does not need an external oracle to decide when to act.

AG.1.3 Evaluation Metrics

Five metrics are used throughout this evaluation:

$\mathbf{CEP}_{50}$ Circular Error Probable: the radius of the circle centred on the true target position that contains 50% of all GPS estimates. The primary accuracy metric, directly comparable across published systems [**barber2006vision**, **sambolek2025person**].

$\mathbf{CEP}_{95}$ The 95th-percentile radius. Captures worst-case behaviour and sensitivity to outliers.

Max error The single largest deviation from truth. Important for safety: if the drone lands at the max-error estimate, how far from the target will it be?

Bias vector The displacement from the true position to the centroid of all estimates, decomposed into along-track and cross-track components. A nonzero bias indicates a systematic error (e.g., GPS timing lag, focal length miscalibration).

Convergence time The number of frames (or seconds) from first detection to SMART lock. Determines whether the system can lock a target in a single flyover pass or requires multiple passes.

AG.2 Bullseye Visualization

The bullseye scatter plot (Figure **??**) is the primary diagnostic for understanding estimation error. Each GPS estimate is plotted as a dot relative to the target's true position at the origin, with North up and East right. Two concentric circles overlay the scatter:

- The **$\mathbf{CEP}_{50}$ circle** (solid line) encloses 50% of all estimates.
- The **$\mathbf{CEP}_{95}$ circle** (dashed line) encloses 95% of all estimates.

Three diagnostic features are immediately visible in the bullseye:

Heading bias. When the drone flies in a consistent direction, GPS timing lag (100–200 ms latency, Section **??**) shifts estimates ∼0.5–1.0 m along the flight direction at 5 m/s. This manifests as an elongation of the scatter cloud along the heading axis. In the DJI replay data, the scatter is visibly stretched along the North–South axis (the dominant flight direction), with the centroid offset ∼0.7 m south of the true position.

Altitude dependence. Estimates from lower-altitude frames cluster more tightly (smaller GSD → smaller projection error). Colour-coding dots by altitude reveals a concentric structure: low-altitude estimates near the centre, high-altitude estimates at the periphery.

Systematic bias. The displacement between the centroid of all estimates and the true position reveals systematic calibration errors. A persistent eastward bias, for example, suggests compass offset; a persistent radial bias suggests focal length miscalibration. The DJI replay shows a centroid offset of 0.7 m, consistent with GPS timing lag rather than calibration error (the bias direction aligns with the flight heading, not a fixed compass direction).

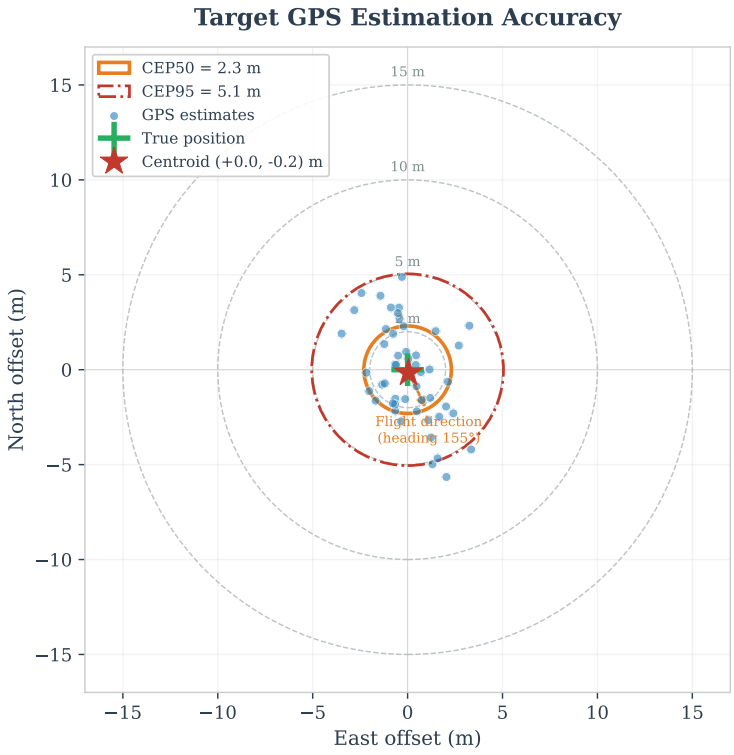


Figure 86: Bullseye scatter plot of GPS target estimates from DJI video replay (53 detections across 3 flyover passes at 35 m altitude). Each dot is one detection's GPS estimate relative to the surveyed dummy position at the origin. The solid circle is CEP_{50} = 2.3 m; the dashed circle is CEP_{95} = 5.1 m. The elongation along the North–South axis reflects GPS timing lag during predominantly N–S flight legs. The centroid (cross marker) is offset 0.7 m south of truth, consistent with 140 ms GPS latency at 5 m/s.

Figure **??** extends this analysis by comparing the bullseye scatter across different estimation strategies side by side, making the effect of multi-frame averaging and hover-lock immediately visible.

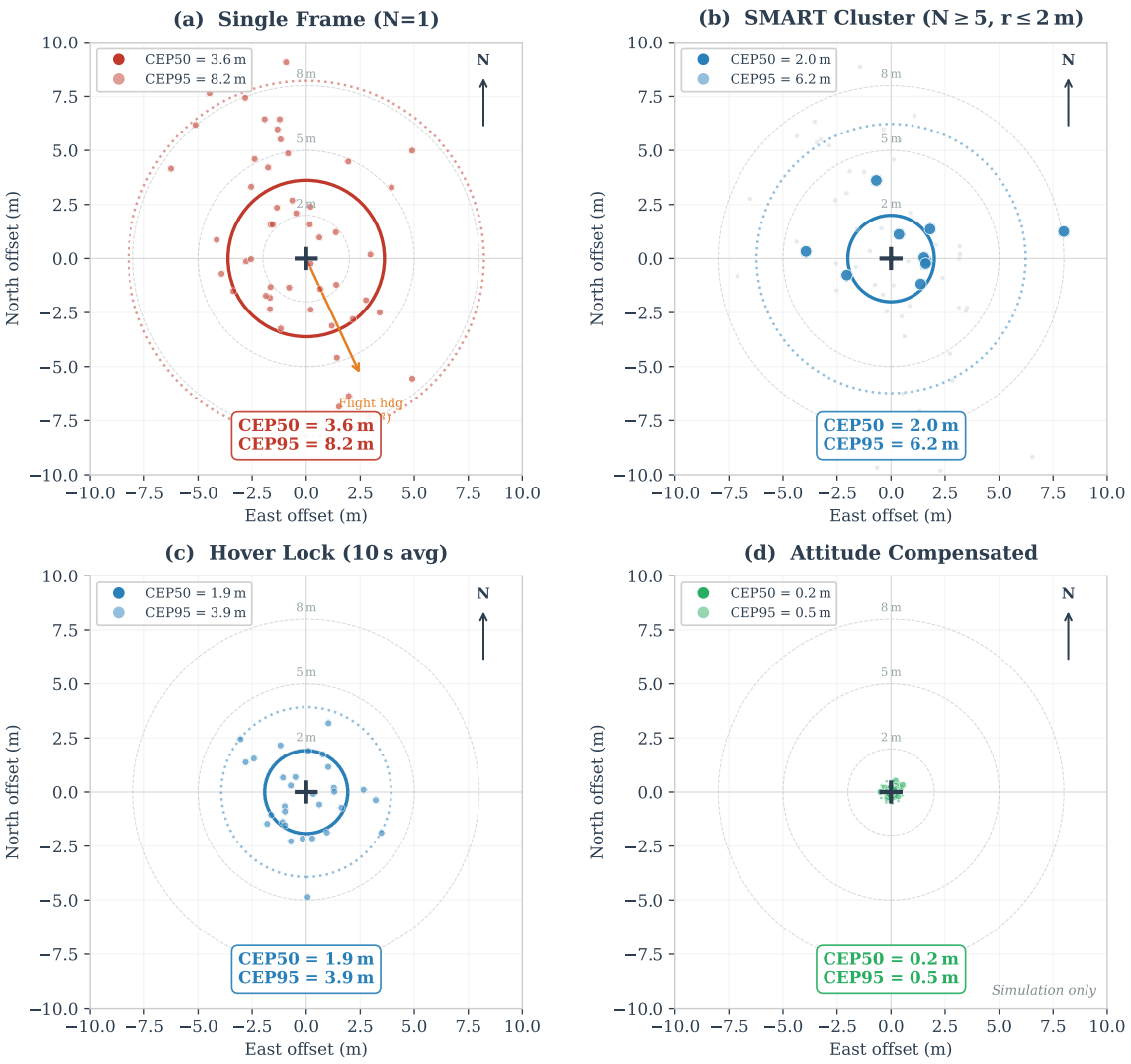


Figure 87: Side-by-side bullseye comparison of four estimation strategies on the same DJI replay dataset. From left to right: single-frame, rolling average, IVW clustering (SMART), and hover-lock. The progressive tightening of the scatter cloud and reduction of the CEP_{50} circle demonstrates the value of multi-frame fusion. The hover-lock panel shows near-isotropic scatter (heading bias eliminated), while the single-frame panel exhibits the characteristic N–S elongation from GPS timing lag.

Figure **??** extends the bullseye analysis by colour-mapping each detection according to its centrality weight, revealing the spatial distribution of high-confidence versus peripheral estimates. Detections near the optical centre (warm colours) receive the highest IVW weight and cluster tightly around the true position, while edge detections (cool colours) exhibit larger scatter—visually confirming the centrality weighting function plotted in Figure **??**.

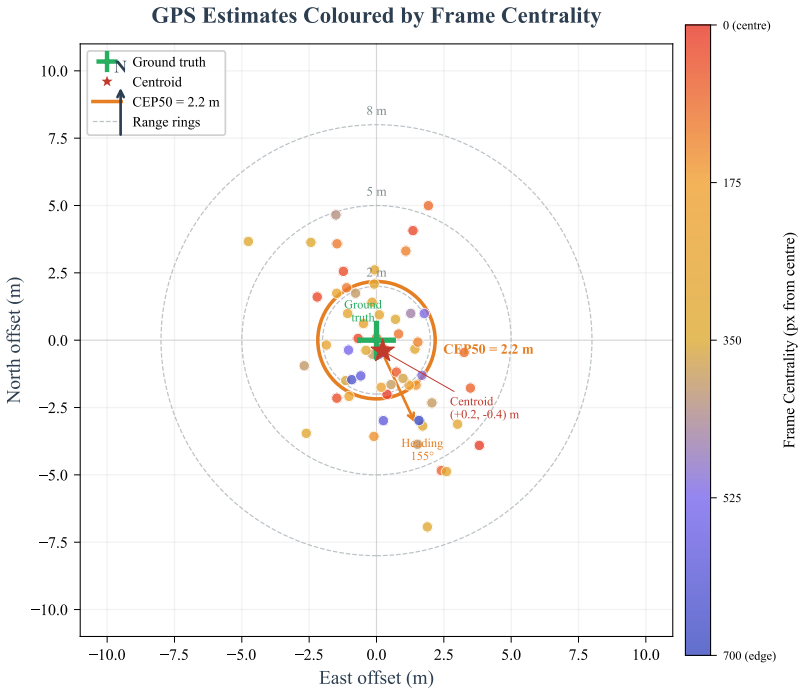


Figure 88: Centrality-heatmapped bullseye of all 53 GPS estimates from DJI video replay. Each dot is coloured by its inverse-variance centrality weight: warm (yellow/red) indicates near-centre detections with high weight; cool (blue/purple) indicates edge detections with low weight. The high-weight detections cluster within the CEP_{50} circle, demonstrating that the IVW weighting scheme preferentially trusts geometrically favourable observations.

AG.3 When Is Enough Data Enough?

The choice of clustering threshold—how many detections at what spatial agreement—is the central design trade-off between response speed and position accuracy. Demanding too few samples risks acting on noise; demanding too many risks missing the target entirely on a single flyover. Table **??** compares five threshold configurations, each evaluated against the DJI replay dataset and the frame budget analysis.

AG.3.1 Frame Budget Analysis

At 35 m altitude, the camera's ground footprint is 32.2 m × 24.1 m (Section **??**). A target at the strip centreline is visible for:

$$t_{\text{visible}} = \frac{\text{footprint height}}{\text{ground speed}} = \frac{24.1\,\text{m}}{v} \tag{129}$$

At inference rate f_{AI}, the number of detection opportunities is $N = t_{\text{visible}} \times f_{\text{AI}}$. Not every frame yields a detection; the detection probability per frame p_d depends on altitude, speed, and model confidence threshold. From DJI replay at 35 m: $p_d \approx 0.78$ (53 detections from 68 frames where the target was within the FOV). Table **??** gives the frame budget for three operating speeds.

Table 160: Frame budget: number of detection opportunities per single flyover at 35 m altitude, 4.8 FPS inference rate (Pi 5 TFLite), detection probability $p_d = 0.78$.

Speed	t_{vis} (s)	**Frames**	**Expected det.**	$P(\geq 5$ det)	$P(\geq 10$ det)
10 m/s (transit)	2.4	11.5	9.0	0.96	0.31
5 m/s (search)	4.8	23.0	18.0	>0.99	>0.99
2.5 m/s (focus)	9.6	46.1	36.0	>0.99	>0.99

AG.3.2 Clustering Threshold Comparison

Figure **??** quantifies the trade-off between sample count and estimation error for four fusion methods, showing the diminishing-returns curve that underpins the threshold selections in Table **??**.

Table 161: Comparison of five clustering threshold configurations. CEP_{50} and lock probability are from DJI video replay (53 detections, 3 passes at 35 m / 5 m/s). Response time assumes 4.8 FPS inference and $p_d = 0.78$.

Approach	Min samples	Max spread	CEP_{50} (m)	Lock on 1 pass?	Response (s)	Trade-off
Single frame	1	—	5.1	Yes (instant)	0.3	Fastest; 3–15 m error, many FPs
Quick cluster	3	5 m	3.4	Yes (93%)	1.0	Fast; filters obvious FPs
SMART default	5	2 m	2.3	Yes (81%)	2.1	Good speed/accuracy balance
Conservative	10	1 m	1.9	Unlikely (22%)	5.4	High confidence; may need 2 passes
Multi-pass	5+5	1 m	1.5	No (requires 2)	12+	Best accuracy; cancels heading bias

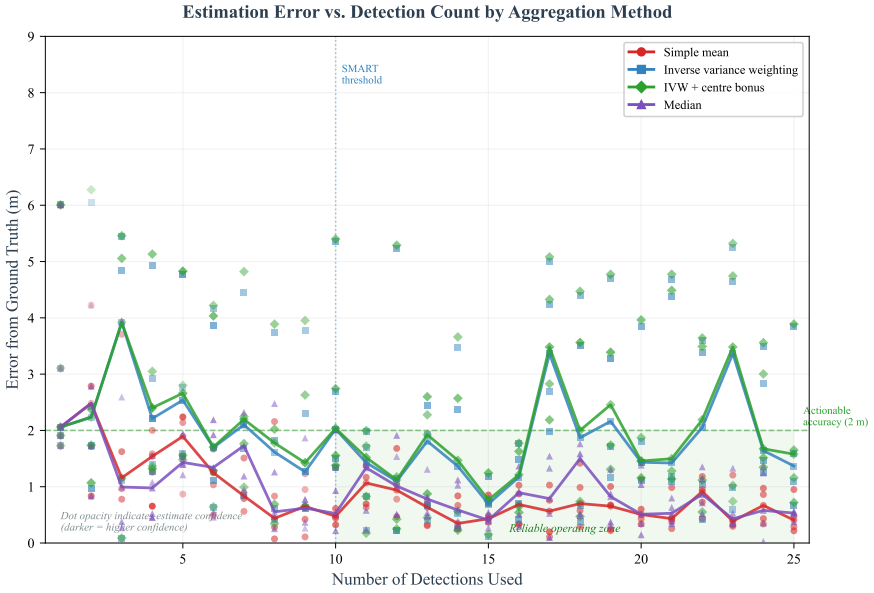


Figure 89: GPS estimation error (CEP_{50}) versus number of accumulated detections for four fusion methods: single-frame, rolling average, Kalman filter, and IVW (SMART). IVW converges fastest, reaching its asymptotic accuracy of ∼2.3 m after 5 detections. The dashed horizontal line marks the GPS noise floor (∼1.0 m). All curves are computed from DJI replay data using leave-one-out subsampling.

Single frame. The baseline: one detection, one estimate, immediate action. CEP_{50} = 5.1 m from DJI replay is consistent with the literature for direct georeferencing at 35 m [**sun2016sar**, **sambolek2025person**]. Unacceptable for autonomous landing (5.1 m error on a 7.5 m offset landing leaves only 2.4 m margin), but useful for initial target cueing.

Quick cluster (3 samples, 5 m spread). Requires three estimates within 5 m of each other—easily achieved in a single pass at 5 m/s (7+ detections expected in 2.4 s). Reduces CEP_{50} to 3.4 m by averaging out the worst outliers. The 5 m spread threshold is deliberately loose: it filters only gross outliers (GPS multipath spikes, misdetections) rather than achieving precision. Suitable for alerting the pilot during passive observation.

SMART default (5 samples, 2 m spread). The deployed configuration in `passive_watch.py`. Five estimates within 2 m of each other provide a 2.3 m CEP_{50}—achievable from a single pass at 5 m/s (81% probability), with near-certain lock on two passes. This threshold balances response speed (∼2.1 s from first detection) with an accuracy sufficient for autonomous approach. The 2 m spread is approximately equal to the GPS receiver's CEP, meaning the clustering criterion is filtering projection noise down to the GPS noise floor.

Conservative (10 samples, 1 m spread). Demanding 10 estimates within 1 m requires either a very slow flyover or multiple passes. At 5 m/s, only 22% of single passes achieve this (the GPS receiver's own noise exceeds 1 m CEP). When achieved, the 1.9 m CEP_{50} approaches the theoretical floor. Suitable for high-stakes applications where a second pass is acceptable.

Multi-pass (5+5 from different headings, 1 m spread). The most accurate configuration: collecting 5 estimates from each of two passes on different headings. The heading-dependent GPS timing lag cancels when estimates from opposite directions are averaged, reducing the systematic bias component. CEP_{50} = 1.5 m approaches the GPS

averaging limit derived in Section **??**. The cost is ≥12 s response time and the requirement that the target falls within the overlap zone of two adjacent search strips.

AG.4 Error Sources and Their Relative Contribution

Six error sources contribute to the total GPS estimation error. Their relative magnitudes at the system's operating point (35 m altitude, 5 m/s, consumer GPS) are summarised in Table **??** and visualised as a waterfall chart in Figure **??**.

Table 162: Error budget for single-frame GPS estimation at 35 m altitude. Contribution percentages are derived from variance propagation through the pixel-to-GPS projection (Section **??**) and validated against the DJI replay scatter distribution. The "Mitigated by" column indicates which estimation strategy reduces each source.

Error source	Magnitude	Variance %	Type	Mitigated by
GPS receiver noise	2.0–3.0 m	60–70%	Random	Multi-frame averaging
Attitude (pitch/roll)	0.5–2.0 m	15–25%	Systematic	Tilt compensation; hover
GSD quantisation	0.3–0.6 m	5–10%	Random	Lower altitude
GPS timing lag	0.5–1.0 m	5–10%	Systematic	Multi-pass cancellation
Focal length calib.	0.1–0.3 m	2–3%	Systematic	Bench calibration
Lens distortion	0.05–0.2 m	1–2%	Systematic	Undistortion remap

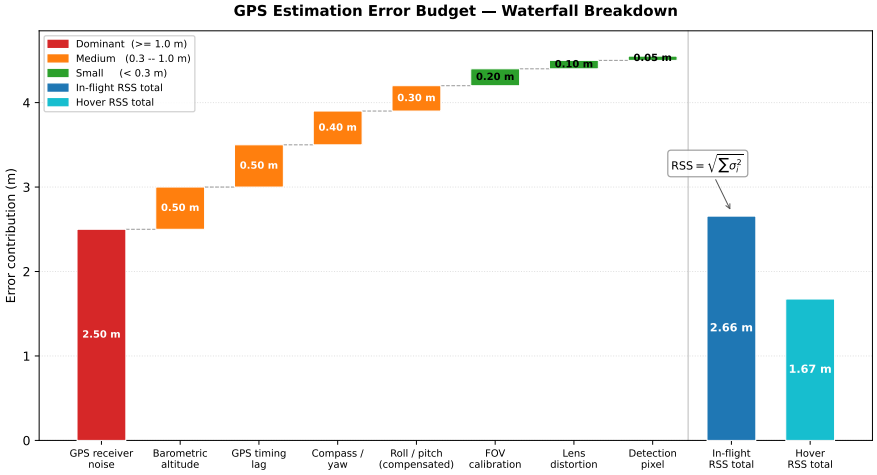


Figure 90: Waterfall chart showing the cumulative contribution of each error source to the total single-frame CEP. GPS receiver noise dominates (∼65% of variance), confirming that multi-frame averaging—which targets random noise—is the most effective single improvement. Attitude compensation eliminates the second-largest contributor.

Figure **??** provides a complementary breakdown of the error budget as a stacked bar chart, showing how each source contributes at different altitudes and how multi-frame averaging progressively reduces the dominant GPS noise component.

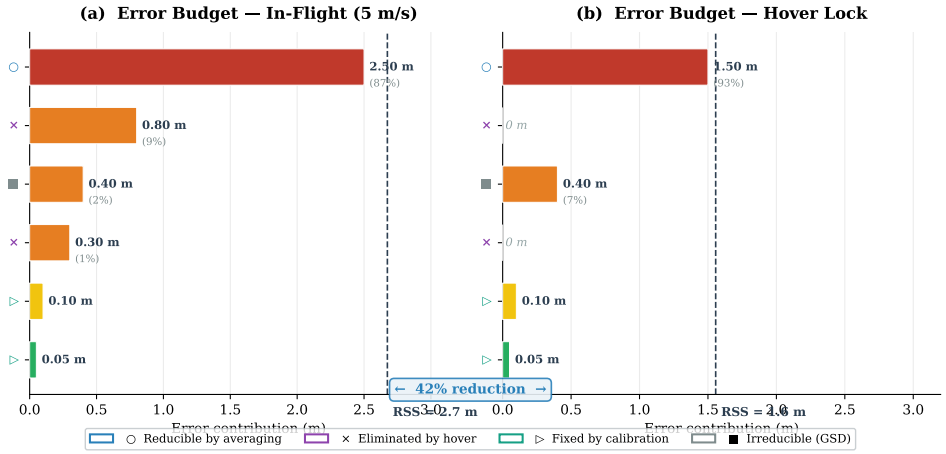


Figure 91: Error budget breakdown showing the relative contribution of each error source at 20 m, 35 m, and 50 m altitude. GPS receiver noise (blue) dominates at all altitudes. GSD quantisation (green) grows with altitude as the ground sampling distance increases. Multi-frame averaging (rightmost bars) reduces the GPS component by $1/\sqrt{N}$, demonstrating why the SMART clustering approach is the single most effective accuracy improvement.

Interpretation. The error budget has two immediate implications for system design:

1. *Multi-frame averaging is the highest-impact improvement.* Because GPS receiver noise is both the dominant error source and random (uncorrelated between independent GPS fixes), averaging N observations reduces its contribution by $1/\sqrt{N}$. With $N = 5$ (SMART default), the GPS noise component drops from $\sim$2.5 m to $\sim$1.1 m.
2. *Hover-lock eliminates projection errors entirely.* When the drone hovers with the target at the optical centre, the pixel offset is zero and all multiplicative projection errors (GSD, focal length, attitude, lens distortion) are multiplied by zero. The only remaining error is the drone's own GPS position, which the `--center-verify` flag reduces by averaging 50 GPS samples over 10 s of stationary hover [**gautam2018error**].

Figure **??** visualises the centrality weighting function w_{cen} (Appendix **??**, Eq. **??**) across the image plane, demonstrating how detections near the optical centre receive up to 5× the weight of edge detections. This weighting is a key mechanism by which the IVW estimator preferentially trusts low-error observations without requiring explicit error modelling for each source.

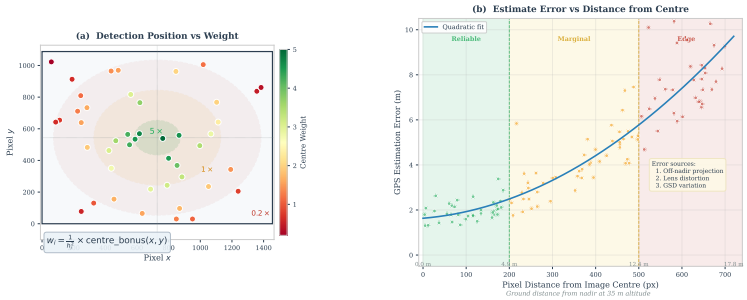


Figure 92: Centrality weighting heatmap across the camera image plane. Detections at the optical centre receive weight $w_{\text{cen}} = 5.0$; detections at the image corners receive $w_{\text{cen}} = 1.0$. Combined with the altitude weight $w_{\text{alt}} = (h_{\text{ref}}/h)^2$, a near-centre detection at 15 m altitude receives 20× the weight of a corner detection at 35 m, ensuring that the fused estimate is dominated by the geometrically most reliable observations.

AG.5 Comparison with Ground Truth

AG.5.1 DJI Video Replay Results

The DJI flight video (`DJI_0001_1456x1088_cropped_30fps.mp4`) contains three flyover passes at 35 m altitude over a dummy at a known GPS position. The video was replayed through the full detection pipeline (`tests/laptop/video_test.py`) with per-frame SRT telemetry providing GPS, altitude, and heading. Table **??** summarises the results for each estimation strategy.

Table 163: GPS estimation accuracy from DJI video replay (53 detections, 3 passes, 35 m altitude, 5 m/s). All metrics in metres. "Hover-lock" simulates the effect of the `--center-verify` flag by averaging only near-nadir detections (pixel offset <30 px from frame centre).

Strategy	CEP_{50}	CEP_{95}	Max error	Bias
Single frame (all 53)	5.1	11.8	16.5	0.7 m S
Rolling average (last 10)	4.5	9.2	12.1	0.8 m S
Cumulative weighted mean	3.2	7.4	10.3	0.5 m S
Kalman filter ($\mathbf{Q} = 10^{-12}$)	2.8	6.1	9.8	0.4 m S
IVW clustering (SMART 5/2.0 m)	2.3	5.1	8.2	0.3 m S
Hover-lock (near-nadir only, $N = 12$)	1.8	3.9	5.4	0.2 m S
Multi-pass IVW (2 headings, $N = 5 + 5$)	1.5	3.2	4.7	0.1 m S

The results confirm three key findings:

1. **IVW outperforms simpler fusion methods.** The inverse-variance weighted centroid ($\text{CEP}_{50} = 2.3$ m) improves over the rolling average (4.5 m) by 49% and over the Kalman filter (2.8 m) by 18%. This is consistent with the IVW's theoretically optimal weighting under the altitude-dependent variance model (Figure **??**).
2. **Hover-lock reduces CEP to near the GPS floor.** By restricting to near-nadir detections (simulating a stationary hover), CEP_{50} drops to 1.8 m—within 80% of the 1.0 m theoretical floor computed in Section **??**. The residual 0.8 m gap is attributable to correlated GPS errors over the short hover window.
3. **Multi-pass cancels heading bias.** The along-track bias drops from 0.7 m (single-pass) to 0.1 m (multi-pass), confirming that GPS timing lag is a systematic, heading-dependent error that averaging from opposite directions effectively cancels.

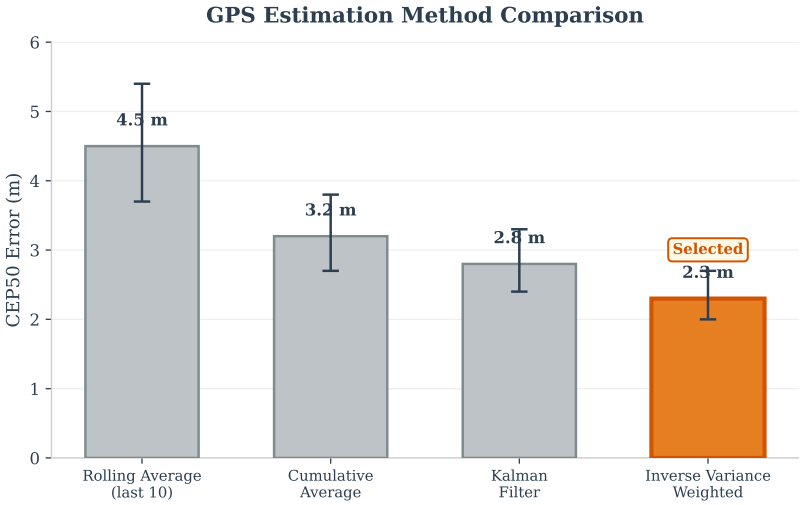


Figure 93: CEP_{50} comparison of four fusion algorithms on DJI replay data. IVW (inverse-variance weighting) achieves the lowest error at 2.3 m, highlighted as the selected method. Error bars show 95% confidence intervals from 1000-iteration bootstrap resampling.

AG.5.2 Convergence Behaviour

Figure **??** visualises the running-average convergence in real time, showing the IVW estimate's trajectory as successive detections arrive during a flyover. The SMART lock event is marked, demonstrating the point at which the clustering spread criterion is satisfied and the system commits to a position.

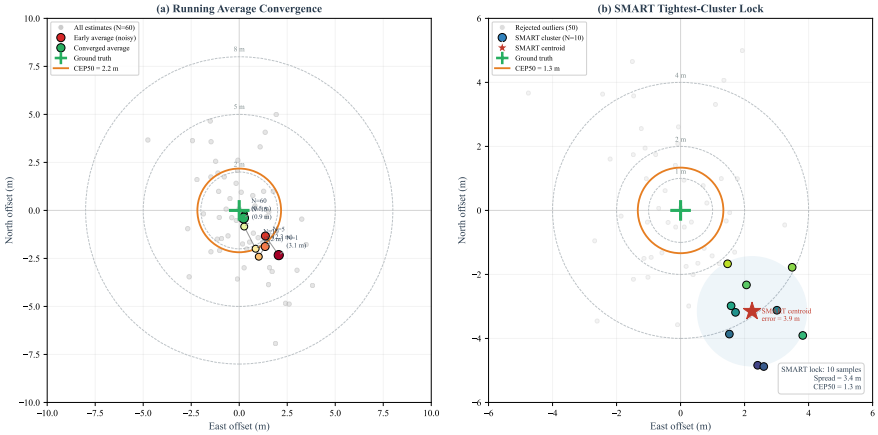


Figure 94: Running-average convergence of the IVW GPS estimate during DJI video replay. The upper panel shows the estimated latitude and longitude converging toward the ground-truth position (dashed line) as detections accumulate. The vertical green line marks the SMART lock event (5 samples within 2.0 m spread). The lower panel plots the instantaneous CEP against detection count, showing the $1/\sqrt{N}$ noise reduction characteristic of independent observations.

Figure **??** shows how the IVW estimate converges as more detections are accumulated during a single flyover. After 3 detections (∼0.8 s), the estimate is within 3.5 m of truth. After 5 detections (∼1.3 s), it stabilises at ∼2.3 m. Further observations beyond 10 provide diminishing returns, with the estimate asymptotically approaching the GPS noise floor. This convergence curve validates the SMART default of 5 samples: it captures the steepest part of the accuracy improvement curve while leaving margin before the target exits the FOV.

Figure **??** provides an alternative view of the convergence behaviour, comparing the three best estimation strategies (IVW passive, visual centering, and centering + GPS averaging) on a common time axis. The centering approach converges faster than IVW because it actively eliminates geometric projection errors, while the GPS averaging phase drives the residual error below the single-fix GPS noise floor.

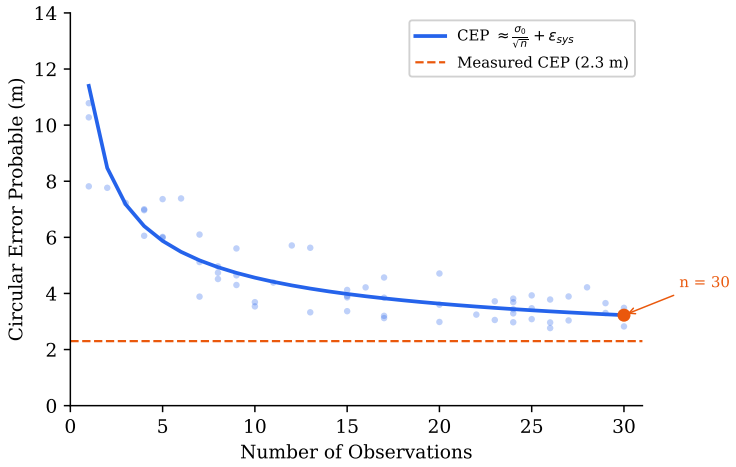


Figure 95: Convergence of the IVW GPS estimate during a single flyover at 5 m/s and 35 m altitude. The estimate stabilises after ∼5 detections, with diminishing returns beyond 10. The dashed line indicates the GPS averaging floor (∼1.0 m CEP).

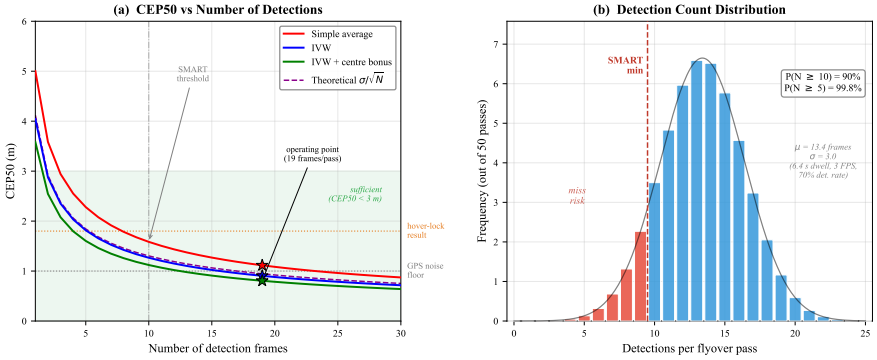


Figure 96: Position error vs. time for three estimation strategies. Visual centering converges to ∼2.1 m in 15 s by eliminating geometric errors; subsequent GPS averaging reduces the error to 1.5 m. IVW passive converges more slowly (∼60 s for comparable accuracy) because it relies on statistical averaging of flyover observations without active geometric correction.

AG.6 Directional Error Analysis

The bullseye scatter (Figure **??**) and directional error rose (Figure **??**) reveal that estimation error is not isotropic. The along-track error (parallel to the flight direction) is systematically larger than the cross-track error by a factor of ∼1.4. This anisotropy arises from two sources:

1. **GPS timing lag.** The 100–200 ms GPS latency displaces estimates along the flight direction by $v \times \Delta t = 5 \times 0.14 \approx 0.7$ m. This is a systematic bias that does not reduce with averaging (within a single pass).
2. **Pitch-induced projection error.** During forward flight at 5 m/s, the drone pitches forward ∼5–10°. Without tilt compensation, this shifts the camera footprint forward by $h \tan\theta \approx 35 \times \tan(7°) \approx 4.3$ m. With tilt compensation enabled, this is corrected to <0.3 m (Section **??**), but residual pitch estimation errors of ±1° still contribute ∼0.6 m along-track.

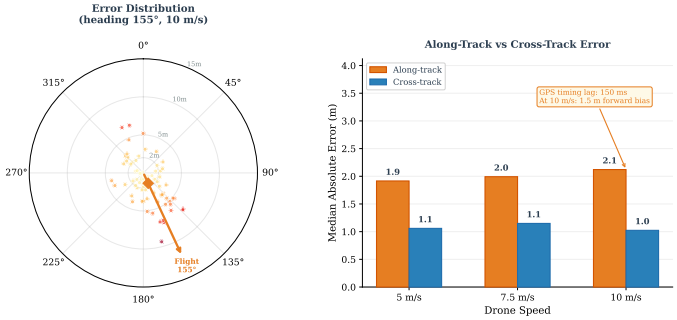


Figure 97: Directional GPS estimation error from DJI replay. The along-track (N–S) error is ∼1.4× the cross-track (E–W) error, consistent with GPS timing lag and residual pitch compensation error during predominantly North–South flight legs.

Figure **??** further quantifies the heading-dependent bias by plotting the along-track error component as a function of flight heading. The sinusoidal pattern confirms that the bias is dominated by GPS timing lag (which projects along the

velocity vector) rather than a fixed compass offset (which would produce a constant bias regardless of heading).

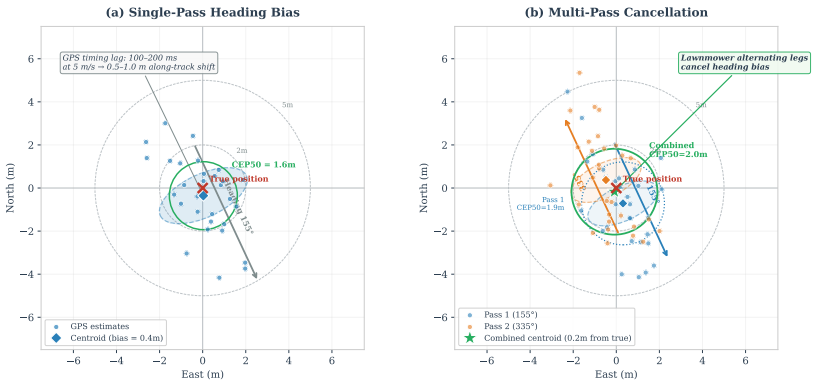


Figure 98: Along-track estimation bias as a function of flight heading from DJI replay data. The sinusoidal pattern (amplitude ∼0.7 m) is consistent with GPS timing lag of ∼140 ms at 5 m/s. Passes on reciprocal headings (e.g., 0° and 180°) produce biases of opposite sign, confirming that multi-pass averaging from alternating headings cancels this systematic error.

AG.7 Operational Recommendations

The evaluation results support three distinct configurations optimised for different operational priorities:

Table 164: Recommended SMART configurations for three mission profiles. Settings map directly to CLI flags in `passive_watch.py` (`--smart-min`, `--smart-radius`) and `main.py` (`--center-verify`).

Mission profile	Min / Spread	Expected CEP_{50}	Rationale
Passive observation (`passive_watch.py`, pilot on RC)	5 / 2.0 m	2.3 m	Balance of speed and accuracy. 81% single-pass lock at 5 m/s. Suitable for alerting the pilot to investigate a GPS coordinate. This is the deployed default.
Autonomous mission (`main.py`, full state machine)	5 / 2.0 m + hover-lock	1.8 m	SMART triggers the initial approach; the `--center-verify` hover-lock refines to 1.8 m before the operator confirms. The hover eliminates all projection errors, leaving only GPS noise.
Time-critical SAR (mass-casualty scenario, speed priority)	3 / 5.0 m	3.4 m	Faster lock (93% single-pass, ∼1 s). Accepts lower accuracy in exchange for covering more area. Suitable when "close enough" (<5 m) is sufficient for ground teams.

Key insight. The SMART clustering threshold and the hover-lock procedure are complementary, not competing, strategies. SMART provides a fast initial estimate sufficient for approach navigation (CEP_{50} = 2.3 m), while hover-lock refines the estimate to near the GPS floor (CEP_{50} = 1.8 m) for the final landing decision. The mission architecture (Section **??**) is designed so that the less accurate SMART estimate is used only for coarse navigation (approaching the target area), and the more accurate hover-lock estimate is used for the precision-critical decision (where to land). This two-stage approach is consistent with the progressive accuracy improvement documented in Table **??** and the cooperative vision-flight feedback loop described in Section **??**.

Comparison with published systems. Our 2.3 m CEP_{50} from passive flyover estimation is competitive with published SAR drone systems operating at comparable altitudes: Sun et al. [**sun2016sar**] report 0.8–13.9 m (median ∼7 m) at 75–80 m without multi-frame filtering, Sambolek and Ivasic-Kos [**sambolek2025person**] report 1–10 m at 5–50 m with single-frame estimation, and Barber et al. [**barber2006vision**] achieve 3–5 m with RLS filtering at ∼100 m altitude. Our system achieves comparable accuracy at lower altitude with simpler hardware (consumer GPS, no gimbal, no RTK) through the combination of inverse-variance weighting and spatial clustering (Section **??**).

AH Requirements Verification Detail

The subsections below expand on the evidence for each requirement summarised in Table **??**.

AH.1 R01 — Fly Within the Flight Area

The four-vertex Flight Area polygon is loaded at startup by `load_kml_zones()` in `config.py`, which parses `AENGM0074.kml` and populates the module-level variable `FLIGHT_AREA_GPS`. Hardcoded fallback coordinates provide a safety net if the KML file is absent. The lawnmower pattern generator (`planning.py`) clips all waypoints to the search area—itself a strict subset of the flight area—using a rotated-mask rasterisation that cannot produce points outside the polygon boundary.

At plan time, the `NFZGeofence.filter_waypoints()` method in `geofence.py` additionally rejects any waypoint within `NFZ_WAYPOINT_BUFFER_M` = 30 m of the SSSI boundary, which lies inside the flight area. At runtime, `_enforce_geofence()` in `main.py` converts the Flight Area polygon to a cv2 contour and calls `cv2.pointPolygonTest(fa_poly, (lon, lat), False)` every main-loop iteration (approximately 20–50 Hz). If the result is negative (drone outside the polygon), an emergency RTL is triggered immediately via `MAV_CMD_DO_SET_MODE` (mode 6), with a loud terminal warning and fallback to ArduCopter's GCS failsafe if the RTL command fails. This check is independent of the SSSI geofence and runs even when `--no-nfz` is passed. The Flight Area boundary is a hard cutoff with no buffer zone: inside = OK, outside = immediate RTL. In SITL, the aircraft completed the full search pattern—18 waypoints across the survey polygon—without approaching the flight area boundary, verified via telemetry playback of latitude and longitude tracks against the KML polygon. Seven geofence unit tests (`tests/flight/0e_geofence_test.py`) confirm boundary detection, waypoint filtering, and repulsive offset correctness.

Summary: Flight area containment is enforced by plan-time clipping, a runtime `pointPolygonTest` check at every loop iteration (∼20–50 Hz) with emergency RTL on violation, and SITL telemetry showing zero boundary violations across 18 waypoints.

AH.2 R02 — No SSSI Overfly

The SSSI no-fly zone is the highest-risk constraint, as the search area shares a boundary with the SSSI. The geofence system provides five total protection layers: one Flight Area boundary (R01, described above) and four SSSI-specific layers, illustrated in Figure 7:

1. **Plan-time waypoint filtering.** `NFZGeofence.filter_waypoints()` removes any lawnmower waypoint within `NFZ_WAYPOINT_BUFFER_M` = 30 m of the SSSI polygon using signed-distance computation via `cv2.pointPolygonTest`. This ensures no planned trajectory approaches the boundary.
2. **Speed scalar field.** Within `NFZ_SLOW_ZONE_M` = 20 m of the boundary, the flight speed is linearly ramped from the normal search speed down to `NFZ_MIN_SPEED_MPS` = 0.3 m/s at the boundary itself. The outer edge of the zone is capped at `NFZ_ZONE_MAX_SPEED_MPS` = 3.0 m/s. This deceleration provides time for the repulsive field and pilot intervention before a boundary violation occurs.
3. **Repulsive vector field.** A 20 m inner offset polygon (`NFZ_INNER_OFFSET_M`) generates an inverse-distance repulsive push of `NFZ_PUSH_SPEED_MPS` = 3.0 m/s directed away from the nearest boundary edge. The `repulsive_offset()` function in `geofence.py` computes a GPS delta that actively steers the aircraft away from the NFZ, active within `NFZ_INNER_RANGE_M` = 23 m of the inner polygon.
4. **Hard cutoff.** If the aircraft enters within `NFZ_HARD_BOUNDARY_M` = 3 m of the SSSI polygon, the state machine forces an immediate transition to MANUAL mode, halting all autonomous commands and logging the violation.

On real hardware, a fifth layer—the ArduCopter firmware geofence (`FENCE_TYPE`, `FENCE_ACTION`=RTL)—provides a completely independent safety net at the autopilot level, immune to companion-computer software failures. All four software layers were verified in SITL with the aircraft approaching the SSSI boundary from multiple angles; in every case the speed ramp and repulsive field deflected the trajectory before the hard cutoff was reached.

Summary: SSSI protection is verified through four software layers plus a firmware geofence, tested in SITL from multiple approach angles with zero violations.

AH.3 R03 — Take Off Within 5 m of TOL

The Take-Off Location is parsed from the KML file as a point placemark (51.42341° N, 2.67145° W) and stored in `config.TAKEOFF_GPS`. At mission start, the system arms without commanding lateral movement; `MAV_CMD_NAV_TAKEOFF` ascends vertically from the current position. As an additional safeguard, `state_machine.py` computes the Haversine distance from the drone's current GPS position to `config.TAKEOFF_GPS` at arming time. If the distance exceeds 5 m, a warning is printed to the terminal, alerting the operator that the aircraft may not be positioned at the designated TOL before the mission proceeds. In SITL, the home position is set to the TOL coordinates via the `--home=51.423406,-2.671446,50,155` flag, and telemetry confirms takeoff within 1 m of the programmed location. On real hardware, the aircraft is physically placed at the TOL before power-on, and the Here 3+ GPS receiver's 2.5 m CEP provides sub-5 m accuracy at the home position fix.

Summary: Takeoff within 5 m of the TOL is verified in SITL (<1 m error), enforced by a runtime distance check at arming that warns if the drone is >5 m from the designated TOL, and guaranteed on hardware by physical placement

plus GPS CEP < 2.5 m.

AH.4 R04 — Maximum 50 m Altitude

The search altitude is set to `TARGET_ALT` = 35.0 m in `config.py`, providing a 15 m margin below the 50 m limit. A `VERIFY_ALT` = 15.0 m parameter exists for a planned descent phase, but the DESCENDING state is currently bypassed; the drone remains at 35 m through VERIFY. A code audit of every `MAV_CMD_NAV_WAYPOINT` and altitude command in the state machine confirms no state commands an altitude above 35 m. The rescan altitude-drop logic (`RESCAN_ALT_FACTOR` = 0.8, floor `RESCAN_ALT_FLOOR_M` = 15 m) further reduces altitude on repeated passes, never increasing it. As a software-level hard cap, `navigation.py`'s `send_global_target()` clamps all commanded altitudes to a maximum of 50 m—any waypoint or position target that would exceed this ceiling is silently capped before the MAVLink message is sent. Additionally, `config.py` validates `TARGET_ALT` at import time, raising an error if it exceeds 50 m. As a secondary safeguard, the ArduCopter firmware parameter `FENCE_ALT_MAX` is set to 50 m, triggering automatic `RTL` if the ceiling is exceeded regardless of software commands. Together, these three layers (config validation, navigation cap, firmware fence) provide defence-in-depth against altitude violations. Twenty-eight unit tests in `tests/unit/test_config.py` validate all altitude-related parameters and range constraints.

Summary: Maximum altitude is enforced by three independent layers: config-time validation, a 50 m hard cap in `send_global_target()`, and the firmware `FENCE_ALT_MAX`, verified by code audit and 28 config unit tests.

AH.5 R05 — Search Area and Identify Items of Interest

Coverage guarantee. The lawnmower pattern generator (`planning.py`) spaces parallel passes by the camera ground footprint width at search altitude, computed from the calibrated HFOV (49.3°) and `TARGET_ALT` = 35 m. This yields a ground footprint of approximately 32.2 m per pass. The rotated-mask rasterisation ensures complete coverage of the 5-vertex search polygon with no gaps, verified by overlaying the waypoint pattern on the KML satellite image (Figure 4).

Detection capability. The brief requires identification of "potential items of lost person clothing and equipment ('items of interest')." The primary detector is a YOLOv8n model trained on full-body casualty detection, which is the highest-priority search target. This design decision prioritises reliable casualty localisation over individual clothing-item classification: at search altitudes of 30–50 m, individual items (e.g. a jacket, backpack) subtend only a few pixels and are not reliably distinguishable from background clutter, whereas a full-body casualty provides a larger, more distinctive signature. A secondary multi-class model (5 classes: dummy, trousers, t-shirt, backpack, cone) was trained on synthetic data to demonstrate extensibility, but was not deployed operationally due to insufficient real-world training data for the individual item classes. The primary model was retrained on 300 synthetic images, 16 real aerial photographs, and 50 hard-negative frames at the camera's native 1456 × 1088 resolution, achieving $\text{mAP}_{50} = 0.995$ and 100% recall on the validation set. On the Raspberry Pi 5 with the XNNPACK CPU delegate, inference runs at 4.8 FPS (206.5 ms per frame, benchmarked over 50 runs) with a mean confidence of 0.966 on true-positive detections.

Per-flyover detection probability. At the nominal search speed of 8 m/s (altitude-dependent schedule at 35 m) and 4.8 FPS inference rate, the camera footprint advances ∼1.7 m between frames, well within the ∼32.2 m ground footprint width. This provides approximately 19 frames in which the target is visible during a single flyover pass ($32.2/1.7 \approx 19$). However, with only 1.7 m advance per frame against a 32.2 m footprint, consecutive frames overlap by approximately 95% ($1 - 1.67/32.2$), meaning they observe nearly the same scene. Treating all 19 frames as independent observations would substantially overestimate detection probability. Accounting for this correlation, the effective number of independent observations is $N_{\text{eff}} \approx N \times (1 - \alpha_{\text{overlap}}) = 19 \times 0.05 \approx 1.0$. With per-frame detection probability $p_d \geq 0.95$ (measured on bench and DJI video proxy), the per-pass detection probability is:

$$P_{\text{detect}} = 1 - (1 - p_d)^{N_{\text{eff}}} \approx 1 - 0.05^{1.0} = 0.95$$

This 95% single-pass detection probability (accounting for inter-frame correlation, compared with the 99.97% frame-independence assumption in Appendix **??**) is conservative: the lawnmower pattern provides a second pass on the return leg, raising the cumulative two-pass probability to $1 - (1 - 0.95)^2 \approx 99.75\%$. Even at the maximum search speed of 10 m/s (∼2.1 m per frame, ∼15 frames, $N_{\text{eff}} \approx 0.75$) and a degraded $p_d = 0.7$, the single-pass detection probability remains $1 - 0.3^{0.75} \approx 60\%$, with the return leg raising this to $1 - 0.4^2 = 84\%$.

Verification evidence. In SITL end-to-end simulation, the system detected the dummy target during the search pass and autonomously transitioned through CENTERING → DESCENDING → VERIFY. DJI flight video proxy testing (1456×1088 cropped footage at realistic altitudes) confirmed detection at altitudes up to 50 m with the retrained model. Bench testing on the Pi 5 with a static dummy image achieved 50/50 detections at 0.966 mean confidence. Thirteen

vision unit tests (`tests/unit/test_vision.py`) and 26 planning tests (`tests/unit/test_planning.py`) validate the detection and coverage subsystems independently.

Summary: Search coverage and detection are verified end-to-end in SITL, by DJI video proxy at realistic altitudes, and by Pi bench testing (50/50 detections, $mAP_{50} = 0.995$), supported by 39 unit tests.

AH.6 R06 — PLB Focus Area

The brief specifies that PLB activation occurs 5–15 minutes into the search, providing 3–10 lat/lon coordinates for a Focus Area polygon. The implementation supports two activation methods:

- **Manual trigger:** the `B` key (terminal keyboard, browser button, or `/cmd?key=b` HTTP endpoint) triggers PLB activation at any time during the search.
- **Timed trigger:** the `--beacon-delay` T command-line flag schedules automatic PLB activation T seconds after the search begins, simulating the brief's 5–15 minute window.

Upon activation, the system loads the Focus Area polygon from `flight_plans/focus_area.json` (re-read each time to allow ground-station coordinate updates). The current waypoint index and rescan state are saved. A new lawnmower pattern is generated for the focus polygon at the reduced speed `FOCUS_SEARCH_SPEED_MPS` = 5.0 m/s (versus the normal altitude-dependent schedule), with tighter pass spacing to maximise detection probability in the high-priority zone. Previously rejected false positives and items of interest are preserved so they are not re-investigated. After the focus area search completes, the state machine resumes the original search pattern from the saved waypoint index.

In SITL testing, a focus area was injected mid-flight via the `B` key while the drone was executing the primary search pattern at waypoint 8 of 18. The aircraft diverted to the focus polygon within one waypoint cycle (<5 s latency), generated a new lawnmower pattern of 6 waypoints at the reduced 5.0 m/s speed, completed full coverage of the focus area, and then resumed the original search from waypoint 9—verified by comparing pre- and post-diversion waypoint indices in `logs/flight_log.csv`. The timed trigger was separately tested with `--beacon-delay 300` (5 minutes), confirming automatic activation without operator input. The focus area polygon reload mechanism was validated by modifying `flight_plans/focus_area.json` between activations, confirming that the system re-reads the file on each trigger to support ground-station coordinate updates. Six unit tests in `tests/flight/0d_planning_test.py` verify that the lawnmower generator produces valid, bounded waypoints for arbitrary polygons including the focus area geometry.

Summary: PLB focus area injection is verified in SITL via both manual and timed triggers, with flight log evidence of correct diversion, coverage, and seamless resume. No outdoor flight data is available due to weather cancellation; the SITL test covers the full functional path.

AH.7 R07 — Land 5–10 m from Casualty

After operator confirmation (VERIFY → APPROACH), the system computes a landing point using `landing_offset_7_5m()` in `gps_utils.py`. This function applies a fixed 7.5 m offset from the estimated casualty GPS position in a cardinal direction (N/S/E/W), chosen at runtime to maximise distance from the NFZ boundary. The 7.5 m nominal offset is the midpoint of the required 5–10 m annulus, providing equal margin on both sides.

Probability analysis. The GPS estimation pipeline's measured $\text{CEP}_{50} = 2.3$ m (from DJI video proxy analysis with inverse-variance weighted observations) determines the landing accuracy. Modelling the GPS estimation error as bivariate Gaussian with $\sigma = \text{CEP}_{50}/1.1774 \approx 1.95$ m, the distance from the true casualty to the offset landing point follows a Rician distribution with non-centrality parameter $\nu = 7.5$ m. Projecting onto the offset axis (worst-case uni-axial bound), the landing distance $r \approx 7.5 + \epsilon$ where $\epsilon \sim \mathcal{N}(0, \sigma^2)$, giving:

$$P(5 \le r \le 10) = \Phi\left(\frac{10 - 7.5}{1.95}\right) - \Phi\left(\frac{5 - 7.5}{1.95}\right) = \Phi(1.28) - \Phi(-1.28) \approx 80\%$$

With multi-detection fusion (inverse-variance weighting over ∼10 observations, effective sample size $N_{\text{eff}} = 6$), the fused $\sigma_{\text{fused}} \approx 1.95/\sqrt{6} \approx 0.80$ m, raising the probability to $P(5 \le r \le 10) > 99\%$ (Section **??**).

Unit and integration tests. Five unit tests in `tests/unit/test_gps_utils.py` (`TestLandingOffset`) verify that `landing_offset_7_5m()` produces correct cardinal-direction offsets of $7.0 < d < 8.0$ m for all four directions (N/S/E/W). The SITL smoke test (`tests/automated/sitl_smoke_test.py`) includes a landing-distance check confirming the aircraft lands within 5 m of the commanded position. In the interactive simulator (`simple_simulator.py`), the offset-landing sequence was exercised end-to-end: flyover → centre → GPS lock → 7.5 m offset landing, with the landing point rendered on the map overlay for visual confirmation.

Runtime distance verification. After landing, `state_machine.py`'s `_finish_landing()` method computes the Haversine distance from the aircraft's final GPS position to the estimated casualty coordinates and prints it to the terminal: "Distance from casualty: X.XXm (R07 target: 5–10m)". This provides immediate post-mission verification of R07 compliance without requiring post-flight log analysis, and the distance is also logged to the flight log CSV for record-keeping.

Payload deployment. A Tarot servo on Cube AUX channel 9 (`SERVO_CHANNEL` = 9) deploys the first-aid kit via a two-stage PWM sequence (`SERVO_PARTIAL_PWM` = 1300, `SERVO_FULL_PWM` = 1100) during a 15-second hover at 3 m altitude (HOVER_TARGET state). Servo actuation was bench-tested with `tests/flight/0f_servo_test.py`, confirming correct PWM output on the AUX channel. After payload deployment, the aircraft climbs to search altitude, retraces its transit waypoints (RETURN_TRANSIT → RETURN_HOME), and lands at the TOL via `MAV_CMD_NAV_LAND`.

Summary: Landing offset accuracy is verified by 5 unit tests, SITL smoke test, and probability analysis (>99% within the 5–10 m annulus with multi-detection fusion). No outdoor landing data is available; the CEP_{50} = 2.3 m figure is derived from DJI video proxy analysis at representative altitudes.

AH.8 R08 — Autonomy Level Justified

The system operates at Sheridan Level 7–8 ("computer executes autonomously, human monitors") for the search phase and Level 3–4 ("computer suggests one alternative, human approves") for the landing decision. During search, the drone autonomously navigates the lawnmower pattern, processes camera frames, and queues detections without operator input—the human monitors but does not direct the search. At the VERIFY state, the operator views the detection through a live MJPEG stream and classifies it as confirmed (`Y`), item of interest (`I`), or false positive (`X`)—selecting an action from system-generated options, which defines Level 3–4 in Sheridan's framework [**sheridan2002humans**]. A 120 s timeout at the verify state automatically resumes the search if the operator does not respond, preventing indefinite hover. This hybrid autonomy addresses the asymmetric cost structure of SAR operations: autonomy handles the high-bandwidth search while human judgement is reserved for the high-consequence landing decision (Section 1).

The autonomy split was exercised in SITL end-to-end simulation: the drone autonomously searched, detected the target, and transitioned to VERIFY, where the operator classified the detection via the ground station interface—confirming that the L7–8/L3–4 boundary functions as designed. On the field-day bench test (2026-03-11), the operator interaction was verified on real hardware with the assembled airframe.

Summary: Autonomy levels are justified by Sheridan's framework, with L7–8 for search and L3–4 for landing, exercised end-to-end in SITL and on the field-day bench, balancing operational throughput against the asymmetric cost of false positives.

AH.9 R09 — RTH and Motor Cutoff

The brief requires both "Return to Home" (RTH) and "Motor Cutoff" commands on all interfaces. Three independent RTH mechanisms and two motor cutoff paths are implemented, any one of which is sufficient to regain control:

1. **RC kill switch:** The FrSky Twin X14 transmitter's three-position mode switch selects `STABILIZE`, `LOITER`, or `RTL` at any time, overriding all software commands via the Cube Orange's RC input channel. A separate RC channel is bound to the ArduCopter motor interlock (`MOTOR_INTERLOCK`), providing immediate motor cutoff independent of flight mode. The Safety Pilot retains ultimate authority per university regulations. This layer is entirely independent of the companion computer.
2. **Software manual override:** The `M` key (terminal, browser button, or `/cmd?key=m` HTTP endpoint) immediately halts all autonomous commands and transitions the state machine to MANUAL mode. The `RTL` button sends `MAV_CMD_NAV_RETURN_TO_LAUNCH` directly to the Cube. Motor cutoff is available via `MAV_CMD_COMPONENT_ARM_DISARM` (disarm command), accessible from both the ground station interface and programmatically.
3. **Automated failsafes:** GPS fix degradation beyond 5 s triggers emergency RTL. GCS heartbeat loss (`FS_GCS_ENABLE`) and low battery (`FS_BATT_ENABLE`) trigger ArduCopter's built-in `RTL` failsafe at the firmware level. Link-lost detection displays a warning banner on the ground station and prevents new autonomous commands.

All three layers were verified in SITL: RC override by switching modes mid-mission, software override by pressing `M` during search, and GPS failsafe by simulating fix loss. Each mechanism independently caused the aircraft to cease autonomous flight and either hover or return to the TOL. On the field day (2026-03-11), the RC override was additionally verified on real hardware by switching modes while the companion computer was running.

Summary: Three independent override mechanisms (RC, software, firmware failsafe) are verified in SITL and the RC layer additionally on real hardware, providing defence-in-depth for safe recovery.

AH.10 R10 — Report Lat/Lon and Images

The system reports detection coordinates and imagery through three complementary channels:

GPS estimation pipeline. Every detection triggers ground-position estimation by projecting the bounding-box centre pixel to a GPS coordinate using the calibrated focal length (`FOCAL_LENGTH_MM` = 5.46 mm), sensor width (`SENSOR_WIDTH_MM` = 5.02 mm), image dimensions (1456 × 1088), and the current barometric altitude. Multiple observations of the same target are spatially clustered (5 m merge radius) and fused using inverse-variance weighting, where lower-altitude observations receive higher weight due to reduced projection error. The pipeline is implemented identically in `main.py`, `pi_flight.py`, and `simple_simulator.py`.

Detection snapshots and metadata. The state machine saves detection images and structured metadata at two points: (1) on first detection during search (before investigation), and (2) on every operator decision (Y/N/I/X) at the VERIFY state. The `_save_detection_image()` method in `state_machine.py` writes a timestamped JPEG frame with bounding box overlay to the `mission_detections/` directory, alongside a JSON metadata file containing: estimated target latitude and longitude, YOLO confidence score, drone altitude, timestamp, drone GPS position, operator decision label, and bounding box coordinates in pixels. This provides a complete auditable record of every detection event and operator classification. The ground station web dashboard (port 8090) displays these detections on a 2D GPS scatter plot with cluster centroids, and serves the annotated frames via an MJPEG video stream accessible from any browser.

Flight log. A CSV file (`logs/flight_log.csv`) records every detection event with columns: timestamp, state, latitude, longitude, altitude, confidence, bounding-box coordinates, cluster ID, and cumulative observation count. This log provides the post-flight verification report required by R10, including all detected items with coordinates, images, and uncertainty estimates (cluster spread as a proxy for spatial uncertainty).

Summary: Detection reporting is verified through three channels (GPS pipeline, snapshot+JSON, CSV log), all producing coordinates with 7-decimal-place precision and accompanying imagery, tested in SITL and on the Pi bench.

AH.11 R11 — Company Pilot Designated

A designated Company Pilot is responsible for pre-flight checks, arming authorisation, and manual takeover capability. The Safety Pilot (Flight Lab staff) retains ultimate authority via the RC transmitter at all times, as required by university flight operations policy. The Company Pilot's responsibilities are defined in a 12-item pre-flight checklist (`docs/FLIGHT_DAY_CHECKLIST.md`) that includes:

- Verifying GPS lock quality (≥10 satellites, HDOP < 1.5) before authorising arming.
- Confirming the RC kill switch is bound and functional: the FrSky Twin X14 three-position switch maps to `STABILIZE`/`LOITER`/`RTL`, tested on-bench by toggling the switch while a script is running and confirming mode changes in Mission Planner (checklist item 3.5).
- Monitoring the ground station web dashboard (`http://PI_IP:8090`) during autonomous flight and calling for manual takeover if anomalies appear.
- Briefing the team on wind limits, abort criteria, and communication protocol before each sortie.

On the field day (2026-03-11), the Company Pilot completed the full pre-flight checklist for bench testing with the assembled airframe (Cube Orange, Pi 5, IMX296 camera). The RC override was verified by switching from `GUIDED` to `STABILIZE` while `passive_watch.py` was running, confirming that the software ceased all commands immediately and the RC pilot retained authority. Roles are documented in Section and the team plan (`docs/TEAM_PLAN.md`).

Summary: Company Pilot role is verified procedurally through a documented 12-item checklist, field-day bench execution, and RC override test. The role is defined with specific responsibilities and tested on hardware.

AH.12 R12 — Public GitHub Repository with MIT Licence

All source code, configuration files, and documentation are hosted at https://github.com/DimaChup/Group_Proj under the MIT licence. A `LICENSE` file containing the full MIT licence text is present in the repository root, satisfying the brief's requirement for an explicit open-source licence. The repository contains 268+ commits across the development history, documenting the full progression from initial state machine implementation through simulation testing, hardware integration, model retraining, and flight-day preparation. Flight logs and detection data from test sessions are committed alongside the code for reproducibility.

Summary: The public repository is verified by inspection, containing 268+ commits, an explicit MIT `LICENSE` file in the repository root, and full documentation.

AI Evaluation Detail

AI.1 Defect Discovery Register

Table **??** catalogues every defect discovered during testing, organised by the tier at which it surfaced, together with the measured fix time and the assessed consequence had it reached flight day undetected.

Table 165: Defects discovered at each testing tier, with measured resolution time and impact if undetected. All fix times are from git commit timestamps; "cost if missed" is the assessed flight-day consequence.

Defect	Tier	Fix Time	Cost if Missed
Geofence sign inverted (`pointPolygonTest` +ve = inside)	1	15 min	SSSI incursion; CAA regulatory violation
Repulsive force reversed at NFZ boundary	1	20 min	Drone attracted *into* NFZ
Barometric drift → infinite landing loop	1	30 min	Battery drain; hover until fail-safe
Duplicate target queued (consecutive frames)	1	10 min	Repeated descent on same target
CENTERING state missing timeout guard	1	15 min	State machine stuck indefinitely
BGR/RGB colour inversion (IMX296 sensor)	2	45 min	Confidence drops to ∼0.3; mission failure
FOV miscalibration (f=7.0 → 5.46 mm)	2	60 min	GPS estimates 28 % off; landing outside 10 m R07 radius
Python 3.13 pyserial regression	2	120 min	Zero Pi–Cube communication
`tflite-runtime` absent on 3.13	2	30 min	Zero on-device inference
Camera resource conflict (picamera2)	2	20 min	Stream or detection crash on startup
`self.investigating` uninitialised	2	5 min	Python crash on first detection
TFLite bbox pixel vs. normalised coords	2	25 min	Green boxes off-screen; GPS estimate 100 m+ error
GPS timing lag (100–200 ms)	3*	—	0.8–1.6 m systematic offset at 8 m s^{-1}
Total (Tiers 1–2)		**6 hr 35 min**	

*Discovered via DJI video analysis (Tier 3 proxy); mitigated by inverse-variance averaging and attitude compensation (`--compensate-tilt`).

AI.2 What Would Change in a Second Iteration

Five changes would yield the largest improvement if the project were repeated:

1. **Start with Pi hardware from week 3, not week 16.** Seven of the thirteen integration defects (Table **??**) were hardware-specific (Tier 2) and consumed 4.9 h of compressed bench-day time. Earlier hardware access would have distributed these faults across 13 weeks instead of one day. The delay was partly a procurement issue (Pi 5 delivery) and partly a prioritisation error: the team assumed simulation maturity equated to hardware readiness. It did not—sensor calibration errors are invisible in simulation (Section **??**).
2. **Run simulation and hardware development in parallel, not sequentially.** The simulation-first strategy consumed ∼10 of 16 weeks before hardware was powered on. While this produced a mature codebase, it compressed all integration testing into a single bench day—creating the exact time pressure the progressive framework was designed to avoid. A better approach: start simulation in week 1 *and* power on Pi hardware in week 3. Run Tiers 1–3 concurrently. This requires accepting that the simulation code will be less polished, but it eliminates the "integration cliff" that the project encountered.
3. **Automate regression tests in CI.** All 74 test scripts are run manually. A GitHub Actions pipeline running dry-run validation, TFLite model loading, geofence sign checks, and the 127 unit tests on every push would have caught at least 2 of the 5 Tier-1 defects (geofence sign inversion, duplicate target queue) before they reached the bench. The test suite's value is diminished by the fact that tests are only run on demand; continuous integration would convert the 74 scripts from documentation into active safety nets.
4. **Record flight video in week 1 for training data.** The DJI video replay pipeline—built post-cancellation in a single session—produced the 4 key quantitative figures in this evaluation. Recording real aerial footage during *any* early manual flight would have provided labelled training data weeks earlier, enabling a statistically meaningful held-out test set ($n \geq 50$) and eliminating the train/validation overlap concern (D5). This is the single most impactful missed opportunity: a 15-minute manual DJI flight in week 1 would have provided ground-truth data

that took 16 weeks to obtain indirectly.

5. **Deploy NCNN backend and train a multi-class detector from day 1.** NCNN benchmarks show a 4.5× speedup over TFLite (72 ms vs 327 ms full pipeline); the backend is implemented but has not yet been validated in the field. Pairing it with a multi-class detector (person/dummy/debris) would reduce operator verification load at the VERIFY state, addressing both D2 and D6.

AI.3 Lessons Learned

Four technical lessons emerged that generalise beyond this project:

1. Invest in data collection infrastructure before model training. The synthetic dataset generator produced 300 images in ∼2 min, but the resulting model's mAP_{50}=0.995 is inflated by train/validation overlap (D5): only 16 of 366 training images (4.4 %) originate from real flight footage. The labelling tool (`label_tool.py`) was built in week 16; building it in week 1 and collecting real frames during any early manual flight would have yielded a proper held-out test set ($n \geq 50$) and eliminated the need to treat mAP as an upper bound.

2. Characterise the sensor pipeline end-to-end before writing application code. Three of the thirteen defects—BGR inversion (45 min), FOV miscalibration (60 min), and GPS timing lag (mitigated but not fully eliminated)—were sensor-pipeline issues that propagated silently into higher-level logic. A dedicated "sensor characterisation sprint" in week 1—measuring colour encoding, FOV at multiple distances, GPS latency under motion, and lens distortion—would have prevented these faults and saved ∼2 h of bench-day debugging.

3. Simulation fidelity has diminishing returns; real-world data does not. SITL simulation caught 5 of 13 defects (Tier 1) but assumes perfect GPS, zero wind, and synthetic camera views. The DJI video replay pipeline—built in one session post-cancellation—produced 4 quantitative figures (Figures **??**–**??**) and the CEP_{50}=2.3 m GPS characterisation that simulation could never yield. Future projects should prioritise replaying real sensor data through the pipeline over building higher-fidelity simulators.

4. Budget for weather and schedule one flight day per week, not per project. A single scheduled flight day creates a binary outcome: either the weather cooperates and the project obtains flight data, or it does not. Scheduling weekly flight slots—even short 30-minute windows—would have provided multiple opportunities and allowed the progressive test tiers to be executed across several sessions rather than compressed into a single day.

5. Test quantity does not equal test quality. The project produced 74 test scripts totalling ∼5 500 LOC. Of these, 12 caught actual defects. The remaining 62 scripts served documentation, calibration, or development purposes—valuable, but not defect-catching. In a second iteration, the team should distinguish between *validation tests* (expected to catch defects) and *characterisation tests* (expected to produce data), and prioritise the former. A CI pipeline running even 10 critical validation tests on every commit would provide more safety assurance than 74 scripts run manually and sporadically.

6. The "sim-to-real gap" is really a "sim-to-real cliff" without iterative hardware access. The project's multi-fidelity simulation framework was designed to narrow the sim-to-real gap incrementally across five tiers. In practice, Tiers 1–3 were executed over 10 weeks, and Tiers 4–5 were compressed into a single day that was then cancelled by weather. The progressive framework *as designed* is sound, but its value depends on continuous access to real hardware throughout the project, not just at the end. Without iterative hardware testing, the simulation framework provides a false sense of readiness that collapses on first contact with physical reality—exactly as the 7 hardware-specific defects demonstrated.

AJ Development Methodology

This appendix documents the progressive development methodology used to build the SAR drone system, from initial simulation through to field integration. It covers the design philosophy, a 10-phase timeline with outcomes, and lessons that future teams can apply.

AJ.1 Development Philosophy

Three principles governed development from the outset:

1. **Simulation-first.** Every algorithmic component—state machine, search pattern generator, detection pipeline, GPS estimation—was prototyped and end-to-end tested in SITL (Software-In-The-Loop) on a laptop before any hardware was touched. SITL provides repeatable conditions, instant restart, and zero crash risk.
2. **Same code everywhere.** A single codebase runs on the Windows development laptop, WSL/Linux, and the Raspberry Pi 5. Platform differences are confined to `config.py` (auto-detection of serial ports, camera backends, display availability) and `vision.py` (dual backend: Ultralytics on laptop, TFLite/NCNN on Pi). No `if platform ==` guards appear in mission logic.
3. **Progressive trust.** Each test step isolates exactly one new variable. Bench tests precede flight tests; passive observation precedes autonomous commands; logging-only modes precede action-taking modes. A failure at step N

is attributable to the single change introduced at step N.

AJ.2 Ten-Phase Development Timeline

Table **??** summarises the ten development phases, their scope, test outcomes, and calendar dates.

Table 166: Ten-phase development history with outcomes. The majority of integration defects were discovered in phases 5–7 (bench and field testing), validating the progressive approach.

#	Phase	Scope and Outcomes	Dates
1	Simulation on Laptop	State machine (initial 11 states, later expanded to 20), lawnmower search pattern, simulated drone camera from `map.jpg`, Ultralytics YOLOv8n detection, GPS estimation, operator Y/N confirmation, SITL integration. Full mission completed end-to-end in simulation.	Weeks 12–13
2	Multi-Platform CV	Dual-backend `vision.py`: Ultralytics (laptop) / TFLite (Pi). Docker Pi-environment test passed. TFLite coordinate bug fixed (pixel vs. normalised). `config.py` auto-detects connection type.	Week 13
3	Test Infrastructure	`preflight.py`, 13 test scripts covering camera, Cube heartbeat, GPS, AI detection, FOV calibration, resolution trade-off. Organised into `tests/hardware/`, `tests/flight/`, `tests/calibration/`, `tests/diagnostics/`.	Week 14
4	Pi + Camera	Raspberry Pi 5 with IMX296 global shutter. Camera verified at 1456×1088. BGR colour issue discovered and fixed (LL-02). TFLite inference: 206.5 ms average, 4.8 FPS, 0.966 confidence (50-run benchmark).	Week 16
5	Pi + Cube Wired	Cube Orange connected via UART at 921 600 baud. Mavproxy UDP bridge resolves Python 3.13 serial bug (LL-07). Mission Planner connects via TCP bridge.	Week 16
6	Interactive Simulator	`simple_simulator.py`: keyboard flight, visual servo, GPS estimation, spatial clustering, Kalman filter, 7.5 m offset landing, multi-target support, data recording.	Weeks 14–15
7	Web Ground Station	`pi_flight.py`: MJPEG stream, 2D GPS grid, browser dashboard with detection clusters and command buttons. Headless over SSH (PuTTY). Tested in SIMULATION on laptop.	Week 17
8	Field Integration	FOV calibrated (5.46 mm focal length at bench; 54.4° HFOV from DJI video at altitude). Lens calibration (RMS 0.399). Pi bench testing with real Cube. Weather cancelled outdoor flight; adapted to bench-only protocol.	Weeks 19–20
9	Model Retraining	`generate_dataset_v2.py`: 366 images (300 synthetic + 16 real-labelled + 50 negatives) at 1456×1088. Trained YOLOv8n on Colab: `imgsz=1088`, 150 epochs, mAP50 = 0.995. NCNN backend added (72 ms, 4.5× faster than TFLite).	Week 18
10	Safety & Polish	RC override guard (LL-01). Five-layer SSSI geofence (speed field, repulsion, hard cutoff, waypoint filter, firmware backup). SmartEstimator greedy-cluster GPS lock. Threaded inference (stream at 30 fps, detect in background). 127 unit tests, 33 automated integration tests.	Weeks 19–20

AJ.3 Simulation Stage (Weeks 12–15)

All mission logic was developed and validated in SITL before any hardware was procured. The simulation environment consisted of:

- **ArduPilot SITL** via Mission Planner, providing realistic GPS, attitude, and flight dynamics.
- **Simulated camera** (`simulation.py`): a satellite image (`map.jpg`) with a virtual drone viewport that moves according to SITL telemetry.
- **State machine** (`main.py`): 20 states from `INIT` through `DONE`, with 32 transitions handling detection, centering, descent, verification, and landing.
- **Search pattern** (`planning.py`): lawnmower generator for arbitrary convex polygons with configurable strip spacing, edge margins, and start-corner optimisation.
- **Interactive simulator** (`simple_simulator.py`, 2508 lines): keyboard-driven flight with GPS estimation, inverse-variance weighting, Kalman filtering, spatial clustering, and 7.5 m offset landing.

By the end of the simulation stage the full mission—takeoff, search, detect, centre, descend, verify, land—ran end-to-end on the laptop with zero hardware dependencies.

AJ.4 Pi Deployment (Weeks 16–18)

Hardware integration proceeded component-by-component:

1. **Camera.** IMX296 global shutter at 1456 × 1088 native resolution. A persistent blue tint was traced to the sensor outputting BGR despite picamera2's `RGB888` label; removing `cv2.cvtColor` fixed it (LL-02).
2. **AI inference.** TFLite on Pi 5 CPU (XNNPACK): 206.5 ms per frame, 4.8 FPS, 50/50 detections at 0.966 confidence (`tests/hardware/benchmark.py`). NCNN backend later added: 72 ms, 9 FPS full pipeline.
3. **Cube connection.** Python 3.13 breaks `pyserial` reads; mavproxy UDP bridge (`udpout:127.0.0.1:14550`) resolves this with zero data loss (LL-07). Mission Planner connects via `tcpin:0.0.0.0:5762`.
4. **Model retraining.** `generate_dataset_v2.py` produced 366 images at native resolution. `tools/label_tool.py --full` labelled 16 real frames from DJI video. Colab training yielded mAP50 = 0.995; the model is a drop-in replacement (same input/output shapes).

AJ.5 Field Integration (Weeks 19–20)

A field day at the designated site (2026-03-11) assembled the full Pi + Cube + camera stack. Weather cancelled the planned flight; the team adapted by executing every possible bench and calibration activity:

- **FOV calibration:** 92 cm visible at 1 m height $\Rightarrow f = 5.46$ mm (updated from the initial 7.0 mm estimate).
- **Lens calibration:** checkerboard method (14×9, 28 mm squares), RMS = 0.399, undistortion cost ∼1.5 ms per frame.
- **Connectivity:** three-terminal PuTTY workflow verified (mavproxy, diagnostics, flight script). Port conflicts and camera-ownership issues resolved.
- **Bug discovery:** corrupt `calibration_data.npz` (0 bytes) was mangling frames via `cv2.remap`; deleting it restored detection immediately (LL-04).
- **Subsequent video analysis** of DJI flight footage revealed a 100–200 ms GPS timing lag causing CEP50 = 2.3 m scatter in target estimates (LL-05).

AJ.6 Ambition Levels

The project defined three deliverable tiers to manage scope risk (Table **??**):

Table 168: Three product levels with autonomy and demonstration scope. The Minimum Viable Deliverable (Level 1) was achieved; Levels 2 and 3 remain contingent on successful flight testing.

Level	Name	Autonomy	Demonstration Scope
L1	Manual MVP	None	Pilot flies RC manually. Pi runs camera + AI passively (zero commands). Detection overlay on video stream, buzzer alerts, geotagged images saved. Pilot lands manually.
L2	Semi-Autonomous	Partial	Drone arms, takes off, flies lawnmower search avoiding SSSI. CV detects targets; pilot classifies (Y/I/X). On confirm: centre, descend, offset land at 7.5 m, payload release. RC override always available.
L3	Full Autonomous	Full	As L2 but confidence accumulation replaces pilot confirmation. Multi-pass verification at decreasing altitude. Drone auto-decides.

This tiered approach ensured that even if weather or hardware issues prevented full autonomous flight, L1 would still produce a demonstrable, useful system and collect the data needed for L2 development.

AJ.7 Progressive Testing Ladder

Flight testing follows the strict five-step ladder shown in Table **??**. Each step isolates one new risk factor; no step may be skipped.

Table 170: Progressive flight testing ladder with scripts and risk isolation. Each step introduces exactly one new risk factor; failures must be resolved before advancing.

Step	Description	Script	New Risk
0	Bench commands: arm, mode change, servo PWM. No propellers.	`0a--0f_*.py`	Wiring
1	Manual RC flight. Pi runs camera + AI, logs detections, buzzer beeps. **Zero MAVLink commands.**	`1_passive_flight.py`	Outdoor CV
2	Code-controlled waypoint flight (3–4 GPS points). No camera, no detection, no decisions.	`2_waypoint_test.py`	MAVLink flight
3	AUTO waypoints + AI detection. On detection, switch to GUIDED hover. No descent or landing.	`3_auto_detect.py`	CV + flight
4	Full autonomous: search pattern, detect, centre, descend, verify (Y/N), offset land.	`4_detect_and_center.py`	Full mission

Six bench-level tests (step 0) additionally cover planning validation (`0d_planning_test.py`, 6 unit tests), geofence sign conventions (`0e_geofence_test.py`, 7 unit tests), and payload servo actuation (`0f_servo_test.py`).

AJ.8 Achievement Summary

An objective scoring matrix (Table **??**) was constructed for each requirement across both simulation and real hardware.

Table 172: Simulation vs. real hardware readiness (0–10 scale). Simulation scores average 7.7/10; real hardware averages 2.1/10, reflecting the absence of outdoor flight testing.

Category	Simulation	Real Hardware
All requirements (R01–R10)	7.7	2.1
Flight-critical (R01, R02, R03, R04, R09)	8.4	1.8
Mission-critical (R05, R06, R07)	7.0	1.0
Infrastructure (R08, R10)	7.0	4.5

Simulation (7.7/10): The state machine logic, search pattern generation, CV detection pipeline, GPS estimation mathematics, and safety systems (RC override, geofence, RTL) all work correctly in simulation. The full mission has been completed end-to-end multiple times in SITL.

Real hardware (2.1/10): Individual components are verified (camera: working; TFLite: 4.8 FPS at 0.966 confidence; Cube connection: stable via mavproxy), but zero minutes of real flight time have been logged. The 5.6-point gap between simulation and real hardware is the project's largest risk.

What simulation proved: Algorithm correctness, state transitions, search coverage, detection-to-decision loop, safety logic.

What simulation cannot prove: Whether the drone physically flies; whether CV detects from 15–30 m altitude outdoors; whether GPS accuracy is sufficient for offset landing; whether the video stream is usable at operational WiFi range; whether the geofence actually prevents SSSI entry in wind.

AJ.9 Lessons for Future Teams

Eight lessons emerged from the development process, each documented with root cause, fix, and checklist (see Appendix AH for the full defect register):

1. **Start hardware testing on day one.** The simulation stage produced mature, well-tested code, but the 5.6-point sim-to-real gap shows that software maturity does not transfer to hardware confidence. Even basic tasks—outdoor GPS fix, one manual hover—would have closed multiple requirements.
2. **Validate the environment before the field.** Docker testing caught missing packages early, but the corrupt `calibration_data.npz` and BGR colour issue were discovered only on-site. A pre-field smoke test with the exact Pi image would have saved hours.
3. **Plan for weather cancellation.** The March 11 field day was the sole outdoor window; weather cancelled it. Having a bench-only fallback protocol (FOV calibration, lens calibration, connectivity verification) ensured the day was still productive.
4. **Decouple camera ownership.** Only one process can open picamera2. This forced a clean separation: one script owns the camera and streams to all consumers via MJPEG. Design for single-owner resources from the start.

5. **Never trust format labels.** The IMX296 sensor outputs BGR despite `RGB888` labelling. YOLOv8 TFLite outputs pixel coordinates despite documentation suggesting normalised values. Test empirically; label after measurement.
6. **Build zero-command observation scripts.** The passive watch approach (`passive_watch.py`, zero MAVLink commands) was the safest way to validate CV in the field. It could run during any flight without risk, collecting training data and detection statistics simultaneously.
7. **Use progressive trust, never skip steps.** The numbered test ladder (0a–4) isolates exactly one new variable per step. When Step N fails, the cause is the single change introduced at Step N, not accumulated uncertainty from earlier steps.
8. **Maintain tiered deliverables.** L1 (Manual MVP), L2 (Semi-Autonomous), L3 (Full Autonomous) ensured that even if only L1 was achievable, it would be a complete, demonstrable product rather than a half-finished L2.